Computer Networks

For Java Developers

What happens between a Java client and a server, layer by layer: addressing, TCP, DNS, HTTP, TLS, load balancing and security, with packet-level diagrams, worked traces, real commands and the questions interviewers ask.

Layers · IP and Subnetting · TCP and UDP · DNS · HTTP · TLS · Load Balancing
Routing · NAT · CORS · WebSockets · Security · Troubleshooting · Interview Q&A

```
Socket s = new Socket("example.com", 443);
// DNS lookup  ->  TCP handshake  ->  TLS handshake
// HTTP request  ->  response  ->  close
out.write(request);
in.read(response);
s.close();
```

Maruthi Sreeram
Prepared for Java full stack interview preparation

Maruthi Sreeram

Version basis

Java listings and local network experiments (loopback sockets, openssl, curl) were run on JDK 21
Experiments that need the public internet could not be run; their output is labelled typical
Prepared for Maruthi Sreeram

How to use this guide

- **Who this is for.** A Java full stack developer preparing for interviews who needs a precise model of what the network does underneath REST calls, sockets, browsers and load balancers.
- **Every chapter follows the same steps:** (1) the concept and the problem it solves, (2) how it works, shown with diagrams of packets, message sequences and state machines, (3) a worked trace or calculation, (4) a hands-on experiment with real commands or Java code, (5) common failures, and (6) the questions interviewers ask, with answers.
- **How to study.** Read each diagram before the prose and try to narrate it aloud. Redo each worked trace on paper before reading the answer. Run the experiments; the output shown was produced on the machine that built this guide.
- **Running the code.** Save a Java listing as `Main.java` and run `java Main.java`. Commands marked as terminal sessions begin with `$`. Several tools (for example `tcpdump`, `dig` and `ping`) were not installed where the guide was built, so those outputs are shown as typical output and labelled.
- **Conventions.** Addresses use the documentation ranges (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and private ranges, never real hosts. Numbers in a diagram match the accompanying trace.

Contents

Part 1: Network Foundations 4

1.1 What a Network Is 4
1.2 Delay, Loss and Throughput 9
1.3 Layered Architecture 14
1.4 The Link Layer 20
1.5 Wireless Basics 28
1.6 Network Tools 33

Part 2: The Network Layer 41

2.1 IPv4 Addressing and the IPv4 Header 41
2.2 Classless Addressing and Subnetting 48
2.3 IPv6 56
2.4 ICMP and Network Diagnostics 63
2.5 Routing 69
2.6 NAT and DHCP 76
2.7 Cloud Networking Basics 83

Part 3: The Transport Layer 89

3.1 Transport Layer, Ports and Sockets 89
3.2 UDP: The User Datagram Protocol 94
3.3 TCP Segments and Connection Management 98
3.4 TCP Reliability 110
3.5 Flow Control 116
3.6 Congestion Control 121
3.7 Sockets in Java 126
3.8 QUIC and HTTP/3 Transport 135

sieram

Part 4: The Application Layer and the Web . . . 139

4.1 The Domain Name System . . . 139
4.2 HTTP/1.1 . . . 147
4.3 HTTP Caching and Conditional Requests . . . 156
4.4 Cookies, Sessions and Authentication over HTTP . . . 161
4.5 HTTP/2 and HTTP/3 . . . 165
4.6 REST API Design from the Network Perspective . . . 171
4.7 Cross-Origin Resource Sharing (CORS) . . . 177
4.8 WebSocket, Server-Sent Events and Long Polling . . . 183
4.9 gRPC and Other Application Protocols . . . 189

Part 5: Security and Infrastructure . . . 196

5.1 Cryptography for Networking . . . 196
5.2 TLS and HTTPS . . . 204
5.3 Authentication in Transit . . . 215
5.4 Network Security . . . 221
5.5 Reverse Proxies, Load Balancers and Gateways . . . 228
5.6 CDNs, Caching and High Availability . . . 239
5.7 Network Design for Microservices . . . 248

Part 6: Practice, Troubleshooting and Interviews . . . 257

6.1 Java Networking in Practice . . . 257
6.2 What Happens When a URL Is Entered . . . 265
6.3 Troubleshooting Playbook . . . 271
6.4 Numerical Problems . . . 280
6.5 Interview Question Bank . . . 288
6.6 Cheat Sheets . . . 294
6.7 Six-Week Study Plan . . . 299

sreeram

PART 1

Network Foundations

Hosts, links, delay, layers and the link layer: the vocabulary and the mechanisms that every later part builds on.

In this part

- **1.1** What a Network Is
- **1.2** Delay, Loss and Throughput
- **1.3** Layered Architecture
- **1.4** The Link Layer
- **1.5** Wireless Basics
- **1.6** Network Tools

1.1 What a Network Is

A computer network is a set of devices that exchange data over shared communication infrastructure. Every REST call made by a Spring Boot service, every JDBC connection to a database and every page load in a browser is a sequence of messages crossing such infrastructure. The application code sees a `Socket` or an `HttpClient`; underneath, the data is cut into pieces, handed from device to device, and reassembled. A developer who understands this machinery can predict latency, explain timeouts and diagnose the failures that appear in production, which is the purpose of this guide.

This chapter defines the components of a network and the structure of the Internet, contrasts the two fundamental ways of moving data (circuit switching and packet switching), and contrasts the two common application architectures (client-server and peer-to-peer).

1.1.1 Concept and mechanism

Components

The elements of a network are classified by function.

Component	Role	Typical example
Host (end system)	Runs applications; originates and consumes data	Laptop, phone, web server, database server
Link	Carries bits between two adjacent devices	Ethernet cable, optical fibre, Wi-Fi channel
Switch	Forwards frames within one local network using link-layer (MAC) addresses	Office access switch
Router	Forwards packets between networks using network-layer (IP) addresses	Home router, ISP edge router
Access point	Bridges wireless stations to a wired network	Wi-Fi access point
Firewall, load balancer	Inspects or distributes traffic	Perimeter firewall, reverse proxy

Network components and their roles.

A **host** is any device that runs application software at the edge of the network. Hosts are also called end systems. A **link** is a physical or wireless medium that joins two devices; its capacity in bits per second is its **bandwidth** or **rate**. A **switch** operates inside one local network and a **router** operates between networks; the difference is examined in the chapters on the link layer and on routing. A **packet** is a unit of data with a header that carries the information needed to deliver it.

Structure of the Internet

The Internet is a network of networks. Hosts connect to an **access network** (home broadband, a campus network, a mobile cellular network). Access networks connect to **Internet Service Providers** (ISPs). ISPs are arranged in tiers: regional ISPs buy transit from larger ISPs, and a small set of **tier-1 ISPs** reach every destination on the Internet without paying for transit, because they **peer** with each other. Content providers frequently operate their own global networks and connect directly to access ISPs. An **Internet Exchange Point** (IXP) is a facility containing a switching fabric at which many networks interconnect and exchange traffic locally.

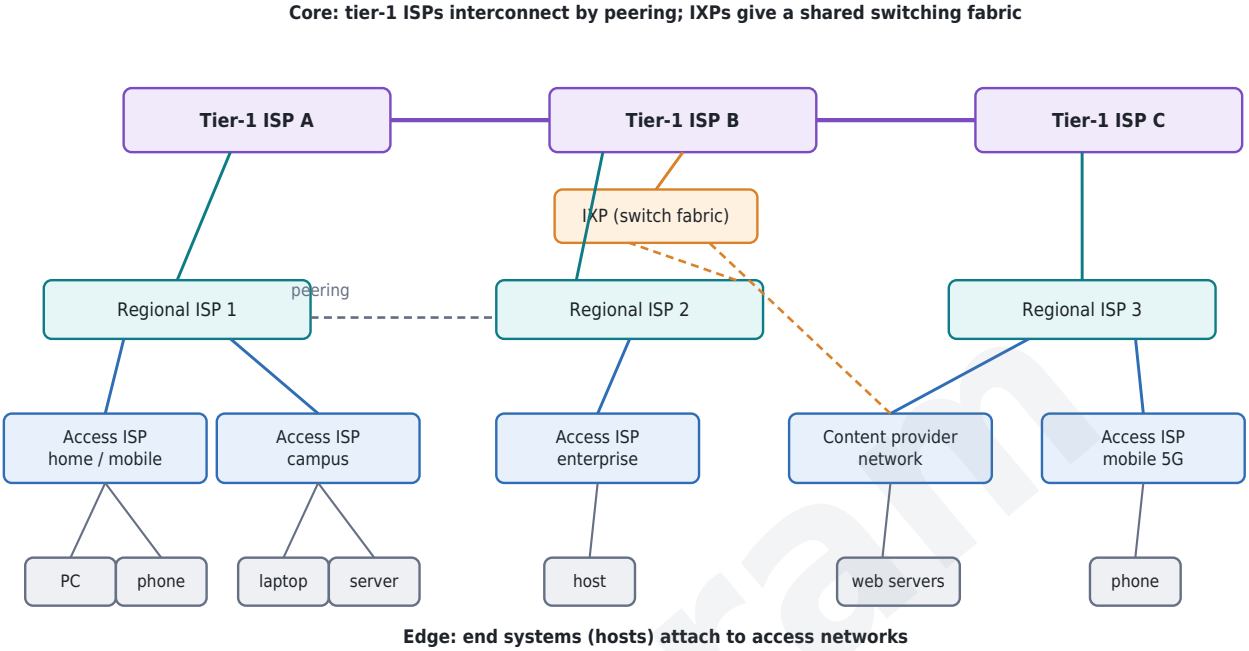


Figure 1.1. The edge (hosts and access networks) attaches through regional ISPs to a core of tier-1 ISPs; solid lines are provider links, dashed lines are peering at an IXP or directly between networks.

Each independently administered network is an **autonomous system** (AS) identified by a number. Autonomous systems exchange reachability information with the Border Gateway Protocol, described in the part on the network layer. Two consequences matter to developers. First, a request from a client to a server normally crosses several administrative domains, so no single party controls its latency. Second, the path is determined hop by hop by routers; the endpoints neither choose nor see it.

Packet switching and circuit switching

Two designs exist for moving data through a network core. In **circuit switching**, a dedicated path with reserved capacity is established between the endpoints before data flows and is released afterwards. The classic telephone network works this way. A link is divided by frequency (FDM) or by time slots (TDM), and each call owns one division for its whole duration, whether or not anything is being said.

In **packet switching**, the message is split into packets. Each packet carries the destination address and is forwarded independently by routers. Packets from many sources are interleaved on the same links; this sharing is called **statistical multiplexing**. A router uses **store-and-forward** operation: it receives a whole packet before it begins to transmit that packet on the outgoing link. If packets arrive faster than the outgoing link can send them, they wait in a buffer (a queue); if the buffer is full, packets are dropped.

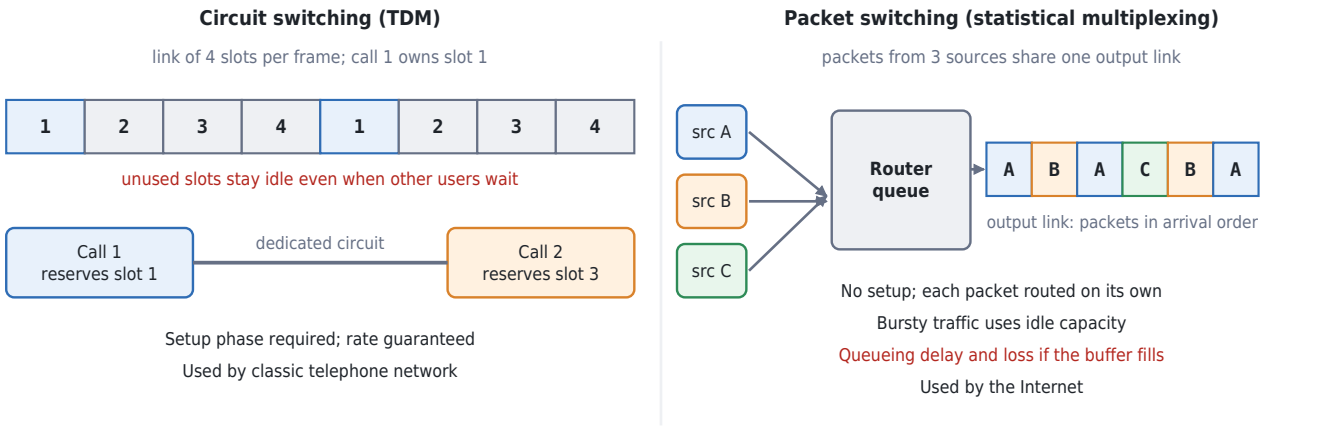


Figure 1.2. Circuit switching reserves fixed slots that stay idle when unused; packet switching interleaves packets from several sources on one link at the price of queueing delay and possible loss.

Property	Circuit switching	Packet switching
Resource allocation	Reserved per call	Shared on demand
Setup phase	Required	None
Rate guarantee	Fixed	None; best effort
Efficiency for bursty data	Low (idle reserved slots)	High
Failure behaviour	Call is dropped if a link fails	Packets are rerouted
Delay	Constant after setup	Variable (queueing)

Circuit switching compared with packet switching.

The Internet uses packet switching because computer traffic is bursty: a web page is fetched in a short burst and then the link is idle while the user reads. Reserving capacity for the idle periods would waste most of it.

Client-server and peer-to-peer

In a **client-server** architecture, a server host is always on, has a well-known address and answers requests from many clients; clients do not communicate with each other. Web servers, database servers and REST APIs follow this model. In a **peer-to-peer** (P2P) architecture, hosts act as client and server at the same time and exchange data directly. File-sharing systems such as BitTorrent use it, and so do some real-time communication paths (WebRTC). The advantage is that capacity grows with the number of peers. The costs are complexity, peers that appear and disappear, and difficulty of connecting hosts that sit behind NAT devices.

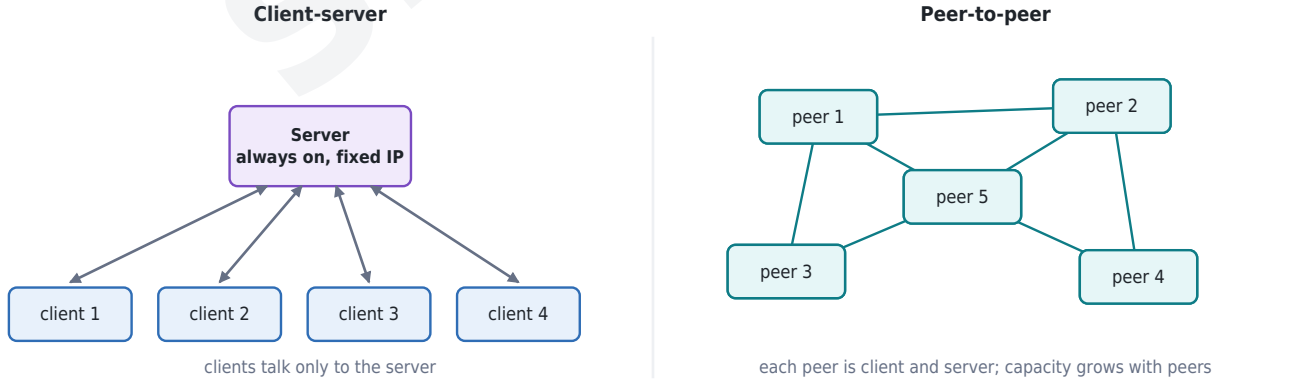


Figure 1.3. In the client-server model all traffic converges on one server; in the peer-to-peer model every host both requests and provides data.

1.1.2 Worked trace

Why packet switching supports more users

Assume a 1 Mbit/s link. Each user, when active, transmits at 100 kbit/s, and a user is active 10 percent of the time. With circuit switching, each user reserves 100 kbit/s for the whole call, so the link supports 1000 / 100 = 10 users. With packet switching, 35 users may share the link. The link is overloaded only when more than 10 of the 35 users are active simultaneously. With independent users, the number of active users is binomially distributed, and the probability of exceeding 10 is the sum over k = 11 to 35 of C(35, k) * 0.1^k * 0.9^(35 - k).

Mux.java JAVA

```java
public class Mux {
    static double choose(int n, int k) {
        double r = 1;
        for (int i = 1; i <= k; i++) r = r * (n - k + i) / i;
        return r;
    }

    public static void main(String[] args) {
        int users = 35, capacityUsers = 10;
        double p = 0.10;
        double tail = 0;
        for (int k = capacityUsers + 1; k <= users; k++)
            tail += choose(users, k) * Math.pow(p, k) * Math.pow(1 - p, users - k);
        System.out.printf("circuit switching supports %d users%n", capacityUsers);
        System.out.printf("packet switching with %d users: P(more than %d active) = %.6f%n",
                users, capacityUsers, tail);
        double circuitSeconds = 640_000 / 64_000.0 + 0.5;
        System.out.printf("640 kbit file, 64 kbit/s circuit, 0.5 s setup: %.1f s%n",
                circuitSeconds);
    }
}
```

Run the calculation TERMINAL

```
$ java Mux.java
circuit switching supports 10 users
packet switching with 35 users: P(more than 10 active) = 0.000424
640 kbit file, 64 kbit/s circuit, 0.5 s setup: 10.5 s
```

The result, 0.000424, means that the link is overloaded in about 0.04 percent of the time. Packet switching therefore carries 3.5 times as many users as circuit switching for almost the same quality, which is the quantitative reason for its dominance. The last line shows the other side of the trade: a circuit offers a fixed rate (64 kbit/s here), so the transfer time is predictable but cannot be shortened when the rest of the network is idle.

1.1.3 Hands-on

The Java standard library exposes the local network interfaces. The following program lists each interface with its MTU, hardware (MAC) address and IP addresses with prefix length.

Ifaces.java JAVA

```java
import java.net.*;
import java.util.*;

public class Ifaces {
    public static void main(String[] args) throws Exception {
        for (NetworkInterface nif : Collections.list(NetworkInterface.getNetworkInterfaces())) {
            byte[] mac = nif.getHardwareAddress();
            String m = mac == null ? "none" : HexFormat.ofDelimiter(":").formatHex(mac);
            System.out.printf("%-5s up=%b loopback=%b mtu=%d mac=%s%n",
                nif.getName(), nif.isUp(), nif.isLoopback(), nif.getMTU(), m);
            for (InterfaceAddress ia : nif.getInterfaceAddresses())
                System.out.printf("      %s/%d%n", ia.getAddress().getHostAddress(),
                    ia.getNetworkPrefixLength());
        }
    }
}
```

Output on the build machine (a Linux container; the values differ on every machine) TERMINAL

```
$ java Ifaces.java
eth0  up=true loopback=false mtu=1400 mac=02:fc:00:00:00:01
      192.0.2.2/24
lo    up=true loopback=true mtu=65536 mac=none
      127.0.0.1/8
```

The interface `lo` is the loopback interface. Traffic sent to `127.0.0.1` never leaves the host, which is why all experiments in this guide that need a server use loopback. The interface `eth0` is a virtual Ethernet interface. The first octet of its MAC address, `02`, has the second-least-significant bit set, which marks a locally administered address (assigned by software rather than by the manufacturer).

A client-server exchange can be reproduced with the netcat utility `nc`, which opens plain TCP connections. One terminal acts as the server and a second as the client.

A one-shot TCP server and client on loopback TERMINAL

```
$ nc -l 127.0.0.1 9000 > received.txt &
$ echo "hello over tcp" | nc -q1 127.0.0.1 9000
$ cat received.txt
hello over tcp
```

The server host listens at a fixed address and port; the client initiates the connection. These two roles correspond exactly to `ServerSocket` and `Socket` in Java.

1.1.4 Failure modes

- **Congestion loss.** A router buffer fills and packets are discarded. The symptom is retransmissions and slow transfers without any link being down. Diagnosis is by measuring loss and delay along the path (the tools chapter).
- **Path asymmetry and dependency on third parties.** The route to a server may cross networks the operator does not control. A fault in a transit network appears as high latency or timeouts from some clients only; the evidence is that the same request succeeds from another network.
- **Single points of failure at the edge.** A single access link or a single server is a client-server weakness: when it fails, every client fails. Remedies are redundancy (several servers behind a load balancer, several uplinks).
- **Assumption of a fixed path.** Packet switching does not guarantee in-order delivery or a fixed route; protocols above IP (TCP) must reorder and retransmit. Code that assumes one `read` returns one `write` is therefore incorrect.

1.1.5 Questions to expect

1. What is the difference between a switch and a router? A switch forwards frames inside one local network using MAC addresses and a learned table. A router connects different networks and forwards packets using IP addresses and a routing table. A router also separates broadcast domains; a switch does not.
2. Why does the Internet use packet switching instead of circuit switching? Computer traffic is bursty. Statistical multiplexing lets many users share link capacity that circuits would reserve and leave idle. The cost is variable delay and possible loss.
3. What is store-and-forward? A router receives an entire packet before it starts to transmit it on the outgoing link. For a packet of L bits over N links of rate R, the end-to-end transmission time is at least N * L / R.
4. What is an Internet Exchange Point? A facility with a switching fabric where many networks connect and exchange traffic directly. It shortens paths and lowers transit cost.
5. What is an autonomous system? A collection of IP networks under one administrative policy that is identified by an AS number and exchanges routes with other autonomous systems using BGP.
6. Give one advantage and one disadvantage of peer-to-peer over client-server. Advantage: capacity scales with the number of peers and no single server is a bottleneck. Disadvantage: peers are unreliable and frequently behind NAT, so connection setup and consistency are harder.
7. What does the loopback interface do? It delivers packets sent to `127.0.0.1` back to the same host without using any physical link. It is used for local testing and inter-process communication.

INTERVIEW ANGLE

Follow-up chain: "Is the Internet a single network?" No. It is a network of tens of thousands of independently operated autonomous systems. Interconnection is by transit (a customer pays a provider) or by peering (two networks exchange traffic with each other), and routes are exchanged with BGP. No participant has a global view or control, which is why end-to-end latency and failures cannot be fully predicted from one side.

KEY POINTS

Key points

- A network consists of hosts at the edge, links, and switches and routers in the core; the Internet is a network of autonomous systems connected by transit and peering.
- Packet switching shares links by statistical multiplexing and uses store-and-forward; it is efficient for bursty traffic but introduces queueing delay and loss.
- Circuit switching reserves capacity per call: predictable but wasteful for bursty data.
- Client-server concentrates service on an always-on server; peer-to-peer distributes it among hosts.
- A packet does not follow a fixed path; applications must not assume ordering or message boundaries below TCP.

1.2 Delay, Loss and Throughput

Network performance is described by three quantities: delay (how long a bit takes to arrive), loss (how many packets never arrive) and throughput (how many bits per second arrive). Application symptoms such as a slow API call or a stalled download are usually explained by one of them. This chapter defines the components of delay with their formulas, works through numeric examples, and introduces the bandwidth-delay product, which later explains TCP window sizes.

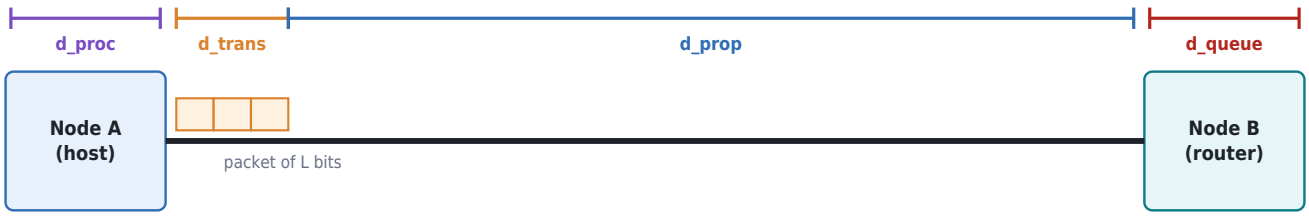
1.2.1 Concept and mechanism

The four components of nodal delay

A packet travelling from one node to the next suffers four delays, whose sum is the **nodal delay**.

Component	Definition	Formula
Processing delay d_proc	Examining the header, checking for bit errors, choosing the output link	Microseconds or less; treated as constant
Queueing delay d_queue	Waiting in the output buffer for earlier packets to be sent	Depends on traffic; variable
Transmission delay d_trans	Time to push all bits of the packet onto the link	L / R (L in bits, R in bit/s)
Propagation delay d_prop	Time for a bit to travel across the medium	d / s (d distance, s propagation speed)

Components of nodal delay.



d_proc	Header inspection, error check, output link selection: microseconds or less
d_trans	L / R: time to push all L bits onto the link at R bit/s
d_prop	distance / propagation speed (about 2 x 10^8 m/s in fibre or copper)
d_queue	Waiting in the output buffer behind earlier packets; varies per packet

d_nodal = d_proc + d_queue + d_trans + d_prop

Transmission delay depends on packet size and link rate; propagation delay on distance only.

Figure 1.4. The four delay components occur at different places: processing and queueing inside the node, transmission when bits are pushed onto the link, and propagation while the bits travel along it.

Transmission delay and propagation delay are frequently confused. Transmission delay depends on the packet length and the link rate; it is the time to emit the packet, and it is not affected by distance. Propagation delay depends on distance and the medium; it is not affected by packet length or rate. The speed of a signal in copper or optical fibre is about 2 x 10^8 m/s, two thirds of the speed of light in vacuum.

Queueing delay and traffic intensity

Let a be the average packet arrival rate (packets per second), L the packet length in bits and R the link rate. The **traffic intensity** is La / R. If La / R is close to 0, packets rarely wait. As La / R approaches 1, the queue grows quickly; if La / R exceeds 1, the arrival rate is higher than the service rate, the queue grows without bound and packets are dropped when the buffer is full. The queueing delay is therefore the main source of variation in end-to-end delay, called **jitter** when measured as the variation between successive packets. Jitter is harmful to real-time media, which is why voice and video applications use playout buffers.

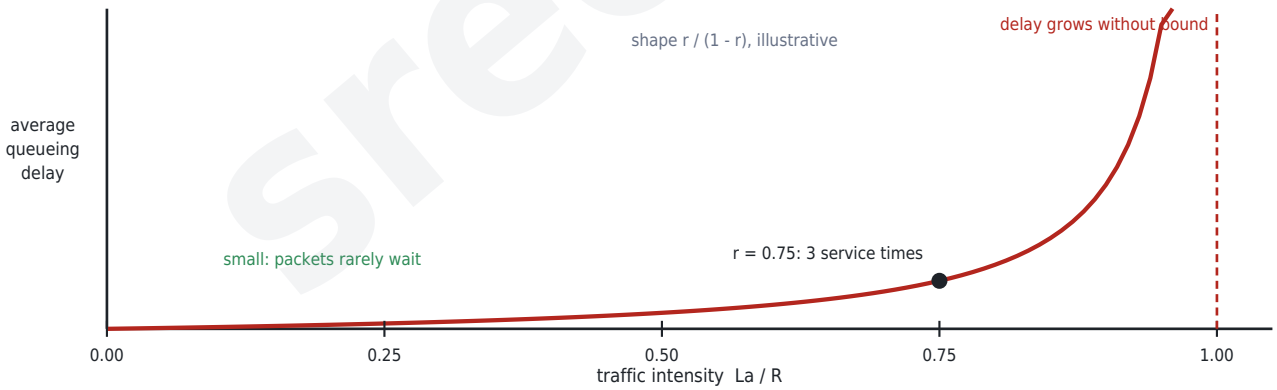


Figure 1.5. Average queueing delay is small at low traffic intensity and rises steeply near 1; the curve shows the shape r / (1 - r) from the simple M/M/1 queueing model and is illustrative, not a measurement.

Store-and-forward end-to-end delay

With N links in series, each of rate R, and no queueing, a single packet of L bits needs N * L / R to reach the destination, because every router receives the whole packet before forwarding it. For P packets sent back to back, the last packet arrives after (N + P - 1) * L / R, since the links work in a pipeline once it is filled.

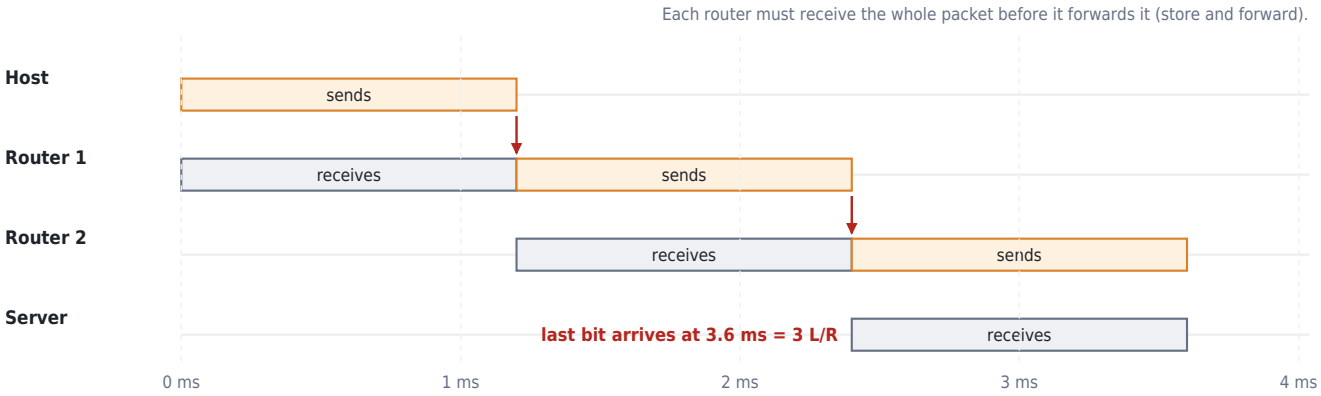


Figure 1.6. A 1500-byte packet (12 000 bits) crossing three 10 Mbit/s links takes three transmission delays of 1.2 ms each; a router cannot begin sending before the last bit has arrived.

Loss

A packet is lost when a router buffer is full on arrival, when a link corrupts it so that a checksum fails, or when a device discards it (for example by a firewall rule or an expired TTL). Loss is a normal part of packet switching. Reliability is added above the network layer: TCP retransmits lost segments, while UDP leaves loss to the application.

Throughput and bottlenecks

Throughput is the rate at which bits are actually delivered between two hosts, as opposed to the **bandwidth**, which is the capacity of a link. For a path of links with rates R1, R2, ..., Rn, the maximum throughput of a single flow is min(R1, ..., Rn). The slowest link is the **bottleneck**. When several flows share a bottleneck of rate R, each obtains about R divided by the number of flows if the sharing is fair.

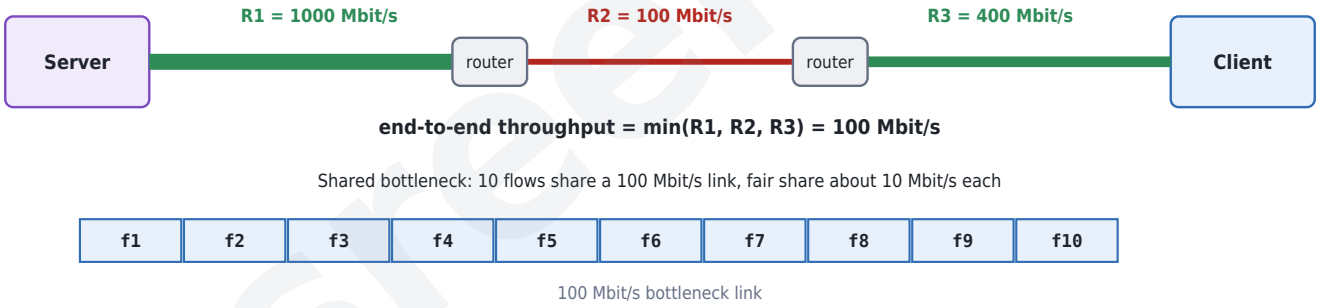


Figure 1.7. The end-to-end throughput equals the rate of the slowest link, here the 100 Mbit/s middle link, not the fastest link or the sum of the links.

Bandwidth-delay product

The **bandwidth-delay product** (BDP) is the number of bits that fit in the link between sender and receiver at one time: rate multiplied by delay. When the delay is the round-trip time (RTT), the BDP is the amount of data a sender must have sent and not yet had acknowledged in order to keep the link fully used. A sender that stops after each window until an acknowledgement arrives cannot exceed window / RTT. This is the basis of TCP window sizing; the transport part applies it.

sreeram

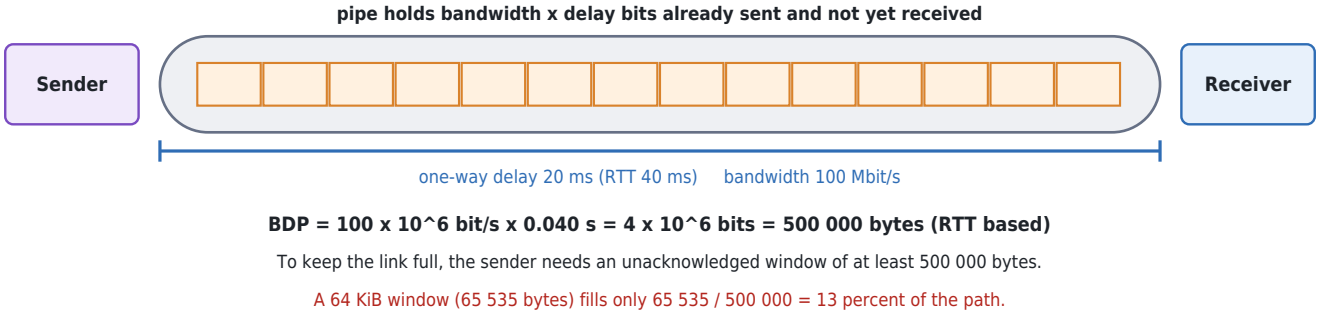


Figure 1.8. The pipe between sender and receiver holds rate x delay bits; a window smaller than this product leaves part of the link idle.

1.2.2 Worked trace

Example 1: transmission and propagation delay

A packet of L = 1500 bytes = 12 000 bits is sent over a link of R = 10 Mbit/s and length 2000 km, with propagation speed 2 x 10^8 m/s.

1. d_trans = L / R = 12 000 / 10 000 000 s = 0.0012 s = 1.2 ms.
2. d_prop = 2 000 000 m / (2 x 10^8 m/s) = 0.010 s = 10 ms.
3. Time from first bit sent to last bit received = d_trans + d_prop = 11.2 ms. The first bit arrives after 10 ms; the last bit leaves the sender at 1.2 ms and arrives at 11.2 ms.
4. If R rises to 100 Mbit/s, d_trans = 0.12 ms and the total is 10.12 ms; if R rises to 1 Gbit/s, d_trans = 0.012 ms. The propagation delay does not change. Beyond some rate, a higher bandwidth no longer shortens the transfer of small messages, because propagation dominates.

Example 2: queueing

Let R = 10 Mbit/s and L = 12 000 bits, so one packet needs 1.2 ms. The service capacity is R / L = 833 packets per second. The traffic intensity for several arrival rates:

Arrival rate a (packets/s)	La / R	Interpretation
400	0.48	Queue usually empty
600	0.72	Occasional waiting
750	0.90	Long queues
800	0.96	Delay large and unstable
830	1.00	Capacity reached; queue grows without bound

Traffic intensity for L = 12 000 bits, R = 10 Mbit/s.

Example 3: end-to-end with three links

Three links of 10 Mbit/s each, no queueing, one packet of 12 000 bits: 3 x 1.2 ms = 3.6 ms of transmission delay (diagram above). With ten such packets sent back to back, the last arrives after (3 + 10 - 1) x 1.2 ms = 14.4 ms.

Example 4: bandwidth-delay product

Rate 100 Mbit/s, RTT 40 ms. BDP = 100 x 10^6 x 0.040 = 4 x 10^6 bits = 500 000 bytes. A sender limited to a window of 65 535 bytes achieves at most 65 535 x 8 / 0.040 = 13.1 Mbit/s, which is 13 percent of the capacity.

Example 5: throughput of a file transfer

An 8 MB file (64 x 10^6 bits) is downloaded over a path whose bottleneck link is 2 Mbit/s. The transfer time is at least 64 x 10^6 / (2 x 10^6) = 32 s, independent of faster links elsewhere on the path.

sreeram

1.2.3 Hands-on

The calculations above are reproduced by a short program.

Delay.java — JAVA

```java
public class Delay {
    public static void main(String[] args) {
        double L = 12_000;                       // packet length in bits (1500 bytes)
        for (double R : new double[]{10e6, 100e6, 1e9})
            System.out.printf("R=%8.0f Mbit/s  d_trans=%.4f ms%n", R / 1e6, L / R * 1000);
        double prop = 2_000_000 / 2.0e8;         // 2000 km at 2 x 10^8 m/s
        System.out.printf("d_prop (2000 km) = %.1f ms%n", prop * 1000);
        double bdp = 100e6 * 0.040;
        System.out.printf("BDP = %.0f bits = %.0f bytes%n", bdp, bdp / 8);
        System.out.printf("store-and-forward, 3 links, 1 packet: %.1f ms%n", 3 * L / 10e6 * 1000);
        System.out.printf("8 MB over a 2 Mbit/s bottleneck: %.0f s%n", 8e6 * 8 / 2e6);
    }
}
```

Run the program — TERMINAL

```
$ java Delay.java
R=      10 Mbit/s  d_trans=1.2000 ms
R=     100 Mbit/s  d_trans=0.1200 ms
R=    1000 Mbit/s  d_trans=0.0120 ms
d_prop (2000 km) = 10.0 ms
BDP = 4000000 bits = 500000 bytes
store-and-forward, 3 links, 1 packet: 3.6 ms
8 MB over a 2 Mbit/s bottleneck: 32 s
```

Delay observed by an application can be measured with `curl` and its `-w` format. The command below requests a local Java `HttpServer` (the server used throughout this guide, listening on 127.0.0.1:8080) and reports the connection time, the time to the first byte and the total time.

Timing a request with curl (loopback; values vary between runs) — TERMINAL

```
$ curl -s -o /dev/null \
    -w 'connect=%{time_connect}s ttfb=%{time_starttransfer}s total=%{time_total}s\n' \
    http://127.0.0.1:8080/hello
connect=0.000104s ttfb=0.001190s total=0.001235s
```

On loopback the propagation delay is zero and the connect time reflects only local processing. Against a remote server, `time_connect` approximates one RTT (the three-way handshake needs one round trip before the client can send data), which makes it a convenient RTT estimate when `ping` is blocked. On a real path the output of `ping` and `traceroute` shows the per-hop and end-to-end delay; they are not installed on the build machine, so a typical ping output is shown in the tools chapter and is labelled as such.

1.2.4 Failure modes

- **Bufferbloat.** Large buffers in routers or modems absorb bursts without dropping, so a saturated uplink adds hundreds of milliseconds of queueing delay. Symptom: ping time rises sharply while a large upload runs. Remedy: smart queue management (for example fq_codel) or limiting the sending rate below the bottleneck rate.
- **Long fat networks.** A high-bandwidth path with large RTT (for example a transcontinental link) has a large BDP. Without a sufficiently large TCP window, throughput is far below the link rate although the link is idle. Diagnosis: throughput is approximately window / RTT.
- **Small-request latency dominated by round trips.** A client that issues many sequential requests pays one RTT per request. Moving from 20 sequential calls at 50 ms RTT (about 1 s) to one batched call (about 50 ms) is a design change, not a bandwidth upgrade.
- **Jitter in real-time traffic.** Variation of queueing delay breaks voice and video unless the playout buffer is dimensioned for it.
- **Loss misread as bandwidth shortage.** A path with 1 percent loss can show a small fraction of the link rate under TCP because the congestion window is reduced after each loss.

Sheram

1.2.5 Questions to expect

1. Name the four components of nodal delay. Processing, queueing, transmission and propagation delay. Only queueing delay varies significantly with load.
2. What is the difference between transmission delay and propagation delay? Transmission delay is L / R, the time to place the bits on the link; it depends on packet size and rate. Propagation delay is distance divided by signal speed; it depends on the medium and the distance only.
3. A 1500-byte packet crosses a 100 Mbit/s link. What is the transmission delay? 12 000 bits / 100 x 10^6 bit/s = 0.12 ms.
4. What is the bandwidth-delay product and why does it matter? It is rate multiplied by RTT, the amount of data in flight on a fully used path. A TCP window smaller than the BDP prevents the sender from filling the link.
5. What happens when traffic intensity approaches 1? The average queueing delay grows without bound and the buffer overflows; packets are lost.
6. What is the throughput of a path with links of 1 Gbit/s, 50 Mbit/s and 200 Mbit/s? 50 Mbit/s, the minimum, because the slowest link is the bottleneck.
7. Why does a faster link not always reduce latency? Higher rate reduces only transmission delay. Propagation, queueing and processing delay stay. For small messages over long distances, propagation delay dominates.
8. What is jitter? The variation in packet delay, mainly caused by variable queueing delay.

KEY POINTS

Key points

- Nodal delay = processing + queueing + transmission + propagation; transmission is L / R and propagation is distance divided by signal speed.
- Queueing delay depends on traffic intensity La / R and rises sharply near 1; overflow produces loss.
- Store-and-forward makes the delay of N links at least N * L / R for one packet.
- Throughput is limited by the bottleneck link; shared bottlenecks divide capacity among flows.
- The bandwidth-delay product is the data in flight needed to fill a path; windows smaller than the BDP waste capacity.

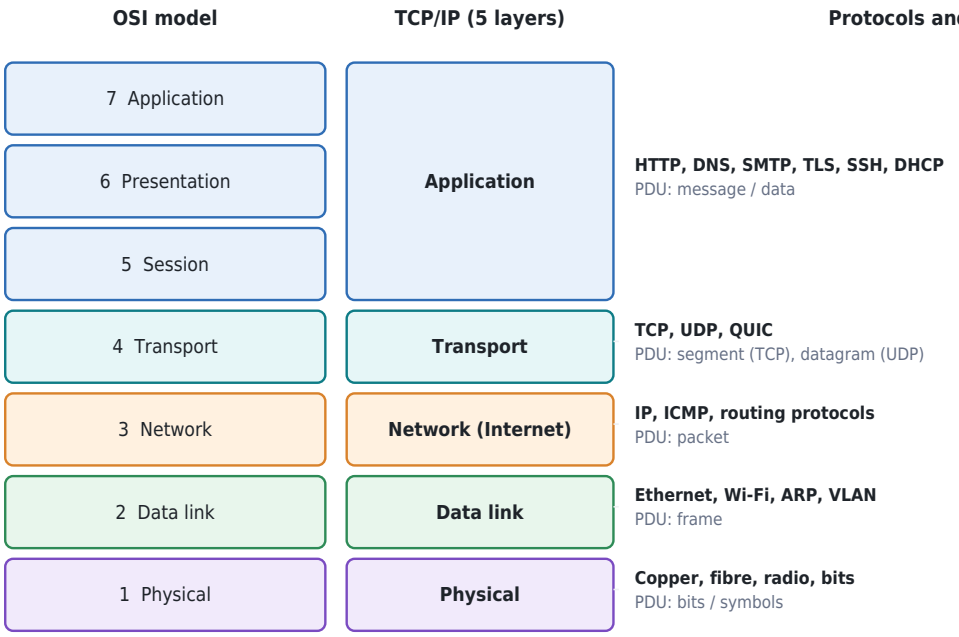
1.3 Layered Architecture

Moving a web page from a server to a browser involves electrical or optical signalling, error detection, addressing, routing, reliable delivery, encryption and the application protocol itself. A single monolithic design for all of this would be impossible to implement, test and evolve. Network architecture therefore divides the problem into **layers**. Each layer offers a service to the layer above, uses the service of the layer below, and hides how it is implemented. A layer can change (Wi-Fi replaces Ethernet, IPv6 replaces IPv4) without changing the layers above it. Two reference models describe the division: the OSI model and the TCP/IP model.

1.3.1 Concept and mechanism

The OSI model and the TCP/IP model

The **OSI model** (Open Systems Interconnection, ISO standard) has seven layers. The **TCP/IP model** reflects the protocols actually deployed on the Internet and has four layers in its original form; textbooks commonly split its bottom layer in two and use five.



TLS is usually placed between application and transport; it does not map cleanly to one OSI layer.

Figure 1.9. The OSI application, presentation and session layers collapse into the single TCP/IP application layer, and the lower four correspond one to one in the five-layer view; the right column gives the protocols and the protocol data unit of each layer.

No.	Layer	Responsibility	Examples
7	Application	Protocols that serve the application: message formats and semantics	HTTP, DNS, SMTP, SSH, DHCP
6	Presentation	Data representation, encoding, encryption, compression	TLS record encoding, JSON, UTF-8
5	Session	Dialogue control, session establishment and recovery	TLS sessions, RPC sessions
4	Transport	Process-to-process delivery (ports); reliability and flow control or neither	TCP, UDP, QUIC
3	Network	Host-to-host delivery across networks: addressing and routing	IP, ICMP
2	Data link	Delivery between adjacent nodes on one link; framing, MAC addressing, error detection	Ethernet, Wi-Fi (802.11), ARP
1	Physical	Transmission of raw bits as signals	Copper, fibre, radio

Responsibilities of each layer (five-layer view with OSI numbers).

In practice the three upper OSI layers are implemented by one application and its libraries, so the TCP/IP model merges them. The OSI numbers remain in everyday use: "layer 2 switch", "layer 3 router", "layer 4 load balancer" and "layer 7 proxy" refer to the OSI numbering. TLS has no clean place: it runs on top of TCP and below HTTP, and is placed at layer 5, 6 or "4.5" depending on the author. This is a property of the model, not an error.

Protocol data units

The unit of data handled by a layer is its **protocol data unit** (PDU). The names are conventional: a message at the application layer, a **segment** for TCP or a **datagram** for UDP at the transport layer, a **packet** (or IP datagram) at the network layer, a **frame** at the data-link layer, and bits at the physical layer. The terms "packet" and "datagram" are often used loosely; this guide uses the precise terms where the layer matters.

Encapsulation and decapsulation

When an application sends data, each layer treats the PDU received from above as opaque payload and prepends its own **header** (the data link layer also appends a trailer). This is **encapsulation**. The receiving host reverses the process: each layer reads and removes its header and passes the payload upward (**decapsulation**). A layer on one host communicates logically with the same layer on the other host (a peer-to-peer relation), but physically the data travels down the stack, across the medium and up the other stack.

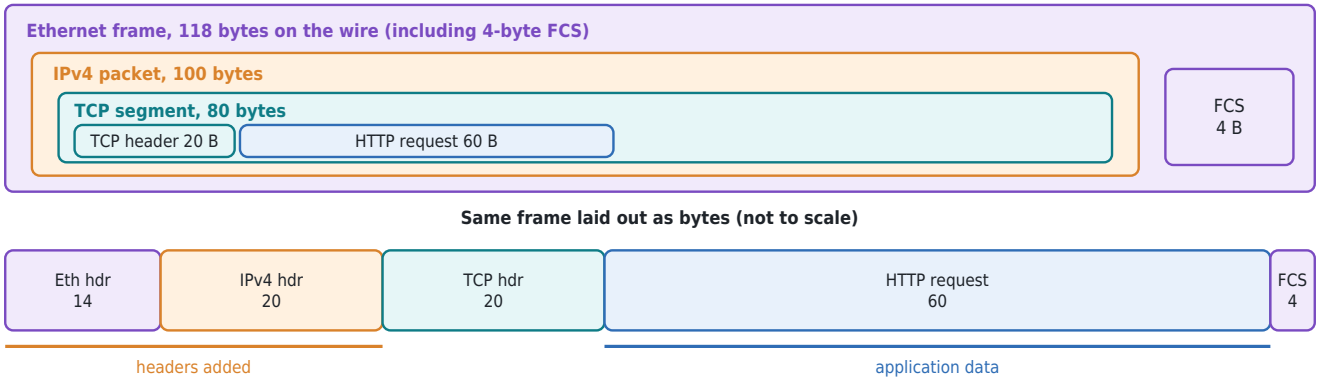


Figure 1.10. A 60-byte HTTP request becomes an 80-byte TCP segment, a 100-byte IPv4 packet and a 114-byte frame (118 bytes including the frame check sequence); headers are 58 bytes, 49 percent of the transmitted bytes.

Intermediate devices implement only the lower layers. A switch processes layers 1 and 2; a router processes layers 1 to 3. Neither examines the TCP or HTTP content in normal operation. Only the end hosts implement all layers. A device that works above its normal layer is a **middlebox**: a firewall that tracks TCP connections, a NAT device that rewrites ports, or a layer 7 proxy that parses HTTP.

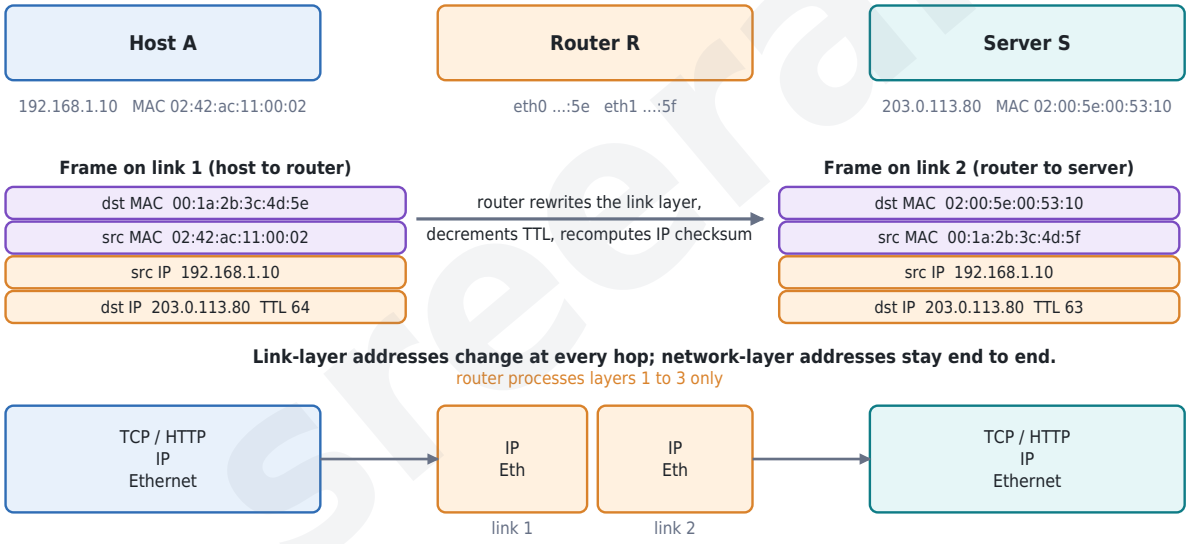


Figure 1.11. The same IP addresses appear on both links, while the MAC addresses are replaced at the router; the router decrements the TTL, which also changes the IP header checksum.

1.3.2 Worked trace

An HTTP request from a host to a server in another network

Host A (192.168.1.10, MAC 02:42:ac:11:00:02) requests `/index.html` from `example.com`, assumed to resolve to the documentation address 203.0.113.80. The host and its default gateway (192.168.1.1, MAC 00:1a:2b:3c:4d:5e) are in one network; the server is in another. The TCP connection is already established; the client source port is 49152, the destination port is 80, the client sequence number is 1000 and the acknowledgement number is 5000. The steps below follow the first data segment.

1. **Application layer.** The HTTP client creates the 60-byte message `GET /index.html HTTP/1.1`, `Host: example.com`, `Accept: */*` and a blank line (each line ends with CR LF). It passes the bytes to the socket.
2. **Transport layer.** TCP adds a 20-byte header: source port 49152 (0xC000), destination port 80 (0x0050), sequence number 1000 (0x000003E8), acknowledgement number 5000 (0x00001388), header length 5 words

(0x50), flags PSH and ACK (0x18), window 64240 (0xFAF0), checksum 0x1670 and urgent pointer 0. The segment is 80 bytes. The checksum covers the segment and a pseudo-header with both IP addresses.

3. **Network layer.** IPv4 adds a 20-byte header: version and header length 0x45, total length 100 (0x0064), identification 0x1C46, flags DF (0x4000), TTL 64 (0x40), protocol 6 for TCP, header checksum 0x204B, source address C0 A8 01 0A (192.168.1.10), destination address CB 00 71 50 (203.0.113.80). The packet is 100 bytes. The routing decision: 203.0.113.80 is not in the local network, so the next hop is the default gateway.
4. **Link layer.** The host needs the MAC address of the gateway (ARP, covered in the link layer chapter) and builds the Ethernet header: destination 00:1a:2b:3c:4d:5e, source 02:42:ac:11:00:02, type 0x0800. The data link layer appends the 4-byte frame check sequence (a CRC-32 over the frame, 0xC37F7484 here as computed by the program below). The frame has 118 bytes.
5. **Physical layer.** The network interface transmits the 118 bytes as signals, preceded by a preamble.
6. **At the router.** The router receives the frame, checks the FCS, removes the Ethernet header and reads the IP header. It selects the outgoing interface from the routing table, decrements the TTL from 64 to 63, recomputes the IP header checksum and builds a new Ethernet header with the MAC addresses of its outgoing link. The TCP segment and HTTP message are not modified.
7. **At the server.** The server's link layer accepts the frame (destination MAC is its own), IP verifies the header checksum and the destination address, delivers the segment to TCP because the protocol field is 6, TCP delivers the data to the process bound to port 80, and the HTTP server reads the 60 bytes.

Layer	PDU	Header or trailer	PDU size (bytes)
Application	HTTP message	none added here	60
Transport	TCP segment	20	80
Network	IPv4 packet	20	100
Data link	Ethernet frame	14 + 4 (FCS)	118

Header sizes and overhead for this request.

The overhead is 58 bytes for 60 bytes of data. For large transfers the proportion falls: a full segment carries 1460 bytes of data in 1518 bytes on the wire (3.8 percent overhead), which is one reason bulk transfers use the largest segments the path allows.

1.3.3 Hands-on

The program below builds the three headers of the trace byte by byte and computes the real checksums. It uses the Internet checksum of RFC 1071 (the same algorithm for IPv4 headers, TCP, UDP and ICMP) and `zlib.crc32` for the Ethernet frame check sequence.

build_frame.py PYTHON

```
import socket
import struct
import zlib


def checksum(data):
    """Internet checksum (RFC 1071): ones-complement sum of 16-bit words, complemented."""
    if len(data) % 2:
        data += b"\0"
    total = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    while total >> 16:
        total = (total & 0xFFFF) + (total >> 16)
    return ~total & 0xFFFF


http = b"GET /index.html HTTP/1.1\r\nHost: example.com\r\nAccept: */*\r\n\r\n"
src_ip, dst_ip = socket.inet_aton("192.168.1.10"), socket.inet_aton("203.0.113.80")

# TCP: ports 49152 -> 80, seq 1000, ack 5000, offset 5 words, flags PSH+ACK, window 64240
tcp = struct.pack("!HHIIBBHHH", 49152, 80, 1000, 5000, 5 << 4, 0x18, 64240, 0, 0) + http
pseudo = src_ip + dst_ip + struct.pack("!BBH", 0, 6, len(tcp))
tcp = tcp[:16] + struct.pack("!H", checksum(pseudo + tcp)) + tcp[18:]

# IPv4: version 4, IHL 5, total length, id 0x1c46, DF flag, TTL 64, protocol 6
ip = struct.pack("!BBHHHBBH4s4s", 0x45, 0, 20 + len(tcp), 0x1C46, 0x4000, 64, 6, 0, src_ip, dst_ip)
ip = ip[:10] + struct.pack("!H", checksum(ip)) + ip[12:]
```

build_frame.py (continued) PYTHON

```python
# Ethernet: destination = gateway MAC, source = host MAC, type 0x0800 (IPv4)
eth = bytes.fromhex("001a2b3c4d5e") + bytes.fromhex("0242ac110002") + b"\x08\x00"
frame = eth + ip + tcp
fcs = zlib.crc32(frame)

print("HTTP payload :", len(http), "bytes")
print("TCP segment  :", len(tcp), "bytes (header 20)")
print("IPv4 packet  :", len(ip) + len(tcp), "bytes (header 20)")
print("Ethernet     :", len(frame), "bytes + 4 FCS =", len(frame) + 4)
print("Ethernet hdr :", eth.hex(" "))
print("IPv4 hdr     :", ip[:20].hex(" "))
print("TCP hdr      :", tcp[:20].hex(" "))
print("IP checksum  : 0x%04x   TCP checksum: 0x%04x   FCS: 0x%08x" % (
    struct.unpack("!H", ip[10:12])[0], struct.unpack("!H", tcp[16:18])[0], fcs))
```

Run the program TERMINAL

```
$ python3 build_frame.py
HTTP payload : 60 bytes
TCP segment  : 80 bytes (header 20)
IPv4 packet  : 100 bytes (header 20)
Ethernet     : 114 bytes + 4 FCS = 118
Ethernet hdr : 00 1a 2b 3c 4d 5e 02 42 ac 11 00 02 08 00
IPv4 hdr     : 45 00 00 64 1c 46 40 00 40 06 20 4b c0 a8 01 0a cb 00 71 50
TCP hdr      : c0 00 00 50 00 00 03 e8 00 00 13 88 50 18 fa f0 16 70 00 00
IP checksum  : 0x204b   TCP checksum: 0x1670   FCS: 0xc37f7484
```

The application layer is plain text, as the following session against the local Java `HttpServer` shows. The `nc` command sends exactly the bytes of an HTTP request and prints the raw response; `curl -v` shows the same exchange with its own annotations (`>` request, `<` response).

Raw HTTP over a TCP socket with nc TERMINAL

```
$ printf 'GET /hello HTTP/1.1\r\nHost: 127.0.0.1:8080\r\nConnection: close\r\n\r\n' \
    | nc -q1 127.0.0.1 8080
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 16:47:53 GMT
Content-type: text/plain
Content-length: 15

Hello, network
```

The same exchange as seen by curl -v TERMINAL

```
$ curl -sv http://127.0.0.1:8080/hello
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET /hello HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sat, 17 Oct 2026 16:47:58 GMT
< Content-type: text/plain
< Content-length: 15
<
Hello, network
```

A Java program observes only the top of the stack: `socket.getOutputStream().write(bytes)` hands 60 bytes to TCP, and the operating system adds the TCP, IP and Ethernet headers. The application never sees the lower headers; they are visible only to a packet capture tool.

1.3.4 Failure modes

- **Layer confusion in diagnosis.** A failure is easiest to locate by working through the layers in order: link up and ARP resolved (layers 1-2), IP reachable (layer 3), port open (layer 4), application responds correctly (layer 7). The troubleshooting part develops this into a playbook.
- **Headers reduce the usable payload.** A path MTU of 1500 bytes leaves 1460 bytes of TCP payload (1500 - 20 - 20). Additional encapsulation (a VPN or tunnel header, or a VLAN tag) reduces it further. If a sender uses a larger segment size than the path allows, packets are fragmented or dropped; the link layer chapter and the ICMP chapter explain path MTU discovery.
- **Violations of layering.** NAT rewrites addresses and ports (layers 3-4); stateful firewalls track TCP state; HTTP proxies terminate and reopen connections. These middleboxes are useful but they break the end-to-end principle: they can modify, delay or reject traffic that the endpoints consider correct, and they explain failures that no endpoint log records.
- **Checksum offload confusion.** A capture taken on the sending host may show incorrect TCP or IP checksums because the network interface computes them after the capture point. The packets are correct on the wire.
- **Assuming message boundaries.** TCP delivers a byte stream; one `write` of 60 bytes may arrive as several `read` results, and several writes may merge in one read. Protocols must define their own framing (HTTP uses Content-Length or chunked encoding).

1.3.5 Questions to expect

1. Name the layers of the OSI model and the TCP/IP model. OSI: physical, data link, network, transport, session, presentation, application. TCP/IP: link (often split into physical and data link), internet, transport, application.
2. What is encapsulation? Each layer adds its own header (and for the link layer a trailer) around the PDU of the layer above. The receiver removes them in reverse order.
3. At which layer do switches, routers and load balancers operate? Switches at layer 2 (MAC addresses), routers at layer 3 (IP addresses), load balancers at layer 4 (IP and port) or layer 7 (HTTP content).
4. What are the PDUs of the transport, network and link layers? Segment (TCP) or datagram (UDP), packet, and frame respectively.
5. What is the purpose of layering? It separates concerns, lets each layer be replaced independently and allows interoperability through standard interfaces. The cost is some overhead and occasional violations for efficiency.
6. Which addresses change when a packet crosses a router, and which do not? MAC addresses change on every hop; IP source and destination addresses stay the same (unless NAT rewrites them). The TTL is decremented and the IP checksum recomputed.
7. Where does TLS belong in the layer models? It sits between the application protocol (HTTP) and TCP; it has no exact OSI layer and is commonly assigned to presentation or session.
8. A request carries 60 bytes of HTTP data. How many bytes are transmitted? With minimal TCP and IPv4 headers and an Ethernet frame: 60 + 20 + 20 + 14 + 4 = 118 bytes (excluding preamble and inter-frame gap).

INTERVIEW ANGLE

Follow-up chain: "Why do routers not look at TCP ports?" A router forwards by the destination IP address using its routing table and needs no transport information; this keeps forwarding fast and independent of the transport protocol. Devices that do examine ports (firewalls, NAT, layer 4 load balancers) are performing functions that go beyond forwarding. They must hold state per flow, which has a memory and failure cost.

KEY POINTS

Key points

- Layers hide implementation and offer a service upward; the TCP/IP model has four or five layers, and the OSI numbers (2, 3, 4, 7) are used when naming devices.
- Each layer encapsulates the PDU from above: HTTP message, TCP segment, IP packet, Ethernet frame.
- For a 60-byte HTTP request the minimum overhead is 58 bytes (14 + 20 + 20 + 4); the figures can be reproduced by building the headers in code.
- MAC addresses are rewritten at every hop; IP addresses identify the endpoints and stay unchanged (apart from NAT).
- Switches work at layers 1-2, routers at 1-3, endpoints at all layers; middleboxes that go beyond their layer cause hard-to-diagnose behaviour.

1.4 The Link Layer

The network layer delivers a packet between two hosts that may be many hops apart. The link layer solves a narrower problem: moving one packet across one link, from one node to an adjacent node. It defines how bits are grouped into **frames**, how nodes on a shared medium are identified (MAC addresses), how transmission errors are detected, and how devices that join several segments (switches) forward frames. Almost every host in a data centre or office uses Ethernet at this layer; wireless networks use Wi-Fi, described in the next chapter. The mechanisms in this chapter explain why a server on the same subnet answers faster than one reached through a router, how an IP address is converted into a hardware address, and why a misconfigured switch can bring down an entire floor.

1.4.1 Concept and mechanism

Frames and MAC addresses

A **frame** is the PDU of the link layer: a header with addresses, the payload (an IP packet or an ARP message), and a trailer with an error-detection code. Every network interface has a **MAC address** (media access control address), a 48-bit number written as six hexadecimal octets, for example `02:42:ac:11:00:02`. The first three octets are the **organizationally unique identifier** (OUI) assigned to the manufacturer; the last three identify the interface. Two bits of the first octet have a defined meaning: the least significant bit marks a group (multicast) address, and the next bit marks a locally administered address, one assigned by software instead of by the manufacturer (virtual machines and containers use these). The address `ff:ff:ff:ff:ff:ff` is the **broadcast** address: every interface on the local network accepts the frame.

A MAC address is meaningful only inside one local network and has no topology information. An IP address identifies a host on the Internet and is assigned according to topology. A frame therefore carries the MAC address of the next hop, while the IP packet inside it carries the address of the final destination.

Ethernet frame format

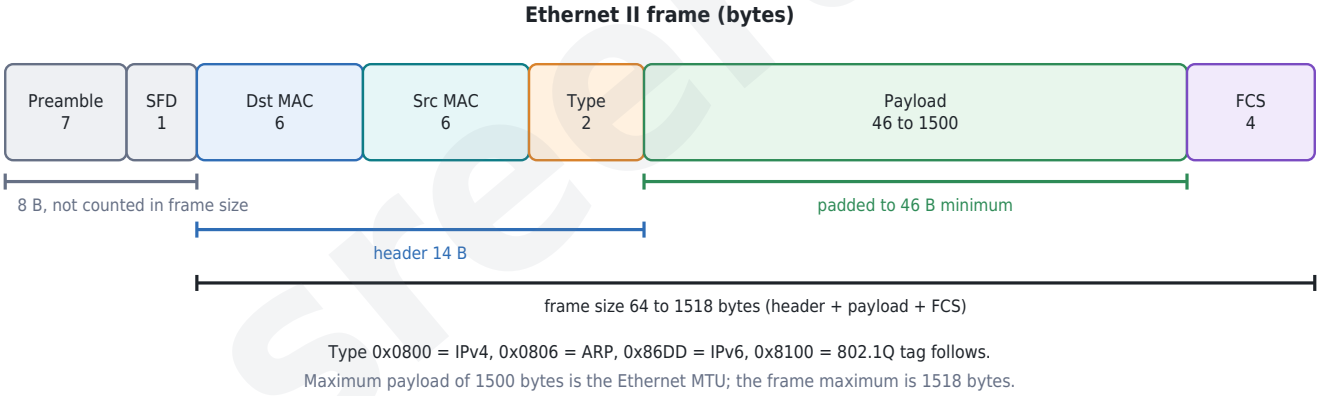


Figure 1.12. An Ethernet II frame consists of a 14-byte header, a payload of 46 to 1500 bytes and a 4-byte frame check sequence; the preamble and start-of-frame delimiter that precede it are not counted in the frame size.

The fields are:

- **Preamble (7 bytes) and start frame delimiter (1 byte).** An alternating bit pattern that allows the receiver to synchronise its clock. These bytes are produced by the interface hardware and do not appear in captures.
- **Destination and source MAC addresses (6 bytes each).**
- **Type (2 bytes).** Identifies the payload protocol: 0x0800 for IPv4, 0x0806 for ARP, 0x86DD for IPv6, 0x8100 for an 802.1Q VLAN tag. Values of 1500 or less in this field are a length in the older IEEE 802.3 format.
- **Payload (46 to 1500 bytes).** A shorter payload is padded to 46 bytes so that the frame is at least 64 bytes. The upper limit of 1500 bytes is the **maximum transmission unit** (MTU).
- **Frame check sequence (4 bytes).** A CRC-32 computed over the header and payload. A receiver that finds a mismatch discards the frame silently; retransmission, if any, is the responsibility of a higher layer.

Between two frames the sender leaves an inter-frame gap of 12 byte times. The maximum frame is 1518 bytes (1522 with a VLAN tag).

Shared media, hubs and collisions

Early Ethernet used a shared coaxial cable and later a **hub**, a layer 1 repeater that copies every incoming bit to all other ports. All hosts then form one **collision domain**: if two transmit at once, the signals collide. Access was controlled by **CSMA/CD** (carrier sense multiple access with collision detection): a host listens before sending; if it detects a collision while sending, it stops, sends a jam signal and waits a random time chosen from a range that doubles after each successive collision (binary exponential backoff) before retrying. Hubs are obsolete. Modern Ethernet links are point-to-point and full duplex, so a host can send and receive at the same time and collisions do not occur.

Switches and MAC learning

A **switch** is a layer 2 device that forwards frames only to the port where the destination is located. It builds its **MAC address table** (also called the forwarding table or CAM table) automatically by **backward learning**:

1. When a frame arrives on a port, the switch records the source MAC address and the arrival port in the table, with a timestamp.
2. It looks up the destination MAC address. If the table has an entry for another port, the switch **forwards** the frame to that port only. If the entry points to the arrival port itself, the switch **filters** (discards) the frame, because the destination is on the same segment.
3. If the destination is unknown, or is a broadcast or multicast address, the switch **floods** the frame to all ports except the one on which it arrived.
4. Entries that are not refreshed expire; a common default is 300 seconds.

The switch is transparent: hosts do not know it exists and it needs no configuration to work.

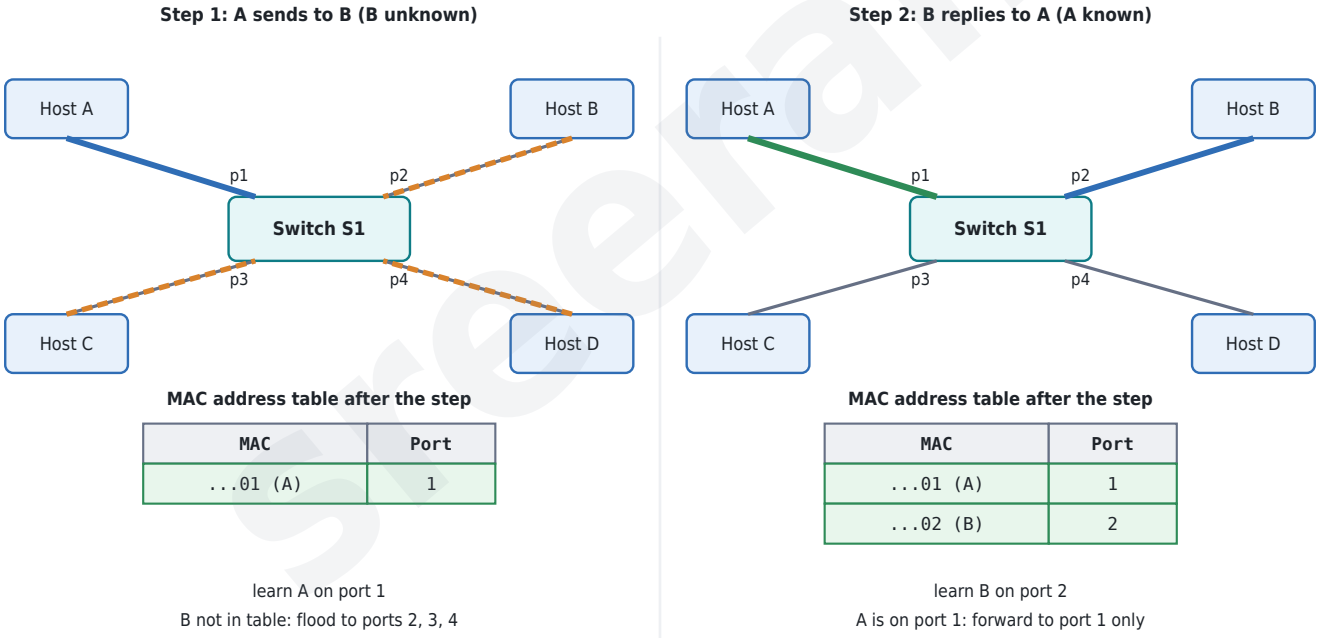


Figure 1.13. In the first step the frame for the unknown destination B is flooded while the source A is learned; in the second step the table already contains A, so the reply is forwarded to port 1 only.

Because each port is its own collision domain and each link is full duplex, a switch gives every connected host the full link rate. A switch can forward in **store-and-forward** mode (the whole frame is received and its FCS checked before forwarding) or in **cut-through** mode (forwarding begins after the destination address has been read, lowering latency but forwarding corrupted frames).

Collision domains and broadcast domains

A **collision domain** is a set of interfaces whose frames can collide. A **broadcast domain** is a set of interfaces that receive each other's broadcast frames. A hub forms a single domain of both kinds. A switch gives each port its own collision domain but keeps all ports in one broadcast domain, because it floods broadcasts. A router does not forward link-layer broadcasts, so each router interface is a separate broadcast domain.

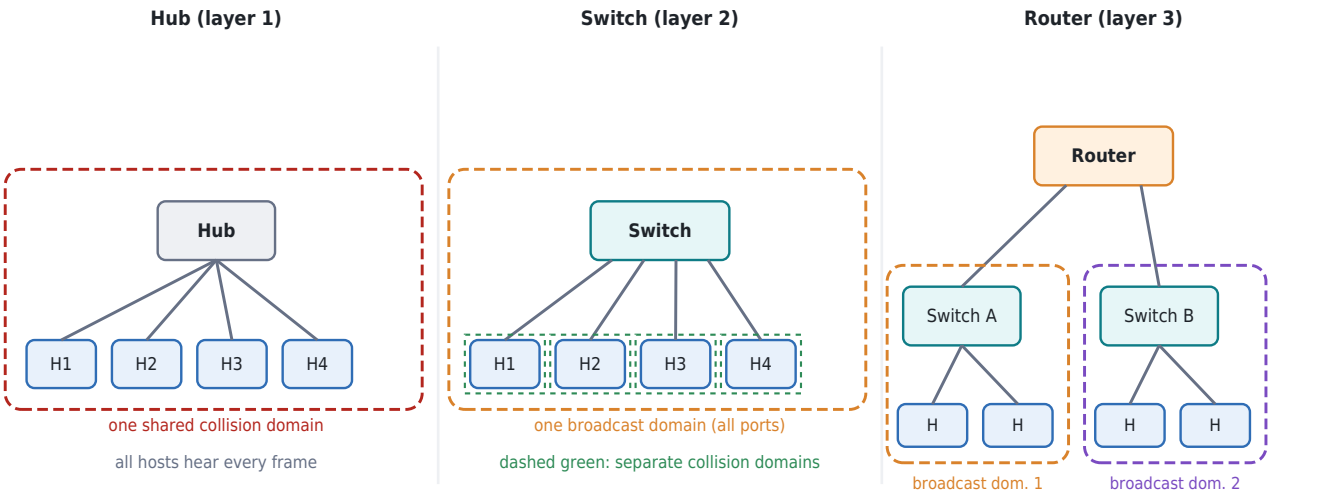


Figure 1.14. A hub is one collision domain; a switch separates collision domains (dashed green) but remains one broadcast domain; a router splits broadcast domains.

Large broadcast domains are undesirable: every host must process every broadcast (ARP requests, DHCP discovery), and a switching loop turns one broadcast into a storm. Switches therefore run the **Spanning Tree Protocol** (STP, IEEE 802.1D and its faster variants), which blocks redundant links to leave a loop-free tree and unblocks them if an active link fails. Ethernet frames have no TTL, so a loop without STP circulates frames until the network is saturated.

VLANs and 802.1Q

A **virtual LAN** (VLAN) divides one physical switch infrastructure into several independent broadcast domains. A department or a service tier (web, application, database) can have its own VLAN without separate hardware. Each VLAN is identified by a 12-bit **VLAN ID**; the values 1 to 4094 are usable. Switch ports are of two kinds:

- An **access port** belongs to one VLAN. Frames on it are untagged; the switch adds the VLAN association internally.
- A **trunk port** carries several VLANs between switches (or to a router or server). Frames are marked with an **802.1Q tag** of 4 bytes inserted after the source MAC: a 16-bit TPID with the value 0x8100, then a 16-bit TCI containing a 3-bit priority (PCP), a 1-bit drop-eligible indicator (DEI) and the 12-bit VLAN ID. Frames of the **native VLAN** may be sent untagged on a trunk.

A frame never crosses from one VLAN to another at layer 2. Communication between VLANs needs a router (or a layer 3 switch), which has one interface or sub-interface per VLAN; each VLAN normally corresponds to one IP subnet.

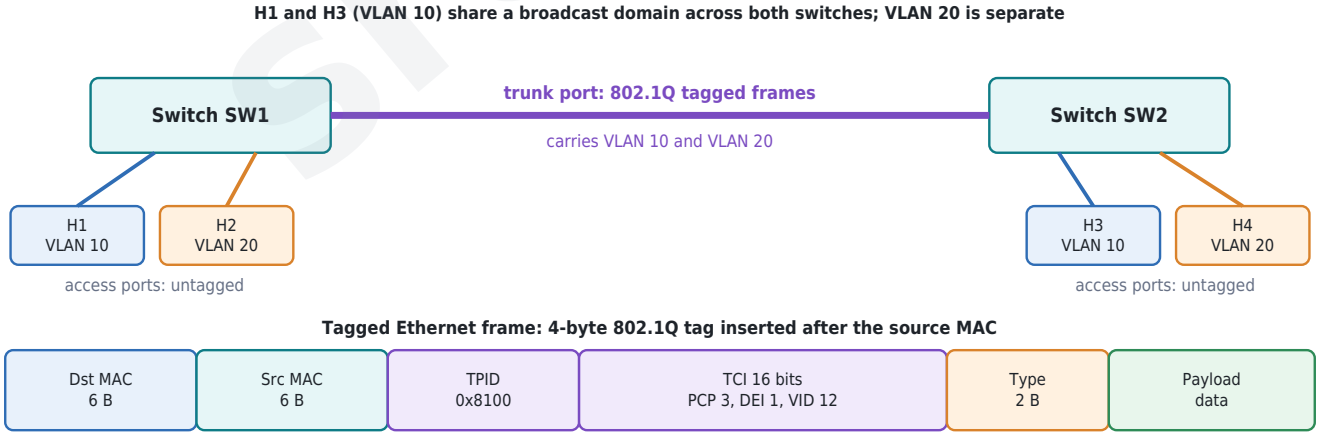


Figure 1.15. Hosts H1 and H3 in VLAN 10 communicate through the tagged trunk as if on one switch, while VLAN 20 stays isolated; the tag adds 4 bytes after the source MAC address.

ARP: from IP address to MAC address

A host that wants to send an IP packet to a destination on the same subnet needs the destination MAC address. For a destination on another subnet it needs the MAC address of its **default gateway**. The **Address Resolution Protocol** (ARP, RFC 826) provides this mapping for IPv4. IPv6 uses the Neighbor Discovery Protocol for the same purpose.

The sender first consults its **ARP cache**. On a miss, it broadcasts an **ARP request** (Ethernet type 0x0806, destination ff:ff:ff:ff:ff:ff) asking "who has IP address X?". Every host on the segment receives it, but only the owner of X answers,

with a unicast **ARP reply** that carries its MAC address. The requester stores the pair in its cache for a limited time (typically tens of seconds up to a few minutes, depending on the operating system) and sends the pending frame. A host also learns the sender's mapping from a received request, so one exchange usually populates both caches. A **gratuitous ARP** is a request or reply that a host sends for its own address, to announce a change or to detect an address conflict.

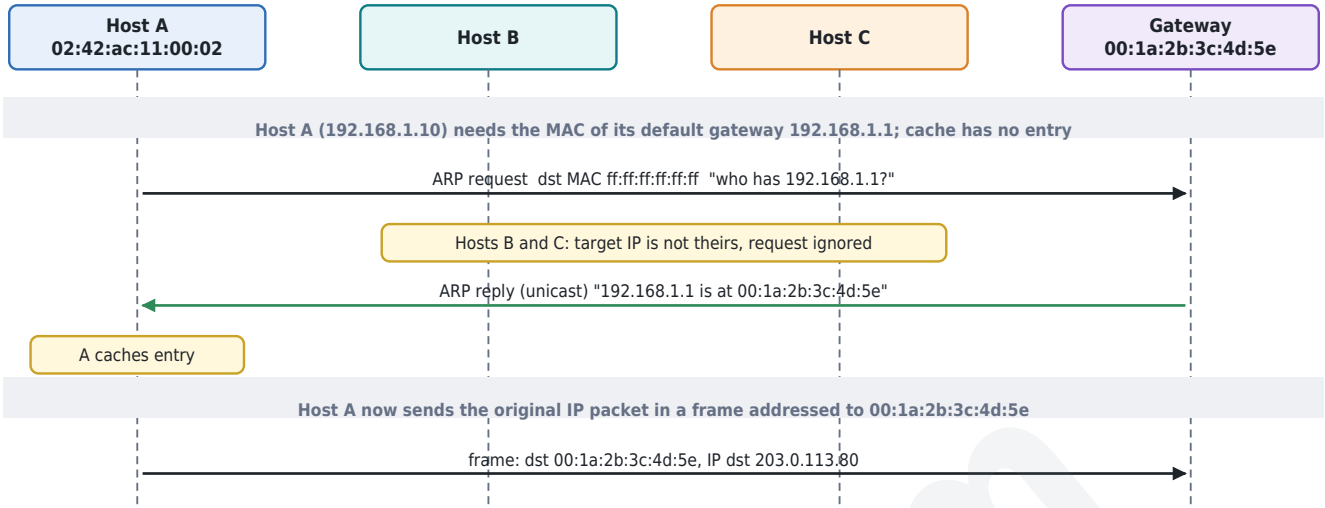


Figure 1.16. The ARP request is a broadcast heard by all hosts; only the gateway replies, by unicast, after which the original IP packet is sent in a frame addressed to the learned MAC address.

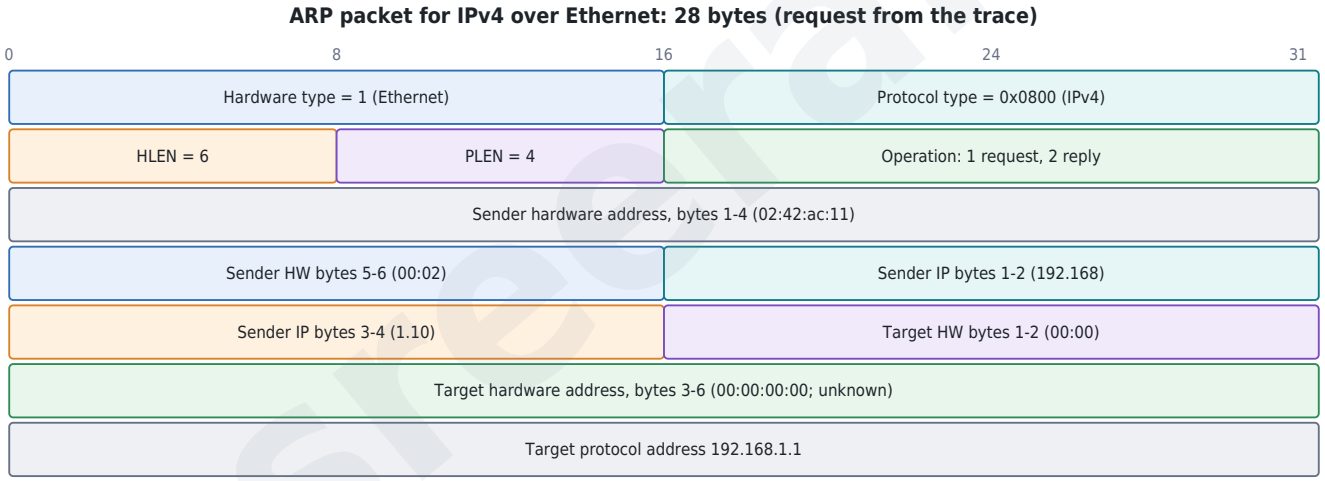


Figure 1.17. The ARP message is a fixed 28-byte structure that names the hardware and protocol types, the operation, and the sender and target addresses; in a request the target hardware address is zero.

ARP has no authentication: any host can send a reply, and the most recent one is accepted. This is the basis of ARP spoofing, covered in the part on security.

MTU and the link layer

The **MTU** is the largest payload, in bytes, that a link carries in one frame. For Ethernet it is 1500. The IP layer must not hand the link a packet larger than its MTU. TCP avoids the problem by limiting its segment to the **maximum segment size** (MSS): MTU - 20 (IPv4) - 20 (TCP) = 1460 bytes on Ethernet. Links differ: PPPoE (DSL) usually has 1492, tunnels subtract their encapsulation header (a VXLAN overlay adds 50 bytes), and data-centre networks sometimes use jumbo frames of about 9000 bytes. The smallest MTU along a path is the **path MTU**; discovery and fragmentation belong to the part on the network layer.

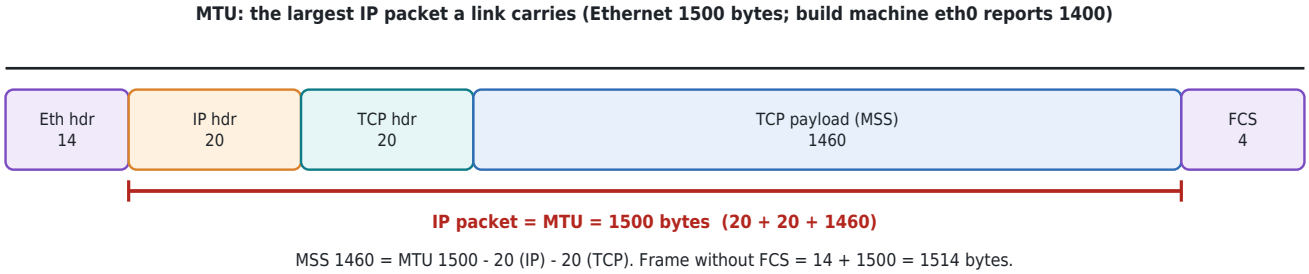


Figure 1.18. The MTU limits the IP packet, so a TCP segment on Ethernet carries at most 1460 bytes of data; the Ethernet header and FCS are outside the MTU.

1.4.2 Worked trace

MAC learning on a four-port switch

Hosts A, B, C and D are attached to ports 1 to 4 of switch S1. Their MAC addresses are `aa:00:00:00:00:01` to `aa:00:00:00:00:04`; the table is initially empty. The following frames arrive, in order. The Java program in the next section computes the same result.

No.	Arrival	Frame	Action and table afterwards
1	port 1	A to B	Learn A on 1. B unknown: flood to ports 2, 3, 4. Table: A=1
2	port 2	B to A	Learn B on 2. A known on port 1: forward to port 1 only. Table: A=1, B=2
3	port 3	C to broadcast	Learn C on 3. Broadcast: flood to ports 1, 2, 4. Table: A=1, B=2, C=3
4	port 1	A to C	A already known. C on port 3: forward to port 3 only
5	port 1	A to 09 (nobody)	Destination unknown: flood to ports 2, 3, 4; no reply, no entry learned

Table updates and forwarding decisions (frames 1 and 2 are the diagram above).

Hosts D and the unknown address never appear in the table because D has not sent a frame; the switch learns only from source addresses. After 300 seconds without traffic, entries age out. If host B is moved from port 2 to port 4, the first frame it sends from the new port overwrites the entry (B=4) and traffic follows without configuration.

ARP resolution of the default gateway

Host A (192.168.1.10, MAC `02:42:ac:11:00:02`) sends to 203.0.113.80 via the gateway 192.168.1.1. Its cache has no entry for the gateway.

1. A builds an ARP request: operation 1, sender hardware address 02:42:ac:11:00:02, sender protocol address 192.168.1.10, target hardware address 00:00:00:00:00:00, target protocol address 192.168.1.1.
2. A wraps it in an Ethernet frame: destination ff:ff:ff:ff:ff:ff, source 02:42:ac:11:00:02, type 0x0806. The 28-byte ARP message is padded to 46 bytes, so the frame is 64 bytes with the FCS.
3. The switch floods the frame to all ports of the VLAN. Hosts B and C compare the target address with their own, find no match and ignore the message.
4. The gateway recognises its address, records the sender's mapping (192.168.1.10 is at 02:42:ac:11:00:02) and sends a unicast reply: operation 2, sender hardware address 00:1a:2b:3c:4d:5e, sender protocol address 192.168.1.1, target hardware address 02:42:ac:11:00:02, target protocol address 192.168.1.10.
5. A stores 192.168.1.1 to 00:1a:2b:3c:4d:5e in its cache and transmits the original IP packet in a frame with destination MAC 00:1a:2b:3c:4d:5e.

Every later packet to an off-subnet destination reuses the cache entry. Note that A never obtains the MAC address of 203.0.113.80: ARP resolves only addresses on the local network.

1.4.3 Hands-on

The simulator below implements the learning rules above with a `LinkedHashMap` as the MAC address table.

SwitchSim.java JAVA

```java
import java.util.*;

public class SwitchSim {
    static final int PORTS = 4;
    static final Map<String, Integer> table = new LinkedHashMap<>();

    static void frame(int inPort, String src, String dst) {
        table.put(src, inPort);                      // learn: source address seen on ingress port
        String action;
        if (dst.equals("ff:ff:ff:ff:ff:ff") || !table.containsKey(dst)) {
            StringBuilder out = new StringBuilder();
            for (int p = 1; p <= PORTS; p++) if (p != inPort) out.append(p).append(' ');
            String kind = dst.startsWith("ff") ? "broadcast" : "unknown unicast, flood";
            action = kind + " to ports " + out.toString().trim();
        } else if (table.get(dst) == inPort) {
            action = "filter (destination on the same port)";
        } else {
            action = "forward to port " + table.get(dst);
        }
        System.out.printf("port %d  %s -> %s : %s%n", inPort, src, dst, action);
        System.out.println("   table = " + table);
    }

    public static void main(String[] args) {
        String a = "aa:00:00:00:00:01", b = "aa:00:00:00:00:02", c = "aa:00:00:00:00:03";
        frame(1, a, b);
        frame(2, b, a);
        frame(3, c, "ff:ff:ff:ff:ff:ff");
        frame(1, a, c);
        frame(1, a, "aa:00:00:00:00:09");
    }
}
```

Run the simulator TERMINAL

```
$ java SwitchSim.java
port 1  aa:00:00:00:00:01 -> aa:00:00:00:00:02 : unknown unicast, flood to ports 2 3 4
   table = {aa:00:00:00:00:01=1}
port 2  aa:00:00:00:00:02 -> aa:00:00:00:00:01 : forward to port 1
   table = {aa:00:00:00:00:01=1, aa:00:00:00:00:02=2}
port 3  aa:00:00:00:00:03 -> ff:ff:ff:ff:ff:ff : broadcast to ports 1 2 4
   table = {aa:00:00:00:00:01=1, aa:00:00:00:00:02=2, aa:00:00:00:00:03=3}
port 1  aa:00:00:00:00:01 -> aa:00:00:00:00:03 : forward to port 3
   table = {aa:00:00:00:00:01=1, aa:00:00:00:00:02=2, aa:00:00:00:00:03=3}
port 1  aa:00:00:00:00:01 -> aa:00:00:00:00:09 : unknown unicast, flood to ports 2 3 4
   table = {aa:00:00:00:00:01=1, aa:00:00:00:00:02=2, aa:00:00:00:00:03=3}
```

Frames 1 and 5 are flooded, frame 3 is a broadcast, and frames 2 and 4 are forwarded to a single port, as in the trace. A real switch adds a timestamp per entry, ageing, VLANs and a table of limited size.

ARP messages can be constructed explicitly. The following program builds the request and the reply of the trace and shows that each is 28 bytes, and that the frames are padded to the 64-byte Ethernet minimum.

arp_frames.py PYTHON

```python
import socket
import struct

HOST_MAC = bytes.fromhex("0242ac110002")
GW_MAC = bytes.fromhex("001a2b3c4d5e")
HOST_IP = socket.inet_aton("192.168.1.10")
GW_IP = socket.inet_aton("192.168.1.1")


def arp(op, sha, spa, tha, tpa):
    # hardware type 1 (Ethernet), protocol 0x0800 (IPv4), HLEN 6, PLEN 4, operation
    return struct.pack("!HHBBH6s4s6s4s", 1, 0x0800, 6, 4, op, sha, spa, tha, tpa)


request = arp(1, HOST_MAC, HOST_IP, bytes(6), GW_IP)
```

arp_frames.py (continued) PYTHON

```
reply = arp(2, GW_MAC, GW_IP, HOST_MAC, HOST_IP)
broadcast = bytes.fromhex("ffffffffffff")

# Ethernet type 0x0806 carries ARP; the payload is padded to the 46-byte minimum
frame1 = broadcast + HOST_MAC + b"\x08\x06" + request
frame2 = HOST_MAC + GW_MAC + b"\x08\x06" + reply
for name, pkt, frame in (("request", request, frame1), ("reply", reply, frame2)):
    padded = len(frame) + max(0, 60 - len(frame))
    print("ARP %-7s %d bytes: %s" % (name, len(pkt), pkt.hex()))
    print("  frame %d bytes before padding, %d padded, %d with FCS"
          % (len(frame), padded, padded + 4))
```

Run the program TERMINAL

```
$ python3 arp_frames.py
ARP request 28 bytes: 0001080006040001024ac110002c0a8010a000000000000c0a80101
  frame 42 bytes before padding, 60 padded, 64 with FCS
ARP reply   28 bytes: 0001080006040002001a2b3c4d5ec0a801010242ac110002c0a8010a
  frame 42 bytes before padding, 60 padded, 64 with FCS
```

The ARP cache of the host can be read from the kernel. On Linux, `ip neigh` (typical output shown after the real one) and `/proc/net/arp` give the same data.

The ARP cache of the build machine (real output) TERMINAL

```
$ cat /proc/net/arp
IP address       HW type     Flags       HW address            Mask     Device
192.0.2.1        0x1         0x2         02:fc:00:00:00:05     *        eth0
```

Typical output of ip neigh on a Linux host (typical, not run on the build machine) TERMINAL

```
$ ip neigh
192.168.1.1 dev eth0 lladdr 00:1a:2b:3c:4d:5e REACHABLE
192.168.1.23 dev eth0 lladdr 02:42:ac:11:00:07 STALE
192.168.1.77 dev eth0  FAILED
```

`REACHABLE` means the entry was confirmed recently; `STALE` means it is still usable but will be re-verified on the next use; `FAILED` means that no reply was received, so the host is down or not on the segment. The flags value `0x2` in `/proc/net/arp` marks a complete entry. The interface MTU, which `Ifaces.java` from the first chapter printed (1400 on the build machine, 1500 on a typical Ethernet), can be changed with `ip link set dev eth0 mtu 1400` only for a value the whole path supports.

1.4.4 Failure modes

- **Switching loop and broadcast storm.** Two switches connected by two cables without STP, or an accidentally patched cable, make frames circulate. Symptoms: all ports show full utilisation, LEDs blink in unison, hosts lose connectivity and CPU use of switches rises. Remedy: enable STP, enable storm control, remove the redundant link.
- **Duplicate IP address.** Two hosts claim one IP address, so the ARP cache of other hosts flips between two MAC addresses; connections to that address break intermittently. Diagnosis: `arp` or `ip neigh` shows the MAC address changing, and the ARP-based duplicate detection of the operating system logs a conflict.
- **ARP cache failures.** `FAILED` or `INCOMPLETE` entries mean that the target does not answer ARP: it is off, on another VLAN, or a firewall blocks ARP. The symptom is "destination host unreachable" from the local host, not from a router.
- **VLAN misconfiguration.** A port in the wrong VLAN, a missing VLAN on a trunk, or a native VLAN mismatch between two switches isolates a host or leaks traffic between segments. A host receives no DHCP offer or cannot reach its gateway although the link is up.
- **MTU mismatch and black holes.** A tunnel or VLAN tag lowers the usable MTU. Small packets (ping, TCP handshake) pass while large ones disappear, so a connection is established but stalls when data flows. The ICMP chapter covers diagnosis with the do-not-fragment flag.
- **MAC table exhaustion (CAM overflow).** An attacker or fault floods a switch with frames with random source addresses; the full table forces the switch to flood all traffic like a hub. Mitigation: port security limits per port.
- **Duplex mismatch.** One end fixed at half duplex while the other negotiates full duplex produces late collisions and poor throughput on an otherwise healthy link. Remedy: auto-negotiation on both ends.

1.4.5 Questions to expect

1. What is a MAC address and how does it differ from an IP address? A 48-bit hardware address of an interface, meaningful on one local network and fixed by the manufacturer or software. An IP address is a logical address assigned by topology that identifies a host across networks. The MAC address changes at every hop; the IP addresses normally do not.
2. How does a switch learn? From the source address of every received frame, it stores the address and the arrival port. Frames for unknown or broadcast destinations are flooded; frames for known destinations are forwarded to one port.
3. What is the difference between a hub, a switch and a router? A hub repeats bits to all ports (layer 1). A switch forwards frames by MAC address (layer 2) and separates collision domains. A router forwards packets by IP address (layer 3) and separates broadcast domains.
4. What is the difference between a collision domain and a broadcast domain? A collision domain is the set of interfaces that can interfere with each other on a shared medium. A broadcast domain is the set that receives a link-layer broadcast. Switch ports are separate collision domains, but all ports of a VLAN share one broadcast domain.
5. What is a VLAN and why is it used? A logical partition of a switched network into separate broadcast domains, identified by a 12-bit ID. It limits broadcast traffic, isolates tiers or departments and avoids separate hardware. Inter-VLAN traffic needs a router.
6. Explain access and trunk ports. An access port carries one VLAN untagged. A trunk carries several VLANs between switches using 802.1Q tags (4 bytes, TPID 0x8100).
7. How does ARP work, and why is it a broadcast? The requester does not know the target MAC address, so it asks everybody on the segment; the owner of the IP address replies by unicast. Results are cached. ARP is limited to one subnet; for other destinations the host resolves the gateway.
8. What is the Ethernet MTU, and what is the MSS? The MTU is 1500 bytes of payload. The TCP MSS is MTU minus the IP and TCP headers, 1460 bytes.
9. Why does Ethernet need a minimum frame size? In the original half-duplex design a sender had to still be transmitting when a collision signal returned from the far end of the cable; a frame of at least 64 bytes guarantees it. The minimum is kept in full duplex for compatibility.
10. What stops a loop in a switched network? The Spanning Tree Protocol, which blocks redundant links, because Ethernet frames have no TTL.

INTERVIEW ANGLE

Follow-up chain: "Host A pings host B on the same subnet for the first time. List every frame." First an ARP request (broadcast, A asks for B's MAC), then the unicast ARP reply from B. Then the ICMP echo request in a frame addressed to B's MAC, and the ICMP echo reply back. The switch learns A from the ARP request and B from the ARP reply, so the echo frames are forwarded to a single port. If A pings an address on another subnet, the ARP exchange is with the default gateway, not with the destination.

KEY POINTS

Key points

- The link layer moves frames across one link; MAC addresses identify interfaces locally and are replaced at every router hop.
- An Ethernet frame has a 14-byte header, 46 to 1500 bytes of payload and a 4-byte FCS; the MTU is 1500 and the TCP MSS 1460.
- A switch learns source addresses, forwards to a known port, and floods unknown and broadcast destinations; each port is a collision domain but all ports of a VLAN are one broadcast domain.
- VLANs create separate broadcast domains using 802.1Q tags (12-bit ID) on trunks; routing is required between VLANs.
- ARP resolves an IPv4 address on the local network to a MAC address using a broadcast request and a unicast reply, and has no authentication.

Sreeram

1.5 Wireless Basics

Wireless local networks, standardised as IEEE 802.11 and marketed as Wi-Fi, replace the cable of the link layer with a shared radio channel. The layers above (IP, TCP, HTTP) are unchanged, but the physical properties of radio alter the link layer design: the medium is shared by all stations within range, signals weaken and reflect, a station cannot listen while it transmits, and any device in range can receive the transmission. This chapter covers the structure of a Wi-Fi network, the association procedure, the access method CSMA/CA and why it differs from the CSMA/CD of classic Ethernet, the hidden terminal problem, and the radio bands. It is deliberately brief; developers meet these topics mostly as intermittent latency and packet loss on client devices.

1.5.1 Concept and mechanism

Components and names

A **station** (STA) is any device with a Wi-Fi interface. An **access point** (AP) bridges the wireless stations to the wired network. An AP and its associated stations form a **basic service set** (BSS). Each BSS is identified by a **BSSID**, which is the MAC address of the radio of the AP. The human-readable network name is the **SSID** (service set identifier), up to 32 bytes. Several APs with the same SSID, connected by a wired network, form an **extended service set** (ESS); a station can move (**roam**) between APs of one ESS while keeping its IP address, because the APs and the wired network are one layer 2 network. One AP can broadcast several SSIDs, each with its own BSSID, often mapped to separate VLANs (for example staff and guests).

An 802.11 frame has up to four address fields, because a frame may be sent between a station, an AP and a wired destination. For a frame from a station to the AP, the receiver address is the BSSID, the transmitter address is the station, and the third address is the final destination on the wired network. Frames are classified as management (beacon, probe, authentication, association), control (RTS, CTS, ACK) and data frames.

Association

A station joins a network in three phases, shown below.

1. **Discovery.** The AP broadcasts a **beacon** frame about ten times per second with its SSID, supported rates and security capabilities. A station finds networks by listening (passive scanning) or by sending a **probe request** and receiving a probe response (active scanning).
2. **Authentication and association.** The station sends an authentication request and then an association request naming the SSID and its capabilities; the AP answers with a status and an association ID. In WPA3-Personal the authentication frames carry a password-based key exchange (SAE) instead of the empty "open system" exchange used by earlier modes.
3. **Key establishment.** In a protected network (WPA2 or WPA3) a four-message **4-way handshake** (carried in EAPOL frames) proves that both sides know the pairwise master key and derives the session keys that encrypt all data frames.

After the link layer association is complete, the station is an ordinary host on the layer 2 network and obtains an IP address with DHCP (described in the part on the network layer).

sreeram

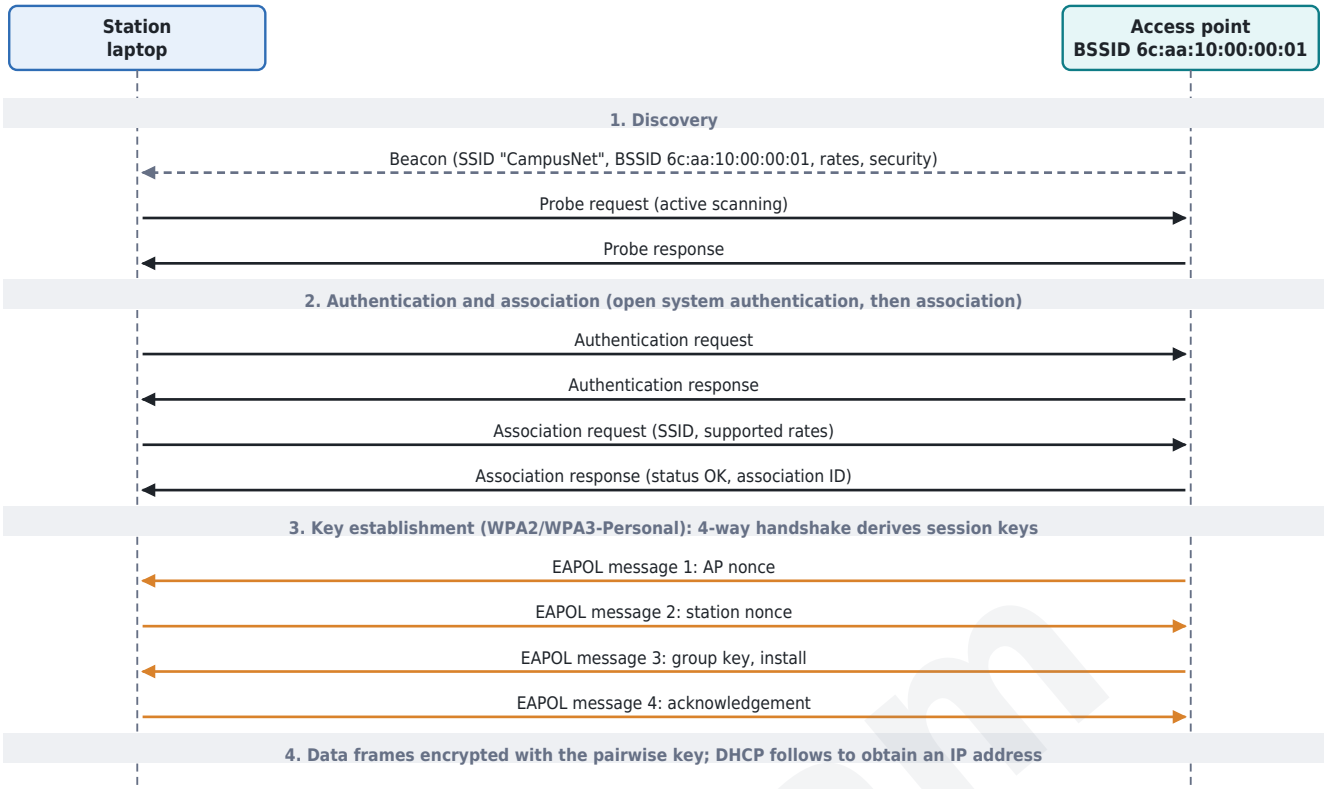


Figure 1.19. Discovery, authentication, association and the 4-way handshake all complete at layer 2 before the first DHCP or IP packet is sent.

CSMA/CA instead of CSMA/CD

Wired Ethernet detected collisions by listening while sending. A radio cannot do this: its own transmission is millions of times stronger than the signal it would need to detect, and a collision occurs at the receiver, which may be out of range of the other sender. 802.11 therefore uses **CSMA/CA** (collision avoidance) in a procedure called the distributed coordination function:

1. A station with data senses the channel. If it has been idle for a **DIFS** (distributed inter-frame space), it transmits immediately.
2. If the channel is busy, the station waits until it becomes idle, then waits one DIFS and counts down a random **backoff**, a number of time slots chosen from the **contention window**. The counter is frozen while the channel is busy and resumes when it is idle again. The station transmits when the counter reaches zero.
3. The receiver sends an **ACK** frame after a shorter gap, the **SIFS**, so that no other station can interrupt between data and ACK.
4. If no ACK arrives, the sender assumes a collision or a corrupted frame, doubles the contention window (from 15 slots up to a maximum of 1023) and retries, up to a retry limit; after the limit the frame is dropped.

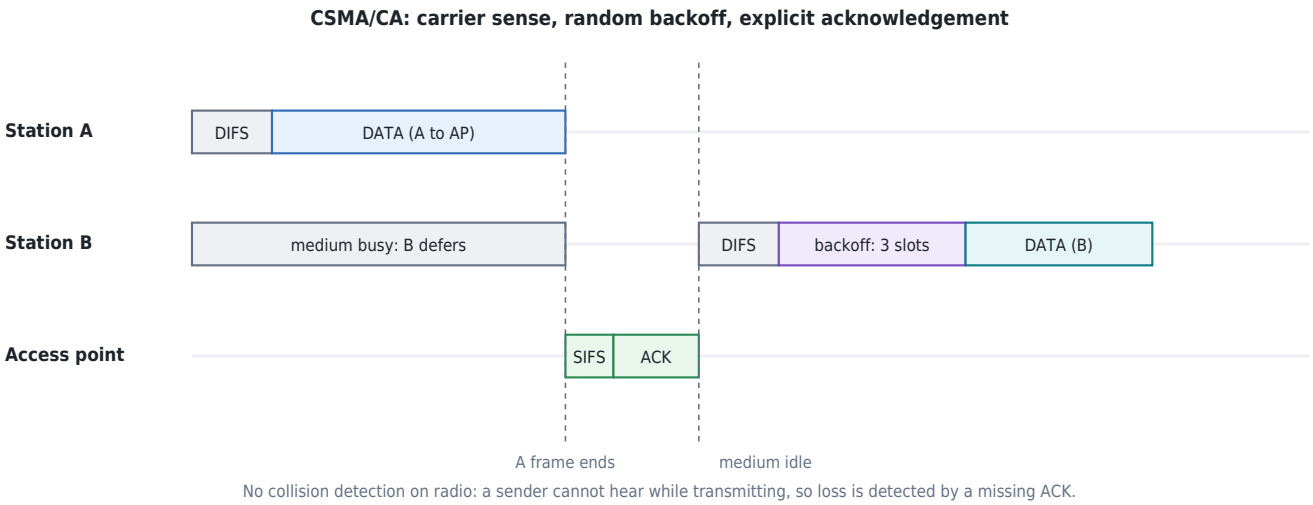


Figure 1.20. Station B defers while A sends, resumes its backoff countdown after the idle period, and the AP acknowledges A after only a SIFS, which is shorter than a DIFS and therefore wins the channel.

Unlike Ethernet, every unicast data frame is individually acknowledged at layer 2. This explains why Wi-Fi hides most radio loss from TCP, and why the cost is variable delay: a frame retried several times adds milliseconds of jitter without any packet being lost at the IP layer.

The hidden terminal problem and RTS/CTS

Two stations may both be in range of the AP but out of range of each other. Each senses an idle channel and transmits; the frames collide at the AP, and neither station can detect it. This is the **hidden terminal problem**. The remedy is optional: the sender first transmits a short **RTS** (request to send), the AP replies with a **CTS** (clear to send), and every station that hears either frame defers for the announced duration. The exchange costs overhead, so it is used only for large frames or in environments with hidden stations.

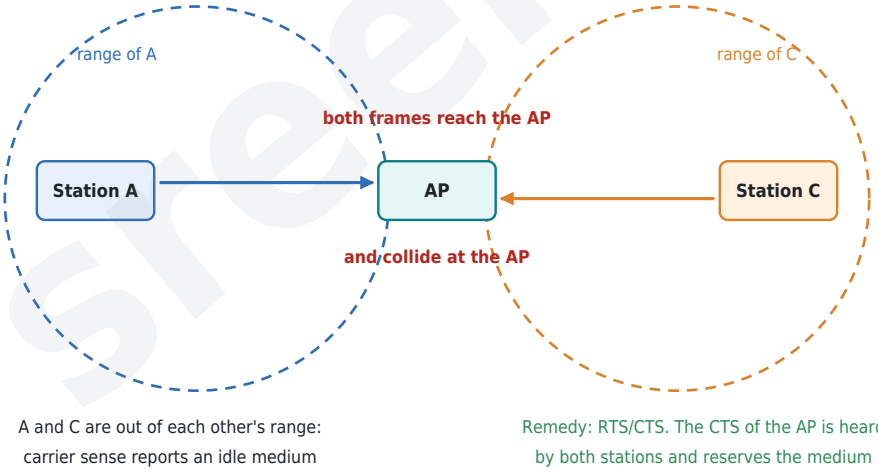


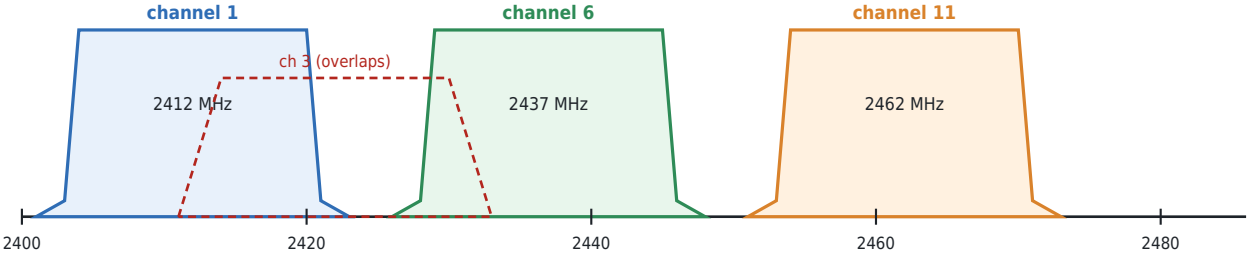
Figure 1.21. Stations A and C cannot hear each other, so carrier sense reports an idle medium while both transmit to the AP; the CTS from the AP is heard by both and reserves the channel.

The opposite case, the **exposed terminal**, occurs when a station defers although its transmission would not interfere with an ongoing one; it reduces efficiency but causes no errors.

Bands, channels and standards

Wi-Fi operates in unlicensed spectrum. The **2.4 GHz band** offers longer range and better wall penetration but only three non-overlapping 20 MHz channels (1, 6 and 11 in most regions), and it is shared with Bluetooth, cordless phones and microwave ovens. The **5 GHz band** offers many more channels, wider channels and less interference but a shorter range. The 6 GHz band was added from Wi-Fi 6E onward for still more channels.

5 GHz offers many non-overlapping 20 MHz channels: less interference, shorter range.



2.4 GHz band: 22 MHz wide channels spaced 5 MHz apart; only 1, 6, 11 do not overlap (in regions with channels 1 to 11).

Figure 1.22. Channels in the 2.4 GHz band are spaced 5 MHz apart but are about 22 MHz wide; only channels 1, 6 and 11 are free of mutual overlap, and a neighbouring AP on channel 3 interferes with both 1 and 6.

Generation	Standard	Bands	Maximum PHY rate
Wi-Fi 4	802.11n	2.4 and 5 GHz	600 Mbit/s
Wi-Fi 5	802.11ac	5 GHz	about 6.9 Gbit/s
Wi-Fi 6 / 6E	802.11ax	2.4, 5 (and 6) GHz	about 9.6 Gbit/s
Wi-Fi 7	802.11be	2.4, 5 and 6 GHz	about 46 Gbit/s

IEEE 802.11 generations (maximum rates are theoretical aggregate PHY rates; real throughput is far lower).

Higher rates depend on signal quality: a station far from the AP uses a more robust but slower modulation, and because all stations share airtime, one slow station reduces the throughput of the others (the "rate anomaly"). Application-visible throughput on Wi-Fi is typically half or less of the PHY rate, because of contention, acknowledgements and protocol overhead.

1.5.2 Worked trace

Backoff after a busy channel

Contention window 0 to 15 (16 values), slot time 9 microseconds. Station A transmits. Station B has a frame ready while the channel is busy.

1. B senses the busy channel and defers; it draws a backoff of 3 slots but does not start counting.
2. A finishes its frame. The AP waits a SIFS and sends the ACK. The channel is busy until the ACK ends.
3. When the channel has been idle for one DIFS, B starts counting: after 3 idle slots (27 microseconds) its counter reaches zero and it transmits.
4. If a third station C had drawn the same value 3, both would transmit simultaneously. Neither receives an ACK; each doubles its window (0 to 31) and draws again.

The probability that two or more stations draw the same smallest value grows quickly with the number of stations, which is why dense Wi-Fi networks lose airtime to collisions. The simulation below draws a backoff from 0 to 15 for each of n stations and counts the rounds in which the smallest value is drawn by at least two stations.

1.5.3 Hands-on

Csma.java JAVA

```java
import java.util.Random;

public class Csma {
    public static void main(String[] args) {
        Random rnd = new Random(42);
        int cw = 16, rounds = 200_000;
        for (int n : new int[]{2, 5, 10, 20}) {
            int collisions = 0;
            for (int r = 0; r < rounds; r++) {
                int min = Integer.MAX_VALUE, countMin = 0;
                for (int s = 0; s < n; s++) {
                    int b = rnd.nextInt(cw);
                    if (b < min) { min = b; countMin = 1; }
                    else if (b == min) countMin++;
                }
                if (countMin > 1) collisions++;
            }
            System.out.printf("stations=%2d  P(collision at first attempt, CW=%d) = %.3f%n",
                    n, cw, collisions / (double) rounds);
        }
    }
}
```

Run the simulation (fixed random seed 42; the values are reproducible) TERMINAL

```
$ java Csma.java
stations= 2  P(collision at first attempt, CW=16) = 0.063
stations= 5  P(collision at first attempt, CW=16) = 0.151
stations=10  P(collision at first attempt, CW=16) = 0.283
stations=20  P(collision at first attempt, CW=16) = 0.503
```

For 2 stations the result matches the analytical value 1/16 = 0.0625. With 20 stations about half of all contention rounds end in a collision at the first attempt, even though each station transmits rarely. This is the reason for contention window doubling, for limiting the number of stations per AP and for using several APs on different channels.

No Wi-Fi interface exists on the build machine, so the following commands are shown as typical output (Linux, `iw` utility; the SSID and addresses are illustrative).

Typical output of iw (typical, not run on the build machine) TERMINAL

```
$ iw dev wlan0 link
Connected to 6c:aa:10:00:00:01 (on wlan0)
        SSID: CampusNet
        freq: 5180
        signal: -58 dBm
        rx bitrate: 433.3 MBit/s VHT-MCS 9 80MHz short GI VHT-NSS 1
        tx bitrate: 390.0 MBit/s VHT-MCS 8 80MHz short GI VHT-NSS 1
```

Reading the output: the BSSID is the AP identifier; `freq: 5180` means 5 GHz channel 36; the signal level in dBm is negative, and the closer to zero the stronger it is (about -50 dBm is excellent, -70 dBm is marginal, -80 dBm and below is unreliable); the bit rates are the currently negotiated PHY rates, which adapt continuously to the signal. A Java program cannot read these values through the standard library; on a laptop they are visible through operating system tools.

1.5.4 Failure modes

- **Co-channel and adjacent-channel interference.** Many APs on the same or overlapping channels share airtime or corrupt each other's frames. Symptom: high latency and low throughput although the signal level is good. Diagnosis: scan for nearby networks and their channels; remedy: use 5 GHz and non-overlapping channels, lower transmit power.
- **Weak signal and rate fallback.** At low signal the PHY rate falls, so each frame occupies the channel longer; all stations on the AP slow down. Remedy: add or move APs, avoid obstacles.
- **Retries masked as latency.** The link layer retransmits a lost frame invisibly, so TCP sees no loss but sees delay jitter. Symptom: video calls stutter while `ping` shows occasional spikes of tens of milliseconds.

- **Roaming failures.** A station stays attached to a distant AP ("sticky client") until the signal is very poor, and a roam between APs on different VLANs changes the subnet and breaks connections. Remedy: roaming standards (802.11r/k/v), one VLAN per SSID across all APs.
- **Connected without connectivity.** Association succeeds, but DHCP fails, the DHCP pool is exhausted or a captive portal blocks traffic until the user signs in; HTTP requests are redirected to a login page. Applications see unexpected redirects or TLS errors.
- **Security misconfiguration.** Open networks and WEP or WPA(1) expose traffic; use WPA2 or WPA3, and never rely on the SSID being hidden.
- **Dense environments.** Hundreds of stations in one room overload the contention mechanism; the collision probability shown above is the mechanism.

1.5.5 Questions to expect

1. What is the difference between SSID and BSSID? The SSID is the network name, which several APs may share. The BSSID is the MAC address of one AP radio and identifies one BSS.
2. Why does Wi-Fi use CSMA/CA and not CSMA/CD? A radio cannot detect collisions while transmitting, and collisions occur at the receiver. Wi-Fi therefore avoids collisions with random backoff and detects loss by a missing link layer ACK.
3. What is the hidden terminal problem and how is it solved? Two stations in range of the AP but not of each other cannot sense one another and collide at the AP. RTS/CTS lets the AP announce the reservation to both.
4. What are the steps for a station to join a WPA2 network? Discovery (beacon or probe), authentication, association, the 4-way handshake (keys), then DHCP for an IP address.
5. Compare 2.4 GHz and 5 GHz. 2.4 GHz has longer range and wall penetration but three non-overlapping channels and heavy interference. 5 GHz has more and wider channels and less interference but a shorter range.
6. Why is Wi-Fi throughput lower than the advertised rate? The advertised value is a theoretical PHY rate. Airtime is shared, every frame needs backoff and an ACK, headers add overhead, and signal quality lowers the rate in use.
7. Why can Wi-Fi make a TCP connection slow without packet loss? The link layer retransmits lost frames, so no loss reaches IP, but each retry adds delay and jitter, which TCP perceives as an increased and variable RTT.
8. What does roaming require? APs of one ESS connected by a layer 2 network so that the station keeps its IP address, plus fast re-association and key handling.

KEY POINTS

Key points

- A BSS is one AP and its stations (identified by the BSSID); an ESS is several APs with one SSID on one layer 2 network that permit roaming.
- Joining involves discovery, authentication, association and (for WPA2 and WPA3) a 4-way handshake before DHCP.
- CSMA/CA avoids collisions with sensing, random backoff with doubling contention windows, and per-frame ACKs; collisions grow quickly with the number of stations.
- Hidden terminals cause collisions that carrier sense cannot see; RTS/CTS reserves the channel.
- 2.4 GHz offers range and three non-overlapping channels; 5 GHz offers capacity and less interference; real throughput is far below the PHY rate.

1.6 Network Tools

A small set of command-line tools answers most questions about a network problem. Each tool observes one layer, so the choice of tool depends on the layer under suspicion. This chapter describes what each tool does and how to read its output. Commands for which the build machine has the tool (`curl`, `nc`, `openssl`) are run for real. The machine has no `ping`, `traceroute`, `ip`, `ss`, `tcpdump` or `dig`, and no access to the public Internet; for those tools the output is labelled **typical** and uses documentation addresses (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24 and the RFC 1918 ranges) and the names `example.com` and `example.org`. Where the kernel exposes the same information in `/proc/net`, a real equivalent is shown, and a small packet-capture program written in Python reproduces the essential function of `tcpdump` on the loopback interface.

1.6.1 Concept and mechanism

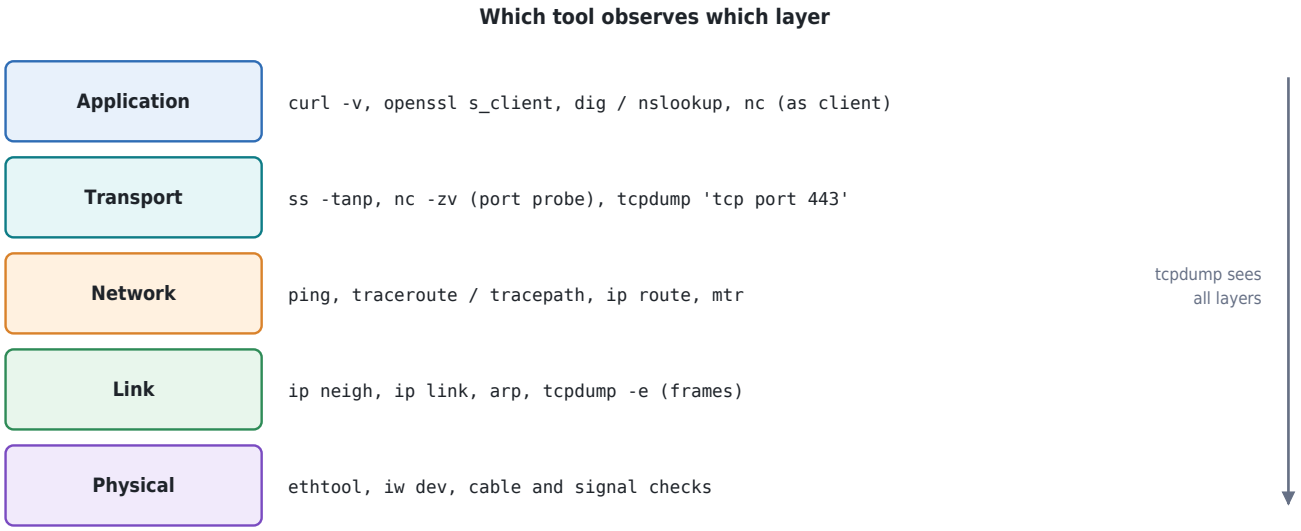


Figure 1.23. Tools are chosen by layer: curl and openssl observe the application, ss and nc the transport, ping and traceroute the network layer, ip neigh the link layer; tcpdump captures at every layer.

Tool	Layer	Question answered	Typical command
ping	3	Is the host reachable, and what are RTT and loss?	`ping -c 4 example.com`
traceroute	3	Which routers lie on the path and where does delay appear?	`traceroute -n example.com`
ip	2-3	Which addresses, routes and neighbours does this host have?	`ip addr`, `ip route`, `ip neigh`
ss	4	Which sockets are listening or connected, in which state?	`ss -tanp`
tcpdump	2-7	What packets actually cross the interface?	`tcpdump -i eth0 -nn 'tcp port 80'`
curl	7	What does the HTTP exchange look like, with timings?	`curl -v http://host/`
nc	4	Is a TCP port open? Send and receive raw bytes	`nc -zv host 443`
openssl s_client	5-7	Which TLS version, cipher and certificate does the server present?	`openssl s_client -connect host:443`
dig	7	What does DNS return, from which server, with which TTL?	`dig example.com A`

The standard tools.

ping

`ping` sends ICMP echo requests and prints one line per echo reply. It measures reachability and the round-trip time at layer 3. It does not test any port or application.

Typical output of ping (typical, not run on the build machine) TERMINAL

```
$ ping -c 4 example.com
PING example.com (203.0.113.80) 56(84) bytes of data.
64 bytes from 203.0.113.80: icmp_seq=1 ttl=52 time=23.4 ms
64 bytes from 203.0.113.80: icmp_seq=2 ttl=52 time=22.9 ms
64 bytes from 203.0.113.80: icmp_seq=3 ttl=52 time=24.4 ms
64 bytes from 203.0.113.80: icmp_seq=4 ttl=52 time=23.7 ms

--- example.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 22.9/23.6/24.4/0.5 ms
```

Reading the output: `56(84) bytes` is the payload and the total IP packet size (56 + 8 bytes ICMP header + 20 bytes IP header). `icmp_seq` shows loss or reordering when numbers are missing. `ttl=52` is the remaining TTL of the reply; if the

remote host started with 64, the reply crossed 12 routers. `time` is the RTT of each probe, and the last line summarises minimum, average, maximum and standard deviation (`mdev`, a measure of jitter). Many firewalls drop ICMP, so no reply does not prove that a host is down.

traceroute

`traceroute` discovers the routers on the path by sending probes with increasing TTL values (1, 2, 3, ...). A router that decrements the TTL to zero discards the packet and returns an ICMP "time exceeded" message that reveals its address; the destination returns a different answer, which ends the trace. Three probes are sent per TTL, so three times are printed.

Typical output of traceroute (typical, not run on the build machine) TERMINAL

```
$ traceroute -n example.com
traceroute to example.com (203.0.113.80), 30 hops max, 60 byte packets
 1  192.168.1.1  1.1 ms  0.9 ms  1.0 ms
 2  198.51.100.1  8.2 ms  8.0 ms  8.4 ms
 3  * * *
 4  203.0.113.9  21.7 ms  21.5 ms  22.0 ms
 5  203.0.113.80  23.5 ms  23.2 ms  23.6 ms
```

A line with `* * *` means that no reply came: the router filters ICMP or rate-limits it; the path continues, as hop 4 shows. A large increase between two hops (hop 2 to 4 here) locates the link where delay is added, although a slow reply from one router alone may reflect only its slow handling of ICMP. Routes may differ for the forward and the return direction, and different probes may take different paths.

ip

`ip` (package iproute2) replaces the older `ifconfig`, `route` and `arp`. `ip addr` lists interfaces with MAC addresses, IP addresses and prefix lengths, `ip link` shows the link state and MTU, `ip route` shows the routing table and `ip neigh` shows the ARP cache.

Typical output of ip addr and ip route (typical, not run on the build machine) TERMINAL

```
$ ip -brief addr
lo      UNKNOWN  127.0.0.1/8
eth0    UP       192.168.1.10/24
$ ip route
default via 192.168.1.1 dev eth0 proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10
```

The first route says that everything not matched by a more specific entry goes to the gateway 192.168.1.1; the second says that the local subnet is directly connected. The build machine reports the same structure in the raw kernel table, as the hands-on section shows.

ss

`ss` (socket statistics) lists sockets with their state, queue sizes and, with `-p`, the owning process. `ss -tln` shows listening TCP sockets, `ss -tan` all TCP sockets, and `ss -s` a summary.

Typical output of ss (typical, not run on the build machine) TERMINAL

```
$ ss -tanp
State      Recv-Q Send-Q  Local Address:Port   Peer Address:Port  Process
LISTEN     0      100     0.0.0.0:8080         0.0.0.0:*          users:(("java",pid=2301,fd=45))
ESTAB      0      0       192.168.1.10:8080    192.168.1.50:51234 users:(("java",pid=2301,fd=61))
TIME-WAIT  0      0       192.168.1.10:8080    192.168.1.51:40022
```

For a listening socket, `Send-Q` is the configured backlog (the length of the accept queue) and `Recv-Q` the number of connections waiting in it; for an established socket they are bytes not yet read by the application and bytes not yet acknowledged by the peer. A growing `Recv-Q` on a connection means that the application is not reading fast enough. The state column is the main diagnostic: many `CLOSE-WAIT` sockets indicate that the application does not close connections, and many `TIME-WAIT` sockets are the normal result of closing many short connections. The transport part explains the states.

tcpdump

`tcpdump` captures packets on an interface and prints one line per packet. It needs administrator rights. A **capture filter** in Berkeley Packet Filter syntax limits the output (`tcp port 80`, `host 192.168.1.50`, `icmp`). Useful options are `-i` (interface), `-nn` (no name or port resolution), `-e` (show link-layer headers), `-w file.pcap` (write a file for Wireshark) and `-X` (hex and ASCII payload).

Typical output of tcpdump for the start of a connection (typical, not run on the build machine) TERMINAL

```
$ tcpdump -i eth0 -nn 'tcp port 80'
12:00:00.000000 IP 192.168.1.10.49152 > 203.0.113.80.80: Flags [S], seq 1000, win 64240
12:00:00.023000 IP 203.0.113.80.80 > 192.168.1.10.49152: Flags [S.], seq 7000, ack 1001, win 65535
12:00:00.023100 IP 192.168.1.10.49152 > 203.0.113.80.80: Flags [.], ack 7001, win 502
```

The flag `S` is SYN, `.` is ACK, `P` is PSH, `F` is FIN and `R` is RST. Each line shows the timestamp, the source address and port, the destination address and port, the flags, the sequence and acknowledgement numbers, the window and the payload length.

dig

`dig` queries DNS servers and shows the complete response. The answer section holds the records, the number after the name is the remaining time to live in seconds, and the footer shows which server answered and how long the query took.

Typical output of dig (typical, not run on the build machine) TERMINAL

```
$ dig example.com A
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41873
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; ANSWER SECTION:
example.com.    3600    IN    A    203.0.113.80

;; Query time: 18 msec
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)
```

The status `NOERROR` with an answer means success; `NXDOMAIN` means that the name does not exist; `SERVFAIL` means that the resolver failed. The flag `ra` shows that the server offers recursion. The DNS chapter in the application layer part describes the protocol.

1.6.2 Worked trace

Reading a packet capture

The program below is a minimal packet capture for the loopback interface. It opens a raw `AF_PACKET` socket, which is the facility that `tcpdump` uses, decodes the Ethernet, IPv4 and TCP headers and prints one line per packet with sequence numbers relative to the start of each direction (the same convention as `tcpdump` with default settings). Administrator rights are required. It needs the Python standard library only.

sniff.py PYTHON

```python
import socket
import struct
import sys
import time

PORT = int(sys.argv[1])
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(0x0003))
s.bind(("lo", 0))
s.settimeout(2)
base = {}
t0 = None
while True:
    try:
        frame, addr = s.recvfrom(65535)
    except socket.timeout:
        break
    # skip outgoing duplicates (packet type 4), non-IPv4 and non-TCP frames
    if addr[2] == 4 or frame[12:14] != b"\x08\x00" or frame[23] != 6:
        continue
    ihl = (frame[14] & 0x0F) * 4
    sp, dp, seq, ack, off, flags, win = struct.unpack("!HHIIBBH", frame[14 + ihl:30 + ihl])
```

Preeram

sniff.py (continued) PYTHON

```
    if PORT not in (sp, dp):
        continue
    plen = len(frame) - 14 - ihl - (off >> 4) * 4
    t0 = t0 or time.time()
    base.setdefault(sp, seq)
    bits = ((2, "S"), (1, "F"), (4, "R"), (8, "P"), (0x10, "."))
    names = "".join(n for b, n in bits if flags & b)
    rel_ack = ack - base[dp] if dp in base and flags & 0x10 else 0
    print("%6.3f ms  %5d > %-5d [%-2s] seq=%-4d ack=%-4d win=%-5d len=%d" % (
        (time.time() - t0) * 1000, sp, dp, names, seq - base[sp], rel_ack, win, plen))
```

While the sniffer is running, one request is made to the local server with `curl`; the output is the real capture of a complete HTTP exchange.

Capture of one HTTP request on the loopback interface (ports and timings vary between runs) TERMINAL

```
$ python3 sniff.py 8080 &
$ curl -s -o /dev/null http://127.0.0.1:8080/hello
 0.031 ms  45390 > 8080  [S ] seq=0    ack=0    win=65495 len=0
 0.137 ms   8080 > 45390 [S.] seq=0    ack=1    win=65483 len=0
 0.153 ms  45390 > 8080  [. ] seq=1    ack=1    win=64    len=0
 0.268 ms  45390 > 8080  [P.] seq=1    ack=1    win=64    len=82
 0.314 ms   8080 > 45390 [. ] seq=1    ack=83   win=64    len=0
 2.337 ms   8080 > 45390 [P.] seq=1    ack=83   win=64    len=102
 2.371 ms  45390 > 8080  [. ] seq=83   ack=103  win=64    len=0
 2.381 ms   8080 > 45390 [P.] seq=103  ack=83   win=64    len=15
 2.391 ms  45390 > 8080  [. ] seq=83   ack=118  win=64    len=0
 2.543 ms  45390 > 8080  [F.] seq=83   ack=118  win=64    len=0
 2.727 ms   8080 > 45390 [F.] seq=118  ack=84   win=64    len=0
 2.741 ms  45390 > 8080  [. ] seq=84   ack=119  win=64    len=0
```

Each line is interpreted as follows (the client port is 45390 in this run).

1. Lines 1-3, **three-way handshake.** The client sends a SYN with initial sequence number 0 (relative), the server answers SYN-ACK acknowledging 1, and the client sends the final ACK. The connection is established after 0.153 ms.
2. Line 4, **request.** The client sends 82 bytes with PSH and ACK. The 82 bytes are the HTTP request: `GET /hello HTTP/1.1` (21 bytes with CR LF), `Host` (22), `User-Agent` (24), `Accept` (13) and the empty line (2).
3. Line 5, **acknowledgement of the request.** The server acknowledges 83 (1 + 82): the acknowledgement number is the next byte expected.
4. Lines 6-9, **response.** The server sends 102 bytes (the status line and headers) and later 15 bytes (the body `Hello, network` and a newline), each acknowledged by the client: ack 103, then ack 118 (1 + 102 + 15). The Java `HttpServer` sends headers and body in two separate segments, which is visible here. The 2.0 ms gap before the response is the server processing the first request, including class initialisation.
5. Lines 10-12, **teardown.** The client sends FIN (sequence 83), the server acknowledges (ack 84) and sends its own FIN (sequence 118), and the client acknowledges that (ack 119). Each FIN consumes one sequence number.

The `win` values are the raw 16-bit window fields; after the window-scale option the effective window is larger (the SYN segments carry the unscaled value 65495 because the scale is not applied in a SYN). The capture shows 12 packets for one tiny request, and the data segments (82, 102 and 15 bytes) are a minority: protocol overhead dominates small exchanges.

1.6.3 Hands-on

curl

`curl -v` prints the connection, the request and the response headers. `-I` sends a HEAD request, `-w` prints timing variables, `--max-time` limits the total time, `-k` accepts an untrusted certificate (testing only), and `--resolve host:port:address` forces an address without changing DNS.

Header-only request with curl -I TERMINAL

```
$ curl -sI http://127.0.0.1:8080/hello
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 16:48:35 GMT
Content-type: text/plain
```

Exit status and error text for a connection that is refused — TERMINAL

```
$ curl -s --max-time 3 http://127.0.0.1:9999/
$ echo "exit status $?"
exit status 7
```

Exit status 7 is "failed to connect"; status 28 would be a timeout. The difference between an immediate refusal (nobody listens on the port, or a firewall actively rejects) and a timeout (packets are dropped silently) is the most useful single fact in network diagnosis.

nc (netcat)

`nc -zv host port` tests whether a TCP port accepts connections without sending data. `nc -l` opens a listening socket, and `nc host port` is a raw TCP client.

Probing an open and a closed port — TERMINAL

```
$ nc -zv 127.0.0.1 8080
Connection to 127.0.0.1 8080 port [tcp/http-alt] succeeded!
$ nc -zv 127.0.0.1 9999
nc: connect to 127.0.0.1 port 9999 (tcp) failed: Connection refused
```

openssl s_client

`openssl s_client` performs a TLS handshake and prints the negotiated parameters and the certificate. The following run connects to a local `openssl s_server` that uses a self-signed certificate created with `openssl req -x509 -newkey rsa:2048 -nodes -keyout k.pem -out c.pem -subj "/CN=localhost"`.

TLS handshake summary with s_client -brief — TERMINAL

```
$ echo | openssl s_client -brief -connect 127.0.0.1:8443
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN = localhost
Hash used: SHA256
Signature type: RSA-PSS
Verification error: self-signed certificate
Server Temp Key: X25519, 253 bits
DONE
```

The protocol version and the cipher suite are those negotiated; `Verification error: self-signed certificate` means that the chain does not lead to a trusted root, which is the cause of the `PKIX path building failed` exception in Java. The TLS part of this guide interprets all fields.

Sockets and routes from /proc

Without `ss` and `ip`, the kernel tables can be read directly. The program below decodes `/proc/net/tcp`, in which addresses are little-endian hexadecimal, and prints the rows that involve one port.

ss_lite.py — PYTHON

```python
import socket
import struct
import sys

STATES = {"01": "ESTAB", "02": "SYN-SENT", "06": "TIME-WAIT", "08": "CLOSE-WAIT", "0A": "LISTEN"}
port = int(sys.argv[1])


def addr(hexaddr):
    ip, p = hexaddr.split(":")
    dotted = socket.inet_ntoa(struct.pack("<I", int(ip, 16)))    # the kernel prints little-endian
    return dotted, int(p, 16)


print("%-10s %-22s %-22s" % ("State", "Local", "Peer"))
with open("/proc/net/tcp") as f:
    next(f)
    for line in f:
        parts = line.split()
```

ss_lite.py (continued) PYTHON

```
        local, peer = addr(parts[1]), addr(parts[2])
        if port in (local[1], peer[1]):
            print("%-10s %-22s %-22s" % (STATES.get(parts[3], parts[3]),
                                         "%s:%d" % local, "%s:%d" % peer))
```

A listener and one established connection (an nc server and client on port 9102) TERMINAL

```
$ nc -l 127.0.0.1 9102 &
$ nc 127.0.0.1 9102 &
$ python3 ss_lite.py 9102
State      Local                Peer
LISTEN     127.0.0.1:9102       0.0.0.0:0
ESTAB      127.0.0.1:43088      127.0.0.1:9102
ESTAB      127.0.0.1:9102       127.0.0.1:43088
```

The listener, the client side and the server side of the connection appear as three sockets, which is what `ss -tan` would show. The routing table is available in `/proc/net/route`; the columns below are the interface, the destination, the gateway and the mask, each in little-endian hexadecimal.

The routing table of the build machine (real output, selected columns) TERMINAL

```
$ awk 'NR>1{print $1, $2, $3, $8}' /proc/net/route
eth0 00000000 010200C0 00000000
eth0 000200C0 00000000 00FFFFFF
```

Decoded, `010200C0` is the bytes C0 00 02 01 reversed, that is 192.0.2.1. The first row is the default route (destination 0.0.0.0, mask 0.0.0.0) through gateway 192.0.2.1; the second is the directly connected network 192.0.2.0 with mask 255.255.255.0 (`00FFFFFF` reversed is FF FF FF 00), the same two lines that `ip route` prints.

1.6.4 Failure modes

- **Concluding from ping alone.** ICMP may be blocked while TCP works, or ICMP may pass while the application port is filtered. Test the actual protocol and port (`nc -zv`, `curl -v`).
- **Misreading traceroute.** Stars and high delay at one intermediate hop do not prove a fault if the following hops are fast: routers deprioritise ICMP generation. Load balancing can show different routers in one line. The path back may differ from the path out.
- **Capturing on the wrong interface or host.** Traffic that never reaches the interface of the capturing host is not visible; a capture on the client shows what the client sent, and a capture on the server shows what arrived. Comparing the two locates the loss.
- **Offload artefacts.** Segmentation and checksum offload make captures taken on the sending host show packets larger than the MTU or with wrong checksums. These are not faults.
- **Encrypted payload.** For HTTPS, `tcpdump` shows only the handshake and sizes; use the application's own logging or `curl -v` for content.
- **Blocking tools and DNS.** Without `-n`, `tcpdump`, `traceroute` and `ss` resolve names, which is slow and may itself generate DNS traffic that disturbs the analysis.
- **Permissions and containers.** Capturing needs root or the `CAP_NET_RAW` capability, and a container sees only its own network namespace.

1.6.5 Questions to expect

1. How does traceroute work? It sends probes with TTL 1, 2, 3 and so on. Each router that reduces the TTL to zero returns an ICMP time-exceeded message that reveals its address; the destination answers differently and ends the trace.
2. What is the difference between "connection refused" and a timeout? Refused: a host answered with a TCP RST (or ICMP unreachable), so the host is up but no process listens or a rule rejects the connection. Timeout: no answer at all; packets are dropped by a firewall, the host is down or the route is broken.
3. A service is slow. Which tools are used first, and why? `curl -w` for the timing breakdown (connect, first byte, total), `ping` and `traceroute` for network delay and loss, `ss` for socket states and queues, and `tcpdump` when the evidence must be at packet level.
4. What does `Recv-Q` mean in `ss`? For an established connection, bytes received but not read by the application; for a listening socket, connections waiting in the accept queue.

sreeram

5. How can an open port be tested without a browser? `nc -zv host port`, `curl -v telnet://host:port`, or in Java `new Socket().connect(address, timeoutMillis)`.
6. What does `ttl=52` in a ping reply tell? The remaining TTL of the reply packet; subtracting it from the likely initial value (64, 128 or 255) estimates the number of hops on the return path.
7. How is the certificate of a server inspected from the command line? `openssl s_client -connect host:443 -servername host`, optionally piped into `openssl x509 -noout -text`, shows the chain, validity, subject and protocol.
8. Why can `tcpdump` show a checksum error for a correct packet? Checksum offload: the network card computes the checksum after the capture point, so the captured copy still holds a placeholder.

INTERVIEW ANGLE

Follow-up chain: "An API call from a service to another fails intermittently. How is the diagnosis organised?" First reproduce with `curl -v` or a small client and note whether the failure is a refusal, a timeout or a reset, because each points to a different layer. Check name resolution (`dig`), reachability and loss (`ping`, `traceroute`), then the port and TLS (`nc -zv`, `openssl s_client`). On the server, check `ss` for listen-queue overflow, many `CLOSE-WAIT` sockets or `TIME-WAIT` build-up. If the evidence is still missing, capture with `tcpdump` on both ends and compare.

KEY POINTS

Key points

- Select the tool by layer: curl and openssl (application), ss and nc (transport), ping and traceroute (network), ip neigh (link), tcpdump (all).
- ping measures RTT and loss with ICMP; traceroute uses increasing TTL; neither proves that a TCP service works.
- Refused, timed out and reset are three different symptoms with different causes.
- A capture shows the handshake, the byte counts of each segment and the sequence and acknowledgement numbers; a small program with an `AF_PACKET` socket reproduces the essence of `tcpdump`.
- The kernel tables in `/proc/net` hold the same information as `ss`, `ip route` and `arp`.

sreeram

PART 2

The Network Layer

Addressing, packet formats, diagnostics, routing, address translation and cloud networking.

In this part

- **2.1** IPv4 Addressing and the IPv4 Header
- **2.2** Classless Addressing and Subnetting
- **2.3** IPv6
- **2.4** ICMP and Network Diagnostics
- **2.5** Routing
- **2.6** NAT and DHCP
- **2.7** Cloud Networking Basics

2.1 IPv4 Addressing and the IPv4 Header

The link layer delivers frames between devices attached to the same network segment. A request from a Spring Boot client to a server on another continent crosses dozens of such segments, owned by different organisations, built from different link technologies and using different frame formats. The network layer solves this problem. It defines a uniform packet format and a uniform address space that are independent of the underlying links, and it moves each packet hop by hop from the source host to the destination host. In the Internet this layer is implemented by the Internet Protocol (IP). Version 4 (IPv4, RFC 791) still carries the majority of traffic, and every other topic in this Part builds on its header and its addressing model.

IP is deliberately minimal. It is **connectionless**: each datagram is routed independently, and no state about a conversation is kept in the routers. It is **best effort**: a datagram can be lost, duplicated, delayed or delivered out of order, and IP does not report these events to the sender except through the optional ICMP error messages described in a later chapter. Reliability, ordering and congestion control are the responsibility of the transport layer (TCP) or of the application. This division of labour keeps routers simple and fast, which is the main reason the design scaled.

2.1.1 Concept and mechanism

The IPv4 address

An IPv4 address is a 32-bit number. It is written in dotted-decimal notation as four octets, each from 0 to 255, for example `192.168.1.10`, which is the binary value `11000000.10101000.00000001.00001010`. An address identifies a network **interface**, not a host. A router has one address per interface, and a multihomed server may have several.

Every address has two logical parts: a **network prefix** that identifies the network (subnet) and a **host part** that identifies the interface within it. Routers forward on the network prefix only. The boundary between the two parts is given by a subnet mask or a prefix length, described in the next chapter. Early IPv4 fixed the boundary by the value of the first bits (classes A, B and C with prefix lengths 8, 16 and 24). This classful scheme wasted address space and was replaced in 1993 by Classless Inter-Domain Routing (CIDR, RFC 4632). Class names survive only in vocabulary.

Class	First octet range	Default prefix and purpose
A	1 - 126	/8, 16.7 million addresses per network
B	128 - 191	/16, 65,536 addresses per network
C	192 - 223	/24, 256 addresses per network
D	224 - 239	multicast (still in use)
E	240 - 255	reserved (still unused)

Historical classful addressing, shown for terminology only.

The header

Every IP datagram begins with a header of 20 to 60 bytes. The fixed part is 20 bytes. The diagram shows the layout in 32-bit rows, which is also the way the specification and packet analysers present it.

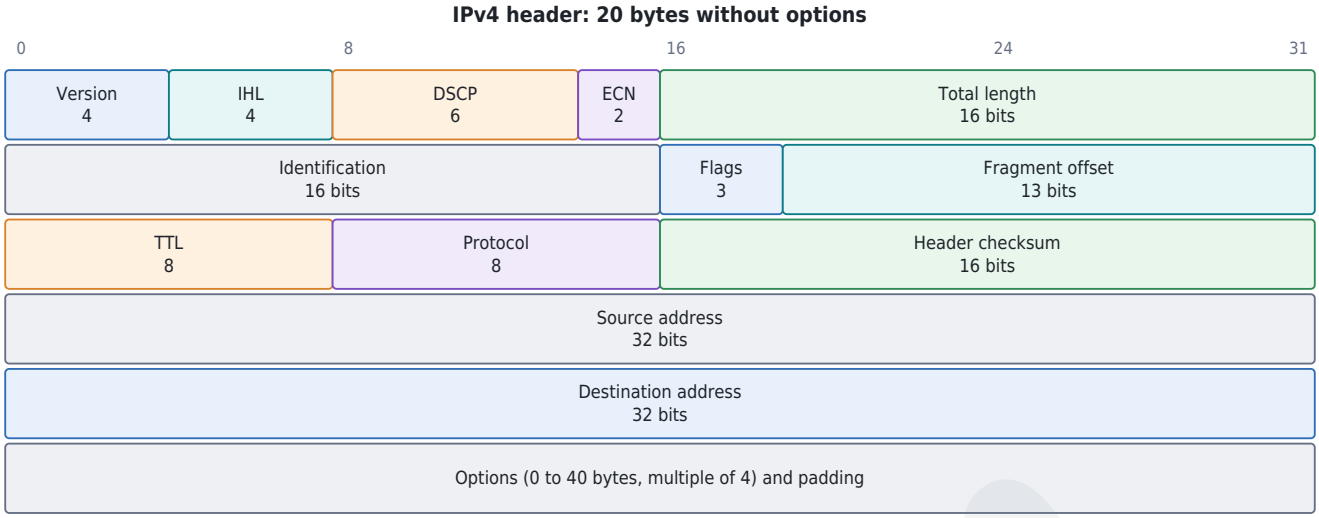


Figure 2.1. The IPv4 header consists of five fixed 32-bit rows; a sixth row of options exists only when the header length field (IHL) is greater than 5, which makes the header grow in steps of 4 bytes up to 60 bytes.

Field	Bits	Purpose
`Version`	4	Always 4 for IPv4 (6 for IPv6). Receivers discard other values in an IPv4 parser.
`IHL`	4	Header length in 32-bit words. Minimum 5 (20 bytes), maximum 15 (60 bytes).
`DSCP`	6	Differentiated services code point: requested forwarding class (RFC 2474), for example voice or best effort.
`ECN`	2	Explicit congestion notification (RFC 3168): routers mark packets instead of dropping them.
`Total length`	16	Header plus payload in bytes. Maximum 65,535.
`Identification`	16	Value chosen by the sender; identical in all fragments of one original datagram.
`Flags`	3	Bit 0 reserved (0); bit 1 DF (do not fragment); bit 2 MF (more fragments follow).
`Fragment offset`	13	Position of this fragment's data in the original payload, in 8-byte units.
`TTL`	8	Hop limit. Each router decrements it; at zero the datagram is discarded.
`Protocol`	8	Identifies the payload type, for example 6 TCP, 17 UDP, 1 ICMP.
`Header checksum`	16	Checksum over the header only; recomputed at every router.
`Source address`	32	Address of the sending interface.
`Destination address`	32	Address of the target interface.
`Options`	0-320	Rarely used (record route, timestamp); many routers process them slowly or drop them.

IPv4 header fields.

Time to live and the protocol field

The **TTL** field prevents a datagram from circulating forever when routing tables contain a loop. A host sets the initial value (commonly 64 on Linux and macOS, 128 on Windows; the exact default is configurable) and each router decrements it by one. When a router decrements a TTL to zero, it discards the datagram and sends an ICMP Time Exceeded message to the source. The traceroute tool exploits exactly this behaviour (see the chapter on ICMP). Despite its name, TTL is a hop count, not a time.

The **protocol** field tells the receiving host which module handles the payload. The receiver demultiplexes on this value before the transport layer applies its own port-based demultiplexing.

Number	Protocol	Remark
1	ICMP	Error reporting and diagnostics

Number	Protocol	Remark
2	IGMP	Multicast group membership
6	TCP	Reliable byte stream
17	UDP	Datagrams; also carries DNS, QUIC, DHCP
41	IPv6	IPv6 packet encapsulated in IPv4 (6in4 tunnels)
47	GRE	Generic routing encapsulation
50	ESP	IPsec encapsulating security payload
89	OSPF	Link-state routing protocol (runs directly over IP)

Frequently seen protocol numbers (IANA protocol numbers registry).

Fragmentation

Each link has a **maximum transmission unit** (MTU), the largest IP datagram it can carry. Ethernet uses 1500 bytes. When a router must forward a datagram larger than the MTU of the outgoing link, IPv4 permits the router to **fragment** it, unless the DF flag is set. Each fragment is an independent datagram with its own IP header. The rules are:

- Every fragment copies the identification, source, destination and protocol fields of the original.
- The fragment offset gives the position of the fragment's data within the original payload, divided by 8. Therefore every fragment except the last must carry a multiple of 8 bytes of data.
- All fragments except the last have MF = 1. The last has MF = 0 and a non-zero offset.
- Routers never reassemble. Only the destination host reassembles, using the tuple (source, destination, protocol, identification) and a reassembly timer; if a fragment is missing when the timer expires, the whole datagram is discarded.

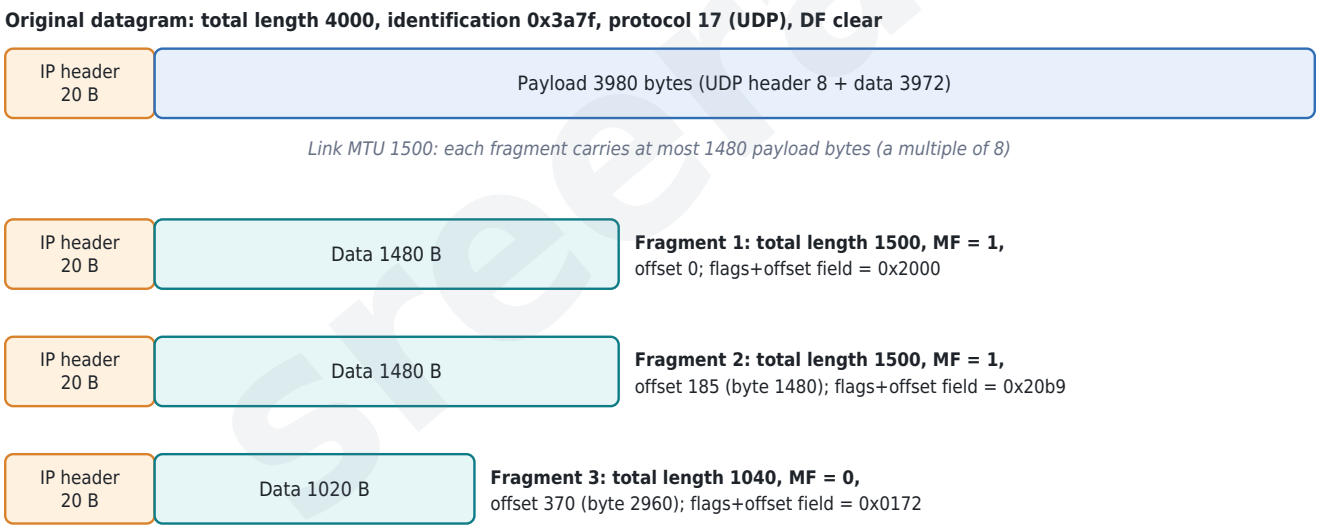


Figure 2.2. A 4000-byte datagram crossing a 1500-byte MTU is split into three fragments whose offsets (0, 185, 370) are counted in 8-byte units; the last fragment is shorter and is the only one without the MF flag.

Fragmentation is costly. A loss of any single fragment destroys the entire datagram, so the delivery probability falls from (1 - p) to (1 - p)^n for n fragments with independent loss probability p. Fragments also defeat stateless packet filters because only the first fragment contains the transport header with the port numbers. For these reasons modern stacks avoid fragmentation: TCP chooses a segment size that fits the path MTU, and Path MTU Discovery sets DF on every packet (see the chapter on ICMP). IPv6 removed router fragmentation altogether.

The header checksum

The header checksum protects the header against corruption in transit and in router memory. It is the 16-bit one's complement of the one's complement sum of all 16-bit words of the header, computed with the checksum field set to zero (RFC 791 and the computation guide in RFC 1071). A receiver recomputes the sum over the complete header including the checksum field. A valid header yields 0xFFFF in the sum, that is, 0 after the final complement. The payload is not covered: TCP and UDP carry their own checksums.

Because each router decrements the TTL, each router must also update the checksum. Implementations do this incrementally (RFC 1624) rather than summing the whole header again.

2.1.2 Worked trace

A host with address 192.168.1.10 sends a TCP segment of 40 bytes to 203.0.113.25 (an address from the documentation range). The IP datagram has a 20-byte header and a 40-byte TCP segment, so the total length is 60. The sender sets DF, uses TTL 64 and identification 0x1c46.

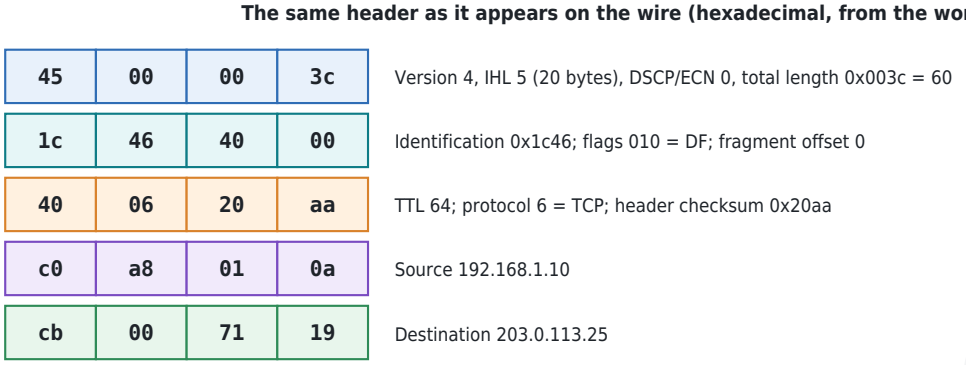


Figure 2.3. The 20 header bytes of the worked trace, each 32-bit row decoded; the checksum 0x20aa is derived in the table that follows.

Checksum computation

The ten 16-bit words of the header, with the checksum field treated as 0x0000, are added with end-around carry.

Word	Value	Running sum	Remark
1	0x4500	0x4500	version 4, IHL 5, DSCP/ECN 0
2	0x003c	0x453c	total length 60
3	0x1c46	0x6182	identification
4	0x4000	0xa182	flags DF, offset 0
5	0x4006	0xe188	TTL 64, protocol 6
6	0x0000	0xe188	checksum field (zero while computing)
7	0xc0a8	0x1a230	source 192.168
8	0x010a	0x1a33a	source 1.10
9	0xcb00	0x26e3a	destination 203.0
10	0x7119	0x2df53	destination 113.25

Step-by-step computation of the header checksum.

The sum 0x2df53 has a carry out of the low 16 bits. Folding the carry gives 0xdf53 + 0x2 = 0xdf55. The one's complement of 0xdf55 is 0x20aa, which is the value placed in the checksum field. The receiver sums all ten words including 0x20aa: 0xdf55 + 0x20aa = 0xffff, and the complement is 0, so the header is accepted.

Effect of one router hop

The first router decrements TTL from 64 to 63. The word 0x4006 becomes 0x3f06, which is 0x0100 smaller. The old checksum no longer verifies, so the router adds 0x0100 to the checksum with end-around carry and writes 0x21aa. This value equals the checksum computed from scratch over the new header, as the program in the next section confirms.

Fragmentation of a 4000-byte datagram

An application sends a UDP datagram with 3972 bytes of data. With the 8-byte UDP header the IP payload is 3980 bytes and the total length is 4000. The outgoing link has an MTU of 1500, so the maximum data per fragment is 1500 - 20 = 1480 bytes, which is divisible by 8. The following table lists the three fragments (identification 0x3a7f, protocol 17).

Fragment	Total length	Data bytes	MF	Offset field	Flags and offset word, header checksum
1	1500	1480	1	0	0x2000, checksum 0x1cc6

Fragment	Total length	Data bytes	MF	Offset field	Flags and offset word, header checksum
2	1500	1480	1	185	0x20b9, checksum 0x1c0d
3	1040	1020	0	370	0x0172, checksum 0x3d20

Fragments of the 4000-byte datagram.

The data bytes add up to 1480 + 1480 + 1020 = 3980. The offsets are 0 / 8 = 0, 1480 / 8 = 185 and 2960 / 8 = 370. The word 0x20b9 combines MF (0x2000) with offset 185 (0x00b9). The destination collects the three fragments, orders them by offset, concatenates their data and passes 3980 bytes to UDP once the fragment with MF = 0 and all bytes before it have arrived.

2.1.3 Hands-on

The header checksum is simple enough to verify in a short Java program. The program builds the header of the worked trace in a `ByteBuffer`, computes the checksum, verifies it, and then repeats the computation after the TTL change of the first hop.

Ipv4Checksum.java JAVA

```java
import java.nio.ByteBuffer;

public class Ipv4Checksum {
    static byte[] header(int total, int id, int flagsFrag, int ttl, int proto, int src, int dst) {
        ByteBuffer b = ByteBuffer.allocate(20);
        b.put((byte) 0x45).put((byte) 0).putShort((short) total).putShort((short) id);
        b.putShort((short) flagsFrag).put((byte) ttl).put((byte) proto);
        b.putShort((short) 0).putInt(src).putInt(dst);
        return b.array();
    }

    static int checksum(byte[] h) {
        long sum = 0;
        for (int i = 0; i < h.length; i += 2) {
            sum += ((h[i] & 0xFF) << 8) | (h[i + 1] & 0xFF);
        }
        while ((sum >> 16) != 0) {
            sum = (sum & 0xFFFF) + (sum >> 16);
        }
        return (int) (~sum & 0xFFFF);
    }

    static int ip(int a, int b, int c, int d) {
        return (a << 24) | (b << 16) | (c << 8) | d;
    }

    public static void main(String[] args) {
        byte[] h = header(60, 0x1c46, 0x4000, 64, 6, ip(192, 168, 1, 10), ip(203, 0, 113, 25));
        int cs = checksum(h);
        System.out.printf("checksum field = 0x%04x%n", cs);
        h[10] = (byte) (cs >> 8);
        h[11] = (byte) cs;
        StringBuilder sb = new StringBuilder();
        for (byte x : h) {
            sb.append(String.format("%02x ", x));
        }
        System.out.println(sb.toString().trim());
        System.out.printf("verification over full header = 0x%04x%n", checksum(h));
        h[8] = 63; // router decrements TTL; checksum is now stale
        System.out.printf("after TTL change, verification = 0x%04x (non-zero: corrupt)%n",
                checksum(h));
        int cs2 = checksum(header(60, 0x1c46, 0x4000, 63, 6, ip(192, 168, 1, 10),
                ip(203, 0, 113, 25)));
        System.out.printf("recomputed checksum with TTL 63 = 0x%04x%n", cs2);
    }
}
```

Output (deterministic) TERMINAL

```
$ java Ipv4Checksum.java
checksum field = 0x20aa
45 00 00 3c 1c 46 40 00 40 06 20 aa c0 a8 01 0a cb 00 71 19
verification over full header = 0x0000
after TTL change, verification = 0x0100 (non-zero: corrupt)
recomputed checksum with TTL 63 = 0x21aa
```

The output reproduces the values of the trace. After the hop the unchanged checksum fails verification with the residue 0x0100, which is exactly the amount by which the TTL change altered the sum, and the recomputed value is 0x21aa.

Reading the address a server actually sees

A server's view of the client address depends on the proxies in between. The next program starts a `com.sun.net.httpserver.HttpServer` on the loopback interface and reports both the address of the TCP peer and the `X-Forwarded-For` header, which load balancers use to carry the original client address.

WhoAmI.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;

public class WhoAmI {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
        server.createContext("/who", ex -> {
            String peer = ex.getRemoteAddress().getAddress().getHostAddress();
            String xff = ex.getRequestHeaders().getFirst("X-Forwarded-For");
            String body = "peer=" + peer + " X-Forwarded-For=" + xff + "\n";
            byte[] out = body.getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, out.length);
            ex.getResponseBody().write(out);
            ex.close();
        });
        server.start();
        int port = server.getAddress().getPort();
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:" + port + "/who"))
                .header("X-Forwarded-For", "203.0.113.50, 10.0.0.7").build();
        System.out.print(client.send(req, HttpResponse.BodyHandlers.ofString()).body());
        server.stop(0);
    }
}
```

Output (deterministic apart from the port, which is not printed) TERMINAL

```
$ java WhoAmI.java
peer=127.0.0.1 X-Forwarded-For=203.0.113.50, 10.0.0.7
```

The peer address is the immediate TCP neighbour. In a deployment behind a load balancer it is the balancer's address, and the original client address is available only in the forwarding header. Spring Boot exposes the original address through `request.getRemoteAddr()` only when `server.forward-headers-strategy` is set to `native` or `framework`, and the header must be trusted only when it is set by infrastructure under the operator's control, because any client can send the header itself.

2.1.4 Failure modes

Symptom	Cause	Diagnosis
Small requests succeed, large responses hang	Path MTU smaller than the sender assumes and ICMP fragmentation-needed messages are filtered	Test with DF-set probes of increasing size; lower the MSS or MTU on the tunnel interface
Intermittent loss of large UDP datagrams	Fragmentation: loss of one fragment loses the datagram; firewalls drop non-first fragments	Keep UDP payloads below the path MTU (about 1200 bytes is a safe application limit)
Traffic loops, TTL exceeded messages	Inconsistent routes between two routers	Traceroute shows the same two hops repeating
Connection works from one network only	Duplicate or overlapping address ranges	Compare routing tables; avoid overlapping private ranges when connecting networks
Logs show only the balancer address	Original client address is in `X-Forwarded-For` or in the PROXY protocol	Configure forwarded-header handling in Spring Boot or the web server

Common IPv4-level failures.

WATCH OUT

The `X-Forwarded-For` header is supplied by the client unless a trusted proxy overwrites it. An application that uses its first value for access control or rate limiting can be bypassed by sending a forged header. Use the rightmost value appended by the trusted proxy, or the framework support that restricts accepted proxies.

2.1.5 Questions to expect

1. What is the difference between connectionless and connection-oriented delivery, and which does IP provide?

IP is connectionless: each datagram is forwarded independently with no setup and no per-flow state in routers. Connection-oriented delivery (TCP) maintains state at the endpoints for sequencing, acknowledgement and flow control. IP provides only best-effort delivery.

2. Why does the IPv4 header contain a checksum that covers only the header?

Routers must verify and update the header at every hop, so a cheap header-only checksum keeps forwarding fast. Payload integrity is the responsibility of TCP and UDP, which carry end-to-end checksums. IPv6 dropped the header checksum entirely, because link layers and transport protocols already verify integrity.

3. What does a router do to the header of a forwarded packet?

It decrements the TTL, discards the packet and signals ICMP Time Exceeded if the TTL reaches zero, updates the header checksum and, if necessary, fragments the packet. The link-layer frame is rebuilt completely with new source and destination MAC addresses; the IP addresses stay unchanged unless NAT is applied.

4. Why must the fragment offset be a multiple of 8 bytes?

The offset field has 13 bits but the datagram can be 65,535 bytes long, so the offset counts units of 8 bytes. Every fragment except the last must therefore carry a multiple of 8 data bytes.

5. A 4000-byte datagram crosses an MTU of 1500. How many fragments result and what are their offsets?

The data per fragment is 1480 bytes (1500 minus the 20-byte header). The payload of 3980 bytes needs three fragments carrying 1480, 1480 and 1020 bytes, with offsets 0, 185 and 370 and MF values 1, 1 and 0.

6. Who reassembles fragments and why not the routers?

The destination host. Fragments can take different paths, so a router cannot be sure that it sees all of them, and reassembly would require buffering and state in every router.

7. What is the purpose of the TTL field, and is it measured in seconds?

It bounds the lifetime of a datagram so that routing loops do not cause permanent circulation. It is a hop count: each router subtracts one, regardless of the time spent.

8. How does a server behind a load balancer learn the client address?

From the `X-Forwarded-For` or `Forwarded` header set by an HTTP load balancer, or from the PROXY protocol for TCP balancers. The socket peer address is the balancer. The header must be trusted only from known proxies.

9. What is the maximum size of an IPv4 datagram and what limits it?

65,535 bytes, because the total length field has 16 bits. In practice datagrams stay near the link MTU, typically 1500 bytes.

INTERVIEW ANGLE

Follow-up chain: why does TCP rarely cause IP fragmentation? TCP advertises a maximum segment size (MSS) during the handshake, derived from the interface MTU (1460 for an MTU of 1500 with 20-byte IP and TCP headers). The sender never builds a segment larger than the smaller of the two MSS values and usually sets DF, so oversized packets are detected through ICMP instead of being fragmented. UDP applications have no such mechanism and must size their datagrams themselves.

KEY POINTS

- IP is connectionless and best effort; reliability belongs to TCP or the application.
- The 20-byte header carries addresses, TTL, protocol, fragmentation fields and a header-only checksum that every router recomputes.
- Fragment offsets are counted in 8-byte units; only the destination reassembles, and one lost fragment loses the datagram.
- TTL is a hop count that stops routing loops and powers traceroute.
- Behind proxies the socket peer address is the proxy; the client address travels in forwarding headers that must be trusted selectively.

2.2 Classless Addressing and Subnetting

Allocating address space in blocks of 256, 65,536 or 16.7 million addresses forced organisations to request far more space than they needed, and it filled the global routing table with thousands of small entries. Classless Inter-Domain Routing (CIDR, RFC 4632) removed the fixed boundaries. A network is described by an address and a **prefix length** of any value from 0 to 32, and routers aggregate adjacent blocks into a single advertisement. Subnetting, which divides one block into smaller blocks, and summarisation, which combines blocks, are the two operations that every network engineer and every cloud architect performs. Both reduce to bit arithmetic.

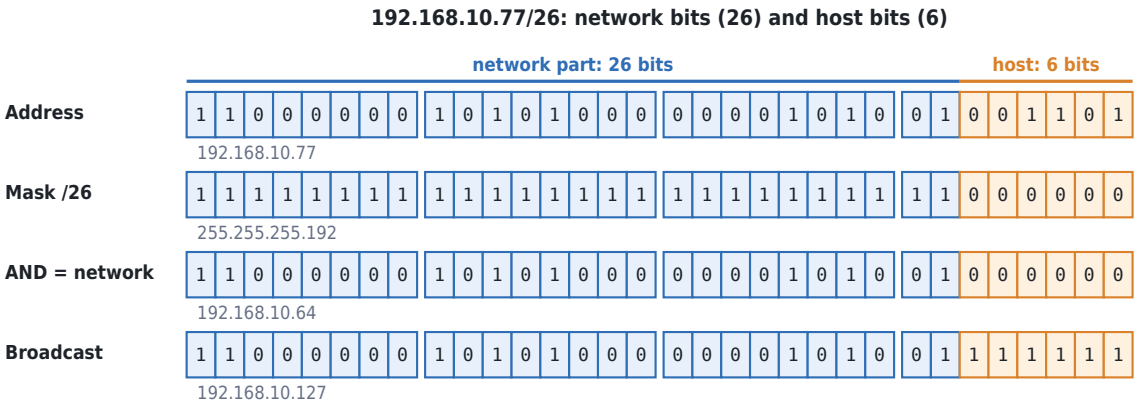
2.2.1 Concept and mechanism

Prefix length and subnet mask

The notation `192.168.10.0/26` states that the first 26 bits are the network prefix and the remaining 6 bits identify hosts. The equivalent **subnet mask** has ones in the prefix positions and zeros in the host positions: 26 ones give `11111111.11111111.11111111.11000000`, which is `255.255.255.192`. A device decides whether a destination is on its own network with one operation: if (destination AND mask) equals (own address AND mask), the destination is local and is reached by ARP on the link; otherwise the packet goes to the default gateway.

The four values that describe a subnet are derived as follows.

- **Network address**: address AND mask (all host bits zero). It names the subnet and is not assigned to a host.
- **Broadcast address**: the network address with all host bits set to one. A datagram sent to it reaches every host in the subnet.
- **Usable host range**: from network + 1 to broadcast - 1.
- **Number of usable hosts**: 2^h - 2 for h host bits. Two point-to-point exceptions exist: a /31 has two usable addresses (RFC 3021) and a /32 identifies exactly one interface, as used for loopback addresses and host routes.



Network = address AND mask. Broadcast = network with all host bits set to 1.

Host bits 000000 (.64) and 111111 (.127) are reserved: 2^6 - 2 = 62 usable hosts.

Figure 2.4. The address 192.168.10.77 with mask /26 written bit by bit: the AND operation clears the six host bits and produces the network 192.168.10.64; setting them to ones produces the broadcast address 192.168.10.127.

A shortcut avoids binary conversion. In the octet where the mask is not 0 or 255, the **block size** is 256 minus the mask value in that octet. For 255.255.255.192 the block size is 64, so the subnets in the last octet start at 0, 64, 128 and 192. An address belongs to the block whose start is the largest multiple of the block size not exceeding the address octet: 77 lies in the block starting at 64.

Prefix	Mask	Block size	Usable hosts	Typical use
/24	255.255.255.0	256	254	ordinary LAN
/25	255.255.255.128	128	126	large VLAN
/26	255.255.255.192	64	62	department VLAN
/27	255.255.255.224	32	30	small VLAN
/28	255.255.255.240	16	14	server segment
/29	255.255.255.248	8	6	small DMZ
/30	255.255.255.252	4	2	router link
/31	255.255.255.254	2	2	point-to-point link (RFC 3021)
/32	255.255.255.255	1	1	host route, loopback

Subnet quick reference for the last octet (the first three octets are 255).

Subnetting into equal parts

To split a block into N equal subnets, borrow b bits from the host part, where 2^b is at least N. The prefix grows by b, each subnet has 2^(h - b) addresses, and the subnet identifiers appear in the borrowed bits. For `192.168.10.0/24` and N = 4, b = 2 and the new prefix is /26.

sreeram

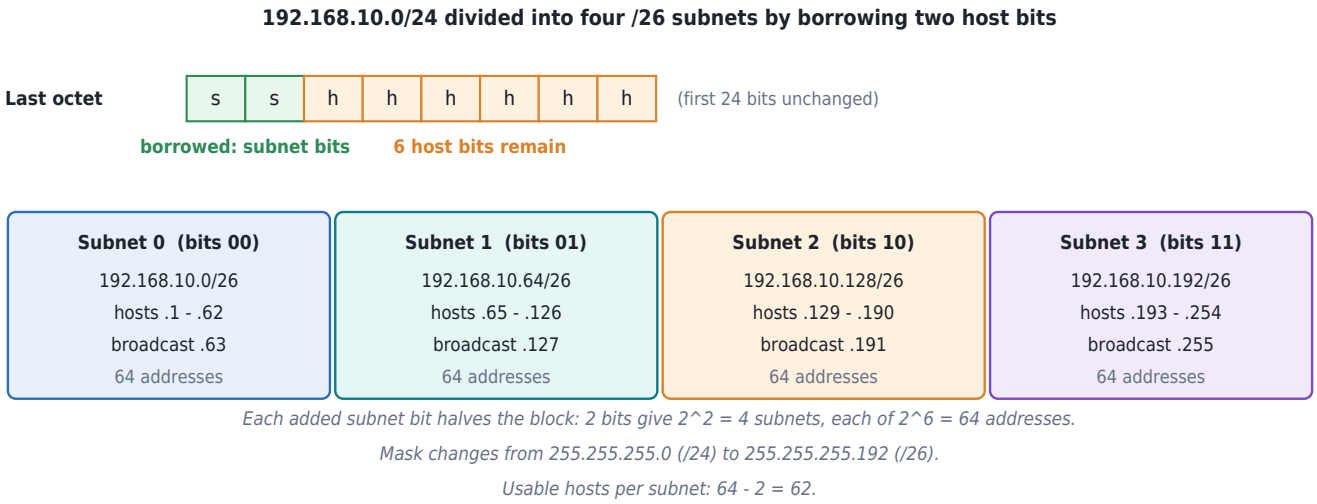


Figure 2.5. Borrowing the two high-order host bits of the last octet yields four /26 subnets whose network addresses differ only in those bits (00, 01, 10, 11); the remaining six host bits give 64 addresses each.

Variable-length subnet masks

Equal subnets waste addresses when the segments differ in size. **Variable-length subnet masking** (VLSM) allocates a block of the smallest sufficient size to each segment. The procedure has four steps.

1. For each segment, find the smallest h such that 2^h - 2 is at least the required number of hosts; the prefix is 32 - h.
2. Sort the segments by size, largest first.
3. Assign blocks consecutively from the start of the parent block. Sorting guarantees that every block begins at an address that is a multiple of its own size (alignment); an unaligned block cannot be expressed as a single prefix.
4. Check that the total fits in the parent block and record the unused remainder for growth.

Summarisation (supernetting)

A router that receives routes to several adjacent subnets can advertise one shorter prefix, provided the blocks are contiguous, the count is a power of two and the first block is aligned to the size of the combined block. Summarisation shrinks routing tables and hides internal changes from the rest of the network. The method is to find the longest common bit prefix of all networks.

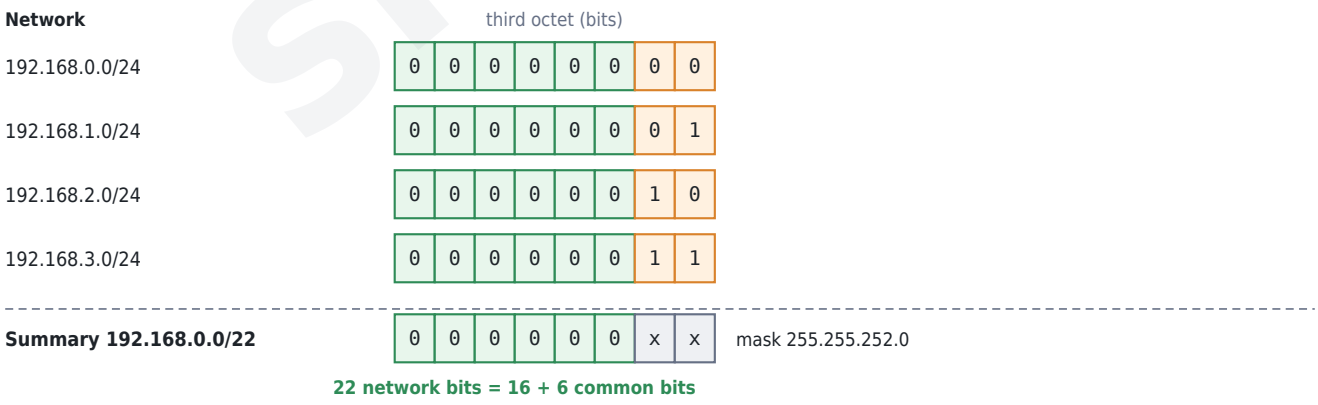


Figure 2.6. The third octets 0, 1, 2 and 3 share their first six bits; the common 16 + 6 = 22 bits form the summary prefix 192.168.0.0/22, which covers exactly the four /24 networks.

Special-purpose ranges

Not every address is globally routable. The table lists the ranges that appear in daily work.

Range	Name	Meaning
`10.0.0.0/8`	private (RFC 1918)	Internal use; not routed on the Internet. 16.7 million addresses.
`172.16.0.0/12`	private (RFC 1918)	172.16.0.0 to 172.31.255.255; 1 million addresses.
`192.168.0.0/16`	private (RFC 1918)	Typical home and office networks.
`100.64.0.0/10`	shared address space (RFC 6598)	Used between a carrier-grade NAT and customer equipment; not for general private use.
`127.0.0.0/8`	loopback	Traffic never leaves the host; 127.0.0.1 is `localhost`.
`169.254.0.0/16`	link-local (RFC 3927)	Self-assigned when no DHCP server answers; valid on one link only.
`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`	documentation (RFC 5737)	Reserved for examples such as the ones in this book.
`224.0.0.0/4`	multicast	Group addresses; 224.0.0.5 and 224.0.0.6 are used by OSPF.
`255.255.255.255`	limited broadcast	Reaches all hosts on the local link; never forwarded.
`0.0.0.0`	unspecified	Source address of a host that has no address yet (DHCP discover); also the default route `0.0.0.0/0`.

Special IPv4 ranges.

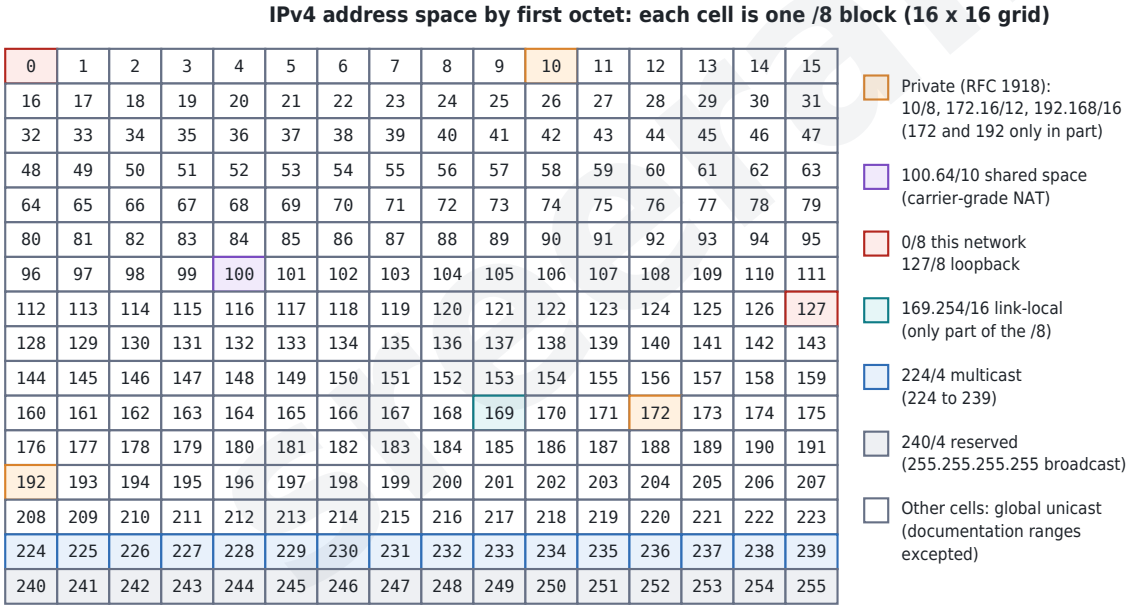


Figure 2.7. The first-octet map shows how little of the 256 /8 blocks is set aside for special use: private, loopback and link-local ranges are small, multicast occupies 16 blocks and the reserved class E occupies another 16.

Private ranges are the reason that NAT exists (see the chapter on NAT and DHCP). Because every organisation uses the same few ranges, two networks that must be connected, for example through a VPN or VPC peering, cannot overlap. Overlap is one of the most frequent avoidable mistakes in cloud and container networking.

2.2.2 Worked trace

Example 1: classify an address and find its boundaries

Given 192.168.10.77/26 (the diagram above), the mask octet is 192, the block size is 256 - 192 = 64 and 77 lies in the block starting at 64. The network is 192.168.10.64, the broadcast is 64 + 63 = 127, the usable range is .65 to .126 (62 hosts). The second example in the program output below uses a larger prefix: 172.16.5.130/20 has a mask of 255.255.240.0, a block size of 16 in the third octet and 5 lies in the block starting at 0, so the network is 172.16.0.0 and the broadcast is 172.16.15.255.

Example 2: equal subnets

Requirement: six departments, at most 25 hosts each, from 192.168.20.0/24. Six subnets need b = 3 bits (2^3 = 8 is at least 6) and leave 5 host bits, which give 30 usable hosts, sufficient for 25. The prefix is /27 and the subnets are 192.168.20.0, .32, .64, .96, .128, .160, .192 and .224 (eight subnets, two spare).

Example 3: VLSM design

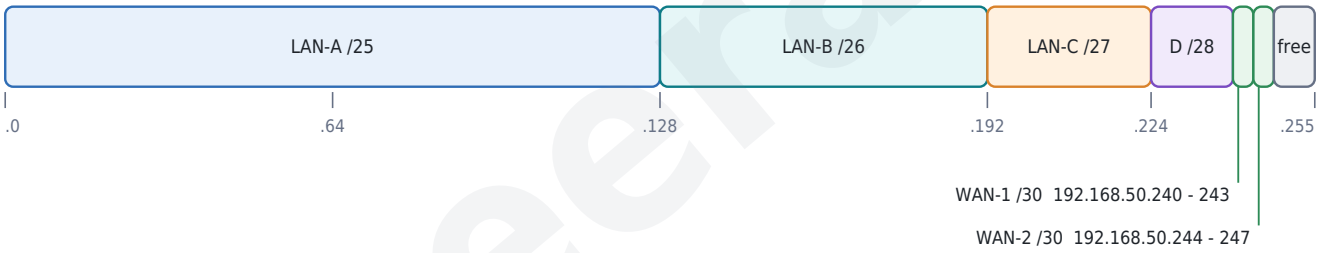
Requirement: from `192.168.50.0/24` allocate LAN-A (100 hosts), LAN-B (50), LAN-C (25), LAN-D (10) and two point-to-point router links (2 hosts each).

Segment	Hosts needed	Host bits	Prefix	Block size	Reason
LAN-A	100	7	/25	128	2^6 - 2 = 62 is too few; 2^7 - 2 = 126
LAN-B	50	6	/26	64	2^6 - 2 = 62
LAN-C	25	5	/27	32	2^5 - 2 = 30
LAN-D	10	4	/28	16	2^4 - 2 = 14
WAN-1, WAN-2	2 each	2	/30	4 each	2^2 - 2 = 2

Derivation of the block sizes.

The blocks are placed in order of size: LAN-A at 0, LAN-B at 128, LAN-C at 192, LAN-D at 224, WAN-1 at 240 and WAN-2 at 244. The total of 128 + 64 + 32 + 16 + 4 + 4 = 248 addresses fits in 256 and leaves 192.168.50.248/29 free.

VLSM allocation of 192.168.50.0/24 (256 addresses, drawn to scale)




```
LAN-A  needs 100  ->  /25  192.168.50.0  usable .1 - .126  (capacity 126)
LAN-B  needs 50  ->  /26  192.168.50.128  usable .129 - .190  (capacity 62)
LAN-C  needs 25  ->  /27  192.168.50.192  usable .193 - .222  (capacity 30)
LAN-D  needs 10  ->  /28  192.168.50.224  usable .225 - .238  (capacity 14)
```

Largest requirement first keeps every block aligned on its own size boundary.

Figure 2.8. The six blocks occupy the /24 without gaps because they were placed largest first; each block starts at an address divisible by its own size, and the final eight addresses stay free for growth.

Example 4: summarisation

Four branch offices use 192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24. The third octets are 00000000, 00000001, 00000010 and 00000011, so the first six bits agree. The summary is 192.168.0.0/22 with mask 255.255.252.0. Three of the four networks (0, 1 and 2) cannot be expressed as one prefix: the best aggregation is 192.168.0.0/23 plus 192.168.2.0/24, which the Python `ipaddress` module also reports for these inputs.

2.2.3 Hands-on

The listing implements a small subnet calculator with `long` arithmetic. `describe` prints the derived values of one CIDR block and `vlsm` assigns blocks to a list of host requirements, largest first.

SubnetCalc.java JAVA

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class SubnetCalc {
    static long parse(String s) {
        String[] p = s.split("\\.");
        long v = 0;
        for (String o : p) {
            v = (v << 8) | Integer.parseInt(o);
        }
        return v;
    }

    static String fmt(long v) {
        return ((v >> 24) & 255) + "." + ((v >> 16) & 255) + "."
                + ((v >> 8) & 255) + "." + (v & 255);
    }

    static String bin(long v) {
        String s = String.format("%32s", Long.toBinaryString(v)).replace(' ', '0');
        return s.substring(0, 8) + "." + s.substring(8, 16) + "." + s.substring(16, 24)
                + "." + s.substring(24);
    }

    static long mask(int prefix) {
        return prefix == 0 ? 0 : (0xFFFFFFFFL << (32 - prefix)) & 0xFFFFFFFFL;
    }

    static void describe(String cidr) {
        String[] p = cidr.split("/");
        long addr = parse(p[0]);
        int len = Integer.parseInt(p[1]);
        long m = mask(len);
        long net = addr & m;
        long bcast = net | (~m & 0xFFFFFFFFL);
        long total = 1L << (32 - len);
        long usable = len >= 31 ? total : total - 2;
        System.out.println("Address      : " + fmt(addr) + "   " + bin(addr));
        System.out.println("Mask         : " + fmt(m) + "   " + bin(m));
        System.out.println("Network      : " + fmt(net) + "/" + len);
        System.out.println("Broadcast    : " + fmt(bcast));
        System.out.println("Host range   : " + fmt(len >= 31 ? net : net + 1) + " - "
                + fmt(len >= 31 ? bcast : bcast - 1));
        System.out.println("Usable hosts : " + usable);
        System.out.println();
    }

    record Need(String name, int hosts) { }

    static void vlsm(String base, List<Need> needs) {
        long cursor = parse(base.split("/")[0]);
        List<Need> sorted = new ArrayList<>(needs);
        sorted.sort(Comparator.comparingInt(Need::hosts).reversed());
        System.out.printf("%-8s %6s %7s %-18s %-15s %s%n", "Name", "Hosts", "Prefix", "Network",
                "Broadcast", "Usable range");
        for (Need n : sorted) {
            int bits = 2;
            while ((1L << bits) - 2 < n.hosts()) {
                bits++;
            }
            int prefix = 32 - bits;
            long size = 1L << bits;
            long bcast = cursor + size - 1;
            System.out.printf("%-8s %6d %7s %-18s %-15s %s - %s%n", n.name(), n.hosts(),
                    "/" + prefix, fmt(cursor) + "/" + prefix, fmt(bcast), fmt(cursor + 1),
                    fmt(bcast - 1));
            cursor += size;
        }
    }

    public static void main(String[] args) {
        describe("192.168.10.77/26");
        describe("172.16.5.130/20");
        vlsm("192.168.50.0/24", List.of(new Need("LAN-A", 100), new Need("LAN-B", 50),
```

sreram

SubnetCalc.java (continued) JAVA

```
                new Need("LAN-C", 25), new Need("LAN-D", 10), new Need("WAN-1", 2),
                new Need("WAN-2", 2)));
    }
}
```

Output (deterministic) TERMINAL

```
$ java SubnetCalc.java
Address      : 192.168.10.77  11000000.10101000.00001010.01001101
Mask         : 255.255.255.192  11111111.11111111.11111111.11000000
Network      : 192.168.10.64/26
Broadcast    : 192.168.10.127
Host range   : 192.168.10.65 - 192.168.10.126
Usable hosts : 62

Address      : 172.16.5.130  10101100.00010000.00000101.10000010
Mask         : 255.255.240.0  11111111.11111111.11110000.00000000
Network      : 172.16.0.0/20
Broadcast    : 172.16.15.255
Host range   : 172.16.0.1 - 172.16.15.254
Usable hosts : 4094

Name      Hosts  Prefix Network            Broadcast       Usable range
LAN-A       100     /25 192.168.50.0/25    192.168.50.127  192.168.50.1 - 192.168.50.126
LAN-B        50     /26 192.168.50.128/26  192.168.50.191  192.168.50.129 - 192.168.50.190
LAN-C        25     /27 192.168.50.192/27  192.168.50.223  192.168.50.193 - 192.168.50.222
LAN-D        10     /28 192.168.50.224/28  192.168.50.239  192.168.50.225 - 192.168.50.238
WAN-1         2     /30 192.168.50.240/30  192.168.50.243  192.168.50.241 - 192.168.50.242
WAN-2         2     /30 192.168.50.244/30  192.168.50.247  192.168.50.245 - 192.168.50.246
```

What to notice: the `vlsm` method finds the smallest bit count with `(1L << bits) - 2 >= hosts`, advances the cursor by the block size, and therefore reproduces the table of Example 3. The 32-bit values are held in a `long` so that masks and addresses stay positive and do not sign-extend when shifted.

Inspecting interface addresses from Java

`java.net.NetworkInterface` exposes the prefix length of each configured address. The output below was produced on the build machine inside a sandbox; the `eth0` address is from the documentation range and the MTU of 1400 reflects the sandbox network, not the usual 1500.

ListInterfaces.java JAVA

```
import java.net.InetAddress;
import java.net.InterfaceAddress;
import java.net.NetworkInterface;
import java.util.Collections;

public class ListInterfaces {
    public static void main(String[] args) throws Exception {
        for (NetworkInterface ni : Collections.list(NetworkInterface.getNetworkInterfaces())) {
            System.out.printf("%s up=%b loopback=%b mtu=%d%n", ni.getName(), ni.isUp(),
                    ni.isLoopback(), ni.getMTU());
            for (InterfaceAddress ia : ni.getInterfaceAddresses()) {
                InetAddress a = ia.getAddress();
                System.out.printf("  %s/%d site-local=%b%n", a.getHostAddress(),
                        ia.getNetworkPrefixLength(), a.isSiteLocalAddress());
            }
        }
    }
}
```

Output (measured on the build machine; differs on other hosts) TERMINAL

```
$ java ListInterfaces.java
eth0 up=true loopback=false mtu=1400
  192.0.2.2/24 site-local=false
lo up=true loopback=true mtu=65536
  127.0.0.1/8 site-local=false
```

2.2.4 Failure modes

Symptom	Cause	Diagnosis and fix
Host reaches some local machines but not others	Wrong mask on the host (for example /25 instead of /24) so that part of the subnet is treated as remote	Compare mask and gateway on both hosts; the AND test shows which peers are considered local
Address `169.254.x.x` on an interface	DHCP discover received no answer	Check the DHCP server, relay and VLAN; the host falls back to link-local addressing
Containers cannot reach an internal network	The default Docker bridge 172.17.0.0/16 overlaps a corporate range	Configure a different default address pool for the container engine
Two connected networks cannot communicate	Overlapping private ranges (both use 10.0.0.0/16, for example)	Renumber one side or use NAT between them; plan ranges before creating cloud networks
Subnet runs out of addresses	Block too small; cloud providers reserve several addresses per subnet (five in AWS)	Allocate with growth in mind; a subnet cannot be resized after creation
Customers behind a carrier-grade NAT cannot accept inbound connections	Address in 100.64.0.0/10 on the WAN interface	Use a relay or an outbound tunnel; request a public address

Subnetting and addressing faults.

GOTCHA

A prefix of /31 or /32 has no network and broadcast address in the usual sense. Calculators that apply the 2^h - 2 rule produce -1 hosts or 0 hosts for these prefixes. The program above applies the exception explicitly.

2.2.5 Questions to expect

1. Given 10.20.30.40/21, what are the network, broadcast and host range?

The mask is 255.255.248.0, the block size in the third octet is 8, and 30 lies in the block starting at 24. The network is 10.20.24.0, the broadcast 10.20.31.255, and the hosts are 10.20.24.1 to 10.20.31.254 (2046 hosts).

2. How many usable hosts does a /27 provide and why minus two?

32 - 27 = 5 host bits give 32 addresses. The all-zero host part names the network and the all-ones host part is the directed broadcast, so 30 addresses remain for interfaces.

3. Split 192.168.1.0/24 into at least 10 subnets. What is the new mask and the hosts per subnet?

Four borrowed bits give 16 subnets (three give only 8). The prefix is /28, the mask 255.255.255.240, and each subnet has 14 usable hosts.

4. What is VLSM and why is the largest-first order important?

VLSM assigns each segment a prefix that fits its size. Placing large blocks first keeps every block aligned on a multiple of its own size, so no address space is lost to alignment gaps and every block is a valid prefix.

5. When can routes be summarised?

When the networks are contiguous, their number is a power of two and the first network address is aligned to the size of the combined block. For example 192.168.0.0/24 through 192.168.3.0/24 summarise to 192.168.0.0/22, whereas 192.168.1.0/24 through 192.168.4.0/24 do not.

6. Why can two hosts in the same subnet communicate without a router?

The sender finds the destination inside its own prefix by the AND test and resolves the destination MAC address with ARP on the local link. A router is involved only when the prefix test fails.

7. What is the difference between a private address and a link-local address?

A private address (RFC 1918) is routable inside an organisation and assigned by an administrator or DHCP. A link-local address (169.254.0.0/16 in IPv4, fe80::/10 in IPv6) is valid only on the attached link and is never forwarded by routers.

sreeram

8. Why is 100.64.0.0/10 not part of RFC 1918?

It is shared address space reserved for carrier-grade NAT (RFC 6598). Using a separate block prevents a service provider from clashing with the private ranges customers already use inside their homes and offices.

9. How would the choice of address ranges affect a VPN between two offices?

Both offices must use non-overlapping ranges. If both use 192.168.1.0/24, a route to that prefix is ambiguous and traffic cannot be directed to the remote side without translation or renumbering.

INTERVIEW ANGLE

Follow-up: a /24 host sends to a destination whose address ANDed with the mask equals its own network. What happens next? The destination is on-link. The host does not use the gateway. It broadcasts an ARP request for the destination address, caches the reply and sends the frame directly to that MAC address. If the destination is not on-link, the host resolves the MAC address of the default gateway instead, and the IP destination address remains the original destination.

KEY POINTS

- A prefix length splits an address into network and host bits; network = address AND mask, broadcast = host bits all ones, usable hosts = 2^h - 2.
- Equal subnetting borrows b bits for 2^b subnets; VLSM sizes each block to its segment, largest first, to keep blocks aligned.
- Summarisation replaces contiguous aligned blocks with the shared bit prefix, for example four /24 networks with a /22.
- Private (RFC 1918), shared (100.64.0.0/10), loopback, link-local and documentation ranges are not globally routable; overlapping private ranges are a common integration fault.
- The same AND test decides on-link delivery versus use of the default gateway.

2.3 IPv6

IPv4 offers about 4.3 billion addresses, and the central pool was exhausted in 2011; regional registries followed. NAT and carrier-grade NAT extended the life of IPv4 but at a cost: hosts lose end-to-end addressing, inbound connections need explicit mappings and applications need workarounds. IPv6 (RFC 8200) enlarges the address to 128 bits, simplifies the header, removes router fragmentation and the header checksum, and replaces ARP and DHCP discovery with ICMPv6-based mechanisms. Adoption is gradual, so engineers work with both protocols for the foreseeable future, and Java applications must handle both address families correctly.

2.3.1 Concept and mechanism

Address notation and compression

An IPv6 address consists of eight groups of 16 bits written as up to four hexadecimal digits separated by colons: `2001:0db8:0000:0000:0000:ff00:0042:8329`. Two rules shorten the text form. The canonical recommendation is RFC 5952.

1. Leading zeros in each group may be dropped: `0db8` becomes `db8`, `0042` becomes `42`.
2. One run of consecutive all-zero groups may be replaced by `::`. The replacement may be used only once, because a second use would make the number of omitted groups ambiguous. RFC 5952 requires that the longest run is replaced (the first one if several runs are equally long), that a single zero group is not replaced, and that lowercase letters are used.

Applying both rules gives `2001:db8::ff00:42:8329`. The address `2001:db8:0:0:1:0:0:1` has two runs of two zero groups, so the first is replaced: `2001:db8::1:0:0:1`. A prefix is written like an IPv4 CIDR prefix, for example `2001:db8:acad::/48`. In a URL the address is enclosed in square brackets, `http://[2001:db8::1]:8080/`, because the colons would otherwise conflict with the port separator.

Address types

IPv6 has no broadcast. Its functions are covered by multicast. A single interface normally holds several addresses of different scope at the same time.

Prefix	Type	Description
`2000::/3`	global unicast	Globally routable, assigned by registries and providers. 2001:db8::/32 is reserved for documentation (RFC 3849).
`fe80::/10`	link-local	Automatically configured on every interface; valid only on the attached link; always required.
`fc00::/7`	unique local	Private addresses (in practice fd00::/8 with a random 40-bit global ID); not routed on the Internet.
`::1/128`	loopback	The local host; equal to 127.0.0.1 in IPv4.
`::/128`	unspecified	Source address before an address has been configured.
`ff00::/8`	multicast	Group addresses; ff02::1 all nodes on the link, ff02::2 all routers.
`::ffff:0:0/96`	IPv4-mapped	Represents an IPv4 address inside an IPv6 socket API, for example ::ffff:192.0.2.128.
`Anycast`	anycast	A unicast address assigned to several interfaces; delivery goes to the nearest one. Not a separate range.

IPv6 address types.

A typical provider assigns a /48 to a site or a /56 to a home. The next 16 bits (or 8 bits) number the subnets, and every subnet uses a /64, so that the last 64 bits form the **interface identifier**. The /64 subnet size is fixed by stateless address autoconfiguration and by many implementations.

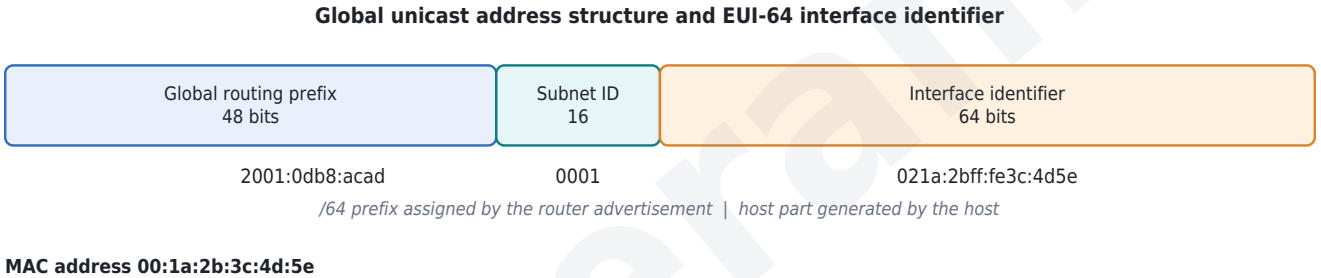


Figure 2.9. A global unicast address splits into a 48-bit routing prefix, a 16-bit subnet identifier and a 64-bit interface identifier; the EUI-64 method derives the identifier from the MAC address by inserting ff:fe and inverting one bit.

The header

The IPv6 header has a fixed length of 40 bytes. All optional information moved into **extension headers** that follow the base header and are chained through the Next Header field. Routers process only the hop-by-hop extension header; all others are examined at the destination.

Sberam

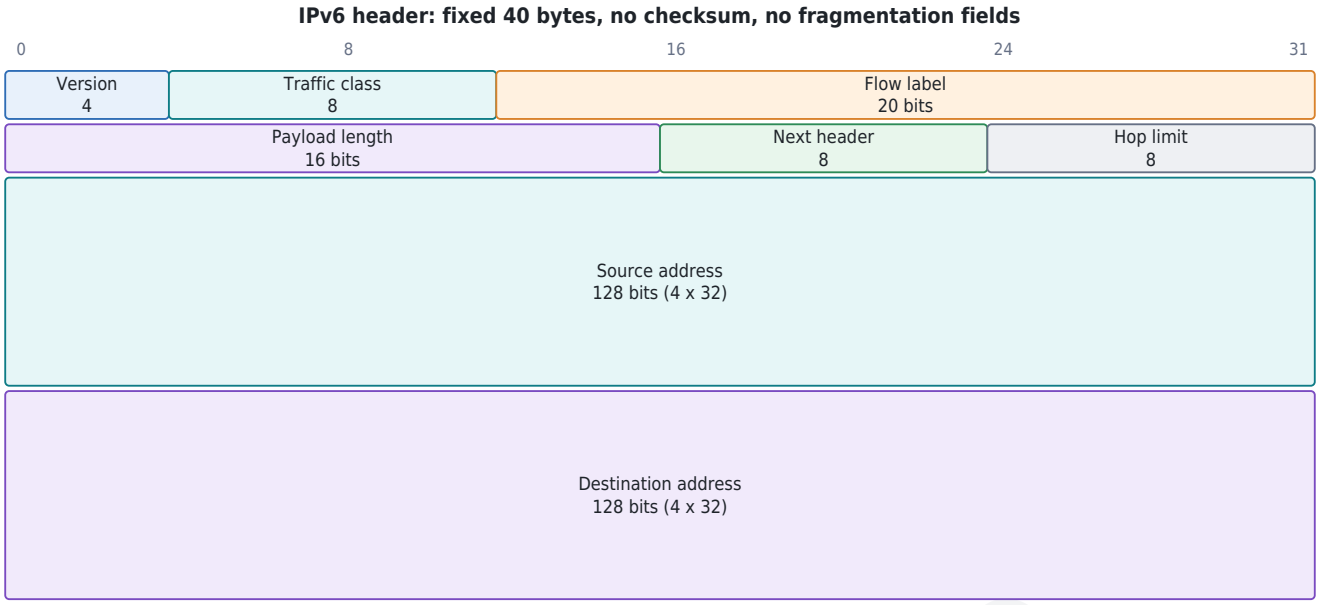


Figure 2.10. The IPv6 base header has eight fields; compared with IPv4 it has no header length, no checksum, no identification, flags or fragment offset, and its addresses are four times as long.

Field	Bits	Remark
Version	4	Value 6.
Traffic class	8	Same role as DSCP plus ECN.
Flow label	20	Identifies a flow; allows load balancing without reading the transport header.
Payload length	16	Length of everything after the base header (extension headers included).
Next header	8	Type of the following header: 6 TCP, 17 UDP, 58 ICMPv6, 44 fragment extension header.
Hop limit	8	Replaces TTL.
Source and destination	128 each	Addresses.

IPv6 header fields compared with IPv4.

Three changes affect behaviour. First, **routers do not fragment**. A source that has more data than the path MTU allows must fragment itself using the fragment extension header, so Path MTU Discovery is mandatory and every link must support at least 1280 bytes (RFC 8200). Second, there is **no header checksum**, which saves router work; the upper-layer checksums of TCP and UDP therefore include a pseudo-header with the IPv6 addresses, and UDP checksums are mandatory. Third, the header fields are aligned on 64-bit boundaries, which suits hardware forwarding.

Neighbour discovery

IPv6 replaces ARP with the **Neighbor Discovery Protocol** (NDP, RFC 4861), carried in ICMPv6 messages.

ICMPv6 type	Name	Purpose
133	Router Solicitation (RS)	A host asks for routers to announce themselves immediately.
134	Router Advertisement (RA)	A router announces prefixes, flags, hop limit, MTU and its own lifetime as default gateway.
135	Neighbor Solicitation (NS)	Resolves an IPv6 address to a MAC address; also used for duplicate address detection.
136	Neighbor Advertisement (NA)	Answer to an NS.
137	Redirect	A router tells a host about a better next hop.

NDP message types.

An NS is not broadcast. It is sent to the **solicited-node multicast address** `ff02::1:ffXX:XXXX`, built from the fixed prefix `ff02::1:ff00:0/104` and the last 24 bits of the target address. Only nodes whose addresses share these 24 bits listen to the group, so the interruption of unrelated hosts is far smaller than with ARP broadcasts.

Address configuration: SLAAC and DHCPv6

A host obtains a global address in one of three ways.

- **SLAAC** (stateless address autoconfiguration, RFC 4862): the host takes the /64 prefix from a router advertisement whose prefix option has the **A** (autonomous) flag set and appends an interface identifier generated by itself. No server keeps state.
- **Stateful DHCPv6** (RFC 8415): the router advertisement sets the **M** (managed) flag and the host requests an address from a DHCPv6 server (UDP ports 546 and 547), which keeps a lease table.
- **Stateless DHCPv6**: SLAAC provides the address and the **O** (other) flag tells the host to ask a DHCPv6 server only for additional settings such as DNS servers.

The router advertisement does not carry a gateway address explicitly: the source link-local address of the RA is the default gateway. RA options also carry recursive DNS servers (RFC 8106).

The interface identifier was originally derived from the MAC address by the EUI-64 procedure, which exposes the hardware address and allows tracking across networks. Current systems use stable but opaque identifiers (RFC 7217) and generate **temporary addresses** (RFC 8981) that change periodically for outgoing connections.

Transition mechanisms

Since the networks are not converted at once, several mechanisms allow IPv4 and IPv6 to coexist.

- **Dual stack**: hosts and routers run both protocols. DNS returns A and AAAA records, and the client applies Happy Eyeballs (RFC 8305): it starts connection attempts over IPv6 and IPv4 with a short stagger and uses the first that succeeds, which hides broken IPv6 paths.
- **Tunnelling**: IPv6 packets are encapsulated inside IPv4 packets (protocol 41) to cross an IPv4-only network.
- **Translation** (NAT64 with DNS64, RFC 6146): an IPv6-only client uses a synthesised address in the prefix `64:ff9b::/96` (RFC 6052); the NAT64 gateway extracts the embedded IPv4 address and forwards the request over IPv4.

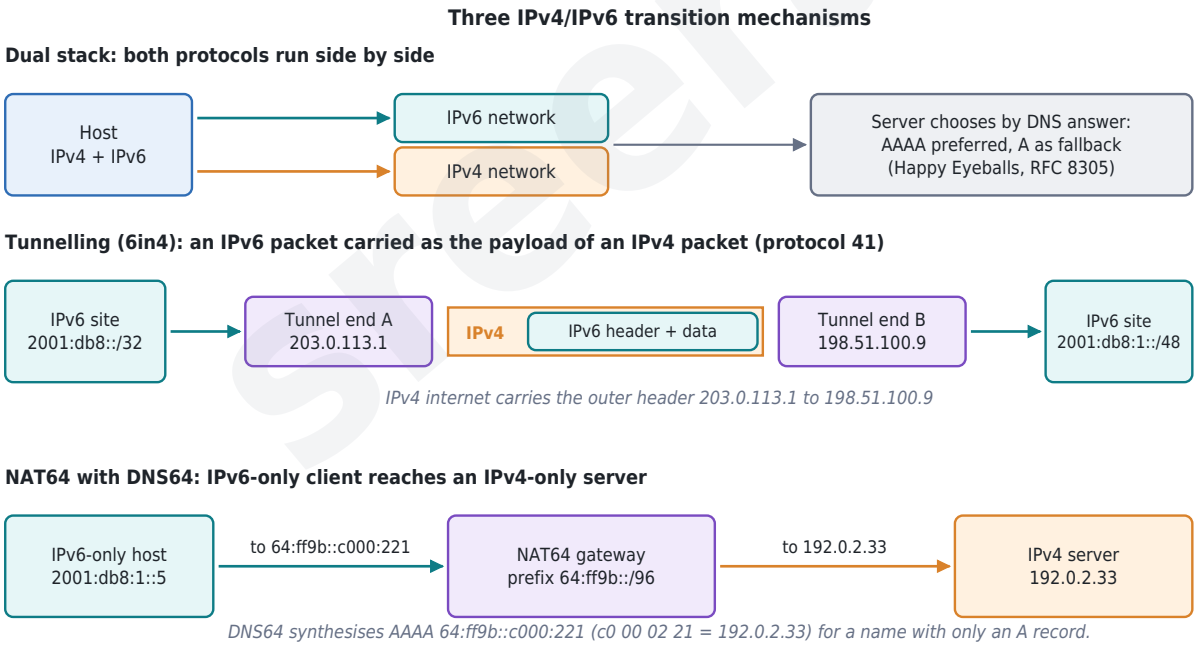


Figure 2.11. Dual stack runs both protocols; tunnelling carries IPv6 as the payload of IPv4; NAT64 converts between an IPv6-only client and an IPv4-only server using the embedded address 64:ff9b::c000:221.

Why NAT is less necessary

The IPv6 address space is large enough to give every device a globally unique address, so address conservation is not a reason for translation. Hosts keep end-to-end connectivity, and filtering is performed by stateful firewalls rather than as a side effect of translation. NAT does not provide security in either protocol: the protection comes from the stateful default-deny rule, which an IPv6 firewall implements equally well. Some organisations still use prefix translation (NPTv6) for provider independence.

sreeram

2.3.2 Worked trace

Interface identifier and SLAAC for a host

A host has the MAC address `00:1a:2b:3c:4d:5e` and receives a router advertisement with prefix `2001:db8:acad:1::/64`.

1. The EUI-64 procedure splits the MAC address into `00:1a:2b` and `3c:4d:5e`, inserts `ff:fe` between them (`00:1a:2b:ff:fe:3c:4d:5e`) and inverts the universal/local bit, which is bit 1 of the first byte: `00` (binary 00000000) becomes `02` (binary 00000010). The identifier is `021a:2bff:fe3c:4d5e`, written `21a:2bff:fe3c:4d5e`.
2. The host forms the link-local address `fe80::21a:2bff:fe3c:4d5e` and performs duplicate address detection: it sends an NS with source `::` to the solicited-node address `ff02::1:ff3c:4d5e` (formed from the last 24 bits 3c:4d:5e). No NA arrives, so the address is unique.
3. The host sends an RS to `ff02::2`. The router answers with an RA that contains the prefix `2001:db8:acad:1::/64` with the A and L flags set (A: autonomous configuration allowed; L: the prefix is on-link).
4. The host forms the global address `2001:db8:acad:1:21a:2bff:fe3c:4d5e`, repeats duplicate address detection, installs the on-link route for the /64 and adds a default route through the RA source `fe80::1`.

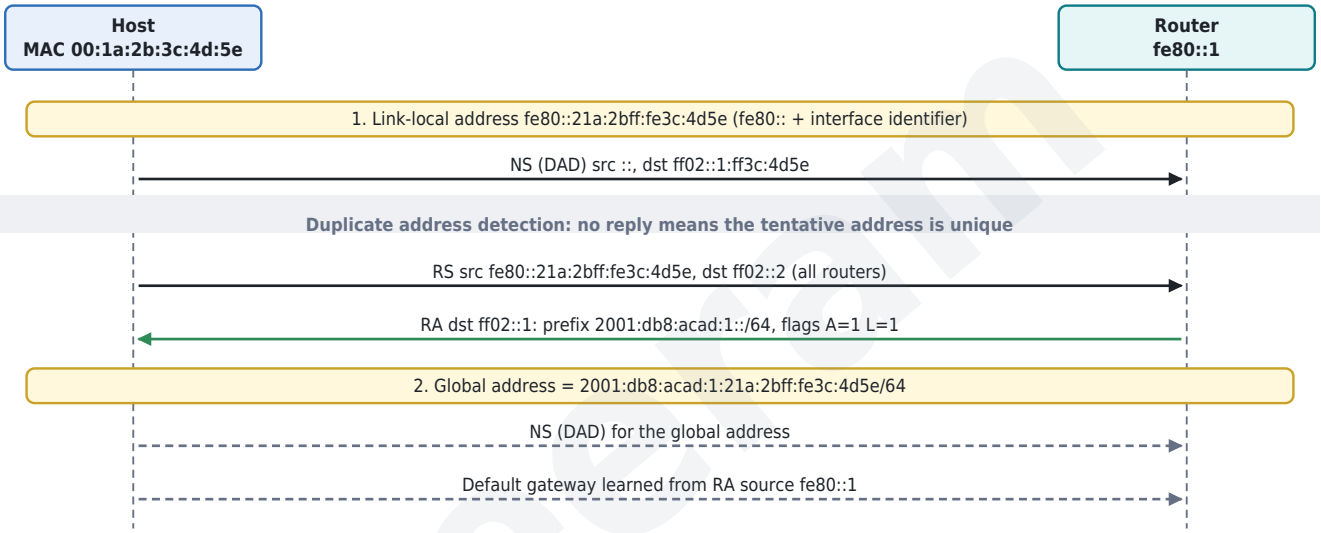


Figure 2.12. The host configures a link-local address, discovers the prefix from a router advertisement and derives its global address without any server that stores state.

Compression exercises

Full form	Shortest canonical form
`2001:0db8:0000:0000:0000:ff00:0042:8329`	`2001:db8::ff00:42:8329`
`2001:0db8:0000:0000:0001:0000:0000:0001`	`2001:db8::1:0:0:1`
`0000:0000:0000:0000:0000:0000:0000:0001`	`::1`
`fe80:0000:0000:0000:01ff:fe23:4567:890a`	`fe80::1ff:fe23:4567:890a`

Compression of IPv6 addresses (verified with the Python ipaddress module).

2.3.3 Hands-on

Java handles both families through `InetAddress`, which returns `Inet4Address` or `Inet6Address`. The program parses several address strings and derives an EUI-64 identifier. The build machine has no IPv6 stack enabled, but parsing literals does not need one.

V6Demo.java JAVA

```java
import java.net.Inet6Address;
import java.net.InetAddress;

public class V6Demo {
    static String eui64(String mac) {
        String[] p = mac.split(":");
        int first = Integer.parseInt(p[0], 16) ^ 0x02;
```

V6Demo.java (continued) JAVA

```
        return String.format("%02x%s:%sff:fe%s:%s%s", first, p[1], p[2], p[3], p[4], p[5]);
    }

    public static void main(String[] args) throws Exception {
        String[] in = {"2001:0db8:0000:0000:0000:ff00:0042:8329", "2001:db8:0:0:1:0:0:1",
                "0:0:0:0:0:0:0:1", "fe80::1ff:fe23:4567:890a", "::ffff:192.0.2.128"};
        for (String s : in) {
            InetAddress a = InetAddress.getByName(s);
            String full = a.getHostAddress() + (a instanceof Inet6Address ? "" : " (IPv4)");
            System.out.println(s + "\n  Java: " + full + "  linkLocal=" + a.isLinkLocalAddress()
                    + " loopback=" + a.isLoopbackAddress());
        }
        System.out.println("EUI-64 of 00:1a:2b:3c:4d:5e = " + eui64("00:1a:2b:3c:4d:5e"));
        System.out.println("localhost -> " + InetAddress.getByName("localhost"));
        for (InetAddress x : InetAddress.getAllByName("localhost")) {
            System.out.println("  " + x.getClass().getSimpleName() + " " + x);
        }
    }
}
```

Output (deterministic) TERMINAL

```
$ java V6Demo.java
2001:0db8:0000:0000:0000:ff00:0042:8329
  Java: 2001:db8:0:0:0:ff00:42:8329  linkLocal=false loopback=false
2001:db8:0:0:1:0:0:1
  Java: 2001:db8:0:0:1:0:0:1  linkLocal=false loopback=false
0:0:0:0:0:0:0:1
  Java: 0:0:0:0:0:0:0:1  linkLocal=false loopback=true
fe80::1ff:fe23:4567:890a
  Java: fe80:0:0:0:1ff:fe23:4567:890a  linkLocal=true loopback=false
::ffff:192.0.2.128
  Java: 192.0.2.128 (IPv4)  linkLocal=false loopback=false
EUI-64 of 00:1a:2b:3c:4d:5e = 021a:2bff:fe3c:4d5e
localhost -> localhost/127.0.0.1
  Inet4Address localhost/127.0.0.1
```

What to notice: `getHostAddress()` prints the uncompressed form with leading zeros removed (`2001:db8:0:0:0:ff00:42:8329`), not the canonical RFC 5952 form, so string comparison of addresses is unreliable; compare `InetAddress` objects or byte arrays. The IPv4-mapped literal `::ffff:192.0.2.128` is converted to an `Inet4Address` by `InetAddress.getByName`.

Sockets and system properties (typical behaviour; not run here)

The JVM selects the address family through two system properties. `java.net.preferIPv4Stack=true` makes the JVM use only IPv4 sockets, and `java.net.preferIPv6Addresses=true` makes name resolution return IPv6 addresses first. By default the JVM prefers IPv4 addresses when both families are available. A server socket bound to the wildcard address on Linux with a dual-stack kernel accepts IPv4 connections as IPv4-mapped IPv6 addresses. In Spring Boot, `server.address=::` (or leaving it unset) listens on all interfaces; `server.address=127.0.0.1` restricts the server to local IPv4 clients, which is a common cause of "works with the address but not with localhost" when `localhost` resolves to `::1` first.

Typical output of a connection to an IPv6 loopback listener (not run: the build machine has no IPv6 stack) TERMINAL

```
$ curl -g -sv -o /dev/null http://[::1]:8080/
*   Trying [::1]:8080...
* Connected to ::1 (::1) port 8080
> GET / HTTP/1.1
> Host: [::1]:8080
```

2.3.4 Failure modes

Symptom	Cause	Diagnosis and fix
Connections stall for several seconds, then succeed over IPv4	DNS returns an AAAA record but the IPv6 path is broken	Test each family separately (`curl -4`, `curl -6`); fix the IPv6 route or remove the AAAA record
Service reachable on `127.0.0.1` but not on `localhost`	`localhost` resolves to `::1` and the server listens only on IPv4	Bind to both families or use the explicit literal
Large responses hang on IPv6 only	ICMPv6 Packet Too Big messages are filtered, so Path MTU Discovery fails	Permit ICMPv6 types 1 to 4 and 128/129 in firewalls; ICMPv6 is not optional
No address is assigned	No router advertisements, or RA guard blocks them	Check the router and switch security features
Rate limiting treats many users as one	Limits applied per address while users hold addresses in one /64	Aggregate IPv6 limits by /64 (or /56), not by single address

IPv6-related faults.

WATCH OUT

Blocking all ICMP, a habit inherited from IPv4 firewalls, breaks IPv6. Neighbor Discovery, duplicate address detection and Path MTU Discovery depend on ICMPv6. Filtering must be selective, never wholesale.

2.3.5 Questions to expect

1. Why does IPv6 not need ARP?

Address resolution is performed by Neighbor Discovery inside ICMPv6, using Neighbor Solicitation sent to a solicited-node multicast address and Neighbor Advertisement replies. The mechanism works for every link technology that supports multicast and avoids broadcast.

2. State the two rules for abbreviating an IPv6 address.

Leading zeros in a group may be omitted, and one run of consecutive zero groups may be replaced by `::`. The `::` may appear only once so that the number of omitted groups is unambiguous.

3. What is the difference between SLAAC and DHCPv6?

With SLAAC the host builds its address from the advertised /64 prefix and a self-generated identifier, and no server stores state. DHCPv6 assigns addresses from a server pool and records leases. The M and O flags in the router advertisement tell hosts which method to use.

4. Which IPv4 header fields disappeared in IPv6 and why?

The header length (no options in the base header), the checksum (link layers and transport protocols verify integrity), and the identification, flags and fragment offset (routers no longer fragment; sources use an extension header). TTL was renamed to hop limit.

5. What happens when an IPv6 packet is larger than the MTU of a link?

The router drops it and sends an ICMPv6 Packet Too Big message that carries the link MTU. The source reduces its packet size or fragments with the fragment extension header. The minimum link MTU is 1280 bytes.

6. What is a link-local address and why is it always present?

It is an address in fe80::/10 that every IPv6 interface configures automatically. Neighbor Discovery, router advertisements and routing protocols use it before any global address exists, and the default gateway is learned as a link-local address.

7. Does IPv6 make NAT unnecessary for security?

NAT is not a security mechanism. Protection against unsolicited inbound traffic comes from a stateful firewall, which works the same way for IPv6 with global addresses. NAT gave IPv4 networks this behaviour as a side effect of translation.

sreeram

8. How does a dual-stack client decide between IPv4 and IPv6?

DNS returns A and AAAA records. The client orders candidate addresses (normally IPv6 first), starts the first attempt and, after a short delay, starts the next family in parallel. The first established connection wins (Happy Eyeballs, RFC 8305).

9. What is an IPv4-mapped IPv6 address?

An address of the form ::ffff:a.b.c.d. It lets an IPv6 socket represent an IPv4 peer, so a single dual-stack server socket can accept both families.

KEY POINTS

- IPv6 uses 128-bit addresses; zero-group compression with `::` is permitted once per address, and URLs need brackets.
- The base header is a fixed 40 bytes with no checksum and no router fragmentation; extension headers carry options; the minimum link MTU is 1280 bytes.
- Neighbor Discovery in ICMPv6 replaces ARP and also performs router discovery and duplicate address detection.
- Hosts configure addresses with SLAAC (prefix from a router advertisement plus a self-generated identifier) or with DHCPv6.
- Dual stack, tunnelling and NAT64 are the transition tools; ICMPv6 must never be blocked wholesale; in Java, `Inet6Address` string forms are not canonical.

2.4 ICMP and Network Diagnostics

IP provides no acknowledgement and no error channel. When a router discards a datagram because the TTL expired, or because no route exists, or because the datagram exceeds the outgoing MTU and DF is set, the sender would otherwise never learn the reason. The Internet Control Message Protocol (ICMP, RFC 792) fills this gap. ICMP messages travel inside IP datagrams with protocol number 1, but ICMP is considered part of the network layer: it reports about IP, not about applications. The tools ping and traceroute, and the Path MTU Discovery mechanism that TCP depends on, are built entirely on ICMP.

2.4.1 Concept and mechanism

Message format and types

Every ICMPv4 message begins with a type (8 bits), a code (8 bits) and a checksum (16 bits, computed over the whole ICMP message with the same algorithm as the IP header). The remaining four bytes depend on the type: the echo messages carry an identifier and a sequence number, and several error messages carry unused bytes or a next-hop MTU.

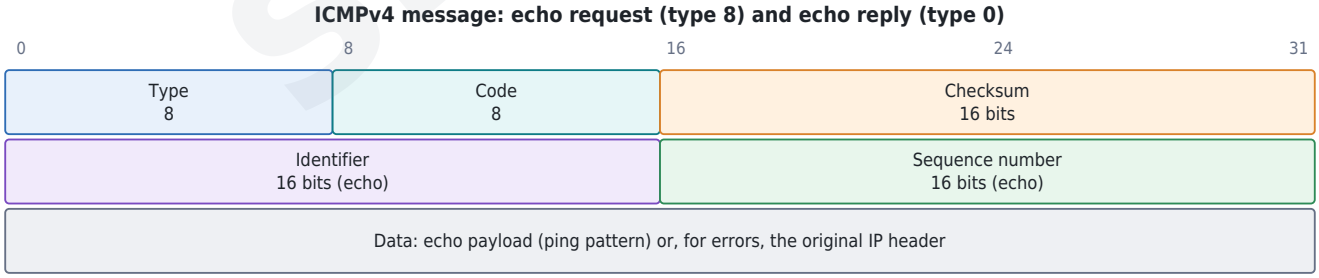
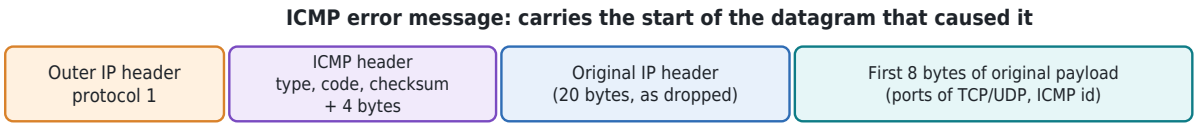


Figure 2.13. An ICMP echo message consists of a type, a code, a checksum, an identifier, a sequence number and optional data; the identifier and sequence number let the sender match each reply to its request.

Error messages (destination unreachable, time exceeded, parameter problem) quote the IP header and the first 8 bytes of the datagram that caused the error. These 8 bytes contain the source and destination ports of TCP and UDP, which lets the sending host associate the error with a specific socket. Java reports the result as an exception: a `ConnectException` ("No route to host", "Network is unreachable") or a `PortUnreachableException` on a connected `DatagramSocket`.



The embedded part lets the sender match the error to the socket that sent the datagram.

Type	Code	Meaning
0	0	Echo reply
3	0 / 1 / 3 / 4	Destination unreachable: net / host / port / fragmentation needed (DF set)
8	0	Echo request
11	0	Time exceeded: TTL reached zero in transit
11	1	Time exceeded: fragment reassembly timeout
12	0	Parameter problem

Figure 2.14. An ICMP error message embeds the original IP header and the first eight payload bytes; the table lists the type and code values that occur most often.

Three rules limit the damage that ICMP itself can cause. An ICMP error is never generated in response to another ICMP error. Errors are generated only for the first fragment of a fragmented datagram. Routers rate-limit the messages they originate. The last rule explains why traceroute output can show gaps even though the path works.

Type	Code	Meaning and typical cause
0	0	Echo reply: answer to ping.
3	0	Network unreachable: the router has no route to the destination network.
3	1	Host unreachable: ARP for the destination failed on the last hop.
3	3	Port unreachable: the destination host has no UDP socket on that port (TCP uses RST instead).
3	4	Fragmentation needed and DF set: carries the next-hop MTU; basis of Path MTU Discovery.
3	13	Communication administratively prohibited: a firewall rejected the packet with feedback.
8	0	Echo request.
11	0	TTL exceeded in transit: basis of traceroute.

The ICMPv4 types needed for diagnostics.

Ping

Ping sends ICMP echo requests and reports the echo replies. It measures reachability at the network layer and the round-trip time (RTT) of a minimal packet. The default Linux request carries 56 data bytes: 56 + 8 bytes of ICMP header + 20 bytes of IP header = 84 bytes on the wire (excluding the link-layer frame). The TTL printed in each reply is the TTL of the reply datagram when it arrives. Because common initial values are 64, 128 and 255, the number of hops can be estimated: a reply with TTL 52 probably started at 64 and crossed 12 routers.

A failed ping proves little. Many hosts and firewalls drop echo requests, and a successful ping proves nothing about TCP or application availability. ICMP is also handled by a different (often slower) path in routers, so its RTT can differ from that of real traffic.

Traceroute

Traceroute discovers the routers on a path by sending probes with increasing TTL. A probe with TTL 1 expires at the first router, which returns ICMP Time Exceeded; the source address of this message identifies the router. A probe with TTL 2 expires at the second router, and so on. The Unix traceroute sends UDP datagrams to an unlikely destination port (33434 and upward), so that the destination answers with ICMP Port Unreachable (type 3, code 3), which signals that the destination has been reached. Windows `tracert` sends ICMP echo requests, and the destination answers with an echo reply. Variants use TCP SYN segments to port 443 to pass firewalls that block UDP and ICMP.

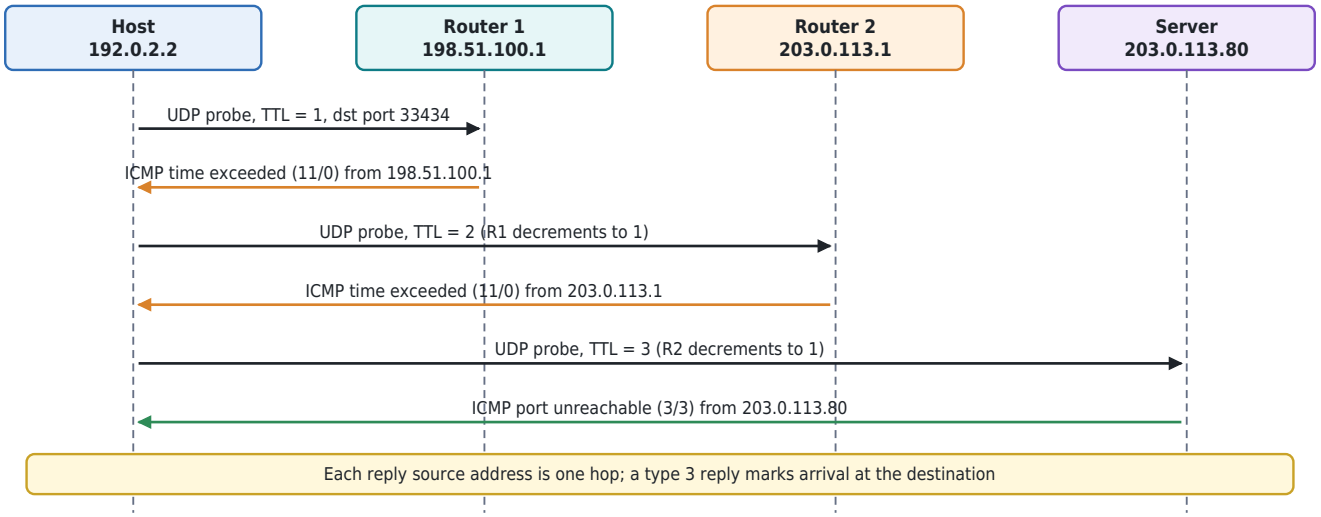


Figure 2.15. Three probes with TTL 1, 2 and 3 are answered by Time Exceeded messages from router 1 and router 2 and by Port Unreachable from the destination; the source address of each reply names one hop of the path.

Limits of the technique: the path found by one probe is not guaranteed for the next one, because load balancing can send probes with different port numbers over different links; the reverse path of the ICMP reply may differ from the forward path (asymmetric routing); and a router that does not generate ICMP errors appears as `* * *`.

Path MTU Discovery

A sender that does not know the smallest MTU on the path (the **path MTU**) risks fragmentation or loss. Path MTU Discovery (RFC 1191) sets the DF flag on every datagram. A router that cannot forward a datagram because of the next link's MTU drops it and returns ICMP type 3 code 4 containing the MTU of that link. The sender lowers its estimate of the path MTU and retransmits smaller packets. TCP translates the path MTU into a maximum segment size: MSS = path MTU - 20 (IP) - 20 (TCP) = 1360 for a path MTU of 1400.

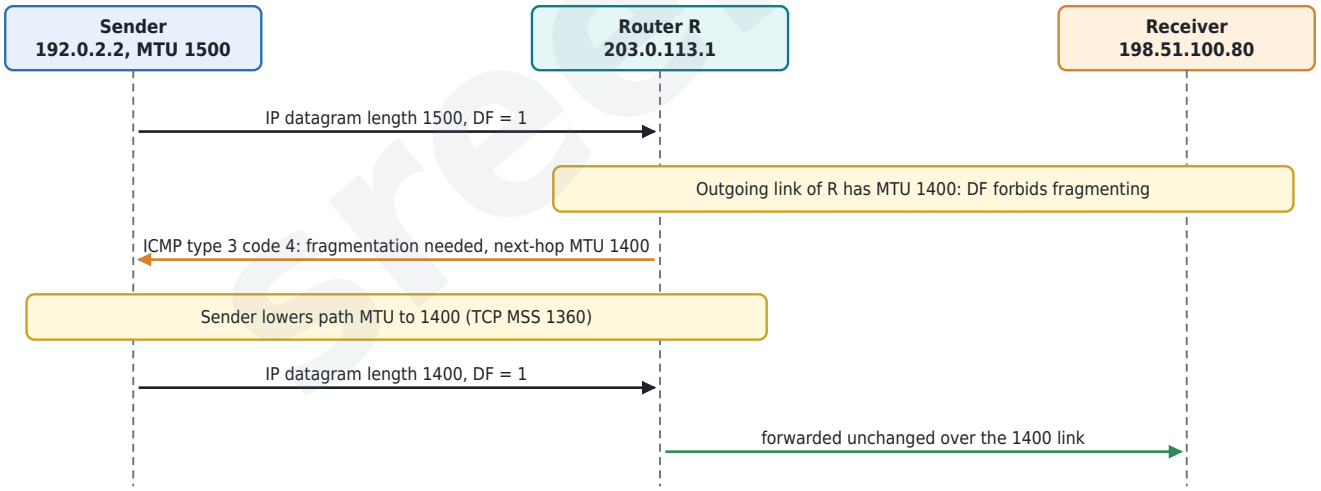


Figure 2.16. A DF-marked datagram of 1500 bytes is rejected by a router whose outgoing link has an MTU of 1400; the ICMP reply carries that MTU and the sender continues with 1400-byte datagrams.

The mechanism fails when ICMP messages are filtered: the sender receives no error, packets above the path MTU disappear without trace, and the connection stalls after the handshake (small packets pass, large ones do not). This condition is called a **PMTU black hole**. Remedies are to permit ICMP type 3 code 4 (and ICMPv6 Packet Too Big), to clamp the TCP MSS on the router at the tunnel (`iptables ... --clamp-mss-to-pmtu`), and to enable Packetization Layer Path MTU Discovery (RFC 4821), which probes using TCP itself.

2.4.2 Worked trace

Traceroute with TTL arithmetic

Host 192.0.2.2 traces the path to 203.0.113.80. The path consists of router 1 (198.51.100.1), router 2 (203.0.113.1) and the destination.

Probe	IP header of probe (TTL)	Event	Reply received
1	UDP to 203.0.113.80:33434, TTL 1	Router 1 decrements TTL to 0 and discards	ICMP 11/0 from 198.51.100.1
2	UDP to 203.0.113.80:33435, TTL 2	Router 1 forwards with TTL 1; router 2 decrements to 0	ICMP 11/0 from 203.0.113.1
3	UDP to 203.0.113.80:33436, TTL 3	Router 2 forwards with TTL 1; destination accepts, finds no socket	ICMP 3/3 from 203.0.113.80

Traceroute probes and replies.

Each hop is probed three times by default, and the three RTTs in the output are the delays to that hop. The round-trip time to hop n includes the time to generate the ICMP message, which can be much slower than forwarding; a single slow hop with normal later hops is not evidence of a problem.

Path MTU Discovery with numbers

A TCP connection between 192.0.2.2 (MTU 1500, MSS 1460) and 198.51.100.80 crosses a tunnel link with MTU 1400. The sender transmits a segment with 1460 data bytes in a 1500-byte datagram with DF set. Router R cannot forward it and returns ICMP type 3 code 4 with next-hop MTU 1400. The sender sets the path MTU to 1400, reduces the effective MSS to 1360 and retransmits the data in two segments of at most 1360 bytes (1400-byte datagrams). From now on the sender caches the value for the destination (Linux keeps it in the route cache for about 10 minutes before probing again).

2.4.3 Hands-on

Raw sockets provide direct access to ICMP. The Python program below sends two echo requests to the loopback address and parses the replies, including the IP header of the reply (the TTL is byte 8). It requires administrative privileges for the raw socket and was run as root on the build machine.

mini_ping.py — PYTHON

```python
import socket, struct, time, os

def checksum(data):
    if len(data) % 2:
        data += b"\0"
    s = sum(struct.unpack("!%dH" % (len(data) // 2), data))
    while s >> 16:
        s = (s & 0xFFFF) + (s >> 16)
    return ~s & 0xFFFF

ident, payload = os.getpid() & 0xFFFF, b"abcdefgh"
for seq in (1, 2):
    hdr = struct.pack("!BBHHH", 8, 0, 0, ident, seq)
    pkt = struct.pack("!BBHHH", 8, 0, checksum(hdr + payload), ident, seq) + payload
    s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
    s.settimeout(2)
    t0 = time.perf_counter()
    s.sendto(pkt, ("127.0.0.1", 0))
    while True:
        data, addr = s.recvfrom(1024)
        ihl = (data[0] & 0x0F) * 4
        typ, code, cs, rid, rseq = struct.unpack("!BBHHH", data[ihl:ihl + 8])
        if typ == 0 and rid == ident:
            break
    rtt = (time.perf_counter() - t0) * 1000
    print("reply from %s: type=%d code=%d id=%d seq=%d ttl=%d bytes=%d time=%.3f ms"
          % (addr[0], typ, code, rid, rseq, data[8], len(data) - ihl, rtt))
```

Output (the RTT values vary; the other fields are deterministic apart from the identifier) TERMINAL

```
$ sudo python3 mini_ping.py
reply from 127.0.0.1: type=0 code=0 id=6977 seq=1 ttl=64 bytes=16 time=0.065 ms
reply from 127.0.0.1: type=0 code=0 id=6977 seq=2 ttl=64 bytes=16 time=0.016 ms
```

What to notice: the reply has type 0 and code 0, the identifier and sequence number equal those of the request, the reply carries the same 8 data bytes plus the 8-byte ICMP header (16 bytes), and the TTL is 64 because the loopback interface does not decrement it.

Java cannot send arbitrary ICMP packets. `InetAddress.isReachable(timeout)` tries an ICMP echo request when the process has the privilege to do so and otherwise falls back to a TCP connection attempt to port 7 (echo); it is unreliable for health checks.

Reach.java JAVA

```java
import java.net.InetAddress;

public class Reach {
    public static void main(String[] args) throws Exception {
        InetAddress a = InetAddress.getByName("127.0.0.1");
        System.out.println("isReachable(127.0.0.1) = " + a.isReachable(1000));
    }
}
```

Output (deterministic) TERMINAL

```
$ java Reach.java
isReachable(127.0.0.1) = true
```

Typical output of ping and traceroute

The public network is not reachable from the build machine, so the following sessions are typical outputs written with documentation addresses; they were not captured.

Typical output of ping (not captured) TERMINAL

```
$ ping -c 3 203.0.113.80
PING 203.0.113.80 (203.0.113.80) 56(84) bytes of data.
64 bytes from 203.0.113.80: icmp_seq=1 ttl=62 time=14.3 ms
64 bytes from 203.0.113.80: icmp_seq=2 ttl=62 time=13.9 ms
64 bytes from 203.0.113.80: icmp_seq=3 ttl=62 time=14.1 ms

--- 203.0.113.80 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 13.9/14.1/14.3/0.2 ms
```

Typical output of traceroute (not captured) TERMINAL

```
$ traceroute -n 203.0.113.80
traceroute to 203.0.113.80 (203.0.113.80), 30 hops max, 60 byte packets
 1  198.51.100.1  1.1 ms  1.0 ms  1.2 ms
 2  203.0.113.1  9.4 ms  9.2 ms  9.6 ms
 3  203.0.113.80  14.1 ms  14.0 ms  14.3 ms
```

TTL 62 in the ping output means an initial TTL of 64 and two routers on the path, which matches the three-line traceroute (two routers and the destination).

2.4.4 Failure modes

Symptom	Cause	Diagnosis and fix
Ping fails but the web server works	Firewall drops ICMP echo	Test with `curl` or a TCP connection; do not use ping as a health check
`* * *` for one or more hops, later hops respond	Router does not answer or rate-limits ICMP	Ignore if later hops respond; the path is intact
`* * *` from some hop to the end	Firewall blocks the probes (UDP or ICMP)	Repeat with TCP probes to the service port
Connection establishes but large transfers stall	PMTU black hole	Capture the traffic; look for retransmitted full-size segments; permit ICMP 3/4 or clamp MSS
`Destination Net Unreachable` from a router	No route in the router's table	Inspect the routing table at the reporting router
`Time to live exceeded` repeated between two routers	Routing loop	Inspect the routes of both routers for opposite next hops

ICMP-related faults.

GOTCHA

ICMP error messages concern the network layer, so a Java application sees them only indirectly: as `ConnectException: No route to host`, as `SocketTimeoutException` when the error is filtered, or as `PortUnreachableException` on a connected UDP socket. A timeout therefore does not distinguish between packet loss, a firewall drop and a dead host; the exception text and a packet capture are needed.

2.4.5 Questions to expect

1. How does traceroute work?

It sends probes with TTL 1, 2, 3 and so on. Each router that decrements the TTL to zero returns ICMP Time Exceeded, which reveals its address. When the probe reaches the destination, an ICMP Port Unreachable (UDP probes) or an echo reply (ICMP probes) ends the trace.

2. What does ping measure and what does it not measure?

It measures network-layer reachability and the RTT of a small ICMP packet. It does not show whether TCP ports are open, whether the application works, or the delay of large packets, and many networks filter or deprioritise ICMP.

3. What is Path MTU Discovery?

A technique to find the smallest MTU along a path without fragmentation. The sender sets DF; a router that cannot forward the packet returns ICMP type 3 code 4 with the next-hop MTU; the sender then sends smaller packets.

4. What is a PMTU black hole?

A condition in which ICMP fragmentation-needed messages are lost, so that packets larger than the path MTU are dropped silently. Small packets (handshake) pass, large ones never arrive, and the connection hangs.

5. Why do TCP connections use an MSS of 1460 on Ethernet?

The MTU of 1500 minus the 20-byte IP header and 20-byte TCP header leaves 1460 bytes of data. Options such as timestamps reduce the usable payload further, by 12 bytes in the usual case.

6. Why does the TTL in a ping reply help estimate the distance?

Operating systems start with well-known initial values (64, 128, 255), each router subtracts one, so initial value minus received value approximates the number of routers on the reverse path.

7. Which ICMP message tells a UDP sender that nobody listens at the destination port?

Destination unreachable, code 3 (port unreachable). TCP instead receives a segment with the RST flag.

8. Why can traceroute show different paths on consecutive runs?

Routers load-balance across equal-cost links, typically by hashing the flow identifiers; probes with different port numbers can take different links. Routing changes also alter the path.

9. Is it safe to block all ICMP at the firewall?

No. Blocking ICMP type 3 code 4 breaks Path MTU Discovery, and for IPv6 the ICMPv6 messages are required for address resolution and autoconfiguration. Only echo requests from untrusted networks should be restricted, if at all.

KEY POINTS

- ICMP reports network-layer events; it is carried in IP (protocol 1) and its errors quote the offending IP header plus 8 bytes.
- Ping uses echo request and reply; traceroute uses increasing TTL and the Time Exceeded and Port Unreachable replies.
- Path MTU Discovery relies on DF and ICMP type 3 code 4; blocked ICMP creates black holes that stall large transfers.
- TCP MSS equals path MTU minus 40 bytes of IP and TCP headers.
- A timeout in Java hides which of loss, filtering or a dead host occurred; combine exception text with captures.

2.5 Routing

A packet from a client in one organisation to a server in another passes through many routers. Each router knows only its own neighbours and a table that maps destination prefixes to next hops. Two distinct activities make this work. **Forwarding** is the per-packet action of looking up the destination in the table and sending the packet out of the chosen interface. **Routing** is the process, run by routing protocols or by an administrator, that computes the table. Forwarding happens in microseconds at line rate; routing happens in seconds and reacts to topology changes. Understanding the table, the lookup rule and the main protocol families is necessary for diagnosing reachability problems, for configuring cloud route tables and for explaining how the Internet scales.

2.5.1 Concept and mechanism

Control plane and data plane

A router separates the two activities architecturally. The **control plane** runs routing protocols, collects candidate routes in the routing information base (RIB), selects the best route for each prefix and installs it in the forwarding information base (FIB). The **data plane** handles each arriving packet: it validates the header, decrements the TTL, looks up the FIB, switches the packet to the output port and queues it there for transmission.

Control plane builds the forwarding table; data plane applies it to every packet

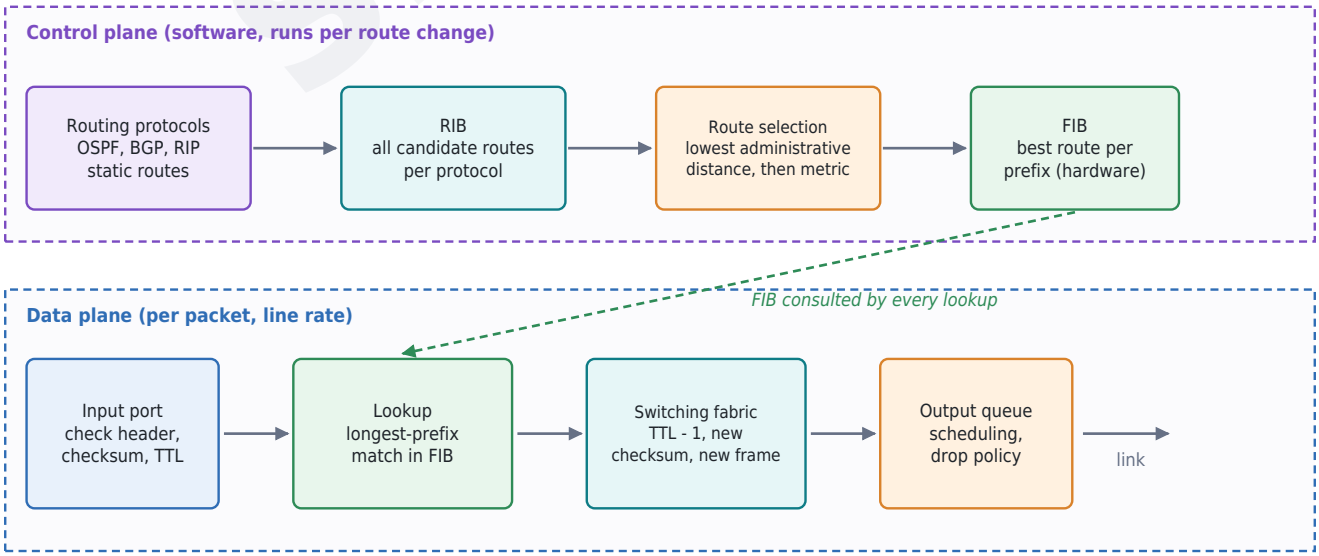


Figure 2.17. The control plane builds the FIB from routing protocols and static routes; the data plane performs one FIB lookup per packet and rewrites the TTL, checksum and link-layer header before queuing on the output port.

The routing table and longest-prefix match

Each entry contains a destination prefix, a next-hop address (or "directly connected"), an outgoing interface and a metric. When a packet arrives, the router compares the destination with the prefix of every entry and considers all entries whose prefix matches. If several match, the entry with the **longest prefix** is chosen. This rule lets a specific route (a /25) override a general one (a /16), and it makes the default route `0.0.0.0/0`, which matches everything with length zero, a last resort. The table is a set, so the order of entries has no influence.

Longest-prefix match: all matching entries qualify, the longest prefix wins

Destination prefix	Length	Next hop	Interface	10.1.2.200	10.1.2.77
0.0.0.0/0	0	10.0.0.1	eth0	match	match
10.1.0.0/16	16	10.0.0.2	eth0	match	match
10.1.2.0/24	24	10.0.0.3	eth0	match	**match, WINS**
10.1.2.128/25	25	10.0.0.4	eth0	**match, WINS**	no
192.168.1.0/24	24	direct	eth1	no	no

10.1.2.200 = 00001010.00000001.00000010.11001000: the /25 prefix 10.1.2.128 requires the next bit (128 place) = 1.

10.1.2.77 = 00001010.00000001.00000010.01001101: the 128 bit is 0, so the /25 entry fails and the /24 is the longest match.

A packet to 8.8.8.8 matches only 0.0.0.0/0 and is sent to the default gateway 10.0.0.1.

Order of entries is irrelevant: the decision depends on prefix length, not position.

Figure 2.18. Two destinations traced through the same five-entry table: 10.1.2.200 matches four entries and the /25 wins; 10.1.2.77 fails the /25 and falls back to the /24; the default route matches every address with the shortest possible prefix.

A **host** has a small routing table of the same form: one connected route per interface and a default route pointing at the **default gateway**, which is the router on the local subnet that receives everything not on-link. The host never needs to know the path beyond the gateway.

Static and dynamic routing

Static routes are entered by an administrator. They are predictable and need no protocol overhead, but they do not react to failures. They are appropriate for stub networks, default routes and cloud route tables. Dynamic routing protocols exchange reachability information and recompute routes automatically when links fail. They are classified by scope and by algorithm.

Family	Protocols	Algorithm	Scope and notes
Distance vector	RIP	Bellman-Ford	Interior; each router tells neighbours its distances; simple, slow to converge
Link state	OSPF, IS-IS	Dijkstra	Interior; each router floods its links and computes shortest paths over the full topology
Path vector	BGP	Policy-based path selection	Exterior; between autonomous systems; carries the list of ASes traversed

Routing protocol families.

An **autonomous system** (AS) is a set of networks under a single administrative policy, identified by an AS number. Protocols that run inside an AS are interior gateway protocols (IGP); BGP is the exterior gateway protocol that joins the ASes into the Internet. When several protocols offer a route for the same prefix, the router prefers the source by an **administrative distance**, a vendor convention (typical values on Cisco equipment: connected 0, static 1, eBGP 20, OSPF 110, RIP 120); among routes from the same protocol the metric decides.

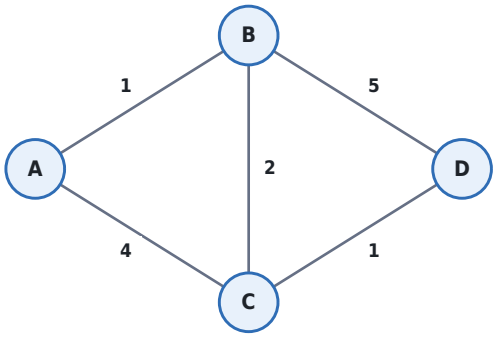
Distance vector: RIP and Bellman-Ford

In a distance vector protocol each router keeps a vector of the best known distance to every destination and sends it periodically to its neighbours. The update rule is the Bellman-Ford equation. Let c(x, v) be the cost of the link from x to neighbour v and d_v(y) the distance that v advertises to destination y. Then

d_x(y) = min over all neighbours v of { c(x, v) + d_v(y) }

and the neighbour that achieves the minimum becomes the next hop. RIP (RFC 2453) uses hop count as the metric, sends updates every 30 seconds, treats 16 as infinity (so networks may be at most 15 hops wide) and runs over UDP port 520. Its weaknesses follow from the algorithm: routers learn about a failure only indirectly, and bad news propagates slowly. The **count-to-infinity** problem arises when a route disappears and routers keep advertising the stale route to each other, increasing the metric step by step. Split horizon (never advertise a route back out of the interface from which it was learned), poison reverse (advertise it with metric infinity) and a small infinity value limit the problem but do not eliminate it.

Distance vector (Bellman-Ford): router A learns D through two rounds of exchanges



Link costs: A-B 1, A-C 4, B-C 2, B-D 5, C-D 1

Distance vector held by A (cost via next hop)

Round	to B	to C	to D
0	1 via B	4 via C	inf
1	1 via B	3 via B	5 via C
2	1 via B	3 via B	4 via B

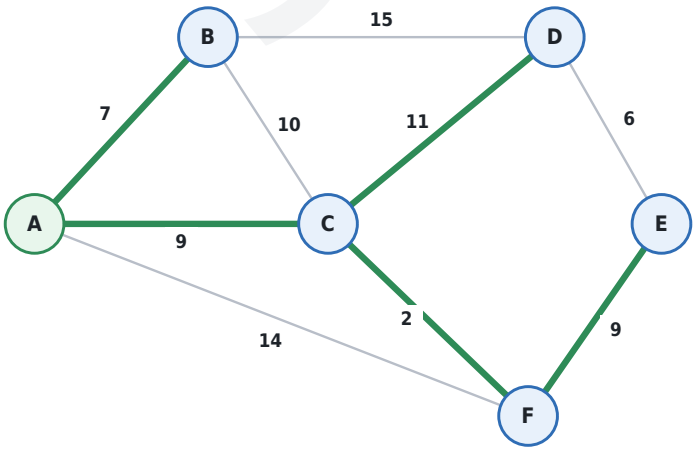
Round 1: B advertises C = 2, so A reaches C at 1 + 2 = 3.
D = min(4 + 1 via C, 1 + 5 via B) = 5 via C.
Round 2: B now advertises D = 3, so A reaches D at 1 + 3 = 4 via B.
Round 3: no vector changes: converged.

Figure 2.19. Router A converges on the shortest costs to B, C and D in two exchange rounds; the cost to D first improves from infinity to 5 through C and then to 4 through B, as neighbours advertise their own new vectors.

Link state: OSPF and Dijkstra

In a link-state protocol each router measures the state and cost of its own links, describes them in link-state advertisements (LSAs) and **floods** these advertisements to every router in the area. All routers therefore hold the same link-state database, a complete map of the topology, and each computes the shortest-path tree rooted at itself with Dijkstra's algorithm. OSPF (version 2, RFC 2328) runs directly over IP (protocol 89), uses multicast addresses 224.0.0.5 (all OSPF routers) and 224.0.0.6 (designated routers), establishes neighbour relationships with Hello packets, and reduces flooding load by dividing a large network into areas around a backbone area 0. The cost of a link is derived from its bandwidth (for example reference bandwidth divided by interface bandwidth). Convergence is fast because a failure is flooded within milliseconds to seconds and every router recomputes locally.

Link state: Dijkstra from router A over the topology database (thick edges: shortest-path tree)



Result at A

Dest	Cost	Next hop
B	7	B
C	9	C
F	11	C
D	20	C
E	20	C

Settling order:
A 0, B 7, C 9, F 11,
D 20, E 20

Every router runs the same algorithm on the same database, so the trees are consistent.

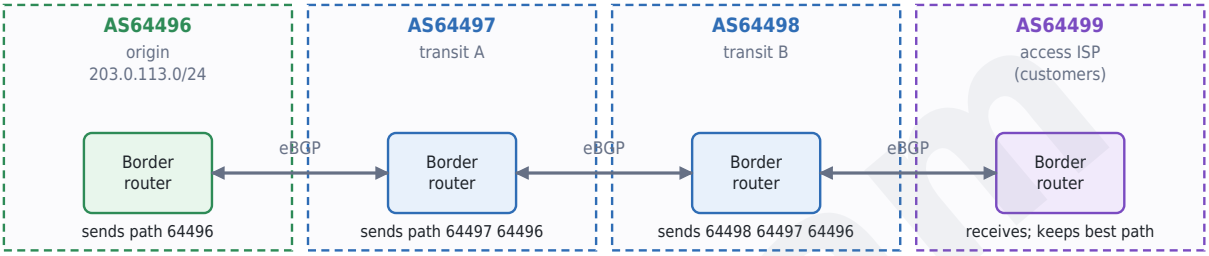
Figure 2.20. A six-router topology with link costs; the thick edges form the shortest-path tree computed from router A, and the table gives cost and next hop for every destination.

Dijkstra's algorithm maintains a set of routers whose shortest distance is final. It repeatedly selects the router with the smallest tentative distance outside the set, adds it to the set, and relaxes the links of the new member: if the distance to the new member plus the link cost beats a neighbour's tentative distance, the neighbour's distance and predecessor are updated. The first router on the tree path from the root to a destination becomes the next hop of the routing table entry.

Border Gateway Protocol

BGP (version 4, RFC 4271) connects the tens of thousands of autonomous systems of the Internet. Its design goal is policy control, not shortest path. Peers form BGP sessions over TCP port 179. A session between routers in different ASes is **eBGP**; a session between routers in the same AS is **iBGP** and distributes the externally learned routes to all border routers. An UPDATE message announces a prefix together with **path attributes**, the most important of which is the **AS_PATH**: the ordered list of ASes the announcement has passed through. A router prepends its own AS number when it passes the route to an eBGP neighbour and rejects any announcement whose AS_PATH already contains its own number, which prevents loops.

BGP between autonomous systems: the AS_PATH grows as the prefix 203.0.113.0/24 is propagated



1. AS64496 originates 203.0.113.0/24: AS_PATH = 64496
2. AS64497 receives it, prepends itself, sends AS_PATH = 64497 64496
3. AS64498 prepends itself: AS_PATH = 64498 64497 64496
4. AS64499 receives it; a router that sees its own ASN in a path rejects it (loop prevention)

Inside an AS an IGP (OSPF) carries internal routes; iBGP distributes the external routes between its border routers.

Route choice: local preference, then shortest AS_PATH, then origin type, MED and lowest router ID.

Figure 2.21. A prefix originated in AS64496 acquires one AS number per eBGP hop; the receiving AS can compare AS_PATH lengths, but policy attributes such as local preference are evaluated first.

For each prefix a router may know several routes and selects one by a fixed sequence of criteria. The usual order is: highest local preference (an operator policy value), shortest AS_PATH, lowest origin type, lowest MED (a hint from the neighbouring AS), eBGP over iBGP, lowest IGP cost to the next hop, and finally the lowest router identifier. Commercial relationships (customer, peer, provider) are expressed through local preference and export filters, which is why traffic does not always follow the shortest AS path. BGP has no built-in verification that an AS is entitled to announce a prefix, so misconfigurations and hijacks can redirect traffic; Resource Public Key Infrastructure (RPKI) route origin validation is the deployed countermeasure.

Convergence

Convergence is the state in which all routers have consistent tables after a change. The time to reach it varies by protocol: OSPF in the order of seconds or less with tuned timers; RIP in minutes in the worst case (a route times out after 180 seconds by default); BGP in tens of seconds to minutes because of per-neighbour advertisement timers and the volume of the global table. During convergence packets can be dropped (a router has no route) or can loop (two routers point to each other), and TTL is the mechanism that ends loops.

2.5.2 Worked trace

Longest-prefix match

The router of the diagram holds five routes. The destination 10.1.2.200 in binary is 00001010.00000001.00000010.11001000. The route 10.1.2.128/25 requires the first 25 bits to equal 00001010.00000001.00000010.1; the destination's 25th bit (the leading bit of the last octet, 200 = 11001000) is 1, so the route matches. The routes /0, /16 and /24 also match. The longest is /25, therefore the packet goes to 10.0.0.4 on eth0. For 10.1.2.77 (last octet 01001101) the 25th bit is 0, the /25 route does not match, and the /24 route to 10.0.0.3 is the longest match.

Distance vector rounds

The topology of the diagram has links A-B 1, A-C 4, B-C 2, B-D 5 and C-D 1. At round 0 every router knows only its direct neighbours. In each round every router recomputes its vector from the vectors its neighbours held at the end of the previous round.

Round	Router A	Router B	Router D
0	`B 1 (B), C 4 (C), D inf`	`A 1 (A), C 2 (C), D 5 (D)`	`A inf, B 5 (B), C 1 (C)`
1	`B 1, C 3 (B), D 5 (C)`	`A 1, C 2, D 3 (C)`	`A 5 (C), B 3 (C), C 1`
2	`B 1, C 3 (B), D 4 (B)`	`A 1, C 2, D 3 (C)`	`A 4 (C), B 3 (C), C 1`
3	`unchanged`	`unchanged`	`unchanged`

Vectors of all routers (cost to A, B, C, D with next hop) after each round.

The entry A to D shows the process. In round 1, A computes min(4 + 1 via C, 1 + 5 via B) = 5 via C, because B still advertises the old value 5 for D. B in the meantime improved its own distance to D to 3 (via C, 2 + 1). In round 2 A sees d_B(D) = 3 and computes 1 + 3 = 4 via B, which is better. Round 3 changes nothing, so the network has converged. The number of rounds needed equals the length of the longest shortest path in hops.

Dijkstra from router A

The topology has costs A-B 7, A-C 9, A-F 14, B-C 10, B-D 15, C-D 11, C-F 2, D-E 6 and E-F 9.

Step	Added	Tentative distances (B, C, D, E, F)	Remark
0	`A (0)`	B 7, C 9, D inf, E inf, F 14	links of A relaxed
1	`B (7)`	C 9, D 22, F 14	D reached via B; C via B would be 17, no change
2	`C (9)`	D 20, F 11	F improves from 14 to 11, D from 22 to 20 through C
3	`F (11)`	D 20, E 20	E reached via F (11 + 9)
4	`D (20)`	E 20	E via D would be 26, no change
5	`E (20)`	-	all routers final

Dijkstra from A. Tentative distances are shown after each router is added to the final set.

The predecessor chain gives the tree: B and C hang off A, F and D off C, and E off F. The routing table of A therefore holds B via B (7), C via C (9), F via C (11), D via C (20) and E via C (20). The direct link A-F with cost 14 is not used because the path through C costs 11.

2.5.3 Hands-on

The program stores the five routes of the example, matches each destination against every route and selects the longest prefix. The `matches` method applies the same mask test that hardware performs.

RouteLookup.java JAVA

```java
import java.net.InetAddress;
import java.net.UnknownHostException;
import java.util.List;

public class RouteLookup {
    record Route(String cidr, String nextHop, String iface) {
        long network() { return toLong(cidr.split("/")[0]); }
        int length() { return Integer.parseInt(cidr.split("/")[1]); }
        boolean matches(long dst) {
            long mask = length() == 0 ? 0 : (0xFFFFFFFFL << (32 - length())) & 0xFFFFFFFFL;
```

RouteLookup.java (continued) JAVA

```
            return (dst & mask) == network();
        }
    }

    static long toLong(String ip) {
        try {
            byte[] b = InetAddress.getByName(ip).getAddress();
            return ((b[0] & 0xFFL) << 24) | ((b[1] & 0xFF) << 16) | ((b[2] & 0xFF) << 8)
                    | (b[3] & 0xFF);
        } catch (UnknownHostException e) {
            throw new IllegalArgumentException(e);
        }
    }

    public static void main(String[] args) {
        List<Route> table = List.of(
                new Route("0.0.0.0/0", "10.0.0.1", "eth0"),
                new Route("10.1.0.0/16", "10.0.0.2", "eth0"),
                new Route("10.1.2.0/24", "10.0.0.3", "eth0"),
                new Route("10.1.2.128/25", "10.0.0.4", "eth0"),
                new Route("192.168.1.0/24", "direct", "eth1"));
        String[] targets = {"10.1.2.200", "10.1.2.77", "10.1.9.9", "192.168.1.50", "8.8.8.8"};
        for (String dst : targets) {
            long d = toLong(dst);
            Route best = null;
            StringBuilder matched = new StringBuilder();
            for (Route r : table) {
                if (r.matches(d)) {
                    matched.append(r.cidr()).append(' ');
                    if (best == null || r.length() > best.length()) {
                        best = r;
                    }
                }
            }
            System.out.printf("%-12s [%s]%n   -> %s via %s (%s)%n", dst, matched.toString().trim(),
                    best.cidr(), best.nextHop(), best.iface());
        }
    }
}
```

Output (deterministic) TERMINAL

```
$ java RouteLookup.java
10.1.2.200   [0.0.0.0/0 10.1.0.0/16 10.1.2.0/24 10.1.2.128/25]
   -> 10.1.2.128/25 via 10.0.0.4 (eth0)
10.1.2.77    [0.0.0.0/0 10.1.0.0/16 10.1.2.0/24]
   -> 10.1.2.0/24 via 10.0.0.3 (eth0)
10.1.9.9     [0.0.0.0/0 10.1.0.0/16]
   -> 10.1.0.0/16 via 10.0.0.2 (eth0)
192.168.1.50 [0.0.0.0/0 192.168.1.0/24]
   -> 192.168.1.0/24 via direct (eth1)
8.8.8.8      [0.0.0.0/0]
   -> 0.0.0.0/0 via 10.0.0.1 (eth0)
```

Inspecting routing tables (typical output; the build machine has no `ip` command)

On Linux the commands `ip route` and `ip route get` show the table and the decision for one destination. The listing below is typical output for a host with documentation addresses.

Typical output of ip route (not captured) TERMINAL

```
$ ip route
default via 192.168.1.1 dev eth0 proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.10 metric 100
$ ip route get 203.0.113.25
203.0.113.25 via 192.168.1.1 dev eth0 src 192.168.1.10 uid 1000
    cache
$ ip route get 192.168.1.50
192.168.1.50 dev eth0 src 192.168.1.10 uid 1000
    cache
```

The first destination is off-link and is sent to the gateway 192.168.1.1; the second is inside 192.168.1.0/24 and is delivered directly (the link-layer address is found with ARP). The same lookup runs in containers: a Docker container

has a default route through the bridge gateway (often 172.17.0.1), which explains why a container cannot reach a service that the host reaches over a VPN unless a route exists.

2.5.4 Failure modes

Symptom	Cause	Diagnosis and fix
`Network is unreachable` (Java `ConnectException`)	No matching route, not even a default route	`ip route get <destination>`; add the default route or fix DHCP
`No route to host`	The destination is on-link but ARP gets no answer, or a router returned ICMP host unreachable	Check ARP table and the last-hop router
Requests succeed, replies never arrive	Return path missing (asymmetric routing, missing route on the far side)	Traceroute in both directions; inspect the route to the client's prefix on the server side
Traffic goes to the wrong network after connecting a VPN	VPN installs a more specific or a default route	Read the routing table after the VPN comes up; use split tunnelling
Packets loop until TTL expires	Inconsistent static routes	Trace the path; inspect the two routers that alternate
Connectivity flaps every few minutes	Unstable link causes repeated reconvergence	Check interface error counters and routing protocol logs
Prefix unreachable from parts of the Internet	BGP announcement withdrawn, filtered or hijacked	Check looking-glass and RPKI data; contact the provider

Routing faults.

REAL WORLD

Asymmetric routing is normal on the Internet: the forward and return paths follow independent policies. It causes problems when a stateful device, such as a firewall or NAT device, sees only one direction and discards the flow. In clouds the same effect appears when a second network interface or a peered network returns traffic along another path.

2.5.5 Questions to expect

1. What is the difference between routing and forwarding?

Forwarding is the per-packet lookup and transmission using the forwarding table. Routing is the computation of that table by protocols or by an administrator. Forwarding is the data plane, routing the control plane.

2. Explain longest-prefix match with an example.

When several routes match the destination, the one with the longest prefix is selected, because it is the most specific. With routes 10.0.0.0/8 and 10.1.0.0/16 a packet to 10.1.5.5 uses the /16 route; a packet to 10.9.9.9 uses the /8 route.

3. What is a default gateway and when does a host use it?

The default gateway is the router on the local subnet to which a host sends every packet whose destination is not on-link. The host determines this by the AND test with its own mask and then resolves the gateway's MAC address.

4. Compare distance vector and link state.

Distance vector routers exchange distance tables with neighbours only and see the network through them; the algorithm is Bellman-Ford, it is simple and converges slowly, and it can count to infinity. Link-state routers flood the topology to all routers and each runs Dijkstra on a complete map; it converges faster and uses more memory and CPU.

5. What is the count-to-infinity problem and how is it mitigated?

After a failure, routers keep advertising stale routes to each other and the metric rises by one per exchange until it reaches the protocol's infinity (16 in RIP). Split horizon, poison reverse, hold-down timers and a small maximum metric limit the effect.

6. How does Dijkstra's algorithm compute the shortest paths?

It keeps a set of routers with final distances, repeatedly adds the router with the smallest tentative distance, and updates the tentative distances of its neighbours. The result is a shortest-path tree rooted at the computing router.

7. What is an autonomous system and why does BGP exist?

An AS is a network or group of networks under one administrative policy with a common routing policy and an AS number. BGP exchanges reachability between ASes and applies business policy; IGPs optimise paths inside one AS and cannot express inter-organisation policy or scale to the whole Internet.

8. How does BGP prevent routing loops?

A router rejects an announcement whose AS_PATH already contains its own AS number, and every AS prepends its number before forwarding the announcement over eBGP.

9. What is convergence and why is it important?

Convergence is the time until all routers hold consistent, loop-free routes after a change. Until then, packets can be lost or can loop, so availability depends on convergence time.

10. A host can ping the gateway but not an Internet address. What should be checked?

The default route and gateway address on the host, the NAT or the route to the Internet on the gateway, the return route from the destination, DNS if names are used, and filtering of ICMP. A traceroute shows the last responding hop.

INTERVIEW ANGLE

Follow-up: a server has two interfaces in different subnets and a single default gateway. Why do replies to clients on the second subnet sometimes fail? The reply is routed by the destination address and the default route, so it can leave through the first interface with a source address of the second subnet. Upstream filters (reverse-path checks) discard it. The remedy is policy routing: a separate table and a rule that selects it by the source address, so that replies leave through the interface on which the request arrived.

KEY POINTS

- Forwarding is a per-packet lookup; routing computes the table. The longest matching prefix wins and the default route is the zero-length prefix.
- Distance vector (RIP) exchanges vectors and uses Bellman-Ford; link state (OSPF) floods topology and runs Dijkstra; BGP exchanges paths between autonomous systems under policy.
- Convergence time determines how long failures cause loss or loops; TTL terminates loops.
- Reachability faults split into missing route, missing return route, filtering and asymmetric paths; `ip route get` and traceroute isolate them.

2.6 NAT and DHCP

Two mechanisms make an ordinary network usable without manual configuration and without a public address for every device. **Network Address Translation** (NAT) lets many hosts with private addresses share one or a few public addresses. The **Dynamic Host Configuration Protocol** (DHCP) assigns an address, a mask, a gateway and DNS servers to a host that has just joined the network. Every home router and every cloud virtual network uses both. They also explain common production symptoms: idle connections that die after a few minutes, services that cannot be reached from outside, and hosts that appear with 169.254.x.x addresses.

2.6.1 Concept and mechanism

Network address translation

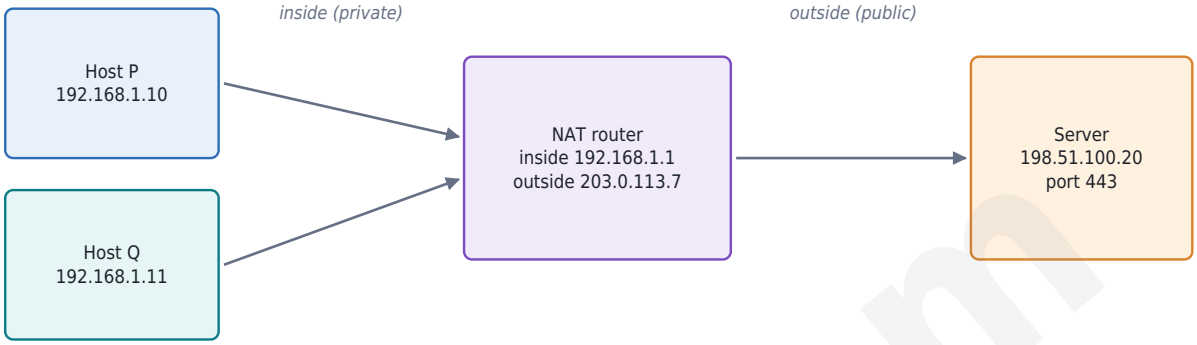
A NAT device sits between a private network and a public network and rewrites addresses in the headers of the packets that pass through it (RFC 3022; behavioural requirements in RFC 4787). Three forms exist.

- **Static NAT** maps one private address to one public address permanently. It is used to expose a server.
- **Dynamic NAT** maps private addresses to a pool of public addresses on demand.

- **Port address translation** (PAT, also NAPT or "NAT overload") maps many private (address, port) pairs to one public address by assigning a distinct public source port to each connection. This is the form used in home routers and in cloud NAT gateways, and it is what the word NAT usually means.

The PAT device maintains a **translation table**. For an outbound packet it looks up the tuple (private address, private port, protocol, destination); if no entry exists it allocates an unused public port and inserts an entry. It then replaces the source address and port and recalculates the IP header checksum and the TCP or UDP checksum, which covers the addresses through the pseudo-header. For an inbound packet it looks up the destination port in the table, restores the private address and port, and forwards the packet. An inbound packet without a matching entry is dropped, which is why NAT behaves like a stateful firewall for unsolicited traffic.

PAT: two inside hosts share one public address; the router rewrites addresses and source ports



Outgoing packets (source:port > destination:port)

P to server, before NAT	`192.168.1.10:51000 > 198.51.100.20:443`
P to server, after NAT	`203.0.113.7:40001 > 198.51.100.20:443`
Q to server, before NAT	`192.168.1.11:51000 > 198.51.100.20:443`
Q to server, after NAT	`203.0.113.7:40002 > 198.51.100.20:443`

Translation table: (inside address, port) <-> (outside address, port); the reply 198.51.100.20:443 > 203.0.113.7:40002 is matched in the table and rewritten to destination 192.168.1.11:51000.

Figure 2.22. Two hosts use the same source port 51000; the NAT router gives their connections the distinct public ports 40001 and 40002 on its single public address 203.0.113.7, so replies from the server can be matched to the correct inside host.

Entries are created by outbound packets and removed after inactivity. The timeout depends on the protocol and the implementation: a UDP mapping may last seconds to minutes, and an established TCP mapping hours or days; Linux connection tracking uses defaults of roughly 30 to 120 seconds for UDP and five days for established TCP, and cloud NAT gateways use their own values (for example 350 seconds for idle TCP connections in AWS NAT gateways). When a mapping expires and the peer later sends data, the NAT device cannot find an entry; it either drops the packet or answers with a reset. This is the most common NAT-related fault in Java services.

Port forwarding and NAT traversal

Inbound connections to a host behind NAT need a mapping that exists before any outbound traffic: **port forwarding** (destination NAT) statically maps public address and port, for example 203.0.113.7:8080, to 192.168.1.10:8080. Without it, a peer on the Internet cannot initiate a connection. Peer-to-peer applications (video calls, games) overcome this with **NAT traversal**: STUN lets a host learn its public address and port as seen from outside, TURN relays traffic through a public server when direct connection fails, and ICE (RFC 8445) combines the techniques and selects the best path. Protocols that embed addresses in their payload, such as FTP active mode and SIP, need an application-level gateway (ALG) in the NAT device to rewrite those addresses.

Consequences of NAT

- End-to-end addressing is lost. A server sees the public address of the NAT device, not the client; all clients behind one NAT appear as one source, which affects per-address rate limiting and logging.
- The NAT device holds per-connection state. It is a point of failure and it limits scale: each connection to one destination address and port consumes a public port, so a single public address supports at most about 64,000 simultaneous connections to the same destination (port exhaustion).
- Idle connections are silently removed; keep-alive traffic must be more frequent than the shortest timeout on the path.

- Some protocols fail or need workarounds: IPsec AH, active FTP, and anything that embeds addresses.
- **Carrier-grade NAT** (CGNAT) places a second translation in the provider network, using the 100.64.0.0/10 range between the customer's router and the provider; the customer then cannot receive inbound connections.
- **Hairpin NAT** (loopback NAT) is needed when an inside host addresses another inside host by its public address.

DHCP

A new host has no address, so it cannot address a server and it does not know the server address. DHCP (RFC 2131, options in RFC 2132) solves this by using broadcast: the client sends from `0.0.0.0` port 68 to `255.255.255.255` port 67 (UDP). The exchange has four messages, remembered as DORA.

1. **Discover**: the client broadcasts a request to locate servers. It identifies itself by its MAC address (`chaddr`) and a random transaction identifier (`xid`).
2. **Offer**: a server proposes an address in the field `yiaddr` together with a lease time and configuration options. Several servers may answer.
3. **Request**: the client broadcasts its selection, naming the chosen server (option 54) and the requested address (option 50). The broadcast also informs the other servers that their offers are declined.
4. **Acknowledge**: the server confirms the lease. The client may verify with an ARP probe that nobody else uses the address and then configures the interface.

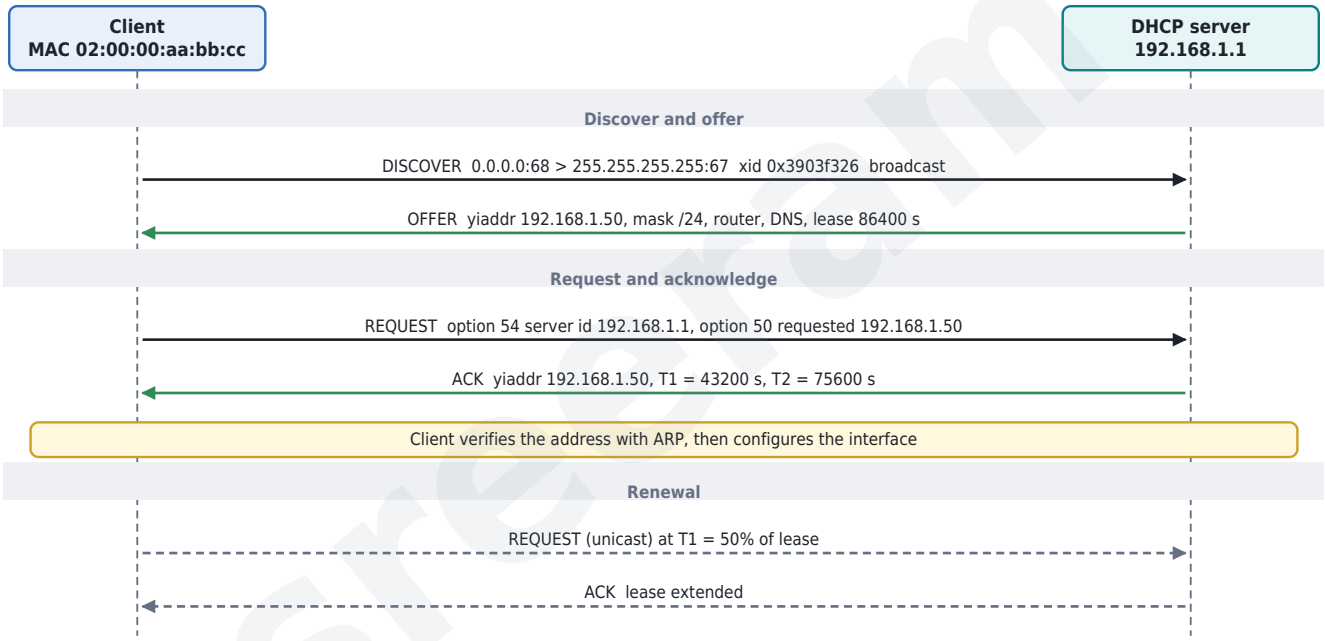


Figure 2.23. The DORA exchange: two broadcasts from the client, two replies from the server, and a later unicast renewal at half of the lease time (T1).

Important options are 1 (subnet mask), 3 (routers, that is, the default gateway), 6 (DNS servers), 51 (lease time), 53 (message type), 54 (server identifier), 58 (T1, renewal time) and 59 (T2, rebinding time). The defaults are T1 = 0.5 and T2 = 0.875 of the lease. At T1 the client asks the same server to extend the lease with a unicast request. If there is no answer, it broadcasts a request at T2 to any server. When the lease expires without renewal, the client must stop using the address. A server that cannot satisfy a request answers with DHCPNAK; a client that finds the address in use sends DHCPDECLINE.

Because routers do not forward broadcasts, one DHCP server for many subnets requires a **relay agent** on each subnet. The relay receives the broadcast, writes its own address on the client's subnet into the field `giaddr`, and forwards the message by unicast to the server, which selects the address pool from `giaddr`.

2.6.2 Worked trace

PAT translation

The router of the diagram has the inside address 192.168.1.1 and the public address 203.0.113.7. The sequence of events for the two hosts is as follows.

1. Host P (192.168.1.10) sends a TCP SYN from port 51000 to 198.51.100.20:443. The router finds no mapping, allocates the public port 40001, stores (192.168.1.10:51000 <-> 203.0.113.7:40001) and forwards the packet with source 203.0.113.7:40001. It updates the IP and TCP checksums.
2. Host Q (192.168.1.11) uses the same source port 51000 for its own connection to the same server. The inside tuples differ by address, so the router allocates the public port 40002.
3. The server replies to 203.0.113.7:40002. The router finds the mapping to 192.168.1.11:51000, rewrites the destination, recalculates the checksums and forwards the segment to Q.
4. A segment from the server to port 40009 has no entry and is dropped.
5. After the last packet of the connection (or a timeout), the entry is removed.

DHCP lease

The client with MAC address 02:00:00:aa:bb:cc sends DISCOVER with `xid` 0x3903f326 and the broadcast flag set. The server 192.168.1.1 answers with OFFER (`yiaddr` 192.168.1.50, options: mask 255.255.255.0, router 192.168.1.1, DNS 192.168.1.1, lease 86400 s). The client broadcasts REQUEST with option 54 = 192.168.1.1 and option 50 = 192.168.1.50. The ACK confirms the lease. The timers are T1 = 43,200 s and T2 = 75,600 s for a lease of 86,400 s (one day). The first four DHCP messages are carried as follows.

Message	IP source > destination	UDP source > destination	Ethernet destination
DISCOVER	`0.0.0.0 > 255.255.255.255`	`68 > 67`	ff:ff:ff:ff:ff:ff
OFFER	`192.168.1.1 > 255.255.255.255` (or unicast to yiaddr)	`67 > 68`	client MAC or broadcast
REQUEST	`0.0.0.0 > 255.255.255.255`	`68 > 67`	ff:ff:ff:ff:ff:ff
ACK	`192.168.1.1 > 255.255.255.255` (or unicast to yiaddr)	`67 > 68`	client MAC or broadcast

Addresses of the four DHCP messages.

2.6.3 Hands-on

The first program simulates the PAT table of the diagram with two maps. It shows the allocation of public ports, the reuse of an existing mapping and the dropping of an unknown inbound packet.

PatSim.java JAVA

```
import java.util.LinkedHashMap;
import java.util.Map;

public class PatSim {
    record Endpoint(String ip, int port) {
        @Override
        public String toString() {
            return ip + ":" + port;
        }
    }

    static final String PUBLIC_IP = "203.0.113.7";
    static final Map<Endpoint, Integer> outbound = new LinkedHashMap<>();
    static final Map<Integer, Endpoint> inbound = new LinkedHashMap<>();
    static int nextPort = 40001;

    static Endpoint translateOut(Endpoint inside, Endpoint server) {
        int pub = outbound.computeIfAbsent(inside, k -> {
            int p = nextPort++;
            inbound.put(p, k);
            return p;
        });
        System.out.printf("OUT  %s > %s  =>  %s:%d > %s%n", inside, server, PUBLIC_IP, pub, server);
        return new Endpoint(PUBLIC_IP, pub);
    }
```

PatSim.java (continued) JAVA

```
    static void translateIn(Endpoint server, int publicPort) {
        Endpoint inside = inbound.get(publicPort);
        if (inside == null) {
            System.out.printf("IN   %s > %s:%d  =>  dropped (no mapping)%n", server, PUBLIC_IP,
                    publicPort);
        } else {
            System.out.printf("IN   %s > %s:%d  =>  %s > %s%n", server, PUBLIC_IP, publicPort,
                    server, inside);
        }
    }

    public static void main(String[] args) {
        Endpoint server = new Endpoint("198.51.100.20", 443);
        Endpoint p = new Endpoint("192.168.1.10", 51000);
        Endpoint q = new Endpoint("192.168.1.11", 51000);
        translateOut(p, server);
        translateOut(q, server);
        translateOut(p, server);
        translateIn(server, 40002);
        translateIn(server, 40001);
        translateIn(server, 40009);
        System.out.println("Table: " + outbound);
    }
}
```

Output (deterministic) TERMINAL

```
$ java PatSim.java
OUT  192.168.1.10:51000 > 198.51.100.20:443  =>  203.0.113.7:40001 > 198.51.100.20:443
OUT  192.168.1.11:51000 > 198.51.100.20:443  =>  203.0.113.7:40002 > 198.51.100.20:443
OUT  192.168.1.10:51000 > 198.51.100.20:443  =>  203.0.113.7:40001 > 198.51.100.20:443
IN   198.51.100.20:443 > 203.0.113.7:40002  =>  198.51.100.20:443 > 192.168.1.11:51000
IN   198.51.100.20:443 > 203.0.113.7:40001  =>  198.51.100.20:443 > 192.168.1.10:51000
IN   198.51.100.20:443 > 203.0.113.7:40009  =>  dropped (no mapping)
Table: {192.168.1.10:51000=40001, 192.168.1.11:51000=40002}
```

What to notice: the third outbound packet from host P reuses port 40001, because the mapping is looked up by the inside endpoint; the packet to port 40009 is dropped because no mapping exists. A real implementation also keys on the protocol and the remote endpoint, tracks the TCP state to remove entries after FIN or RST, and expires idle entries.

The second program builds a DHCPDISCOVER message byte by byte, which exposes the fixed 236-byte BOOTP layout, the magic cookie `63 82 53 63` and the options.

DhcpDiscover.java JAVA

```
import java.nio.ByteBuffer;

public class DhcpDiscover {
    static String hex(byte[] b, int from, int to) {
        StringBuilder sb = new StringBuilder();
        for (int i = from; i < to; i++) {
            sb.append(String.format("%02x", b[i]));
            if (i < to - 1) {
                sb.append(' ');
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        byte[] mac = {0x02, 0x00, 0x00, (byte) 0xaa, (byte) 0xbb, (byte) 0xcc};
        ByteBuffer b = ByteBuffer.allocate(300);
        b.put((byte) 1).put((byte) 1).put((byte) 6).put((byte) 0);   // op, htype, hlen, hops
        b.putInt(0x3903f326);                                       // xid
        b.putShort((short) 0).putShort((short) 0x8000);             // secs, flags (broadcast)
        b.put(new byte[16]);                                        // ciaddr yiaddr siaddr giaddr
        b.put(mac).put(new byte[10]);                               // chaddr (16 bytes)
        b.put(new byte[64 + 128]);                                  // sname, file
        b.putInt(0x63825363);                                       // magic cookie
        b.put((byte) 53).put((byte) 1).put((byte) 1);               // option 53: DHCPDISCOVER
        b.put((byte) 55).put((byte) 3).put((byte) 1).put((byte) 3).put((byte) 6); // parameter list
        b.put((byte) 255);                                          // end
        byte[] msg = new byte[b.position()];
```

DhcpDiscover.java (continued) JAVA

```
        b.flip();
        b.get(msg);
        System.out.println("BOOTP/DHCP payload length: " + msg.length + " bytes");
        System.out.println("op htype hlen hops : " + hex(msg, 0, 4));
        System.out.println("xid                : " + hex(msg, 4, 8));
        System.out.println("secs flags         : " + hex(msg, 8, 12));
        System.out.println("chaddr             : " + hex(msg, 28, 34));
        System.out.println("magic cookie       : " + hex(msg, 236, 240));
        System.out.println("options            : " + hex(msg, 240, msg.length));
        System.out.println("UDP/IP total       : " + (msg.length + 8 + 20) + " bytes");
    }
}
```

Output (deterministic) TERMINAL

```
$ java DhcpDiscover.java
BOOTP/DHCP payload length: 249 bytes
op htype hlen hops : 01 01 06 00
xid                : 39 03 f3 26
secs flags         : 00 00 80 00
chaddr             : 02 00 00 aa bb cc
magic cookie       : 63 82 53 63
options            : 35 01 01 37 03 01 03 06 ff
UDP/IP total       : 277 bytes
```

The bytes decode as follows: `35 01 01` is option 53 (message type) of length 1 with value 1 (DISCOVER); `37 03 01 03 06` is option 55 (parameter request list) asking for the mask (1), router (3) and DNS (6); `ff` ends the options. The flag word `80 00` has the broadcast bit set, so the server broadcasts its reply because the client cannot yet receive unicast on an unconfigured interface. The datagram length of 277 bytes adds the 8-byte UDP header and the 20-byte IP header; some clients pad the message to 300 bytes for compatibility with old BOOTP relay agents.

Keeping connections alive behind NAT

A Java service that keeps idle connections in a pool (JDBC, HTTP) across a NAT gateway loses them when the mapping expires. The next use fails with `Connection reset` or hangs until a timeout. Three measures help: set the pool's maximum connection lifetime below the NAT idle timeout (for HikariCP `spring.datasource.hikari.max-lifetime`, whose default is 30 minutes); enable TCP keep-alive with `socket.setKeepAlive(true)` and shorten the operating-system probe interval, since the Linux default of 7200 seconds before the first probe (`net.ipv4.tcp_keepalive_time`) is far above typical NAT timeouts; and validate connections before use.

2.6.4 Failure modes

Symptom	Cause	Diagnosis and fix
First request after an idle period fails with reset or timeout	NAT or firewall removed the idle mapping	Compare idle time with the gateway's timeout; shorten pool lifetime; use keep-alive
Intermittent connection failures under high load to one destination	Source port exhaustion on the NAT gateway	Check gateway metrics (port allocation errors); add public addresses or reuse connections
Service unreachable from the Internet	Missing port forward or security rule; CGNAT in front	Check the public address of the router against the address seen by an external service
All users appear as one address in logs and limits	Shared NAT address	Use forwarded-header or authenticated identity for limits
Host has 169.254.x.x	No DHCP reply (server down, wrong VLAN, relay missing)	Check relay `giaddr`, pool exhaustion, VLAN
Two hosts report an address conflict	Static address inside the DHCP pool, or two servers on one segment	Exclude static addresses from pools; find the rogue server
Address changes and service disappears	Lease expired and a different address was issued	Use DHCP reservations or static configuration for servers

NAT and DHCP faults.

WATCH OUT
Two DHCP servers on one segment hand out conflicting configurations. A home router plugged into a corporate network by its LAN port is the classic cause: clients receive the wrong gateway and DNS server. Switches offer DHCP snooping to accept server replies only from trusted ports.

2.6.5 Questions to expect

1. What is PAT and how does it differ from basic NAT?

Basic NAT maps one private address to one public address. PAT also rewrites the transport port, so many private hosts share one public address, distinguished by public source port numbers in the translation table.

2. How does a NAT device handle the reply to an outbound packet?

It looks up the destination (public) port in its translation table, restores the original private address and port, recalculates the checksums and forwards the packet inside. Without a matching entry the packet is dropped.

3. Why can a server on a private network not be contacted from the Internet without configuration?

No mapping exists until the inside host sends traffic. A port-forwarding rule (destination NAT) or a public address creates the mapping in advance.

4. What problems does NAT cause?

Loss of end-to-end addressing, per-connection state in the middle of the network, idle timeouts, port exhaustion, trouble for protocols that embed addresses, shared source addresses for rate limiting, and the need for traversal techniques in peer-to-peer applications.

5. Describe the DHCP exchange and the addresses used.

DISCOVER (client broadcast from 0.0.0.0:68 to 255.255.255.255:67), OFFER (server proposes `yiaddr` and options), REQUEST (client broadcasts the chosen offer, naming the server), ACK (server confirms the lease). Broadcasts are necessary because the client has no address yet.

6. What are T1 and T2?

T1 (default 50 percent of the lease) is the time at which the client unicasts a renewal to its server. T2 (default 87.5 percent) is the time at which it broadcasts a rebinding request to any server if renewal failed.

7. How can a single DHCP server serve several subnets?

Through relay agents on the routers. A relay forwards the client's broadcast as a unicast to the server and sets `giaddr` to its own address on the client's subnet, which the server uses to choose the address pool.

8. Why does a pooled database connection fail after a quiet period in a cloud deployment?

A NAT gateway or firewall between the application and the database expired the idle flow. The pool still holds the socket, but the middle device no longer knows the connection. Keep-alives, a shorter maximum connection lifetime and validation before use prevent it.

9. Does NAT improve security?

It blocks unsolicited inbound connections as a side effect of its state table, but it is not a security control: the same behaviour is provided explicitly by a stateful firewall, and outbound malware connections are unaffected.

INTERVIEW ANGLE
Follow-up: how can two peers behind two different NATs communicate directly? Each peer contacts a public STUN server to learn its mapped public address and port, the peers exchange these candidates through a signalling channel, and both send packets to each other at nearly the same time (UDP hole punching). Each outbound packet creates a mapping that admits the reply of the other side. If the NAT behaviour prevents this (symmetric NAT), the traffic is relayed through a TURN server. ICE automates the candidate selection.

sreeram

KEY POINTS

- PAT rewrites the source address and port of outbound packets and keeps a table that maps replies back; unsolicited inbound packets are dropped.
- NAT state expires; idle pooled connections die silently and must be handled with keep-alive, validation and a connection lifetime below the NAT timeout.
- Port forwarding creates inbound mappings; CGNAT (100.64.0.0/10) prevents them; STUN, TURN and ICE support peer-to-peer traffic.
- DHCP uses DORA over broadcast UDP (ports 68 and 67), assigns address, mask, gateway and DNS, and renews at T1 and T2; relays carry requests across routers through `giaddr`.

2.7 Cloud Networking Basics

A Spring Boot application deployed to a cloud provider runs inside a **virtual network** whose addressing, routing and filtering the operator defines. The concepts are those of the preceding chapters: CIDR blocks, subnets, route tables, default gateways and NAT. Cloud platforms package them as managed objects. This chapter uses AWS terminology (VPC, internet gateway, NAT gateway, security group, network ACL); Azure (virtual network, network security group) and Google Cloud (VPC network, firewall rules) provide equivalents with differences in detail, notably that some providers apply routes and rules per network rather than per subnet. A misconfigured route table or security rule is the most frequent reason that a freshly deployed service "does not work", so the model deserves a precise understanding.

2.7.1 Concept and mechanism

The virtual private cloud and its subnets

A **VPC** is an isolated virtual network with an address range chosen by the operator, for example `10.0.0.0/16` (65,536 addresses). The range comes from the private blocks of RFC 1918 and must not overlap with any network that will later be connected (other VPCs, on-premises networks over VPN). A VPC is divided into **subnets**, each of which lies in a single availability zone. A common layout uses one subnet per tier per zone, for example /24 subnets: `10.0.1.0/24` (public, zone a), `10.0.2.0/24` (private application, zone a), `10.0.3.0/24` (private data, zone a) and the same pattern in the other zones with the next third-octet values. The provider reserves several addresses in every subnet (in AWS the first four and the last, so a /24 provides 251 usable addresses).

A subnet is **public** or **private** only by its route table. A subnet whose route table sends `0.0.0.0/0` to an **internet gateway** is public; its instances can reach the Internet and be reached from it if they also have a public address. A subnet without such a route is private. The route table always contains an implicit `local` route for the VPC range, so all subnets of a VPC can reach each other by default.

Gateways

An **internet gateway** (IGW) is a horizontally scaled, managed component that connects the VPC to the Internet. For instances with a public address it performs one-to-one NAT between the private address and the public address; the instance itself only knows its private address.

A **NAT gateway** lets instances in private subnets start outbound connections (software updates, calls to external APIs) without being reachable from the Internet. It is placed in a public subnet with an elastic (static public) address, and the route table of the private subnet sends `0.0.0.0/0` to it. It performs the PAT function described in the previous chapter. Its idle-connection timeout (350 seconds for TCP in AWS) is the cause of the stale-connection faults described there.

Further components attach other networks: VPC peering and transit gateways connect VPCs (the connected ranges must not overlap, and peering is not transitive), VPN gateways and dedicated links connect on-premises networks, and gateway or interface **VPC endpoints** reach provider services such as object storage without traversing the Internet.

Security groups and network ACLs

Two filtering mechanisms exist at different layers of the VPC.

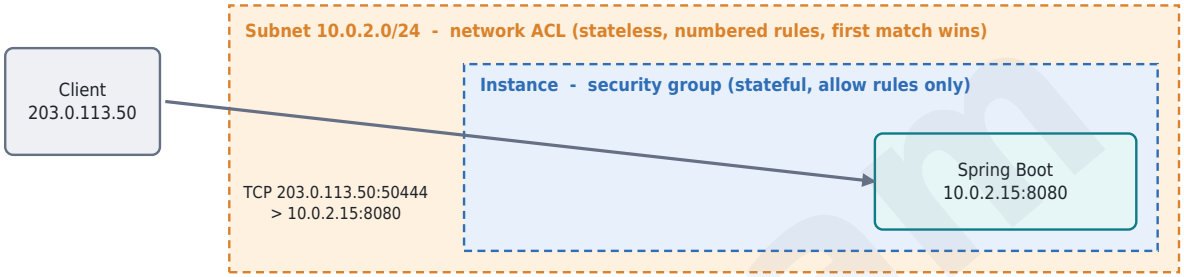
Property	Security group	Network ACL
Attached to	Network interface of an instance (or load balancer, database)	Subnet
State	Stateful: reply traffic of an allowed connection is allowed automatically	Stateless: request and reply are evaluated separately

Property	Security group	Network ACL
Rule type	Allow rules only; everything not allowed is denied	Allow and deny rules with numbers; first matching rule wins
Source or destination	CIDR block or another security group	CIDR block only
Default	All inbound denied, all outbound allowed	The default ACL allows everything
Typical use	Primary control: application access between tiers	Coarse extra layer, for example blocking an address range

Security group compared with network ACL.

The ability of a security group to refer to another security group is the most useful property in practice: the rule "allow TCP 8080 from the load balancer's group" remains correct when instances are added or replaced, and it does not hardcode addresses. A network ACL is stateless, so a rule that admits inbound TCP port 8080 must be accompanied by an outbound rule for the **ephemeral port range** (1024 to 65535) to let the replies leave; forgetting it is a classic fault.

Packet filtering layers in a VPC: network ACL at the subnet edge, security group at the instance



Rule examples

Network ACL inbound	`100 allow TCP 8080 from 0.0.0.0/0`	*200 deny all (implicit * rule)*
Network ACL outbound	`100 allow TCP 1024-65535 to 0.0.0.0/0`	*ephemeral ports needed: stateless*
Security group inbound	`allow TCP 8080 from sg-lb`	*reply traffic allowed automatically*

A security group tracks the connection, so the response needs no outbound rule. A network ACL evaluates each packet independently, so the response (source port 8080, destination port 50444) needs its own outbound allow.

Figure 2.24. A request reaches the subnet boundary, where the stateless network ACL evaluates it, and then the instance, where the stateful security group evaluates it; the reply needs its own outbound rule in the ACL but no rule in the security group.

A complete layout for a Spring Boot application

sreeram

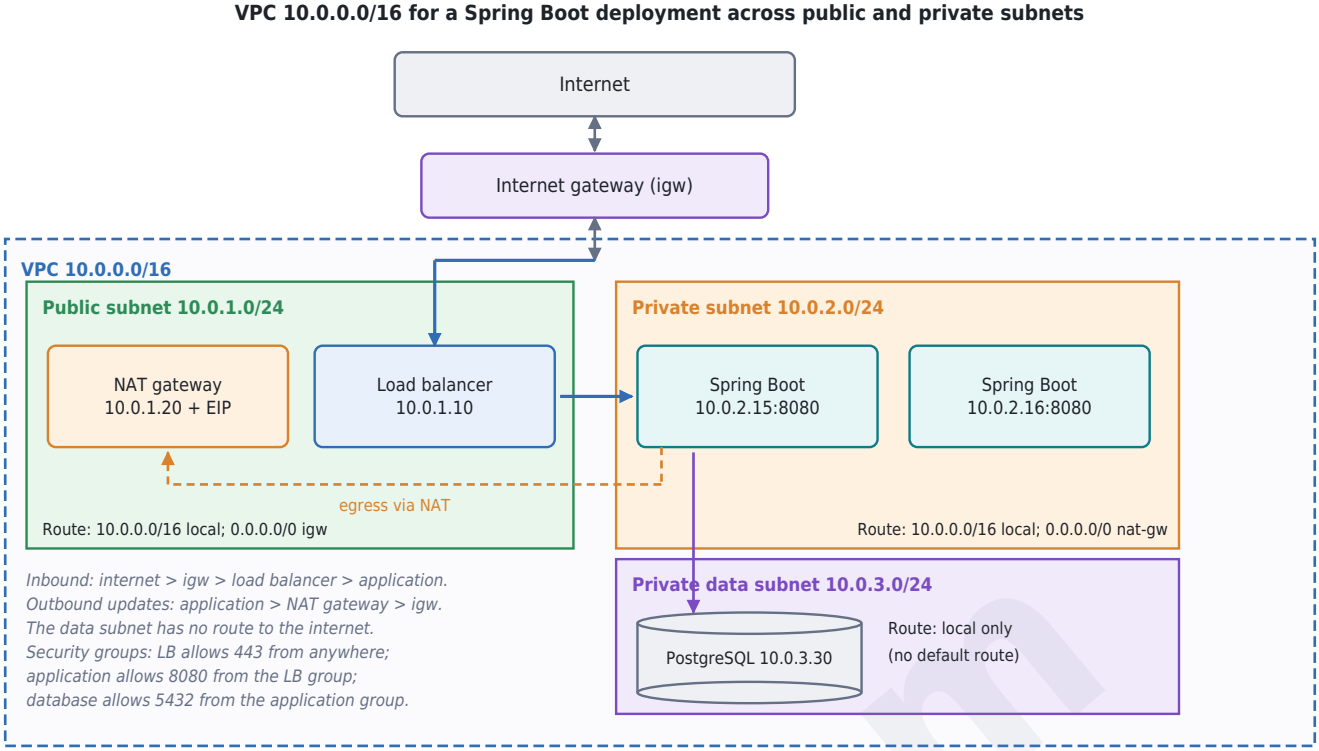


Figure 2.25. A three-tier layout: a load balancer and a NAT gateway in the public subnet, Spring Boot instances in a private subnet that reach the Internet only through the NAT gateway, and a database in a data subnet with no default route at all.

The three tiers use different routes and different security group rules.

Subnet	Routes	Effect
Public 10.0.1.0/24	`10.0.0.0/16 local; 0.0.0.0/0 igw`	Load balancer reachable from the Internet; NAT gateway can send out
Private application 10.0.2.0/24	`10.0.0.0/16 local; 0.0.0.0/0 nat-gw`	Application starts outbound connections through NAT; no inbound from the Internet
Private data 10.0.3.0/24	`10.0.0.0/16 local`	Database reachable only from inside the VPC

Route tables of the layout.

The security groups chain the tiers: the load balancer group admits TCP 443 from `0.0.0.0/0`, the application group admits TCP 8080 only from the load balancer group, and the database group admits TCP 5432 only from the application group. The application reaches the database by its private DNS name, which resolves to a 10.0.3.x address inside the VPC.

2.7.2 Worked trace

A request and an outbound call

A client at 203.0.113.50 requests `https://app.example.com/orders`. Public DNS returns the public address of the load balancer, which is a node in the public subnet with private address 10.0.1.10.

Step	Source > destination (IP:port)	Event
1	`203.0.113.50:50444 > load balancer public address:443`	Client connects; the IGW translates the public address to 10.0.1.10
2	`203.0.113.50:50444 > 10.0.1.10:443`	Security group of the load balancer allows TCP 443 from anywhere; TLS ends at the balancer
3	`10.0.1.10:41312 > 10.0.2.15:8080`	New connection to the application; the network ACL of the private subnet allows 8080 from 10.0.1.0/24; the application group allows the load balancer group
4	`10.0.2.15:38722 > 10.0.3.30:5432`	JDBC connection to the database; allowed by the group rule; the route is the `local` route

Step	Source > destination (IP:port)	Event
5	`10.0.2.15:47010 > 198.51.100.20:443`	Application calls an external API; route 0.0.0.0/0 sends the packet to the NAT gateway
6	`10.0.1.20:1024 + n > 198.51.100.20:443`	NAT gateway translates the source to its elastic address (public); the IGW forwards to the Internet
7	`replies in reverse order`	NAT gateway restores 10.0.2.15:47010; the security group admits replies because it is stateful

Hop-by-hop path of one request and the application's outbound call.

The client sees only the load balancer; the application's private address never appears outside the VPC, and the source address of the external API call is the elastic address of the NAT gateway, which is the address a partner must allow-list.

Planning the address space

A VPC of 10.0.0.0/16 with six /24 subnets (public, application, data across two zones) uses 6 x 256 = 1536 of the 65,536 addresses. Each subnet offers 251 usable addresses, which limits an application tier to 251 instances per zone. Containers or pods that receive VPC addresses (for example in a managed Kubernetes service with VPC-native networking) consume one address each and exhaust small subnets quickly; such deployments use /20 or /19 subnets and a larger VPC range, and a secondary range for pods if the platform supports it.

2.7.3 Hands-on

A service must listen on an address that the load balancer can reach. The following program binds one server socket to `127.0.0.1` and another to the wildcard address, then connects to the machine's own non-loopback address, as a load balancer's health check would.

BindDemo.java JAVA

```
import java.net.InetAddress;
import java.net.InetSocketAddress;
import java.net.NetworkInterface;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Collections;

public class BindDemo {
    static InetAddress firstExternalIpv4() throws Exception {
        for (NetworkInterface ni : Collections.list(NetworkInterface.getNetworkInterfaces())) {
            if (ni.isUp() && !ni.isLoopback()) {
                for (InetAddress a : Collections.list(ni.getInetAddresses())) {
                    if (a.getAddress().length == 4) {
                        return a;
                    }
                }
            }
        }
        throw new IllegalStateException("no external IPv4 address");
    }

    static void tryConnect(String label, ServerSocket server, InetAddress target) {
        try (Socket s = new Socket()) {
            s.connect(new InetSocketAddress(target, server.getLocalPort()), 1000);
            System.out.printf("%-22s connect to %s: OK%n", label, target.getHostAddress());
        } catch (Exception e) {
            System.out.printf("%-22s connect to %s: %s%n", label, target.getHostAddress(),
                    e.getClass().getSimpleName() + " (" + e.getMessage() + ")");
        }
    }

    public static void main(String[] args) throws Exception {
        InetAddress external = firstExternalIpv4();
        InetAddress loopback = InetAddress.getByName("127.0.0.1");
        try (ServerSocket loopbackOnly = new ServerSocket(0, 50, loopback);
             ServerSocket wildcard = new ServerSocket(0)) {
            System.out.println("loopback-only bound to " + loopbackOnly.getInetAddress());
            System.out.println("wildcard bound to      " + wildcard.getInetAddress()
                    + " (any-local=" + wildcard.getInetAddress().isAnyLocalAddress() + ")");
            tryConnect("server on 127.0.0.1", loopbackOnly, external);
```

BindDemo.java (continued) JAVA

```
            tryConnect("server on 0.0.0.0", wildcard, external);
        }
    }
}
```

Output (measured on the build machine; the address 192.0.2.2 is that of its `eth0`) TERMINAL

```
$ java BindDemo.java
loopback-only bound to /127.0.0.1
wildcard bound to      0.0.0.0/0.0.0.0 (any-local=true)
server on 127.0.0.1    connect to 192.0.2.2: ConnectException (Connection refused)
server on 0.0.0.0      connect to 192.0.2.2: OK
```

What to notice: a server bound to `127.0.0.1` refuses connections that arrive on the external address, whereas the wildcard binding accepts them. In Spring Boot the equivalent setting is `server.address`; leaving it unset binds the wildcard. Embedded Tomcat therefore listens correctly in a container by default, and a hardened configuration that sets `server.address=127.0.0.1` makes the health check of the load balancer fail.

The following configuration is typical for an application behind a load balancer. It is illustrative and was not run here.

application.yml YAML

```yaml
server:
  port: 8080
  forward-headers-strategy: native
  shutdown: graceful
  tomcat:
    connection-timeout: 20s
    keep-alive-timeout: 60s
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s
management:
  endpoint:
    health:
      probes:
        enabled: true
```

The choices have network reasons. `forward-headers-strategy: native` lets Tomcat use the `X-Forwarded-*` headers from the balancer, so that the client address and the scheme (https) are correct. `shutdown: graceful` lets in-flight requests finish after the balancer stops sending new ones; the balancer's deregistration delay should be shorter than the shutdown timeout. The Tomcat keep-alive timeout should exceed the idle timeout of the load balancer (60 seconds by default on many balancers), otherwise Tomcat can close a connection at the moment the balancer reuses it, which produces sporadic 502 responses. The probe endpoints give the balancer an HTTP path (`/actuator/health/readiness`) to check.

2.7.4 Failure modes

Symptom	Cause	Diagnosis and fix
Request to the service times out from the Internet	No route to the IGW for the subnet, or no public address, or security group closed	Check route table, public address assignment and the group of the balancer
Load balancer reports unhealthy targets	Application group does not admit the balancer's group, wrong port, application bound to loopback	Verify group rule and health check port; check `server.address`
Application cannot call external APIs	Private subnet without NAT route, or NAT gateway in a subnet without IGW route	Check `0.0.0.0/0` in the private route table and in the NAT gateway's subnet
Works in one zone, fails in another	Route table or ACL associated with only one subnet	Compare subnets; automate through infrastructure as code
Database unreachable from a new service	Database group lacks a rule for the new application group	Add the group reference, not a CIDR
Peering configured but no traffic	Overlapping ranges, missing routes on both sides, or security rules	Add routes in both route tables; check ranges
Replies lost despite an inbound allow rule	Network ACL denies the ephemeral port range outbound	Add the stateless return rule

Cloud networking faults.

REAL WORLD
Infrastructure as code (Terraform, CloudFormation) is the standard way to define these objects. The reason is not convenience alone: a route table association or a security group rule that exists only through manual changes cannot be reviewed, compared between zones or reproduced after an incident.

2.7.5 Questions to expect

1. What makes a subnet public or private?

Its route table. A subnet with a route to an internet gateway for 0.0.0.0/0 is public; without it, private. Instances in a public subnet are reachable from the Internet only if they also have a public address and the security rules permit it.

2. What does a NAT gateway do and where is it placed?

It lets instances in private subnets initiate outbound connections by translating their private source addresses to its elastic address. It sits in a public subnet, and the private subnets route 0.0.0.0/0 to it. It does not allow inbound connections initiated from the Internet.

3. Compare security groups and network ACLs.

A security group is stateful, attached to a network interface and has only allow rules; a network ACL is stateless, attached to a subnet and evaluates numbered allow and deny rules in order. Replies need explicit rules in the ACL.

4. Why reference a security group instead of an IP range in a rule?

Membership follows the instances, so the rule stays correct when addresses change or instances scale in and out, and it grants access to exactly the intended tier.

5. Why should the database be in a subnet with no route to the Internet?

It removes an entire class of exposure: even a mistaken security rule or a compromised host cannot route the database to or from the Internet. Updates arrive through managed services or endpoints.

6. How does an application behind a load balancer know the client address?

The balancer terminates the connection and appends the client address to `X-Forwarded-For` (or uses the PROXY protocol for TCP balancers). Spring Boot honours it with `server.forward-headers-strategy` set to `native` or `framework`, and it must be trusted only from the balancer.

7. Why do overlapping CIDR blocks matter?

Routes are matched by prefix, so two networks with the same range cannot be distinguished. VPC peering and VPN connections need disjoint ranges, which makes the choice of range an early and hard-to-change decision.

8. A service is healthy locally but the load balancer reports it unhealthy. What are the likely causes?

The application listens only on loopback, the security group does not admit the balancer, the health check uses a wrong port or path, or the application needs longer to start than the health check allows.

9. How does the application reach the database in a private subnet?

Through the `local` route of the VPC using the database's private address or DNS name; the security group of the database admits the application group on the database port.

KEY POINTS
- A VPC is a private CIDR block divided into zone-bound subnets; a subnet is public only when its route table sends 0.0.0.0/0 to an internet gateway.
- A NAT gateway provides outbound-only access for private subnets and is a PAT device with an idle timeout.
- Security groups are stateful allow lists attached to interfaces and can reference each other; network ACLs are stateless, ordered and subnet-wide.
- Spring Boot behind a load balancer needs a reachable bind address, forwarded-header handling, a keep-alive timeout above the balancer's idle timeout and graceful shutdown.
- Plan non-overlapping, generously sized address ranges early; they are difficult to change later.

skeram

PART 3

The Transport Layer

UDP, TCP, reliability, flow and congestion control, the Java socket API and QUIC.

In this part

- **3.1** Transport Layer, Ports and Sockets
- **3.2** UDP: The User Datagram Protocol
- **3.3** TCP Segments and Connection Management
- **3.4** TCP Reliability
- **3.5** Flow Control
- **3.6** Congestion Control
- **3.7** Sockets in Java
- **3.8** QUIC and HTTP/3 Transport

3.1 Transport Layer, Ports and Sockets

The network layer delivers packets between hosts. It does not distinguish between the many programs on a host, it offers no guarantee that a packet arrives, and it does not preserve order. The transport layer closes two gaps. First, it extends host-to-host delivery to process-to-process delivery by means of port numbers. Second, depending on the protocol chosen, it adds services that IP does not provide: reliable delivery, ordering, flow control and congestion control (TCP), or almost nothing beyond the port numbers and an optional integrity check (UDP).

For a Java backend developer the transport layer is the layer that is exposed directly by `java.net.Socket`, `ServerSocket` and `DatagramSocket`. Every `HttpClient` call, every JDBC connection and every Tomcat request passes through it. Many production symptoms (connection refused, connect timeout, read timeout, connection reset, exhausted ephemeral ports, sockets stuck in `TIME_WAIT` or `CLOSE_WAIT`) are transport-layer events, and diagnosing them requires an exact model of what the layer does.

3.1.1 Concept and mechanism

Process-to-process delivery

An IP address identifies a network interface. A transport-layer **port** is a 16-bit unsigned integer that identifies an endpoint within one host. The pair of an IP address and a port is a **transport address**, written `198.51.100.20:443`. The sending transport entity writes a source port and a destination port into the transport header; the receiving transport entity uses the destination port to choose the application that receives the data.

Multiplexing is the sender-side operation: data from several application sockets is gathered, each piece is wrapped in a transport header carrying the port numbers, and the segments are handed to the network layer. **Demultiplexing** is the receiver-side operation: the transport layer inspects the header of each arriving segment and delivers the payload to the correct socket.

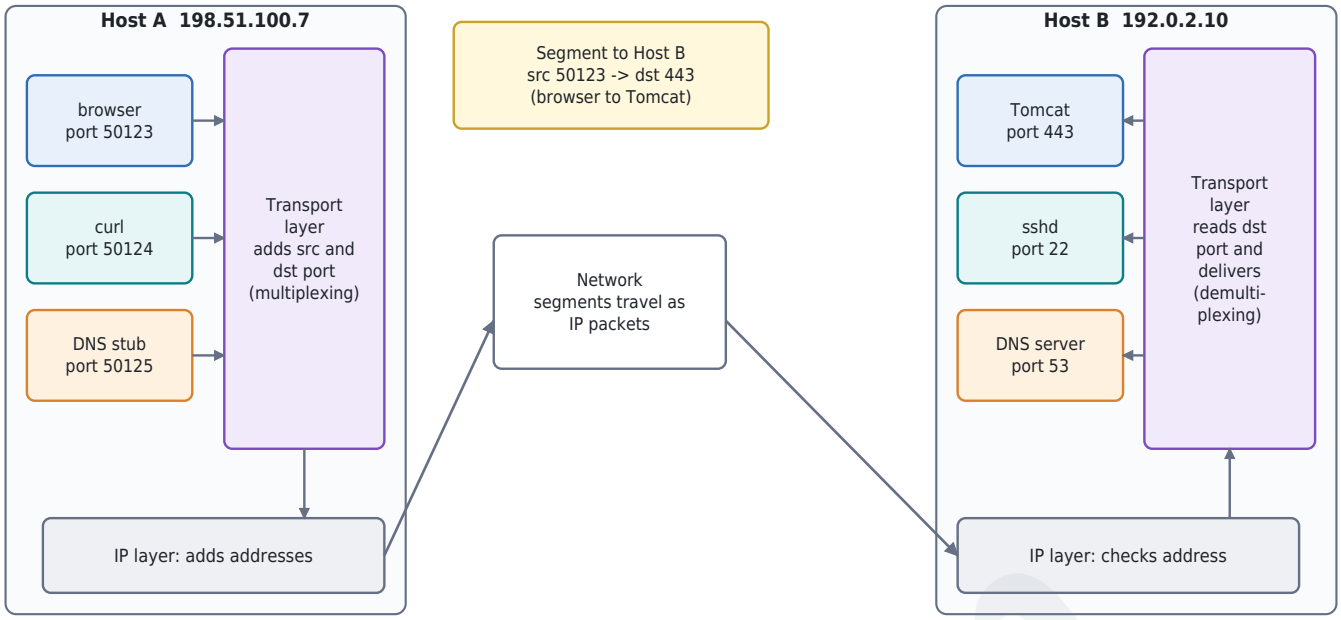


Figure 3.1. Three applications on Host A share one IP layer. The transport layer on Host A writes the source and destination ports; the transport layer on Host B reads the destination port 443 and delivers the segment to the Tomcat process.

Port number ranges

Port numbers are partitioned by the Internet Assigned Numbers Authority (IANA, RFC 6335). The partition is administrative: the operating system enforces only the privilege rule for low ports on some systems.

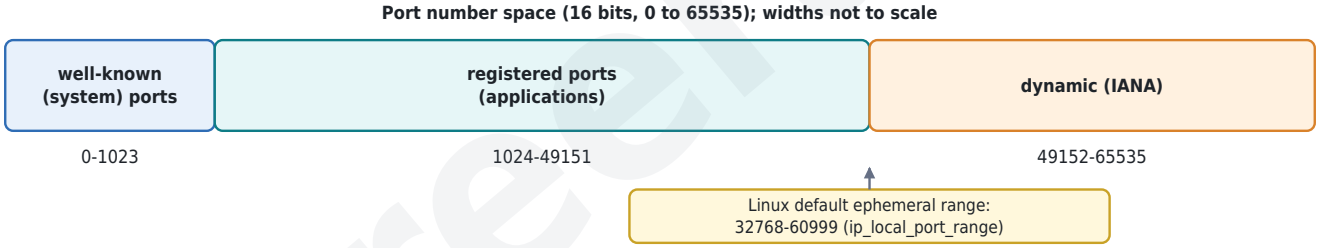


Figure 3.2. The 16-bit port space is divided into well-known ports (0 to 1023), registered ports (1024 to 49151) and dynamic ports (49152 to 65535); Linux draws ephemeral ports from 32768 to 60999 by default.

Range	Name	Use
`0-1023`	Well-known (system)	Assigned services such as 22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS). Binding below 1024 requires privilege on Unix-like systems unless the capability is granted.
`1024-49151`	Registered	Services registered with IANA, for example 3306 (MySQL), 5432 (PostgreSQL), 8080 (alternative HTTP).
`49152-65535`	Dynamic (private)	Never assigned. Client operating systems pick **ephemeral ports** here; Linux uses 32768-60999 by default.

Port ranges and their use.

A Spring Boot application listens on port 8080 by default (`server.port`). The value is a convention, not a protocol requirement, and it falls in the registered range. A process that is not privileged cannot bind to port 80 or 443 on Linux; deployments therefore either run the application on 8080 behind a reverse proxy that owns port 443, or redirect traffic with a packet-filter rule.

Sockets and the 5-tuple

A **socket** is the endpoint abstraction that the operating system presents to a program. A TCP connection is identified by the **5-tuple**: protocol, source address, source port, destination address, destination port. The kernel keeps one table entry per tuple. Two connections are different if any one element differs.

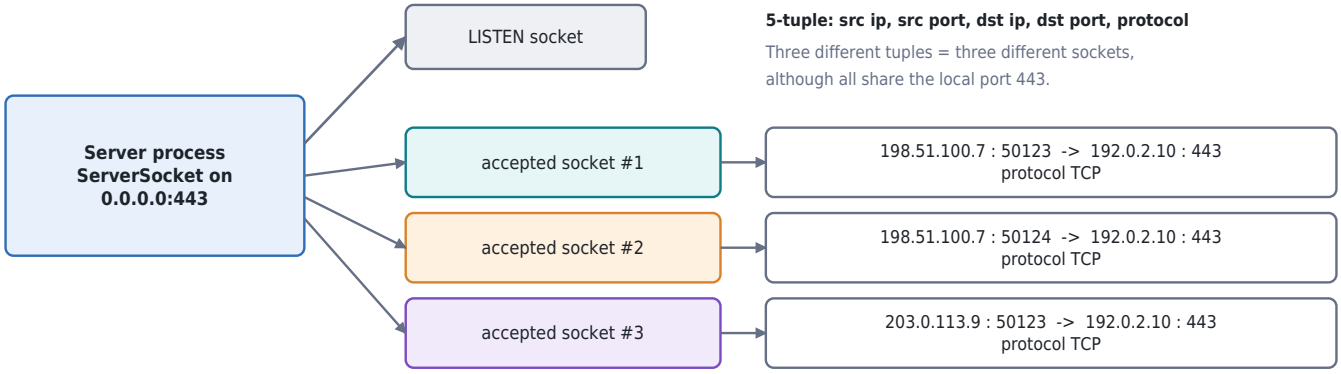


Figure 3.3. A server that listens on port 443 owns one listening socket and one accepted socket per connection. The three accepted sockets share the local port 443 and are distinguished only by the client address and client port.

This explains why a server needs only one port for thousands of clients. The `ServerSocket` bound to port 443 is the **listening socket**; it is in the `LISTEN` state and carries no data. Each call to `accept()` returns a new `Socket` object that represents a different connection, and the local port of that `Socket` is still 443.

For UDP the demultiplexing key is smaller. A UDP socket is identified by the pair (destination address, destination port) only. Datagrams from different senders to the same port are delivered to the same socket, and the application learns the sender from the source address in each datagram (`DatagramPacket.getSocketAddress()` in Java).

Ephemeral ports and capacity limits

A client that opens a connection without binding a local port receives an ephemeral source port. Because the 5-tuple must be unique and the destination address, destination port and protocol are fixed for a given target, the number of simultaneous connections from one client address to one server address and port is bounded by the size of the ephemeral range. For the Linux default of 32768 to 60999 this is 60999 - 32768 + 1 = 28232 connections. A load generator, an API gateway or a service that calls one downstream address at a high rate can reach this limit. The symptom is the error `Cannot assign requested address` (`java.net.BindException` or `NoRouteToHostException: Cannot assign requested address` in Java), usually combined with a large number of sockets in `TIME_WAIT`. Remedies are discussed under the TCP connection-management chapter below: reuse connections (HTTP keep-alive, connection pools), widen the range, or spread the traffic over more destination addresses.

3.1.2 Worked trace

The following trace shows the demultiplexing decision of Host B (192.0.2.10) for three arriving TCP segments. The listening socket is bound to `0.0.0.0:443`; two connections already exist.

Source	Destination	Matches	Action
`198.51.100.7:50123`	`192.0.2.10:443`	Established socket #1 (same 5-tuple)	Payload appended to the receive buffer of socket #1
`198.51.100.7:50124`	`192.0.2.10:443`	Established socket #2 (different source port)	Payload appended to the receive buffer of socket #2
`203.0.113.9:50123`	`192.0.2.10:443`	No established tuple; listening socket matches the destination port	If the segment is a SYN, a new connection is created (socket #3); any other segment is answered with RST

Demultiplexing of three arriving segments on Host B.

The first match is always the most specific one: the kernel prefers an established socket whose full 5-tuple matches over a listening socket that matches only the local address and port. A segment addressed to a port without any socket is answered with a `RST` segment; the sender then reports `Connection refused`. For UDP, the equivalent answer is an ICMP "port unreachable" message, which Java surfaces only on a connected `DatagramSocket` (`PortUnreachableException`).

3.1.3 Hands-on

The loopback interface is sufficient to observe the 5-tuple. The program below starts a server, connects a client, and prints the local and remote ports as seen by each side.

EchoServer.java JAVA

```java
import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) throws Exception {
        try (ServerSocket server = new ServerSocket(19100, 50, InetAddress.getLoopbackAddress())) {
            System.out.println("listening on " + server.getLocalSocketAddress());
            Thread client = Thread.ofPlatform().start(() -> {
                try (Socket s = new Socket(InetAddress.getLoopbackAddress(), 19100)) {
                    System.out.println("client: local=" + s.getLocalPort()
                        + " remote=" + s.getPort());
                    PrintWriter out = new PrintWriter(s.getOutputStream(), true);
                    BufferedReader in = new BufferedReader(
                        new InputStreamReader(s.getInputStream()));
                    out.println("ping");
                    System.out.println("client: got '" + in.readLine() + "'");
                    out.println("quit");
                    System.out.println("client: got '" + in.readLine() + "'");
                } catch (IOException e) { e.printStackTrace(); }
            });
            try (Socket conn = server.accept()) {
                System.out.println("server: accepted " + conn.getRemoteSocketAddress()
                    + " on local port " + conn.getLocalPort());
                BufferedReader in = new BufferedReader(
                    new InputStreamReader(conn.getInputStream()));
                PrintWriter out = new PrintWriter(conn.getOutputStream(), true);
                String line;
                while ((line = in.readLine()) != null) {
                    if (line.equals("quit")) { out.println("bye"); break; }
                    out.println("echo:" + line);
                }
            }
            client.join();
        }
    }
}
```

Run EchoServer.java TERMINAL

```
$ java EchoServer.java
listening on localhost/127.0.0.1:19100
server: accepted /127.0.0.1:43104 on local port 19100
client: local=43104 remote=19100
client: got 'echo:ping'
client: got 'bye'
```

The client reports `local=43104 remote=19100` and the server reports `accepted /127.0.0.1:43104 on local port 19100`: the two descriptions are mirror images of the same tuple. The ephemeral port differs between runs (typical output).

The Linux kernel exposes its socket tables in `/proc/net/tcp`. The next Python fragment reads that file to list the sockets that involve port 7000, after the server has called `listen()` and one client has connected. The output is real (state codes translated by the script).

Socket table entries for port 7000 (output of the script) TERMINAL

```
$ python3 states.py
-- after listen
  local :7000  remote :0     LISTEN
-- after connect+accept
  local :7000  remote :0     LISTEN
  local :39270 remote :7000  ESTABLISHED
  local :7000  remote :39270 ESTABLISHED
```

One listening entry and two established entries exist: the client side (ephemeral port 39270) and the server side (port 7000) of the same connection. On a host with the `ss` command the same information is shown by `ss -tan`; the command is not available on the build machine, and its typical output is reproduced in the tools chapter of Part 1.

3.1.4 Failure modes

Symptom	Cause	Diagnosis and remedy
`Address already in use` on startup	Another socket is bound to the same local address and port (a previous instance, or a port collision) or a closing connection prevents the bind without `SO_REUSEADDR`	Identify the owner of the port; enable `SO_REUSEADDR` (the default for `ServerSocket` on Linux); change `server.port`
`Connection refused`	No process listens on the destination port; the kernel answered the SYN with RST	Verify the process, the port, the bind address (127.0.0.1 versus 0.0.0.0) and the container port mapping
`Cannot assign requested address`	Ephemeral port exhaustion, or binding to an address that does not belong to the host	Count sockets per destination; reuse connections; widen `ip_local_port_range`
Service reachable locally but not remotely	Server bound to `127.0.0.1` only	Bind to `0.0.0.0` or the interface address (`server.address` in Spring Boot)

Transport-layer failures that originate in addressing.

INTERVIEW ANGLE

How can thousands of clients connect to one server on a single port? The connection is identified by the full 5-tuple, not by the server port alone. Each client contributes a different source address or source port, so every connection has a distinct tuple. The server uses one listening socket and one accepted socket per connection, all with local port 443.

3.1.5 Questions to expect

1. **What is the difference between a port and a socket?** A port is a 16-bit number that identifies an endpoint on a host. A socket is the operating-system object bound to a protocol, a local address and port, and (for TCP connections) a remote address and port. The same port can be used by many sockets that differ in the remote endpoint.
2. **Which header fields support multiplexing and demultiplexing?** The source and destination port fields of the transport header, together with the protocol number and the addresses of the IP header that complete the 5-tuple.
3. **What is an ephemeral port and who chooses it?** A temporary source port selected by the client operating system when an application connects without binding a port. The default range on Linux is 32768 to 60999.
4. **Why does a client occasionally fail to open new connections although the server is healthy?** Ephemeral port exhaustion toward one destination address and port, typically with many `TIME_WAIT` entries. Reusing connections or increasing the range removes the limit.
5. **How does a UDP server tell clients apart?** A UDP socket has no per-peer state. Each received datagram carries the source address and port, and the server addresses its reply to that endpoint.
6. **Why can a non-root process not bind to port 443 on Linux, and what is done in practice?** Ports below 1024 are privileged. The process either runs behind a privileged reverse proxy or load balancer, receives the capability `CAP_NET_BIND_SERVICE`, or listens on a high port with an external redirect.
7. **What happens if a segment arrives for a port where nothing listens?** TCP answers with RST (the sender sees `Connection refused`); UDP typically triggers an ICMP destination-port-unreachable message.

KEY POINTS

The transport layer provides process-to-process delivery; the destination and source ports are the demultiplexing keys.

- A TCP connection is identified by the 5-tuple (protocol, source address and port, destination address and port); a UDP socket by (address, port) at the receiver.
- One listening socket produces one accepted socket per connection; all share the local port.
- Ports: 0-1023 well-known, 1024-49151 registered, 49152-65535 dynamic; Linux ephemeral range 32768-60999.
- The ephemeral range bounds the number of simultaneous connections from one address to one destination address and port.

3.2 UDP: The User Datagram Protocol

UDP (RFC 768, 1980) is the minimal transport protocol. It adds to IP only the port numbers, a length field and an optional checksum. Each `send` produces exactly one datagram, each datagram is delivered whole or not at all, and the protocol never retransmits, reorders, paces or acknowledges. These omissions are the purpose of the protocol: an application or a higher-level protocol that implements its own recovery, or that values freshness over completeness, avoids the delay and the state of TCP.

3.2.1 Concept and mechanism

The UDP header

The header occupies 8 bytes. The **length** field counts the header plus the payload, so its minimum value is 8. The 16-bit field limits a datagram to 65535 bytes in total; after the 20-byte IPv4 header and the 8-byte UDP header, the largest payload is 65507 bytes.

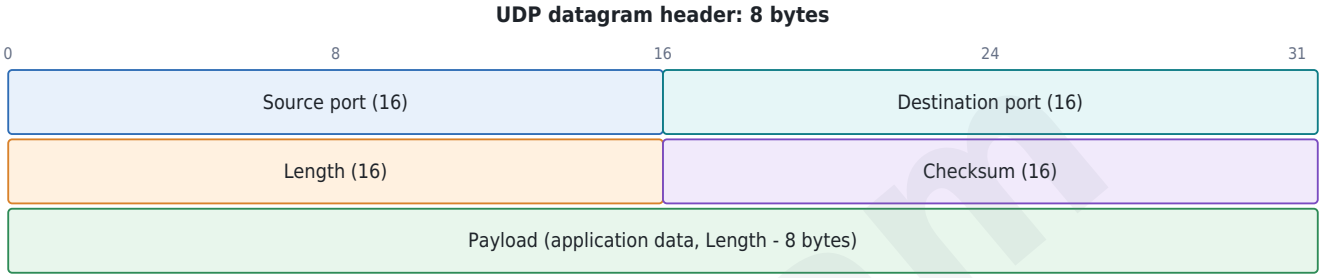


Figure 3.4. The UDP header consists of four 16-bit fields: source port, destination port, length and checksum, followed directly by the payload.

Message boundaries and unreliability

A UDP socket preserves message boundaries: one `DatagramSocket.send` call corresponds to one `receive` call at the peer, and a receive buffer that is smaller than the datagram silently truncates it. UDP provides none of the following: delivery confirmation, retransmission, ordering, duplicate suppression, flow control or congestion control. A datagram may be lost, duplicated or reordered; the application detects these conditions by itself, for example with sequence numbers.

An application that sends a datagram larger than the path MTU relies on IP fragmentation. If one fragment is lost, the whole datagram is lost. Applications that need robustness therefore keep datagrams below the MTU; typical choices are 1200 to 1472 bytes (1500 minus 20 minus 8 for IPv4 on Ethernet).

The checksum

The checksum covers a **pseudo-header**, the UDP header and the payload. The pseudo-header consists of the source address, the destination address, a zero byte, the protocol number (17) and the UDP length. It is not transmitted; it is included so that a datagram delivered to the wrong host or with a corrupted address is rejected. The algorithm is the Internet checksum (RFC 1071): the data is treated as a sequence of 16-bit words (padded with a zero byte if the length is odd), the words are added using one's-complement arithmetic (any carry out of bit 15 is added back), and the result is complemented.

In IPv4 the checksum field may be zero, meaning that no checksum was computed. In IPv6 the checksum is mandatory (RFC 8200), because the IPv6 header has no checksum of its own. If the computed value is zero it is transmitted as 0xFFFF.

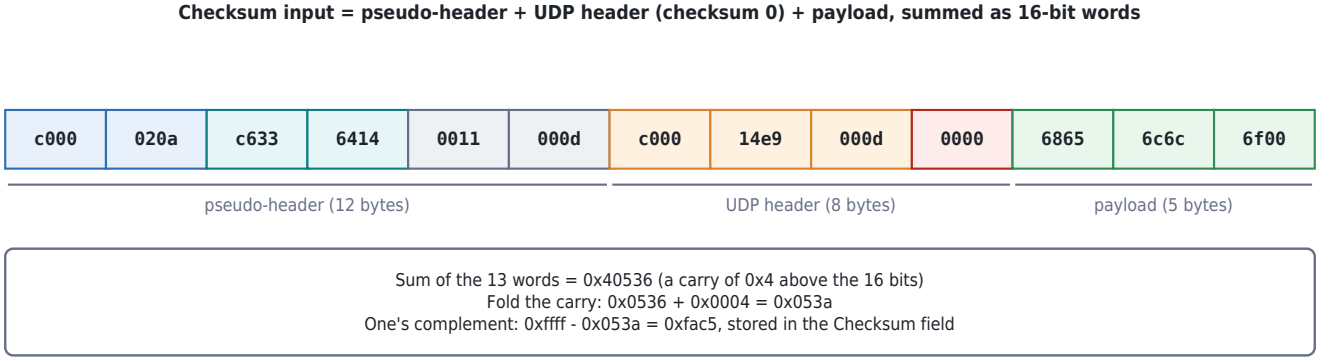


Figure 3.5. The checksum is computed over thirteen 16-bit words: six from the pseudo-header, four from the UDP header (with the checksum field set to zero) and three from the payload. The final value 0xfac5 is the complement of the folded sum 0x053a.

3.2.2 Worked trace

The datagram is sent from `192.0.2.10:49152` to `198.51.100.20:5353` and carries the five bytes `hello`. The UDP length is 8 + 5 = 13 (0x000d). The script below builds the pseudo-header and the UDP header with `struct`, then prints every running total. The output is real.

udpck.py PYTHON

```python
import struct, socket
def csum(b):
    if len(b)%2: b+=b'\0'
    s=sum(struct.unpack('!%dH'%(len(b)//2),b))
    while s>>16: s=(s&0xffff)+(s>>16)
    return (~s)&0xffff
src=socket.inet_aton('192.0.2.10'); dst=socket.inet_aton('198.51.100.20')
payload=b'hello'
length=8+len(payload)
pseudo=src+dst+struct.pack('!BBH',0,17,length)
hdr=struct.pack('!HHHH',49152,5353,length,0)
data=pseudo+hdr+payload
print('pseudo hdr:',pseudo.hex(' '))
print('udp hdr   :',hdr.hex(' '),' payload:',payload.hex(' '))
words=struct.unpack('!%dH'%((len(data)+1)//2), data+b'\0' if len(data)%2 else data)
t=0
for x in words:
    t+=x; print('  +0x%04x = 0x%05x'%(x,t))
c=csum(data); print('sum folded -> complement = 0x%04x'%c)
full=pseudo+struct.pack('!HHHH',49152,5353,length,c)+payload
print('verify (should be 0):',hex(csum(full)))
print('final datagram:',(struct.pack('!HHHH',49152,5353,length,c)+payload).hex(' '))
```

Output of udpck.py TERMINAL

```
$ python3 udpck.py
pseudo hdr: c0 00 02 0a c6 33 64 14 00 11 00 0d
udp hdr   : c0 00 14 e9 00 0d 00 00  payload: 68 65 6c 6c 6f
  +0xc000 = 0x0c000
  +0x020a = 0x0c20a
  +0xc633 = 0x1883d
  +0x6414 = 0x1ec51
  +0x0011 = 0x1ec62
  +0x000d = 0x1ec6f
  +0xc000 = 0x2ac6f
  +0x14e9 = 0x2c158
  +0x000d = 0x2c165
  +0x0000 = 0x2c165
  +0x6865 = 0x329ca
  +0x6c6c = 0x39636
  +0x6f00 = 0x40536
sum folded -> complement = 0xfac5
verify (should be 0): 0x0
final datagram: c0 00 14 e9 00 0d fa c5 68 65 6c 6c 6f
```

The steps are as follows.

Sreeram

1. The pseudo-header words are `c000 020a` (192.0.2.10), `c633 6414` (198.51.100.20), `0011` (zero byte and protocol 17) and `000d` (UDP length).
2. The UDP header words are `c000` (port 49152), `14e9` (port 5353), `000d` (length) and `0000` (checksum placeholder).
3. The payload `68 65 6c 6c 6f` forms the words `6865`, `6c6c` and `6f00`; the last byte is padded with zero.
4. The sum of the thirteen words is 0x40536. The carry above bit 15 is 0x4; adding it back gives 0x0536 + 0x0004 = 0x053a.
5. The one's complement is 0xffff - 0x053a = 0xfac5. This value is written into the checksum field.
6. The receiver repeats the sum including the checksum field. Folding the result yields 0xffff, and the complement is 0, which indicates a valid datagram (printed as `verify`).

The checksum detects most single-bit and burst errors, but it cannot detect the swapping of two 16-bit words, and it is weak compared with a CRC. Ethernet adds a CRC-32 on each frame; the end-to-end checksum protects against faults inside routers and network interface cards.

3.2.3 Hands-on

A Java echo exchange over loopback

The program runs a UDP server and a client in one JVM. The server echoes two datagrams. The last block sends a datagram to a port without a listener and shows the timeout that results, because UDP gives no error indication to an unconnected sender.

UdpEcho.java — JAVA

```
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UdpEcho {
    public static void main(String[] args) throws Exception {
        try (DatagramSocket server = new DatagramSocket(19091, InetAddress.getLoopbackAddress())) {
            Thread t = Thread.ofPlatform().start(() -> {
                try {
                    byte[] buf = new byte[512];
                    for (int i = 0; i < 2; i++) {
                        DatagramPacket p = new DatagramPacket(buf, buf.length);
                        server.receive(p);
                        System.out.println("server: " + p.getLength() + " bytes from "
                            + p.getAddress().getHostAddress() + ":" + p.getPort());
                        server.send(new DatagramPacket(p.getData(), p.getLength(),
                            p.getSocketAddress()));
                    }
                } catch (Exception e) { e.printStackTrace(); }
            });
            try (DatagramSocket client = new DatagramSocket()) {
                client.setSoTimeout(1000);
                System.out.println("client local port: " + (client.getLocalPort() > 0));
                for (String msg : new String[] {"hello", "datagram two"}) {
                    byte[] data = msg.getBytes(StandardCharsets.UTF_8);
                    client.send(new DatagramPacket(data, data.length,
                        InetAddress.getLoopbackAddress(), 19091));
                    DatagramPacket r = new DatagramPacket(new byte[512], 512);
                    client.receive(r);
                    System.out.println("client: echo '" + new String(r.getData(), 0, r.getLength(),
                        StandardCharsets.UTF_8) + "' from port " + r.getPort());
                }
            }
            t.join();
        }
        try (DatagramSocket c = new DatagramSocket()) {
            c.setSoTimeout(300);
            c.send(new DatagramPacket(new byte[1], 1, InetAddress.getLoopbackAddress(), 19099));
            c.receive(new DatagramPacket(new byte[8], 8));
        } catch (Exception e) {
            System.out.println("no listener: " + e.getClass().getSimpleName()
                + ": " + e.getMessage());
        }
    }
}
```

Run UdpEcho.java — TERMINAL

```
$ java UdpEcho.java
client local port: true
server: 5 bytes from 127.0.0.1:41229
client: echo 'hello' from port 19091
server: 12 bytes from 127.0.0.1:41229
client: echo 'datagram two' from port 19091
no listener: SocketTimeoutException: Receive timed out
```

What to notice: the server sees the datagram length (5 bytes, 12 bytes), not a stream; the client port is chosen by the operating system and changes between runs; the reply is addressed to `p.getSocketAddress()`; and the send to an unused port produces no exception on the sender, only a `SocketTimeoutException` after the configured `setSoTimeout`. A receive without a timeout would block indefinitely.

Datagram size limits and a quick test with nc

The maximum payload is observable. A datagram of 70000 bytes is rejected by the operating system, while 65507 bytes are accepted (the data goes to the discard port 9; the output is real):

Datagram size limits (fragment of the Opts.java output) — TERMINAL

```
$ java Opts.java
UDP 70000 bytes: SocketException: Message too long
UDP 65507 bytes: sent
```

The `nc` utility sends and receives UDP from the shell. The first command is a listener, the second sends the datagram. The listener printed `hello` (real output):

A UDP exchange with nc — TERMINAL

```
$ nc -u -l 127.0.0.1 9400 > udp.out &
$ echo -n "hello" | nc -u -w1 127.0.0.1 9400
$ cat udp.out
hello
```

3.2.4 Where UDP is used

Protocol	Reason for UDP	Notes
DNS (port 53)	One small request and one small response; no connection set-up cost	Falls back to TCP when the response is truncated (TC bit) or for zone transfers; DNS over TLS and over HTTPS use TCP
DHCP (67, 68)	The client has no address yet and cannot establish a TCP connection	Broadcast on the local network
NTP (123)	Short, timestamp-sensitive exchanges	Retransmission would distort timing
QUIC and HTTP/3 (usually 443)	User-space transport with its own streams, loss recovery and encryption	Avoids kernel and middlebox ossification of TCP; see the final chapter of this Part
RTP/WebRTC media, VoIP, live video	A late packet is useless; loss is concealed instead of retransmitted	Application adds sequence numbers and jitter buffers
Syslog, SNMP traps, metrics (StatsD)	Fire-and-forget telemetry	Loss is acceptable

UDP-based protocols and the reason for the choice.

3.2.5 Failure modes

- **Silent loss.** A burst that exceeds the receiver socket buffer is dropped by the kernel without notice. The remedy is a larger `SO_RCVBUF` (`DatagramSocket.setReceiveBufferSize`), faster consumption, or an application-level acknowledgement.
- **Amplification attacks.** Because the source address of a UDP datagram can be forged, a small request to a service that returns a large response (open DNS resolvers, NTP monitor commands, memcached) can be turned into a denial-of-service attack against the forged victim. Services restrict responses and rate-limit.
- **Middlebox filtering.** Firewalls and corporate networks often block or throttle UDP other than DNS, which causes QUIC to fall back to TCP.

- **Fragmentation.** Datagrams above the path MTU are fragmented; fragments are frequently filtered, and loss of one fragment loses the datagram.
- **Missing congestion control.** A UDP sender that ignores the network can starve TCP flows. Protocols built on UDP therefore implement their own congestion control (QUIC does).

3.2.6 Questions to expect

1. **Why is UDP faster than TCP?** It has no handshake, no retransmission state, no congestion control and a smaller header (8 against 20 bytes or more). The saving is latency and overhead, at the cost of reliability. The higher rate is not guaranteed: UDP can send faster than the path can carry, and the excess is lost.
2. **Does UDP preserve message boundaries?** Yes. One send corresponds to one datagram and one receive. TCP offers a byte stream without boundaries.
3. **Why does DNS use UDP although the answer matters?** A query and answer fit in one datagram, so the connection set-up of TCP would more than double the cost; the resolver retries on timeout. Large answers use TCP.
4. **What does the UDP checksum cover and why is a pseudo-header used?** The UDP header, the payload and a pseudo-header with the IP addresses, protocol and length. The pseudo-header detects misdelivery caused by corrupted or wrong addresses without transmitting them again.
5. **What is the maximum UDP payload over IPv4?** 65507 bytes: 65535 minus 20 (IP header) minus 8 (UDP header). In practice datagrams stay below the MTU to avoid fragmentation.
6. **How is reliability built on UDP when it is needed?** Sequence numbers, acknowledgements, timers and retransmission, usually with congestion control, in the application or in a library; QUIC is the standard example.
7. **How does a Java program detect that a UDP peer is unreachable?** Not reliably. A timeout (`setSoTimeout`) is the normal mechanism. A connected `DatagramSocket` may surface an ICMP port-unreachable message as `PortUnreachableException`.

KEY POINTS

UDP is a thin layer: ports, length and an optional checksum over IP.

- It preserves message boundaries and offers no reliability, ordering, flow control or congestion control.
- The checksum is the Internet checksum over a pseudo-header, header and payload; it is mandatory in IPv6.
- Suitable for small request-response exchanges (DNS), real-time media, telemetry and as the base of QUIC.
- The application must handle loss, duplication, reordering and the absence of congestion control.

3.3 TCP Segments and Connection Management

The Transmission Control Protocol (RFC 9293, which in 2022 replaced RFC 793 and its later amendments) provides a reliable, ordered, full-duplex byte stream between two endpoints over the unreliable datagram service of IP. Before data flows the endpoints establish state: initial sequence numbers, window sizes and option settings. The same state is removed in an orderly way at the end. The costs of these steps (one round trip to open, a lingering state after close, memory per connection) shape the behaviour of every HTTP client and server.

3.3.1 Concept and mechanism

The properties of TCP

- **Connection-oriented.** A connection has a defined set-up, data transfer and termination phase; both ends hold state.
- **Reliable.** Lost data is detected and retransmitted; corrupted segments are discarded through the checksum.
- **Ordered byte stream.** The receiver delivers bytes in the order in which they were sent. Message boundaries are not preserved: two writes of 100 bytes may arrive as one read of 200 bytes.
- **Full duplex.** Each direction has its own sequence number space, windows and close.
- **Flow controlled and congestion controlled.** The sender never exceeds the window advertised by the receiver, nor the window that the network path appears to permit.

The segment header

A TCP **segment** is the TCP header plus a payload of up to the maximum segment size. The fixed header is 20 bytes; options extend it up to 60 bytes.

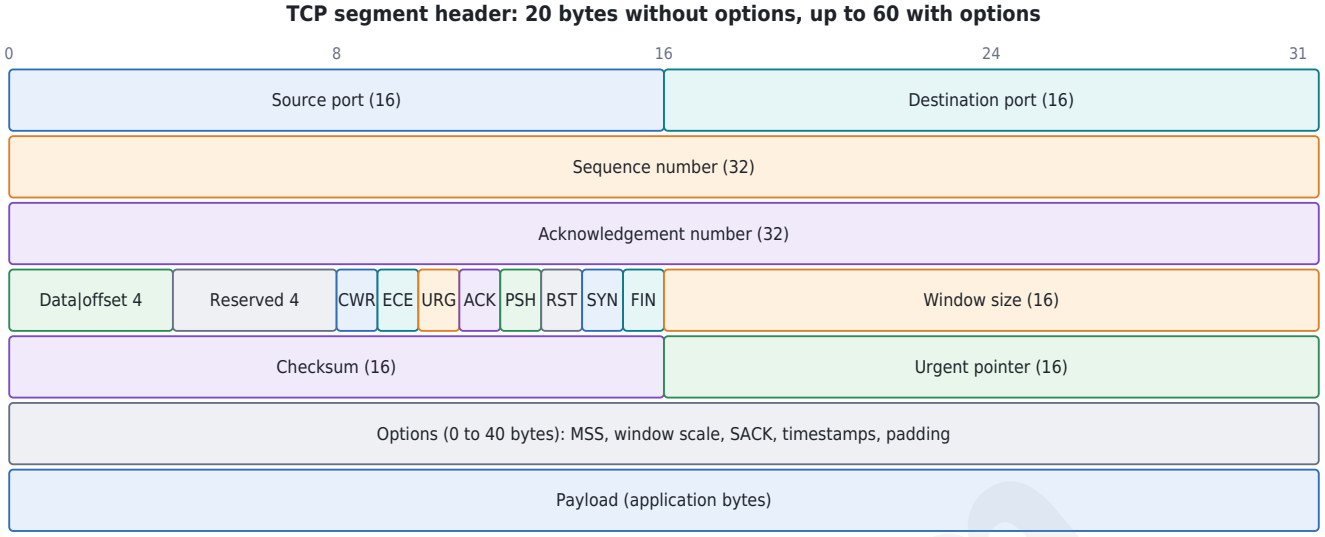


Figure 3.6. The TCP header places the 32-bit sequence number and acknowledgement number after the ports, followed by the data offset, eight flag bits, the window, the checksum and the urgent pointer; options extend the header when the data offset exceeds 5.

Field	Size	Meaning
Source port, destination port	16 bits each	Demultiplexing keys, see the chapter on ports.
Sequence number	32 bits	Position in the byte stream of the first payload byte of this segment. A SYN or FIN flag consumes one sequence number.
Acknowledgement number	32 bits	Valid when ACK is set: the next sequence number the sender of the segment expects to receive. It acknowledges every byte below this number (cumulative acknowledgement).
Data offset	4 bits	Header length in 32-bit words (5 to 15, so 20 to 60 bytes). It locates the start of the payload.
Reserved	4 bits	Zero. (An earlier experimental NS flag, RFC 3540, is historic.)
CWR, ECE	1 bit each	Explicit Congestion Notification (RFC 3168): ECE echoes a congestion signal, CWR confirms that the sender reduced its rate.
URG	1 bit	The urgent pointer is valid. Practically unused; its use is discouraged.
ACK	1 bit	The acknowledgement number is valid. Set in every segment except the first SYN.
PSH	1 bit	The sender asks the receiver to deliver buffered data to the application promptly; most stacks set it on the last segment of a write.
RST	1 bit	Abort the connection immediately.
SYN	1 bit	Synchronise sequence numbers (connection set-up).
FIN	1 bit	The sender has finished sending data (connection termination).
Window	16 bits	The number of bytes beyond the acknowledgement number that the sender of this segment is willing to receive (flow control). The window scale option multiplies it.
Checksum	16 bits	Internet checksum over a pseudo-header (addresses, protocol 6, TCP length), header and payload. Mandatory.
Urgent pointer	16 bits	Offset of urgent data, when URG is set.

TCP header fields.

Options

Options appear after the fixed header. The maximum header size of 60 bytes leaves 40 bytes for options; the SYN segments of a modern stack use about 20.

Option	Kind	Length	Purpose
`MSS (maximum segment size)`	2	4	Largest payload the sender of the option accepts in one segment; typically MTU minus 40 (1460 on Ethernet with IPv4). Exchanged in SYN segments only.
`Window scale (RFC 7323)`	3	3	Shift count from 0 to 14 that multiplies the 16-bit window field, allowing windows up to about 1 GiB. Negotiated in the SYN segments only.
`SACK permitted, SACK (RFC 2018)`	4, 5	2, variable	Permits and carries selective acknowledgements, see the reliability chapter.
`Timestamps (RFC 7323)`	8	10	TSval and TSecr give round-trip samples on every ACK and protect against wrapped sequence numbers (PAWS).
`NOP`	1	1	Padding used to align following options to 32 bits.

Common TCP options.

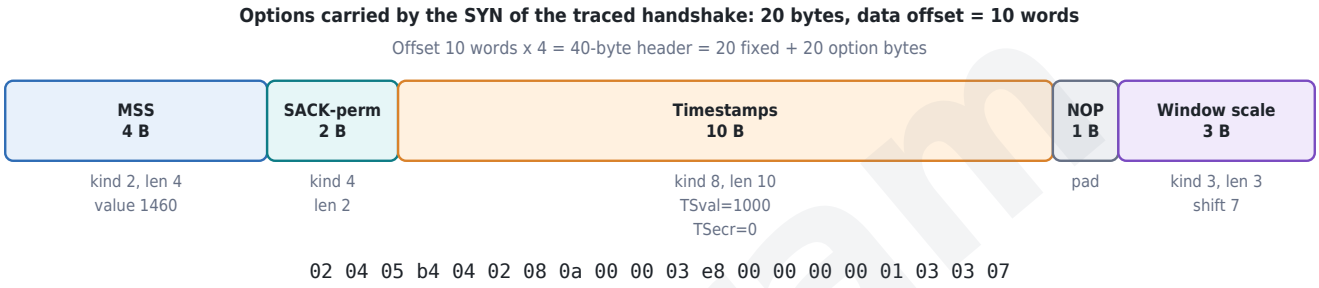


Figure 3.7. The 20 option bytes of the SYN in the traced handshake: MSS 1460, SACK permitted, timestamps, one padding byte and a window scale shift of 7. The data offset of 10 words follows from 20 + 20 bytes.

The MSS must not be confused with the MTU. The MTU is a property of a link (1500 bytes for Ethernet); the MSS is the largest TCP payload that fits into one IP packet of that size: 1500 - 20 (IPv4 header) - 20 (TCP header) = 1460 bytes. With the 12 bytes of the timestamp option in every data segment, the effective payload is 1448 bytes.

3.3.2 The three-way handshake

Both endpoints choose an **initial sequence number** (ISN), and each must know that the other has accepted it. Three segments accomplish this exchange. The client performs an **active open** (`connect()`), the server a **passive open** (`listen()` followed by `accept()`).

1. The client sends a segment with SYN set and its ISN (`seq = x`). It enters `SYN_SENT`.
2. The server answers with SYN and ACK set, its own ISN (`seq = y`) and `ack = x + 1`. It enters `SYN_RCVD`.
3. The client sends an ACK with `ack = y + 1` and enters `ESTABLISHED`. The server enters `ESTABLISHED` when this segment arrives.

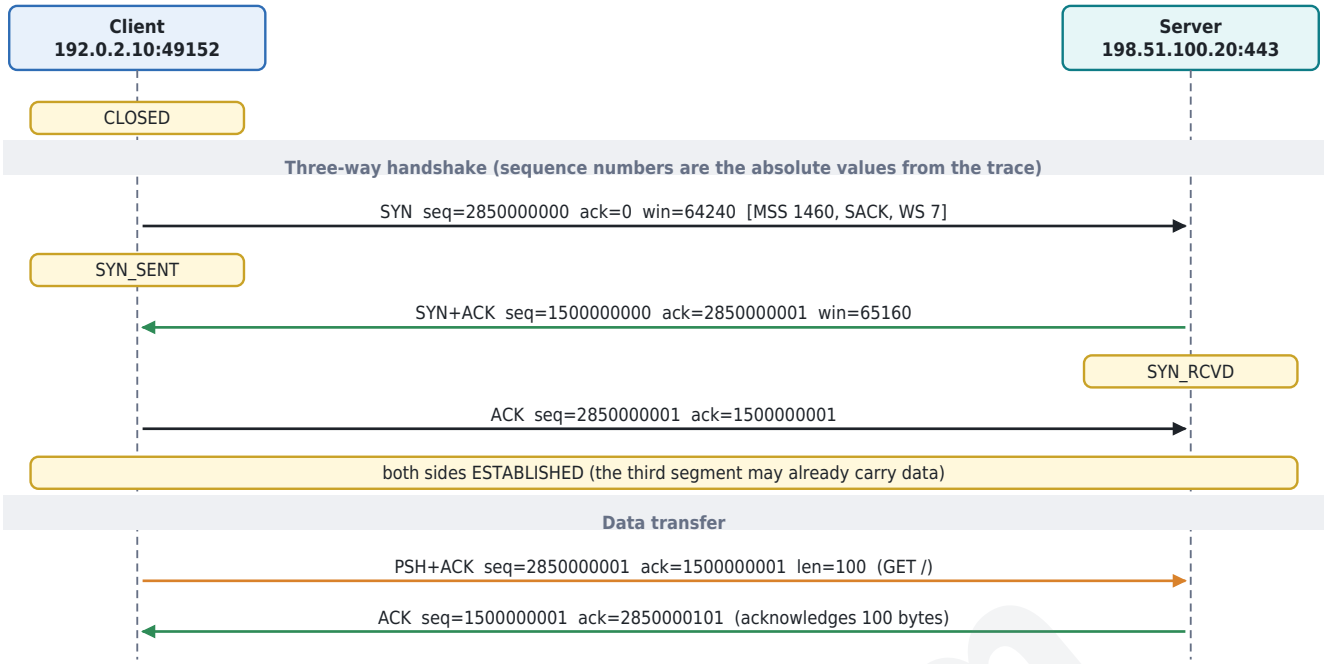


Figure 3.8. The handshake of the trace: the client 192.0.2.10:49152 sends SYN with seq 2850000000; the server answers with seq 1500000000 and ack 2850000001; the client acknowledges with ack 1500000001. The first data segment of 100 bytes is then acknowledged with ack 2850000101.

The SYN and FIN flags each occupy one position in the sequence space, which is why the acknowledgement is `x + 1` and not `x`. A pure ACK occupies none. The initial sequence number is not zero: it is derived from a clock that advances about every 4 microseconds, combined with a keyed hash of the 4-tuple (RFC 6528, now part of RFC 9293). The reason is to make delayed duplicates from an earlier incarnation of the same connection unlikely to fall into the window of the new connection, and to make blind spoofing of segments difficult.

A connection attempt costs one round trip before the client can send data (the third segment may carry the first data). If the SYN is lost the client retransmits it after 1 second (initial RTO, RFC 6298) with exponential backoff; Linux retries `tcp_syn_retries` times (6 by default on the build machine), which gives a total of 1 + 2 + 4 + 8 + 16 + 32 + 64 = 127 seconds before `connect()` fails with a timeout.

Sequence numbers and acknowledgements during data transfer

After the handshake the sequence number of a segment is the number of the first payload byte it carries. A segment with `seq = 2850000001` and 100 payload bytes covers bytes 2850000001 to 2850000100; the receiver acknowledges with `ack = 2850000101`, the number of the next byte it expects. Sequence numbers are 32 bits wide and wrap after 4 GiB of data, which is the reason for the timestamp option on fast links.

The listen backlog and the SYN flood

The kernel completes the handshake before the application calls `accept()`. Two queues exist.

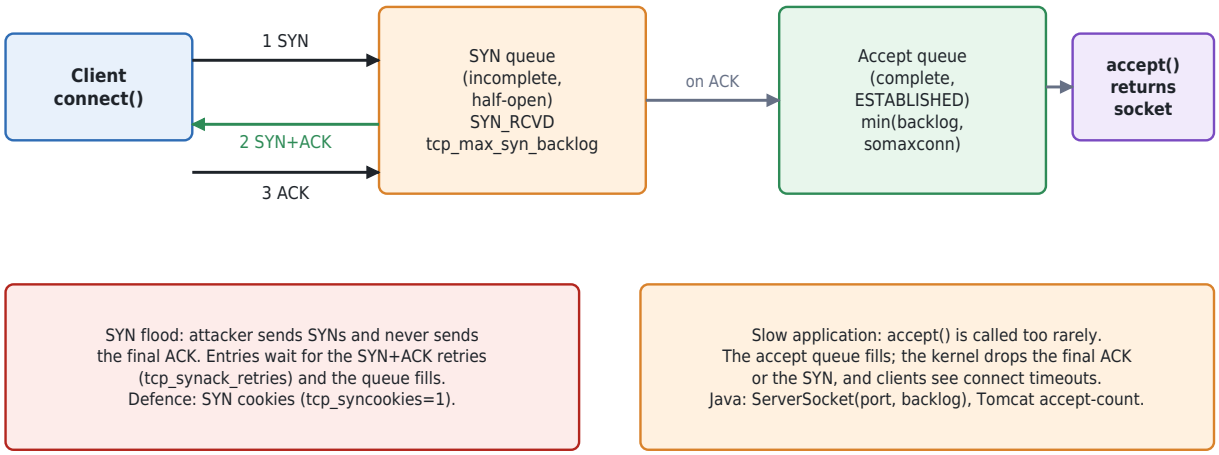


Figure 3.9. The kernel keeps half-open connections (SYN received, SYN+ACK sent) in the SYN queue and completed connections in the accept queue until the application calls accept(). Both queues can be exhausted, by an attack or by a slow application.

- The **SYN queue** holds entries in `SYN_RCVD`. An entry remains until the final ACK arrives or the SYN+ACK has been retransmitted `tcp_synack_retries` times (5 by default on the build machine). Its size is bounded by `net.ipv4.tcp_max_syn_backlog` (512 on the build machine).
- The **accept queue** holds established connections that wait for `accept()`. Its length is the smaller of the `backlog` argument of `listen()` and `net.core.somaxconn` (4096 on the build machine). Java `ServerSocket` uses a default backlog of 50; Tomcat uses `server.tomcat.accept-count` (default 100).

A **SYN flood** sends many SYN segments, usually with forged source addresses, and never completes the handshake. The SYN queue fills with entries that wait for retransmission timeouts, and legitimate clients cannot connect. **SYN cookies** (`net.ipv4.tcp_syncookies=1`, enabled on the build machine) remove the state: when the queue is full the server encodes the connection parameters (a hash of the tuple, a timestamp and a compressed MSS) into its own ISN, sends the SYN+ACK and forgets the connection. If the client's ACK returns `ack = ISN + 1`, the server validates the cookie and reconstructs the connection. The price is that some options (such as window scale and SACK) can be lost unless the timestamp option carries them.

3.3.3 Connection termination

Each direction of a TCP connection is closed independently by a FIN segment. A normal close therefore uses four segments, although the second and third can be combined into one.

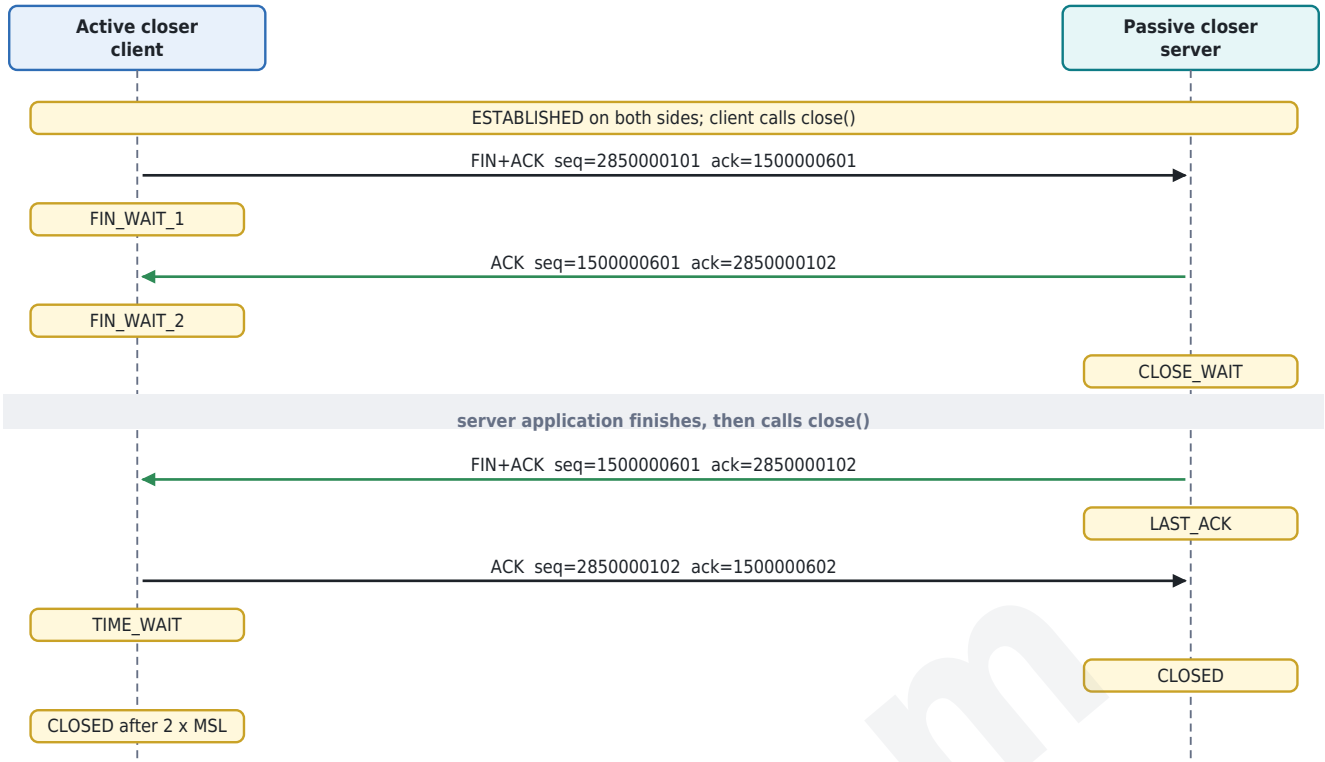


Figure 3.10. The active closer sends FIN and moves through FIN_WAIT_1 and FIN_WAIT_2; the passive closer stays in CLOSE_WAIT until its application calls close(), then sends its FIN from LAST_ACK. The active closer remains in TIME_WAIT for twice the maximum segment lifetime.

The side that calls `close()` first is the **active closer**. After its FIN is acknowledged it still reads data until the peer's FIN arrives. The **passive closer** receives the FIN, acknowledges it (the kernel does this) and enters `CLOSE_WAIT`. The kernel then waits for the application to read the end of stream (`read()` returns -1 in Java) and call `close()`. Only then does the passive side send its FIN, move to `LAST_ACK`, and release the connection after the final ACK.

Half-close

Because the directions are independent, an endpoint may declare that it has finished sending but continue to receive. `Socket.shutdownOutput()` in Java (the `shutdown(SHUT_WR)` system call) sends a FIN while the socket stays readable. The peer sees end of stream, can finish its processing and send a response, then close its own direction. This is the mechanism by which a client can mark the end of a request without any framing, as the `HalfClose` program in the chapter on sockets shows.

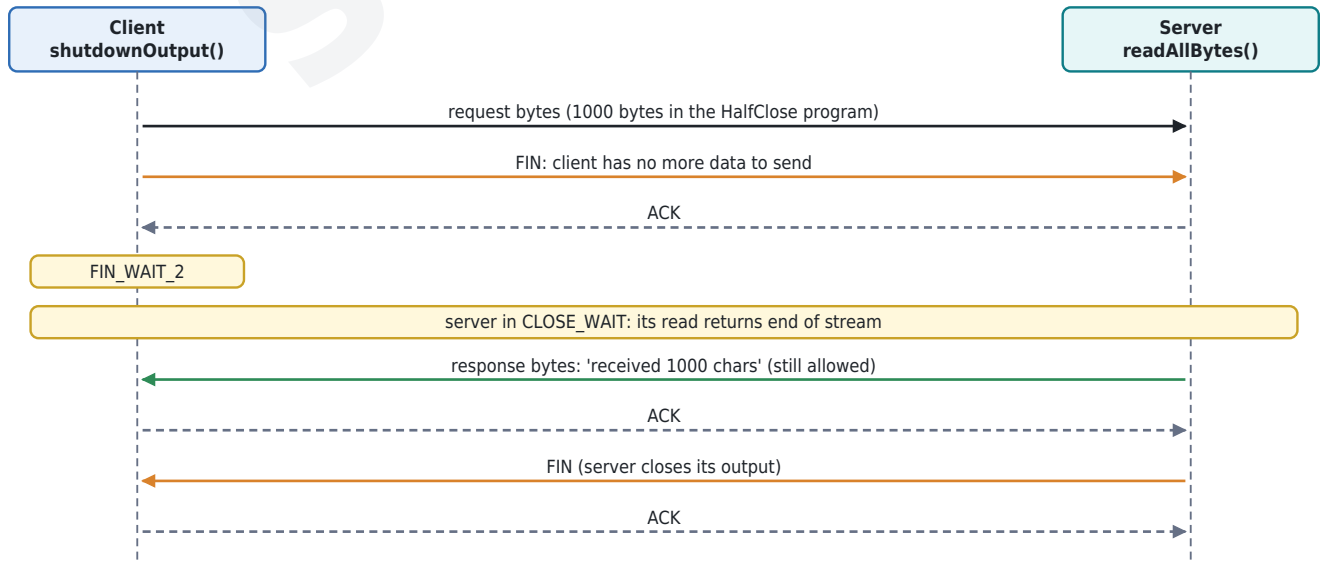


Figure 3.11. A half-close: after the client calls shutdownOutput() the FIN reaches the server, whose read returns end of stream; the server can still send its response, and then closes its own direction with a second FIN.

TIME_WAIT

The endpoint that sends the last ACK (normally the active closer) keeps the connection record in the `TIME_WAIT` state for 2 x MSL, where MSL is the maximum segment lifetime. RFC 9293 takes MSL as 2 minutes; Linux uses a fixed 60 seconds for the state itself (a compile-time constant that cannot be configured, unlike `tcp_fin_timeout`, which governs the `FIN_WAIT_2` state of orphaned sockets). The state has two purposes:

1. **Reliable termination.** If the final ACK is lost, the peer retransmits its FIN. The record must still exist so that the ACK can be repeated; otherwise the peer would receive RST and report an error.
2. **Isolation of incarnations.** Delayed segments of the old connection must expire before the same 4-tuple is reused, otherwise they could be accepted by the new connection.

`TIME_WAIT` is therefore a normal and correct state, not a fault. It becomes a problem when a client makes many short connections to one server address and port: each closed connection keeps its ephemeral port occupied for 60 seconds, and at 28232 ports the sustainable rate is about 28232 / 60 = 470 new connections per second to a single destination. Remedies in order of preference: reuse connections (persistent HTTP connections, pools); let the server close idle connections only when necessary, because the closer holds `TIME_WAIT`; widen the port range; use several destination addresses. The setting `net.ipv4.tcp_tw_reuse` allows a new outgoing connection to reuse a `TIME_WAIT` slot when timestamps make it safe (value 2 on the build machine: enabled for loopback traffic only). Shortening `TIME_WAIT` by disabling the protection is not recommended.

CLOSE_WAIT

`CLOSE_WAIT` means that the peer has closed its direction and the local application has not yet called `close()`. The state has no timeout. A growing number of `CLOSE_WAIT` sockets is a defect in the local application: connections are not closed after errors or after end of stream, typically because a `Socket`, an `InputStream` or an HTTP response body is not closed (missing try-with-resources) or because a worker thread holds the connection while it is blocked. The file descriptors accumulate until the process reports `Too many open files`.

RST

A segment with RST set aborts the connection without the closing sequence; both sides discard their state, and any buffered unread data is lost. RST is sent when:

- a SYN arrives for a port without a listener (the client sees `Connection refused`);
- a segment arrives for a connection that the host does not know (for example after a server reboot, or after a middlebox dropped the state);
- an application calls `close()` while unread data remains in the receive buffer (Linux), or sets `SO_LINGER` to zero (`Socket.setSoLinger(true, 0)`);
- a firewall or load balancer rejects or resets a connection, for example because its idle timeout expired.

In Java, a read after RST fails with `SocketException: Connection reset`, whereas a read after a FIN returns -1. The two outcomes are demonstrated in the hands-on section.

sreeram

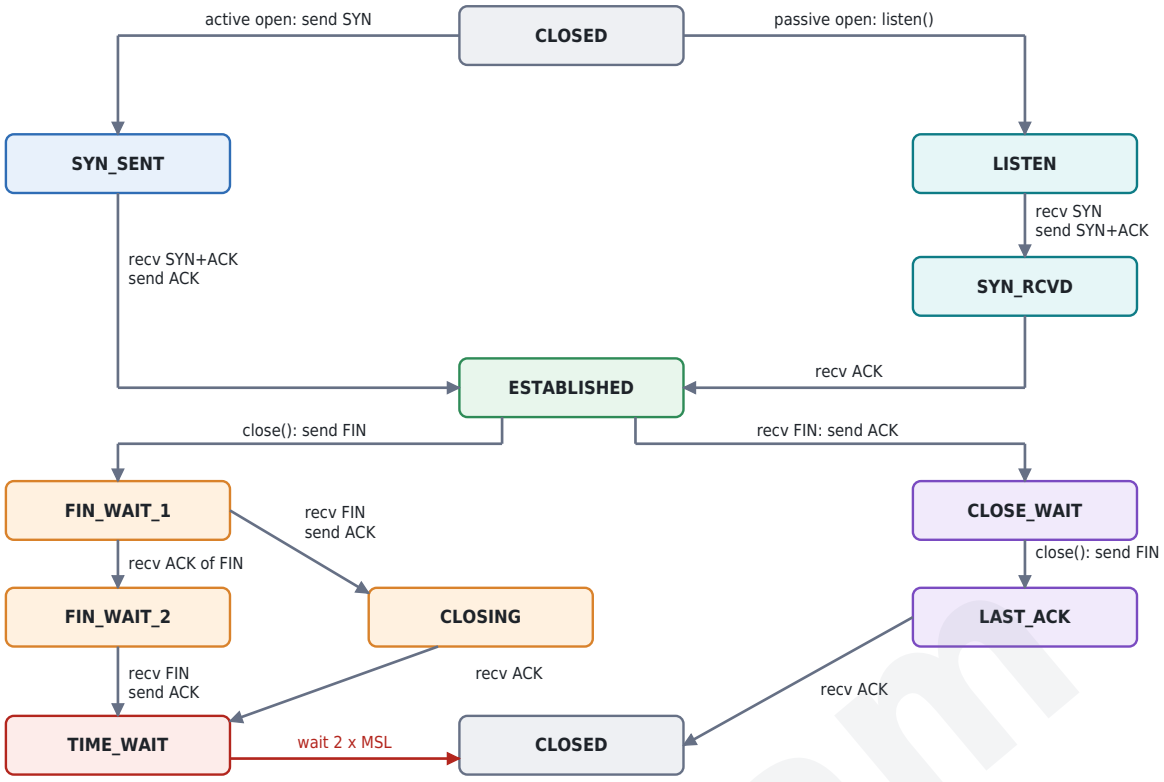


Figure 3.12. The TCP state machine. The left branch below ESTABLISHED is the active close (FIN_WAIT_1, FIN_WAIT_2 or CLOSING, then TIME_WAIT); the right branch is the passive close (CLOSE_WAIT, LAST_ACK). A FIN that arrives together with the ACK of the local FIN moves FIN_WAIT_1 directly to TIME_WAIT.

State	Meaning	Persistent when
LISTEN	Waiting for connection requests	Normal for servers
SYN_SENT	SYN sent, waiting for SYN+ACK	The peer is down or filtered; ends after the retry limit
SYN_RCVD	SYN received, SYN+ACK sent	SYN flood or lost final ACKs
ESTABLISHED	Data transfer	Normal; idle connections remain here without keep-alive
FIN_WAIT_1 and FIN_WAIT_2	Local FIN sent; waiting for the ACK, then for the peer's FIN	FIN_WAIT_2 accumulates when the peer never closes its direction; Linux limits the orphaned case with `tcp_fin_timeout`
CLOSE_WAIT	Peer sent FIN; the local application has not closed	Application bug (leak)
LAST_ACK	Local FIN sent after CLOSE_WAIT; waiting for the final ACK	Brief
TIME_WAIT	Final ACK sent; waiting 2 x MSL	Normal on the active closer; excessive with short-lived connections
CLOSING	Both sides sent FIN simultaneously	Brief

TCP states and the conditions under which they persist.

3.3.4 Worked trace

Building the SYN segment

The SYN of the handshake diagram can be constructed byte by byte. Source `192.0.2.10:49152`, destination `198.51.100.20:443`, ISN 2850000000, window 64240, and the 20 option bytes shown earlier. The script below computes the checksum over the pseudo-header and verifies it (real output follows).

tcphdr.py PYTHON

```python
import struct, socket
def csum(b):
    if len(b)%2: b+=b'\0'
    s=sum(struct.unpack('!%dH'%(len(b)//2),b))
    while s>>16: s=(s&0xffff)+(s>>16)
    return (~s)&0xffff
src=socket.inet_aton('192.0.2.10'); dst=socket.inet_aton('198.51.100.20')
opts=(struct.pack('!BBH',2,4,1460) + b'\x04\x02'
      + struct.pack('!BBII',8,10,1000,0) + b'\x01' + struct.pack('!BBB',3,3,7))
print('options len',len(opts))
off=(20+len(opts))//4
seq=2850000000
hdr=lambda ck: struct.pack('!HHIIBBHHH',49152,443,seq,0,off<<4,0x02,64240,ck,0)+opts
pseudo=src+dst+struct.pack('!BBH',0,6,20+len(opts))
ck=csum(pseudo+hdr(0))
print('data offset',off,'checksum 0x%04x'%ck)
h=hdr(ck)
print(h.hex(' '))
print('verify',hex(csum(pseudo+h)))
```

Header bytes of the SYN segment TERMINAL

```
$ python3 tcphdr.py
options len 20
data offset 10 checksum 0x64b9
c0 00 01 bb a9 df 8c 80 00 00 00 00 a0 02 fa f0 64 b9 00 00
02 04 05 b4 04 02 08 0a 00 00 03 e8 00 00 00 00 01 03 03 07
verify 0x0
```

Bytes	Hex	Meaning
0-1	`c0 00`	Source port 49152
2-3	`01 bb`	Destination port 443
4-7	`a9 df 8c 80`	Sequence number 2850000000
8-11	`00 00 00 00`	Acknowledgement number 0 (ACK flag not set)
12	`a0`	Data offset 10 (40 bytes), reserved bits 0
13	`02`	Flags: only SYN (0x02)
14-15	`fa f0`	Window 64240
16-17	`64 b9`	Checksum
18-19	`00 00`	Urgent pointer
20-39	`02 04 05 b4 04 02 08 0a ... 03 03 07`	Options: MSS 1460, SACK permitted, timestamps (TSval 1000, TSecr 0), NOP, window scale 7

Decoding of the 40 header bytes.

The complete exchange

The connection of the diagrams exchanges the following segments. Sequence numbers are absolute. The client sends a 100-byte request; the server returns a 600-byte response in one or more segments, and the client closes first.

No.	Direction	Header values	Effect
1	C to S	`SYN seq=2850000000`	Client SYN_SENT
2	S to C	`SYN+ACK seq=1500000000 ack=2850000001`	Server SYN_RCVD
3	C to S	`ACK seq=2850000001 ack=1500000001`	Both ESTABLISHED
4	C to S	`PSH+ACK seq=2850000001 ack=1500000001 len=100`	Request delivered
5	S to C	`ACK seq=1500000001 ack=2850000101`	100 bytes acknowledged
6	S to C	`PSH+ACK seq=1500000001 ack=2850000101 len=600`	Response of 600 bytes (numbers 1500000001 to 1500000600)
7	C to S	`FIN+ACK seq=2850000101 ack=1500000601`	Client FIN_WAIT_1; the FIN consumes number 2850000101

No.	Direction	Header values	Effect
8	S to C	ACK seq=1500000601 ack=2850000102	Client FIN_WAIT_2; server CLOSE_WAIT
9	S to C	FIN+ACK seq=1500000601 ack=2850000102	Server LAST_ACK, after its application closed
10	C to S	ACK seq=2850000102 ack=1500000602	Client TIME_WAIT; server CLOSED

Segments of the trace.

The sequence numbers add up: the server sends 600 bytes, so its next number is 1500000601; the FIN consumes that number and the final acknowledgement is therefore 1500000602.

3.3.5 Hands-on

Observing states from Linux

A Python script of about 25 lines (not listed; it reads `/proc/net/tcp` and maps the state codes to names) opens a listener, connects a client, closes the client first, and prints the entries of `/proc/net/tcp` for port 7000 at each step. The state codes are translated by the script (real output):

States after each operation TERMINAL

```
$ python3 states.py
-- after listen
  local :7000  remote :0     LISTEN
-- after connect+accept
  local :7000  remote :0     LISTEN
  local :39270 remote :7000  ESTABLISHED
  local :7000  remote :39270 ESTABLISHED
-- client called close(); server has not yet closed
  local :7000  remote :0     LISTEN
  local :39270 remote :7000  FIN_WAIT2
  local :7000  remote :39270 CLOSE_WAIT
-- server called close() too
  local :7000  remote :0     LISTEN
  local :39270 remote :7000  TIME_WAIT
```

The third block shows the passive closer in `CLOSE_WAIT` (the server application has not yet closed its accepted socket) and the active closer in `FIN_WAIT_2`. After the server closes, the client side, which closed first, holds `TIME_WAIT`.

Counting TIME_WAIT

A client and a server on loopback perform 200 short connections in which the server closes first. The program then counts the states for port 7001:

TIME_WAIT accumulation (real output) TERMINAL

```
$ python3 tw.py
{'0A': 1, 'TIME_WAIT': 200}
```

Code `0A` is `LISTEN`. Each of the 200 closed connections left one `TIME_WAIT` record, here on the server side because the server closed first. A server that closes connections after each response therefore carries the `TIME_WAIT` records, and a client that closes first carries them.

sreeram

FIN versus RST in Java

Opts.java JAVA

```java
import java.io.*;
import java.net.*;
import jdk.net.ExtendedSocketOptions;

public class Opts {
    public static void main(String[] args) throws Exception {
        try (ServerSocket ss = new ServerSocket(0, 50, InetAddress.getLoopbackAddress());
             Socket c = new Socket(InetAddress.getLoopbackAddress(), ss.getLocalPort());
             Socket a = ss.accept()) {
            System.out.println("TCP_NODELAY      = " + c.getTcpNoDelay());
            System.out.println("SO_KEEPALIVE     = " + c.getKeepAlive());
            System.out.println("SO_TIMEOUT       = " + c.getSoTimeout() + " (0 = infinite)");
            System.out.println("SO_LINGER        = " + c.getSoLinger() + " (-1 = disabled)");
            System.out.println("SO_RCVBUF        = " + c.getReceiveBufferSize());
            System.out.println("SO_SNDBUF        = " + c.getSendBufferSize());
            System.out.println("server SO_REUSEADDR = " + ss.getReuseAddress());
            System.out.println("server SO_RCVBUF    = " + ss.getReceiveBufferSize());
            c.setKeepAlive(true);
            c.setOption(ExtendedSocketOptions.TCP_KEEPIDLE, 60);
            c.setOption(ExtendedSocketOptions.TCP_KEEPINTERVAL, 10);
            c.setOption(ExtendedSocketOptions.TCP_KEEPCOUNT, 3);
            System.out.println("keepalive idle/interval/count = "
                + c.getOption(ExtendedSocketOptions.TCP_KEEPIDLE) + "/"
                + c.getOption(ExtendedSocketOptions.TCP_KEEPINTERVAL) + "/"
                + c.getOption(ExtendedSocketOptions.TCP_KEEPCOUNT));
        }
        // RST via SO_LINGER 0
        try (ServerSocket ss = new ServerSocket(0, 5, InetAddress.getLoopbackAddress())) {
            Thread t = Thread.ofPlatform().start(() -> {
                try (Socket s = ss.accept()) {
                    s.setSoLinger(true, 0);   // close() sends RST instead of FIN
                } catch (IOException e) { e.printStackTrace(); }
            });
            try (Socket c = new Socket(InetAddress.getLoopbackAddress(), ss.getLocalPort())) {
                t.join();
                Thread.sleep(100);
                try { c.getInputStream().read(); }
                catch (IOException e) { System.out.println("after peer RST: " + e.getMessage()); }
            }
        }
        // FIN gives end of stream (-1), not an exception
        try (ServerSocket ss = new ServerSocket(0, 5, InetAddress.getLoopbackAddress())) {
            Thread t = Thread.ofPlatform().start(() -> {
                try (Socket s = ss.accept()) { } catch (IOException e) { e.printStackTrace(); }
            });
            try (Socket c = new Socket(InetAddress.getLoopbackAddress(), ss.getLocalPort())) {
                t.join();
                System.out.println("after peer FIN, read() = " + c.getInputStream().read());
            }
        }
        // UDP too large
        try (DatagramSocket d = new DatagramSocket()) {
            d.send(new DatagramPacket(new byte[70000], 70000, InetAddress.getLoopbackAddress(), 9));
        } catch (IOException e) {
            System.out.println("UDP 70000 bytes: " + e.getClass().getSimpleName()
                + ": " + e.getMessage());
        }
        try (DatagramSocket d = new DatagramSocket()) {
            d.send(new DatagramPacket(new byte[65507], 65507, InetAddress.getLoopbackAddress(), 9));
            System.out.println("UDP 65507 bytes: sent");
        }
    }
}
```

Run Opts.java TERMINAL

```
$ java Opts.java
TCP_NODELAY      = false
SO_KEEPALIVE     = false
SO_TIMEOUT       = 0 (0 = infinite)
SO_LINGER        = -1 (-1 = disabled)
SO_RCVBUF        = 65536
SO_SNDBUF        = 1969920
```

Run Opts.java (continued) TERMINAL

```
server SO_REUSEADDR = true
server SO_RCVBUF    = 65536
keepalive idle/interval/count = 60/10/3
after peer RST: Connection reset
after peer FIN, read() = -1
UDP 70000 bytes: SocketException: Message too long
UDP 65507 bytes: sent
```

What to notice: after the peer sent RST (`setSoLinger(true, 0)` followed by `close()`), the read fails with `Connection reset`; after an orderly close, the read returns -1. An application that treats only exceptions as failures misses the orderly close, and the connection then remains in `CLOSE_WAIT` if the application neither reads the end of stream nor closes the socket. The first lines also show the defaults of a new socket: keep-alive off, no timeout, Nagle enabled (`TCP_NODELAY` false) and the buffer sizes of the build machine.

Connection refused and port probes

The `nc` option `-z` probes a port without sending data. The listener on 9300 completes the handshake; port 9301 answers the SYN with RST (real output):

A probe of an open and a closed port TERMINAL

```
$ nc -zv 127.0.0.1 9300
Connection to 127.0.0.1 9300 port [tcp/*] succeeded!
$ nc -zv 127.0.0.1 9301
nc: connect to 127.0.0.1 port 9301 (tcp) failed: Connection refused
```

`curl -v` against a listening server shows the same event at the application level. The line `Connected to 127.0.0.1 (127.0.0.1) port 8088` is printed after the three-way handshake completed, and `connect=0.000105s` (the `time_connect` variable) is the time of that handshake on loopback. Against a closed port curl reports `Failed to connect ... Connection refused` immediately. When the server host silently drops packets the same command waits until the connect timeout instead; this difference between a refusal (RST, immediate) and a timeout (no answer) is the first diagnostic split in troubleshooting.

3.3.6 Failure modes

Symptom	Mechanism	Diagnosis and remedy
Connect timeout after many seconds	SYN or SYN+ACK is dropped (firewall, security group, full accept queue, wrong route)	Compare with `Connection refused`; check security groups, the listen backlog (`accept-count`) and `ss -ltn` for Recv-Q at the backlog limit
Many `TIME_WAIT` sockets, `Cannot assign requested address`	Short-lived connections from one client to one destination	Connection pooling and keep-alive; verify that the client library reuses connections
Growing `CLOSE_WAIT` count, `Too many open files`	The application does not close sockets after the peer closed	Fix resource handling; take a thread dump to find threads holding connections
`Connection reset by peer`	RST from the peer or a middlebox (idle timeout, server restart, closed connection reused from a pool)	Set pool idle times below the server and load balancer idle timeout; retry idempotent requests
SYN flood symptoms: many `SYN_RECV` entries	Handshakes never complete	Enable SYN cookies; rate limit at the edge; use a load balancer or CDN that absorbs the flood
Half-open connection (one side believes the connection exists)	Peer crashed or a NAT mapping expired without FIN or RST	Application timeouts; TCP keep-alive; heartbeats

Connection-management failures in production.

WATCH OUT

Closing a pooled connection on the server side before the client notices causes a reset on the next request. If a server or load balancer closes idle connections after 60 seconds and the client pool keeps them for 120 seconds, the client may send a request on a connection that the peer has already abandoned. The first segment is answered with RST or the write fails. The idle timeout of the client pool must be smaller than that of every device on the path.

3.3.7 Questions to expect

1. **Why does TCP use a three-way handshake and not two segments?** Both sides must choose an initial sequence number and have it acknowledged. Two segments confirm only one direction. The third segment confirms the server's ISN and lets the server discard connection attempts that are old duplicates of a SYN, because the client answers a stale SYN+ACK with RST.
2. **What are the sequence and acknowledgement numbers in the second segment?** The server's ISN `y` as sequence number and `x + 1` as acknowledgement number, where `x` is the client ISN. The SYN consumes one number.
3. **Why is the ISN random-looking instead of zero?** To prevent segments of an earlier connection with the same tuple from being accepted and to resist off-path spoofing. RFC 6528 specifies a clock plus a keyed hash of the tuple.
4. **Explain the four-way termination and why it is not three.** Each direction closes separately. The passive side may still send data after it acknowledged the FIN, so the ACK and the FIN are separate unless the application closes at once.
5. **What is TIME_WAIT, who holds it, and why is it needed?** The active closer holds it for 2 x MSL (60 s on Linux). It allows retransmission of the last ACK and lets old segments expire before the tuple is reused.
6. **A server shows thousands of CLOSE_WAIT sockets. What is wrong?** The application does not call `close()` after the peer has closed. This is a resource leak in the application, not a network problem.
7. **What is a SYN flood and how is it mitigated?** Many SYNs without completing the handshake exhaust the SYN queue. SYN cookies avoid per-connection state until the final ACK; filtering and rate limiting at the edge complement them.
8. **Differentiate Connection refused, Connection timed out and Connection reset.** Refused: the SYN was answered with RST, so no process listens. Timed out: no answer at all, usually a drop by a firewall or a dead host. Reset: an established connection was aborted with RST.
9. **What does half-close mean and how does Java expose it?** One direction is closed while the other stays open. `Socket.shutdownOutput()` sends the FIN; `shutdownInput()` discards incoming data locally.

INTERVIEW ANGLE

The server closes every connection after sending the response and the monitoring shows 30000 sockets in TIME_WAIT. Is that a defect? It is the expected consequence of the server being the active closer, and it is harmless as long as the table has capacity: a TIME_WAIT record on the server consumes memory but, unlike on a client, does not consume an ephemeral port toward one destination. The problem appears on the client side of high-rate short connections. The solution there is persistent connections rather than lowering the timer.

KEY POINTS

A TCP connection starts with SYN, SYN+ACK, ACK and ends with a FIN in each direction; SYN and FIN each consume one sequence number.

- The header carries ports, 32-bit sequence and acknowledgement numbers, flags, window, checksum and options (MSS, window scale, SACK, timestamps).
- The listen backlog (SYN queue, accept queue) and SYN cookies determine behaviour under overload and under SYN flood.
- TIME_WAIT lasts 2 x MSL on the active closer and is correct behaviour; CLOSE_WAIT without end is an application leak.
- RST aborts a connection; Java reports it as `Connection reset`, whereas an orderly FIN gives end of stream (-1).

3.4 TCP Reliability

IP may lose, duplicate, delay and reorder packets. TCP converts this service into a reliable byte stream with a small set of mechanisms: sequence numbers identify every byte, cumulative acknowledgements report progress, a retransmission timer repairs loss that no other signal reveals, duplicate acknowledgements trigger earlier repair, and selective acknowledgements describe exactly which data arrived. The sender keeps a copy of every unacknowledged byte, and the receiver reassembles the stream in order.

3.4.1 Concept and mechanism

Acknowledgements

TCP acknowledgements are **cumulative**: `ack = n` states that all bytes with numbers below `n` have been received in order and that `n` is the next byte expected. A cumulative acknowledgement has two consequences. A lost acknowledgement is harmless if a later acknowledgement arrives, because the later one covers the earlier data. And the receiver cannot acknowledge data that follows a gap; it can only repeat the number of the first missing byte.

To reduce overhead, receivers use **delayed acknowledgements** (RFC 1122, RFC 5681): an ACK is sent for at least every second full-sized segment, and no later than 500 ms after the first unacknowledged segment arrived (Linux uses a shorter timer, typically around 40 ms). A segment that arrives out of order or fills a gap is acknowledged at once, because the sender needs the information.

An acknowledgement that carries no data is not itself acknowledged and not retransmitted. When the receiver sends data in the opposite direction, the acknowledgement is **piggybacked** on that data segment.

The retransmission timer

The sender starts a timer for the oldest unacknowledged segment. If the timer expires before that segment is acknowledged, the segment is retransmitted and the timer value is doubled (**exponential backoff**). The timer value is the **retransmission timeout** (RTO). It must exceed the round-trip time (RTT), otherwise segments are retransmitted needlessly (spurious retransmission); it must not greatly exceed it, otherwise a loss stalls the connection for a long time. Because the RTT varies with queueing, the sender cannot use a constant.

RFC 6298 defines the estimator. The sender measures RTT samples `R` (the time between sending a segment and receiving its acknowledgement) and maintains two variables: the smoothed round-trip time `SRTT` and the round-trip time variation `RTTVAR`.

DEFINITION

RTO estimation (RFC 6298, alpha = 1/8, beta = 1/4, K = 4, G = clock granularity) First sample R: SRTT = R; RTTVAR = R / 2; RTO = SRTT + max(G, 4 x RTTVAR). Later samples R': RTTVAR = (1 - beta) x RTTVAR + beta x |SRTT - R'| (computed first, with the old SRTT); then SRTT = (1 - alpha) x SRTT + alpha x R'; RTO = SRTT + max(G, 4 x RTTVAR). Initial RTO before any sample: 1 second. A computed RTO below 1 second SHOULD be rounded up to 1 second. On timer expiry the RTO is doubled.

Three rules complete the algorithm. **Karn's algorithm** forbids taking an RTT sample from a segment that was retransmitted, because the acknowledgement could belong to either copy (the timestamp option removes the ambiguity and permits sampling). The timer is restarted whenever new data is acknowledged. After a timeout, the congestion window is reduced (see the congestion control chapter), so a timeout is a more severe event than a fast retransmission.

Linux does not apply the 1-second floor. It uses a minimum RTO of 200 ms, and a maximum of 120 seconds, which is why datacentre connections recover from tail losses within a few hundred milliseconds.

3.4.2 Worked trace: RTO estimation

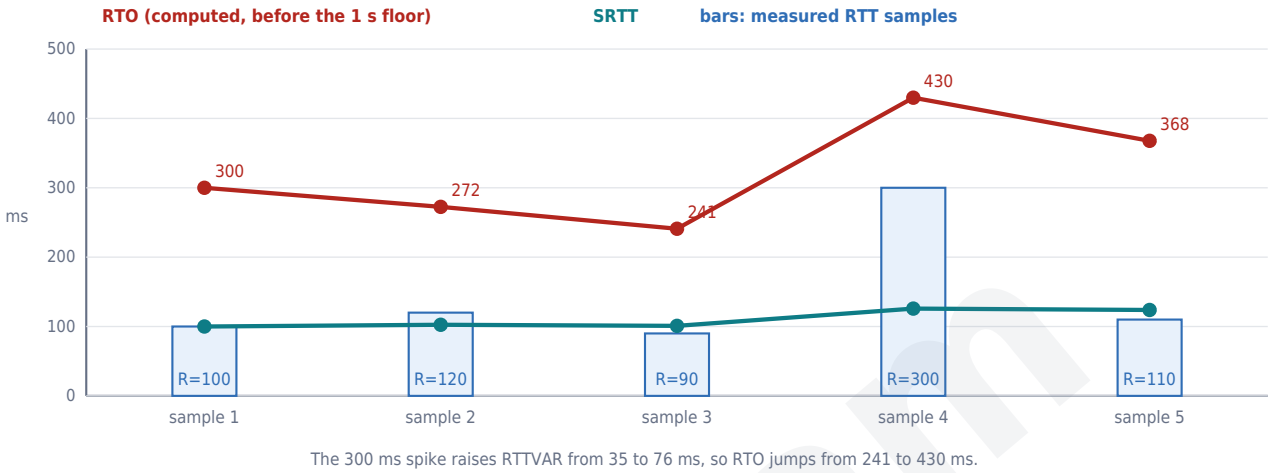
The path has a base RTT of about 100 ms. Five samples are measured: 100, 120, 90, 300 (a queueing spike) and 110 milliseconds. The script reproduces the algorithm with the RFC constants (real output):

rto.py PYTHON

```
samples=[100,120,90,300,110]
srtt=rttvar=None
print('n  R     SRTT     RTTVAR   RTO(calc)')
for i,r in enumerate(samples,1):
    if srtt is None:
        srtt=r; rttvar=r/2
    else:
        rttvar=0.75*rttvar+0.25*abs(srtt-r)
        srtt=0.875*srtt+0.125*r
    print('%d  %-4d  %7.3f  %7.3f  %7.3f'%(i,r,srtt,rttvar,srtt+4*rttvar))
print('backoff from 200ms:',[200*2**k for k in range(6)])
```

RTO estimation, values in ms TERMINAL

```
$ python3 rto.py
n  R     SRTT     RTTVAR   RTO(calc)
1  100   100.000   50.000  300.000
2  120   102.500   42.500  272.500
3  90    100.938   35.000  240.938
4  300   125.820   76.016  429.883
5  110   123.843   60.967  367.710
backoff from 200ms: [200, 400, 800, 1600, 3200, 6400]
```



The 300 ms spike raises RTTVAR from 35 to 76 ms, so RTO jumps from 241 to 430 ms.

Figure 3.13. The measured samples (bars), the smoothed RTT (teal) and the computed RTO (red). The single 300 ms sample raises RTTVAR from 35 to 76 ms and the RTO from 241 to 430 ms; it moves SRTT only by 25 ms.

The calculation of the first two rows shows the mechanism.

1. First sample: `R = 100`. `SRTT = 100`, `RTTVAR = 100 / 2 = 50`, `RTO = 100 + 4 x 50 = 300` ms.
2. Second sample: `R' = 120`. `RTTVAR = 0.75 x 50 + 0.25 x |100 - 120| = 37.5 + 5 = 42.5`; then `SRTT = 0.875 x 100 + 0.125 x 120 = 87.5 + 15 = 102.5`; `RTO = 102.5 + 4 x 42.5 = 272.5` ms.
3. The fourth sample, `R' = 300`, gives `RTTVAR = 0.75 x 35 + 0.25 x |100.938 - 300| = 26.25 + 49.766 = 76.016` and `SRTT = 0.875 x 100.938 + 0.125 x 300 = 125.82`, so `RTO = 125.82 + 4 x 76.016 = 429.883` ms.
4. If the RFC floor of 1 second applied, every computed RTO above would be raised to 1000 ms. With the Linux floor of 200 ms none of them changes.
5. When a segment is lost and the timer expires with RTO = 200 ms, the timer doubles for each further failure: 200, 400, 800, 1600, 3200, 6400 ms. Six consecutive losses of the same segment cost 12.6 seconds of waiting.

The estimator is deliberately conservative: the variance term is weighted four times, so a sudden increase in delay does not immediately cause spurious timeouts, and the smoothed value follows the mean slowly.

3.4.3 Fast retransmit and duplicate acknowledgements

Waiting for the timer is wasteful when later segments still arrive. A receiver that gets a segment beyond a gap sends an immediate acknowledgement that repeats the old number: a **duplicate ACK**. When the sender receives **three duplicate ACKs** (four identical acknowledgements in total) for the same number, it concludes that the segment starting at that number was lost and retransmits it without waiting for the timer. This mechanism is **fast retransmit** (RFC 5681). The threshold of three tolerates ordinary reordering of up to two positions without a false alarm.

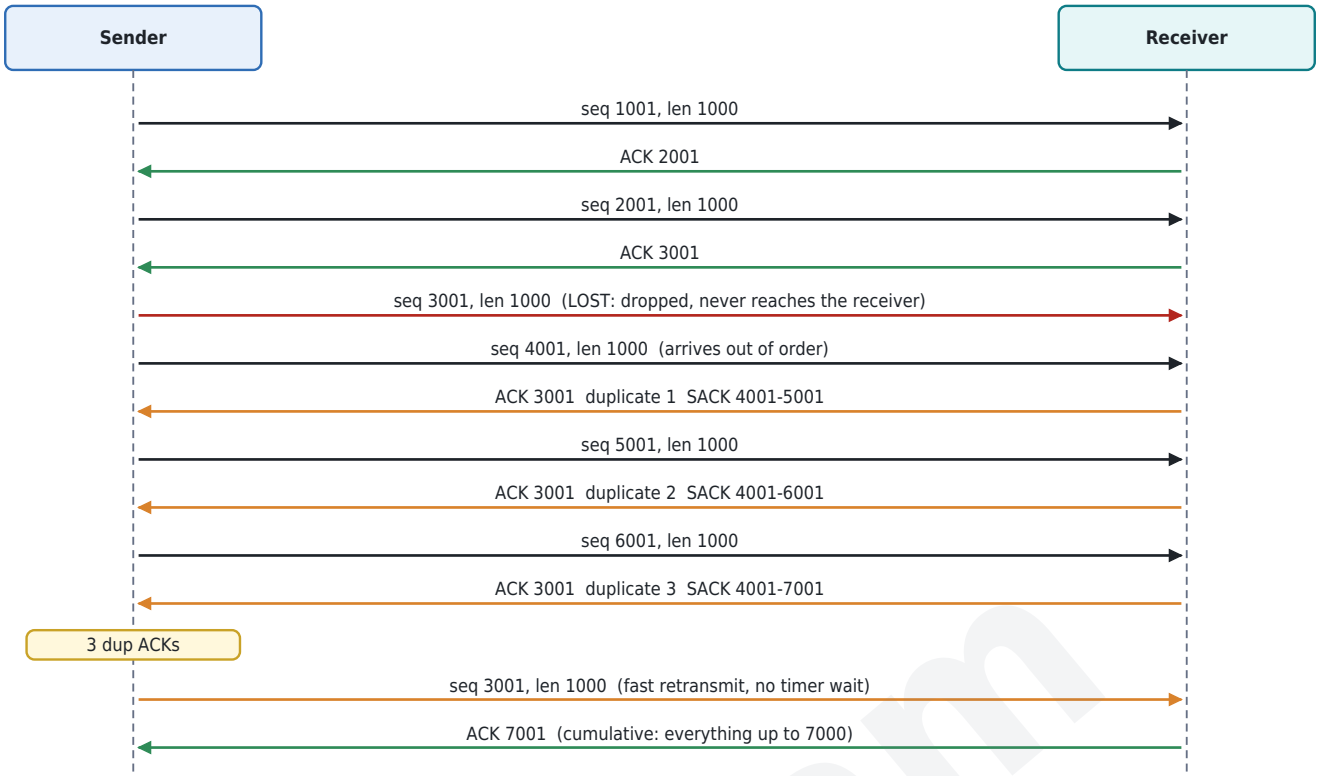


Figure 3.14. Fast retransmit with SACK: segment 3001-4000 is lost. Each later segment produces a duplicate ACK 3001 with a growing SACK block. After the third duplicate the sender retransmits 3001 at once; the receiver then acknowledges 7001 cumulatively.

No.	Segment	Receiver state	Receiver sends
1	`seq 1001 len 1000`	next expected 2001	ACK 2001
2	`seq 2001 len 1000`	next expected 3001	ACK 3001
3	`seq 3001 len 1000 (lost)`	none	none
4	`seq 4001 len 1000`	gap at 3001; 4001-5000 buffered	ACK 3001 (dup 1), SACK 4001-5001
5	`seq 5001 len 1000`	4001-6000 buffered	ACK 3001 (dup 2), SACK 4001-6001
6	`seq 6001 len 1000`	4001-7000 buffered	ACK 3001 (dup 3), SACK 4001-7001
7	`seq 3001 len 1000 (fast retransmit)`	gap filled; next expected 7001	ACK 7001

Trace of the fast retransmit (MSS = 1000 bytes, relative sequence numbers).

After a fast retransmit the sender halves its congestion window instead of collapsing it to one segment, because the arrival of duplicate ACKs proves that the network still delivers packets (see the next chapter but one). The recovery completes at the first acknowledgement that covers the retransmitted data (ACK 7001 here). Total recovery time is about one RTT, against at least one RTO when the timer is the only signal.

Fast retransmit fails when too few segments follow the loss: a loss at the end of a transfer or of a small request produces no duplicate ACKs. Modern stacks add a **tail loss probe** (RACK-TLP, RFC 8985), which sends a probe after about two RTTs without an acknowledgement and uses the time of delivery of later segments to detect loss instead of counting duplicates.

3.4.4 Selective acknowledgement

Cumulative acknowledgements reveal only the first hole. After a burst loss the sender learns one lost segment per round trip, or it retransmits everything after the hole. The **SACK** option (RFC 2018), negotiated in the SYN, lets the receiver list up to four (three with timestamps) blocks of data that arrived beyond the cumulative acknowledgement. Each block is a pair (left edge, right edge + 1).

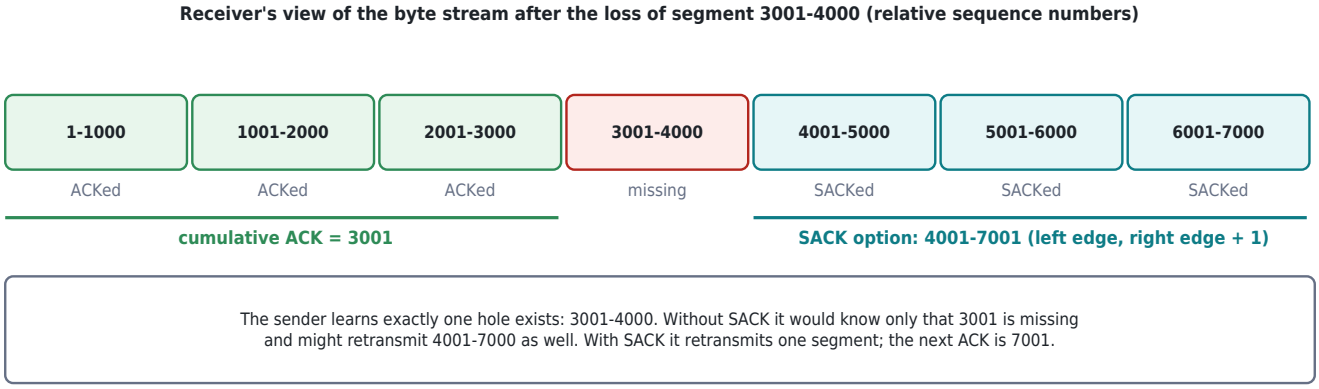


Figure 3.15. After the loss of 3001-4000 the receiver reports a cumulative ACK of 3001 and a SACK block 4001-7001. The sender knows that exactly one hole exists and retransmits only that segment.

With SACK, the sender maintains a scoreboard of the blocks, retransmits only the holes, and does not resend data the receiver already holds. SACK is enabled by default on Linux (`net.ipv4.tcp_sack=1`, verified on the build machine) and by essentially all other stacks. SACK blocks are advisory: the receiver may later discard data held out of order, so the sender must not free the data from its buffer until the cumulative acknowledgement covers it.

3.4.5 Reordering, duplicates and corruption

- **Out-of-order arrival.** The receiver stores segments beyond the gap in its receive buffer and delivers to the application only the contiguous prefix. The application never observes the disorder; it observes a delay. The buffered bytes count against the receive window.
- **Duplicates.** A duplicate segment (from a spurious retransmission or a network duplicate) carries sequence numbers that the receiver has already passed or already holds. It is discarded and answered with an immediate ACK; the D-SACK extension (RFC 2883) reports it to the sender, which allows the sender to detect that a retransmission was unnecessary.
- **Corruption.** A segment whose checksum fails is discarded silently, so that it is handled as loss. The 16-bit checksum is weaker than the link-layer CRC; applications that must detect corruption use TLS or an application-level digest.
- **Sequence number wrap.** On fast paths 32-bit numbers wrap within seconds (at 10 Gbit/s about every 3.4 seconds). The timestamp option provides PAWS (protection against wrapped sequence numbers): a segment with an older timestamp than the most recent accepted one is discarded.

3.4.6 Hands-on

TCP reliability cannot be observed on loopback because loopback does not lose packets. Two observations are still possible with the tools available.

First, the retransmission parameters of the kernel are readable. On the build machine:

Retransmission parameters TERMINAL

```
$ cat /proc/sys/net/ipv4/tcp_retries2 /proc/sys/net/ipv4/tcp_syn_retries /proc/sys/net/ipv4/tcp_sack
15
6
1
```

`tcp_retries2 = 15` is the number of retransmissions of established-connection data before the connection is declared dead; with the Linux RTO bounds (200 ms to 120 s) the total is roughly 15 minutes. An application that writes to a vanished peer therefore keeps seeing a working socket for many minutes unless it uses its own timeouts, `SO_KEEPALIVE` or the `TCP_USER_TIMEOUT` option.

Second, the effect of loss on a local connection can be emulated with `tc netem`, which is not available on the build machine. On a machine with `tc` the typical commands would be as follows (typical, not run here):

Typical loss emulation (not run on the build machine) TERMINAL

```
$ sudo tc qdisc add dev lo root netem loss 2% delay 20ms
$ curl -o /dev/null -w "%{time_total}s\n" http://127.0.0.1:8080/large
$ sudo tc qdisc del dev lo root
```

In `ss -ti` output (typical) the fields `rto:204`, `rtt:21.5/10.7`, `retrans:0/3`, `cwnd:10` and `sacked:` expose the estimator, the retransmission counters and the congestion window of every connection; they are the practical way to read the quantities of this chapter on a live system.

3.4.7 Failure modes

Symptom	Cause	Diagnosis and remedy
Requests hang for about 200 ms, 1 s or multiples of these	Retransmission timeouts after tail loss or a lost SYN	Look for `retrans` in `ss -ti`; enable RACK-TLP (default in recent Linux); fix packet loss on the path
Throughput collapses on a lossy path although bandwidth is available	Loss interpreted as congestion shrinks the window; every timeout restarts slow start	Reduce loss (Wi-Fi, faulty link, overloaded device); use SACK, a recent congestion control algorithm such as BBR where appropriate
Application sees no error for minutes after the peer disappears	Retransmissions continue until `tcp_retries2` is exhausted	Application read timeouts, heartbeats, `TCP_USER_TIMEOUT`
Spurious retransmissions in a mobile network	Sudden delay increase (handover) exceeds the RTO	Timestamps, F-RTO (RFC 5682) detect the spurious timeout; no action normally needed

Reliability-related problems.

3.4.8 Questions to expect

1. **How does TCP guarantee reliable delivery?** Sequence numbers on every byte, cumulative acknowledgements, a retransmission timer, fast retransmit on three duplicate ACKs, and a checksum that turns corruption into loss. The receiver reorders and delivers an ordered stream.
2. **What is a cumulative acknowledgement?** An ACK of `n` confirms all bytes below `n` and names the next byte expected. It cannot confirm data beyond a gap.
3. **How is the RTO computed?** `RTO = SRTT + 4 x RTTVAR`, with `SRTT` an exponentially weighted average (weight 1/8) of RTT samples and `RTTVAR` an average of the deviation (weight 1/4). The RTO doubles after each timeout, and samples from retransmitted segments are ignored (Karn).
4. **Explain fast retransmit.** Three duplicate ACKs indicate that a segment was lost while later segments arrived. The sender retransmits it immediately instead of waiting for the timer and halves the window.
5. **What does SACK add?** The receiver reports the ranges that arrived beyond the cumulative ACK, so the sender retransmits only the missing ranges and several holes can be repaired within one round trip.
6. **Why are samples from retransmitted segments discarded?** The acknowledgement might refer to the first transmission or to the retransmission, which yields a wrong RTT. Karn's algorithm excludes them; the timestamp option allows sampling safely.
7. **What happens to the RTO after a timeout?** It doubles (exponential backoff) until new data is acknowledged, and the congestion window falls to one segment.
8. **TCP is reliable. Why do applications still need timeouts?** TCP retries for many minutes and reports success only for data that reached the peer's kernel, not the application. A peer that crashed, or a path that was cut, produces no signal until the retries are exhausted.

KEY POINTS

Reliability rests on sequence numbers, cumulative ACKs, retransmission and checksums.

- RTO = SRTT + 4 x RTTVAR (RFC 6298), doubled after each timeout; Karn's algorithm excludes retransmitted segments.
- Three duplicate ACKs trigger fast retransmit; SACK describes the exact holes.
- A timeout is more costly than a fast retransmit because it collapses the congestion window.
- A successful `write()` means that the data entered the local kernel buffer, not that the peer processed it.

Preview

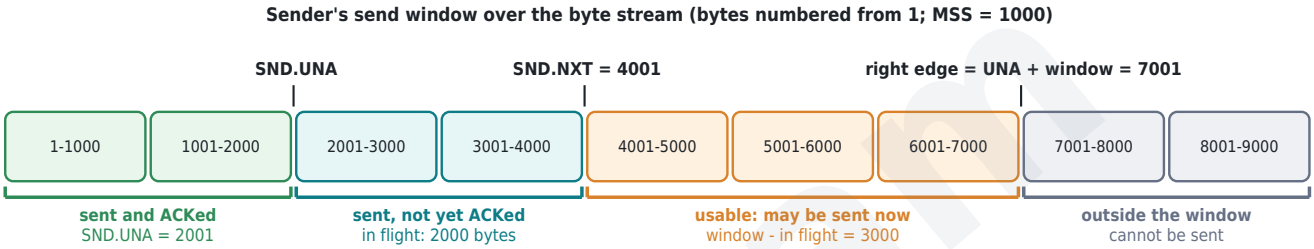
3.5 Flow Control

Reliability alone does not protect a slow receiver from a fast sender. If the sender transmitted without limit, the receiver would have to discard segments when its buffer overflowed, and the retransmissions would waste bandwidth. TCP prevents this with **flow control**: the receiver advertises in every segment the amount of buffer space that it has available (the **receive window**, `rwnd`), and the sender never keeps more unacknowledged data in flight than this value. Flow control concerns the endpoints; congestion control (next chapter) concerns the network between them.

3.5.1 Concept and mechanism

The sliding window

The sender tracks three positions in the byte stream: `SND.UNA` (the oldest unacknowledged byte), `SND.NXT` (the next byte to send) and the right edge of the window, `SND.UNA + window`. Bytes below `SND.UNA` are acknowledged and can leave the send buffer. Bytes between `SND.UNA` and `SND.NXT` are in flight. Bytes between `SND.NXT` and the right edge may be sent immediately. Each acknowledgement that advances `SND.UNA`, or that increases the window, moves the right edge and lets the window **slide** forward.



Advertised receive window = 5000 bytes (and cwnd >= 5000): the window spans 2001-7000. An ACK of 4001 slides it to 4001-9000.

Figure 3.16. With an advertised window of 5000 bytes and SND.UNA = 2001, the sender may have bytes 2001 to 7000 outstanding. 2000 bytes are in flight and 3000 bytes remain usable; an ACK of 4001 slides the window to 4001-9000.

The window field is 16 bits wide in the header, so without scaling its maximum is 65535 bytes. The **window scale option** (RFC 7323) is exchanged in the SYN segments and defines a shift `s` (0 to 14): the real window is the field value multiplied by 2^s. The scale applies to all later segments of the connection and cannot be changed. If either SYN omits the option (a middlebox may strip it), no scaling is used and the window stays below 64 KiB.

The effective amount of data that a sender may have in flight is the smaller of the receive window and the congestion window: `min(rwnd, cwnd)`.

Window size and throughput: the bandwidth-delay product

A sender can have at most one window of data in flight per round trip. The maximum throughput of a connection is therefore `window / RTT`. To fill a path, the window must be at least the **bandwidth-delay product** (BDP): bandwidth x RTT.

Path	Bandwidth, RTT	BDP	Throughput limit with an unscaled 65535-byte window
LAN	1 Gbit/s, 0.5 ms	62.5 KB	1 Gbit/s (window sufficient)
Same continent	1 Gbit/s, 20 ms	2.5 MB	65535 x 8 / 0.02 = 26.2 Mbit/s
Intercontinental	1 Gbit/s, 150 ms	18.75 MB	3.5 Mbit/s

Window requirements for typical paths.

The second row uses BDP = 10^9 / 8 x 0.02 = 2.5 x 10^6 bytes: without window scaling the 1 Gbit/s path is used at 2.6 percent. Linux sizes receive buffers dynamically (**auto-tuning**) within `net.ipv4.tcp_rmem`, which holds minimum, initial and maximum values (4096, 131072 and 33554432 on the build machine). A large maximum permits a large window; a small maximum, for example in a container with restricted sysctls, caps throughput on long paths.

3.5.2 Zero window and window probes

When the application reads slowly, the receive buffer fills and the advertised window shrinks. At zero, the sender must stop. To avoid a deadlock in case the window update is lost, the sender starts a **persist timer** and periodically sends a **window probe** (one byte, or a segment that the receiver may reject); the receiver answers with its current window. The probe interval backs off exponentially. When the application drains the buffer, the receiver sends a **window update**.

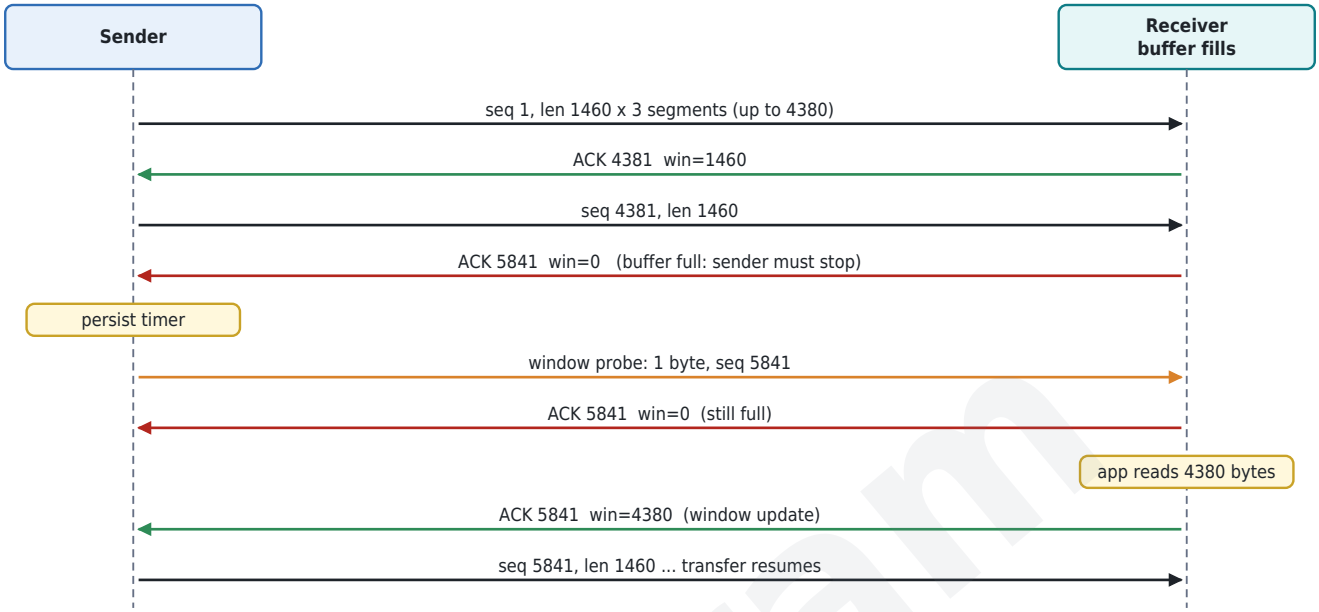


Figure 3.17. The receiver's buffer fills: the advertised window shrinks to 1460, then to 0. The sender stops and probes with a one-byte segment on its persist timer; after the application reads 4380 bytes the receiver sends a window update of 4380 and the transfer resumes.

Silly window syndrome

If the receiver advertises every small increment of free space and the sender transmits tiny segments to fill it, the connection degenerates into many segments with a few bytes each (**silly window syndrome**). Two rules prevent it. The receiver does not advertise a larger window until it can accommodate at least one full segment or half of its buffer (Clark's algorithm). The sender does not transmit small segments unless it has no unacknowledged data outstanding or the segment fills a full MSS (Nagle's algorithm).

3.5.3 Nagle's algorithm and delayed acknowledgement

Nagle's algorithm (RFC 896) reduces the number of tiny segments produced by interactive applications that write a few bytes at a time. The rule is: if unacknowledged data is in flight, a new small write is held in the buffer until either the outstanding data is acknowledged or enough data has accumulated for a full segment. The first small write after an idle period is sent at once.

The receiver's **delayed acknowledgement** postpones an ACK in the hope that data to piggyback on, or a second segment, arrives soon. Each algorithm is reasonable alone. Together they produce a stall in the **write-write-read** pattern: the application writes a small header, then a small body, then waits for the response. The header is sent at once; the body is held by Nagle until the header is acknowledged; the receiver delays the ACK of the header because it has nothing to send and does not yet have a second segment; the server cannot respond before it has the body. The stall lasts until the delayed-ACK timer fires (about 40 ms on Linux).

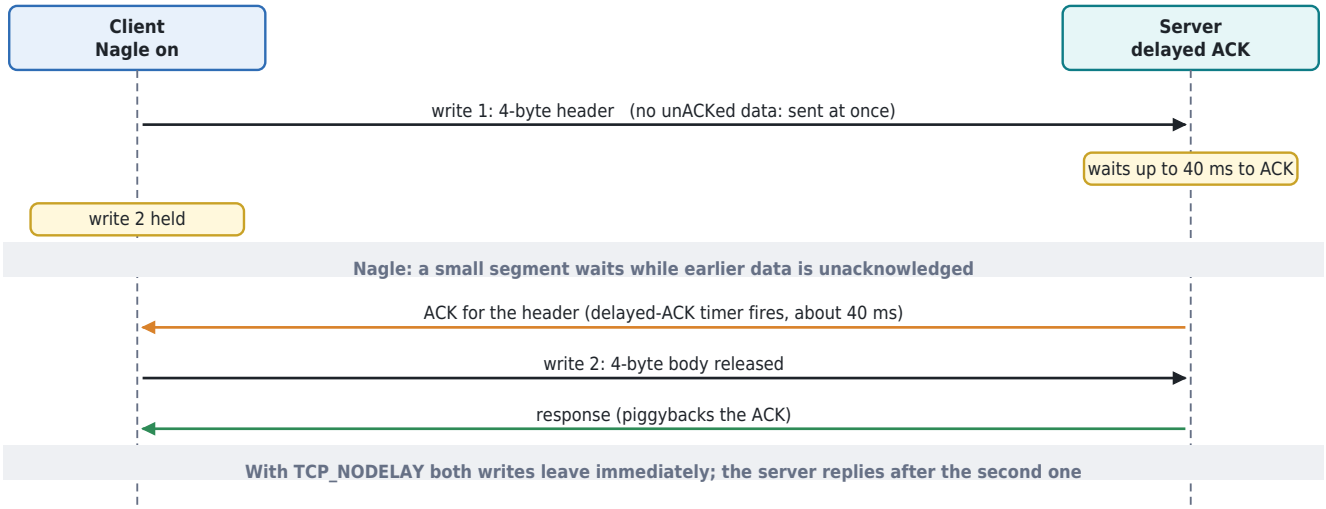


Figure 3.18. The write-write-read stall: the second small write waits for the acknowledgement of the first, and the receiver holds that acknowledgement for its delayed-ACK timer. With TCP_NODELAY both writes leave immediately and the stall disappears.

The remedy is `TCP_NODELAY` (`Socket.setTcpNoDelay(true)` in Java), which disables Nagle's algorithm, preferably together with writing the message in one call (a buffered output stream, a single `write` of header and body). The next section demonstrates the effect.

3.5.4 Worked trace

The sender uses an MSS of 1460 bytes and the receiver announces window 5840 (four segments) after the handshake. The receiving application stops reading.

1. The sender transmits three segments of 1460 bytes (4380 bytes in flight). The receiver acknowledges with `ack = 4381` and `win = 1460` (5840 - 4380 bytes still unread in the buffer).
2. The sender transmits one more segment of 1460 bytes. The buffer is full; the receiver acknowledges with `ack = 5841` and `win = 0`.
3. The sender stops and starts the persist timer. After the timeout it sends a 1-byte probe with `seq = 5841`. The receiver rejects the byte (no space) and answers `ack = 5841`, `win = 0`. The interval before the next probe is doubled.
4. The application reads 4380 bytes, which frees that much buffer space; the receiver sends an unsolicited window update `ack = 5841`, `win = 4380`.
5. The sender transmits 1460-byte segments again, up to 4380 bytes (three segments) in flight.

The diagram above shows exactly these numbers. The window field in each ACK is the free buffer space at the time of sending, and the sender never has more than `win` bytes beyond `ack` outstanding.

3.5.5 Hands-on

Receive buffer exhaustion in Java

The program connects a client to a server that does not read. The client writes 4 MiB in 4 KiB chunks from a second thread and the main thread counts how many bytes `write()` accepted. When both buffers are full, `write()` blocks: this is the zero window observed from the application.

ZeroWindow.java JAVA

```java
import java.io.*;
import java.net.*;
import java.util.concurrent.atomic.AtomicLong;

public class ZeroWindow {
    public static void main(String[] args) throws Exception {
        try (ServerSocket ss = new ServerSocket(0, 5, InetAddress.getLoopbackAddress())) {
            ss.setReceiveBufferSize(64 * 1024);
            AtomicLong sent = new AtomicLong();
            Socket client = new Socket(InetAddress.getLoopbackAddress(), ss.getLocalPort());
            client.setSendBufferSize(64 * 1024);
            Socket server = ss.accept();       // the server never calls read() for now
```

ZeroWindow.java (continued) JAVA

```
            Thread writer = Thread.ofPlatform().start(() -> {
                byte[] chunk = new byte[4096];
                try {
                    OutputStream out = client.getOutputStream();
                    for (int i = 0; i < 1024; i++) {       // 4 MiB in total
                        out.write(chunk);
                        sent.addAndGet(chunk.length);
                    }
                } catch (IOException e) { System.out.println("writer: " + e.getMessage()); }
            });
            Thread.sleep(500);
            long stalled = sent.get();
            Thread.sleep(500);
            System.out.println("bytes accepted by write() while the reader is idle: " + stalled);
            System.out.println("progress during the next 500 ms: "
                + (sent.get() - stalled) + " bytes");
            System.out.println("writer state: " + writer.getState());
            long total = 0;
            byte[] buf = new byte[65536];
            InputStream in = server.getInputStream();
            while (total < 4096L * 1024) {
                total += in.read(buf);
            }
            writer.join();
            System.out.println("after the reader started: received " + total
                + " bytes, writer state " + writer.getState());
            client.close();
            server.close();
        }
    }
}
```

Run ZeroWindow.java TERMINAL

```
$ java ZeroWindow.java
bytes accepted by write() while the reader is idle: 241664
progress during the next 500 ms: 0 bytes
writer state: RUNNABLE
after the reader started: received 4194304 bytes, writer state TERMINATED
```

What to notice: `write()` accepted about 240 KB, which equals the send buffer of the client plus the receive buffer and queues of the server side (the exact figure varies, typical output). Then the progress stopped completely for the next 500 ms. The thread state is `RUNNABLE`, not `BLOCKED` or `WAITING`: Java reports a thread inside a blocking native socket call as running, which is why such a stall does not appear as lock contention in a thread dump. As soon as the reader consumed data, the window update let the writer finish. In a production service this pattern appears as handler threads that remain in `SocketOutputStream.socketWrite0` while a slow client or a slow downstream service does not read.

Nagle and delayed ACK measured

The program below issues 20 requests, each consisting of two small writes (4 bytes, 4 bytes), and waits for a 1-byte response. It measures the time per request with `TCP_NODELAY` on and off.

Nagle.java JAVA

```
import java.io.*;
import java.net.*;

public class Nagle {
    static long run(boolean noDelay) throws Exception {
        try (ServerSocket ss = new ServerSocket(0, 5, InetAddress.getLoopbackAddress())) {
            Thread srv = Thread.ofPlatform().daemon().start(() -> {
                try (Socket c = ss.accept()) {
                    InputStream in = c.getInputStream();
                    OutputStream out = c.getOutputStream();
                    byte[] b = new byte[16];
                    while (true) {
                        // a request is 2 writes: 4-byte header + 4-byte body
                        int got = 0;
                        while (got < 8) {
                            int n = in.read(b, got, 8 - got);
                            if (n < 0) return;
                            got += n;
                        }
```

Nagle.java (continued) JAVA

```
                        out.write(1); out.flush();
                    }
                } catch (IOException e) { }
            });
            try (Socket s = new Socket(InetAddress.getLoopbackAddress(), ss.getLocalPort())) {
                s.setTcpNoDelay(noDelay);
                OutputStream out = s.getOutputStream();
                InputStream in = s.getInputStream();
                long t0 = System.nanoTime();
                for (int i = 0; i < 20; i++) {
                    out.write(new byte[4]); out.flush();   // write 1
                    out.write(new byte[4]); out.flush();   // write 2 (small, previous unACKed)
                    in.read();
                }
                return (System.nanoTime() - t0) / 20 / 1000;
            }
        }
    }
    public static void main(String[] a) throws Exception {
        run(true);
        System.out.println("TCP_NODELAY=true : " + run(true) + " us per request");
        System.out.println("TCP_NODELAY=false: " + run(false) + " us per request");
    }
}
```

Run Nagle.java TERMINAL

```
$ java Nagle.java
TCP_NODELAY=true : 37 us per request
TCP_NODELAY=false: 42411 us per request
```

The difference is about 500-fold. With Nagle's algorithm enabled the second write waited for the delayed ACK of the first, and each request took about 43 ms, which is the Linux delayed-ACK timer; with `TCP_NODELAY` a request completes in less than 0.1 ms. The figures are measured on the build machine and are typical, not exact. The standard remedies, in order of preference, are to write each message in one `write` call, and to set `TCP_NODELAY` when the pattern cannot be changed. Tomcat sets `TCP_NODELAY` on its connectors by default (the `tcpNoDelay` attribute is true), as do most HTTP client libraries; custom protocols written directly on `Socket` are the usual victims.

3.5.6 Failure modes

Symptom	Cause	Diagnosis and remedy
Transfers on long fat paths reach only a fraction of the bandwidth	The window (rwnd or socket buffer) is smaller than the BDP; scaling disabled or buffers capped	Compare `window / RTT` with the link rate; raise `tcp_rmem` and `tcp_wmem` maxima; do not set `SO_RCVBUF` explicitly on Linux unless required, because it disables auto-tuning
Writers block indefinitely	The peer does not read (zero window)	Write timeouts at the application level; bounded queues; monitor `Send-Q` in `ss -tn`
Periodic stalls of about 40 ms in request-response protocols	Nagle plus delayed ACK	`TCP_NODELAY`; single write per message
Many tiny segments, high CPU	Silly window syndrome or small unbuffered writes	Buffered streams; larger writes
`Recv-Q` growing on the server	The application reads slower than data arrives	Thread pool saturation; profile the handler; apply backpressure

Flow-control problems.

GOTCHA

Setting `SO_RCVBUF` or `SO_SNDBUF` on Linux disables auto-tuning for that socket. `Socket.setReceiveBufferSize` fixes the buffer (the kernel doubles the requested value for bookkeeping). A fixed small buffer limits throughput to `buffer / RTT`; the setting must be done before `connect()` or `bind()` so that the window scale is chosen with it. In the absence of a measured need, the defaults are preferable.

3.5.7 Questions to expect

1. **What is the difference between flow control and congestion control?** Flow control protects the receiver and uses the window advertised by the receiver (`rwnd`). Congestion control protects the network and uses a window computed by the sender (`cwnd`). The sender uses the minimum of both.
2. **What is the receive window and where is it carried?** The number of bytes the receiver can currently buffer. It is carried in the 16-bit window field of every segment and scaled by the option negotiated in the SYN.
3. **Why is the window scale option needed?** The 16-bit field limits the window to 65535 bytes, which restricts throughput on paths with a large bandwidth-delay product. The shift count (0 to 14) allows windows up to about 1 GiB.
4. **Compute the maximum throughput of a connection with a 64 KiB window and an RTT of 100 ms.** 65536 x 8 / 0.1 = 5.24 Mbit/s, independent of the link capacity.
5. **What is a zero window and how does the sender recover?** The receiver's buffer is full and it advertises 0. The sender pauses, probes with the persist timer, and resumes after a window update.
6. **Explain Nagle's algorithm and when to disable it.** Small writes wait while unacknowledged data is in flight. It is disabled with `TCP_NODELAY` for latency-sensitive request-response protocols that write a message in several pieces.
7. **How do Nagle and delayed ACK interact?** The sender waits for an ACK that the receiver delays, producing a stall of up to the delayed-ACK timer (about 40 ms on Linux) per request.
8. **What does it mean for a Java write to block?** The kernel send buffer is full because the peer's window is closed or the network is slow. The blocked thread is shown as `RUNNABLE`.

KEY POINTS

The receiver advertises a window in every segment; the sender keeps at most `min(rwnd, cwnd)` bytes in flight.

- Throughput is limited to window / RTT, so a window of at least the bandwidth-delay product is needed to fill a path; window scaling extends the 16-bit field.
- A zero window stops the sender; probes with exponential backoff recover from a lost window update.
- Nagle's algorithm with delayed ACK causes stalls of about 40 ms in write-write-read protocols; `TCP_NODELAY` and single writes remove them.
- A blocked `write()` is flow control observed by the application.

3.6 Congestion Control

A network path has finite capacity. If all senders transmit as fast as their receivers allow, the queues in routers overflow, packets are dropped, retransmissions add load, and useful throughput collapses (**congestion collapse**, observed on the early Internet in 1986). TCP prevents this by letting each sender estimate the capacity of the path and adjust its rate continuously. The algorithms are executed at the endpoints; routers take no explicit part except when Explicit Congestion Notification (ECN) is used.

The control variable is the **congestion window** (`cwnd`), measured in bytes or in segments (MSS). The sender never has more than `min(cwnd, rwnd)` unacknowledged bytes in flight. Since a window of `cwnd` bytes is sent per round trip, the average sending rate is about `cwnd / RTT`.

3.6.1 Concept and mechanism

The phases of the classical algorithm

RFC 5681 defines four algorithms: slow start, congestion avoidance, fast retransmit and fast recovery. A second variable, the **slow start threshold** (`ssthresh`), separates the first two.

1. **Slow start.** The connection starts with a small `cwnd` (the **initial window**, IW). For every acknowledged segment `cwnd` grows by one MSS, which doubles the window per RTT. The growth is exponential, and the name refers only to the small starting point. Slow start ends when `cwnd` reaches `ssthresh` or when loss occurs. RFC 6928 permits an initial window of 10 segments; Linux has used 10 since kernel 3.0.
2. **Congestion avoidance.** When `cwnd >= ssthresh`, the window grows by about one MSS per RTT (a quantity of MSS x MSS / cwnd per ACK). The growth is linear.

3. **Reaction to loss detected by three duplicate ACKs.** The network is congested but still delivering. The sender sets `ssthresh = max(FlightSize / 2, 2 x MSS)`, retransmits the lost segment (fast retransmit) and continues with `cwnd = ssthresh` (after fast recovery) in congestion avoidance. The multiplicative decrease halves the rate.
4. **Reaction to a retransmission timeout.** Severe congestion is assumed. `ssthresh = max(FlightSize / 2, 2 x MSS)` and `cwnd` falls to 1 MSS (the loss window); slow start restarts.

The combination of additive increase (one segment per RTT) and multiplicative decrease (halving) is **AIMD**. The resulting graph of `cwnd` against time is the familiar sawtooth.

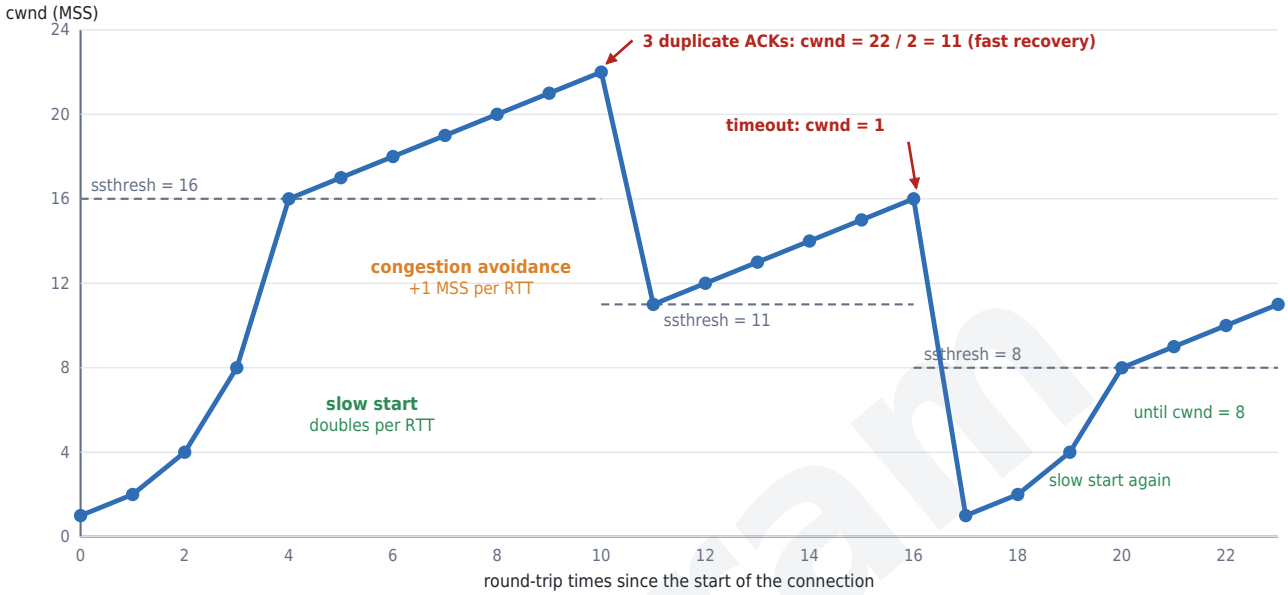


Figure 3.19. The congestion window of the worked trace in units of MSS. Slow start doubles cwnd to ssthresh = 16, congestion avoidance adds one per RTT until three duplicate ACKs at cwnd = 22 halve it to 11; a timeout at cwnd = 16 resets it to 1 and ssthresh to 8.

Why AIMD converges to fairness

Two flows that share a bottleneck of capacity C should each receive C / 2. Under AIMD, additive increase adds the same amount to both flows and therefore leaves the difference between them unchanged, whereas multiplicative decrease halves both and therefore halves the difference. After each cycle of increase and decrease the gap has become smaller and the combined throughput oscillates around C.

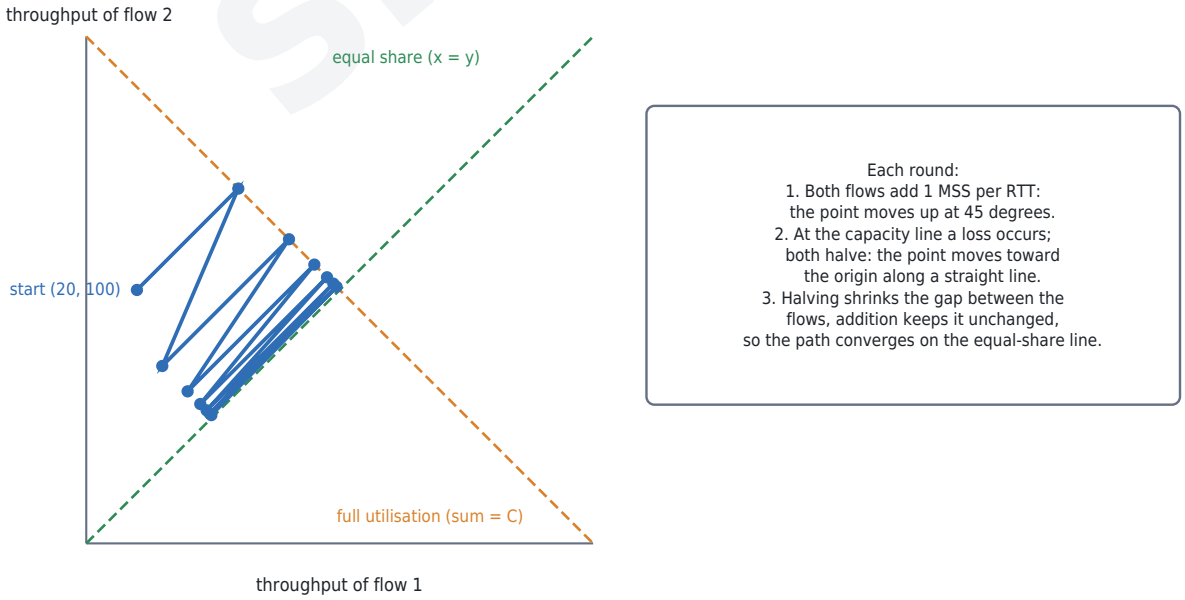


Figure 3.20. Throughput of flow 1 against flow 2. Additive increase moves the operating point along a 45-degree line; at the capacity line both flows halve, which moves the point toward the origin along a straight line. Repeated cycles converge on the equal-share line.

Fairness holds between flows with similar round-trip times. A flow with a shorter RTT increases its window more often and obtains more bandwidth than one with a longer RTT, and an application that opens several parallel connections takes a proportionally larger share. This is the reason why browsers limit connections per host and why HTTP/2 multiplexes requests on one connection.

Throughput as a function of loss

For a long-running flow with loss probability `p`, the classical estimate (Mathis et al.) is `throughput ~ 1.22 x MSS / (RTT x sqrt(p))`. With MSS = 1460 bytes and RTT = 50 ms the estimate is 28.5 Mbit/s at `p = 0.01 %` and 2.85 Mbit/s at `p = 1 %`. A hundredfold increase in loss therefore reduces throughput tenfold, and a longer RTT lowers throughput proportionally. This is the reason why a lossy Wi-Fi link or a long path performs poorly even when bandwidth is plentiful.

3.6.2 Modern algorithms

Algorithm	Principle	Characteristics
Reno, NewReno (RFC 5681, RFC 6582)	Loss-based AIMD; NewReno repairs several losses of one window during fast recovery	Reference behaviour; slow to use large windows after a loss
CUBIC (RFC 9438)	Loss-based; after a loss the window follows a cubic function `W(t) = C x (t - K)^3 + Wmax` with C = 0.4, reduction factor 0.7	Grows quickly back toward the previous maximum, then probes carefully; window growth depends on elapsed time, not on RTT; default on Linux
BBR (Google; Linux module `tcp_bbr`)	Model-based: estimates the bottleneck bandwidth and the minimum RTT, and paces at about their product instead of reacting to loss	High throughput on lossy paths and with shallow queues; avoids filling buffers; fairness toward loss-based flows has been a subject of study and was revised in later versions
Delay-based and datacentre variants (for example DCTCP with ECN)	React to queue length or ECN marks rather than loss	Used inside datacentres where all endpoints are controlled

Congestion control algorithms in common use.

On Linux the algorithm is selected per system with `net.ipv4.tcp_congestion_control` and can be set per connection through the `TCP_CONGESTION` socket option; the available algorithms are listed in `net.ipv4.tcp_available_congestion_control`. On the build machine the readings were:

Congestion control settings of the build machine TERMINAL

```
$ cat /proc/sys/net/ipv4/tcp_congestion_control
bbr
$ cat /proc/sys/net/ipv4/tcp_available_congestion_control
reno bbr cubic
$ cat /proc/sys/net/ipv4/tcp_slow_start_after_idle
1
```

The default of a distribution is usually `cubic`; the build machine uses `bbr`. The standard Java socket API does not expose the congestion control choice; it is a property of the operating system of the server. `ECN` (RFC 3168) lets a router mark packets instead of dropping them when its queue grows; the receiver echoes the mark (ECE flag) and the sender reduces the window once (CWR flag) without any loss.

GOTCHA

`tcp_slow_start_after_idle` resets the window of an idle persistent connection. With the value 1, a connection that was idle for longer than one RTO restarts with the initial window. A service that keeps HTTP connections open between bursts of traffic and sends large responses may see slow-start behaviour on every burst despite the reuse. The setting 0 disables the reset; whether this is appropriate depends on the network.

3.6.3 Worked trace

The following program applies the rules of RFC 5681 to a scripted sequence with `ssthresh = 16` and an initial window of 1 MSS (the book uses 1 MSS to make the growth visible; Linux starts with 10). A loss is signalled by three duplicate ACKs in round 10 and by a timeout in round 16. The output is real and is the source of the diagram above.

CwndSim.java JAVA

```java
public class CwndSim {
    public static void main(String[] args) {
        int cwnd = 1;          // in MSS; the initial window of 1 MSS keeps the example small
        int ssthresh = 16;
        System.out.println("RTT  cwnd  ssthresh  phase / event");
        for (int rtt = 0; rtt <= 23; rtt++) {
            String phase = cwnd < ssthresh ? "slow start" : "congestion avoidance";
            String event = "";
            System.out.printf("%3d  %4d  %8d  %s", rtt, cwnd, ssthresh, phase);
            if (rtt == 10) {                        // three duplicate ACKs
                ssthresh = Math.max(cwnd / 2, 2);
                cwnd = ssthresh;                    // fast recovery ends with cwnd = ssthresh
                event = "  <- 3 dup ACKs: ssthresh = cwnd/2, cwnd = ssthresh";
            } else if (rtt == 16) {                 // retransmission timeout
                ssthresh = Math.max(cwnd / 2, 2);
                cwnd = 1;
                event = "  <- timeout: ssthresh = cwnd/2, cwnd = 1";
            } else if (cwnd < ssthresh) {
                cwnd = Math.min(cwnd * 2, ssthresh);   // doubling per RTT, capped at ssthresh
            } else {
                cwnd += 1;                              // one MSS per RTT
            }
            System.out.println(event);
        }
    }
}
```

Run CwndSim.java TERMINAL

```
$ java CwndSim.java
RTT  cwnd  ssthresh  phase / event
  0     1        16  slow start
  1     2        16  slow start
  2     4        16  slow start
  3     8        16  slow start
  4    16        16  congestion avoidance
  5    17        16  congestion avoidance
  6    18        16  congestion avoidance
  7    19        16  congestion avoidance
  8    20        16  congestion avoidance
  9    21        16  congestion avoidance
 10    22        16  congestion avoidance  <- 3 dup ACKs: ssthresh = cwnd/2, cwnd = ssthresh
 11    11        11  congestion avoidance
 12    12        11  congestion avoidance
 13    13        11  congestion avoidance
 14    14        11  congestion avoidance
 15    15        11  congestion avoidance
 16    16        11  congestion avoidance  <- timeout: ssthresh = cwnd/2, cwnd = 1
 17     1         8  slow start
 18     2         8  slow start
 19     4         8  slow start
 20     8         8  congestion avoidance
 21     9         8  congestion avoidance
 22    10         8  congestion avoidance
 23    11         8  congestion avoidance
```

The trace has the following steps.

1. Rounds 0 to 3: slow start. The window is 1, 2, 4, 8: it doubles every round trip.
2. Round 4: `cwnd = 16 = ssthresh`; congestion avoidance begins. Rounds 5 to 10 add one segment per round: 17 to 22.
3. Round 10: three duplicate ACKs. `ssthresh = 22 / 2 = 11`, the lost segment is retransmitted, and `cwnd = 11` (fast recovery completes). The sender keeps transmitting and does not return to slow start.
4. Rounds 11 to 16: congestion avoidance again, 11 to 16.
5. Round 16: retransmission timeout. `ssthresh = 16 / 2 = 8`, `cwnd = 1`.

6. Rounds 17 to 19: slow start, 1, 2, 4; round 20: `cwnd = 8 = ssthresh`, congestion avoidance resumes (9, 10, 11).

The cost difference of the two loss signals is visible: after three duplicate ACKs the window remained at 11, whereas the timeout reduced it to 1, which required four rounds to return to 8. The time to transfer a small response also depends on the initial window. With a 10-segment initial window and a 1460-byte MSS, a 1 MB response needs 7 round trips (10, 20, 40, 80, 160, 320, 640 segments); with an initial window of 1 segment it would need 10 round trips. At an RTT of 50 ms this is a difference of 150 ms, and it explains why the initial window and connection reuse matter for web latency.

3.6.4 Hands-on

Congestion cannot be created on loopback, because the loopback device has no bottleneck and no loss. The quantities are readable on any Linux host with `ss -ti`, which prints the per-connection state (typical output, not available on the build machine):

Typical output of ss -ti for one established connection TERMINAL

```
$ ss -ti dst 198.51.100.20
State  Recv-Q Send-Q Local Address:Port    Peer Address:Port
ESTAB  0      0      192.0.2.10:49152      198.51.100.20:443
         cubic wscale:7,7 rto:204 rtt:3.2/1.6 mss:1448 cwnd:10 ssthresh:7 bytes_sent:20380
         bytes_retrans:1448 retrans:0/1 send 36.2Mbps pacing_rate 72.4Mbps
```

The fields identify the algorithm (`cubic`), the scale factors (`wscale:7,7`), the RTO (204 ms, the Linux floor plus variance), the smoothed RTT and its variation (3.2 ms and 1.6 ms), the MSS actually in use (1448 bytes, because the timestamp option takes 12 bytes), the congestion window in segments and the slow start threshold. `send` is `cwnd x mss / rtt`, here 10 x 1448 x 8 / 0.0032 = 36.2 Mbit/s, which confirms the formula `rate = cwnd / RTT`. `retrans:0/1` reports no retransmission in progress and one in total.

3.6.5 Failure modes

- **Bufferbloat.** Large router buffers delay the loss signal; loss-based algorithms fill the buffer and the RTT rises by hundreds of milliseconds. Remedies are active queue management (CoDel, fq_codel), ECN, and BBR.
- **Throughput collapse on random loss.** Loss that is not caused by congestion (radio errors) is interpreted as congestion by loss-based algorithms. BBR is less sensitive.
- **Incast.** Many senders respond at once to one receiver (a scatter-gather query in a datacentre); the switch buffer overflows and timeouts of 200 ms or more dominate. Remedies: ECN-based algorithms, application-level staggering, smaller minimum RTO.
- **Slow restart.** Idle connections returning to the initial window (see the warning above).
- **Unfair sharing.** An application that opens many connections, or a UDP flow without congestion control, takes more than a fair share.

3.6.6 Questions to expect

1. **Explain slow start and congestion avoidance.** Slow start doubles `cwnd` every RTT (one MSS per ACK) until `ssthresh`. Congestion avoidance then adds about one MSS per RTT. The start is slow only in the sense of a small initial window.
2. **What happens on three duplicate ACKs and on a timeout?** Three duplicate ACKs: retransmit, `ssthresh = cwnd / 2`, `cwnd = ssthresh`, continue in congestion avoidance. Timeout: `ssthresh = cwnd / 2`, `cwnd = 1 MSS`, slow start.
3. **What is AIMD and why does it lead to fairness?** Additive increase, multiplicative decrease. Increases add equally, decreases scale proportionally and shrink the difference between flows.
4. **How are flow control and congestion control related?** The sender uses `min(rwnd, cwnd)`. `rwnd` comes from the receiver, `cwnd` from the sender's estimate of the network.
5. **How does CUBIC differ from Reno?** CUBIC uses a cubic function of the time since the last loss to regrow the window, which is faster on high-bandwidth, high-latency paths, and reduces by 0.7 instead of 0.5.
6. **What is BBR?** A congestion control algorithm that estimates bottleneck bandwidth and minimum RTT and sends at the estimated rate, instead of using loss as the signal.
7. **Why does a new connection to a nearby server still take several round trips to deliver 1 MB?** The window starts at about 10 segments and doubles per RTT; 1 MB needs about 7 round trips. This is why connection reuse and a larger initial window help.

8. **Why can opening more parallel connections be considered unfair?** Each flow receives a fair share of the bottleneck, so n connections obtain n shares.

KEY POINTS

Congestion control adjusts `cwnd` from loss and delay signals so that all senders share capacity without collapse.

- Slow start doubles the window per RTT; congestion avoidance adds one MSS per RTT; AIMD yields the sawtooth.
- Three duplicate ACKs halve the window; a timeout resets it to one segment.
- Throughput is about cwnd / RTT and, under random loss, about MSS / (RTT x sqrt(p)).
- CUBIC is the common default; BBR is model-based; the algorithm is a server operating-system setting.

3.7 Sockets in Java

The Java socket API is a thin wrapper around the system calls `socket`, `bind`, `listen`, `accept`, `connect`, `read`, `write` and `close`. The classes `java.net.ServerSocket` and `java.net.Socket` provide blocking, stream-oriented access; `java.nio.channels` offers non-blocking channels with a `Selector`; and since Java 21 virtual threads allow the simple blocking style at the scale that previously required selectors. The behaviour of each call is determined by the TCP mechanisms of the preceding chapters.

3.7.1 Concept and mechanism

The call sequence

The server calls `new ServerSocket(port, backlog, bindAddress)`, which performs `socket`, `bind` and `listen`; the connection state `LISTEN` begins. `accept()` blocks until a completed connection is in the accept queue and returns a `Socket`. The client calls `new Socket(host, port)` or `connect(SocketAddress, timeout)`, which sends the SYN and blocks until the handshake completes, fails or times out. Both sides then use `getInputStream()` and `getOutputStream()`. `close()` releases the descriptor and starts the termination of the connection.

Java call	TCP event	Failure signal
`new Socket(host, port)` / `connect`	SYN, SYN+ACK, ACK	`ConnectException: Connection refused` (RST); `SocketTimeoutException: Connect timed out` (no answer); `UnknownHostException` (DNS failed before any SYN)
`ServerSocket.accept()`	Removes an entry from the accept queue	Blocks while the queue is empty; `SocketException: Socket closed` when closed
`InputStream.read()`	Reads from the receive buffer	-1: peer sent FIN; `SocketException: Connection reset`: RST; `SocketTimeoutException: Read timed out`: `SO_TIMEOUT` elapsed
`OutputStream.write()`	Copies to the send buffer; blocks if full	`SocketException: Broken pipe` or `Connection reset` after the peer has reset
`Socket.close()`	FIN (or RST when `SO_LINGER` is zero or unread data exists)	None; the connection continues in the background until closed
`shutdownOutput()`	FIN; socket remains readable	See half-close

Where Java calls meet the TCP events.

sreeram

3.7.2 A blocking echo server and many clients

The simplest design accepts connections in a loop and dedicates one thread to each. The program below starts a server that uses a virtual thread per connection and then opens 1000 simultaneous client connections, each from its own virtual thread. The reply begins with `true` when the handler runs in a virtual thread.

MultiEcho.java JAVA

```
import java.io.*;
import java.net.*;
import java.util.*;
import java.util.concurrent.*;

public class MultiEcho {
    static void handle(Socket s) {
        try (s) {
            BufferedReader in = new BufferedReader(
                            new InputStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(s.getOutputStream(), true);
            String line;
            while ((line = in.readLine()) != null) {
                out.println(Thread.currentThread().isVirtual() + ":" + line.toUpperCase());
            }
        } catch (IOException ignored) { }
    }

    public static void main(String[] args) throws Exception {
        ServerSocket server = new ServerSocket(19101, 128, InetAddress.getLoopbackAddress());
        Thread acceptor = Thread.ofPlatform().daemon().start(() -> {
            try {
                while (true) {
                    Socket s = server.accept();
                    Thread.ofVirtual().start(() -> handle(s));
                }
            } catch (IOException e) { /* closed */ }
        });
        int clients = 1000;
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> fs = new ArrayList<>();
            for (int i = 0; i < clients; i++) {
                final int id = i;
                fs.add(pool.submit(() -> {
                    try (Socket s = new Socket(InetAddress.getLoopbackAddress(), 19101)) {
                        PrintWriter out = new PrintWriter(s.getOutputStream(), true);
                        BufferedReader in = new BufferedReader(
                            new InputStreamReader(s.getInputStream()));
                        out.println("client" + id);
                        return in.readLine();
                    }
                }));
            }
            int ok = 0;
            String first = null;
            for (Future<String> f : fs) { String r = f.get(); if (first == null) first = r; ok++; }
            System.out.println(ok + " clients served concurrently; first reply = " + first);
        }
        server.close();
    }
}
```

Run MultiEcho.java TERMINAL

```
$ java MultiEcho.java
1000 clients served concurrently; first reply = true:CLIENT0
```

What to notice: the accept loop runs on one platform thread and hands every connection to `Thread.ofVirtual().start(...)`; each handler blocks in `readLine()` in an ordinary, sequential style. A virtual thread that blocks on a socket is unmounted from its carrier thread, so 1000 concurrent connections do not need 1000 operating-system threads. The same program with `Thread.ofPlatform()` also works at this scale, but at tens of thousands of connections the memory for thread stacks and the scheduling cost become the limit.

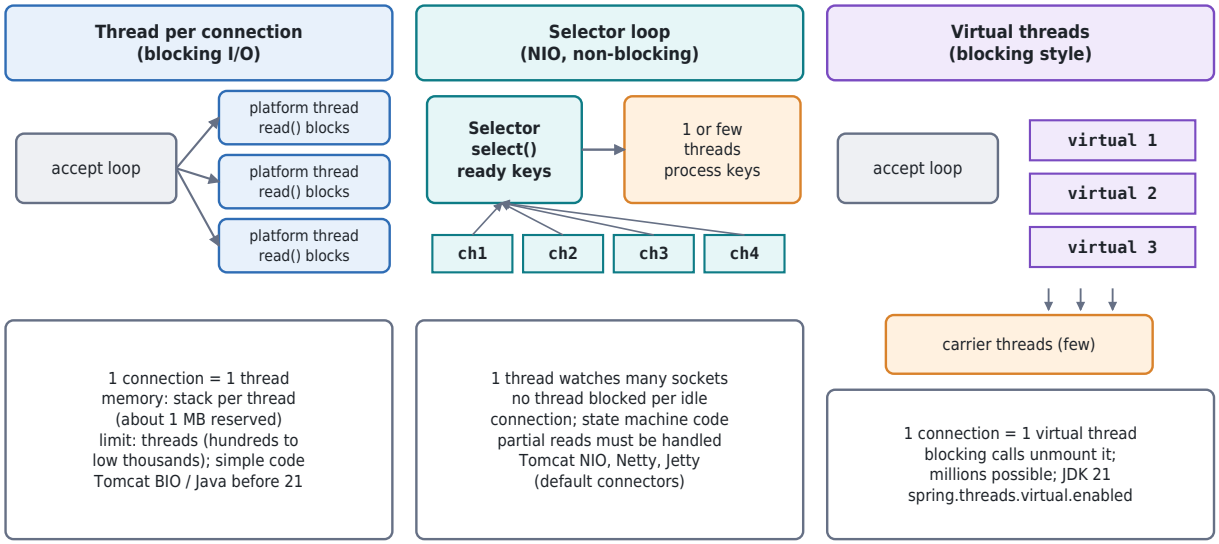


Figure 3.21. Three models of concurrency for connection handling: a platform thread per connection (blocking I/O), a selector loop that multiplexes many non-blocking channels onto a few threads, and a virtual thread per connection that keeps blocking code but multiplexes onto a few carrier threads.

The selector model

`java.nio` registers channels with a `Selector`; one thread calls `select()` and receives the set of channels that are ready (acceptable, readable, writable). No thread is blocked for an idle connection. The model needs an explicit state machine per connection, because a read returns whatever is available and a write may accept only part of the buffer.

NioEcho.java JAVA

```java
import java.io.IOException;
import java.net.InetAddress;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.ByteBuffer;
import java.nio.channels.*;
import java.util.Iterator;

public class NioEcho {
    public static void main(String[] args) throws Exception {
        Selector selector = Selector.open();
        ServerSocketChannel server = ServerSocketChannel.open();
        server.bind(new InetSocketAddress(InetAddress.getLoopbackAddress(), 19108));
        server.configureBlocking(false);
        server.register(selector, SelectionKey.OP_ACCEPT);

        Thread loop = Thread.ofPlatform().name("selector-loop").start(() -> {
            ByteBuffer buf = ByteBuffer.allocate(1024);
            try {
                while (selector.isOpen()) {
                    selector.select();
                    Iterator<SelectionKey> it = selector.selectedKeys().iterator();
                    while (it.hasNext()) {
                        SelectionKey key = it.next();
                        it.remove();
                        if (key.isAcceptable()) {
                            SocketChannel ch = server.accept();
                            ch.configureBlocking(false);
                            ch.register(selector, SelectionKey.OP_READ);
                            System.out.println("accepted " + ch.getRemoteAddress());
                        } else if (key.isReadable()) {
                            SocketChannel ch = (SocketChannel) key.channel();
                            buf.clear();
                            int n = ch.read(buf);
                            if (n < 0) { ch.close(); System.out.println("peer closed"); continue; }
                            buf.flip();
                            ch.write(buf);   // may write fewer bytes; a real server must handle it
                        }
                    }
                }
            } catch (IOException | ClosedSelectorException e) { /* shutting down */ }
```

NioEcho.java (continued) JAVA

```
        });

        for (int i = 1; i <= 3; i++) {
            try (Socket s = new Socket(InetAddress.getLoopbackAddress(), 19108)) {
                s.getOutputStream().write(("msg" + i).getBytes());
                byte[] r = new byte[16];
                int n = s.getInputStream().read(r);
                System.out.println("client " + i + " got " + new String(r, 0, n)
                    + " (handled by thread " + loop.getName() + ")");
            }
        }
        Thread.sleep(200);
        selector.close();
        server.close();
        loop.join();
    }
}
```

Run NioEcho.java TERMINAL

```
$ java NioEcho.java
accepted /127.0.0.1:54144
client 1 got msg1 (handled by thread selector-loop)
peer closed
accepted /127.0.0.1:54154
client 2 got msg2 (handled by thread selector-loop)
peer closed
accepted /127.0.0.1:54156
client 3 got msg3 (handled by thread selector-loop)
peer closed
```

All three clients were served by the single thread `selector-loop`. The listing omits the handling that production code needs: a partial `write` (a `ByteBuffer` with remaining data must be kept and `OP_WRITE` registered), a per-connection buffer, and error handling. Frameworks such as Netty, and Tomcat's NIO connector, implement these concerns.

3.7.3 Timeouts, backlog and reuse

Three timeouts exist and each applies to a different phase.

Timeout	API	Applies to
Connect timeout	`socket.connect(addr, millis)`; `HttpClient.Builder.connectTimeout`	The three-way handshake. Default: none (the operating-system limit, about 127 s on Linux with 6 SYN retries)
Read timeout	`socket.setSoTimeout(millis)`; `HttpRequest.Builder.timeout` (whole exchange)	A single blocking `read`. The exception leaves the socket usable. Default 0 (infinite)
Write	none in the standard API	A blocked `write` cannot be timed out; use a watchdog that closes the socket, or NIO channels

Socket timeouts in Java.

The next program demonstrates the three observable results: a refused connection, a read timeout, and the effect of the listen backlog when `accept()` is never called.

Timeouts.java JAVA

```
import java.io.*;
import java.net.*;

public class Timeouts {
    public static void main(String[] args) throws Exception {
        // 1. Connection refused: nothing listens on the port
        try (Socket s = new Socket()) {
            s.connect(new InetSocketAddress("127.0.0.1", 19199), 1000);
        } catch (IOException e) {
            System.out.println("1) " + e.getClass().getSimpleName() + ": " + e.getMessage());
        }
        // 2. Read timeout: peer accepts but never sends
        try (ServerSocket silent = new ServerSocket(19102, 1, InetAddress.getLoopbackAddress());
             Socket c = new Socket(InetAddress.getLoopbackAddress(), 19102)) {
```

Timeouts.java (continued) JAVA

```
            c.setSoTimeout(500);
            long t0 = System.nanoTime();
            try {
                c.getInputStream().read();
            } catch (SocketTimeoutException e) {
                System.out.printf("2) %s: %s after %d ms; socket still open=%b%n",
                    e.getClass().getSimpleName(), e.getMessage(),
                    (System.nanoTime() - t0) / 1_000_000, !c.isClosed());
            }
        }
        // 3. Backlog: listen backlog 1, never accept, count successful connects
        try (ServerSocket full = new ServerSocket(19103, 1, InetAddress.getLoopbackAddress())) {
            int ok = 0;
            String last = "";
            java.util.List<Socket> keep = new java.util.ArrayList<>();
            for (int i = 0; i < 6; i++) {
                Socket s = new Socket();
                try {
                    s.connect(new InetSocketAddress("127.0.0.1", 19103), 500);
                    keep.add(s);
                    ok++;
                } catch (IOException e) {
                    last = e.getClass().getSimpleName() + ": " + e.getMessage();
                    break;
                }
            }
            System.out.println("3) completed with backlog=1, no accept(): " + ok
                + (last.isEmpty() ? "" : "; next -> " + last));
            for (Socket s : keep) s.close();
        }
    }
}
```

Run Timeouts.java TERMINAL

```
$ java Timeouts.java
1) ConnectException: Connection refused
2) SocketTimeoutException: Read timed out after 501 ms; socket still open=true
3) completed with backlog=1, no accept(): 2; next -> SocketTimeoutException: Connect timed out
```

1. Connecting to port 19199, where nothing listens, fails at once with `Connection refused`, because the kernel answered the SYN with RST.
2. The read timeout fires after 501 ms. The peer accepted the connection (the kernel completed the handshake) but never sent data, which is the pattern of a hung downstream service. The exception does not close the socket.
3. With `backlog = 1` and no `accept()`, two connections completed (the Linux accept queue holds `backlog + 1` entries) and the third attempt timed out: the kernel drops the SYN or the final ACK while the queue is full, and the client retransmits and finally reports `Connect timed out`. This is the mechanism by which a saturated server produces connect timeouts instead of refusals.

SO_REUSEADDR

After a server process terminates, connections that it closed first remain in `TIME_WAIT` on the server port. Without `SO_REUSEADDR` a new server cannot bind that port until they expire.

Reuse.java JAVA

```
import java.io.*;
import java.net.*;

public class Reuse {
    static ServerSocket open(boolean reuse) throws IOException {
        ServerSocket ss = new ServerSocket();
        ss.setReuseAddress(reuse);
        ss.bind(new InetSocketAddress(InetAddress.getLoopbackAddress(), 19107));
        return ss;
    }
    public static void main(String[] args) throws Exception {
        System.out.println("default getReuseAddress() = " + new ServerSocket().getReuseAddress());
        ServerSocket ss = open(true);
        Socket c = new Socket(InetAddress.getLoopbackAddress(), 19107);
        Socket a = ss.accept();
        a.close();   // server side closes first -> TIME_WAIT stays on the server's port 19107
```

Reuse.java (continued) JAVA

```
        c.getInputStream().read();
        c.close();
        ss.close();
        try (ServerSocket x = open(false)) {
            System.out.println("rebind with reuse=false: ok");
        } catch (BindException e) {
            System.out.println("rebind with reuse=false: " + e.getMessage());
        }
        try (ServerSocket x = open(true)) {
            System.out.println("rebind with reuse=true: ok");
        }
    }
}
```

Run Reuse.java TERMINAL

```
$ java Reuse.java
default getReuseAddress() = true
rebind with reuse=false: Address already in use
rebind with reuse=true: ok
```

`ServerSocket` enables `SO_REUSEADDR` by default on Linux (the first line). With the option disabled the rebind fails with `Address already in use`; with it enabled the bind succeeds while the old connections remain in `TIME_WAIT`. The option does not allow two live listeners on the same address and port, and on Linux it must not be confused with `SO_REUSEPORT`, which does.

Keep-alive

`SO_KEEPALIVE` makes the kernel send probe segments on an idle connection. The kernel parameters (`tcp_keepalive_time` 7200 s, `tcp_keepalive_intvl` 75 s and `tcp_keepalive_probes` 9 on the build machine) mean that a dead peer is detected after more than two hours by default. Since Java 11, `jdk.net.ExtendedSocketOptions` provides `TCP_KEEPIDLE`, `TCP_KEEPINTERVAL` and `TCP_KEEPCOUNT` per socket (the `Opts` program in the chapter on TCP sets 60, 10 and 3, which detects a dead peer after 60 + 3 x 10 = 90 seconds). Keep-alive also keeps NAT and firewall mappings alive when the probe interval is shorter than their idle timeout, which is often 300 s or less in cloud environments. It is not a substitute for an application heartbeat, because it confirms only that the peer kernel is alive.

3.7.4 Partial reads and message framing

TCP has no message boundaries. `InputStream.read(byte[])` returns between 1 and `length` bytes: whatever is available. The next program shows a framing protocol with a 4-byte length prefix, which is robust against both the merging and the splitting of messages, and then a single `read` that returns only part of the data.

Framing.java JAVA

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class Framing {
    static void writeFrame(DataOutputStream out, String msg) throws IOException {
        byte[] b = msg.getBytes(StandardCharsets.UTF_8);
        out.writeInt(b.length);
        out.write(b);
    }
    static String readFrame(DataInputStream in) throws IOException {
        int len = in.readInt();
        byte[] b = new byte[len];
        in.readFully(b);               // loops until all len bytes have arrived
        return new String(b, StandardCharsets.UTF_8);
    }
    public static void main(String[] args) throws Exception {
        try (ServerSocket server = new ServerSocket(19105, 5, InetAddress.getLoopbackAddress())) {
            Thread t = Thread.ofPlatform().start(() -> {
                try (Socket c = server.accept()) {
                    DataInputStream in = new DataInputStream(
                        new BufferedInputStream(c.getInputStream()));
                    for (int i = 0; i < 3; i++) {
                        System.out.println("server frame " + i + ": " + readFrame(in));
                    }
                } catch (IOException e) { e.printStackTrace(); }
```

Framing.java (continued) JAVA

```
            });
            try (Socket s = new Socket(InetAddress.getLoopbackAddress(), 19105)) {
                s.setTcpNoDelay(true);
                DataOutputStream out = new DataOutputStream(
                    new BufferedOutputStream(s.getOutputStream()));
                writeFrame(out, "alpha");
                writeFrame(out, "beta beta");
                writeFrame(out, "gamma");
                out.flush();   // three frames may share one segment; framing makes that harmless
            }
            t.join();
        }
        // The partial-read pitfall: read(byte[]) returns whatever is available
        try (ServerSocket server = new ServerSocket(19106, 5, InetAddress.getLoopbackAddress())) {
            Thread t = Thread.ofPlatform().start(() -> {
                try (Socket c = server.accept()) {
                    OutputStream o = c.getOutputStream();
                    o.write("12345".getBytes()); o.flush();
                    Thread.sleep(200);
                    o.write("67890".getBytes()); o.flush();
                } catch (Exception e) { e.printStackTrace(); }
            });
            try (Socket s = new Socket(InetAddress.getLoopbackAddress(), 19106)) {
                byte[] buf = new byte[10];
                int n = s.getInputStream().read(buf);
                System.out.println("single read(byte[10]) returned " + n + " bytes");
            }
            t.join();
        }
    }
}
```

Run Framing.java TERMINAL

```
$ java Framing.java
server frame 0: alpha
server frame 1: beta beta
server frame 2: gamma
single read(byte[10]) returned 5 bytes
```

`DataInputStream.readFully` repeats the underlying read until the requested number of bytes has arrived. The three frames were written with one `flush`, and may have been sent in one segment; the receiver still recovered three messages. The last line shows the pitfall: the server wrote `12345` and `67890` 200 ms apart, so a single `read` into a 10-byte buffer returned only 5 bytes. Code that assumes one `read` per `write` works on loopback tests and fails on a real network. The usual framing methods are a length prefix (used by Kafka, many RPC systems and TLS records), a delimiter (the newline of HTTP/1.1 header lines and of `readLine()`), and a declared length in a header (`Content-Length`) or chunk lengths (chunked encoding).

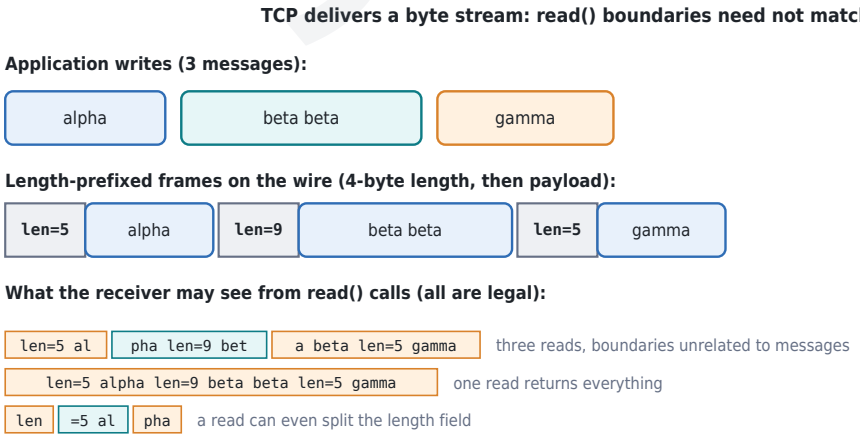


Figure 3.22. The sender writes three messages; each is prefixed with its length. The receiver may see the stream split or merged in any way, and still reconstructs the messages from the length fields.

3.7.5 What Tomcat does with these mechanisms

An embedded Tomcat in Spring Boot uses the NIO connector by default. The stages correspond to the mechanisms above.

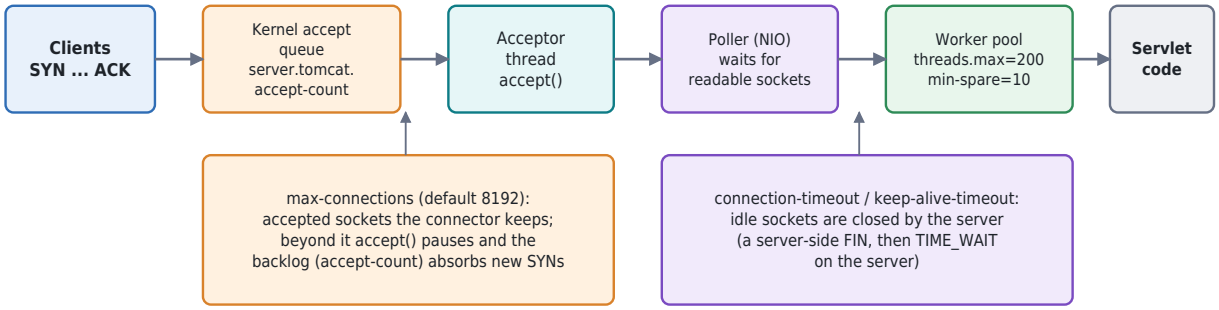


Figure 3.23. The path of a connection through a Tomcat connector: the kernel accept queue (bounded by accept-count), the acceptor thread that calls accept(), the poller that waits for readable sockets, and the worker pool that runs the servlet. The limits max-connections and the timeouts apply between these stages.

Property	Default	Effect at the TCP level
`server.tomcat.accept-count`	100	Backlog of the listening socket once `max-connections` is reached; excess SYNs are dropped by the kernel
`server.tomcat.max-connections`	8192	Connections that Tomcat accepts and keeps; beyond it new connections wait in the backlog
`server.tomcat.threads.max`	200	Worker threads, which limit concurrent request processing, not connections
`server.tomcat.threads.min-spare`	10	Threads kept ready
`server.tomcat.connection-timeout`	60 s unless set	Time to wait for the request line after accepting a connection; also the `SO_TIMEOUT`-style limit
`server.tomcat.keep-alive-timeout`	as `connection-timeout`	Idle time of a persistent connection before the server closes it (the server then becomes the active closer)
`server.tomcat.max-keep-alive-requests`	100	Requests per persistent connection before the server closes it

Spring Boot properties for the Tomcat connector (defaults from the Tomcat 10.1 documentation).

A connection is therefore limited at three levels: backlog (`accept-count`), connections (`max-connections`) and threads (`threads.max`). A common production observation is that thousands of idle keep-alive connections are held by the poller without threads, while a slow downstream call blocks 200 workers and the request queue grows although CPU use is low. Setting `spring.threads.virtual.enabled=true` (Spring Boot 3.2 and later, Java 21 and later) runs request handling on virtual threads, which removes the 200-thread ceiling but not the limit of the downstream resource, such as a database connection pool. `TCP_NODELAY` is enabled by Tomcat by default.

3.7.6 Failure modes

Symptom	Cause	Remedy
Threads stuck in `socketRead0`	No read timeout on a call to a slow or dead peer	Always set `SO_TIMEOUT` or the client library's read timeout
`Too many open files`	Leaked sockets (`CLOSE_WAIT`) or too low a descriptor limit (`ulimit -n`)	try-with-resources; close response bodies; raise the limit for high connection counts
Requests that hang until the connect timeout	Accept queue full or packets dropped	Tune backlog; check security groups; short connect timeout with retries
`Connection reset` on the first request after idle	Server or load balancer closed an idle pooled connection	Evict idle connections earlier than the peer's idle timeout; retry idempotent calls
Truncated or merged messages	Assumption of one read per write	Length-prefix or delimiter framing; `readFully`
Server cannot restart (`Address already in use`)	`SO_REUSEADDR` disabled, or another process	Keep the default; find the owner of the port
Thread-per-connection collapse under load	Platform threads exhausted	NIO or virtual threads; bounded queues

Socket-level failures in Java services.

3.7.7 Questions to expect

1. **Describe the lifecycle of a server socket in Java.** `ServerSocket` binds and listens; `accept()` returns a `Socket` per completed connection; the server reads and writes streams and closes the socket; closing the `ServerSocket` ends the listening.
2. **What is the backlog parameter?** The requested length of the accept queue of completed connections. The kernel limits it with `net.core.somaxconn`. The Java default is 50.
3. **What is the difference between a connect timeout and a read timeout?** The connect timeout bounds the handshake; `SO_TIMEOUT` bounds one blocking `read`. Neither bounds the total time of a request; that needs an overall deadline.
4. **Why can one `read` return fewer bytes than were written?** TCP is a byte stream. Data may be split over segments and arrive over several reads. Code must loop or use framing.
5. **How does an application learn that the peer closed the connection?** `read` returns -1 after FIN; an RST raises `SocketException`. Writes to a closed peer fail after the RST arrives.
6. **Compare blocking I/O, NIO selectors and virtual threads.** Blocking I/O uses a thread per connection and is simple but limited by threads. Selectors serve many connections per thread with explicit state machines. Virtual threads provide blocking-style code at selector-like scale.
7. **What does `SO_REUSEADDR` do?** It allows binding a port that has connections in `TIME_WAIT`, so a restarted server can listen immediately.
8. **How does Tomcat cope with 10000 idle connections?** The NIO poller watches them with a small number of threads; worker threads are allocated only for requests. The limits are `max-connections` and the file descriptor limit.
9. **What is TCP keep-alive and when is it insufficient?** Kernel probes on idle connections. By default the first probe is sent after two hours; the application needs its own heartbeat or timeouts for faster detection.

KEY POINTS

Java sockets expose TCP behaviour directly: connect, read and write outcomes map to SYN, RST, FIN and window states.

- Always set connect and read timeouts; a blocked `read` or `write` has no other limit.
- TCP delivers a byte stream; use length prefixes or delimiters, and loop on partial reads.
- Thread per connection, selector and virtual-thread models trade simplicity against scale; Tomcat uses NIO with a worker pool.
- Backlog, `max-connections`, thread count and the file-descriptor limit are separate capacity limits.

3.8 QUIC and HTTP/3 Transport

TCP is implemented in operating-system kernels and is understood by every middlebox, which made its design almost impossible to change. Its limits are structural: the handshake of TCP and the handshake of TLS are separate, a single lost segment delays all data behind it (head-of-line blocking), a connection is bound to the address pair of the endpoints so that a change of network breaks it, and new extensions must pass through firewalls that drop unknown options. **QUIC** (RFC 9000, May 2021) is a transport protocol built on UDP that integrates reliability, congestion control, stream multiplexing and TLS 1.3 encryption, and is implemented in user space. HTTP/3 (RFC 9114) is HTTP mapped onto QUIC.

3.8.1 Concept and mechanism

Position in the protocol stack

QUIC runs over UDP, usually on port 443. UDP supplies ports and delivery through NATs and firewalls that already allow it; everything else is part of QUIC. The TLS 1.3 handshake is carried inside QUIC packets and encrypts almost all of the packet header and all of the payload; the specification of the TLS usage is RFC 9001, and loss detection and congestion control are in RFC 9002 (a Reno-like algorithm is the default, and CUBIC or BBR can be used).

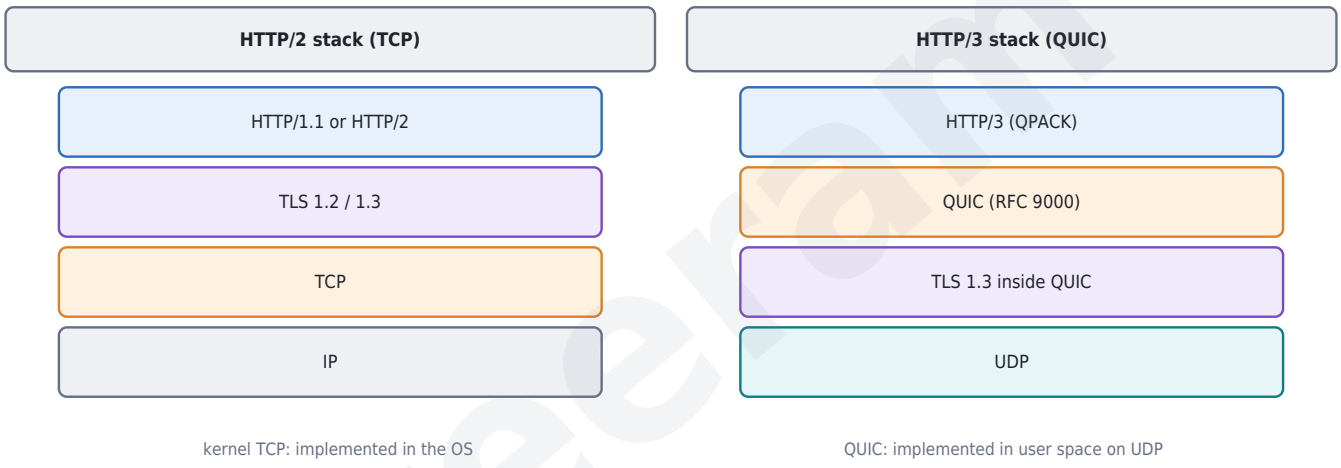


Figure 3.24. HTTP/2 uses HTTP over TLS over TCP over IP, with TCP in the kernel. HTTP/3 uses HTTP/3 and its header compression QPACK over QUIC, which contains the TLS 1.3 handshake and the transport functions, over UDP.

Connections and streams

A QUIC connection is identified by a **connection ID** chosen by each endpoint, not by the 4-tuple. If a client changes network (for example from Wi-Fi to mobile data) its address changes, but the connection ID stays the same, and the connection **migrates** after validating the new path; a TCP connection would have to be reopened. The connection ID also lets load balancers route packets of one connection to the same backend.

Within a connection, data is carried in independent **streams**, each with its own ordering. Streams are created without a handshake, and each has its own flow control as well as the connection-level limit. Packets carry frames; a packet may contain frames of several streams, and every packet has a packet number that is never reused (a retransmission of data is sent in a new packet), which removes the retransmission ambiguity of TCP.

Head-of-line blocking

In HTTP/2 over TCP several requests are multiplexed as streams within one TCP byte stream. TCP has no knowledge of the streams. If one segment is lost, the kernel withholds all later data from the application until the retransmission arrives, even data that belongs to other streams and arrived intact. QUIC orders data per stream, so a loss stalls only the streams whose data was in the lost packet.

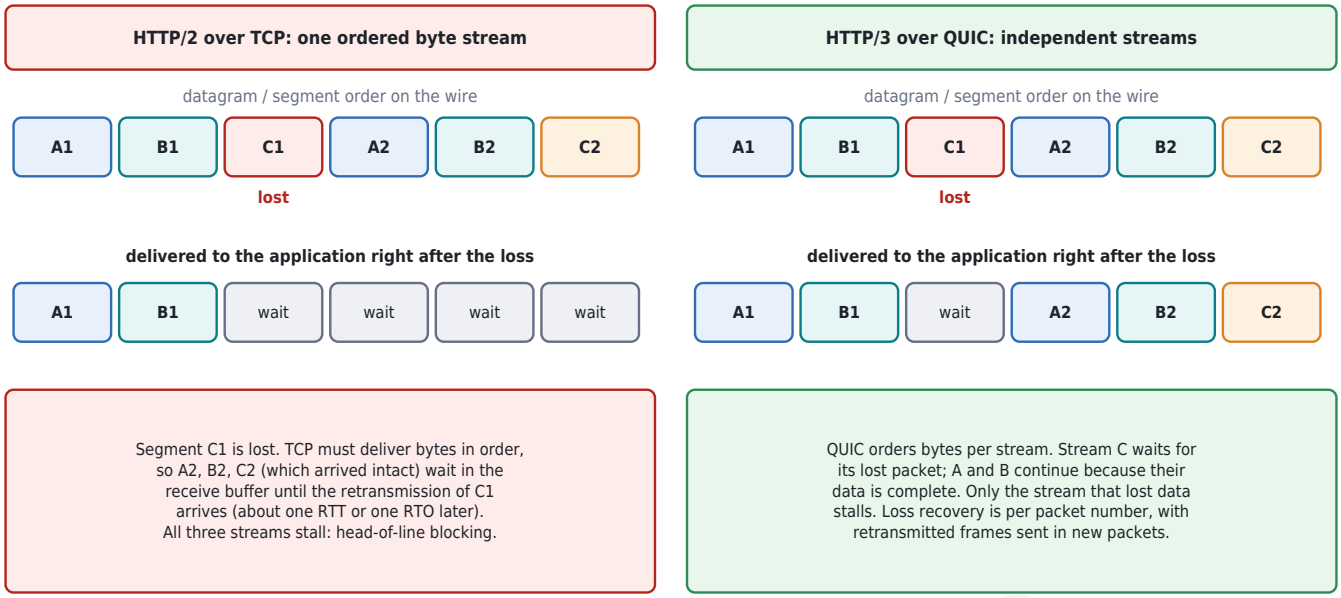


Figure 3.25. A lost segment C1 blocks every stream behind it on TCP (left); on QUIC only stream C waits and streams A and B continue to deliver data (right).

The effect is most visible on lossy paths such as mobile networks. On a clean path the two protocols perform similarly.

Connection establishment: 1-RTT and 0-RTT

A TCP connection with TLS 1.3 needs one round trip for the TCP handshake and one for the TLS handshake before the first request can be sent; with TLS 1.2 it needs three. QUIC combines the transport and the cryptographic handshake: the client's first flight contains the TLS ClientHello, and the first request can be sent after one round trip. A client that has connected before holds a resumption ticket and may send the request in the very first flight (**0-RTT**). 0-RTT data is not protected against replay by an attacker, so servers accept it only for idempotent requests, and the same restriction applies to TLS 1.3 early data over TCP.

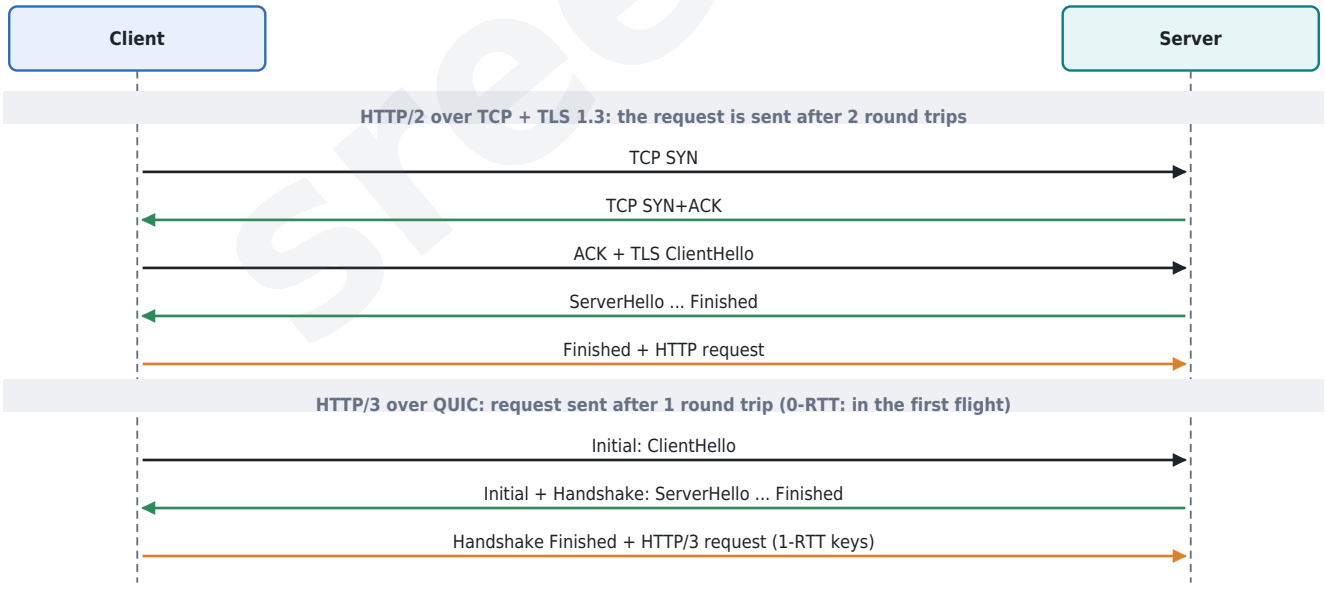


Figure 3.26. Time to the first request: TCP plus TLS 1.3 needs the TCP handshake and the TLS handshake (two round trips); QUIC combines them and needs one round trip, and zero with 0-RTT resumption.

The first packet of a client (**Initial**) must be padded to at least 1200 bytes, which limits the amplification that a forged source address could cause, and which means that QUIC requires a path MTU of at least that size.

3.8.2 Worked trace: time to first byte

The effect of the handshake structure can be calculated. The round-trip time is 80 ms and the server needs no processing time. Time is counted from the first packet that the client sends to the arrival of the first response byte at the client.

Protocol	Round trips before the request leaves	First response byte
HTTP/1.1 or HTTP/2 over TCP and TLS 1.2	3 (TCP 1, TLS 2)	3 x 80 + 80 = 320 ms
HTTP/2 over TCP and TLS 1.3	2 (TCP 1, TLS 1)	2 x 80 + 80 = 240 ms
HTTP/3 over QUIC, new connection	1	80 + 80 = 160 ms
HTTP/3 over QUIC, 0-RTT resumption	0	80 ms

Latency of the first response byte for RTT = 80 ms.

The figures neglect DNS resolution, server processing, and the initial congestion window; they show how the round trips accumulate, and why connection reuse and QUIC matter on high-latency paths. The same calculation for RTT = 10 ms gives 40, 30, 20 and 10 ms respectively.

3.8.3 Hands-on

HTTP/3 requires a QUIC implementation. The JDK `HttpClient` supports HTTP/2 since Java 11 and HTTP/3 since Java 26 (JEP 517, March 2026); the build machine runs Java 21 and has no external network, so the fragment below could not be run here. The client is configured with the preferred version and falls back to an older version if the server does not offer HTTP/3.

Http3Client.java (Java 26; not run on 21) JAVA

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class Http3Client {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newBuilder()
            .version(HttpClient.Version.HTTP_3)      // preferred version
            .build();
        HttpRequest req = HttpRequest.newBuilder(URI.create("https://example.org/")).build();
        HttpResponse<String> resp = client.send(req, HttpResponse.BodyHandlers.ofString());
        System.out.println(resp.version() + " " + resp.statusCode());
    }
}
```

With `curl` built with HTTP/3 support the equivalent test is `curl --http3 -I https://example.org/` (typical, not run). The negotiation is visible in the `alt-svc: h3=":443"` response header that an HTTP/2 server sends to advertise HTTP/3; the client then tries QUIC and keeps the TCP connection as a fallback. Server-side support in the Java ecosystem (Tomcat, Jetty, Netty) is still partial, so HTTP/3 is normally terminated at a CDN, a load balancer or a proxy such as nginx or Envoy, and the Spring Boot application behind it continues to see HTTP/1.1 or HTTP/2.

3.8.4 Failure modes

- **UDP blocked or throttled.** Some enterprise networks drop UDP 443. Clients fall back to TCP transparently; the visible effect is the loss of the latency benefits.
- **Load balancers without QUIC support.** A balancer that routes by the 4-tuple breaks connection migration; connection-ID-aware routing is required.
- **CPU cost.** User-space packet processing and encryption cost more CPU per byte than kernel TCP with offload; large-scale servers watch for this.
- **Observability.** Packet inspection tools cannot read QUIC headers because they are encrypted; diagnosis depends on endpoint logs (qlog) and metrics.
- **0-RTT replay.** Non-idempotent requests sent as early data can be replayed; servers must reject early data for such requests.

3.8.5 Questions to expect

1. **Why was QUIC built on UDP and not as a new IP protocol?** A new IP protocol number would be blocked by NATs and firewalls; UDP passes through them. User-space implementation also allows rapid deployment of changes.
2. **What is head-of-line blocking and how does QUIC avoid it?** A lost segment delays all data behind it in the same ordered stream. TCP has one stream per connection; QUIC keeps independent ordering per stream, so only the affected stream waits.
3. **How many round trips does an HTTPS request need on a new connection with TCP and with QUIC?** TCP with TLS 1.3: two round trips before the request is sent. QUIC: one; with 0-RTT resumption: none.
4. **What is connection migration?** A QUIC connection is identified by a connection ID rather than by the address 4-tuple, so it survives a change of IP address or network after path validation.
5. **Is QUIC encrypted?** Yes. TLS 1.3 is mandatory and protects the payload and most of the header, which also prevents middleboxes from interfering with the protocol.
6. **What is the risk of 0-RTT?** Replay of the early data. It is acceptable only for idempotent requests.
7. **Is HTTP/3 always faster than HTTP/2?** No. The advantage is largest on lossy and high-latency paths; on clean low-latency networks the difference is small and CPU use may be higher.
8. **Which versions of the HTTP semantics run over which transport?** HTTP/1.1 and HTTP/2 over TCP (usually with TLS); HTTP/3 over QUIC. The request and response semantics are the same; only the framing and transport differ.

KEY POINTS

QUIC (RFC 9000) is a UDP-based, encrypted transport with streams, loss recovery and congestion control in user space; HTTP/3 (RFC 9114) runs over it.

- Independent streams remove transport-level head-of-line blocking.
- The combined transport and TLS 1.3 handshake takes one round trip, zero with resumption (replay risk for early data).
- Connection IDs allow connection migration and load-balancer routing.
- Deployment is usually at a CDN or proxy; the JDK HttpClient supports HTTP/3 from Java 26.

sreeram

PART 4

The Application Layer and the Web

Name resolution, HTTP in its three generations, state, security headers, real-time channels and the protocols behind them.

In this part

- **4.1** The Domain Name System
- **4.2** HTTP/1.1
- **4.3** HTTP Caching and Conditional Requests
- **4.4** Cookies, Sessions and Authentication over HTTP
- **4.5** HTTP/2 and HTTP/3
- **4.6** REST API Design from the Network Perspective
- **4.7** Cross-Origin Resource Sharing (CORS)
- **4.8** WebSocket, Server-Sent Events and Long Polling
- **4.9** gRPC and Other Application Protocols

4.1 The Domain Name System

Network layers address hosts by IP address, but applications and users refer to services by name. The Domain Name System (DNS) is the distributed database that maps names such as `www.example.com` to records, most often IP addresses. Every outbound call made by a Spring Boot service to a hostname begins with a DNS lookup. A slow, stale or failing lookup therefore appears to the application as a slow, misrouted or failed request, although no HTTP exchange has taken place. DNS is also the first component that determines which server receives traffic, which makes it a central tool for load distribution and failover.

DNS solves three problems at once: it provides stable, human-readable names; it permits an address to change without changing the name; and it distributes both storage and administration, so that no single organisation holds the entire database. The design relies on a hierarchy of delegated zones and on aggressive caching governed by a time-to-live (TTL) value on every record.

4.1.1 Concept and mechanism

Namespace hierarchy and zones

A domain name is a sequence of labels separated by dots and read from right to left. The rightmost label is the top-level domain (TLD); the implicit empty label at the far right is the root. Each label is at most 63 bytes and a complete name is at most 253 characters in text form. A **zone** is the portion of the tree that one administrative authority manages. A parent zone delegates a child zone by publishing NS (name server) records for it. The root zone delegates `com`, `org` and `in`; the `com` zone delegates `example.com`; the `example.com` zone holds the records for `www.example.com`.

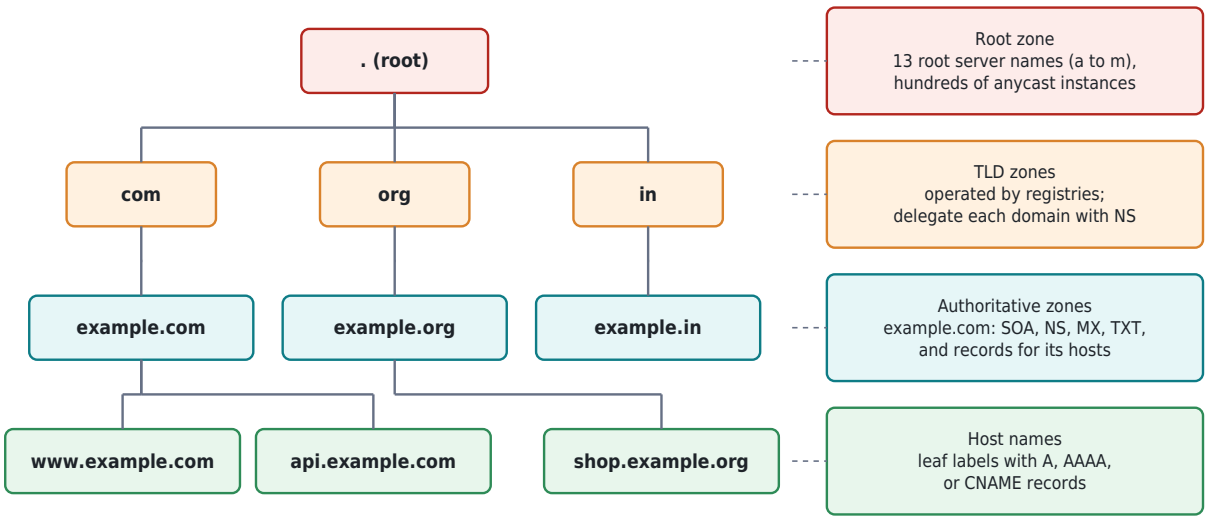


Figure 4.1. The namespace is a tree of delegations: each level knows only the name servers of the level directly below it, and the leaf records are held by the authoritative servers of the zone that owns the name.

Server roles

- **Authoritative name server**: holds the zone data for a zone and answers with the AA (authoritative answer) flag set. It does not search other servers.
- **Recursive resolver**: accepts a query from a client and performs whatever lookups are needed to produce a final answer, caching every result. Internet service providers, public resolvers and cloud virtual networks all operate recursive resolvers.
- **Stub resolver**: the minimal resolver inside the operating system or runtime. It forwards the query to a configured recursive resolver (`/etc/resolv.conf` on Linux) and waits for the answer.
- **Root and TLD servers**: authoritative servers for the two topmost levels. They never return the final address of an ordinary host; they return **referrals** to the next set of name servers.

A referral contains NS records for the child zone. When the name server names are inside the child zone itself, the parent also supplies **glue** records (the A or AAAA records of those name servers) to break the circular dependency.

Recursive and iterative resolution

The client sets the RD (recursion desired) bit and asks its recursive resolver for the complete answer; this is a **recursive query**. The resolver in turn asks each authoritative server in a chain and follows the referrals each returns; these are **iterative queries**. The resolver starts from a built-in list of root server addresses (the root hints) unless its cache already holds a closer starting point, such as the NS records for `com`.

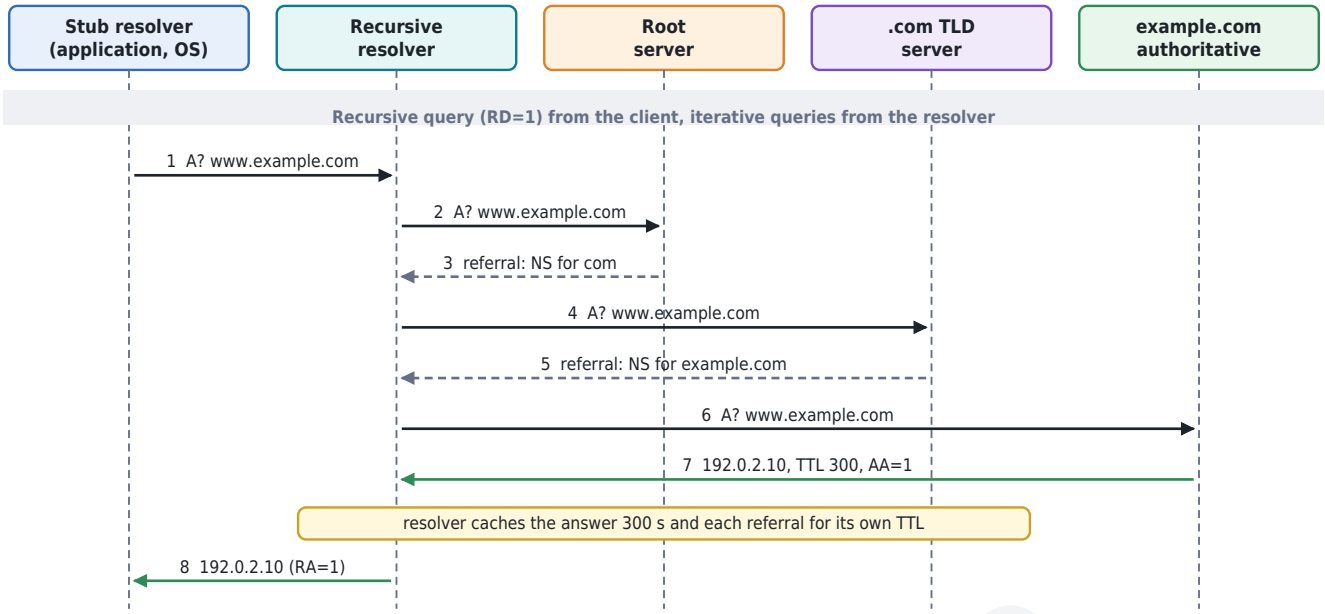


Figure 4.2. Resolution of www.example.com with an empty cache: one recursive query from the stub resolver, then three iterative queries from the recursive resolver to the root, the com TLD and the example.com authoritative server, with the final answer and its TTL cached by the resolver.

Transport and message format

DNS uses UDP port 53 for most queries because a single request and a single response need no connection. A server sets the TC (truncated) flag when the response does not fit; the client then repeats the query over TCP port 53. Extension mechanism EDNS(0) lets clients advertise a larger UDP payload size and is required by DNSSEC. DNS over TLS (port 853) and DNS over HTTPS encrypt the stub-to-resolver leg; they do not change the resolution logic.

Every DNS message has a 12-byte header followed by four variable sections: question, answer, authority and additional.

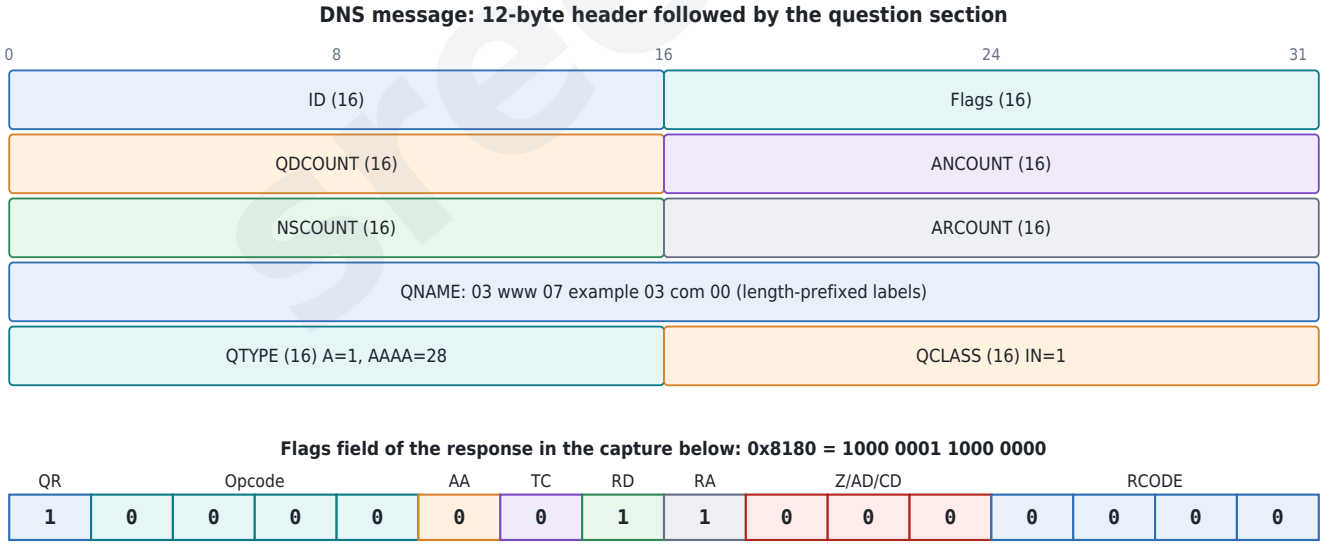


Figure 4.3. The 12-byte DNS header carries an identifier that matches responses to queries, a 16-bit flags field and four section counts; the question section encodes the name as length-prefixed labels terminated by a zero byte, and the flags of the captured response are expanded bit by bit.

Resource record types

Type	Content	Typical use
A	one IPv4 address	name to IPv4 address of a host
AAAA	one IPv6 address	name to IPv6 address
CNAME	canonical name	alias; the resolver restarts the lookup at the target name
NS	name server host name	delegation of a zone and publication of its servers
MX	priority and mail host	destination of e-mail for a domain
TXT	free text	SPF, DKIM, domain ownership verification
SOA	primary server, serial, refresh, retry, expire, negative TTL	one per zone; the last field governs negative caching
SRV	priority, weight, port, target	service location, used by Kubernetes headless services and SIP
PTR	host name	reverse lookup (address to name) under `in-addr.arpa`

Common DNS record types.

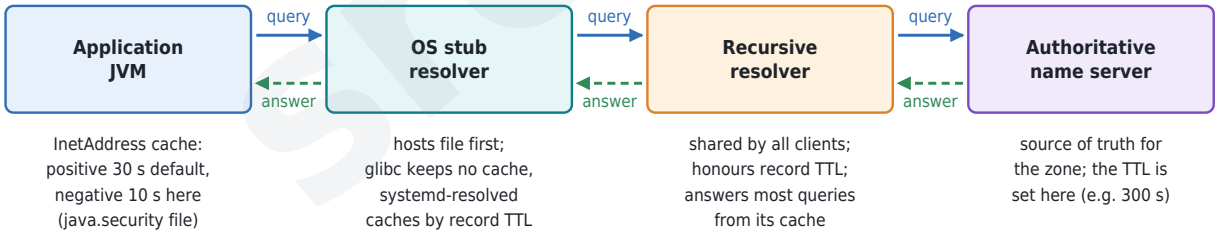
A CNAME record cannot coexist with other records at the same name, which is why the zone apex (`example.com`) cannot be a CNAME. Several cloud providers offer an ALIAS or ANAME pseudo-record to work around this restriction on the server side.

TTL, caching and negative caching

Every resource record carries a TTL in seconds. A cache may serve the record without a new query until the TTL expires and must decrement the remaining TTL as time passes. A response that proves a name does not exist (RCODE 3, NXDOMAIN) or has no record of the requested type (NODATA) is cached as well, for the lesser of the SOA record TTL and the SOA minimum field (RFC 2308). This **negative caching** protects authoritative servers from repeated queries but also delays the visibility of a newly created name.

The TTL is therefore the price of agility. A 300-second TTL allows traffic to move to a new address within about five minutes; a 86400-second TTL generates fewer queries but keeps a stale address alive for a day. The standard operational practice before a planned migration is to lower the TTL at least one old-TTL period in advance, perform the change, then raise it again.

Caches exist at four levels in a typical Java deployment.



A cache hit at any level ends the lookup there. Each level obeys the TTL it received (the JVM caps it with its own setting). Stale data survives until the nearest cache entry expires.

Figure 4.4. Four caches stand between a Java call and the authoritative data: the JVM address cache, the operating-system stub resolver, the shared recursive resolver and the authoritative server; a hit at any level terminates the lookup.

DNS-based load distribution

An authoritative server can return several A records for one name (round-robin DNS); clients choose among them, and `InetAddress.getAllByName` exposes the full list. More advanced services answer according to the client location (geo-DNS), the health of each endpoint, or fixed weights. DNS-based balancing has structural limits: the answer is cached for the TTL, so the failure of one address remains visible to some clients until the cache expires; clients often use the first address returned; and the authoritative server sees the recursive resolver address, not that of the end client, unless EDNS Client Subnet is in use. Layer-4 and layer-7 load balancers, covered in Part 5, complement rather than replace DNS.

sreeram

4.1.2 Worked trace

The following trace follows `www.example.com` through the diagram above. The addresses use the documentation range 192.0.2.0/24 and the example name servers are illustrative.

Step	Message	Key fields
1	Stub resolver to recursive resolver (UDP 53)	ID 0xA1B2, flags 0x0100 (RD=1), QNAME www.example.com, QTYPE A (1), QCLASS IN (1)
2	Recursive resolver to a root server	same question, RD=0 (iterative)
3	Root server to resolver	no answer; authority section: NS records for `com`; additional section: glue A/AAAA records
4	Resolver to a `com` TLD server	same question
5	TLD server to resolver	referral: NS records for `example.com` (for instance `ns1.example.com`) with glue 198.51.100.53
6	Resolver to 198.51.100.53	same question
7	Authoritative server to resolver	flags 0x8400 (QR=1, AA=1); answer: `www.example.com` A 192.0.2.10, TTL 300
8	Resolver to stub resolver	flags 0x8180 (QR=1, RD=1, RA=1); ANCOUNT 1; the answer is cached for 300 s

Resolution of www.example.com with a cold cache.

A second lookup of the same name within 300 seconds ends at step 1: the recursive resolver answers from its cache, with the remaining TTL in the answer. A lookup of `api.example.com` immediately afterwards skips steps 2 to 5, because the resolver already holds the NS records of `example.com` and the TLD referral is still cached.

4.1.3 Hands-on

Building a DNS query by hand

The following Python program encodes a query for `www.example.com`, builds the matching response that an authoritative server would send, parses the answer and exchanges both messages over UDP on the loopback interface. No external resolver is contacted. The byte values are the real output of the program.

dns_demo.py — PYTHON

```python
import socket
import struct
import threading


def encode_name(name):
    return b"".join(bytes([len(p)]) + p.encode("ascii") for p in name.split(".")) + b"\x00"


def hexdump(data):
    for off in range(0, len(data), 16):
        chunk = data[off:off + 16]
        print("%04x  %s" % (off, " ".join("%02x" % b for b in chunk)))


QUESTION = encode_name("www.example.com") + struct.pack("!HH", 1, 1)      # QTYPE=A, QCLASS=IN
query = struct.pack("!HHHHHH", 0xA1B2, 0x0100, 1, 0, 0, 0) + QUESTION     # RD=1, one question
answer = (b"\xc0\x0c" + struct.pack("!HHIH", 1, 1, 300, 4) + bytes([192, 0, 2, 10]))
response = struct.pack("!HHHHHH", 0xA1B2, 0x8180, 1, 1, 0, 0) + QUESTION + answer

print("query: %d bytes" % len(query))
hexdump(query)
print("response: %d bytes" % len(response))
hexdump(response)

# Parse the answer section: NAME is a compression pointer (0xC00C = offset 12).
pos = 12 + len(QUESTION)
name, rtype, rclass, ttl, rdlen = struct.unpack("!HHHIH", response[pos:pos + 12])
rdata = response[pos + 12:pos + 12 + rdlen]
print("answer: name=0x%04x type=%d class=%d ttl=%d rdlength=%d address=%s"
      % (name, rtype, rclass, ttl, rdlen, ".".join(str(b) for b in rdata)))
```

dns_demo.py (continued) PYTHON

```python
# Exchange both messages over UDP on the loopback interface (a stand-in for a real resolver).
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 0))


def serve():
    data, addr = server.recvfrom(512)
    server.sendto(data[:2] + response[2:], addr)      # echo the ID of the query


threading.Thread(target=serve).start()
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.settimeout(2)
client.sendto(query, server.getsockname())
reply, _ = client.recvfrom(512)
print("reply over UDP: %d bytes, ID=0x%04x, ANCOUNT=%d"
      % (len(reply), struct.unpack("!H", reply[:2])[0], struct.unpack("!H", reply[6:8])[0]))
```

Output of dns_demo.py TERMINAL

```
$ python3 dns_demo.py
query: 33 bytes
0000  a1 b2 01 00 00 01 00 00 00 00 00 00 03 77 77 77
0010  07 65 78 61 6d 70 6c 65 03 63 6f 6d 00 00 01 00
0020  01
response: 49 bytes
0000  a1 b2 81 80 00 01 00 01 00 00 00 00 03 77 77 77
0010  07 65 78 61 6d 70 6c 65 03 63 6f 6d 00 00 01 00
0020  01 c0 0c 00 01 00 01 00 00 01 2c 00 04 c0 00 02
0030  0a
answer: name=0xc00c type=1 class=1 ttl=300 rdlength=4 address=192.0.2.10
reply over UDP: 49 bytes, ID=0xa1b2, ANCOUNT=1
```

Reading the hexadecimal dump: bytes 0 to 1 are the ID `a1 b2`; bytes 2 to 3 are the flags (`01 00` for the query, `81 80` for the response); bytes 4 to 11 are the four counts. The name starts at byte 12 as `03 77 77 77` (length 3, "www"), `07 ...` (length 7, "example"), `03 63 6f 6d` (length 3, "com") and the terminating `00`. Two 16-bit values follow: type `00 01` and class `00 01`. In the response, `c0 0c` is a **compression pointer**: the two high bits `11` mark a pointer and the remaining 14 bits give the offset (12) of the earlier occurrence of the name. The TTL `00 00 01 2c` is 300 and the RDATA `c0 00 02 0a` is 192.0.2.10.

Name resolution in Java

`java.net.InetAddress` is the entry point. It delegates to the platform resolver (on Linux the C library, which consults `/etc/hosts` and then the servers in `/etc/resolv.conf`) and caches the result inside the JVM.

ResolveDemo.java JAVA

```java
import java.net.InetAddress;
import java.net.UnknownHostException;
import java.security.Security;

public class ResolveDemo {
    public static void main(String[] args) throws Exception {
        InetAddress lo = InetAddress.getByName("localhost");
        System.out.println(lo + " loopback=" + lo.isLoopbackAddress());
        for (InetAddress a : InetAddress.getAllByName("localhost")) {
            System.out.println("getAllByName: " + a);
        }
        InetAddress literal = InetAddress.getByName("192.0.2.10");
        System.out.println(literal + " is a " + literal.getClass().getSimpleName()
                + " (no lookup)");
        System.out.println("cache.ttl = " + Security.getProperty("networkaddress.cache.ttl"));
        System.out.println("cache.negative.ttl = "
                + Security.getProperty("networkaddress.cache.negative.ttl"));
        try {
            InetAddress.getByName("no-such-host.invalid");
        } catch (UnknownHostException e) {
            System.out.println("UnknownHostException: " + e.getMessage());
        }
    }
}
```

Output of ResolveDemo.java (JDK 21 on the build machine) TERMINAL

```
$ java ResolveDemo.java
localhost/127.0.0.1 loopback=true
getAllByName: localhost/127.0.0.1
/192.0.2.10 is a Inet4Address (no lookup)
cache.ttl = null
cache.negative.ttl = 10
UnknownHostException: no-such-host.invalid: Name or service not known
```

What to notice. A literal address is parsed locally and produces no query. `toString()` prints `hostname/address`; the host name part is empty for a literal. The property `networkaddress.cache.ttl` is unset (`null`) in the `java.security` file of this JDK; the documented behaviour is then a 30-second cache when no `SecurityManager` is installed, and a cache that never expires when one is installed. The negative cache lasts 10 seconds. The `.invalid` top-level domain is reserved (RFC 6761) and never resolves, so it is the standard choice for testing failure handling.

Controlling the JVM cache

The cache policy is set either in the file `conf/security/java.security` of the JDK, or programmatically before the first lookup in the process:

DnsTtl.java JAVA

```java
import java.security.Security;

public class DnsTtl {
    public static void main(String[] args) {
        // Must run before the first name lookup in the JVM.
        // seconds; -1 = forever, 0 = off
        Security.setProperty("networkaddress.cache.ttl", "30");
        Security.setProperty("networkaddress.cache.negative.ttl", "5");
        System.out.println(Security.getProperty("networkaddress.cache.ttl"));
    }
}
```

Long-running services that connect to endpoints whose addresses change (managed load balancers, database failover endpoints, service discovery names) must keep this value small. A value of -1, which older application server images sometimes configure for security reasons, pins the first resolved address for the life of the process and turns a routine address change into an outage. The HTTP connection pools described in later chapters add a second layer: an open pooled connection is bound to an IP address and does not re-resolve until it is closed.

The dig command (typical output)

The `dig` tool is not installed on the build machine; the following is typical output for the authoritative answer in the trace, using documentation addresses.

Typical dig output (not run here) TERMINAL

```
$ dig www.example.com A +noall +answer +comments
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41394
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
www.example.com.    300    IN    A    192.0.2.10
$ dig +trace www.example.com
# +trace performs the iterative walk itself: root, then com, then example.com
```

4.1.4 Failure modes

Symptom	Likely cause	Diagnosis
`UnknownHostException` immediately	name does not exist, typing error, wrong search domain, negative cache	resolve the same name with `dig` from the same host or container; check RCODE (NXDOMAIN versus SERVFAIL)
Call hangs for 2 to 5 seconds, then fails or succeeds	first resolver in `resolv.conf` unreachable; timeout and retry settings (`timeout:2 attempts:3`) apply	compare latency with each resolver; remove the dead entry
Traffic keeps reaching an old server after migration	TTL not yet expired in a resolver or in the JVM cache	inspect the remaining TTL in the answer; check `networkaddress.cache.ttl`
Intermittent failures in Kubernetes	`ndots:5` search list multiplies queries; CoreDNS under load; UDP packet loss	count queries per lookup; use fully qualified names with a trailing dot
SERVFAIL	broken delegation, unreachable authoritative servers, DNSSEC validation failure	`dig +trace`; check the NS records at the parent and at the child

Typical DNS failures and their diagnosis.

Kubernetes deserves separate mention. Pods receive a `resolv.conf` that points to the cluster DNS service (CoreDNS) and contains a search list such as `shop.svc.cluster.local svc.cluster.local cluster.local` with `options ndots:5`. A service is reachable as `orders.shop.svc.cluster.local`. Because a name with fewer than five dots is first tried with each search suffix, a lookup of an external name such as `api.example.com` can generate several failed queries before the correct one. Using fully qualified names that end in a dot, or lowering `ndots` in the pod specification, removes the overhead.

WATCH OUT
The JVM caches successful lookups for 30 seconds by default, but only when no `SecurityManager` is set. Container images and application servers that set `networkaddress.cache.ttl=-1` keep the first resolved address forever. A managed load balancer or database endpoint that changes address then requires a restart of the service. Verify the effective value in the deployed JDK, not only in development.

GOTCHA
An answer to `getAllByName` can contain both IPv6 and IPv4 addresses. If the host has no working IPv6 route, a connection attempt to the first (IPv6) address waits for a timeout before the next address is tried. The JVM options `-Djava.net.preferIPv4Stack=true` and `-Djava.net.preferIPv6Addresses` change the ordering and family used.

4.1.5 Questions to expect

1. What is the difference between recursive and iterative resolution?

In a recursive query the client delegates the entire task and receives either the final answer or an error. In an iterative query the server returns the best information it has, usually a referral, and the asker contacts the next server itself. A stub resolver sends recursive queries to a recursive resolver; the recursive resolver sends iterative queries to root, TLD and authoritative servers.

2. What happens in DNS when a user opens a URL for the first time?

The browser or JVM checks its own cache, then the operating-system resolver (hosts file, then the configured recursive resolver). The recursive resolver checks its cache; on a miss it queries a root server, a TLD server and the authoritative server in turn and caches every answer for its TTL. The address is returned to the client, which then opens a TCP connection.

3. Why is DNS usually carried over UDP, and when does it use TCP?

A query and its response each fit in one datagram, so UDP avoids the handshake latency and the per-connection state. TCP is used when the response is truncated (TC bit), for zone transfers, and for DNS over TLS. Responses larger than the advertised UDP size, which DNSSEC makes common, trigger the TCP retry.

4. What does a TTL of 300 mean, and what is the cost of a low TTL?

Caches may reuse the record for 300 seconds after retrieval. A low TTL speeds up changes and failover but multiplies queries to the authoritative servers and adds lookup latency for clients whose cache has expired. A high TTL has the opposite effect.

5. Why can a CNAME not be placed at the zone apex?

A CNAME states that a name is an alias of another and forbids other record types at that name. The apex must hold SOA and NS records, so a CNAME there would be invalid. Provider-specific ALIAS records synthesise the answer on the server side.

6. How does the JVM cache DNS, and how can it cause an outage?

`InetAddress` caches results according to `networkaddress.cache.ttl` (30 s by default without a security manager; forever when set to -1 or when a security manager is present). A forever-cache keeps an obsolete address after a failover or load balancer change. The remedy is a short positive TTL and awareness that pooled connections are not re-resolved.

7. What is negative caching?

Caching of the fact that a name or record type does not exist. The TTL is derived from the SOA record (RFC 2308). It prevents repeated queries for non-existent names but delays visibility of newly created names.

8. What is the difference between NXDOMAIN and SERVFAIL?

NXDOMAIN (RCODE 3) is an authoritative statement that the name does not exist. SERVFAIL (RCODE 2) means that the server could not complete the resolution: an unreachable authoritative server, a broken delegation or a failed DNSSEC validation.

INTERVIEW ANGLE

Follow-up chain: "A service was migrated to a new IP address an hour ago; some clients still reach the old one. Why?" Candidate causes in order: the record TTL has not expired in a recursive resolver; the operating-system or JVM cache holds the old address; a pooled connection is still open to the old server; a client or proxy uses a hard-coded address. The first remedy is to check the remaining TTL with `dig`, then the JVM property, then the connection pool lifetime.

KEY POINTS

- DNS maps names to records through a hierarchy of delegated zones; root and TLD servers return referrals, authoritative servers return answers.
- Stub resolvers ask recursive resolvers; recursive resolvers walk the hierarchy iteratively and cache every answer for its TTL, including negative answers.
- Queries use UDP port 53 with a TCP fallback; the message is a 12-byte header plus question, answer, authority and additional sections, with name compression pointers.
- The JVM has its own cache (`networkaddress.cache.ttl`, 30 s by default without a security manager); a value of -1 can cause outages after address changes.
- Many failures that look like HTTP problems are DNS problems: check resolution first when a call fails before any connection is made.

4.2 HTTP/1.1

The HyperText Transfer Protocol (HTTP) is the application-layer protocol on which web pages, REST APIs, service-to-service calls and most mobile back ends are built. HTTP defines how a client formulates a request, how a server formulates a response, and which parts of the exchange are visible to intermediaries such as proxies, caches and load balancers. HTTP/1.1, specified in RFC 9112 (message syntax) and RFC 9110 (semantics shared by all versions), remains the default for many internal services and the baseline that HTTP/2 and HTTP/3 preserve at the semantic level: methods, status codes and header fields keep their meaning; only the encoding on the wire changes.

HTTP is **stateless**: each request carries all information the server needs to process it, and the protocol itself keeps no memory of earlier requests. State (sessions, authentication) is layered on top by cookies and tokens, treated in a later chapter. HTTP/1.1 is also a **text protocol** over a reliable byte stream, normally TCP, which makes it easy to inspect with `curl -v` or `nc` and easy to misframe when a message length is wrong.

4.2.1 Concept and mechanism

Message format

A request and a response share one structure: a start line, zero or more header fields, an empty line and an optional body. Lines end with CRLF (carriage return, line feed). The start line of a request holds the method, the request target and the protocol version; the start line of a response holds the version, a three-digit status code and a reason phrase.

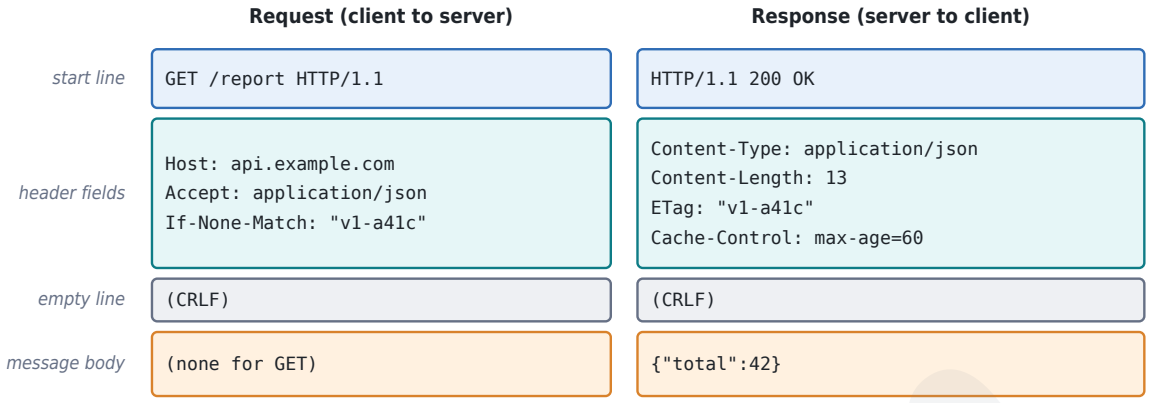


Figure 4.5. The anatomy of an HTTP/1.1 exchange: a start line, header fields (name, colon, value), an empty line that ends the header section, and an optional body whose length is announced by Content-Length or by chunked transfer coding.

Header field names are case-insensitive. The implementation in the JDK `HttpServer`, for example, emits `Content-type` and `Etag`, while Spring Boot emits `Content-Type` and `ETag`; both are equivalent on the wire. Values are not case-insensitive in general.

Methods, safety and idempotency

A **safe** method is defined not to change server state; a client may call it freely, and crawlers and prefetchers do. An **idempotent** method has the same intended effect on the server whether it is performed once or many times. Idempotency determines whether a failed call can be retried automatically after a timeout, when the client cannot know whether the first attempt succeeded.

Method	Safe	Idempotent	Cacheable	Typical use
`GET`	yes	yes	yes	read a representation of a resource
`HEAD`	yes	yes	yes	same headers as GET, no body; check existence or size
`POST`	no	no	rarely	create a resource, submit a command, run a non-idempotent action
`PUT`	no	yes	no	replace a resource at a known URI
`PATCH`	no	no	no	partial update; idempotent only if the patch format makes it so
`DELETE`	no	yes	no	remove a resource; a repeated call may return 404, the state is the same
`OPTIONS`	yes	yes	no	discover capabilities; used by CORS preflight

Request methods (RFC 9110).

Safe and idempotent describe the intended semantics, not what the code does. A `GET` handler that decrements a stock counter violates the contract and will cause damage when a cache, a link prefetcher or an automatic retry replays it.

Status codes

The first digit of the status code gives the class. Clients must treat an unrecognised code as the generic code of its class (for example, an unknown 4xx as 400).

Class	Meaning	Common codes
1xx	informational	100 Continue (client may send the body), 101 Switching Protocols (WebSocket upgrade)
2xx	success	200 OK, 201 Created (with `Location`), 202 Accepted (asynchronous work), 204 No Content, 206 Partial Content

Class	Meaning	Common codes
3xx	redirection	301 Moved Permanently, 302 Found, 304 Not Modified, 307 and 308 (redirect that preserves the method)
4xx	client error	400 Bad Request, 401 Unauthorized (authentication missing or invalid), 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 409 Conflict, 415 Unsupported Media Type, 422 Unprocessable Content, 429 Too Many Requests
5xx	server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable (with `Retry-After`), 504 Gateway Timeout

Status code classes and the codes used most often.

The distinction between 502, 503 and 504 matters in production behind a proxy: 502 means the proxy received an invalid response or no response from the upstream, 503 means the server or proxy is unable to serve right now, and 504 means the upstream did not answer within the proxy timeout. These are revisited in the troubleshooting chapters.

Header fields that matter

Header	Direction	Purpose
`` `Host` ``	request	mandatory in HTTP/1.1; selects the virtual host and, behind proxies, the application
`` `Content-Type`, `Accept` ``	both	media type of the body; media types the client accepts (content negotiation)
`` `Content-Length` ``	both	exact body size in bytes
`` `Transfer-Encoding: chunked` ``	both	body sent as length-prefixed chunks when the size is not known in advance
`` `Connection` ``	both	`close` ends the connection after this message; `keep-alive` or absence keeps it open (HTTP/1.1 default)
`` `Authorization` ``	request	credentials (Basic, Bearer)
`` `Cache-Control`, `ETag`, `If-None-Match` ``	both	caching and validation (next chapter)
`` `Location` ``	response	URI of a created resource (201) or redirect target (3xx)
`` `Retry-After` ``	response	seconds or date at which a retry is acceptable (429, 503)
`` `X-Forwarded-For`, `Forwarded` ``	request	original client address added by proxies
`` `User-Agent`, `Origin`, `Referer` ``	request	client software; page origin for CORS; previous page

Header fields with the largest operational effect.

Persistent connections, pipelining and head-of-line blocking

HTTP/1.0 opened a TCP connection for each request, and every request therefore paid the three-way handshake (one round trip) and the TCP slow start. HTTP/1.1 makes connections **persistent** by default: the connection stays open after a response, and the next request reuses it. The connection ends when either side sends `Connection: close`, when an idle timeout expires, or when the server reaches its maximum number of requests per connection.

HTTP/1.1 also permits **pipelining**: sending several requests without waiting for the responses. The server must return the responses in the order of the requests. A slow response at the head of the pipeline therefore blocks all later responses, even those that are ready: this is **head-of-line (HOL) blocking at the HTTP layer**. Because of this behaviour, together with failures of intermediaries to handle pipelines, browsers disabled pipelining. They open up to six parallel TCP connections per host instead, which multiplies handshakes and congestion windows. HTTP/2 removes the problem with multiplexed streams.



Figure 4.6. Three connection strategies for three requests: a new connection per request costs six round trips, a persistent connection costs four, and pipelining on one connection still forces responses 2 and 3 to wait for the slow response 1.

Message framing

The receiver must know where a message ends, otherwise it cannot find the start of the next one on a persistent connection. HTTP/1.1 offers two mechanisms for the body.

- `Content-Length: N`: exactly N bytes follow the empty line.
- `Transfer-Encoding: chunked`: the body is a series of chunks, each preceded by its size in hexadecimal and CRLF, and the series ends with a chunk of size 0. The sender needs no advance knowledge of the total size, which suits generated content, streaming and Server-Sent Events.

If a message has neither, the response body extends until the server closes the connection, which prevents reuse. When both headers appear in one message, a request is ambiguous; intermediaries and servers that disagree about which one governs enable **request smuggling** attacks, so current servers reject such messages.

Host header and virtual hosting

A single IP address and port can serve many sites. The client sends the name it used in the URL as the `Host` header; the server (or the reverse proxy in front of it) selects the site from that value. HTTPS uses a corresponding mechanism at the TLS layer, Server Name Indication, described in Part 5.

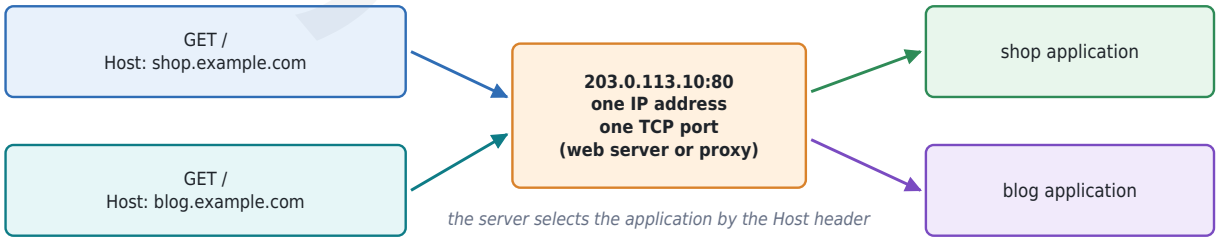


Figure 4.7. Two requests reach the same address and port; the server distinguishes the applications only by the value of the Host header.

Other features of HTTP/1.1

- **Range requests**: `Range: bytes=0-1023` with a `206 Partial Content` response enables resumable downloads and media seeking.
- **Expect: 100-continue**: the client sends the headers first and waits for `100 Continue` before transmitting a large body, so that a rejection (401, 413) saves the upload.
- **Redirects**: 301 and 302 allow clients to change a POST into a GET; 307 and 308 require the method and body to be preserved.

sreram

- **Content negotiation**: `Accept`, `Accept-Language` and `Accept-Encoding` (gzip, br) let the server select a representation; the response must then carry `Vary` so that caches key on those headers.

4.2.2 Worked trace

The trace below follows the exchange produced by `POST /orders` against the local server of the hands-on section. The values are those captured with `curl -v` and the Java `HttpClient` on the loopback interface.

Step	Bytes on the wire (summarised)	Meaning
1	TCP SYN, SYN-ACK, ACK to 127.0.0.1:18080	connection established, one round trip
2	`POST /orders HTTP/1.1`	start line: method, target, version
3	`Host: 127.0.0.1:18080`, `Content-Type: application/json`, `Content-Length: 23`, `Idempotency-Key: 7c9e-41`	header fields; the length announces the body
4	empty line, then `{"item":"book","qty":2}`	end of headers; 23 bytes of body
5	`HTTP/1.1 201 Created`	status line
6	`Content-type: application/json`, `Content-length: 59`, `Location: /orders/1001`	response header fields
7	empty line, then `{"id":1001,"echo":{"item":"book","qty":2},"key":"7c9e-41"}`	59 bytes of body
8	connection remains open	the next request reuses it without a new handshake

Exchange of a POST request on one persistent TCP connection.

The request body `{"item":"book","qty":2}` has 23 characters, which is the value of `Content-Length`; the response body has 59. The size of each message is thus derivable from its visible content, and a mismatch (a body longer or shorter than announced) corrupts the framing of the next message on the same connection.

4.2.3 Hands-on

A local HTTP server

The following class uses `com.sun.net.httpserver.HttpServer`, which is part of the JDK (module `jdk.httpserver`) and suitable for experiments and tests. It listens on 127.0.0.1:18080 and defines the endpoints used by this and the following chapters: Host echo, validation with an ETag, chunked output, a session cookie, a POST with an idempotency key, a CORS endpoint and a Server-Sent Events stream. The listing is long but each handler is independent.

LocalServer.java JAVA

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;

public class LocalServer {
    static final String ETAG = "\"v1-a41c\"";

    static void send(HttpExchange ex, int code, String body) throws IOException {
        byte[] b = body.getBytes(StandardCharsets.UTF_8);
        ex.sendResponseHeaders(code, b.length == 0 ? -1 : b.length);   // -1 = no body
        if (b.length > 0) {
            try (OutputStream os = ex.getResponseBody()) {
                os.write(b);
            }
        }
        ex.close();
    }

    public static void main(String[] args) throws Exception {
        HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", 18080), 0);

        s.createContext("/hello", ex -> {                      // echoes the Host header
            ex.getResponseHeaders().add("Content-Type", "text/plain; charset=utf-8");
            send(ex, 200, "Hello, " + ex.getRequestHeaders().getFirst("Host") + "\n");
```

LocalServer.java (continued) JAVA

```
        });

        s.createContext("/report", ex -> {                    // validation with an ETag
            String inm = ex.getRequestHeaders().getFirst("If-None-Match");
            ex.getResponseHeaders().add("ETag", ETAG);
            ex.getResponseHeaders().add("Cache-Control", "max-age=60, must-revalidate");
            if (ETAG.equals(inm)) {
                send(ex, 304, "");
                return;
            }
            ex.getResponseHeaders().add("Content-Type", "application/json");
            send(ex, 200, "{\"total\":42}\n");
        });

        s.createContext("/stream", ex -> {                    // length 0 selects chunked encoding
            ex.getResponseHeaders().add("Content-Type", "text/plain");
            ex.sendResponseHeaders(200, 0);
            try (OutputStream os = ex.getResponseBody()) {
                os.write("first part\n".getBytes());
                os.flush();
                os.write("second part\n".getBytes());
            }
        });

        s.createContext("/login", ex -> {                     // issues a session cookie
            ex.getResponseHeaders().add("Set-Cookie",
                    "SESSION=abc123; Path=/; HttpOnly; SameSite=Lax; Max-Age=3600");
            send(ex, 204, "");
        });

        s.createContext("/orders", ex -> {                    // POST with an idempotency key
            String key = ex.getRequestHeaders().getFirst("Idempotency-Key");
            String body = new String(ex.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
            ex.getResponseHeaders().add("Content-Type", "application/json");
            ex.getResponseHeaders().add("Location", "/orders/1001");
            send(ex, 201, "{\"id\":1001,\"echo\":" + body + ",\"key\":\"" + key + "\"}\n");
        });

        s.createContext("/api", ex -> {                       // CORS for one allowed origin
            String origin = ex.getRequestHeaders().getFirst("Origin");
            if ("https://app.example.com".equals(origin)) {
                ex.getResponseHeaders().add("Access-Control-Allow-Origin", origin);
                ex.getResponseHeaders().add("Vary", "Origin");
            }
            if ("OPTIONS".equals(ex.getRequestMethod())) {
                ex.getResponseHeaders().add("Access-Control-Allow-Methods", "GET, PUT, DELETE");
                ex.getResponseHeaders().add("Access-Control-Allow-Headers",
                        "Content-Type, Authorization");
                ex.getResponseHeaders().add("Access-Control-Max-Age", "600");
                send(ex, 204, "");
                return;
            }
            send(ex, 200, "{\"ok\":true}\n");
        });

        s.createContext("/events", ex -> {                    // Server-Sent Events
            ex.getResponseHeaders().add("Content-Type", "text/event-stream");
            ex.getResponseHeaders().add("Cache-Control", "no-cache");
            ex.sendResponseHeaders(200, 0);
            try (OutputStream os = ex.getResponseBody()) {
                for (int i = 1; i <= 3; i++) {
                    String event = "id: " + i + "\nevent: price\n"
                            + "data: {\"sku\":\"A1\",\"v\":" + (100 + i) + "}\n\n";
                    os.write(event.getBytes(StandardCharsets.UTF_8));
                    os.flush();
                    try {
                        Thread.sleep(200);
                    } catch (InterruptedException e) {
                        return;
                    }
                }
            }
        });
```

LocalServer.java (continued) JAVA

```
        s.start();
        System.out.println("listening on 127.0.0.1:18080");
    }
}
```

What to notice. `sendResponseHeaders(code, length)` takes three kinds of length: a positive value is sent as `Content-length`; 0 selects chunked transfer coding; -1 means that no body follows (used for 204 and 304). The server runs handlers on its default executor, which is a single thread; this is acceptable for experiments and not for load tests. Port 18080 is used because 8080 is frequently occupied.

Start the server in one terminal (`java LocalServer.java`) and run the following commands in another.

Reading a request and response with curl

curl -v against the local server (trimmed to request and response lines) TERMINAL

```
$ curl -sv http://127.0.0.1:18080/hello
> GET /hello HTTP/1.1
> Host: 127.0.0.1:18080
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Sat, 17 Oct 2026 16:48:27 GMT
< Content-type: text/plain; charset=utf-8
< Content-length: 23
<
Hello, 127.0.0.1:18080
```

Lines prefixed with `>` are sent by the client, lines prefixed with `<` are received, and lines prefixed with `*` are curl diagnostics. The empty `>` and `<` lines are the CRLF that ends each header section.

Virtual hosting with the Host header

The same address and port serve a different response when only the `Host` header changes; no DNS entry is required for the experiment.

Changing only the Host header TERMINAL

```
$ curl -s -H 'Host: shop.example.com' http://127.0.0.1:18080/hello
Hello, shop.example.com
$ curl -s -H 'Host: blog.example.com' http://127.0.0.1:18080/hello
Hello, blog.example.com
```

Connection reuse

Two URLs on the same host given to one curl invocation share a connection. The diagnostic line `Re-using existing connection` confirms it, and the second request required no new handshake.

Two requests on one connection TERMINAL

```
$ U=http://127.0.0.1:18080
$ curl -sv $U/hello $U/report -o /dev/null -o /dev/null 2>&1 | grep -E 'Connected|Re-using|GET'
* Connected to 127.0.0.1 (127.0.0.1) port 18080
> GET /hello HTTP/1.1
* Re-using existing connection with host 127.0.0.1
> GET /report HTTP/1.1
```

Chunked transfer coding on the wire

The option `--raw` disables curl decoding, so that the chunk framing becomes visible. Sizes are hexadecimal: `b` is 11 bytes (`first part` and a newline), `c` is 12 bytes, and `0` terminates the body.

Raw chunked response TERMINAL

```
$ curl -si --raw http://127.0.0.1:18080/stream
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 16:48:27 GMT
Transfer-encoding: chunked
Content-type: text/plain

b
```

Raw chunked response (continued) — TERMINAL

```
first part

c
second part

0
```

Pipelining by hand

The Python fragment sends two requests in one write on one connection and reads both responses. The server returns them in request order.

Pipe.py — PYTHON

```python
import socket
import time

s = socket.create_connection(("127.0.0.1", 18080))
s.sendall(b"GET /hello HTTP/1.1\r\nHost: a.example.com\r\n\r\n"
          b"GET /report HTTP/1.1\r\nHost: a.example.com\r\n\r\n")
time.sleep(0.5)
print(s.recv(4096).decode())
```

Output of Pipe.py (the Date header is omitted here) — TERMINAL

```
$ python3 Pipe.py
HTTP/1.1 200 OK
Content-type: text/plain; charset=utf-8
Content-length: 21

Hello, a.example.com
HTTP/1.1 200 OK
Content-type: application/json
Etag: "v1-a41c"
Content-length: 13
Cache-control: max-age=60, must-revalidate

{"total":42}
```

Spring Boot and Tomcat settings for HTTP/1.1 connections

An embedded Tomcat connector, which serves a Spring Boot application, exposes the connection behaviour discussed above through the following properties. They are configuration keys; the defaults depend on the Boot and Tomcat versions and should be confirmed in the deployed version.

Property	Effect at the TCP and HTTP level
`server.tomcat.threads.max`	maximum number of request-processing threads (one blocking request per thread)
`server.tomcat.max-connections`	maximum number of connections accepted and held open by the connector
`server.tomcat.accept-count`	length of the queue of pending connections when all of `max-connections` are in use
`server.tomcat.connection-timeout`	time to wait, after accepting a connection, for the request line and headers
`server.tomcat.keep-alive-timeout`	idle time before a persistent connection is closed
`server.tomcat.max-keep-alive-requests`	number of requests served on one connection before the server closes it

Spring Boot properties relevant to HTTP/1.1 connections.

4.2.4 Failure modes

Symptom	Cause	Diagnosis
Client hangs after a POST	`Content-Length` larger than the body actually sent; the server waits for the rest	`curl -v`; compare the announced and actual lengths
Garbled response on a reused connection	an earlier response had a wrong length, so the parser started the next message in the middle	disable reuse (`Connection: close`) to confirm; fix the length
`Connection reset` on the first request after idle	the server or a load balancer closed the idle connection while the client still held it in its pool	set the client idle-eviction time below the server keep-alive timeout
Intermittent 502 behind a proxy	proxy reuses an upstream connection that the application has just closed	align keep-alive timeouts: the upstream must outlast the proxy
Wrong site served	missing or wrong `Host` header; a proxy rewrote it	`curl -v -H 'Host: ...'` and inspect the proxy configuration
Unexpected redirect loop	the application sees `http` because TLS terminates at a proxy	forward `X-Forwarded-Proto` and enable forwarded-header handling in the framework

Typical HTTP/1.1 failures and their diagnosis.

WATCH OUT

A client that retries a POST after a timeout without an idempotency key can create duplicate orders or payments. A timeout leaves the outcome unknown: the request may have been processed. Retry automatically only idempotent methods, or add an idempotency key that the server stores (see the chapter on REST API design).

GOTCHA

A response without `Content-Length` and without chunked coding ends only when the server closes the connection. An HTTP/1.1 client then cannot reuse the connection, and a truncated transfer is indistinguishable from a complete one. Frameworks set the length or chunk the body automatically; hand-written servers and proxies sometimes do not.

4.2.5 Questions to expect

1. What does it mean that HTTP is stateless?

Each request is self-contained; the server keeps no protocol-level memory between requests. Sessions are built above HTTP with cookies or tokens that the client attaches to every request. Statelessness allows any server instance to answer any request, which is the basis of horizontal scaling.

2. What is the difference between a safe and an idempotent method?

Safe: the method does not change server state (GET, HEAD, OPTIONS). Idempotent: repeating the call has the same effect as one call (GET, PUT, DELETE, and the safe methods). POST is neither, and PATCH is not guaranteed to be idempotent. Idempotency decides whether automatic retries are permitted.

3. How does a client know where a response body ends?

By `Content-Length` (an exact byte count), by chunked transfer coding (a terminating zero-size chunk), or, for HTTP/1.0-style responses, by connection close. Bodiless responses (1xx, 204, 304, responses to HEAD) have no body regardless of headers.

4. What is head-of-line blocking in HTTP/1.1?

On one connection responses must return in the order of the requests. A slow first response delays all later ones. Browsers therefore open several parallel connections per host. HTTP/2 multiplexes streams on one connection, but TCP-level head-of-line blocking remains until HTTP/3.

5. What does the Host header do?

It carries the authority from the URL, which allows one IP address and port to host many sites; the server or proxy routes by it. It is mandatory in HTTP/1.1.

6. How do 301, 302, 307 and 308 differ?

301 and 308 are permanent, 302 and 307 temporary. 307 and 308 require the client to repeat the same method and body; for 301 and 302 clients commonly change POST to GET.

7. What are persistent connections and what is the cost of not using them?

Connections reused for several requests. Without reuse each request pays a TCP handshake (one round trip), a TLS handshake for HTTPS, and TCP slow start. Java clients must pool connections to obtain the benefit.

8. Which status code should a service return when an upstream dependency times out?

504 Gateway Timeout when acting as a gateway or proxy; 503 Service Unavailable with `Retry-After` when overloaded or in maintenance. 500 is reserved for unexpected internal errors.

INTERVIEW ANGLE

Follow-up chain: "A client reports `Connection reset by peer` on the first request after a quiet period." The server or a load balancer closed an idle persistent connection; the client sent the request on a connection that was already closed. Remedies: evict idle connections in the client pool before the server idle timeout, enable a stale-connection check, and retry once when the method is idempotent.

KEY POINTS

- An HTTP/1.1 message is a start line, header fields, an empty line and a body; the length is given by `Content-Length` or chunked coding.
- Methods carry semantics of safety and idempotency that decide cacheability and whether a retry is permitted.
- Status classes (2xx, 3xx, 4xx, 5xx) are the contract; 502, 503 and 504 identify different gateway situations.
- Persistent connections avoid repeated handshakes; pipelining suffers from head-of-line blocking and is unused in practice.
- The `Host` header enables virtual hosting; keep-alive timeouts along the chain must be ordered so that no peer reuses a connection that the other has closed.

4.3 HTTP Caching and Conditional Requests

The fastest request is the one that is never sent, and the cheapest response is one that carries no body. HTTP caching serves both goals: a stored response can be reused without any network traffic while it is fresh, and a stale response can be revalidated with a small conditional request that the server answers with `304 Not Modified` when nothing changed. Caching reduces latency, bandwidth and server load, and it is the mechanism by which content delivery networks (CDNs) absorb most of the traffic to popular sites. Incorrect caching, in contrast, produces stale pages, leaked personal data and bugs that cannot be reproduced. The rules are specified in RFC 9111.

4.3.1 Concept and mechanism

Where responses are stored

A **private cache** serves one user; the browser cache is the main example. A **shared cache** serves many users: a CDN, a reverse proxy such as nginx or Varnish, or a corporate proxy. The distinction governs which responses may be stored: a response marked `private` must not enter a shared cache, because it may be personal.

Freshness

A stored response is **fresh** while its age is less than its freshness lifetime. The lifetime is taken from `Cache-Control: max-age=N` (seconds), or, for shared caches, `s-maxage=N`, or from the older `Expires` date. When a response has none of these but has `Last-Modified`, caches may apply a heuristic, commonly 10 percent of the time since the last modification; explicit lifetimes are preferable. The `Age` header reports how long a response has already spent in shared caches.

Directive	Effect
`max-age=N`	fresh for N seconds in any cache
`s-maxage=N`	overrides `max-age` for shared caches only

Directive	Effect
`no-cache`	may be stored, but must be revalidated with the origin before each reuse (the name is misleading)
`no-store`	must not be stored at all; used for sensitive data
`private` / `public`	storable only by the browser / storable by any cache, even if the request carried credentials
`must-revalidate`	once stale, must not be served without successful revalidation (no serving stale on error)
`immutable`	the content never changes during its lifetime; do not revalidate on reload
`stale-while-revalidate=N`	a stale response may be served for N seconds while a background refresh runs

Cache-Control directives in responses.

Validation

When a response is stale, the cache does not discard it. It sends a **conditional request** that carries a validator taken from the stored response, and the origin answers either `304 Not Modified` without a body (the stored body is still valid and the cache refreshes the headers and the lifetime) or `200 OK` with a new representation. Two validators exist.

- `ETag`: an opaque identifier of a specific version of the representation, such as `"v1-a41c"`. The client returns it in `If-None-Match`. A weak ETag (`W/"..."`) states semantic equivalence rather than byte equality.
- `Last-Modified`: a timestamp with one-second resolution. The client returns it in `If-Modified-Since`. It is coarser and depends on synchronised clocks.

The same validators protect writes. `If-Match: "v1-a41c"` on a `PUT` or `DELETE` asks the server to act only if the stored version still matches, and the server answers `412 Precondition Failed` otherwise; this is optimistic concurrency control at the HTTP level and prevents lost updates.

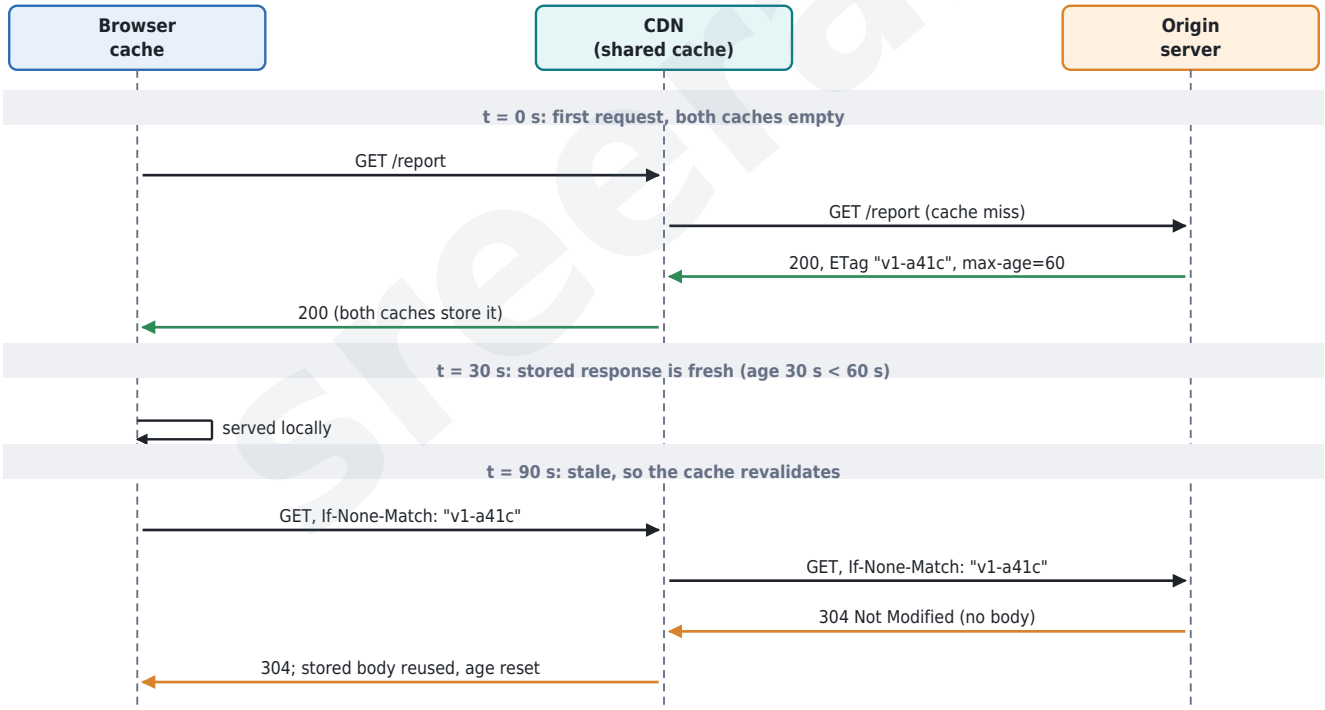


Figure 4.8. The life of one response across a browser cache, a CDN and the origin: the first request fills both caches, a request at 30 s is answered locally, and a request at 90 s revalidates with If-None-Match and receives a body-less 304.

sreram

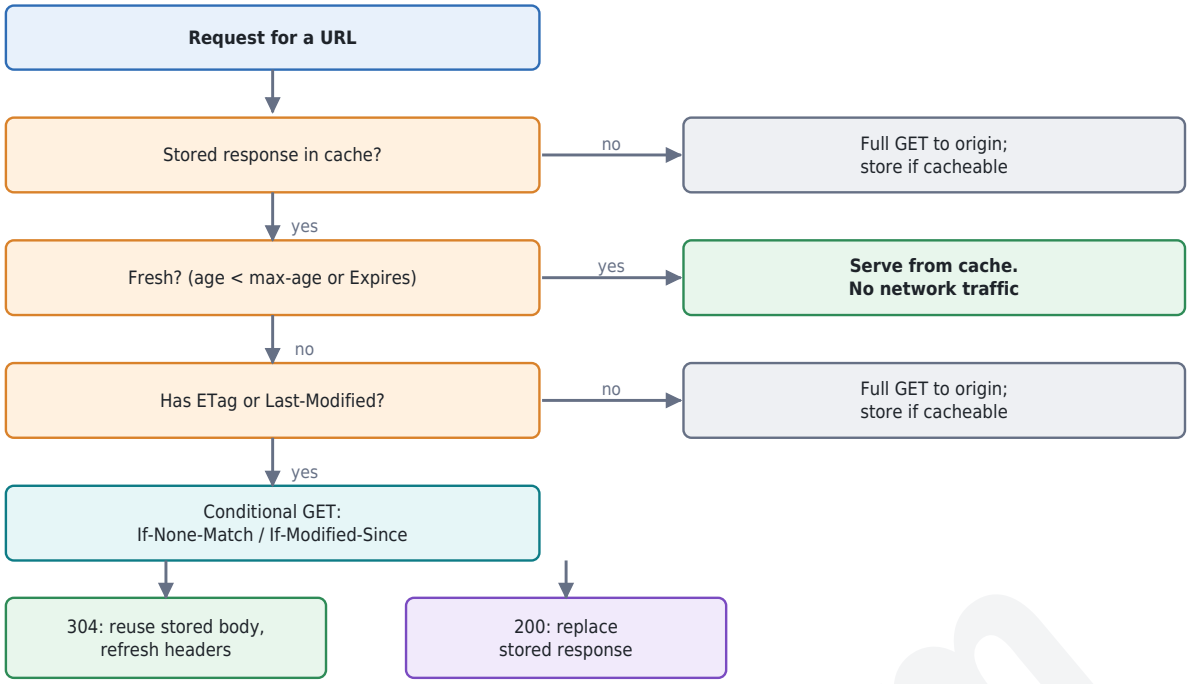


Figure 4.9. The decision a cache makes for each request: serve a fresh stored response with no network traffic, revalidate a stale response that has a validator, or fetch the full response from the origin.

The Vary header and cache keys

A cache normally keys a stored response on the request method and URL. When the representation depends on request headers (for example `Accept-Encoding`, `Accept-Language`), the server lists them in `Vary`, and the cache then stores one variant per combination of those header values. A missing `Vary: Accept-Encoding` on a compressed response can cause a cache to deliver gzip data to a client that did not ask for it. `Vary: Cookie` or `Vary: Authorization` makes shared caching nearly useless and is better replaced by `private` or `no-store`.

Effect of a CDN

A CDN is a shared cache placed close to users. Requests that hit the CDN edge never reach the origin, which reduces origin load and round-trip latency. The response lifetime at the edge is controlled by `s-maxage` (or by CDN configuration), while `max-age` controls the browser. Immutable, fingerprinted assets (`app.4f2a9c.js`) can carry `max-age=31536000, immutable`; HTML documents typically carry short lifetimes or `no-cache`, so that new releases take effect immediately by referencing new asset names.

4.3.2 Worked trace

The trace corresponds to the first diagram of this chapter and to the output of the hands-on program. The origin sends `Cache-Control: max-age=60, must-revalidate` and `ETag: "v1-a41c"`.

Time	Event	Cache behaviour
0 s	first GET /report	miss in browser and CDN; origin returns 200 with the body; both caches store it with age 0
30 s	second GET	age 30 s is less than max-age 60 s: fresh; answered from the browser cache with no packet sent
60 s	lifetime reached	the stored response is stale from now on
90 s	third GET	age 90 s exceeds 60 s: stale; browser sends `GET` with `If-None-Match: "v1-a41c"`
90 s	CDN	also stale; forwards the conditional request to the origin
90 s	origin	the ETag still matches: returns `304 Not Modified`, no body (about 150 bytes instead of the full body)
90 s	browser	updates the stored headers, sets age to 0, and serves the stored body with status 200 to the application

Timeline of a cached resource.

For a 2 MB resource the validation round trip transfers roughly a hundred bytes of headers instead of two megabytes; the latency saved is the transfer time, while the round trip itself remains. A fresh hit removes the round trip as well.

4.3.3 Hands-on

The program below uses the Java `HttpClient` against the local server of the HTTP/1.1 chapter. It reads the validator from the first response and sends it back in a conditional request. `HttpClient` itself does not store responses; the caching decision stays with the application.

ConditionalGet.java JAVA

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class ConditionalGet {
    static HttpRequest.Builder get(String url) {
        return HttpRequest.newBuilder(URI.create(url)).timeout(Duration.ofSeconds(3)).GET();
    }

    public static void main(String[] args) throws Exception {
        HttpClient http = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(2))
                .build();
        String url = "http://127.0.0.1:18080/report";

        HttpResponse<String> first = http.send(get(url).build(),
                HttpResponse.BodyHandlers.ofString());
        String etag = first.headers().firstValue("etag").orElseThrow();
        System.out.println("1st: " + first.statusCode() + " etag=" + etag
                + " cache-control=" + first.headers().firstValue("cache-control").orElse("-"));
        System.out.println("     body=" + first.body().trim());

        HttpResponse<String> second = http.send(get(url).header("If-None-Match", etag).build(),
                HttpResponse.BodyHandlers.ofString());
        System.out.println("2nd: " + second.statusCode()
                + " body-length=" + second.body().length());
    }
}
```

Output of ConditionalGet.java (server of the previous chapter running) TERMINAL

```
$ java ConditionalGet.java
1st: 200 etag="v1-a41c" cache-control=max-age=60, must-revalidate
     body={"total":42}
2nd: 304 body-length=0
```

The same exchange with curl shows the raw headers of both responses:

Validation with curl TERMINAL

```
$ curl -si http://127.0.0.1:18080/report
HTTP/1.1 200 OK
Content-type: application/json
Etag: "v1-a41c"
Content-length: 13
Cache-control: max-age=60, must-revalidate

{"total":42}
$ curl -si -H 'If-None-Match: "v1-a41c"' http://127.0.0.1:18080/report
HTTP/1.1 304 Not Modified
Etag: "v1-a41c"
Cache-control: max-age=60, must-revalidate
```

In Spring MVC the corresponding configuration is short. `ResponseEntity.ok().cacheControl(CacheControl.maxAge(Duration.ofSeconds(60)).mustRevalidate()).eTag("\"v1-a41c\"").body(report)` sets both headers, and `ShallowEtagHeaderFilter` computes an ETag from the response body of every request, which saves bandwidth but not server computation. Static resources can be given a lifetime with `spring.web.resources.cache.cachecontrol.max-age`, and

`spring.web.resources.chain.strategy.content.enabled=true` produces fingerprinted resource URLs. These are configuration facts of Spring Boot 3; the snippet is not run in this guide.

4.3.4 Failure modes

Symptom	Cause	Remedy
Users see the old version after a release	HTML or an unversioned asset cached with a long `max-age`	version asset names by content hash; give HTML `no-cache`
Personal data shown to another user	personalised response stored by a shared cache (`public`, or no `Cache-Control` with a heuristic lifetime)	`Cache-Control: private` or `no-store` on authenticated responses
Garbled content or wrong language from a CDN	`Vary` missing for a header that changes the body	list the header in `Vary`
Constant `200` instead of `304`	ETag differs between server instances (generated from file timestamps or inode)	derive the ETag from content or a version number
`no-cache` believed to disable storage	misunderstanding of the directive	use `no-store` when the response must not be stored
Cache never refreshes	clock skew between servers when `Expires` or `Last-Modified` is used	prefer `max-age` and content-based ETag

Typical caching failures.

WATCH OUT

`Cache-Control: no-cache` does not prevent storage; it forces revalidation. Responses that contain secrets, tokens or personal records require `no-store`. Authenticated API responses without any `Cache-Control` header may receive a heuristic lifetime in a shared cache, so a missing header is not a safe default.

REAL WORLD

A common production pattern combines three rules: fingerprinted static assets with `max-age=31536000, immutable`; HTML pages with `no-cache` and an ETag; and API responses for user data with `private, no-store`. A deployment then changes the HTML only, and every client fetches new assets on the next page load while unchanged assets remain cached for a year.

4.3.5 Questions to expect

1. What is the difference between `no-cache` and `no-store`?

`no-cache` allows storage but requires revalidation before every reuse. `no-store` forbids storing the response in any cache. Sensitive data uses `no-store`.

2. How does ETag validation work?

The server sends `ETag` with the response. After the response becomes stale, the client sends `If-None-Match` with that value. If the current ETag matches, the server returns `304` without a body, and the client reuses its stored body. Otherwise the server returns `200` with a new body and ETag.

3. When is ETag preferable to Last-Modified?

When changes happen within one second, when the timestamp is not meaningful (generated content), or when several servers must agree. An ETag based on content or version identifies the representation exactly.

4. What does `Vary` do?

It names the request headers that influenced the response. Caches use their values as part of the key, so that each variant is stored and served separately.

5. How should a single-page application be cached?

Fingerprinted JavaScript and CSS files with a one-year `immutable` lifetime; `index.html` with `no-cache` so the browser always checks for a new version that references new file names.

6. What does the status 304 contain?

Only headers: the validators and caching headers of the current representation. It has no body. The cache combines these headers with the stored response.

7. How does a conditional write prevent lost updates?

The client reads a resource with its ETag and sends `If-Match` with the update. If another client changed the resource in between, the ETag differs and the server returns `412 Precondition Failed`; the client re-reads and merges.

KEY POINTS

- A fresh stored response is reused with no network traffic; a stale one is revalidated with a conditional request.
- Validators are `ETag` (with `If-None-Match`) and `Last-Modified` (with `If-Modified-Since`); `304` carries no body.
- `no-cache` means revalidate, `no-store` means never store, `private` excludes shared caches, `s-maxage` targets CDNs.
- `Vary` defines cache keys; missing `Vary` or `private` causes content leaks and wrong variants.
- Version static assets by content hash and give the HTML document a short lifetime or `no-cache`.

4.4 Cookies, Sessions and Authentication over HTTP

HTTP is stateless, yet applications need to recognise a user across requests: a shopping cart, a login, a preference. Two mechanisms provide this. A **cookie** is a small piece of data that the server asks the browser to store and to return automatically with later requests to the same site. A **token** in the `Authorization` header is attached explicitly by the client code. Sessions, cookie attributes and authentication headers determine both the behaviour of a Spring Boot back end and the security of its users.

4.4.1 Concept and mechanism

The cookie mechanism

The server sends one `Set-Cookie` header per cookie in a response. The browser stores the cookie in its cookie jar, associated with the host that set it, and returns it in a `Cookie` header on every later request that matches the cookie scope. The `Cookie` header carries only names and values; the attributes are never sent back.

Attribute	Effect
`` `Max-Age=N`, `Expires=date` ``	lifetime; without either the cookie is a session cookie, discarded when the browser session ends
`` `Domain=example.com` ``	include subdomains; without it the cookie is returned only to the exact host that set it
`` `Path=/` ``	return the cookie only for URL paths below this prefix
`` `Secure` ``	send only over HTTPS
`` `HttpOnly` ``	invisible to JavaScript (`document.cookie`), which limits theft through cross-site scripting
`` `SameSite=Strict` ``	never sent on cross-site requests
`` `SameSite=Lax` ``	sent on top-level navigations with safe methods, not on cross-site subrequests or POSTs; the default in current browsers
`` `SameSite=None` ``	sent on all cross-site requests; requires `Secure`
`Name prefix `__Host-` `	browser enforces `Secure`, `Path=/` and no `Domain`, so that subdomains cannot overwrite the cookie

Set-Cookie attributes (RFC 6265 and its successors).

A request is **cross-site** when the registrable domain of the page differs from that of the requested URL. Cross-site requests with cookies are the basis of cross-site request forgery (CSRF); `SameSite=Lax` blocks the most common form, an automatic POST from another site.

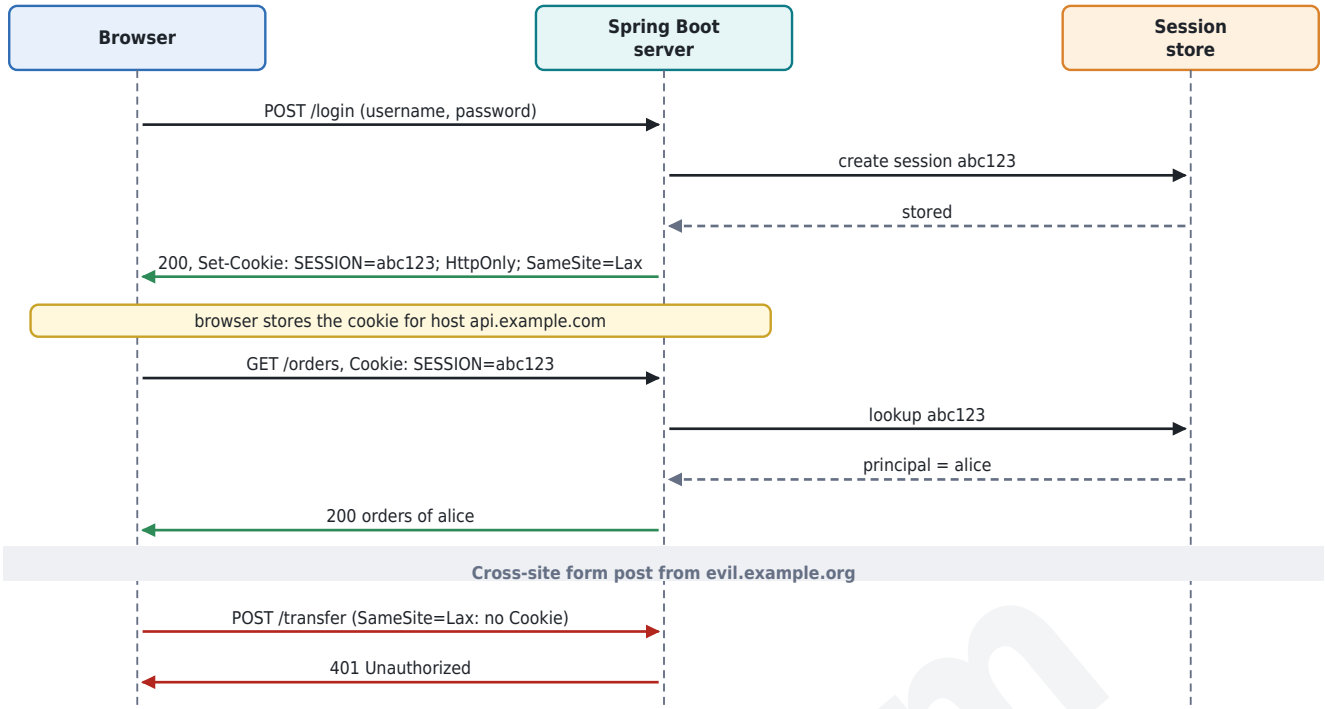


Figure 4.10. Cookie-based session: the login response creates a server-side session and sets an HttpOnly cookie, later requests return it automatically, and a cross-site form post from another origin does not carry it because of SameSite=Lax.

Session or token

With a **server-side session** the cookie holds only a random identifier (`SESSION=abc123`); the data lives on the server, in memory or in a shared store. Revocation is immediate, since the server deletes the session. Scaling requires either sticky routing (a load balancer sends the user to the same instance) or a shared store such as Redis, which Spring Session provides.

With a **token** (commonly a JSON Web Token) the client stores a signed token that contains the claims and presents it with each call. Servers validate the signature and hold no per-user state, which suits many independent services. Revocation is harder, because a token is valid until it expires; short lifetimes and refresh tokens compensate. The format and flows are the subject of the chapter on authentication in transit in Part 5.

Aspect	Session cookie	Bearer token in `Authorization`
Sent by	browser automatically	client code explicitly
CSRF exposure	yes, mitigated by `SameSite` and CSRF tokens	none, as the browser does not attach it
XSS exposure	`HttpOnly` hides the cookie from scripts	a token stored in browser storage is readable by scripts
Server state	session store required	none (stateless validation)
Revocation	immediate	expiry, or a deny-list
Typical use	server-rendered applications, same-site SPAs	APIs, mobile clients, service-to-service calls

Session cookie compared with bearer token.

The Authorization header

An unauthenticated request to a protected resource receives `401 Unauthorized` with a `WWW-Authenticate` header that names the scheme. The client repeats the request with `Authorization: <scheme> <credentials>`.

- **Basic** (RFC 7617): `Authorization: Basic` followed by the Base64 encoding of `user:password`. Base64 is an encoding, not encryption: anyone who sees the header reads the password. Basic is acceptable only over TLS.
- **Bearer** (RFC 6750): `Authorization: Bearer <token>`. Whoever holds the token can use it; it must therefore also travel over TLS only.

Status `401` means the credentials are missing or invalid (the client may retry with better ones). Status `403` means the credentials are valid but the permission is insufficient. Returning 404 instead of 403 for resources whose existence is itself confidential is a deliberate design choice.

4.4.2 Worked trace

Step	Exchange	Header values
1	`POST /login`	body with username and password (over TLS in production)
2	server creates the session in the store	identifier abc123, principal alice
3	`204 No Content`	`Set-Cookie: SESSION=abc123; Path=/; HttpOnly; SameSite=Lax; Max-Age=3600`
4	browser stores the cookie	host 127.0.0.1, path `/`, expiry now + 3600 s
5	`GET /orders`	`Cookie: SESSION=abc123`
6	server loads session abc123	principal alice; response contains alice's orders
7	form post from evil.example.org to `/transfer`	cookie omitted by the browser (cross-site, `SameSite=Lax`); server answers 401

Session life cycle as in the diagram.

4.4.3 Hands-on

The `/login` endpoint of the local server issues the cookie of step 3. The `-c` option of curl writes the cookie jar and `-b` reads it.

Cookie issued and returned (trimmed) TERMINAL

```
$ curl -sv -c jar.txt http://127.0.0.1:18080/login 2>&1 | grep -E '^<'
< HTTP/1.1 204 No Content
< Date: Sat, 17 Oct 2026 16:58:36 GMT
< Set-cookie: SESSION=abc123; Path=/; HttpOnly; SameSite=Lax; Max-Age=3600
<
$ tail -1 jar.txt
#HttpOnly_127.0.0.1     FALSE   /       FALSE   1792259916      SESSION abc123
$ curl -sv -b jar.txt http://127.0.0.1:18080/hello 2>&1 | grep -E '^> (GET|Cookie)'
> GET /hello HTTP/1.1
> Cookie: SESSION=abc123
```

The jar file stores the expiry as a Unix timestamp (login time plus 3600 s). The second request shows that only name and value are returned.

Basic authentication header TERMINAL

```
$ curl -sv -u alice:s3cret http://127.0.0.1:18080/hello 2>&1 | grep -E '^> (GET|Auth)'
> GET /hello HTTP/1.1
> Authorization: Basic YWxpY2U6czNjcmV0
$ echo -n 'YWxpY2U6czNjcmV0' | base64 -d
alice:s3cret
```

The decoding on the second command line demonstrates the point: the credentials are recoverable by any observer of the header.

In Spring Security, the session cookie attributes are set in configuration, for example `server.servlet.session.cookie.http-only=true`, `server.servlet.session.cookie.secure=true` and `server.servlet.session.cookie.same-site=lax`. The cookie name defaults to `JSESSIONID` in servlet containers.

4.4.4 Failure modes

Symptom	Cause	Remedy
Cookie not stored by the browser	`Secure` cookie set over plain HTTP; `SameSite=None` without `Secure`; cookie larger than about 4 KB	use HTTPS; correct attributes; keep cookies small
Login works, next request is anonymous	cookie `Domain` or `Path` does not match; or the API is on another site and `SameSite=Lax` blocks it	align the sites, or use `SameSite=None; Secure` with CORS credentials
Users logged out at random behind a load balancer	in-memory sessions on several instances without sticky routing	shared session store or stateless tokens
Session fixation	identifier not renewed at login	issue a new session identifier after authentication
Password visible in logs	Basic credentials or URLs with credentials written by a proxy or access log	mask the header; never put credentials in URLs

Typical cookie and authentication failures.

WATCH OUT

An identifier in a cookie is a bearer credential. Set `HttpOnly` and `Secure` on every session cookie, regenerate the identifier at login, and treat the `Authorization` header as sensitive in logs and traces.

4.4.5 Questions to expect

1. How does a server recognise a returning user over a stateless protocol?

It sends `Set-Cookie` with a session identifier; the browser returns it in `Cookie` on every request to that site; the server maps the identifier to stored state. Alternatively the client sends a token in `Authorization`.

2. What do `HttpOnly`, `Secure` and `SameSite` protect against?

`HttpOnly` prevents scripts from reading the cookie, limiting damage from XSS. `Secure` prevents transmission over plain HTTP. `SameSite` restricts cross-site sending and mitigates CSRF.

3. What is the difference between 401 and 403?

401: authentication missing or invalid, accompanied by `WWW-Authenticate`. 403: the identity is known but lacks permission.

4. Why is Basic authentication considered weak?

The credentials are only Base64-encoded and are sent with every request. Without TLS they are exposed; with TLS they are acceptable for simple internal use, but the password is still transmitted repeatedly.

5. How do sessions scale across several servers?

Sticky sessions at the load balancer, replication of session data, or an external store such as Redis (Spring Session). A stateless token avoids the need for shared state.

6. Why does a bearer token in local storage differ in risk from a cookie?

Scripts can read local storage, so an XSS flaw leaks the token; an `HttpOnly` cookie is unreadable by scripts but is attached automatically, so CSRF must be countered with `SameSite` and tokens.

KEY POINTS

- Cookies are set by `Set-Cookie`, returned by `Cookie`, and scoped by host, `Domain`, `Path`, `Secure`, `HttpOnly` and `SameSite`.
- Server-side sessions keep state in a store and are revocable; tokens keep claims in the credential and are stateless but harder to revoke.
- `Authorization: Basic` is Base64, not encryption; Basic and Bearer both require TLS.
- 401 means unauthenticated, 403 means unauthorised.
- `SameSite=Lax` (the browser default) blocks most CSRF; `SameSite=None` requires `Secure`.

4.5 HTTP/2 and HTTP/3

HTTP/1.1 has three structural weaknesses. A connection carries one response at a time, so parallelism requires several connections, each with its own handshake and congestion window. Header fields are textual and repeated in full on every request, although cookies and user-agent strings change rarely. And a slow response blocks the ones behind it on the same connection. HTTP/2 (RFC 9113, 2015 as RFC 7540) and HTTP/3 (RFC 9114) address these weaknesses without changing HTTP semantics: the methods, status codes, header fields and URIs of the previous chapters are unchanged, and application code written against a servlet or `HttpClient` API does not change. Only the encoding on the wire and the transport differ.

4.5.1 Concept and mechanism

HTTP/2: binary framing, streams and multiplexing

HTTP/2 replaces the text format by a **binary framing layer** on one TCP connection (normally with TLS). Every message is divided into **frames**, each of which carries the identifier of the **stream** to which it belongs. A stream is one request and its response; stream identifiers are odd for client-initiated streams (1, 3, 5, ...) and even for server-initiated ones. Frames of different streams may be interleaved on the connection in any order, and the receiver reassembles each stream by its identifier. This is **multiplexing**: many concurrent exchanges share one connection, and no response waits for another at the HTTP layer.

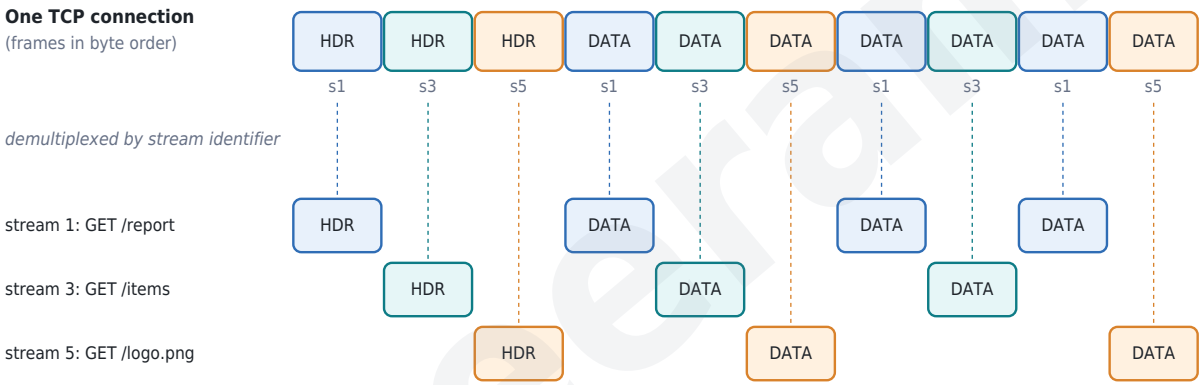


Figure 4.11. Three requests share one TCP connection: header and data frames of streams 1, 3 and 5 are interleaved in byte order and reassembled into three independent streams by the stream identifier in each frame header.

Every frame starts with a fixed 9-byte header: a 24-bit payload length, an 8-bit type, 8 bits of flags, one reserved bit and a 31-bit stream identifier. The default maximum frame payload is 16,384 bytes.

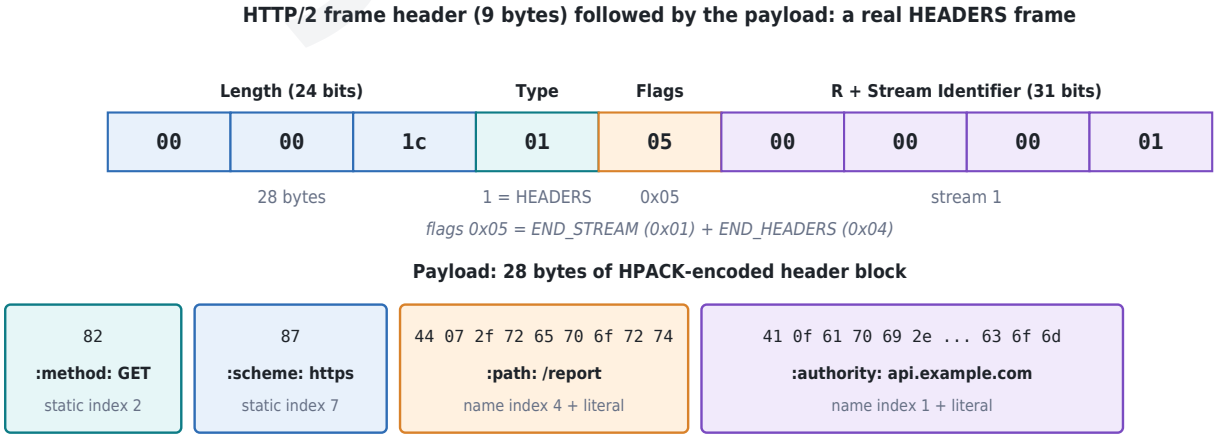


Figure 4.12. A real HEADERS frame for GET /report: the nine header bytes (length 28, type 1, flags END_STREAM and END_HEADERS, stream 1) and the 28-byte HPACK block that encodes the four request pseudo-header fields.

Code	Type	Purpose
0	DATA	body bytes of a stream
1	HEADERS	header block that opens a stream (request or response) or carries trailers
3	RST_STREAM	abnormal termination of one stream; the connection continues
4	SETTINGS	connection parameters (for example SETTINGS_MAX_CONCURRENT_STREAMS), acknowledged by the peer
5	PUSH_PROMISE	server push announcement
6	PING	liveness and round-trip measurement
7	GOAWAY	graceful connection shutdown; names the last processed stream
8	WINDOW_UPDATE	flow-control credit for a stream or for the whole connection

HTTP/2 frame types.

Requests are expressed through **pseudo-header fields** that begin with a colon: `:method`, `:scheme`, `:authority` (the replacement for `Host`), `:path` and, in responses, `:status`. A client opens an HTTP/2 connection with the 24-byte preface `PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n` followed by a SETTINGS frame. Over TLS the protocol is selected during the handshake by ALPN (Application-Layer Protocol Negotiation) with the identifier `h2`; over plain TCP (`h2c`) it is selected by prior knowledge or an HTTP/1.1 upgrade.

Header compression (HPACK)

HPACK (RFC 7541) compresses header fields with three devices: a **static table** of 61 common entries (index 2 is `:method: GET`, index 7 is `:scheme: https`), a **dynamic table** of entries that the two peers add as the connection proceeds, and Huffman coding of literal strings. A field present in a table is sent as a single byte index. A header that was sent before, such as a long cookie or authorisation header, costs one or two bytes on every later request. This is where much of the saving of HTTP/2 on small API calls arises.

Flow control, priorities and push

Each stream and the connection as a whole have a **flow-control window**, initially 65,535 bytes, that limits how many DATA bytes the sender may transmit before it receives `WINDOW_UPDATE` credit. It prevents a fast stream from monopolising the receiver buffers; it is independent of TCP flow control. The original priority tree of RFC 7540 was deprecated by RFC 9113; the replacement is the simple Extensible Priorities scheme of RFC 9218 (an urgency level and an incremental flag). **Server push**, in which the server sends responses that the client has not yet requested, was rarely beneficial, was difficult to tune against the client cache, and has been removed from major browsers; it should not be used in new designs.

Remaining weakness: head-of-line blocking in TCP

HTTP/2 removes head-of-line blocking at the HTTP layer, but TCP delivers one ordered byte stream. When one TCP segment is lost, the kernel withholds all later bytes of the connection from the application until the retransmission arrives, and every multiplexed stream stalls, including streams whose data was not lost. On lossy networks, a single HTTP/2 connection can perform worse than several HTTP/1.1 connections.

HTTP/3 and QUIC

HTTP/3 maps HTTP semantics onto **QUIC** (RFC 9000), a transport that runs over UDP and integrates TLS 1.3 (RFC 9001). QUIC provides streams itself: loss on one stream delays only that stream, because ordering is enforced per stream, not per connection. Further properties follow from the design: connection establishment combines the transport and cryptographic handshakes into one round trip (zero round trips with 0-RTT resumption, subject to replay restrictions); a connection is identified by a connection identifier instead of the address and port tuple, so it survives a change of network (migration from Wi-Fi to mobile); and the transport is implemented in user space, so it evolves faster than operating-system TCP stacks. HTTP/3 uses QPACK (RFC 9204) for header compression, a variant of HPACK that tolerates out-of-order delivery.

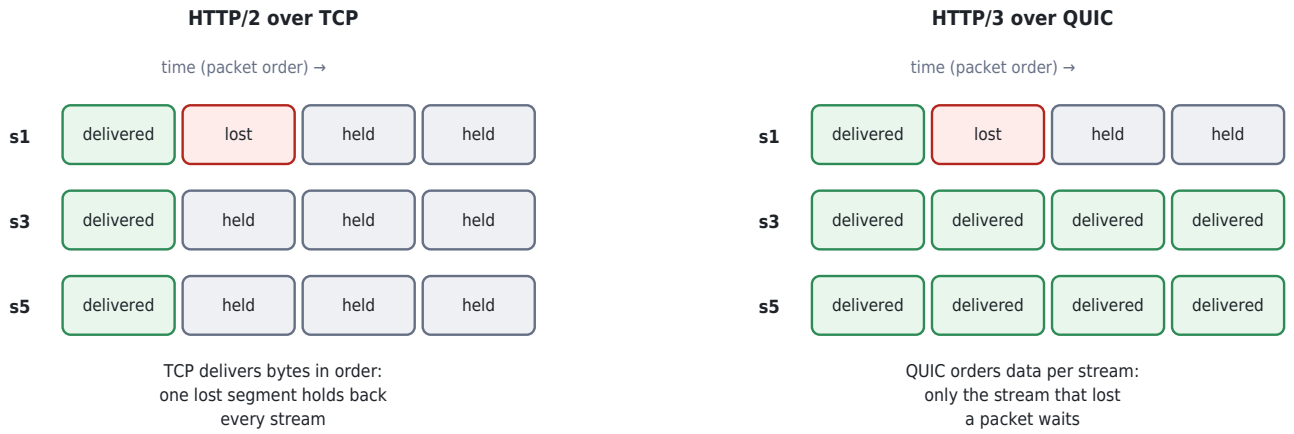


Figure 4.13. One lost packet on stream 1: over TCP every stream behind it waits for the retransmission, whereas over QUIC only stream 1 waits and streams 3 and 5 are delivered at once.

A client learns that a server offers HTTP/3 from the `Alt-Svc` response header (for example `Alt-Svc: h3=":443"; ma=86400`) or from an HTTPS DNS record, and it retries over TCP when UDP is blocked. The transport design is treated in detail in Part 3.

Property	HTTP/1.1	HTTP/2	HTTP/3
Encoding	text	binary frames	binary frames
Transport	TCP	TCP (usually with TLS)	QUIC over UDP (TLS 1.3 built in)
Concurrency per connection	one request at a time	multiplexed streams	multiplexed streams
Header compression	none	HPACK	QPACK
HOL blocking	HTTP layer and TCP	TCP layer only	per stream only
Negotiation	none	ALPN `h2`	`Alt-Svc` or HTTPS record, ALPN `h3`

Comparison of HTTP versions.

4.5.2 Worked trace

The size effect of HPACK is easily computed for the request of the diagram. The program of the hands-on section encodes the request, decodes it with a minimal decoder, and repeats the request with the dynamic table in place.

Bytes	Field	Rule applied
`82`	`:method: GET`	indexed field: high bit set, static index 2
`87`	`:scheme: https`	indexed field, static index 7
`44 07 2f 72 65 70 6f 72 74`	`:path: /report`	literal with incremental indexing (`01`), name index 4, length 7, raw string; added to the dynamic table
`41 0f 61 70 69 ...`	`:authority: api.example.com`	literal with incremental indexing, name index 1, length 15; added to the dynamic table
`82 87 bf be`	the same four fields, second request	`bf` = indexed 63 (`:path: /report`), `be` = indexed 62 (`:authority: ...`); 4 bytes in total

HPACK encoding of GET /report to api.example.com over HTTPS.

The first block is 28 bytes, against 47 bytes for the minimal HTTP/1.1 text of the same request (`GET /report HTTP/1.1`, `Host: api.example.com` and the empty line, without `User-Agent` or `Accept`). The repeated block is 4 bytes. Add the 9-byte frame header and the first HEADERS frame is 37 bytes on the wire; each later request of this form is 13 bytes.

4.5.3 Hands-on

Building HTTP/2 frames and HPACK by hand

h2_frames.py PYTHON

```python
import struct

STATIC = {1: (":authority", ""), 2: (":method", "GET"), 4: (":path", "/"), 7: (":scheme", "https")}


def frame(ftype, flags, stream, payload):
    length = struct.pack("!I", len(payload))[1:]              # 24-bit length
    return length + bytes([ftype, flags]) + struct.pack("!I", stream) + payload


def literal(name_index, value):
    # 01xxxxxx: literal header field with incremental indexing, name taken from the table
    return bytes([0x40 | name_index, len(value)]) + value.encode()


def encode_first(path, authority):
    return bytes([0x82, 0x87]) + literal(4, path) + literal(1, authority)


def show(fields):
    return ", ".join("%s=%s" % kv for kv in fields)


def decode(block, dynamic):
    """Minimal HPACK decoder: indexed fields and literals with incremental indexing, no Huffman."""
    out, i = [], 0
    while i < len(block):
        b = block[i]
        if b & 0x80:                                  # indexed header field
            idx = b & 0x7F
            table = STATIC if idx < 62 else {idx: dynamic[idx - 62]}
            out.append(table[idx])
            i += 1
        else:                                         # literal with incremental indexing
            name = STATIC[b & 0x3F][0]
            n = block[i + 1]
            value = block[i + 2:i + 2 + n].decode()
            dynamic.insert(0, (name, value))          # newest entry gets index 62
            out.append((name, value))
            i += 2 + n
    return out


block1 = encode_first("/report", "api.example.com")
hdr = frame(1, 0x05, 1, block1)                       # HEADERS, END_STREAM | END_HEADERS, stream 1
print("HEADERS frame, %d bytes:" % len(hdr))
print("frame header :", " ".join("%02x" % b for b in hdr[:9]))
print("payload      :", " ".join("%02x" % b for b in hdr[9:]))
dyn = []
print("decoded      :", show(decode(block1, dyn)))
print("dynamic table:", show(dyn))

block2 = bytes([0x82, 0x87, 0xBF, 0xBE])              # same request again: indexed 63 and 62
print("second request block:", " ".join("%02x" % b for b in block2), "=", len(block2), "bytes")
print("decoded      :", show(decode(block2, dyn)))

text = "GET /report HTTP/1.1\r\nHost: api.example.com\r\n\r\n"
print("HTTP/1.1 text request: %d bytes; HPACK block: %d bytes first, %d bytes repeated"
      % (len(text), len(block1), len(block2)))
print("connection preface   :", b"PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n".hex(" "))
settings = frame(4, 0, 0, struct.pack("!HI", 3, 100))   # SETTINGS_MAX_CONCURRENT_STREAMS = 100
print("SETTINGS frame       :", settings.hex(" "))
```

Output of h2_frames.py — TERMINAL

```
$ python3 h2_frames.py
HEADERS frame, 37 bytes:
frame header : 00 00 1c 01 05 00 00 00 01
payload      : 82 87 44 07 2f 72 65 70 6f 72 74 41 0f 61 70 69 2e 65 78 61 6d 70 6c 65 2e 63 6f 6d
decoded      : :method=GET, :scheme=https, :path=/report, :authority=api.example.com
dynamic table: :authority=api.example.com, :path=/report
second request block: 82 87 bf be = 4 bytes
decoded      : :method=GET, :scheme=https, :path=/report, :authority=api.example.com
HTTP/1.1 text request: 47 bytes; HPACK block: 28 bytes first, 4 bytes repeated
connection preface   : 50 52 49 20 2a 20 48 54 54 50 2f 32 2e 30 0d 0a 0d 0a 53 4d 0d 0a 0d 0a
SETTINGS frame       : 00 00 06 04 00 00 00 00 00 00 03 00 00 00 64
```

The `SETTINGS` frame in the last line has length 6, type 4, flags 0, stream 0 (connection-level) and one setting: identifier 3 (`SETTINGS_MAX_CONCURRENT_STREAMS`) with value 100.

Protocol negotiation in the JDK HttpClient

`java.net.http.HttpClient` prefers HTTP/2 by default. For an `https` URL it offers `h2` through ALPN. For a cleartext `http` URL it sends an HTTP/1.1 request with an upgrade offer, and continues with HTTP/1.1 when the server ignores it. The experiment below uses the HTTP/1.1 server of the previous chapter, so it falls back. The offer itself was captured by a listening socket and is shown first.

The cleartext upgrade offer sent by HttpClient (captured on a listening socket) — TERMINAL

```
$ python3 listen.py
GET /hello HTTP/1.1
Connection: Upgrade, HTTP2-Settings
Host: 127.0.0.1:19091
HTTP2-Settings: AAEAAEAAAAIAAAAAAAMAAAAAAAQBAAAAAAUAAEAAAAYABgAA
Upgrade: h2c
User-Agent: Java-http-client/21.0.12.1
```

H2Probe.java — JAVA

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class H2Probe {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();          // default preference: HTTP/2
        System.out.println("client preference: " + client.version());
        HttpResponse<String> r = client.send(
                HttpRequest.newBuilder(URI.create("http://127.0.0.1:18080/hello")).build(),
                HttpResponse.BodyHandlers.ofString());
        System.out.println("negotiated: " + r.version() + ", status " + r.statusCode());
    }
}
```

Output of H2Probe.java against the HTTP/1.1 server — TERMINAL

```
$ java H2Probe.java
client preference: HTTP_2
negotiated: HTTP_1_1, status 200
```

`response.version()` reports the version actually used, which is the reliable way to verify in a test or a log line that HTTP/2 was negotiated. In Spring Boot the server side is enabled with `server.http2.enabled=true`; with embedded Tomcat HTTP/2 over TLS needs `server.ssl.*` configured (ALPN is available in current JDKs), and clients such as `curl --http2 -v https://...` display `ALPN: server accepted h2`. HTTP/3 support in `HttpClient` arrived in JDK 26 (JEP 517, opt-in per client); it was not run here because the build machine has JDK 21. In practice HTTP/3 is terminated by a CDN, a load balancer or a reverse proxy in front of Java services.

4.5.4 Failure modes

Symptom	Cause	Diagnosis and remedy
`PROTOCOL_ERROR` or `GOAWAY` from a proxy	a downstream peer does not speak HTTP/2 or an intermediary mishandles pseudo-headers	check ALPN result with `curl -v --http2`; force HTTP/1.1 on that hop
Slow transfers on a lossy mobile network although HTTP/2 is enabled	TCP head-of-line blocking; one connection with a shrunken congestion window	enable HTTP/3 at the edge
`431 Request Header Fields Too Large` or reset streams	very large cookies exceed `SETTINGS_MAX_HEADER_LIST_SIZE`	reduce cookie size; raise the server limit
Stalled large upload	stream or connection flow-control window exhausted; the receiver does not consume data	raise `initialWindowSize` or consume the body promptly
HTTP/3 never used	UDP 443 blocked by a firewall; `Alt-Svc` not sent	client silently falls back to TCP; open UDP/443
Too many concurrent streams error	client exceeds `SETTINGS_MAX_CONCURRENT_STREAMS` of the server	queue requests on the client; the server limit is typically 100 or more

Typical HTTP/2 and HTTP/3 failures.

GOTCHA

HTTP/2 does not make a service faster when the bottleneck is on the server. Multiplexing removes connection overhead and header cost; it does not reduce database latency. Moreover, one connection per host concentrates load balancing: a layer-4 balancer distributes connections, not requests, so a few long-lived HTTP/2 connections can leave instances unevenly loaded. Layer-7 balancing or periodic connection recycling (`GOAWAY`) corrects this.

4.5.5 Questions to expect

1. How does HTTP/2 improve on HTTP/1.1?

Binary framing, multiplexed streams on one connection, HPACK header compression and per-stream flow control. A single connection replaces the six parallel ones, and requests no longer queue behind each other at the HTTP layer.

2. What is multiplexing and what identifies the pieces?

Interleaving of frames from many streams on one connection. Each frame header contains the 31-bit stream identifier, and the receiver reassembles per stream.

3. If HTTP/2 removes head-of-line blocking, why was HTTP/3 needed?

HTTP/2 removed it at the HTTP layer only. All streams share one TCP byte stream, so one lost segment blocks every stream. QUIC orders data per stream, so only the affected stream waits.

4. What is HPACK?

A header compression scheme with a static table, a per-connection dynamic table and Huffman coding. Repeated headers cost a table index of one or two bytes. QPACK is its counterpart for HTTP/3.

5. Why was server push removed?

Servers could not know what the client cache already held, so pushes often wasted bandwidth, and the benefit over `Link: rel=preload` was small. Major browsers removed support.

6. How is HTTP/2 negotiated?

Over TLS by ALPN with the identifier `h2` during the handshake; over cleartext by prior knowledge or an `Upgrade: h2c` request. HTTP/3 is discovered through `Alt-Svc` or an HTTPS DNS record.

7. Does an application need code changes to use HTTP/2?

Generally no. Servlet and `HttpClient` code is unchanged; the server configuration, the TLS setup and intermediaries must support the protocol. Practices such as domain sharding and asset concatenation, which compensated for HTTP/1.1 limits, become unnecessary or counterproductive.

KEY POINTS

- HTTP/2 keeps HTTP semantics and changes the encoding: binary frames with a 9-byte header, streams, multiplexing, HPACK and per-stream flow control.
- It removes head-of-line blocking at the HTTP layer; TCP head-of-line blocking remains, and is solved by QUIC streams in HTTP/3.
- HTTP/2 uses ALPN `h2` over TLS; HTTP/3 runs over QUIC on UDP and is advertised by `Alt-Svc`.
- Server push is obsolete; the original priority tree is replaced by Extensible Priorities (RFC 9218).
- In Java, `HttpResponse.version()` shows what was negotiated; `server.http2.enabled=true` enables HTTP/2 in Spring Boot.

4.6 REST API Design from the Network Perspective

A REST call from one Java service to another is a distributed operation with partial failure: the request may be lost before it arrives, executed and then lost on the way back, delayed beyond the caller's patience, or rejected by an intermediary that the caller never sees. API design choices that look cosmetic in a controller (status code, method, idempotency, pagination) decide how the caller behaves in each of these cases and how much load a failure creates. This chapter reads the REST conventions of Spring Boot services through the network behaviour studied so far. It does not repeat the semantics of methods and status codes from the HTTP/1.1 chapter; it applies them.

4.6.1 Concept and mechanism

Resources and methods

A resource is addressed by a URI, a noun (`/orders/1001`); the method states the action. Collections accept `POST` to create and `GET` to list; items accept `GET`, `PUT` (replace), `PATCH` (partial update) and `DELETE`. Choosing the method fixes the retry policy: a client library may repeat `GET`, `PUT` and `DELETE` automatically after a failure, because repetition is harmless by definition, but it must not repeat `POST` unless the API provides a means of deduplication.

Unknown outcome and idempotency keys

When a request times out, the caller has no information on whether the server executed it. For `POST /orders` this is the dangerous case: a retry may create a second order. An **idempotency key** solves it. The client generates a unique value (a UUID) for each logical operation and sends it in an `Idempotency-Key` header, and repeats the same value on each retry. The server records the key together with the stored response on first execution; a repeated request with the same key returns the stored response without executing again. The header is a widely used convention rather than a finished standard (an IETF draft exists). The server must store the key and the result atomically with the effect, for example in the same database transaction with a unique constraint on the key, otherwise two concurrent retries can both pass the check.

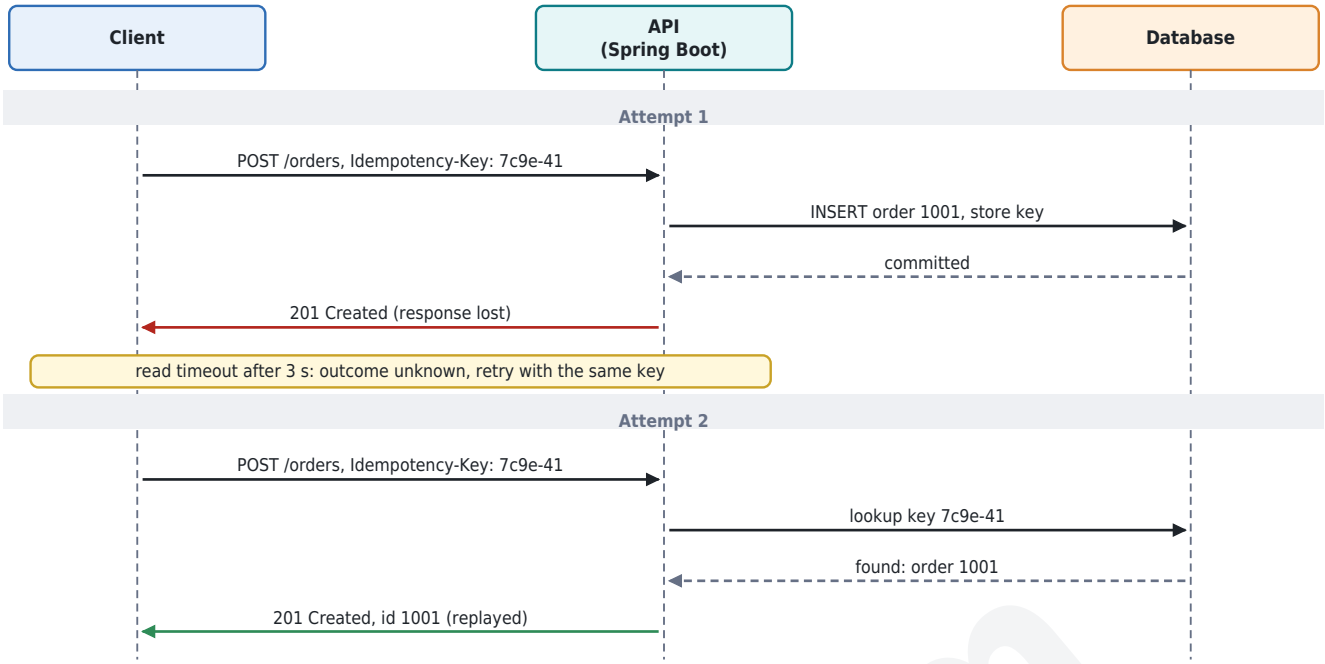


Figure 4.14. A lost response forces a retry: the second POST carries the same Idempotency-Key, the server finds the stored result and returns order 1001 again instead of creating a second order.

Status code choices

Situation	Status	Notes
resource created	201	with `Location` header of the new resource
update succeeded, body returned or not	200 or 204	204 when there is nothing to return
work accepted for asynchronous processing	202	`Location` of a status resource the caller can poll
malformed syntax or invalid parameter	400	422 is used for well-formed but semantically invalid payloads
no or invalid credentials / not permitted	401 / 403	401 includes `WWW-Authenticate`
state conflict, duplicate unique value, stale `If-Match`	409 / 412	the caller must re-read and decide
too many requests	429	with `Retry-After`; the client must back off
dependency failed or overloaded	502, 503, 504	retryable for idempotent calls; 503 with `Retry-After`

Status codes for common API outcomes.

Errors should have one machine-readable shape. RFC 9457 defines `application/problem+json` with the members `type`, `title`, `status`, `detail` and `instance`; Spring Framework 6 supports it through `ProblemDetail`. A uniform shape lets clients distinguish retryable from permanent failures without parsing text.

Pagination

Returning an unbounded collection loads the server, the network and the client. **Offset pagination** (`?page=3&size=20`, translated to `LIMIT 20 OFFSET 40`) is simple, but the cost of deep pages grows because the database still scans the skipped rows, and a row inserted during traversal shifts the following pages so that items are skipped or repeated. **Cursor pagination** (`?limit=20&after=1042`) continues from the last seen key; the cost is constant per page and the traversal is stable, at the price of no random access to page N. The location of the next page may be given in a `Link` header (RFC 8288), for example `Link: <https://api.example.com/orders?limit=20&after=1062>; rel="next"`, or in the body.

Timeouts, retries and backoff

A call has several independent time limits, and each corresponds to a different layer.

Limit	What it bounds	Where it is set
Connect timeout	TCP handshake (and TLS in many clients)	`HttpClient.Builder.connectTimeout`; `RestClient`/`WebClient` through the underlying client configuration
Response (request) timeout	wait for the response to a request	`HttpRequest.Builder.timeout`; read timeout on `HttpURLConnection` (`setReadTimeout`)
Pool acquire timeout	wait for a free pooled connection	connection pool configuration (Apache HttpClient, Reactor Netty)
Overall deadline	total time including retries	application logic; propagate a deadline downstream

Time limits of an HTTP call in Java.

A call without a response timeout can wait indefinitely on a connection that the peer has silently dropped, and the threads of a blocking server then accumulate. Retries must be limited and spaced. **Exponential backoff** doubles the delay cap after each failure; **jitter** randomises the delay (for example, uniform between 0 and the cap, known as full jitter) so that many clients that failed together do not retry together. Without jitter, a short outage ends in synchronised retry waves, which can overload a recovering service (the retry storm problem, treated in Part 5). A client must honour `Retry-After` when present, retry only idempotent calls or calls with an idempotency key, and retry only on failures that can be transient: connection errors, timeouts, 502, 503, 504 and 429, never on 400 or 404.

Content negotiation, versioning and compression

The client states acceptable representations in `Accept: application/json` and `Accept-Encoding: gzip`; the server answers `406 Not Acceptable` when it cannot comply, or `415 Unsupported Media Type` for a request body it cannot read. Compression trades CPU for bytes and pays off for text payloads above about one kilobyte; small responses and already compressed content gain nothing. In Spring Boot it is enabled with `server.compression.enabled=true`. Version identifiers appear in the path (`/v2/orders`), in a header or in a media type; the path form is the easiest to route, cache and observe.

4.6.2 Worked trace

The retry budget of a client that sends an idempotent `GET` with a 3-second response timeout and at most three attempts is computed as follows.

Step	Event	Time consumed
1	attempt 1 fails by timeout	3.0 s
2	backoff before attempt 2: uniform in [0, 200 ms)	at most 0.2 s
3	attempt 2 fails by timeout	3.0 s
4	backoff before attempt 3: uniform in [0, 400 ms)	at most 0.4 s
5	attempt 3 fails by timeout	3.0 s
6	total worst case	3 x 3.0 + 0.2 + 0.4 = 9.6 s

Worst-case duration of three attempts with exponential backoff and full jitter.

A caller that itself has a 5-second deadline for its own client must therefore not allow this policy: the retries would outlive the caller. The per-attempt timeout and the number of attempts are chosen from the end-to-end deadline, not independently. When each of ten services in a chain retries three times, a single failure at the bottom can generate 3 to the power 10 requests; this multiplication is the reason retries are normally confined to one layer.

4.6.3 Hands-on

The program below starts a small server on 127.0.0.1:18081 that implements the idempotency-key behaviour of the diagram, and then calls it with a repeated key, a new key and no key. The lookup and the store are separate steps, so the listing is a demonstration of the contract only and not safe for concurrent retries; a production implementation would use a unique constraint in the database.

IdempotencyDemo.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

public class IdempotencyDemo {
    static final Map<String, String> stored = new ConcurrentHashMap<>();   // key -> response body
    static final AtomicInteger nextId = new AtomicInteger(1001);

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 18081), 0);
        server.createContext("/orders", ex -> {
            String key = ex.getRequestHeaders().getFirst("Idempotency-Key");
            boolean replay = key != null && stored.containsKey(key);
            String body = replay ? stored.get(key) : "{\"id\":" + nextId.getAndIncrement() + "}";
            if (key != null && !replay) {
                stored.put(key, body);
            }
            ex.getResponseHeaders().add("Idempotent-Replayed", String.valueOf(replay));
            byte[] b = body.getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(201, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        server.start();

        HttpClient http = HttpClient.newHttpClient();
        String[] keys = {"7c9e-41", "7c9e-41", "a0b1-77", null};
        for (String key : keys) {
            HttpRequest.Builder rb = HttpRequest
                    .newBuilder(URI.create("http://127.0.0.1:18081/orders"))
                    .timeout(Duration.ofSeconds(3))
                    .POST(HttpRequest.BodyPublishers.ofString("{\"item\":\"book\"}"));
            if (key != null) {
                rb.header("Idempotency-Key", key);
            }
            HttpResponse<String> r = http.send(rb.build(), HttpResponse.BodyHandlers.ofString());
            System.out.printf("key=%-8s -> %d %s replayed=%s%n", key, r.statusCode(), r.body(),
                    r.headers().firstValue("idempotent-replayed").orElse("?"));
        }
        server.stop(0);
    }
}
```

Output of IdempotencyDemo.java TERMINAL

```
$ java IdempotencyDemo.java
key=7c9e-41  -> 201 {"id":1001} replayed=false
key=7c9e-41  -> 201 {"id":1001} replayed=true
key=a0b1-77  -> 201 {"id":1002} replayed=false
key=null     -> 201 {"id":1003} replayed=false
```

The second call returns the stored result with the same order identifier and the header `Idempotent-Replayed: true`; no second order exists. A request without a key creates a new order on every call, which is why unprotected `POST` calls must not be retried.

The second program implements the retry policy: a server that fails twice with 503 and a client that retries a `GET` with exponential backoff and full jitter. The random generator has a fixed seed only to make the output reproducible; real clients use an unseeded generator.

RetryDemo.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.Random;
import java.util.concurrent.atomic.AtomicInteger;

public class RetryDemo {
    public static void main(String[] args) throws Exception {
        AtomicInteger calls = new AtomicInteger();
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 18082), 0);
        server.createContext("/flaky", ex -> {                 // fails twice, then succeeds
            boolean fail = calls.incrementAndGet() <= 2;
            if (fail) {
                ex.getResponseHeaders().add("Retry-After", "0");
            }
            byte[] body = (fail ? "busy" : "ok").getBytes();
            ex.sendResponseHeaders(fail ? 503 : 200, body.length);
            ex.getResponseBody().write(body);
            ex.close();
        });
        server.start();

        HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(1)).build();
        HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:18082/flaky"))
                .timeout(Duration.ofSeconds(3)).GET().build();   // GET is idempotent: retry allowed
        Random random = new Random(42);
        long baseMs = 200;
        for (int attempt = 1; attempt <= 4; attempt++) {
            HttpResponse<String> r = http.send(req, HttpResponse.BodyHandlers.ofString());
            System.out.println("attempt " + attempt + ": status " + r.statusCode()
                    + " body=" + r.body());
            if (r.statusCode() < 500) {
                break;
            }
            long cap = baseMs << (attempt - 1);                  // 200, 400, 800 ...
            long sleep = (long) (random.nextDouble() * cap);     // full jitter: uniform in [0, cap)
            System.out.println("  retryable; backoff cap " + cap + " ms, sleeping "
                    + sleep + " ms");
            Thread.sleep(sleep);
        }
        server.stop(0);
    }
}
```

Output of RetryDemo.java (the sleep values depend on the seed) TERMINAL

```
$ java RetryDemo.java
attempt 1: status 503 body=busy
  retryable; backoff cap 200 ms, sleeping 145 ms
attempt 2: status 503 body=busy
  retryable; backoff cap 400 ms, sleeping 273 ms
attempt 3: status 200 body=ok
```

The conventional libraries provide the same logic: Spring Retry (`@Retryable` with `@Backoff(delay = 200, multiplier = 2, random = true)`) and Resilience4j (`Retry` with an exponential randomised backoff). They are referenced here and not run.

4.6.4 Failure modes

Symptom	Cause	Remedy
Duplicate orders or payments	retry of a non-idempotent POST after a timeout	idempotency key with atomic storage
Thread pool exhaustion in the caller	no response timeout; slow dependency holds threads	per-call timeouts, bulkheads, circuit breaker
Synchronised retry waves	exponential backoff without jitter; many clients	add jitter, honour `Retry-After`
Slow list endpoints that degrade with the page number	offset pagination over a large table	cursor pagination on an indexed key
Clients cannot tell permanent from transient errors	all errors returned as 500	correct 4xx/5xx classes and a problem-details body
Intermittent 413 or 414	very large payload or query string rejected by a proxy	limit sizes in the API; move large data to the body or to object storage

Typical API-level failures.

WATCH OUT

Never retry a non-idempotent request on a timeout without a deduplication mechanism. A timeout means unknown outcome, not failure. The same rule applies to HTTP client libraries that retry automatically on connection reset: confirm the retry policy of the library for POST requests.

INTERVIEW ANGLE

Follow-up chain: "A payment API sometimes charges customers twice. The client retries on timeout." The first request was executed but its response was lost, so the retry executed it again. Fix: the client sends an `Idempotency-Key` that is stable across retries; the server stores the key and result in the same transaction as the charge, with a unique constraint; a repeated key returns the stored response. Add a retention period for keys and reject a repeated key whose payload differs.

4.6.5 Questions to expect

1. What does idempotent mean, and which HTTP methods are idempotent?

Repeating a request has the same effect on the server state as sending it once. GET, HEAD, OPTIONS, PUT and DELETE are idempotent; POST and, in general, PATCH are not.

2. How can a POST be made safe to retry?

With an idempotency key supplied by the client and stored by the server together with the first result; a repeated key returns the stored response.

3. Which status codes are worth retrying?

Connection failures, timeouts, 429, 502, 503 and 504, for idempotent requests or with an idempotency key, with backoff and honouring `Retry-After`. 4xx errors other than 408 and 429 are permanent.

4. Why add jitter to exponential backoff?

Without it, clients that failed at the same time retry at the same times, and the load arrives in synchronised bursts that can keep a recovering service down.

5. What is the difference between 401, 403 and 404 for a protected resource?

401: not authenticated. 403: authenticated but not permitted. 404: the resource does not exist, or its existence is concealed from the caller.

6. Compare offset and cursor pagination.

Offset is simple and permits jumping to a page, but costs more for deep pages and is unstable under concurrent inserts. A cursor continues after the last key; the cost is constant and the traversal stable, but random access is lost.

sreeram

7. Which timeouts should every HTTP client configure?

Connect timeout, response (read) timeout and pool acquire timeout, plus an overall deadline that bounds retries. Defaults are often unlimited or very long.

8. What does 202 Accepted mean and how does the client obtain the result?

The request has been accepted for processing and is not complete. The response carries a `Location` of a status resource; the client polls it, or the server calls back through a webhook or event.

KEY POINTS

- A timeout means unknown outcome; automatic retries are permitted only for idempotent methods or requests protected by an `Idempotency-Key`.
- Use precise status classes and a uniform error body (`application/problem+json`), so that clients can separate transient from permanent failures.
- Every call needs a connect timeout, a response timeout and a bounded retry budget derived from the end-to-end deadline.
- Exponential backoff with jitter and respect for `Retry-After` prevent retry storms.
- Prefer cursor pagination for large or changing collections.

4.7 Cross-Origin Resource Sharing (CORS)

A single-page application served from `https://app.example.com` calls a Spring Boot API at `https://api.example.com`. The call works in `curl` and in Postman, and fails in the browser with a console message that mentions CORS. This is one of the most frequent problems encountered by full-stack developers, and it is frequently misdiagnosed as a server or network fault. The behaviour is neither: the request reaches the server, and the **browser** decides whether the page script may read the response. CORS is the protocol by which the server tells the browser which cross-origin reads it permits.

4.7.1 Concept and mechanism

The same-origin policy

Browsers isolate documents by **origin**, the triple of scheme, host and port. Scripts running in a page may freely interact with resources of the same origin; reading data from another origin is blocked by the **same-origin policy**. The policy exists because browsers attach ambient credentials (cookies, client certificates) automatically: without it, a script in a malicious page could call `https://bank.example.com/account` with the victim's cookie and read the answer.

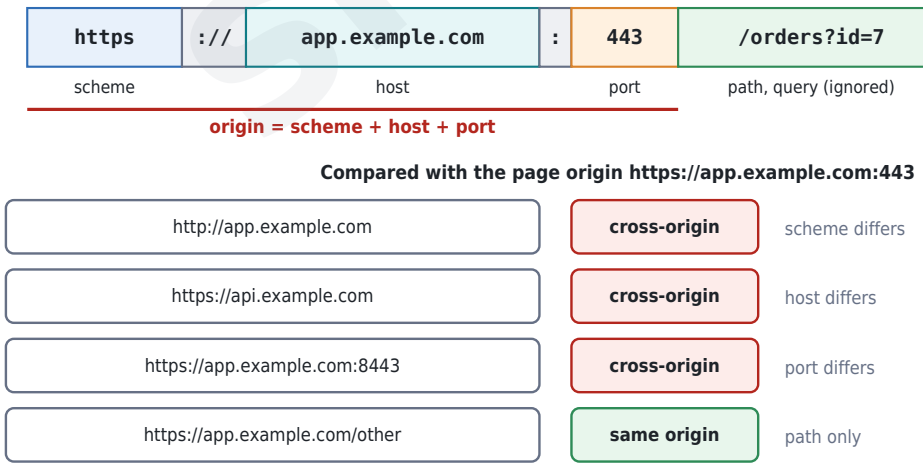


Figure 4.15. An origin is scheme, host and port; the path and query are not part of it. Of four URLs compared with the page origin, only the one that differs in path alone is same-origin.

The policy restricts reading, not sending. A page can always cause the browser to send a cross-site `GET` or a form `POST`; this is why CSRF exists and why `SameSite` cookies and CSRF tokens are needed independently of CORS.

Simple requests and preflighted requests

CORS relaxes the policy by letting the server opt in with response headers. The browser distinguishes two classes of cross-origin request.

A **simple request** (in the terminology of the Fetch standard, one that uses only CORS-safelisted elements) is sent directly. It must use the method `GET`, `HEAD` or `POST`; it may carry only safelisted headers such as `Accept` and `Accept-Language`; and its `Content-Type`, if present, must be `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`. These are exactly the kinds of request that HTML forms could already send before CORS existed. The browser adds an `Origin` header, sends the request, and then inspects the response: if `Access-Control-Allow-Origin` does not match, the script is denied access to the response.

Any other request is **preflighted**: a `PUT` or `DELETE`, a request with an `Authorization` header or a custom header, or a `POST` with `Content-Type: application/json`. Before sending it, the browser sends an `OPTIONS` request to the same URL, without a body and without credentials, asking whether the actual request would be acceptable. Only if the preflight response permits it does the browser send the real request. A preflight protects older servers that never expected such requests from cross-origin pages.

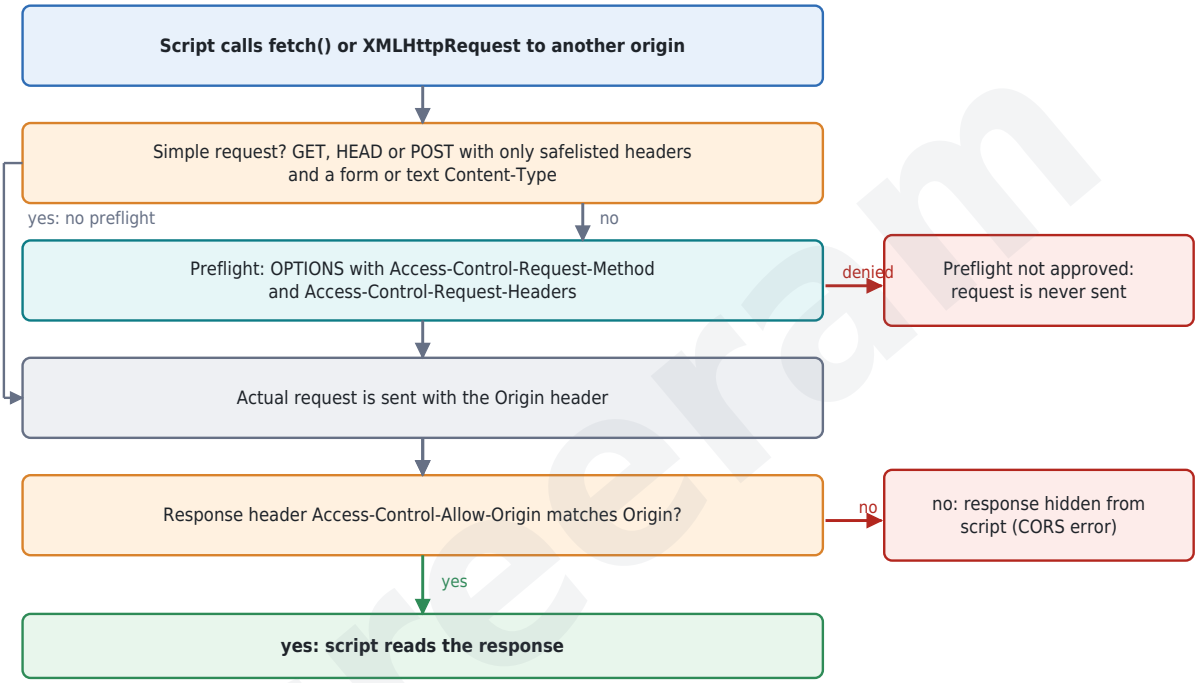


Figure 4.16. The browser's decision procedure: a simple request goes straight to the origin check on the response; any other request first needs an approving preflight, and in every case the script reads the response only when Access-Control-Allow-Origin matches.

The headers

Header	Sent by	Meaning
`Origin`	browser, every cross-origin request	the page origin, for instance `https://app.example.com`
`Access-Control-Request-Method`	browser, preflight	method of the intended request
`Access-Control-Request-Headers`	browser, preflight	non-safelisted headers of the intended request
`Access-Control-Allow-Origin`	server	the single allowed origin, or `*`
`Access-Control-Allow-Methods`	server, preflight response	methods permitted
`Access-Control-Allow-Headers`	server, preflight response	request headers permitted
`Access-Control-Allow-Credentials`	server	`true` permits cookies and `Authorization` credentials to be sent and the response to be read
`Access-Control-Max-Age`	server, preflight response	seconds for which the browser may cache the preflight result (browsers apply their own upper limits)
`Access-Control-Expose-Headers`	server	response headers other than the safelisted ones that the script may read
`Vary: Origin`	server	required when the allowed origin is chosen per request, so that caches do not serve one origin's answer to another

CORS header fields.

Credentials impose strict rules. For a request made with credentials (`fetch(url, {credentials: 'include'})`), the response must carry `Access-Control-Allow-Credentials: true`, and `Access-Control-Allow-Origin` must name one explicit origin; the wildcard `*` is rejected. Wildcards in the other headers are likewise not honoured for credentialed requests. The wildcard in `Access-Control-Allow-Headers` never covers `Authorization`, which must be listed.

What CORS is not

CORS is enforced by the browser alone. It does not authenticate anyone, does not stop `curl`, scripts or other servers from calling the API, and does not stop a simple cross-site request from being executed by the server. An API must still authenticate and authorise every request. A permissive CORS configuration (`Access-Control-Allow-Origin: *` on an API that uses cookies and reflects the `Origin` header) exposes authenticated data to any website the user visits.

4.7.2 Worked trace

The page `https://app.example.com` runs `fetch('https://api.example.com/api/items/7', {method: 'PUT', headers: {'Content-Type': 'application/json', 'Authorization': 'Bearer ...'}, body: ...})`.

srenem

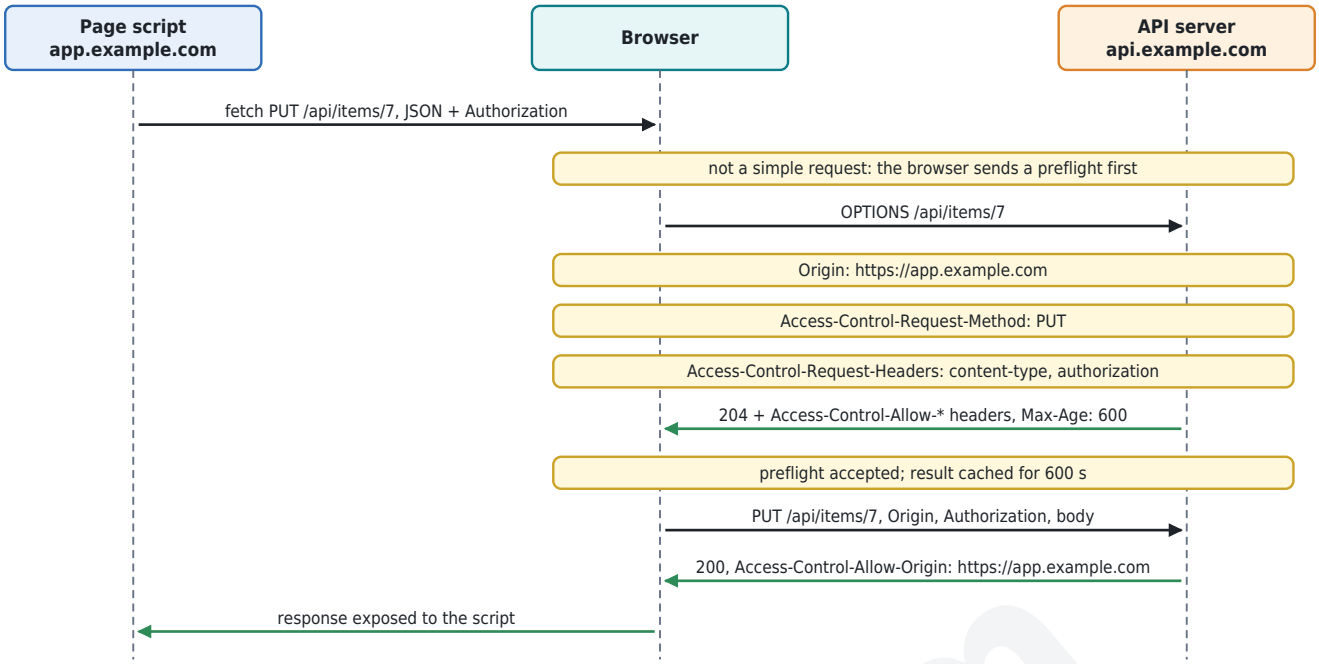


Figure 4.17. Complete preflight sequence for a PUT with a JSON body and an Authorization header: the browser sends OPTIONS with the Access-Control-Request headers, caches the approval for 600 seconds, sends the PUT with the Origin header, and finally exposes the response to the script.

Step	Message	Decision
1	script calls `fetch` with `PUT`, JSON content type, `Authorization`	the request is not simple (method, header, content type)
2	browser sends `OPTIONS /api/items/7`, `Origin: https://app.example.com`, `Access-Control-Request-Method: PUT`, `Access-Control-Request-Headers: content-type, authorization`	no cookies and no body are sent on the preflight
3	server answers `204` with `Access-Control-Allow-Origin: https://app.example.com`, `Access-Control-Allow-Methods: GET, PUT, DELETE`, `Access-Control-Allow-Headers: Content-Type, Authorization`, `Access-Control-Max-Age: 600`	origin, method and headers all appear in the allowed lists: approved
4	browser stores the approval for 600 s for this origin, URL and method combination	later identical calls within 600 s skip the preflight
5	browser sends `PUT /api/items/7` with `Origin` and `Authorization`	the real request, now permitted
6	server answers `200` with `Access-Control-Allow-Origin: https://app.example.com`	the final check on the response header matches the page origin
7	`fetch` resolves	the script reads the response; on any mismatch it would see a network-type error with no status or body

Step-by-step trace.

4.7.3 Hands-on

The `/api` endpoint of the local server from the HTTP/1.1 chapter allows the origin `https://app.example.com`. The commands imitate what the browser sends; `curl` itself performs no CORS check, which is the instructive point.

Preflight request and response TERMINAL

```
$ curl -si -X OPTIONS -H 'Origin: https://app.example.com' \
    -H 'Access-Control-Request-Method: PUT' \
    -H 'Access-Control-Request-Headers: content-type,authorization' \
    http://127.0.0.1:18080/api
HTTP/1.1 204 No Content
Access-control-allow-headers: Content-Type, Authorization
Access-control-max-age: 600
Date: Sat, 17 Oct 2026 17:03:42 GMT
Access-control-allow-methods: GET, PUT, DELETE
Access-control-allow-origin: https://app.example.com
Vary: Origin
```

Actual request from an allowed and from a foreign origin TERMINAL

```
$ curl -si -H 'Origin: https://app.example.com' http://127.0.0.1:18080/api
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 17:03:42 GMT
Access-control-allow-origin: https://app.example.com
Vary: Origin
Content-length: 12

{"ok":true}
$ curl -si -H 'Origin: https://evil.example.org' http://127.0.0.1:18080/api
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 17:03:42 GMT
Content-length: 12

{"ok":true}
```

What to notice. The response to the foreign origin has status 200 and a full body: the server executed the request. Only the missing `Access-Control-Allow-Origin` header would cause a browser to withhold the body from the page script. A server therefore must not rely on CORS to stop an unwanted state-changing request.

Configuration in Spring Boot

The following fragment shows the two usual forms. It requires the Spring dependencies and was not run in this guide.

CorsConfigFragment.java JAVA

```java
// Spring MVC (not run here): global rules for one origin
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("https://app.example.com")
                .allowedMethods("GET", "PUT", "DELETE")
                .allowedHeaders("Content-Type", "Authorization")
                .allowCredentials(false)
                .maxAge(600);
    }
}
// Per controller or method: @CrossOrigin(origins = "https://app.example.com")
// With Spring Security: http.cors(Customizer.withDefaults()) and a CorsConfigurationSource bean
```

With Spring Security the CORS processing must run before the authentication filters, which is what `http.cors(...)` arranges. Otherwise the credential-less preflight is answered with 401 and the browser reports a failed preflight although the actual endpoint is correct. Behind a reverse proxy, the CORS headers may be added at the proxy or at the gateway instead (`spring.cloud.gateway.globalcors` in Spring Cloud Gateway); they must be set in one place only, because duplicated `Access-Control-Allow-Origin` headers are rejected by browsers.

4.7.4 Failure modes

Console message (abridged)	Cause	Remedy
No 'Access-Control-Allow-Origin' header is present	server does not send the header for this origin, or an error response (4xx/5xx, proxy page) lacks it	send the header on all responses including errors
The value of the 'Access-Control-Allow-Origin' header must not be the wildcard '*' when the credentials mode is 'include'	credentials with *	echo the exact allowed origin, add `Allow-Credentials: true` and `Vary: Origin`
Response to preflight request doesn't pass access control check: It does not have HTTP ok status	preflight answered with 401, 403, 404, 405 or a redirect	allow `OPTIONS` without authentication; no redirects on preflight
Request header field authorization is not allowed by Access-Control-Allow-Headers	header not listed	add it explicitly to `Allow-Headers`
Method PUT is not allowed by Access-Control-Allow-Methods	method not listed	add it to `Allow-Methods`
The 'Access-Control-Allow-Origin' header contains multiple values	filter and proxy both add the header	configure CORS in exactly one layer

Common CORS errors. The wording is that of Chromium-based browsers and differs slightly elsewhere.

WATCH OUT
Reflecting any `Origin` header value into `Access-Control-Allow-Origin` together with `Access-Control-Allow-Credentials: true` allows every website to read authenticated responses. Match the origin against an explicit list, and send `Vary: Origin`.

GOTCHA
A CORS failure and a network failure look the same to the script: `fetch` rejects with a `TypeError` and no status code. The browser developer console, not the script, holds the explanation. If the Network tab shows the request with status 200 but the console reports a CORS error, the server executed the request and the browser withheld the response.

4.7.5 Questions to expect

1. What is the same-origin policy and why does it exist?

Scripts of one origin (scheme, host, port) may not read responses from another origin. It prevents a malicious page from using the user's ambient credentials to read data from other sites.

2. What problem does CORS solve?

It lets a server selectively relax the same-origin policy for specified origins, so that legitimate front ends on another origin can read its API responses.

3. When does a browser send a preflight?

When the request is not simple: a method other than GET, HEAD or POST; a non-safelisted header such as `Authorization`; or a `Content-Type` other than form-urlencoded, multipart or text/plain, for instance `application/json`. The preflight is an `OPTIONS` request with `Origin` and `Access-Control-Request-*` headers.

4. Does CORS protect my API from other clients?

No. It is a browser mechanism. Other clients ignore it, and a server executes cross-origin simple requests regardless. Authentication, authorisation and CSRF defences remain necessary.

5. Why can `Access-Control-Allow-Origin: *` not be used with cookies?

The browser rejects it for credentialed requests, because it would allow any site to read personalised responses. The server must name one origin and send `Access-Control-Allow-Credentials: true`.

6. Why does the preflight fail with 401 in a Spring Security application?

The browser sends the preflight without credentials, and the security filter chain rejects it before the CORS handling runs. Enable CORS in the security configuration so that it is processed first.

7. What is `Access-Control-Max-Age` for?

It lets the browser cache the preflight result, which saves one round trip per call. Browsers apply their own maximum (for example two hours in Chromium).

8. How can CORS be avoided altogether?

By serving the front end and the API from the same origin, for example through a reverse proxy that maps `/api` to the back end. Then no cross-origin request exists.

KEY POINTS

- Origin = scheme + host + port; the same-origin policy blocks reading cross-origin responses, not sending requests.
- Simple requests go directly and are checked afterwards; all others need an approving `OPTIONS` preflight first.
- The server grants access with `Access-Control-Allow-Origin` (and `-Methods`, `-Headers`, `-Credentials`, `-Max-Age`); credentials forbid `*`; send `Vary: Origin`.
- CORS is enforced by browsers only and is not a security control for the API; the server still executes the request.
- In Spring, configure CORS once, before authentication, and let the preflight pass without credentials.

4.8 WebSocket, Server-Sent Events and Long Polling

HTTP follows a request and response pattern: the server speaks only when asked. Applications such as chat, collaborative editing, live prices, notifications and dashboards need the server to deliver data when it becomes available. Three techniques provide this over the web: **long polling**, which stays within ordinary HTTP; **Server-Sent Events** (SSE), a one-way stream over one long HTTP response; and **WebSocket**, a full-duplex message channel that starts as an HTTP request and then leaves HTTP. Each has different costs in latency, connection count, proxy compatibility and implementation effort.

4.8.1 Concept and mechanism

Short polling and long polling

With **short polling** the client repeats a request at a fixed interval and the server answers immediately, usually with nothing new. The latency of an event is up to one interval, and most requests are wasted, each paying the HTTP header overhead. **Long polling** improves this: the server holds the request open until an event occurs or a timeout (commonly 20 to 60 seconds) expires, then responds, and the client immediately sends the next request. The latency is close to the network delay, and the number of empty responses drops. The cost is one pending request per client at all times, and event bursts can be missed between the response and the next request unless the server queues them by sequence number.

Server-Sent Events

SSE (part of the HTML standard) keeps one response open with `Content-Type: text/event-stream` and writes events as text lines. The browser API `EventSource` parses them, reconnects automatically after a failure, and sends the identifier of the last event received in the `Last-Event-ID` request header so that the server can resume. The channel is one-way (server to client); the client sends data through ordinary requests. Because SSE is plain HTTP, it works through proxies and benefits from HTTP/2 multiplexing; over HTTP/1.1 a browser limits the number of connections per host (six), which restricts the number of simultaneous event streams.

An event is a block of `field: value` lines ended by an empty line. The fields are `id`, `event` (the event name), `data` (repeated for multi-line payloads) and `retry` (reconnect delay in milliseconds).

WebSocket

WebSocket (RFC 6455) provides a persistent, bidirectional channel of messages between a browser, or any client, and a server. The connection starts as an HTTP/1.1 request with `Upgrade: websocket`; the server answers `101 Switching Protocols`, and from then on the same TCP connection carries WebSocket frames, not HTTP. The URL schemes are `ws://` (port 80) and `wss://` (port 443, WebSocket over TLS).

The server proves that it understood the upgrade request through the `Sec-WebSocket-Accept` header. The value is the Base64 encoding of the SHA-1 hash of the client's `Sec-WebSocket-Key` concatenated with the fixed string `258EAFA5-E914-47DA-95CA-C5AB0DC85B11`. The mechanism does not provide security; it prevents an ordinary HTTP server or cache from mistaking the exchange for something else.

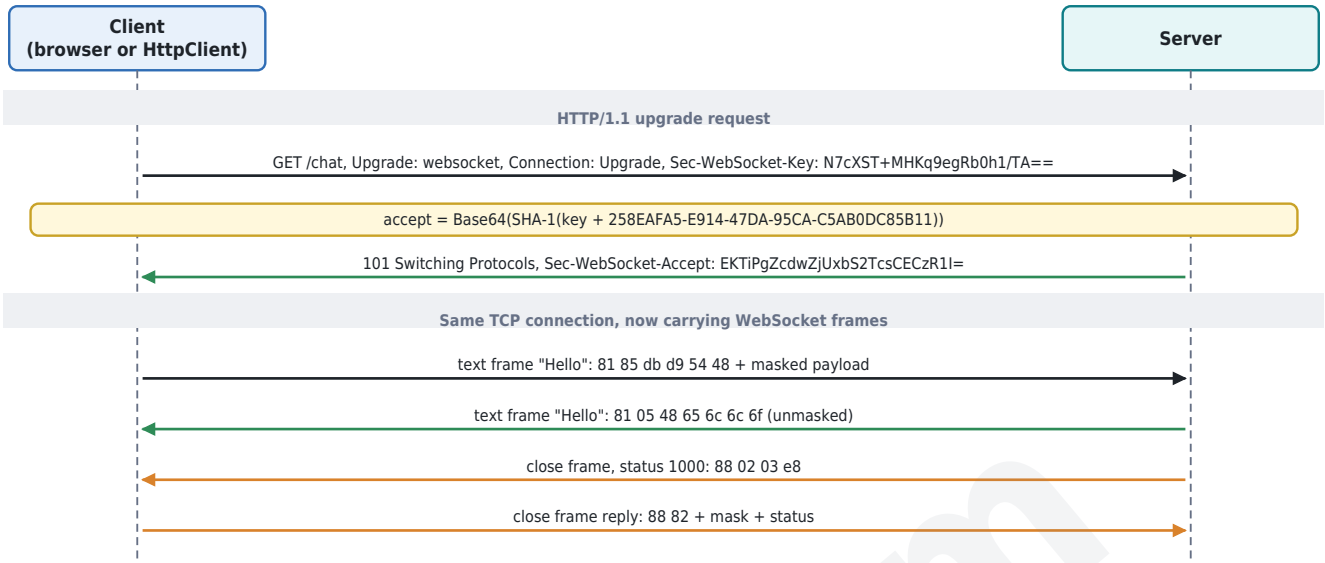


Figure 4.18. The WebSocket handshake and message exchange on one TCP connection: an HTTP upgrade request with a random key, a 101 response with the derived accept value, then data frames in both directions and a close handshake with status 1000.

Frame format

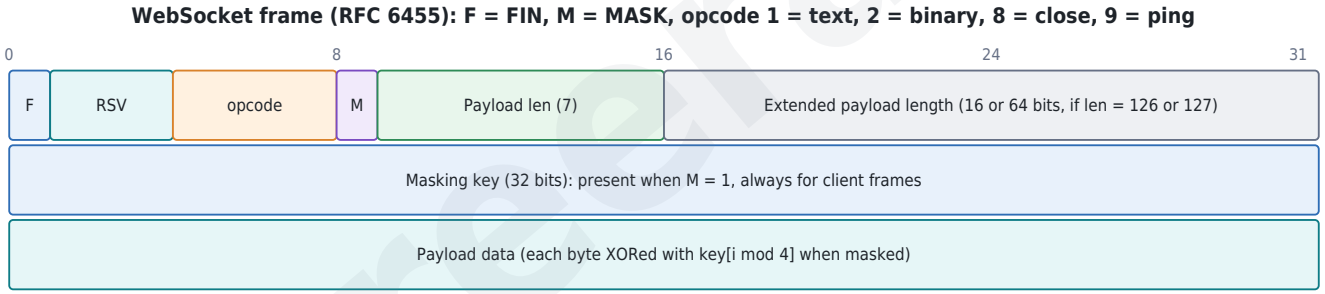


Figure 4.19. The WebSocket frame header is 2 to 14 bytes: FIN and opcode, a mask bit with a 7-bit length (126 selects a 16-bit extended length, 127 a 64-bit one), an optional 32-bit masking key, and the payload.

The first byte holds the FIN bit (last fragment) and the opcode: 0 continuation, 1 text, 2 binary, 8 close, 9 ping, 10 pong. Messages larger than one frame are fragmented. **Masking** is mandatory for frames from client to server: each payload byte is XORed with one of four key bytes, selected cyclically. The purpose is not confidentiality; it prevents a malicious script from crafting bytes that a transparent proxy might misinterpret as an HTTP message. Frames from server to client are not masked. **Ping** and **pong** frames check liveness. A **close** frame carries a 2-byte status code: 1000 normal closure, 1001 going away, 1008 policy violation, 1011 internal error; 1006 (abnormal closure, no close frame received) is reported locally and never sent.

Comparison

Property	Long polling	Server-Sent Events	WebSocket
Direction	client request, server response	server to client only	both directions
Transport	repeated HTTP requests	one long HTTP response	TCP after an HTTP upgrade
Message format	any HTTP body	UTF-8 text events	text or binary frames
Reconnection	client code	built into `EventSource`, with `Last-Event-ID`	client code
Proxy and firewall behaviour	best	good, buffering must be disabled	needs `Upgrade` support and long idle timeouts
Overhead per message	full HTTP headers	few bytes	2 to 14 bytes
Typical use	fallback, rare events	notifications, live feeds, progress, token streaming	chat, games, collaboration, trading UIs

Real-time techniques compared.

The decision rule is short: choose SSE when the data flows from the server only and the events are text; choose WebSocket when the client sends frequent messages as well, when binary payloads or very low per-message overhead matter; choose long polling only as a fallback or for infrequent events in restrictive environments.

Scaling and infrastructure

Every open connection holds a socket (a file descriptor), buffers and, in a thread-per-connection model, a thread. A server with 100,000 concurrent WebSocket clients needs non-blocking I/O or virtual threads, raised file-descriptor limits and attention to memory per connection. Connections are stateful: when a message must reach clients connected to different instances, the instances exchange it through a message broker or a publish/subscribe channel (Redis, Kafka, a STOMP broker relay). Load balancers must support the upgrade and must not close idle connections too early; typical idle timeouts of 60 seconds (nginx `proxy_read_timeout`, cloud load balancers) require **heartbeats** (ping frames or SSE comment lines) shorter than the timeout. Rolling deployments disconnect all clients of an instance at once, so clients must reconnect with backoff and jitter, as in the REST chapter, or the new instances face a reconnection storm.

In the Java ecosystem, Spring offers `spring-websocket` with raw handlers and STOMP messaging (`@MessageMapping`, `SimpMessagingTemplate`), `SseEmitter` for SSE in Spring MVC and `Flux<ServerSentEvent<T>>` in WebFlux, and the Jakarta WebSocket API (`@ServerEndpoint`). `java.net.http.WebSocket` is a client API in the JDK.

4.8.2 Worked trace

The trace is the one captured in the hands-on section, on the loopback interface.

Step	Bytes	Meaning
1	`GET /chat HTTP/1.1`, `Upgrade: websocket`, `Connection: Upgrade`, `Sec-WebSocket-Key: N7cXST+MHKq9egRb0h1/TA==`, `Sec-WebSocket-Version: 13`	upgrade request; the key is 16 random bytes in Base64
2	server computes Base64(SHA-1(key + GUID))	result `EKTiPgZcdwZjUxbS2TcsCECzR1I=`
3	`HTTP/1.1 101 Switching Protocols` with `Sec-WebSocket-Accept: EKTiPgZcdwZjUxbS2TcsCECzR1I=`	HTTP ends; frames begin
4	`81 85 db d9 54 48 93 bc 38 24 b4`	client text frame: `81` = FIN 1, opcode 1; `85` = MASK 1, length 5; key `db d9 54 48`; masked payload `93 bc 38 24 b4`
5	unmasking: `93 xor db`, `bc xor d9`, `38 xor 54`, `24 xor 48`, `b4 xor db`	`48 65 6c 6c 6f` = "Hello"
6	`81 05 48 65 6c 6c 6f`	server text frame: no mask bit, length 5, plain payload
7	`88 02 03 e8`	server close frame: opcode 8, length 2, status 0x03e8 = 1000
8	`88 82 53 7d 9f 12 50 95`	client reply: mask `53 7d 9f 12`, payload `50 95` xor `53 7d` = `03 e8`

Handshake and first messages.

The overhead of the exchange is visible: a 5-byte message costs 11 bytes upstream and 7 bytes downstream, against several hundred bytes for the headers of an HTTP request and response.

4.8.3 Hands-on

A WebSocket server in 50 lines and a JDK client

The Python server accepts one connection, performs the handshake, decodes one client frame, echoes it, and closes. It is deliberately minimal and handles exactly one short exchange.

ws_server.py — PYTHON

```python
import base64
import hashlib
import socket

GUID = "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"      # fixed value defined by RFC 6455

srv = socket.socket()
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 19090))
srv.listen(1)
print("ws server on 19090", flush=True)
conn, _ = srv.accept()


def read_exact(n):
    data = b""
    while len(data) < n:
        data += conn.recv(n - len(data))
    return data


request = b""
while b"\r\n\r\n" not in request:
    request += conn.recv(4096)
print("--- client handshake ---")
print(request.decode().rstrip())
key = [l.split(": ")[1] for l in request.decode().split("\r\n")
       if l.lower().startswith("sec-websocket-key")][0]
accept = base64.b64encode(hashlib.sha1((key + GUID).encode()).digest()).decode()
response = ("HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection: Upgrade\r\n"
            "Sec-WebSocket-Accept: %s\r\n\r\n" % accept)
conn.sendall(response.encode())
print("--- server handshake ---")
print(response.rstrip())

head = read_exact(2)                                  # FIN/opcode byte, MASK/length byte
fin, opcode = head[0] >> 7, head[0] & 0x0F
masked, length = head[1] >> 7, head[1] & 0x7F
mask = read_exact(4) if masked else b""
payload = read_exact(length)
text = bytes(b ^ mask[i % 4] for i, b in enumerate(payload)) if masked else payload
print("--- frame from client ---")
print("bytes   :", (head + mask + payload).hex(" "))
print("FIN=%d opcode=%d MASK=%d len=%d mask-key=%s" % (fin, opcode, masked, length, mask.hex()))
print("payload : %s -> %r" % (payload.hex(" "), text.decode()))

reply = b"\x81" + bytes([len(text)]) + text            # unmasked text frame
conn.sendall(reply)
print("server text frame :", reply.hex(" "))
conn.sendall(b"\x88\x02\x03\xe8")                      # close frame, status 1000
print("server close frame: 88 02 03 e8")
conn.settimeout(2)
print("client close frame:", conn.recv(64).hex(" "))
```

The client uses the WebSocket API of `java.net.http`. The `Listener` receives callbacks; `request(1)` signals readiness for the next message, since the API applies backpressure.

WsClient.java JAVA

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.WebSocket;
import java.util.concurrent.CompletionStage;
import java.util.concurrent.CountDownLatch;

public class WsClient {
    public static void main(String[] args) throws Exception {
        CountDownLatch done = new CountDownLatch(1);
        WebSocket.Listener l = new WebSocket.Listener() {
            public CompletionStage<?> onText(WebSocket ws, CharSequence d, boolean last) {
                System.out.println("client got text: " + d);
                ws.request(1);
                return null;
            }
            public CompletionStage<?> onClose(WebSocket ws, int code, String reason) {
                System.out.println("client got close: " + code);
                done.countDown();
                return ws.sendClose(WebSocket.NORMAL_CLOSURE, "");
            }
        };
        WebSocket ws = HttpClient.newHttpClient().newWebSocketBuilder()
                .buildAsync(URI.create("ws://127.0.0.1:19090/chat"), l).join();
        System.out.println("negotiated subprotocol: '" + ws.getSubprotocol() + "'");
        ws.sendText("Hello", true).join();
        done.await();
        Thread.sleep(300);
    }
}
```

Output of both programs, started in this order (the key and the mask are random on each run) TERMINAL

```
$ python3 ws_server.py &
$ java WsClient.java
negotiated subprotocol: ''
client got text: Hello
client got close: 1000
```

Output printed by ws_server.py TERMINAL

```
$ cat ws.out
ws server on 19090
--- client handshake ---
GET /chat HTTP/1.1
Connection: Upgrade
Host: 127.0.0.1:19090
Upgrade: websocket
User-Agent: Java-http-client/21.0.12.1
Sec-WebSocket-Key: N7cXST+MHKq9egRb0h1/TA==
Sec-WebSocket-Version: 13
--- server handshake ---
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: EKTiPgZcdwZjUxbS2TcsCECzR1I=
--- frame from client ---
bytes   : 81 85 db d9 54 48 93 bc 38 24 b4
FIN=1 opcode=1 MASK=1 len=5 mask-key=dbd95448
payload : 93 bc 38 24 b4 -> 'Hello'
server text frame : 81 05 48 65 6c 6c 6f
server close frame: 88 02 03 e8
client close frame: 88 82 53 7d 9f 12 50 95
```

Server-Sent Events with curl

The `/events` endpoint of the local server writes three events at 200 ms intervals. The option `-N` disables curl output buffering, so the events appear as they are sent. The response uses chunked transfer coding with no `Content-Length`, as in the HTTP/1.1 chapter.

An SSE stream TERMINAL

```
$ curl -si -N http://127.0.0.1:18080/events
HTTP/1.1 200 OK
Date: Sat, 17 Oct 2026 16:48:49 GMT
Transfer-encoding: chunked
Content-type: text/event-stream
Cache-control: no-cache

id: 1
event: price
data: {"sku":"A1","v":101}

id: 2
event: price
data: {"sku":"A1","v":102}

id: 3
event: price
data: {"sku":"A1","v":103}
```

Each event ends with an empty line. A browser `EventSource` would dispatch a `price` event to its listeners with the `data` text, and after a dropped connection would reconnect with `Last-Event-ID: 3`.

4.8.4 Failure modes

Symptom	Cause	Remedy
WebSocket handshake returns 400 or 200	proxy does not forward `Upgrade` and `Connection` headers (HTTP/1.1 required)	configure the proxy (`proxy_http_version 1.1`, `Upgrade` and `Connection` headers in nginx)
Connection drops every 60 seconds	idle timeout in a load balancer or proxy	send ping frames or SSE comment lines (`: keepalive`) more often than the timeout
SSE events arrive in bursts or at the end	proxy or compression buffers the response	disable buffering (for nginx `X-Accel-Buffering: no`), flush after each event
Only six SSE streams per browser work (HTTP/1.1)	per-host connection limit	use HTTP/2; share one stream per page
Memory growth with many clients	one buffer or thread per connection; slow consumers accumulate queued messages	bounded per-client queues, drop or disconnect slow clients; non-blocking I/O
Reconnection storm after a deployment	all clients reconnect at once	randomised reconnect delay with backoff; connection draining
Messages missed during reconnect	no replay	sequence numbers or `Last-Event-ID` with a replay buffer

Typical failures of long-lived connections.

WATCH OUT

Browser WebSocket connections are not subject to the same-origin policy and are not restricted by CORS. The server must check the `Origin` header during the handshake and authenticate the connection, otherwise any web page the user visits can open an authenticated socket (cross-site WebSocket hijacking). Credentials in the URL query are logged by proxies; prefer a short-lived ticket obtained beforehand.

4.8.5 Questions to expect

1. How does a WebSocket connection start?

With an HTTP/1.1 `GET` containing `Upgrade: websocket`, `Connection: Upgrade`, `Sec-WebSocket-Key` and `Sec-WebSocket-Version: 13`. The server answers `101 Switching Protocols` with `Sec-WebSocket-Accept`, computed from the key. The TCP connection then carries WebSocket frames.

2. Why are client frames masked?

To prevent attacks on intermediaries that might interpret attacker-controlled bytes as HTTP (cache poisoning). The mask is a 32-bit random key in each frame; it is not encryption. TLS (`wss://`) provides confidentiality.

3. When would SSE be chosen over WebSocket?

For one-way, text-based streams: notifications, live feeds, progress and streamed model output. SSE needs no special protocol support in proxies, reconnects automatically and resumes with `Last-Event-ID`.

4. What is long polling and what are its drawbacks?

The client sends a request that the server holds until an event occurs or a timeout passes; the client then immediately sends a new request. Drawbacks: header overhead per event, a pending connection per client, and a gap between response and next request in which events can be missed unless they are queued.

5. How is a WebSocket application scaled to several instances?

A client is connected to one instance, so messages for it must reach that instance: a broker or pub/sub layer (Redis, a STOMP relay, Kafka) fans events out to all instances. The load balancer needs upgrade support, long idle timeouts and a way to drain connections on deployment.

6. How can a server detect that a client has silently disappeared?

TCP keep-alive operates on a coarse scale (hours by default). Application-level ping and pong frames with a timeout detect it in seconds; the server closes the connection when no pong arrives.

7. What are the security considerations of WebSocket?

Validate the `Origin` header, authenticate during or immediately after the handshake, use `wss://`, limit message size and rate, and validate every message as untrusted input, because the HTTP-level protections (CORS, cookies policy) do not apply to messages.

KEY POINTS

- Long polling, SSE and WebSocket differ in direction, transport and cost: SSE is one-way text over HTTP, WebSocket is full duplex over an upgraded TCP connection.
- The WebSocket handshake derives `Sec-WebSocket-Accept` from the key with SHA-1 and a fixed GUID; frames are 2 to 14 bytes of header, and client frames are masked.
- Long-lived connections need heartbeats shorter than every idle timeout, proxy support for upgrade and unbuffered streaming, and reconnection with jitter.
- Scaling stateful connections requires a broker for fan-out, non-blocking or virtual-thread I/O, and bounded per-client queues.
- Check `Origin` and authenticate WebSocket connections; CORS does not protect them.

4.9 gRPC and Other Application Protocols

HTTP with JSON is the default for public APIs, but it is not the only application-layer protocol that a Java developer meets. Service-to-service calls inside a platform often use gRPC for its compact encoding and streaming; e-mail still depends on SMTP and IMAP; file transfer and administration use FTP and SSH. This chapter describes how gRPC is built on HTTP/2 and gives a compact overview of the classic protocols and their ports. All of them share the structure studied so far: a well-known port, a text or binary framing on a TCP connection, and a handshake that establishes capabilities.

4.9.1 Concept and mechanism

gRPC

gRPC is a remote-procedure-call framework. A service and its messages are defined in a `.proto` file, and code generators produce client stubs and server skeletons for Java, Go, Python and other languages. A call `orders.Get(request)` looks like a local method call; the framework serialises the request, sends it, and deserialises the response.

Protocol Buffers (protobuf) is the default encoding. A message is a sequence of fields, each written as a key and a value. The key is a varint holding `(field_number << 3) | wire_type`. Integers use varints (7 data bits per byte, least significant group first, the high bit marks continuation), and strings use wire type 2 with a length prefix. Field names

never appear on the wire, only numbers, which makes messages small and parsing fast, and also requires both sides to share the schema. Adding a field with a new number is backward compatible; reusing or renumbering a field is not.

Transport. gRPC runs on HTTP/2, one call per stream. The request is a `POST` to the path `/<package.Service>/<Method>` with `content-type: application/grpc`. Each protobuf message is wrapped in a 5-byte prefix: one byte for the compression flag and four bytes for the message length, followed by the message. The response carries the status not in the HTTP status line, which is always 200, but in the **trailers** (HEADERS frames after the body) as `grpc-status` and `grpc-message`. Metadata (authentication tokens, trace identifiers) travels as ordinary HTTP/2 headers. A **deadline** set by the caller is sent as `grpc-timeout` and propagates through a chain of calls.

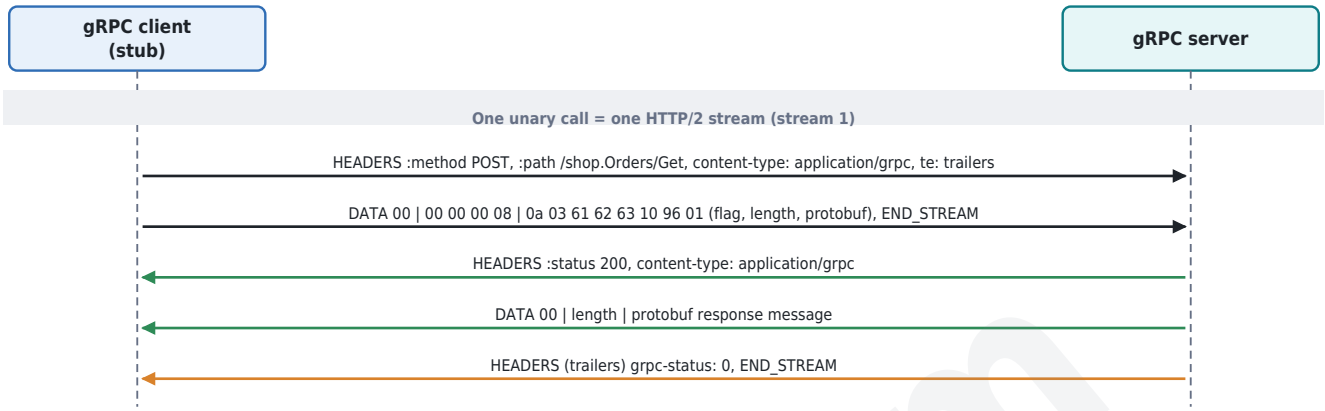


Figure 4.20. One unary gRPC call as HTTP/2 frames on stream 1: request HEADERS, a DATA frame with the length-prefixed protobuf message, response HEADERS and DATA, and trailers that carry grpc-status.

Type	Pattern	Example
Unary	one request, one response	`GetOrder`
Server streaming	one request, a stream of responses	price updates, log tail
Client streaming	a stream of requests, one response	bulk upload with a summary
Bidirectional streaming	independent streams in both directions on one call	chat, interactive sessions

The four call types.

Code	Name	Typical meaning and handling
0	OK	success
3	INVALID_ARGUMENT	caller error; do not retry
4	DEADLINE_EXCEEDED	deadline elapsed; the outcome is unknown, as with an HTTP timeout
5	NOT_FOUND	missing resource
14	UNAVAILABLE	transient failure (connection lost, server not ready); retry with backoff
16	UNAUTHENTICATED	missing or invalid credentials

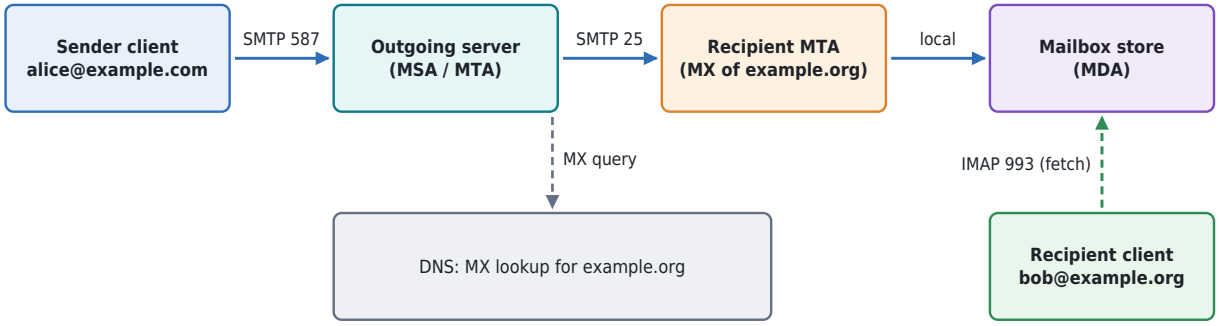
gRPC status codes that matter in operation.

Aspect	gRPC	REST with JSON
Schema	mandatory `.proto`, generated code	optional (OpenAPI)
Encoding	binary protobuf	text JSON
Transport	HTTP/2 only	HTTP/1.1, HTTP/2, HTTP/3
Streaming	native, four call types	SSE or WebSocket as add-ons
Browser use	needs gRPC-Web and a proxy	native
Debugging	needs tooling (grpcurl, reflection)	`curl`, browser
Caching	not cacheable by HTTP caches	HTTP caching applies
Typical role	internal service-to-service	public and browser-facing APIs

gRPC compared with REST over JSON.

Two operational consequences follow from HTTP/2. A client opens few, long-lived connections, so a layer-4 load balancer distributes connections and not calls: one client may send all its traffic to one backend. Layer-7 balancing at a proxy that understands HTTP/2, or client-side balancing over a list of addresses obtained from DNS or a registry, distributes calls. Second, a proxy that does not support HTTP/2 end to end, including trailers, breaks gRPC.

Classic protocols



Submission is authenticated and uses STARTTLS (587) or implicit TLS (465).

Figure 4.21. The path of an e-mail: the sender client submits over SMTP on port 587 to an outgoing server, which finds the recipient server through an MX lookup in DNS and relays the message over SMTP on port 25; the recipient fetches it from the mailbox with IMAP over TLS.

- **SMTP** (RFC 5321) transfers mail between servers and from clients to their outgoing server. It is a text dialogue of commands (`EHLO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`) answered by three-digit codes (2xx success, 4xx temporary failure, 5xx permanent failure). Server-to-server delivery uses port 25; authenticated client submission uses 587 (STARTTLS upgrade) or 465 (implicit TLS). The recipient server is located by the MX records of the domain.
- **IMAP** (143, or 993 with implicit TLS) lets clients read and manage mail that stays on the server; **POP3** (110, 995) downloads and usually removes it.
- **FTP** uses a control connection on port 21 and separate data connections. In active mode the server connects back from port 20 to a client port, which fails behind NAT and firewalls; in passive mode the server opens a port and the client connects to it. Plain FTP sends credentials unencrypted; SFTP (a subprotocol of SSH, port 22) or FTPS should replace it.
- **SSH** (port 22) provides encrypted remote login, command execution, file transfer (SFTP, SCP) and port forwarding. The two sides first exchange version banners (`SSH-2.0-...`), negotiate algorithms, perform a key exchange, and authenticate the server by its host key and the user by password or public key. SSH tunnels (`ssh -L`) are a common way to reach private databases.

Service	Port	Transport and note
`HTTP / HTTPS`	`80 / 443`	TCP; HTTP/3 uses UDP 443
`DNS`	`53`	UDP, TCP fallback; DNS over TLS 853
`SSH, SFTP`	`22`	TCP
`SMTP, submission, SMTPS`	`25, 587, 465`	TCP
`IMAP, IMAPS`	`143, 993`	TCP
`POP3, POP3S`	`110, 995`	TCP
`FTP control, data`	`21, 20`	TCP
`NTP`	`123`	UDP
`DHCP server, client`	`67, 68`	UDP
`LDAP, LDAPS`	`389, 636`	TCP
`PostgreSQL, MySQL`	`5432, 3306`	TCP
`Redis, MongoDB`	`6379, 27017`	TCP
`Kafka broker`	`9092`	TCP (default listener)
`Spring Boot embedded server`	`8080`	TCP; configured by `server.port`

Well-known and common ports.

4.9.2 Worked trace

The protobuf message of the hands-on program, `GetOrderRequest { string id = 1; int32 qty = 2; }` with `id = "abc"` and `qty = 150`, is encoded as follows.

Bytes	Field	Derivation
`0a`	key of field 1	(1 << 3) or 2 = 0x0a; wire type 2 is length-delimited
`03`	length	the string has 3 bytes
`61 62 63`	value	ASCII "abc"
`10`	key of field 2	(2 << 3) or 0 = 0x10; wire type 0 is varint
`96 01`	value 150	150 = 0b1001_0110; low 7 bits `0010110` with continuation bit gives `0x96`; remaining bits `1` give `0x01`

Encoding of GetOrderRequest.

The message is 8 bytes, against 22 bytes for the equivalent JSON `{"id":"abc","qty":150}`. As gRPC payload it is prefixed with `00 00 00 00 08` (not compressed, length 8) and sent in a DATA frame of stream 1, as in the diagram.

4.9.3 Hands-on

Protobuf and gRPC framing by hand

proto_demo.py — PYTHON

```python
import struct


def varint(n):
    out = bytearray()
    while True:
        b = n & 0x7F
        n >>= 7
        if n:
            out.append(b | 0x80)
        else:
            out.append(b)
            return bytes(out)


def field_string(number, text):
    data = text.encode()
    key = bytes([(number << 3) | 2])                    # wire type 2: length-delimited
    return key + varint(len(data)) + data


def field_int(number, value):
    return bytes([(number << 3) | 0]) + varint(value)                  # wire type 0: varint


# message GetOrderRequest { string id = 1; int32 qty = 2; }
msg = field_string(1, "abc") + field_int(2, 150)
print("protobuf message :", msg.hex(" "), "(%d bytes)" % len(msg))
print("varint(150)      :", varint(150).hex(" "), " 150 = 0b10010110 -> low 7 bits first")

grpc_message = b"\x00" + struct.pack("!I", len(msg)) + msg             # compressed flag + length
print("gRPC DATA payload:", grpc_message.hex(" "))
print("JSON equivalent  :", '{"id":"abc","qty":150}', "(%d bytes)" % len('{"id":"abc","qty":150}'))
```

Output of proto_demo.py — TERMINAL

```
$ python3 proto_demo.py
protobuf message : 0a 03 61 62 63 10 96 01 (8 bytes)
varint(150)      : 96 01  150 = 0b10010110 -> low 7 bits first
gRPC DATA payload: 00 00 00 00 08 0a 03 61 62 63 10 96 01
JSON equivalent  : {"id":"abc","qty":150} (22 bytes)
```

A real gRPC server in Java needs the `grpc-java` libraries and generated stubs, or the Spring gRPC starter, which are not available on the build machine; the framing above is exactly what such a server receives in a DATA frame.

sreeram

An SMTP dialogue over loopback

The Python standard library client `smtplib` speaks SMTP; its debug mode prints each command and reply. A minimal server (below) answers the dialogue on 127.0.0.1:12525, so that no mail leaves the machine. It recognises only the commands used here.

smtp_server.py PYTHON

```python
import socket

srv = socket.socket()
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 12525))
srv.listen(1)
conn, _ = srv.accept()
f = conn.makefile("rwb", buffering=0)


def say(line):
    f.write((line + "\r\n").encode())


say("220 mail.example.org ESMTP demo")
in_data = False
while True:
    raw = f.readline()
    if not raw:
        break
    line = raw.decode().rstrip("\r\n")
    if in_data:
        if line == ".":
            in_data = False
            say("250 2.0.0 queued as 7F3A")
        continue
    verb = line.split(" ")[0].upper()
    if verb == "EHLO":
        say("250-mail.example.org")
        say("250 8BITMIME")
    elif verb in ("MAIL", "RCPT"):
        say("250 2.1.0 OK")
    elif verb == "DATA":
        in_data = True
        say("354 End data with <CR><LF>.<CR><LF>")
    elif verb == "QUIT":
        say("221 2.0.0 Bye")
        break
    else:
        say("502 5.5.2 command not recognised")
```

smtp_client.py PYTHON

```python
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "alice@example.com"
msg["To"] = "bob@example.org"
msg["Subject"] = "Test"
msg.set_content("Hello Bob.")

with smtplib.SMTP("127.0.0.1", 12525) as smtp:
    smtp.set_debuglevel(1)
    smtp.send_message(msg)
```

Command and reply lines of the dialogue (repeated debug lines omitted, one long line wrapped) TERMINAL

```
$ python3 smtp_client.py
send: 'ehlo [127.0.0.1]\r\n'
reply: b'250-mail.example.org\r\n'
reply: b'250 8BITMIME\r\n'
send: 'mail from:<alice@example.com>\r\n'
reply: b'250 2.1.0 OK\r\n'
send: 'rcpt to:<bob@example.org>\r\n'
reply: b'250 2.1.0 OK\r\n'
send: 'data\r\n'
reply: b'354 End data with <CR><LF>.<CR><LF>\r\n'
```

sreeram

Command and reply lines of the dialogue (repeated debug lines omitted, one long line wrapped) (continued) TERMINAL

```
send: b'From: alice@example.com\r\nTo: bob@example.org\r\nSubject: Test\r\nContent-Type: text/plain;
 charset="utf-8"\r\nContent-Transfer-Encoding: 7bit\r\nMIME-Version: 1.0\r\n\r\nHello Bob.\r\n.\r\n'
reply: b'250 2.0.0 queued as 7F3A\r\n'
send: 'QUIT\r\n'
reply: b'221 2.0.0 Bye\r\n'
```

What to notice. Every client command receives a reply that begins with a three-digit code; `250-` with a hyphen marks a continuation line of a multi-line reply, and `250 ` with a space ends it. The message body is terminated by a line that holds a single dot. The envelope (`MAIL FROM`, `RCPT TO`) is separate from the message headers (`From:`, `To:`), which is why spoofed headers are possible and why SPF, DKIM and DMARC records (TXT entries in DNS) exist. In Java the corresponding API is Jakarta Mail, and Spring Boot configures it with `spring.mail.host`, `spring.mail.port` and `spring.mail.properties.mail.smtp.starttls.enable`.

The SSH banner (typical output)

An SSH server announces itself as the first line of the connection, before any negotiation. The `nc` command shows it; no SSH server runs on the build machine, so the output is typical.

Typical output (not run here) TERMINAL

```
$ nc 192.0.2.20 22
SSH-2.0-OpenSSH_9.6
```

4.9.4 Failure modes

Symptom	Cause	Diagnosis and remedy
gRPC client always reaches the same backend	long-lived HTTP/2 connection behind a layer-4 balancer	layer-7 or client-side balancing; limit connection age so that clients reconnect
gRPC call fails with UNAVAILABLE after a proxy was introduced	proxy lacks HTTP/2 end to end or drops trailers	use an HTTP/2-aware proxy; check `grpc-status` handling
`DEADLINE_EXCEEDED` in a chain of services	each hop sets a long or no deadline	propagate the caller deadline downstream
Protobuf decode errors or silently wrong data	field numbers reused or changed incompatibly	never reuse numbers; mark removed fields `reserved`
Mail rejected with 550 or classified as spam	missing SPF, DKIM or DMARC; sender reputation	publish the DNS records; check bounce codes
Connection to port 25 from cloud hosts times out	outbound port 25 blocked by the provider	use submission (587) through an authorised relay
FTP listing hangs	passive mode data port blocked by firewall or NAT	open the passive port range or use SFTP
SSH `Host key verification failed`	host key changed (reinstallation, address reused) or an interception	verify the new fingerprint out of band before updating `known_hosts`

Typical failures of these protocols.

REAL WORLD

Inside a Kubernetes cluster, a gRPC service is commonly exposed through a headless service so that a client-side balancer receives the addresses of all pods from DNS, or through a service mesh sidecar that balances each call. A default ClusterIP service balances only the connections, which causes the single-backend symptom above.

4.9.5 Questions to expect

1. What is gRPC and how does it use HTTP/2?

An RPC framework with protobuf schemas and generated stubs. Each call is one HTTP/2 stream: a `POST` to `/package.Service/Method` with `content-type: application/grpc`, length-prefixed protobuf messages in DATA frames, and the status in trailers (`grpc-status`).

2. Why is protobuf smaller than JSON?

Field names are replaced by numbers, integers use varints, and there is no punctuation or whitespace. The cost is a required shared schema and no human readability.

3. When would REST be preferred to gRPC?

For public and browser-facing APIs, when HTTP caching, simple debugging and wide client compatibility matter. gRPC suits internal, high-volume, low-latency service calls and streaming.

4. What are the four gRPC call types?

Unary, server streaming, client streaming and bidirectional streaming.

5. Why does load balancing gRPC need care?

HTTP/2 multiplexes many calls on one long-lived connection, so connection-level balancing places all calls on one backend. The balancing must be per call, by a layer-7 proxy or by the client.

6. Which ports and protocols are used for e-mail?

SMTP on 25 between servers, 587 or 465 for client submission, IMAP on 143 or 993 and POP3 on 110 or 995 for retrieval; the recipient server is found by DNS MX records.

7. Why is FTP difficult to use through firewalls and NAT?

It uses a separate data connection whose address and port are negotiated in the control connection. Active mode requires an inbound connection to the client; passive mode requires a range of open server ports. SFTP uses a single connection.

8. What does SSH provide beyond remote login?

Encrypted tunnels, port forwarding, file transfer (SFTP, SCP), public-key authentication and host-key verification for protection against interception.

KEY POINTS

- gRPC maps one call to one HTTP/2 stream: length-prefixed protobuf messages in DATA frames, status in trailers, deadlines in `grpc-timeout`.
- Protobuf encodes `(field_number << 3) | wire_type` keys and varint values; a field number must never be reused.
- HTTP/2 connection reuse makes layer-4 balancing ineffective for gRPC; balance per call.
- SMTP delivers mail (25, 587, 465) using MX records; IMAP (143, 993) and POP3 (110, 995) retrieve it; FTP needs a data connection; SSH (22) secures remote access and tunnels.
- Remembering default ports and the transport (TCP or UDP) of each protocol is a common interview requirement.

sreeram

PART 5

Security and Infrastructure

Cryptography, TLS, authentication, network defence, proxies, load balancing, content delivery and the network design of microservices.

In this part

- **5.1** Cryptography for Networking
- **5.2** TLS and HTTPS
- **5.3** Authentication in Transit
- **5.4** Network Security
- **5.5** Reverse Proxies, Load Balancers and Gateways
- **5.6** CDNs, Caching and High Availability
- **5.7** Network Design for Microservices

5.1 Cryptography for Networking

A packet crosses many devices that the sender neither owns nor controls: Wi-Fi access points, switches, routers of several operators, proxies and load balancers. Any of them can read a packet, modify it, record it or inject a new one. The protocols of the earlier Parts do nothing against this. Cryptography supplies four properties that a secure channel needs.

- **Confidentiality**: only the intended parties can read the data.
- **Integrity**: any modification in transit is detected.
- **Authentication**: each party can verify the identity of the peer.
- **Non-repudiation**: the author of a message cannot plausibly deny it later (relevant for signatures, not for session keys).

No single primitive provides all four efficiently. TLS, SSH, IPsec and WireGuard all combine the same small set of building blocks: symmetric ciphers, key exchange, hash functions, message authentication codes, digital signatures and certificates. This chapter defines each block, shows a complete key exchange computed with small numbers, and explains how the blocks compose in TLS.

5.1.1 Concept and mechanism

Symmetric encryption

A symmetric cipher uses the same secret key to encrypt and to decrypt. Modern network protocols use **AEAD** ciphers (authenticated encryption with associated data) such as AES-GCM and ChaCha20-Poly1305. An AEAD cipher takes a key, a nonce (a number used once), the plaintext and optional associated data, and returns ciphertext plus an authentication tag. Decryption fails if a single bit of the ciphertext, the tag or the associated data was changed. Symmetric ciphers are fast (AES uses dedicated CPU instructions) and are the only primitive used for bulk data. Their weakness is key distribution: both parties must already share the key.

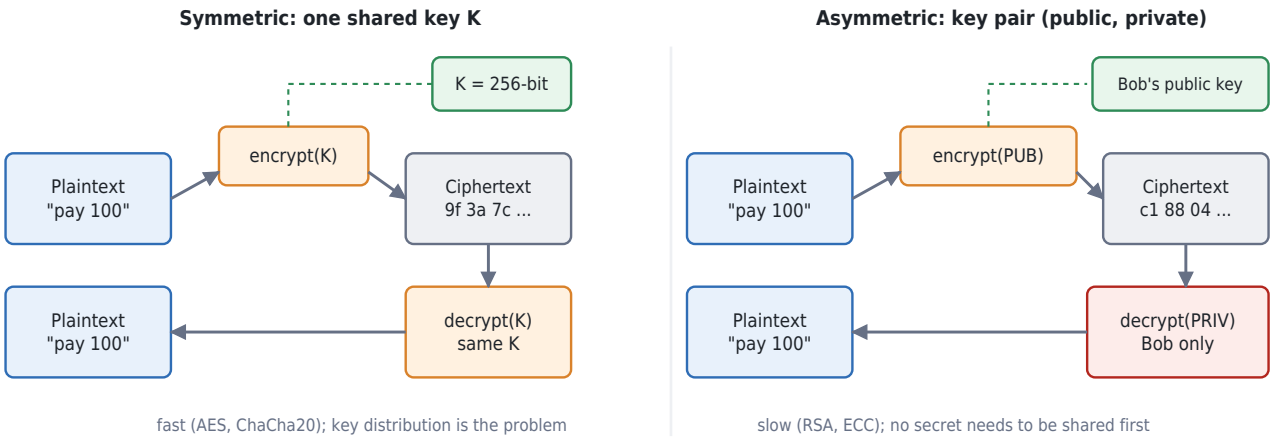


Figure 5.1. Symmetric encryption (left) uses one key at both ends, whereas asymmetric encryption (right) encrypts with a public key that anyone may hold and decrypts with a private key that never leaves its owner.

Asymmetric cryptography

An asymmetric (public-key) scheme uses a key pair. Data encrypted with the public key can be decrypted only with the private key, and a signature produced with the private key can be verified with the public key. The mathematics (integer factorisation for RSA, discrete logarithms on elliptic curves for ECDSA, EdDSA and ECDH) makes derivation of the private key from the public key computationally infeasible. Public-key operations are roughly two to three orders of magnitude slower than AES and are therefore used only for key exchange and signatures, never for bulk data.

Security strength	Symmetric	RSA modulus	Elliptic curve
112 bits	3DES (legacy)	2048 bits	224 bits
128 bits	AES-128	3072 bits	256 bits (P-256, X25519)
192 bits	AES-192	7680 bits	384 bits (P-384)
256 bits	AES-256	15360 bits	521 bits (P-521)

Approximate key sizes for equal security strength (NIST SP 800-57 style comparison).

Elliptic-curve keys are much shorter for the same strength, which is why TLS certificates with ECDSA P-256 keys produce smaller handshakes than 3072-bit RSA keys.

Key exchange: Diffie-Hellman

Diffie-Hellman (DH) allows two parties to derive a shared secret over an open channel. Both agree on public parameters: a prime p and a generator g. Each party picks a private number (a and b), publishes g to the power of that number modulo p (A and B), and raises the received value to its own private number. Both arrive at g^(ab) mod p. An observer sees p, g, A and B but must solve the discrete logarithm problem to recover a or b, which is infeasible for parameters of 2048 bits (finite-field DH) or for a 255-bit elliptic curve such as X25519 (ECDH).

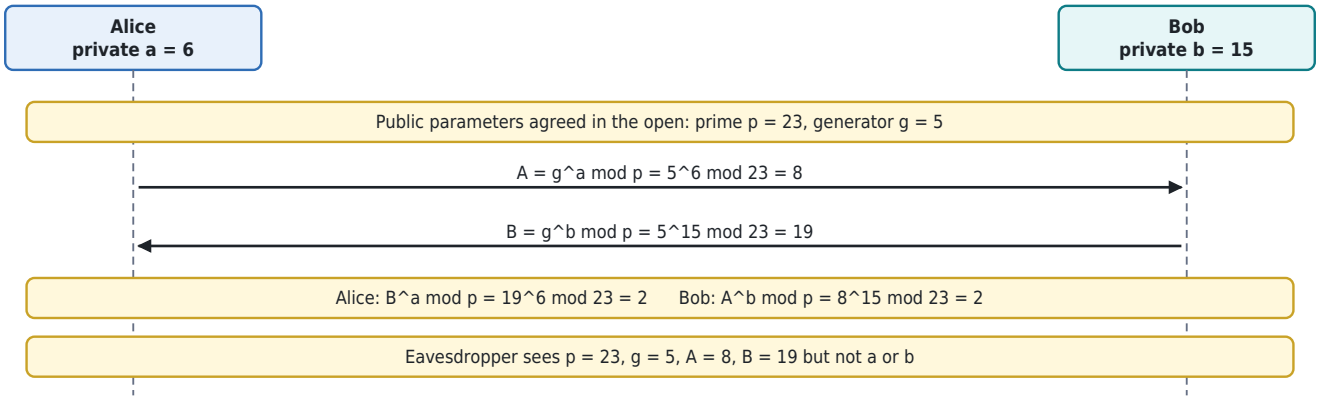


Figure 5.2. Diffie-Hellman with the toy parameters p = 23 and g = 5: only the values 8 and 19 cross the network, yet both parties compute the shared secret 2.

DH has two properties that TLS relies on. First, when the private numbers are generated fresh for each connection (**ephemeral** DH, written DHE or ECDHE), compromise of a long-term key later does not reveal the session keys. This

property is **forward secrecy**. Second, plain DH is not authenticated: an active attacker can run one exchange with each party and relay traffic between them (a man-in-the-middle attack). DH must therefore be combined with a signature or certificate that proves who sent each public value.

Hash functions, MAC and HMAC

A cryptographic hash function maps input of any length to a fixed-length digest (SHA-256: 32 bytes). Required properties: preimage resistance (the input cannot be recovered from the digest), second-preimage resistance and collision resistance. A one-bit change in the input changes about half of the output bits. MD5 and SHA-1 have practical collision attacks and are not acceptable for signatures or certificates. SHA-256 and SHA-384 are used throughout TLS.

A hash alone gives no authentication, because an attacker can recompute it after modifying the message. A **message authentication code** (MAC) mixes a secret key into the computation. **HMAC** (RFC 2104) builds a MAC from a hash function: HMAC(K, m) = H((K xor opad) || H((K xor ipad) || m)). Only holders of K can produce or verify the value. HMAC-SHA256 is used in TLS 1.3 for the Finished message and in key derivation (HKDF), and in JWT for the HS256 algorithm.

Digital signatures

A signature binds a message to the holder of a private key. The signer hashes the message and applies the private-key operation to the digest. The verifier hashes the received message, applies the public-key operation to the signature and compares the two digests. Verification succeeds only if the message is unchanged and the signature was produced by the matching private key. Common algorithms: RSA-PSS and RSA PKCS#1 v1.5, ECDSA, and EdDSA (Ed25519).

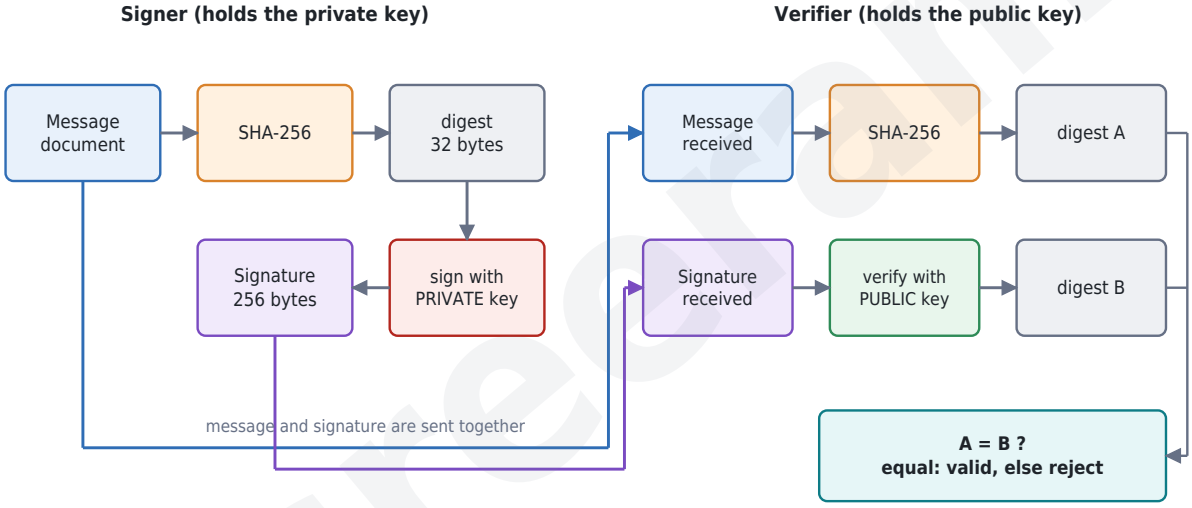


Figure 5.3. A signature is computed over a digest of the message with the private key; the verifier recomputes the digest and compares it with the value recovered from the signature using the public key.

Certificates and the chain of trust

A public key is useful for authentication only when the verifier knows whom it belongs to. An **X.509 certificate** (RFC 5280) binds a public key to an identity, expressed for web servers as one or more DNS names in the Subject Alternative Name (SAN) extension. The binding is asserted by a **certificate authority** (CA), which signs the certificate with its own private key.

Clients do not trust individual servers; they trust a small set of root CAs stored in a **truststore** (operating system, browser or the Java `cacerts` file). Roots rarely sign server certificates directly. A root signs **intermediate CA** certificates, and an intermediate signs the **leaf** certificate of the server. The server sends the leaf and the intermediates; the client builds a path from the leaf to a root in its truststore and verifies each signature along the way.

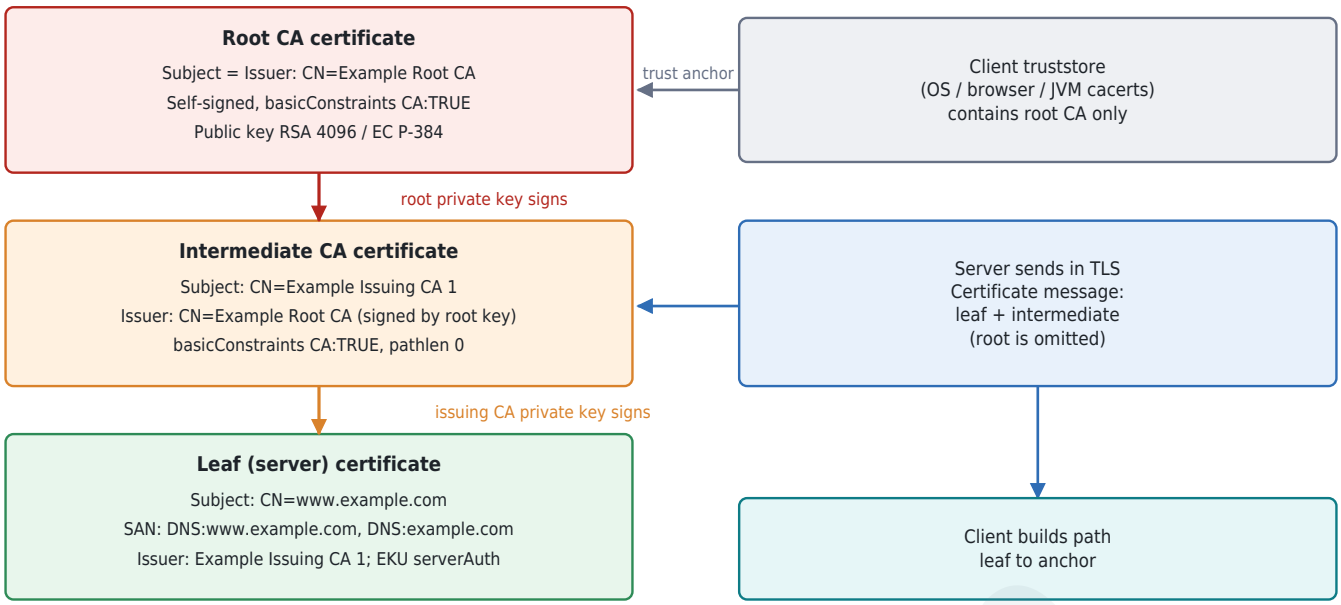


Figure 5.4. A leaf certificate is trusted because its signature verifies against an intermediate, whose signature verifies against a root already present in the client truststore; the root is not transmitted.

Further mechanisms complement the chain. **Revocation** announces certificates that must no longer be trusted before their expiry: Certificate Revocation Lists (CRL), the Online Certificate Status Protocol (OCSP) and OCSP stapling, in which the server attaches a recent signed OCSP response to the handshake. **Certificate Transparency** logs every publicly trusted certificate in append-only public logs so that misissuance can be detected. Validity periods are shrinking: under CA/Browser Forum Ballot SC-081 the maximum validity of publicly trusted TLS certificates fell to 200 days on 15 March 2026 and is scheduled to fall to 100 days in March 2027 and 47 days in March 2029, which makes automated renewal (ACME, for example with Let's Encrypt) mandatory in practice.

How the primitives compose in TLS

Primitive	Used for	Example in TLS 1.3
Ephemeral key exchange	Agree on a fresh shared secret	X25519 (ECDHE)
Signature	Prove that the server owns the certificate key	RSA-PSS or ECDSA in CertificateVerify
Certificate	Bind the server name to its public key	X.509 chain, validated against the truststore
Hash and HMAC	Transcript hash, Finished, key derivation (HKDF)	SHA-256 or SHA-384
AEAD cipher	Encrypt and authenticate every record	AES-GCM, ChaCha20-Poly1305

Role of each primitive in a TLS 1.3 connection.

The design follows from the cost differences: slow public-key operations occur once per connection to authenticate the peer and to agree on a secret; the cheap symmetric cipher then protects all data with keys derived from that secret.

5.1.2 Worked trace

The exchange of the diagram above, with every step computed. Parameters: p = 23, g = 5. Alice chooses a = 6, Bob chooses b = 15.

1. Alice computes A = 5^6 mod 23. 5^2 = 25 = 2 (mod 23); 5^4 = 4; 5^6 = 5^4 x 5^2 = 4 x 2 = 8. A = 8 is sent to Bob.
2. Bob computes B = 5^15 mod 23 = 19 and sends it to Alice.
3. Alice computes B^a mod p = 19^6 mod 23 = 2.
4. Bob computes A^b mod p = 8^15 mod 23 = 2.
5. Both hold the shared secret 2 = 5^(6 x 15) mod 23 = 5^90 mod 23. The value 2 is never transmitted.

The attacker knowledge is p, g, A and B. With p = 23 an exhaustive search over exponents succeeds after at most 22 trials; real deployments use 2048-bit primes or 255-bit curve points, for which the search space is not feasible. The program below performs the computation with `BigInteger.modPow` and also runs the brute-force search.

Dh.java JAVA

```java
import java.math.BigInteger;

public class Dh {
    public static void main(String[] args) {
        BigInteger p = BigInteger.valueOf(23);
        BigInteger g = BigInteger.valueOf(5);
        BigInteger a = BigInteger.valueOf(6);   // Alice private
        BigInteger b = BigInteger.valueOf(15);  // Bob private
        BigInteger A = g.modPow(a, p);
        BigInteger B = g.modPow(b, p);
        System.out.println("public p=" + p + " g=" + g);
        System.out.println("Alice sends A = g^a mod p = 5^6 mod 23 = " + A);
        System.out.println("Bob sends   B = g^b mod p = 5^15 mod 23 = " + B);
        System.out.println("Alice computes B^a mod p = " + B.modPow(a, p));
        System.out.println("Bob computes   A^b mod p = " + A.modPow(b, p));
        for (int x = 1; x < 23; x++) {
            if (g.modPow(BigInteger.valueOf(x), p).equals(A)) {
                System.out.println("Eavesdropper finds a=" + x
                        + " by exhaustive search (22 tries at most)");
                break;
            }
        }
    }
}
```

Output of Dh.java TERMINAL

```
$ java Dh.java
public p=23 g=5
Alice sends A = g^a mod p = 5^6 mod 23 = 8
Bob sends   B = g^b mod p = 5^15 mod 23 = 19
Alice computes B^a mod p = 2
Bob computes   A^b mod p = 2
Eavesdropper finds a=6 by exhaustive search (22 tries at most)
```

5.1.3 Hands-on

Hashing and HMAC in Java

The program hashes two messages that differ in a single character, then computes an HMAC of the first message under two different keys. The digests of the two messages share no visible structure, and the HMAC depends on the key.

Hm.java JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.HexFormat;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class Hm {
    public static void main(String[] args) throws Exception {
        byte[] msg = "amount=100&to=acct42".getBytes(StandardCharsets.UTF_8);
        byte[] msg2 = "amount=900&to=acct42".getBytes(StandardCharsets.UTF_8);
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        System.out.println("SHA-256(m1) = " + HexFormat.of().formatHex(md.digest(msg)));
        System.out.println("SHA-256(m2) = " + HexFormat.of().formatHex(md.digest(msg2)));
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec("shared-secret".getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        System.out.println("HMAC(m1)    = " + HexFormat.of().formatHex(mac.doFinal(msg)));
        mac.init(new SecretKeySpec("other-secret".getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        System.out.println("HMAC(m1,k2) = " + HexFormat.of().formatHex(mac.doFinal(msg)));
    }
}
```

Output of Hm.java TERMINAL

```
$ java Hm.java
SHA-256(m1) = 9baea67157754c47c0a5a0261dd98ebdcf218c01dbb36858d81b45d791d5b32d
SHA-256(m2) = 00b174043ba58171e6a65bfe34b9934a971a6e2b2073f1978c1c8a0e1ce51bb8
HMAC(m1)    = f0314ad7f62d17492d6afc70708b20919f9f718daec839592ddcf877d33f87cc
HMAC(m1,k2) = 358aeca2735b9b4229e7c088195207fc122309f3d607c7aaba9f498eb8027d92
```

The `openssl` command line gives the same functions without code. The first command is a plain SHA-256 of the string `hello`; the second is its HMAC under a key.

Hash and HMAC with openssl TERMINAL

```
$ echo -n "hello" | openssl dgst -sha256
SHA2-256(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
$ echo -n "hello" | openssl dgst -sha256 -hmac "shared-secret"
SHA2-256(stdin)= dad60f181b1f0d72f63b2dff4e9aa1de277989cb1e08a8b0e64e43f774ac036e
```

Signatures and key agreement with the JDK

The `java.security.Signature` class implements signing and verification, and `javax.crypto.KeyAgreement` implements DH and ECDH variants, including X25519 (available since Java 11).

Sg.java JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.MessageDigest;
import java.security.Signature;
import javax.crypto.KeyAgreement;

public class Sg {
    public static void main(String[] args) throws Exception {
        KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
        kpg.initialize(2048);
        KeyPair kp = kpg.generateKeyPair();
        byte[] m = "hello".getBytes(StandardCharsets.UTF_8);
        Signature s = Signature.getInstance("SHA256withRSA");
        s.initSign(kp.getPrivate());
        s.update(m);
        byte[] sig = s.sign();
        System.out.println("signature length bytes = " + sig.length);
        s.initVerify(kp.getPublic());
        s.update(m);
        System.out.println("verify original = " + s.verify(sig));
        s.initVerify(kp.getPublic());
        s.update("hellp".getBytes(StandardCharsets.UTF_8));
        System.out.println("verify altered  = " + s.verify(sig));

        KeyPairGenerator ec = KeyPairGenerator.getInstance("X25519");
        KeyPair a = ec.generateKeyPair();
        KeyPair b = ec.generateKeyPair();
        KeyAgreement ka = KeyAgreement.getInstance("X25519");
        ka.init(a.getPrivate());
        ka.doPhase(b.getPublic(), true);
        byte[] s1 = ka.generateSecret();
        KeyAgreement kb = KeyAgreement.getInstance("X25519");
        kb.init(b.getPrivate());
        kb.doPhase(a.getPublic(), true);
        byte[] s2 = kb.generateSecret();
        System.out.println("X25519 secrets equal = " + MessageDigest.isEqual(s1, s2)
                + ", length " + s1.length);
    }
}
```

Output of Sg.java TERMINAL

```
$ java Sg.java
signature length bytes = 256
verify original = true
verify altered  = false
X25519 secrets equal = true, length 32
```

The RSA signature is 256 bytes because it has the size of the 2048-bit modulus. Changing one character of the message invalidates it. Each X25519 shared secret is 32 bytes and identical at both ends. The method `MessageDigest.isEqual` compares in constant time and is the correct way to compare secrets and MACs.

Inspecting a certificate

A self-signed certificate for `localhost` was generated with `openssl req -x509 -newkey rsa:2048 -nodes -days 30 -subj "/CN=localhost/O=Example Lab" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"`. The relevant fields of `openssl x509 -in cert.pem -noout -text` are shown below (serial number, modulus and extension bytes omitted).

Fields of an X.509 certificate TERMINAL

```
$ openssl x509 -in cert.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = localhost, O = Example Lab
        Validity
            Not Before: Oct 17 16:48:15 2026 GMT
            Not After : Nov 16 16:48:15 2026 GMT
        Subject: CN = localhost, O = Example Lab
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints: critical
                CA:TRUE
            X509v3 Subject Alternative Name:
                DNS:localhost, IP Address:127.0.0.1
```

Issuer and Subject are identical because the certificate signs itself; it is its own trust anchor, which is why a client rejects it unless the file is placed explicitly in its truststore.

5.1.4 Failure modes

Reusing a nonce with an AEAD cipher

GCM and ChaCha20-Poly1305 are secure only if a (key, nonce) pair is never used twice. With a repeated nonce the same key stream encrypts both messages, so the XOR of two ciphertexts equals the XOR of the two plaintexts. The program below encrypts two 23-byte messages with AES-GCM under one key and one nonce.

Nonce.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.util.HexFormat;
import javax.crypto.Cipher;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;

public class Nonce {
    static byte[] enc(byte[] key, byte[] iv, String msg) throws Exception {
        Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
        c.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"), new GCMParameterSpec(128, iv));
        return c.doFinal(msg.getBytes(StandardCharsets.UTF_8));
    }

    public static void main(String[] args) throws Exception {
        byte[] key = new byte[16];
        for (int i = 0; i < 16; i++) key[i] = (byte) (i * 7 + 1);
        byte[] iv = new byte[12];       // the same 96-bit nonce used twice: a protocol bug
        String m1 = "transfer 100 to acct 42";
        String m2 = "transfer 900 to acct 77";
        byte[] c1 = enc(key, iv, m1), c2 = enc(key, iv, m2);
        System.out.println("c1 = " + HexFormat.of().formatHex(c1, 0, m1.length()));
        System.out.println("c2 = " + HexFormat.of().formatHex(c2, 0, m2.length()));
        StringBuilder x = new StringBuilder();
        for (int i = 0; i < m1.length(); i++) {
            int cx = (c1[i] ^ c2[i]) & 0xFF, px = (m1.charAt(i) ^ m2.charAt(i)) & 0xFF;
            if (cx != px) throw new IllegalStateException("mismatch at " + i);
            x.append(String.format("%02x", cx));
        }
        System.out.println("c1 xor c2 = " + x);
```

Nonce.java (continued) JAVA

```
        System.out.println("m1 xor m2 equals c1 xor c2 for all " + m1.length()
                + " bytes: the key stream cancelled");
    }
}
```

Output of Nonce.java TERMINAL

```
$ java Nonce.java
c1 = 502ef72886d280a46c507230b64c8dcd5e54839b73f421
c2 = 502ef72886d280a46c587230b64c8dcd5e54839b73f724
c1 xor c2 = 0000000000000000000080000000000000000000000305
m1 xor m2 equals c1 xor c2 for all 23 bytes: the key stream cancelled
```

The XOR is zero wherever the plaintexts agree, and it exposes exactly where they differ (the digits 1 and 9, and 4 and 7). In practice the attacker learns the XOR of the plaintexts and, with GCM, can additionally forge tags. TLS avoids the problem by deriving the nonce from a per-record sequence number.

Other recurring errors

- **Using a hash for passwords or as a MAC.** Fast hashes allow billions of guesses per second. Passwords require a deliberately slow, salted function (bcrypt, scrypt, Argon2, PBKDF2). Message authentication requires HMAC, not `H(key || message)`.
- **Unauthenticated key exchange.** DH without a signed or certified public value is open to man-in-the-middle attacks.
- **Weak randomness.** Keys, nonces and session identifiers must come from `java.security.SecureRandom`, never from `java.util.Random`.
- **Non-constant-time comparison.** Comparing MACs or tokens with `String.equals` can leak the matching prefix length through timing. Use `MessageDigest.isEqual`.
- **Obsolete algorithms.** MD5, SHA-1, RC4, DES, 3DES, RSA key exchange without forward secrecy, and RSA keys below 2048 bits.
- **Custom constructions.** Combining primitives manually (for example, encrypt without authentication in ECB or CBC mode) has repeatedly produced vulnerabilities; protocols such as TLS should be used instead.

5.1.5 Questions to expect

1. What is the difference between symmetric and asymmetric encryption, and why does TLS use both?

Symmetric encryption uses one shared key and is fast; asymmetric encryption uses a key pair and is slow but needs no pre-shared secret. TLS uses asymmetric operations (key exchange and signatures) once per connection to authenticate the server and to agree on a secret, and then uses symmetric AEAD encryption with keys derived from that secret for all application data.

2. How does Diffie-Hellman produce a shared secret without sending it?

Each side combines its private exponent with the peer public value: (g^b)^a = (g^a)^b = g^(ab) mod p. An observer has only g^a and g^b and would have to solve the discrete logarithm problem. The exchange is unauthenticated, so TLS signs the server public value (or the handshake transcript) with the certificate key.

3. What is forward secrecy and which cipher suites provide it?

Forward secrecy means that recorded traffic stays confidential even if the server long-term private key is compromised later. It requires ephemeral key exchange (DHE or ECDHE), because the session key then depends on values that were discarded after the handshake. TLS 1.3 mandates ephemeral exchange for every connection; in TLS 1.2 it is provided by suites beginning with ECDHE or DHE and absent from static RSA key exchange.

4. How does a hash differ from a MAC and from a digital signature?

A hash has no key and gives no authentication. A MAC (HMAC) uses a shared secret key, so only key holders can create or verify it; it provides integrity and authentication between two parties but not non-repudiation. A signature uses a private key and is verified with the public key, so anyone can verify it and only the key holder could have produced it.

sreram

5. How does a client decide to trust a server certificate?

It builds a chain from the leaf through the intermediates the server sent to a root in its truststore, verifies each signature, checks validity dates, checks that the requested host name matches a SAN entry, checks CA and key-usage constraints, and (browsers) checks revocation status. The server key must also prove possession in the handshake by signing the transcript.

6. Why can a self-signed certificate not authenticate a server by itself?

Anyone can create a certificate that names any host. Trust originates only from a root or from a certificate that was installed explicitly in the truststore. A self-signed certificate is acceptable when it is distributed through a trusted channel, for example inside a private deployment, and not for public Internet services.

7. Why must a nonce never be reused with AES-GCM?

Reuse repeats the key stream, so the XOR of two ciphertexts equals the XOR of the plaintexts, and the authentication key of GCM can be recovered, which allows forgery. TLS derives each nonce from a record sequence number combined with a per-connection IV, so repetition cannot occur within a connection.

8. Why are certificate lifetimes being shortened?

Revocation is unreliable in practice, so a shorter lifetime bounds the exposure after a key compromise or a misissuance. The cost is that renewal must be automated, typically with the ACME protocol.

KEY POINTS

- Confidentiality, integrity and authentication are obtained by composing a key exchange, signatures, certificates, hashes or MACs, and an AEAD cipher.
- Symmetric ciphers protect bulk data; asymmetric operations occur once per connection for key agreement and authentication.
- Ephemeral Diffie-Hellman (ECDHE, X25519) provides forward secrecy; unauthenticated DH is vulnerable to man-in-the-middle attacks.
- HMAC provides keyed integrity; a signature provides integrity and origin proof verifiable by anyone with the public key.
- A certificate chain ends at a root in the client truststore; the leaf name is matched against the SAN extension.
- Never reuse an AEAD nonce, never use a plain hash as a MAC or password store, and compare secrets in constant time.

5.2 TLS and HTTPS

HTTP as defined in the earlier Part transmits requests and responses in clear text. Every router on the path can read cookies, tokens and form data, and can modify a response before it reaches the browser. **Transport Layer Security** (TLS) inserts a protected channel between TCP and the application protocol. **HTTPS** is HTTP carried inside a TLS connection, normally on TCP port 443. The same mechanism protects SMTP, IMAP, database connections, gRPC and most service-to-service traffic. TLS provides the three properties of the previous chapter: the server (and optionally the client) is authenticated by a certificate, the data is encrypted, and each record carries an authentication tag that detects modification.

Versions in use: SSL 2.0 and 3.0 are obsolete; TLS 1.0 and 1.1 were deprecated by RFC 8996 (March 2021); TLS 1.2 (RFC 5246) remains widely deployed; TLS 1.3 (RFC 8446, since consolidated into RFC 9846) is the current version. Java 21 enables TLS 1.3 and TLS 1.2 by default for client and server sockets.

5.2.1 Concept and mechanism

Where TLS sits and what the handshake achieves

TLS runs after the TCP three-way handshake. A **handshake** of one or two round trips negotiates the protocol version and cipher suite, authenticates the server, agrees on shared secrets by ephemeral Diffie-Hellman, and derives symmetric keys. Afterwards the **record layer** fragments application data into records of at most 16,384 bytes of plaintext, encrypts each with an AEAD cipher and sends it over TCP.

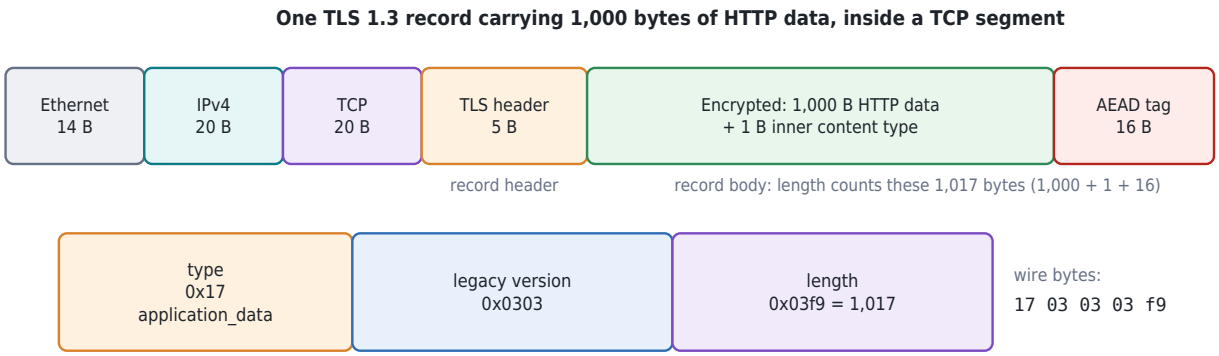


Figure 5.5. A TLS 1.3 record consists of a five-byte header (content type 0x17, legacy version 0x0303 and a length) followed by the encrypted fragment and a 16-byte authentication tag; the length field counts everything after the header.

In TLS 1.3 the visible content type of an encrypted record is always 0x17 (application_data), and the real type (handshake, alert or application data) is hidden as the last byte of the encrypted fragment. Observers on the path therefore cannot distinguish handshake records from data records after the ServerHello.

The TLS 1.2 handshake

The full TLS 1.2 handshake needs two round trips before application data can flow. The messages of a handshake with ECDHE key exchange and an RSA certificate are drawn below, with the message lengths printed by `openssl s_client -msg` against a local server (length includes the four-byte handshake header).

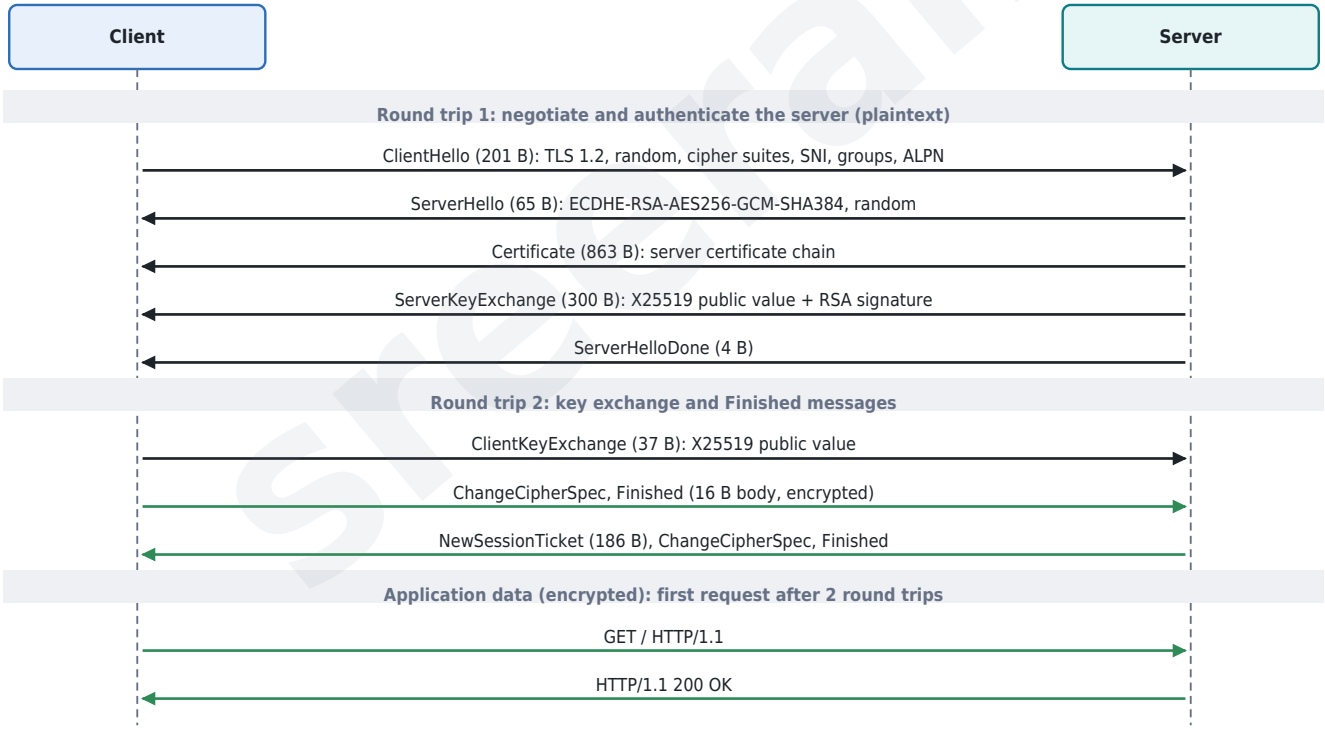


Figure 5.6. In TLS 1.2 the certificate and the key exchange values travel in plaintext and the first HTTP request is sent only after the second round trip; encrypted messages are drawn in green.

1. **ClientHello** lists the highest protocol version, a 32-byte client random, the supported cipher suites, supported groups (for example X25519, secp256r1), signature algorithms and extensions: Server Name Indication (SNI), ALPN, session ticket.
2. **ServerHello** selects the version and one cipher suite (here `ECDHE-RSA-AES256-GCM-SHA384`) and returns the server random.
3. **Certificate** carries the leaf and intermediate certificates.
4. **ServerKeyExchange** carries the server ephemeral ECDH public value, signed with the private key of the certificate. The signature covers both randoms, so it cannot be replayed from another session.
5. **ServerHelloDone** ends the server flight.

6. **ClientKeyExchange** carries the client ephemeral public value. Both sides compute the same pre-master secret and derive a master secret and the traffic keys from it and from the two randoms.
7. **ChangeCipherSpec** and **Finished** (each side): from this point records are encrypted; Finished contains a MAC over the whole handshake transcript, which detects any tampering with the negotiation (such as a downgrade of the cipher suite list).

The TLS 1.3 handshake

TLS 1.3 removes a round trip by sending the client key share in the first message. The client guesses a supported group (nearly always X25519) and includes its public value in the ClientHello. The server answers with its own key share in the ServerHello, after which both sides hold the handshake secret and everything that follows is encrypted, including the server certificate.



Figure 5.7. In TLS 1.3 only the ClientHello and ServerHello are plaintext; the certificate, its signature and both Finished messages are encrypted, and the client can send its HTTP request after one round trip.

If the server does not support the offered group it replies with a HelloRetryRequest, which costs one additional round trip. The CertificateVerify message is a signature over the hash of the transcript so far, which proves possession of the private key matching the certificate and binds the authentication to this specific handshake.

Aspect	TLS 1.2	TLS 1.3
Round trips before data	2 (full handshake)	1 (full), 0 (resumption with early data)
Key exchange	RSA key transport, static DH or ephemeral DH	Ephemeral (EC)DHE only: forward secrecy always
Certificate visibility	Plaintext	Encrypted
Cipher suites	Many combinations, including CBC and RC4	Five AEAD suites (AES-GCM, ChaCha20-Poly1305, AES-CCM)
Removed features	none	Renegotiation, compression, static RSA, MD5, SHA-1 signatures in handshake
Handshake integrity	Finished MAC	Finished MAC plus transcript-bound CertificateVerify

Main differences between TLS 1.2 and TLS 1.3.

Server Name Indication and ALPN

One IP address and port often serves many host names, so the server must choose the certificate before it has read the HTTP `Host` header, which is still encrypted. The **SNI** extension (RFC 6066) carries the requested host name in the ClientHello in plaintext, which lets a load balancer or reverse proxy select the matching certificate and route the connection. Because SNI is visible to the network, a passive observer learns which site is contacted even though the content is encrypted; Encrypted Client Hello (ECH) is the mechanism that encrypts it where deployed.

Application-Layer Protocol Negotiation (ALPN, RFC 7301) lets client and server agree on the protocol that will run inside the tunnel during the handshake, without an additional round trip. A browser offers `h2` and `http/1.1`; the server selects one. This is how HTTP/2 is negotiated over TLS. HTTP/3 uses its own QUIC transport and is advertised differently (the `Alt-Svc` header or DNS records).

Certificate validation

The client validates the certificate chain with the steps shown below. Failure at any step aborts the handshake with an alert, and in Java produces a `javax.net.ssl.SSLHandshakeException` caused by a `java.security.cert.CertPathValidatorException` or a `SunCertPathBuilderException`.

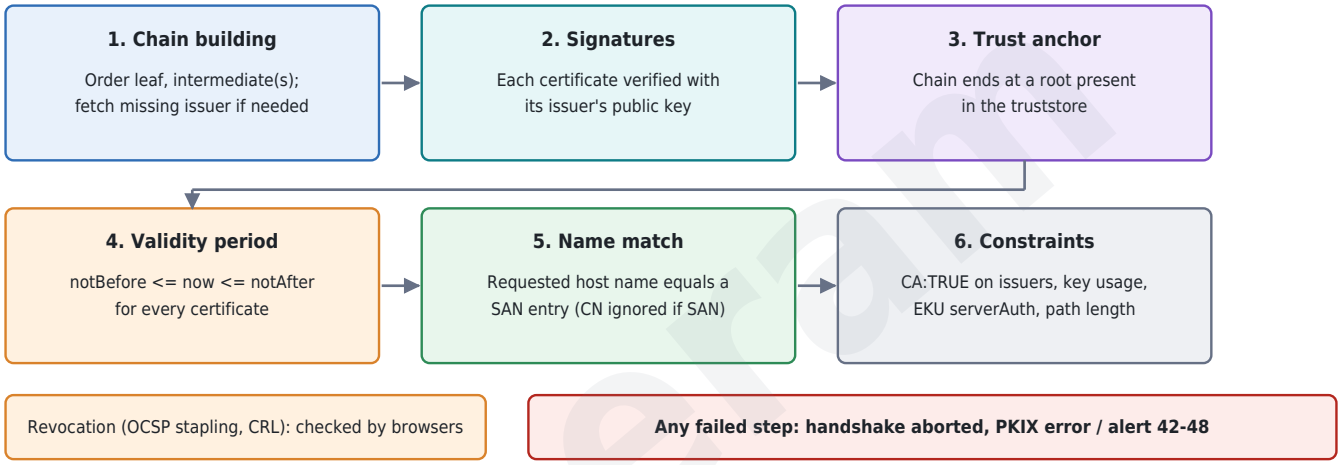


Figure 5.8. Certificate validation proceeds from chain building through signatures, trust anchor, validity period, name match and constraints; the host name check ties the certificate to the name the user typed.

The host name check follows RFC 9525 (which obsoletes RFC 6125): the requested name must equal a `dNSName` entry of the SAN extension, where a wildcard such as `*.example.com` matches exactly one label (`api.example.com` but not `a.b.example.com`). The Common Name is not consulted when a SAN is present. For IP address URLs the certificate needs an `iPAddress` SAN entry.

Session resumption and 0-RTT

A repeat connection to the same server can skip certificate transfer and public-key operations. In TLS 1.2 this is done by session IDs or session tickets; in TLS 1.3 by a **pre-shared key** derived from the previous connection and delivered in a NewSessionTicket message. The client presents the ticket in the ClientHello and the server replies with a fresh ECDHE share, so forward secrecy is retained.

TLS 1.3 additionally allows the client to send application data immediately after the ClientHello (**0-RTT early data**), encrypted under a key derived from the ticket. RFC 8446 section 2.3 states that the security properties of this data are weaker: there is no guarantee of non-replay between connections, and the data is not forward secret. An attacker who records the first flight can resend it to the server, or to another server in the same cluster, and the request may be executed twice.

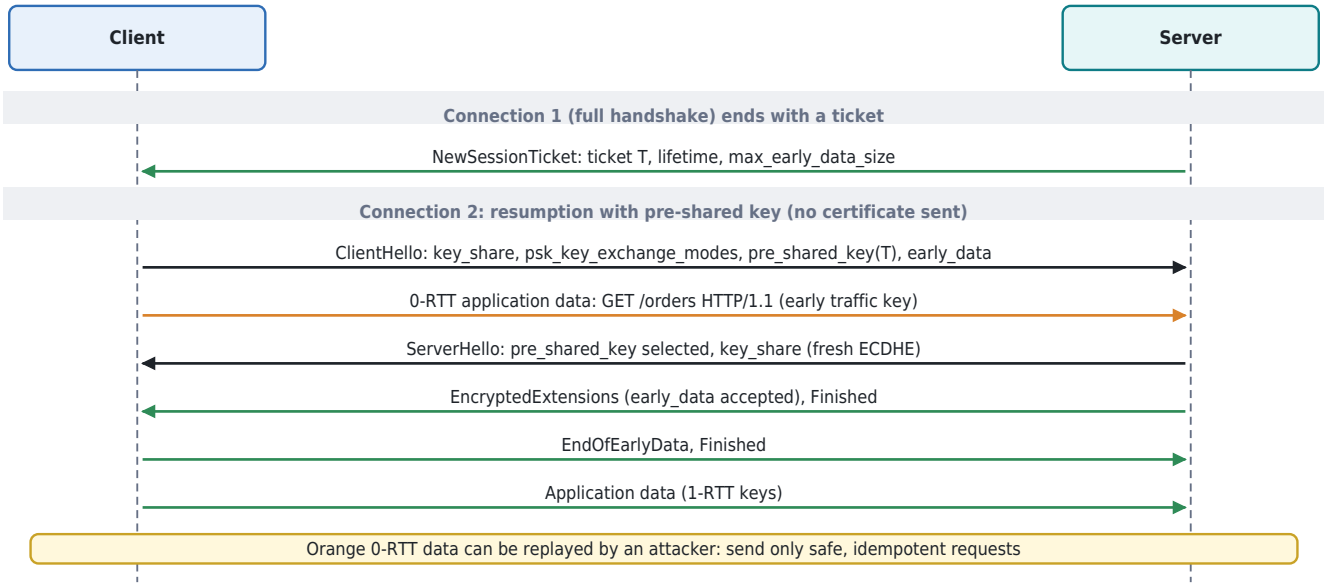


Figure 5.9. In a resumed TLS 1.3 connection no certificate is sent, and the HTTP request travels in the very first flight as 0-RTT data (orange), which is replayable and must be limited to safe, idempotent requests.

Servers that enable early data should accept it only for idempotent requests (`GET` that does not change state) and reject or delay anything else; CDNs and reverse proxies commonly add an `Early-Data: 1` header (RFC 8470) to the request so that the application can answer `425 Too Early`.

Mutual TLS

In ordinary TLS only the server is authenticated. In **mutual TLS** (mTLS) the server also sends a CertificateRequest and the client presents its own certificate and a CertificateVerify signature. mTLS is used between services inside a cluster (identity of the calling service), by API partners and in zero-trust designs. Each side needs a keystore (own key and certificate) and a truststore (CAs it accepts).

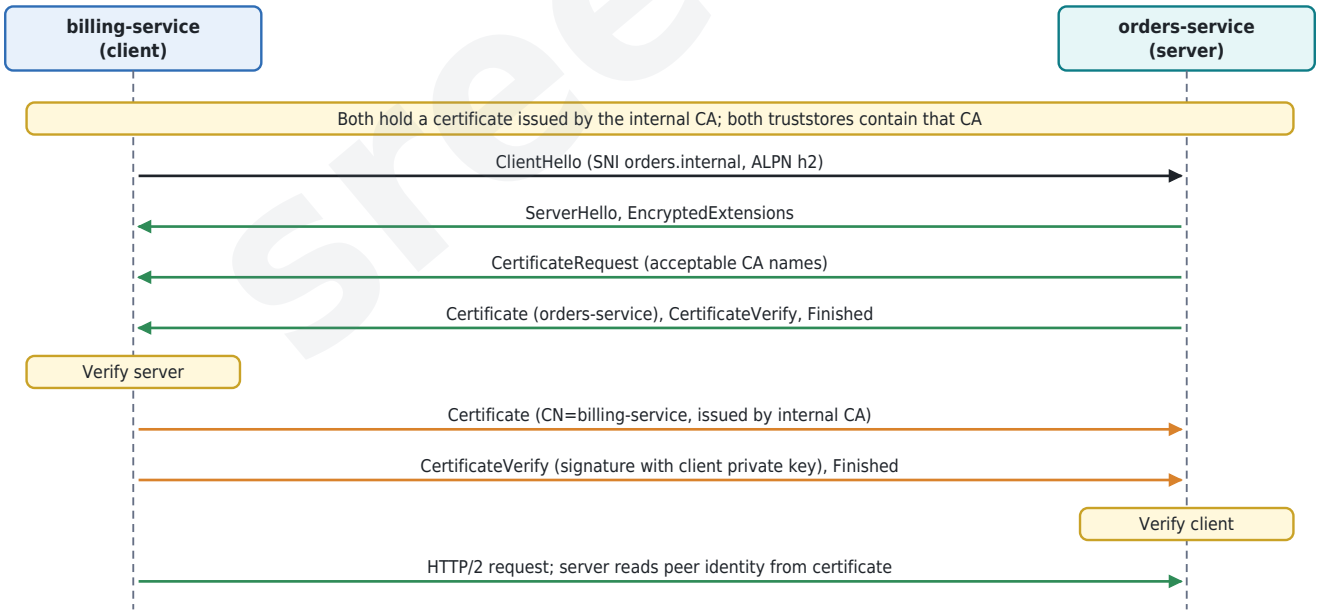


Figure 5.10. Mutual TLS adds a CertificateRequest from the server and a client Certificate and CertificateVerify (orange); both services verify the peer certificate against a truststore that contains the internal CA.

5.2.2 Worked trace

The server below is `openssl s_server` with the self-signed certificate of the previous chapter (CN=localhost, RSA 2048, SAN DNS:localhost and IP:127.0.0.1) listening on the loopback interface. The client requests TLS 1.3, offers ALPN `http/1.1` and validates against the certificate file.

TLS 1.3 handshake against a local server TERMINAL

```
$ openssl s_server -accept 8443 -cert cert.pem -key key.pem -www -alpn http/1.1,h2 &
$ echo "GET / HTTP/1.0" | openssl s_client -connect 127.0.0.1:8443 -servername localhost \
    -CAfile cert.pem -alpn http/1.1 -verify_hostname localhost
depth=0 CN = localhost, O = Example Lab
verify return:1
CONNECTED(00000003)
---
Certificate chain
 0 s:CN = localhost, O = Example Lab
   i:CN = localhost, O = Example Lab
   a:PKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
   v:NotBefore: Oct 17 16:48:15 2026 GMT; NotAfter: Nov 16 16:48:15 2026 GMT
---
Server certificate
subject=CN = localhost, O = Example Lab
issuer=CN = localhost, O = Example Lab
---
Peer signing digest: SHA256
Peer signature type: RSA-PSS
Server Temp Key: X25519, 253 bits
---
SSL handshake has read 1424 bytes and written 406 bytes
Verification: OK
Verified peername: localhost
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Compression: NONE
ALPN protocol: http/1.1
Early data was not sent
Verify return code: 0 (ok)
```

Reading the output field by field:

1. **Certificate chain** contains one certificate whose subject and issuer are identical (self-signed); `Verify return code: 0 (ok)` is reported only because `-CAfile cert.pem` placed that certificate in the truststore.
2. **Peer signature type RSA-PSS** with digest SHA256 is the CertificateVerify signature produced by the server RSA key.
3. **Server Temp Key X25519, 253 bits** is the ephemeral ECDHE key that gives forward secrecy. The certificate key is used only for the signature, never for key transport.
4. **TLSv1.3, TLS_AES_256_GCM_SHA384** is the negotiated version and cipher suite: AES-256 in GCM mode for records, SHA-384 for HKDF and the transcript hash.
5. **ALPN protocol http/1.1** shows the negotiated application protocol. When the same server is started with `-alpn h2,http/1.1` and the client offers `h2,http/1.1` the result is `ALPN protocol: h2`; offering only `spdy/3` yields `No ALPN negotiated`.
6. **Handshake bytes**: 1,424 bytes read and 406 written. The server flight is dominated by the 853-byte DER certificate, which is why large certificate chains cost real time on lossy links.

The same server with a forced protocol version gives the comparison below (measured on the build machine; the byte counts depend on certificate size and extensions).

Protocol	Cipher suite	Bytes read	Bytes written	Messages in the server flight
TLS 1.2	ECDHE-RSA-AES256-GCM-SHA384	1,494	281	ServerHello, Certificate, ServerKeyExchange, ServerHelloDone
TLS 1.3	TLS_AES_256_GCM_SHA384	1,409	305	ServerHello, EncryptedExtensions, Certificate, CertificateVerify, Finished
TLS 1.2 resumed (ticket)	ECDHE-RSA-AES256-GCM-SHA384	141	447	ServerHello, Finished (no certificate)

Handshake with the same certificate, forced protocol versions (openssl 3.0.13, loopback).

The resumed TLS 1.2 handshake read only 141 bytes because no certificate and no key-exchange message was transmitted. `openssl s_client -tls1_3 -sess_out file` followed by `-sess_in file` prints `Reused, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384` for TLS 1.3 in the same way.

Failure cases are produced by changing one parameter. Without `-CAfile` the client does not trust the certificate; with a wrong expected name the host name check fails; TLS 1.1 is not available in this build at all.

Three failures with openssl s_client TERMINAL

```
$ echo | openssl s_client -connect 127.0.0.1:8443 -servername localhost
verify error:num=18:self-signed certificate
Verification error: self-signed certificate
Verify return code: 18 (self-signed certificate)
$ echo | openssl s_client -connect 127.0.0.1:8443 -CAfile cert.pem -verify_hostname example.org
verify error:num=62:hostname mismatch
Verification error: hostname mismatch
Verify return code: 62 (hostname mismatch)
$ echo | openssl s_client -connect 127.0.0.1:8443 -tls1_1
error:0A0000BF:SSL routines:tls_setup_handshake:no protocols available
```

`openssl s_client` continues the handshake after the verification error and only reports it; browsers and Java clients abort. The `-msg` option prints every handshake message with its length, which is the source of the sizes in the diagrams (for TLS 1.3, ClientHello 238 bytes, ServerHello 122, EncryptedExtensions 6, Certificate 866, CertificateVerify 264 and two Finished messages of 52 bytes each).

5.2.3 Hands-on

Keystores and truststores in Java

Java keeps keys and certificates in a **keystore** file, normally PKCS12 (the default type since Java 9). A **keystore** holds the private key and certificate chain that the application presents; a **truststore** holds the certificates of the CAs or servers the application accepts. Technically both are the same file format; the roles differ. The JVM default truststore is `$JAVA_HOME/lib/security/cacerts`, replaceable with `-Djavax.net.ssl.trustStore=path` and `-Djavax.net.ssl.trustStorePassword=secret`.

Create a server keystore and a client truststore with keytool TERMINAL

```
$ keytool -genkeypair -alias server -keyalg RSA -keysize 2048 -validity 30 \
    -dname "CN=localhost,O=Example Lab" -ext san=dns:localhost,ip:127.0.0.1 \
    -keystore server.p12 -storetype PKCS12 -storepass changeit
Generating 2,048 bit RSA key pair and self-signed certificate (SHA384withRSA)
        with a validity of 30 days for: CN=localhost, O=Example Lab
$ keytool -exportcert -alias server -keystore server.p12 -storepass changeit -rfc -file server.crt
Certificate stored in file <server.crt>
$ keytool -importcert -noprompt -alias server -file server.crt -keystore trust.p12 \
    -storetype PKCS12 -storepass changeit
Certificate was added to keystore
$ keytool -list -keystore server.p12 -storepass changeit
Keystore type: PKCS12
Keystore provider: SUN

Your keystore contains 1 entry

server, Oct 17, 2026, PrivateKeyEntry,
Certificate fingerprint (SHA-256):
  39:7C:1D:E7:D5:C3:FE:FF:56:CC:EB:A2:7A:A5:E8:5A:6B:19:3C:20:8F:38:E8:34:76:7D:56:EE:20:83:7D:33
```

The next program starts a `com.sun.net.httpserver.HttpsServer` on `127.0.0.1:8444` using `server.p12` and calls it twice with `java.net.http.HttpClient`: first with the default truststore, then with an `SSLContext` built from `trust.p12`.

TlsDemo.java JAVA

```
import com.sun.net.httpserver.HttpsConfigurator;
import com.sun.net.httpserver.HttpsServer;
import java.io.FileInputStream;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
```

TlsDemo.java (continued) JAVA

```java
import java.security.KeyStore;
import javax.net.ssl.KeyManagerFactory;
import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManagerFactory;

public class TlsDemo {
    static SSLContext context(String keyStoreFile, String trustStoreFile) throws Exception {
        char[] pw = "changeit".toCharArray();
        KeyManagerFactory kmf = null;
        TrustManagerFactory tmf = null;
        if (keyStoreFile != null) {
            KeyStore ks = KeyStore.getInstance("PKCS12");
            try (var in = new FileInputStream(keyStoreFile)) { ks.load(in, pw); }
            kmf = KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
            kmf.init(ks, pw);
        }
        if (trustStoreFile != null) {
            KeyStore ts = KeyStore.getInstance("PKCS12");
            try (var in = new FileInputStream(trustStoreFile)) { ts.load(in, pw); }
            tmf = TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
            tmf.init(ts);
        }
        SSLContext c = SSLContext.getInstance("TLS");
        c.init(kmf == null ? null : kmf.getKeyManagers(),
               tmf == null ? null : tmf.getTrustManagers(), null);
        return c;
    }

    static void call(String name, SSLContext c) {
        try {
            HttpClient hc = HttpClient.newBuilder().sslContext(c).build();
            HttpRequest req = HttpRequest.newBuilder(
                    URI.create("https://localhost:8444/hi")).build();
            HttpResponse<String> r = hc.send(req, HttpResponse.BodyHandlers.ofString());
            var tls = r.sslSession().get();
            System.out.println(name + ": " + r.statusCode() + " " + r.body() + " via "
                    + tls.getProtocol() + " " + tls.getCipherSuite());
        } catch (Exception e) {
            Throwable t = e;
            while (t.getCause() != null) t = t.getCause();
            System.out.println(name + ": " + e.getClass().getSimpleName());
            System.out.println("  root cause: " + t.getClass().getName() + ": " + t.getMessage());
        }
    }

    public static void main(String[] args) throws Exception {
        HttpsServer s = HttpsServer.create(new InetSocketAddress("127.0.0.1", 8444), 0);
        s.setHttpsConfigurator(new HttpsConfigurator(context("server.p12", null)));
        s.createContext("/hi", ex -> {
            byte[] b = "hello over TLS".getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        s.start();
        call("default truststore", SSLContext.getDefault());
        call("custom truststore ", context(null, "trust.p12"));
        s.stop(0);
    }
}
```

Output of TlsDemo.java TERMINAL

```
$ java TlsDemo.java
default truststore: SSLHandshakeException
  root cause: sun.security.provider.certpath.SunCertPathBuilderException:
    unable to find valid certification path to requested target
custom truststore : 200 hello over TLS via TLSv1.3 TLS_AES_256_GCM_SHA384
```

What to notice. The first call fails with the characteristic **PKIX path building failed** message because the self-signed certificate does not chain to any root in `cacerts`. The second call succeeds because the `TrustManagerFactory` was initialised from a store that contains the server certificate. `HttpClient` negotiated TLS 1.3 without configuration. The

root cause is found by walking `getCause()`, since the visible exception message is the generic `PKIX path building failed`.

Spring Boot configuration

Spring Boot terminates TLS in the embedded server (Tomcat by default) when `server.ssl.*` is configured. Since Spring Boot 3.1 the recommended form uses **SSL bundles**, which also support reloading on file change. The following configuration is shown for reference and was not run here.

application.properties (not run) PROPERTIES

```
server.port=8443
server.ssl.bundle=web
spring.ssl.bundle.jks.web.key.alias=server
spring.ssl.bundle.jks.web.keystore.location=file:/etc/app/server.p12
spring.ssl.bundle.jks.web.keystore.password=${KEYSTORE_PASSWORD}
spring.ssl.bundle.jks.web.keystore.type=PKCS12
# require client certificates (mutual TLS): none, want or need
server.ssl.client-auth=need
spring.ssl.bundle.jks.web.truststore.location=file:/etc/app/trust.p12
spring.ssl.bundle.jks.web.truststore.password=${TRUSTSTORE_PASSWORD}
server.ssl.enabled-protocols=TLSv1.3,TLSv1.2
```

`server.ssl.client-auth=need` makes the server reject connections without a valid client certificate; `want` requests one but allows anonymous clients. For outgoing calls, `RestClient` and `WebClient` accept an `SslBundle` through `SslBundles`, and `java.net.http.HttpClient` takes an `SSLContext` as shown above. In most production deployments TLS terminates at a load balancer instead, covered later in this Part.

Mutual TLS with openssl and curl

The server below requires a client certificate (`-Verify 1`) issued by `ccert.pem`. The curl client is run once without and once with the client certificate.

Mutual TLS against openssl s_server TERMINAL

```
$ openssl s_server -accept 8445 -cert cert.pem -key key.pem -CAfile ccert.pem -Verify 1 -www &
$ curl -s --cacert cert.pem --cert ccert.pem --key ckey.pem https://localhost:8445/ \
    -o /dev/null -w "http=%{http_code}\n"
http=200
$ curl -sS --cacert cert.pem https://localhost:8445/
curl: (56) OpenSSL SSL_read: OpenSSL/3.0.13: error:0A00045C:SSL routines::
  tlsv13 alert certificate required, errno 0
```

In the second call the TLS 1.3 handshake appears to succeed from the client perspective: the client Finished is sent before the server has validated the (absent) certificate, so the rejection arrives later as an alert while the client reads the response. Applications therefore see a read error, not a connect error, when a client certificate is missing. The server log shows `peer did not return a certificate`.

Common PKIX and handshake errors

Symptom	Cause	Fix
PKIX path building failed: unable to find valid certification path	Issuer not in truststore; server omitted the intermediate certificate	Import the CA into the truststore; configure the server to send the full chain
No subject alternative names matching IP address / DNS name found	Certificate SAN does not contain the requested name	Reissue with the correct SAN; use the host name that the certificate names
CertificateExpiredException / NotAfter in the past	Expired certificate, or wrong system clock	Renew; automate renewal; check NTP
handshake_failure / No appropriate protocol	No common protocol version or cipher suite (for example a TLS 1.0 client)	Align `enabled-protocols` and cipher lists; upgrade the client
Received fatal alert: unknown_ca	Peer does not trust the certificate that this side sent (common in mTLS)	Add the issuing CA to the peer truststore
bad_certificate / certificate_required	Client certificate missing or invalid in mTLS	Configure the client keystore and its alias
Works in browser, fails in Java	Server sends leaf only; the browser fetches the missing intermediate itself, Java does not	Serve the complete chain
Wrong certificate returned	Client sent no SNI (IP address URL, old client)	Send SNI; configure a default virtual host

Typical TLS errors in Java and their usual causes.

WATCH OUT
A common shortcut is a `TrustManager` that accepts every certificate, or `HostnameVerifier` that returns true, installed to make an error disappear. This removes the only protection TLS gives against an active attacker and is reported by security scanners as a critical finding. Import the missing certificate or CA into a truststore instead.

The option `-Djavax.net.debug=ssl:handshake` makes the JDK print every handshake message, the selected protocol and the certificate path validation result; it is the first diagnostic tool for Java handshake failures.

5.2.4 Failure modes

- **Expired certificate.** The most frequent production TLS outage. Symptom: every client fails at once with a validity error. Prevention: ACME automation, monitoring of `notAfter` (alert at 30 and 7 days), and an inventory of internal certificates.
- **Incomplete chain.** Browsers repair a missing intermediate through the Authority Information Access extension; Java, curl and many API clients do not. The server must send leaf plus intermediates.
- **Clock skew.** A client clock outside `notBefore` and `notAfter` rejects a valid certificate. Containers and embedded devices without NTP are typical victims.
- **TLS interception.** Corporate proxies re-sign traffic with a private CA. Java then reports PKIX errors until that CA is added to the truststore.
- **Protocol and cipher mismatch.** Old Java 7 clients or legacy devices that support only TLS 1.0 fail against servers that require TLS 1.2 or higher.
- **0-RTT replay.** Enabling early data on an endpoint that executes non-idempotent requests permits replayed payments or orders.
- **Handshake cost under load.** Each new connection performs public-key operations: one signature on the server and one verification on the client. Connection reuse (HTTP keep-alive, HTTP/2 multiplexing, connection pools) and session resumption reduce CPU use and latency.
- **Mixed content and downgrade.** A page loaded over HTTPS that references `http://` resources is blocked or warned. **HSTS** (`Strict-Transport-Security: max-age=31536000; includeSubDomains`, RFC 6797) instructs the browser to use HTTPS only for the host.

REAL WORLD
In production the certificate private key is rarely stored in the application. Kubernetes deployments typically hold certificates in secrets managed by cert-manager (ACME renewal) and terminate TLS at an ingress controller or service mesh sidecar; cloud load balancers use certificates from a managed service. Service-to-service mTLS with automatically rotated short-lived certificates (hours, not months) is the standard in meshes, because the short lifetime removes the need for revocation.

5.2.5 Questions to expect

1. Describe the TLS 1.2 handshake.

The client sends ClientHello (version, random, cipher suites, extensions). The server replies with ServerHello (chosen suite), Certificate, ServerKeyExchange (signed ECDHE public value) and ServerHelloDone. The client verifies the certificate, sends ClientKeyExchange, ChangeCipherSpec and Finished. The server answers with ChangeCipherSpec and Finished. Both derive the same session keys from the shared secret and the randoms. Two round trips elapse before application data.

2. What improves in TLS 1.3?

One round trip instead of two (the key share travels in the ClientHello), mandatory forward secrecy, encryption of the certificate and later handshake messages, removal of weak algorithms (RSA key transport, CBC, RC4, SHA-1, compression, renegotiation), a smaller set of cipher suites, and optional 0-RTT resumption.

3. What is forward secrecy and how does TLS achieve it?

Session keys are derived from ephemeral ECDHE values that exist only in memory during the handshake. Recorded traffic cannot be decrypted later even if the server private key is stolen, because that key only signs and never encrypts the session secret. TLS 1.3 uses ephemeral exchange in every handshake.

4. How does the client verify the server identity?

By validating the certificate chain to a trusted root, checking dates, key usage and revocation status, and matching the requested host name against the SAN entries. The server then proves possession of the private key by signing the handshake transcript (ServerKeyExchange in TLS 1.2, CertificateVerify in TLS 1.3).

5. What are SNI and ALPN?

SNI is a ClientHello extension that carries the requested host name in plaintext so that a server hosting several sites can choose the right certificate. ALPN is an extension in which the client lists application protocols (`h2`, `http/1.1`) and the server picks one, avoiding an extra round trip after the handshake.

6. What is mutual TLS and where is it used?

The server requests a client certificate, and the client proves possession of its key. Both identities are verified against configured CAs. It is used for service-to-service authentication in meshes, for partner APIs and for device identity.

7. What risk does TLS 1.3 0-RTT introduce?

Early data is encrypted only under the resumption key, so it has no forward secrecy and an attacker can replay the recorded first flight. Servers should accept early data only for idempotent requests and may answer `425 Too Early`.

8. A Java client reports "PKIX path building failed". How is it diagnosed?

It means that no chain from the presented certificate leads to a trusted root. Inspect the chain with `openssl s_client -connect host:443 -showcerts`, check whether the intermediate is sent, identify the issuing CA, and import it into the truststore used by the JVM (`-Djavax.net.ssl.trustStore`) or an `SSLContext`. Disabling validation is not an acceptable fix.

9. Does HTTPS hide the URL?

The path, query string, headers and body are encrypted. The server IP address and, unless ECH is used, the host name in the SNI extension remain visible, as do packet sizes and timing.

10. Why is TLS termination at a load balancer common, and what is the trade-off?

Termination centralises certificates and offloads cryptographic work, and allows the balancer to read HTTP for routing. The segment behind the balancer is plaintext unless it is re-encrypted, which can violate compliance requirements; the later chapter on load balancers compares termination, passthrough and re-encryption.

KEY POINTS

- TLS runs over TCP and provides authentication, encryption and integrity; HTTPS is HTTP over TLS on port 443.
- TLS 1.3 needs one round trip, always uses ephemeral key exchange and encrypts the certificate; TLS 1.2 needs two round trips.
- SNI selects the certificate before HTTP starts; ALPN selects the application protocol (`h2` or `http/1.1`).
- Resumption avoids the certificate and public-key cost; 0-RTT data is replayable and restricted to idempotent requests.
- Java uses keystores (own identity) and truststores (accepted CAs); PKIX errors are fixed by supplying the missing trust or chain, never by disabling validation.
- Mutual TLS adds a client certificate and is the basis of service identity in zero-trust and mesh deployments.

5.3 Authentication in Transit

TLS authenticates the server to the client and protects bytes on the wire, but it does not tell the server who the user or calling service is. That second problem is solved at the application layer with credentials that accompany every request or that establish a session. This chapter covers the four mechanisms found in nearly every Spring Boot deployment: session cookies, API keys, JSON Web Tokens (JWT) and the OAuth 2.0 / OpenID Connect authorisation-code flow. For each, the focus is on how the credential travels on the wire, where it is stored, and what an attacker can do with it.

All of these mechanisms are secure only over TLS. A bearer credential (one that grants access to whoever presents it) sent over plain HTTP can be read and replayed by any device on the path.

5.3.1 Concept and mechanism

Credentials on the wire

Mechanism	Header carrying it	Typical value	Who validates it
HTTP Basic (RFC 7617)	`Authorization`	`Basic` + base64(user:password)	Server checks the password on every request
Session cookie	`Cookie`	`SESSION=7f3a9c2e1b` (opaque identifier)	Server looks the identifier up in a session store
API key	`X-API-Key` or `Authorization`	Opaque random string	Server looks up the key and its owner
Bearer token (RFC 6750)	`Authorization`	`Bearer` + JWT or opaque token	Resource server verifies a signature or asks the issuer

How common credentials appear in an HTTP request.

Basic authentication only encodes, it does not encrypt: `base64("alice:s3cret")` is `YWxpY2U6czNjcmV0`, which decodes back with a single command. It is acceptable only inside TLS and only for simple internal tools.

Session cookies

After a login the server creates a session record, stores it (memory, Redis, database) and sends a random identifier in a `Set-Cookie` header. The browser returns the cookie automatically with every request to that site. The cookie attributes determine most of the attack surface.

- `HttpOnly` hides the cookie from JavaScript, which limits theft through cross-site scripting (XSS).
- `Secure` sends the cookie only over HTTPS.
- `SameSite=Lax` or `Strict` stops the browser from attaching the cookie to most cross-site requests, which limits cross-site request forgery (CSRF).
- `Path`, `Domain`, `Max-Age` restrict scope and lifetime.

The following server, run on loopback, issues a cookie and then echoes the credentials it receives. The responses show how each mechanism appears on the wire.

Cookie, Basic and Bearer credentials observed with curl TERMINAL

```
$ curl -si http://127.0.0.1:8082/login
HTTP/1.1 204 No Content
Date: Sat, 17 Oct 2026 17:04:35 GMT
Set-cookie: SESSION=7f3a9c2e1b; Path=/; Max-Age=1800; HttpOnly; Secure; SameSite=Lax

$ curl -si http://127.0.0.1:8082/api
HTTP/1.1 401 Unauthorized
$ curl -s -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.e30.abc" http://127.0.0.1:8082/api
Authorization=Bearer eyJhbGciOiJIUzI1NiJ9.e30.abc Cookie=null
$ curl -s -u alice:s3cret http://127.0.0.1:8082/api
Authorization=Basic YWxpY2U6czNjcmV0 Cookie=null
$ curl -s -b "SESSION=7f3a9c2e1b" http://127.0.0.1:8082/api
Authorization=null Cookie=SESSION=7f3a9c2e1b
```

A session is **stateful**: the server must hold the session record, which complicates horizontal scaling (shared session store or sticky sessions at the load balancer, see the chapter on load balancers). Revocation is trivial: delete the record.

API keys

An API key is a long random string that identifies a calling application. It is sent in a header (never in the URL, where it is logged by proxies, browsers and servers). It carries no signature and no expiry by itself; the server must look it up. Keys are suitable for identifying a client and applying quotas, and are weak as the only defence because they are long-lived and often leak through source control and logs.

JSON Web Tokens

A **JWT** (RFC 7519) is a compact, self-contained token of three base64url parts separated by dots: a header naming the signing algorithm, a payload of claims, and a signature computed over the first two parts (JWS, RFC 7515). The resource server validates the token locally by verifying the signature and the claims, without a call to the issuer. This makes tokens **stateless** and well suited to services behind a gateway.

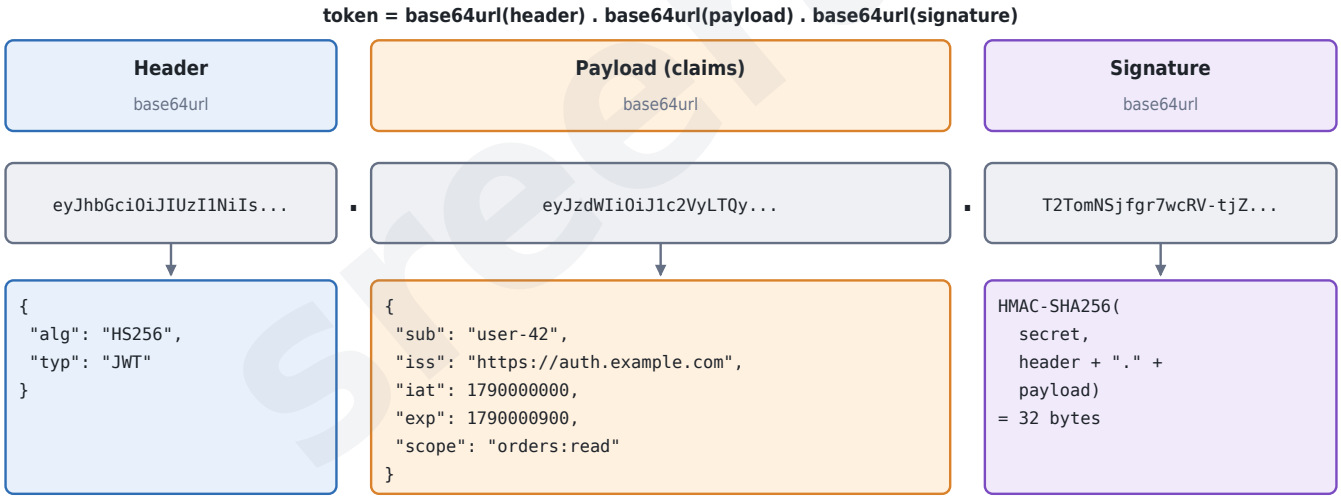


Figure 5.11. A JWT is the base64url encoding of a header and a payload, joined by dots to a signature that covers both; anyone can read the claims, but only the holder of the key can produce a valid signature.

Registered claims: `iss` (issuer), `sub` (subject), `aud` (audience), `exp` (expiry), `nbf` (not before), `iat` (issued at), `jti` (unique identifier). The payload is encoded, not encrypted, so it must never contain secrets. Two signature families exist: symmetric (HS256, HMAC with a shared secret, so every verifier can also forge tokens) and asymmetric (RS256, ES256, EdDSA), where the issuer signs with a private key and services verify with a public key published at a **JWKS** endpoint (RFC 7517).

A correct validation checks, in order: the algorithm is the expected one (never taken from the token header blindly), the signature verifies against the trusted key, `iss` equals the expected issuer, `aud` contains this service, `exp` and `nbf` are valid with a small clock tolerance, and the required scope or role is present.

OAuth 2.0 and OpenID Connect

OAuth 2.0 (RFC 6749) lets an application obtain a limited access token for a resource without handling the user password. **OpenID Connect** (OIDC) adds an identity layer: the authorisation server also returns an **ID token** (a JWT whose claims describe the authenticated user) and offers a `userinfo` endpoint. For browser and mobile applications the recommended grant is the **authorisation code flow with PKCE** (RFC 7636); the implicit flow is no longer recommended by the current security best practice (RFC 9700).

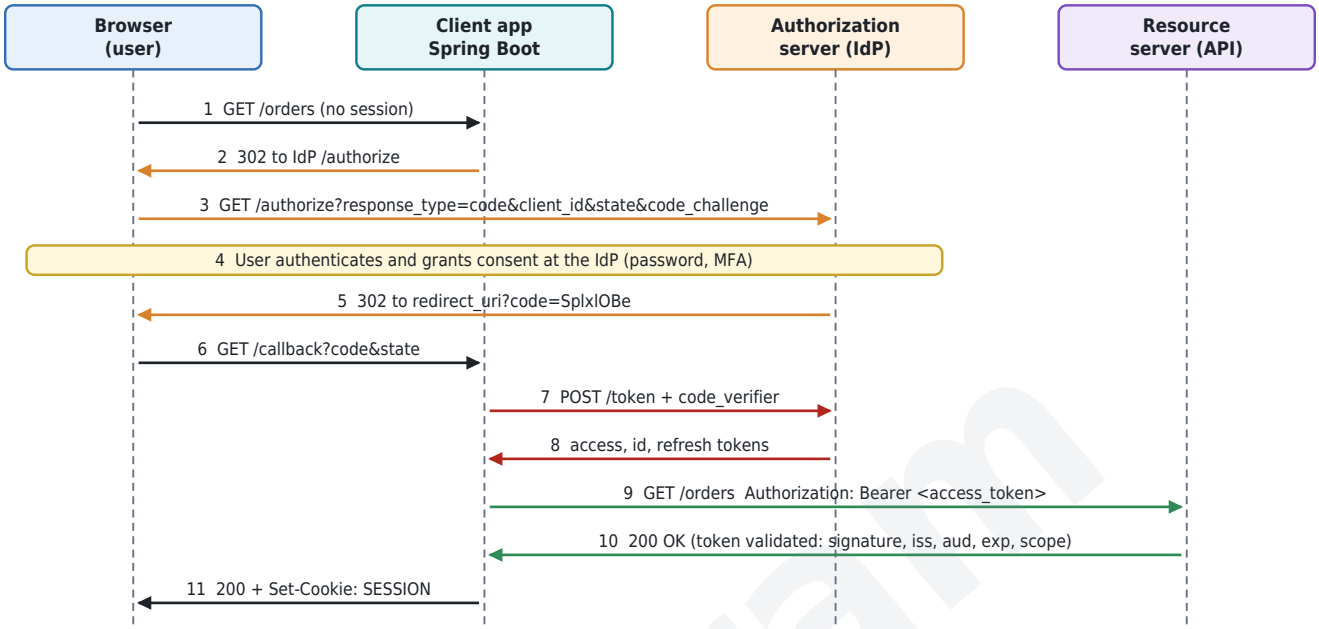


Figure 5.12. In the authorisation code flow the browser carries only a short-lived code (orange); the tokens (red) are exchanged on a direct back channel between the client application and the authorisation server over TLS.

1. The user requests a protected page; the client application (Spring Boot with `oauth2Login`) has no session and redirects the browser to the authorisation endpoint with `response_type=code`, `client_id`, `redirect_uri`, `scope=openid`, a random `state` and a `code_challenge` (SHA-256 of a secret `code_verifier`).
2. The user authenticates at the authorisation server (password, MFA) and consents.
3. The server redirects the browser to the `redirect_uri` with a single-use `code` and the original `state`. The client compares `state` with the stored value, which prevents CSRF on the callback.
4. The client calls the token endpoint directly, sending the `code`, the `code_verifier` and (for confidential clients) a client secret. The server checks that the verifier hashes to the earlier challenge, so a stolen code is useless to anyone who did not start the flow.
5. The server returns an access token, an ID token and optionally a refresh token. The client creates its own session cookie for the browser and calls APIs with `Authorization: Bearer <access_token>`.

5.3.2 Worked trace

The program builds a JWT with HS256 by hand using `javax.crypto.Mac`, decodes its parts and verifies the signature, then verifies again after altering the payload.

Jwt.java JAVA

```
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class Jwt {
    static String b64(byte[] b) {
        return Base64.getUrlEncoder().withoutPadding().encodeToString(b);
    }

    static String sign(String signingInput, String secret) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), "HmacSHA256"));
        return b64(mac.doFinal(signingInput.getBytes(StandardCharsets.UTF_8)));
    }
```

Jwt.java (continued) JAVA

```java
    static String text(String part) {
        return new String(Base64.getUrlDecoder().decode(part), StandardCharsets.UTF_8);
    }

    public static void main(String[] args) throws Exception {
        String header = b64("{\"alg\":\"HS256\",\"typ\":\"JWT\"}".getBytes(StandardCharsets.UTF_8));
        String payload = b64(("{\"sub\":\"user-42\",\"iss\":\"https://auth.example.com\","
                + "\"iat\":1790000000,\"exp\":1790000900,\"scope\":\"orders:read\"}")
                .getBytes(StandardCharsets.UTF_8));
        String token = header + "." + payload + "."
                + sign(header + "." + payload, "demo-secret-key");
        System.out.println(token);
        String[] parts = token.split("\\.");
        System.out.println("header  = " + text(parts[0]));
        System.out.println("payload = " + text(parts[1]));
        boolean ok = sign(parts[0] + "." + parts[1], "demo-secret-key").equals(parts[2]);
        System.out.println("signature valid with correct key: " + ok);
        String tampered = b64("{\"sub\":\"admin\",\"exp\":1790000900}"
                .getBytes(StandardCharsets.UTF_8));
        System.out.println("signature valid after payload change: "
                + sign(parts[0] + "." + tampered, "demo-secret-key").equals(parts[2]));
    }
}
```

Output of Jwt.java TERMINAL

```
$ java Jwt.java
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyLTQyIiwiaXNzIjoiaHR0cHM6Ly9hdXRoLmV4Y
W1wbGUuY29tIiwiaWF0IjoxNzkwMDAwMDAwLCJleHAiOjE3OTAwMDA5MDAsInNjb3BlIjoib3JkZXJzOnJlYWQifQ.
T2TomNSjfgr7wcRV-tjZFoMGeiRY0lV5MGsRoRTeYgs
header  = {"alg":"HS256","typ":"JWT"}
payload = {"sub":"user-42","iss":"https://auth.example.com","iat":1790000000,
  "exp":1790000900,"scope":"orders:read"}
signature valid with correct key: true
signature valid after payload change: false
```

The token and the payload line are printed on one line each by the program and wrapped here. What to notice: the payload is readable by anyone who holds the token, and the lifetime is 900 seconds (`exp` minus `iat`). Changing any byte of the payload invalidates the signature, but nothing prevents replay of an unmodified token before it expires, which is why access tokens are short-lived and always travel over TLS.

In a Spring Boot resource server the validation is configuration. With a JWKS-publishing issuer the service needs only the issuer URI; Spring Security then fetches the signing keys, verifies the signature and checks `exp`, `nbf` and `iss`. The configuration is shown for reference (not run here).

application.properties (not run) PROPERTIES

```properties
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth.example.com
# optional: fixed key set location instead of OIDC discovery
# spring.security.oauth2.resourceserver.jwt.jwk-set-uri=\
#   https://auth.example.com/.well-known/jwks.json
```

5.3.3 Hands-on

Choosing a mechanism

Mechanism	Client storage	Main attack	Revocation
Session cookie	Browser cookie jar (HttpOnly)	CSRF, session fixation, cookie theft over HTTP	Immediate (delete server record)
JWT in `Authorization`	Application memory; browser `localStorage` is readable by any script	XSS theft, replay until `exp`, wrong algorithm acceptance	Hard: wait for `exp`, or keep a denylist of `jti`
JWT in HttpOnly cookie	Browser cookie jar	CSRF (needs SameSite or token), size limits	As above
API key	Server-side secret store	Leakage in logs and repositories	Delete the key
Refresh token	Secure server-side or HttpOnly cookie	Theft gives long-lived access	Rotate on each use; revoke family on reuse

Storage, attack surface and revocation of each mechanism.

Guidance that follows from the table: browser applications served by one backend are simplest and safest with an HttpOnly, Secure, SameSite session cookie; JWT bearer tokens suit APIs called by other services or mobile clients; access tokens should live minutes and refresh tokens should rotate.

CSRF and token handling in Spring Security

Cookie-based sessions require CSRF protection because the browser sends the cookie even for requests triggered by another site. Spring Security enables CSRF protection by default for browser form flows and expects a token that an attacker page cannot read. Stateless APIs that authenticate with the `Authorization` header (not a cookie) are not CSRF-prone and normally disable it explicitly. The configuration is a reference sample (not run).

SecurityConfig.java (not run) JAVA

```java
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(a -> a.requestMatchers("/actuator/health").permitAll()
                                         .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
            .csrf(AbstractHttpConfigurer::disable);   // bearer tokens are not sent automatically
        return http.build();
    }
}
```

5.3.4 Failure modes

- **Accepting `alg: none` or the wrong algorithm.** Early libraries accepted unsigned tokens, or accepted an HS256 token signed with the RSA public key used as the HMAC secret. A verifier must pin the expected algorithm and key.
- **Not checking `aud` and `iss`.** A valid token issued for another service of the same issuer is accepted. Symptom: privilege escalation across services.
- **Tokens in URLs.** `?access_token=` ends up in access logs, browser history and `Referer` headers.
- **Long-lived bearer tokens.** A token leaked through a log or a proxy is usable until it expires. Keep access tokens to minutes.
- **Missing `state` or PKCE.** The callback becomes vulnerable to CSRF and code interception.
- **Cookie without `Secure` or `SameSite`.** The cookie is sent over HTTP and with cross-site requests.
- **Clock skew.** Services with clocks several minutes apart reject valid tokens (`exp`, `nbf`). Symptom: intermittent 401 responses. Allow a clock tolerance of 30 to 60 seconds and synchronise with NTP.
- **JWKS outage or key rotation.** A resource server that cannot fetch or refresh keys rejects every token. Cache the key set and refresh on an unknown `kid`.

INTERVIEW ANGLE
Where should a single-page application store its JWT? Storing it in `localStorage` exposes it to any script that runs on the page, so one XSS flaw leaks the token. An HttpOnly, Secure, SameSite cookie is invisible to scripts but needs CSRF protection. Many designs use the backend-for-frontend pattern: the browser holds only a session cookie and the backend keeps the tokens and calls the APIs.

5.3.5 Questions to expect

1. What is the difference between authentication and authorisation, and where does OAuth fit?

Authentication establishes who the caller is; authorisation decides what that caller may do. OAuth 2.0 is an authorisation framework that issues access tokens with scopes; OpenID Connect adds authentication by returning an ID token.

2. Explain the authorisation code flow with PKCE.

The client redirects the browser to the authorisation server with a code challenge; after login the server returns a single-use code through the browser; the client exchanges the code together with the code verifier at the token endpoint over a direct TLS connection and receives the tokens. PKCE binds the code to the client that started the flow.

3. Session cookie or JWT?

A session cookie is stateful, revocable at once and kept out of scripts by HttpOnly; it needs shared session storage to scale and CSRF protection. A JWT is stateless and verified locally, convenient across services, but cannot be revoked before expiry without extra state. Browser applications often use cookies; service-to-service calls use JWT.

4. Is a JWT encrypted?

A signed JWT (JWS) is only encoded; anyone can read the claims. Confidentiality requires JWE or TLS, and sensitive data should not be placed in the token.

5. Why do access tokens expire quickly while refresh tokens last longer?

A short lifetime bounds the damage of a leaked bearer token. The refresh token is sent only to the token endpoint, is stored more carefully and is rotated on each use, so its exposure is lower.

6. What does the `state` parameter protect against?

CSRF on the redirect callback. The client stores a random value before the redirect and rejects a callback whose `state` differs, so an attacker cannot make the victim log in to the attacker account.

7. How does a service validate a JWT without calling the issuer?

It verifies the signature with the issuer public key (from the JWKS endpoint, cached by `kid`), checks `iss`, `aud`, `exp`, `nbf` and the scope. Only the key set is fetched, and only occasionally.

KEY POINTS
- Credentials travel in `Authorization` or `Cookie` headers and are protected only by TLS; bearer credentials can be replayed by whoever obtains them.
- Session cookies are stateful and revocable; set HttpOnly, Secure and SameSite and add CSRF protection.
- A JWT is a signed, readable token; validate algorithm, signature, `iss`, `aud` and `exp`, and keep lifetimes short.
- The authorisation code flow with PKCE keeps tokens off the browser redirect; `state` protects the callback.
- Choose storage by attack surface: HttpOnly cookies against XSS theft, headers and short lifetimes for service calls.

5.4 Network Security

A network is attacked at every layer: forged link-layer addresses, spoofed IP sources, connection-exhaustion at TCP, poisoned DNS answers, intercepted TLS sessions and malicious HTTP requests. Defence therefore uses several independent controls (**defence in depth**), each assuming that another one may fail. This chapter presents the threat model, the filtering devices, tunnels and proxies that form the perimeter, and the main attacks with their mitigations.

5.4.1 Concept and mechanism

Threat model

A threat model states what an attacker can do and where the attacker is located.

Position	Capability	Typical defence
Passive on-path (shared Wi-Fi, ISP tap)	Read traffic	Encryption (TLS, VPN)
Active on-path (rogue router, ARP spoofing)	Read, modify, drop and inject traffic	Authenticated encryption, certificate validation
Off-path (remote host)	Send forged packets, flood, probe	Firewalls, random sequence numbers, rate limits
Insider or compromised host	Reach internal services	Segmentation, mutual TLS, least privilege
Application-level client	Send valid but malicious requests	Input validation, authentication, WAF, rate limiting

Attacker positions and capabilities.

Firewalls

A **firewall** enforces a rule set on traffic crossing a boundary. Rules are evaluated top to bottom, the first match decides, and a default policy (preferably deny) applies when nothing matches.

- A **stateless packet filter** (router ACL, cloud network ACL) judges each packet by its header fields alone: source and destination address, protocol, ports and flags. Return traffic must be allowed by an explicit rule.
- A **stateful firewall** keeps a connection table. Only the first packet of a flow is checked against the rules; later packets and replies that match a table entry are admitted. Unsolicited packets that do not belong to a tracked connection are dropped. Linux `nftables` and `iptables` use the `conntrack` module for this, and cloud security groups are stateful.
- An **application firewall** (web application firewall, WAF) inspects HTTP content for attack patterns such as SQL injection.
- A **next-generation firewall** adds application identification, TLS inspection and intrusion prevention.

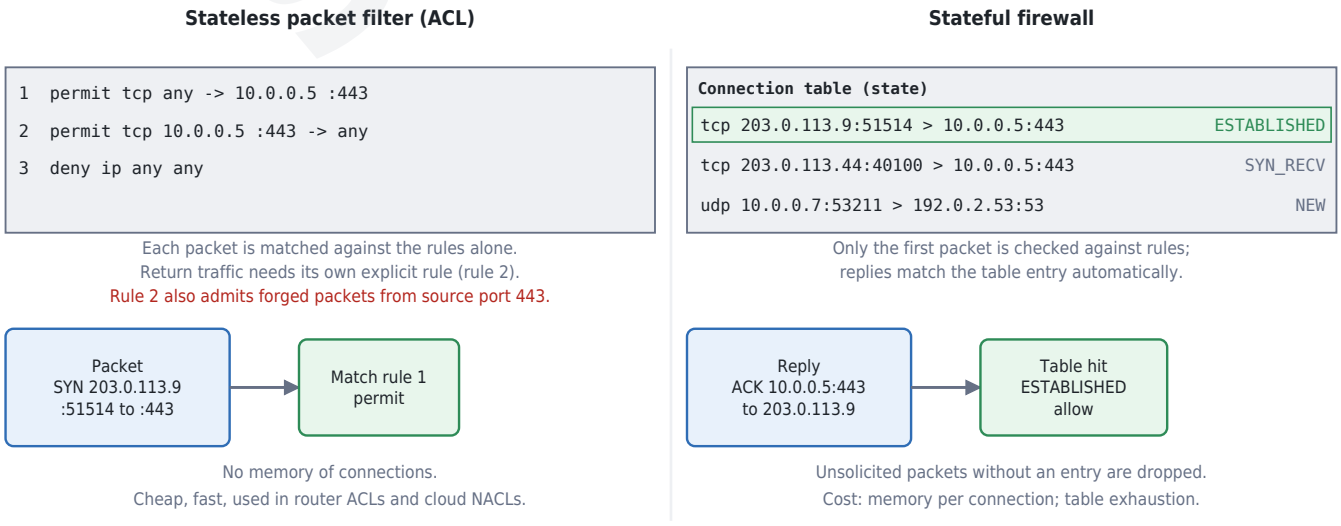


Figure 5.13. A stateless filter needs explicit return-path rules and cannot tell a genuine reply from a forged packet with source port 443, whereas a stateful firewall admits only packets that match an entry in its connection table.

Network zones and the DMZ

Segmentation limits the damage of a compromise. A **demilitarised zone** (DMZ) is a network segment between the Internet and the internal network that hosts the services which must be reachable from outside (reverse proxy, mail relay, bastion host). The edge firewall allows Internet traffic only to selected DMZ services; the inner firewall allows the DMZ to reach only the specific internal ports it needs and blocks everything in the opposite direction. A compromised proxy in the DMZ then cannot talk to the database directly.

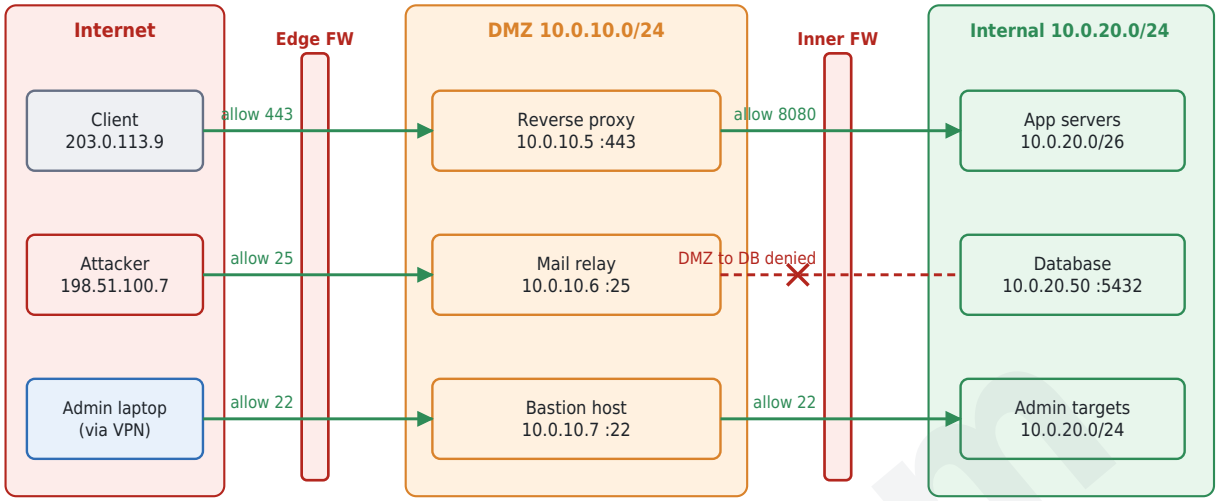


Figure 5.14. Two firewalls create three zones: the Internet may reach only listed DMZ services, the DMZ may reach only listed internal ports, and the direct path from the DMZ to the database is denied.

Intrusion detection and prevention

An **intrusion detection system** (IDS) watches a copy of the traffic (a mirror port or tap) and raises alerts from signatures (known patterns) or anomaly models. An **intrusion prevention system** (IPS) sits in line and drops matching traffic. The trade-off: an in-line device adds latency and can block legitimate traffic after a false positive; a passive device cannot stop an attack. Encrypted traffic limits payload inspection unless TLS is terminated at the inspection point.

VPN and tunnels

A **virtual private network** carries private traffic across an untrusted network by encapsulating each original packet inside an encrypted outer packet. Two topologies are used: site-to-site (gateways connect networks) and remote access (a client connects to a gateway).

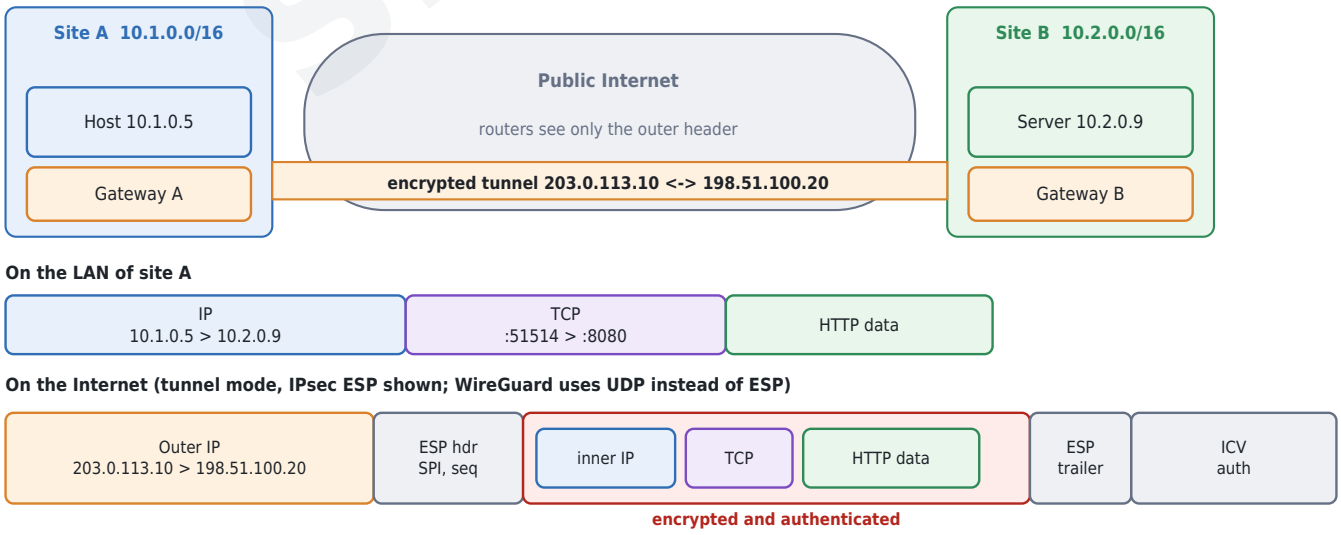


Figure 5.15. In tunnel mode the original packet from 10.1.0.5 to 10.2.0.9 is encrypted as a whole and wrapped in a new IP header between the two gateways; Internet routers see only the outer addresses.

Technology	Layer and transport	Characteristics	Typical port
IPsec (IKEv2 + ESP)	Network layer; IP protocol 50 (ESP), UDP 500 and 4500 for IKE and NAT traversal	Standard for site-to-site tunnels; transport and tunnel modes; complex configuration	UDP 500, 4500
WireGuard	UDP	Small code base, fixed modern cryptography (Curve25519, ChaCha20-Poly1305), one-round-trip handshake over UDP	Configurable, 51820 by convention
TLS VPN (OpenVPN, SSL VPN)	TCP or UDP carrying TLS	Passes through restrictive firewalls on 443; user-space implementation	443 or 1194

Common VPN technologies.

Running a TCP application over a TCP-based tunnel (TCP inside TCP) can interact badly because both layers retransmit; UDP-based tunnels avoid it.

Forward and reverse proxies

A **forward proxy** acts on behalf of clients: clients are configured to send requests through it, and servers see the proxy address. It is used for egress filtering, caching and policy. A **reverse proxy** acts on behalf of servers: clients connect to one public address and the proxy forwards each request to a back-end server. It hides the topology, terminates TLS, balances load, caches and filters. A reverse proxy is the usual enforcement point for rate limiting, authentication and WAF rules; load balancers and API gateways are specialised reverse proxies (see the next chapter).

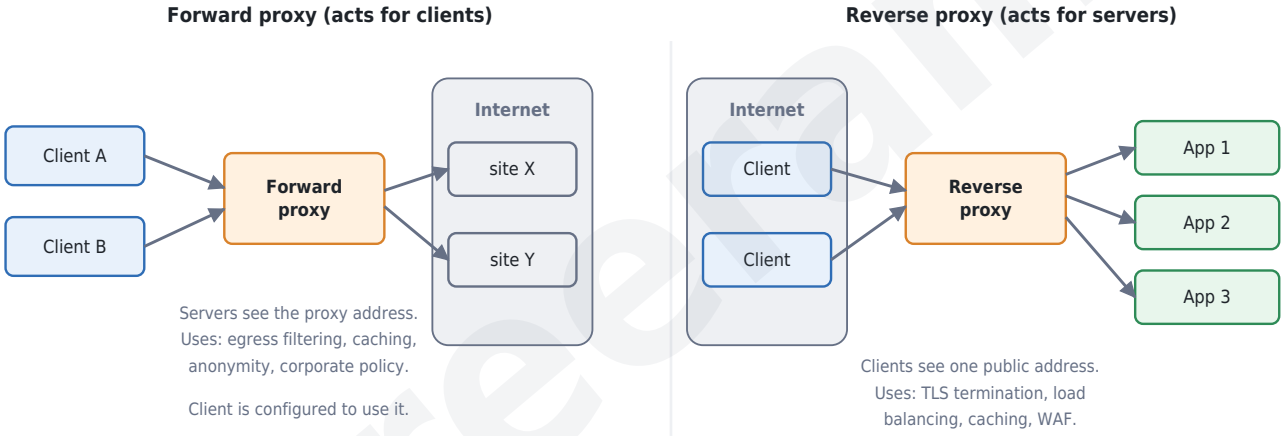


Figure 5.16. A forward proxy represents the clients to the Internet, whereas a reverse proxy represents a set of servers to the Internet; the roles differ in who is configured to use the proxy and which side is hidden.

5.4.2 Worked trace

A packet trace through the firewalls of the DMZ diagram, using a stateful edge firewall and a stateful inner firewall with default-deny policy. The rules are those drawn: Internet to 10.0.10.5 on TCP 443, Internet to 10.0.10.6 on TCP 25, administrators to 10.0.10.7 on TCP 22, DMZ proxy to 10.0.20.0/26 on TCP 8080, bastion to 10.0.20.0/24 on TCP 22. The addresses are documentation and private ranges chosen for the example; the table is a worked decision trace, not a capture.

No	Packet	Rule or state	Verdict
1	203.0.113.9:51514 to 10.0.10.5:443, SYN	Edge rule "Internet to 10.0.10.5 TCP 443"; entry created	Permit
2	10.0.10.5:443 to 203.0.113.9:51514, SYN-ACK	Matches table entry (reverse direction)	Permit
3	198.51.100.7:40000 to 10.0.10.5:22, SYN	No rule for port 22 on this host; default policy	Drop
4	198.51.100.7:40001 to 10.0.10.6:25, SYN	Edge rule "Internet to 10.0.10.6 TCP 25"	Permit
5	10.0.10.5:34000 to 10.0.20.10:8080, SYN	Inner rule "proxy to app servers TCP 8080"	Permit

No	Packet	Rule or state	Verdict
6	10.0.10.5:51000 to 10.0.20.50:5432, SYN	No inner rule for the database; default policy	Drop
7	198.51.100.7:443 to 10.0.10.5:55555, ACK	No table entry; a stateless filter would permit it because of source port 443	Drop

Verdicts for seven packets (stateful, default deny).

Packet 7 illustrates the weakness of stateless rules: an attacker can craft packets with source port 443 to pass a rule that was written for return traffic, and use them for scanning. A stateful device recognises that no connection exists. The equivalent rule set in `nftables` syntax is shown below (typical configuration, not run on the build machine, which lacks the tool).

nftables.conf (not run) CONF

```
table inet filter {
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    ip daddr 10.0.10.5 tcp dport 443 accept
    ip daddr 10.0.10.6 tcp dport 25 accept
    ip saddr 10.0.10.5 ip daddr 10.0.20.0/26 tcp dport 8080 accept
  }
}
```

5.4.3 Hands-on

Attacks at the network and transport layers

Attack	Mechanism	Mitigation
SYN flood	Many SYN segments (often with forged sources) fill the SYN queue of the listener with half-open connections	SYN cookies, larger backlog, rate limiting, upstream scrubbing
Volumetric DDoS	Traffic from many hosts exceeds link capacity; amplification through UDP services (DNS, NTP, memcached) multiplies it	Anycast and scrubbing centres, ingress filtering (BCP 38), disable open resolvers
Application-layer DDoS	Many valid-looking HTTP requests exhaust CPU or connections	Rate limits, caching, WAF, bot detection, autoscaling
ARP spoofing	Forged ARP replies map the gateway IP to the attacker MAC	Dynamic ARP inspection, port security, TLS
DNS spoofing and cache poisoning	A forged answer is accepted by a resolver and cached	Random source port and transaction ID, DNSSEC, DoT/DoH
Man in the middle	Attacker relays and alters traffic between two parties	TLS with certificate validation, HSTS, certificate pinning where appropriate
Replay	A captured valid message is sent again	Nonces, timestamps, sequence numbers (built into TLS records), idempotency keys

Network and transport attacks and mitigations.

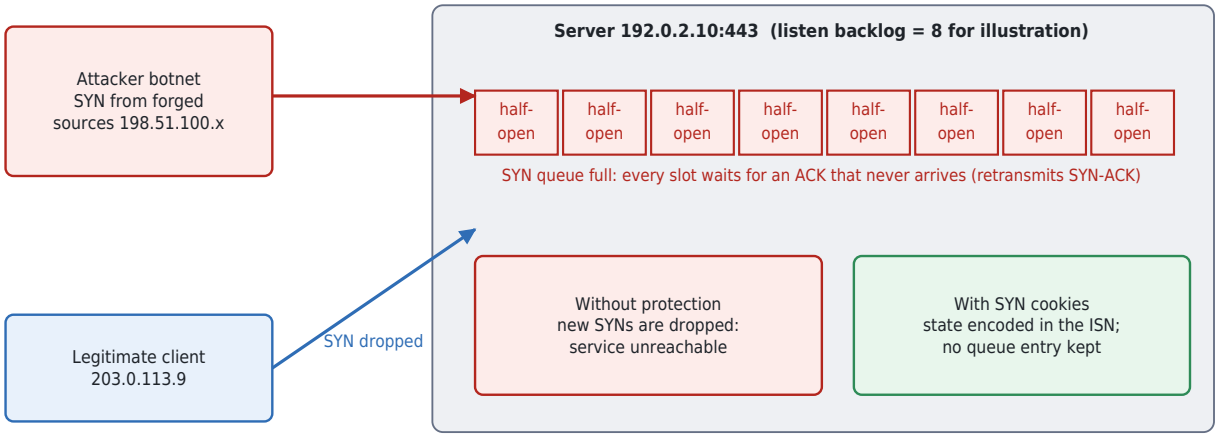


Figure 5.17. Each forged SYN occupies a queue slot until the SYN-ACK retransmissions time out; once the queue is full, genuine SYN segments are dropped, whereas SYN cookies encode the connection state in the initial sequence number and keep no entry.

The SYN queue holds connections in the SYN_RECV state, waiting for the final ACK of the three-way handshake. Because the source addresses are forged, the final ACK never arrives. Linux counters this with **SYN cookies** (`net.ipv4.tcp_syncookies`): when the queue overflows, the server encodes a hash of the connection parameters into the initial sequence number of its SYN-ACK and allocates no state; a legitimate client returns the value in its ACK and the connection is then reconstructed. Cost: some TCP options may be lost during cookie mode.

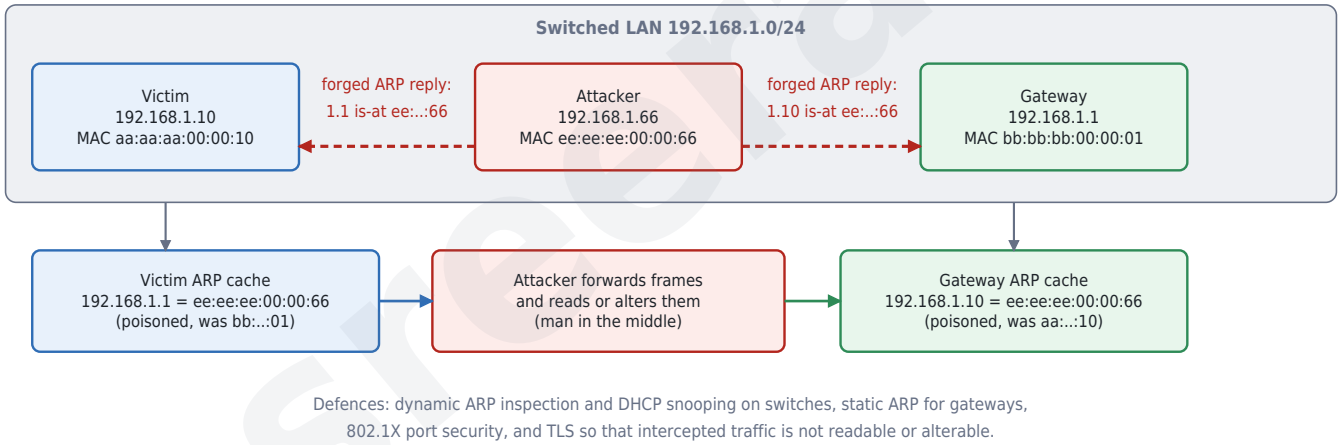


Figure 5.18. ARP has no authentication, so two forged replies are enough to place the attacker between the victim and the gateway; neither ARP cache shows an error, and all traffic flows through the attacker.

Application-level attacks

- **Cross-site request forgery (CSRF).** A malicious page causes the victim browser to send a request that carries the victim session cookie. Defences: `SameSite` cookies, anti-CSRF tokens, and checking `Origin` for state-changing requests.
- **Cross-site scripting (XSS).** Attacker-controlled data is rendered as HTML or script. Defences: context-aware output encoding, a `Content-Security-Policy` header, `HttpOnly` cookies.
- **Server-side request forgery (SSRF).** The attacker makes the server issue a request to an internal address, for example the cloud metadata service at 169.254.169.254. The check below resolves a host name and rejects loopback, private, link-local and wildcard addresses.

Ssrf.java JAVA

```java
import java.net.InetAddress;

public class Ssrf {
    static boolean blocked(InetAddress a) {
        return a.isLoopbackAddress() || a.isSiteLocalAddress() || a.isLinkLocalAddress()
```

Ssrf.java (continued) JAVA

```
                || a.isAnyLocalAddress() || a.isMulticastAddress();
    }

    public static void main(String[] args) throws Exception {
        for (String h : new String[] {"127.0.0.1", "10.1.2.3", "172.16.5.9", "192.168.0.10",
                "169.254.169.254", "0.0.0.0", "192.0.2.10", "203.0.113.7"}) {
            InetAddress a = InetAddress.getByName(h);
            System.out.printf("%-16s blocked=%b%n", h, blocked(a));
        }
    }
}
```

Output of Ssrf.java TERMINAL

```
$ java Ssrf.java
127.0.0.1        blocked=true
10.1.2.3         blocked=true
172.16.5.9       blocked=true
192.168.0.10     blocked=true
169.254.169.254  blocked=true
0.0.0.0          blocked=true
192.0.2.10       blocked=false
203.0.113.7      blocked=false
```

A host-name check is not sufficient on its own: DNS can return a public address at validation time and a private address at connection time (DNS rebinding). The validated `InetAddress` must be the one that is connected to, redirects must be re-validated, and an allow-list of destinations is stronger than a deny-list. Because `isSiteLocalAddress` in Java covers only the IPv4 private ranges and the deprecated IPv6 site-local range, IPv6 unique local addresses (`fc00::/7`) need a separate check.

Rate limiting with a token bucket

A **token bucket** admits a request when a token is available. The bucket holds at most `capacity` tokens and is refilled at a constant rate, which allows bursts up to the capacity and a sustained rate equal to the refill rate. It is the algorithm behind most API gateways and libraries. The program simulates a bucket of capacity 5 refilled at 2 tokens per second, with one request every 100 ms.

Bucket.java JAVA

```
public class Bucket {
    final double capacity, refillPerSec;
    double tokens;
    long lastNanos;

    Bucket(double capacity, double refillPerSec, long nowNanos) {
        this.capacity = capacity;
        this.refillPerSec = refillPerSec;
        this.tokens = capacity;
        this.lastNanos = nowNanos;
    }

    boolean tryAcquire(long nowNanos) {
        tokens = Math.min(capacity, tokens + (nowNanos - lastNanos) / 1e9 * refillPerSec);
        lastNanos = nowNanos;
        if (tokens >= 1) { tokens -= 1; return true; }
        return false;
    }

    public static void main(String[] args) {
        Bucket b = new Bucket(5, 2, 0);          // burst of 5, sustained 2 requests per second
        long t = 0;
        for (int i = 0; i < 12; i++) {           // one request every 100 ms for 1.2 s
            boolean ok = b.tryAcquire(t);
            System.out.printf("t=%4d ms  %s  tokens left %.2f%n",
                    t / 1_000_000, ok ? "200 OK " : "429 Too Many", b.tokens);
            t += 100_000_000L;
        }
    }
}
```

Output of Bucket.java TERMINAL

```
$ java Bucket.java
t=   0 ms  200 OK   tokens left 4.00
t= 100 ms  200 OK   tokens left 3.20
t= 200 ms  200 OK   tokens left 2.40
t= 300 ms  200 OK   tokens left 1.60
t= 400 ms  200 OK   tokens left 0.80
t= 500 ms  200 OK   tokens left 0.00
t= 600 ms  429 Too Many  tokens left 0.20
t= 700 ms  429 Too Many  tokens left 0.40
t= 800 ms  429 Too Many  tokens left 0.60
t= 900 ms  429 Too Many  tokens left 0.80
t=1000 ms  200 OK   tokens left 0.00
t=1100 ms  429 Too Many  tokens left 0.20
```

The first six requests (the initial five tokens plus the tokens refilled during the burst) pass, then the bucket is empty and requests are rejected with `429 Too Many Requests` until a full token has accumulated (every 500 ms at 2 tokens per second). A real service returns a `Retry-After` header with the 429 response so that clients can back off.

5.4.4 Failure modes

- **Rule order and over-broad rules.** A permissive rule such as "any to any" placed above a deny rule makes the deny unreachable. Review rules with the first-match principle in mind and keep the default policy at deny.
- **Connection-table exhaustion.** A stateful firewall or NAT device holds memory per flow. A flood or very many idle connections fills the table, after which new connections fail. Symptom on Linux: `nf_conntrack: table full, dropping packet` in the kernel log.
- **Idle timeout resets.** Stateful devices discard idle entries (commonly minutes). A long-lived idle TCP connection (database connection pool) then fails with a reset on the next use. Mitigation: TCP keep-alive or pool validation with a period shorter than the device timeout.
- **Asymmetric routing.** If a stateful firewall sees only one direction of a flow, the packets that it sees do not match a state entry and are dropped.
- **Trusting the internal network.** Flat internal networks allow lateral movement after one host is compromised. Segmentation, mutual TLS between services and least-privilege rules reduce the exposure.
- **Blocking ICMP entirely.** Dropping all ICMP breaks path MTU discovery (type 3 code 4) and causes connections that hang on large packets; allow the required types.
- **VPN split-tunnel and MTU.** Encapsulation reduces the usable MTU; large packets are dropped if fragmentation or path MTU discovery fails. Symptom: small requests succeed, large downloads stall.

WATCH OUT
Spoofed source addresses make IP-based allow-lists weak against attackers who can send packets that appear to come from a trusted address, and make rate limiting per source address bypassable. Combine network filtering with authentication at the application layer; do not treat a source address as an identity.

5.4.5 Questions to expect

1. What is the difference between a stateless and a stateful firewall?

A stateless filter evaluates every packet against the rules independently and needs explicit rules for return traffic. A stateful firewall tracks connections, so replies to permitted outbound connections are allowed automatically and unsolicited inbound packets are dropped. The cost is memory per flow and exposure to table exhaustion.

2. What is a DMZ and why is it used?

A segment between the Internet and the internal network that hosts publicly reachable services. Two firewalls restrict what may enter the DMZ and what the DMZ may reach, so that compromise of a public service does not give direct access to internal systems.

3. How does a SYN flood work and how is it mitigated?

The attacker sends many SYN segments, usually with forged sources, so that the server allocates half-open connection state and waits for ACKs that never arrive. When the queue is full, legitimate connections are refused. Mitigations: SYN cookies, a larger backlog, shorter SYN-ACK retries, rate limiting and upstream filtering.

4. How does ARP spoofing work and how is it prevented?

ARP replies are accepted without authentication, so an attacker answers with its own MAC address for the gateway IP and for the victim IP and relays traffic. Prevention uses dynamic ARP inspection and port security on switches, static entries for critical hosts and end-to-end encryption.

5. What is DNS cache poisoning?

An attacker races the real answer with a forged one that matches the transaction ID and source port of a pending query. If accepted, the resolver caches the wrong record. Random source ports and IDs make guessing infeasible; DNSSEC authenticates the records themselves.

6. Forward proxy versus reverse proxy?

A forward proxy is configured by clients and hides them from servers; a reverse proxy is fronting servers and hides them from clients. Reverse proxies provide TLS termination, load balancing, caching and filtering.

7. Compare IPsec, WireGuard and TLS-based VPNs.

IPsec works at the network layer with IKE negotiation and ESP encapsulation and is the standard for site-to-site tunnels. WireGuard is a compact UDP-based protocol with fixed modern cryptography. TLS VPNs run over TCP or UDP port 443, which traverses most firewalls, at the cost of user-space processing.

8. What are CSRF, XSS and SSRF?

CSRF makes a victim browser send an authenticated request that the user did not intend; XSS runs attacker script in the victim page; SSRF makes a server send a request to an internal destination on behalf of the attacker. They are prevented by SameSite cookies and tokens, output encoding and CSP, and destination validation plus network egress rules respectively.

9. How does rate limiting with a token bucket work?

Tokens accumulate at a fixed rate up to a maximum; each request consumes one. Bursts up to the bucket size are allowed, the long-term rate equals the refill rate, and requests without a token receive `429 Too Many Requests`.

KEY POINTS

- Security uses layered controls: encryption, filtering, segmentation, authentication and rate limiting, each assuming that another can fail.
- Stateful firewalls track flows and admit only replies to permitted connections; stateless filters judge packets in isolation. Default policy should be deny.
- A DMZ with two firewalls confines public services; VPNs encapsulate private traffic in encrypted outer packets; reverse proxies enforce policy in front of servers.
- SYN floods exhaust half-open queue entries (SYN cookies), ARP and DNS spoofing exploit unauthenticated protocols, and TLS defeats on-path tampering.
- Application attacks (CSRF, XSS, SSRF) are mitigated with SameSite cookies and tokens, output encoding and destination allow-lists; rate limiting uses token buckets and returns 429.

5.5 Reverse Proxies, Load Balancers and Gateways

One server process has finite capacity and is a single point of failure. A production service therefore runs as several identical instances behind a component that accepts client connections and distributes them. That component is a reverse proxy, a load balancer or an API gateway; in practice one product (nginx, HAProxy, Envoy, a cloud load balancer, Spring Cloud Gateway) often performs all three roles. The functions are distinguished by the layer at which they operate and by the policies they apply.

The tasks of this tier are: distribute load across instances, detect failed instances and stop sending traffic to them, terminate TLS, hide the internal topology, and apply cross-cutting policy (authentication, rate limiting, routing) in one place.

5.5.1 Concept and mechanism

Layer 4 and layer 7 balancing

A **layer 4** (transport) balancer decides from the IP addresses and TCP or UDP ports of a packet. It selects a back end when the first packet (the SYN) arrives and then forwards every packet of that connection to the same back end, either by rewriting the destination address (NAT) or by forwarding frames unchanged (direct server return). It never parses the payload, so it works for any TCP or UDP protocol and handles very high packet rates.

A **layer 7** (application) balancer terminates the client TCP connection (and usually TLS), parses the HTTP request and opens or reuses a separate connection to a back end. It can route by `Host`, path, header or cookie, rewrite requests, retry on another instance, cache responses and compress. The cost is higher CPU use and memory per connection, and the original client address is no longer visible to the back end unless the balancer passes it on.

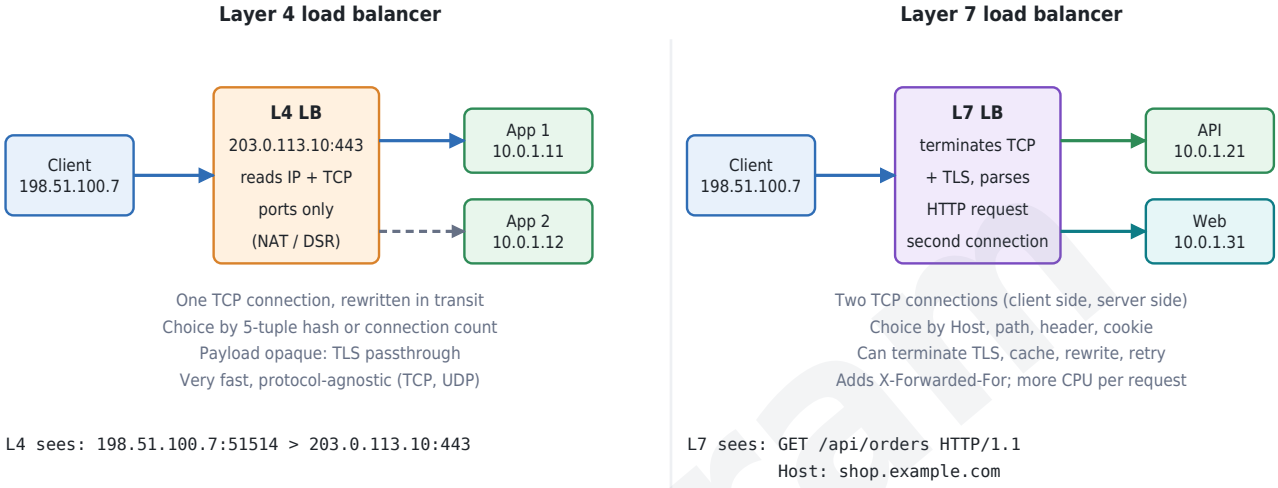


Figure 5.19. A layer 4 balancer rewrites and forwards one TCP connection on the basis of addresses and ports; a layer 7 balancer terminates the client connection, reads the HTTP request and opens a second connection to the chosen back end.

Property	Layer 4	Layer 7
Decision input	5-tuple (addresses, ports, protocol)	HTTP method, host, path, headers, cookies
TCP connections	One per client, rewritten in transit	Two: client to balancer, balancer to back end
TLS	Passthrough (back end terminates) or terminate	Terminate, then optionally re-encrypt
Protocols	Any TCP or UDP	HTTP, HTTP/2, gRPC, WebSocket
Client address at the back end	Preserved (DSR) or replaced (NAT, PROXY protocol needed)	Replaced; carried in `X-Forwarded-For`
Per-request features	None	Retry, caching, rewrite, authentication
Examples	AWS NLB, LVS, HAProxy in TCP mode	AWS ALB, nginx, Envoy, Spring Cloud Gateway

Comparison of layer 4 and layer 7 load balancing.

Balancing algorithms

- **Round robin** sends request i to server i mod N. Simple and fair when requests cost the same. **Weighted round robin** gives larger instances a larger share.
- **Least connections** sends the request to the server with the fewest active connections. It adapts to uneven request durations. **Least response time** also considers measured latency.
- **IP hash** (or hash of a cookie or header) maps a key to a server so that the same client reaches the same instance. When the server count changes, plain `hash mod N` remaps almost every key.
- **Consistent hashing** places servers at positions on a ring of hash values; a key belongs to the first server position clockwise from the key hash. Adding or removing a server moves only the keys adjacent to its positions. This is used for caches, sharded stores and affinity routing.
- **Power of two choices** samples two random servers and picks the less loaded one; it approximates least connections without global state.

The effect of uneven request durations is shown by a small model: three servers, each processing one request at a time, with one request arriving per time unit, where every third request takes 9 units and the others take 1 unit.

LbSim.java JAVA

```java
import java.util.Arrays;

public class LbSim {
    // One request arrives per time unit. Every third request is heavy (9 units), the others take 1.
    static int cost(int i) { return i % 3 == 2 ? 9 : 1; }

    static void run(String name, boolean leastConn) {
        int servers = 3, n = 12;
        double[] busyUntil = new double[servers];        // one worker per server, FIFO queue
        double[] finish = new double[n];
        int[] server = new int[n];
        int[] peak = new int[servers];
        for (int i = 0; i < n; i++) {
            double t = i;
            int pick = i % servers;
            if (leastConn) {
                int best = Integer.MAX_VALUE;
                for (int s = 0; s < servers; s++) {
                    int a = 0;
                    for (int j = 0; j < i; j++) if (server[j] == s && finish[j] > t) a++;
                    if (a < best) { best = a; pick = s; }
                }
            }
            server[i] = pick;
            double start = Math.max(t, busyUntil[pick]);
            finish[i] = start + cost(i);
            busyUntil[pick] = finish[i];
            int a = 0;
            for (int j = 0; j <= i; j++) if (server[j] == pick && finish[j] > t) a++;
            peak[pick] = Math.max(peak[pick], a);
        }
        double worst = 0, sumWait = 0;
        for (int i = 0; i < n; i++) {
            worst = Math.max(worst, finish[i] - i);
            sumWait += finish[i] - i - cost(i);
        }
        System.out.printf("%s%n  server per request %s%n  peak concurrent %s, mean queueing %.2f, "
                + "slowest response %.0f%n", name, Arrays.toString(server), Arrays.toString(peak),
                sumWait / n, worst);
    }

    public static void main(String[] args) {
        run("round robin", false);
        run("least connections", true);
    }
}
```

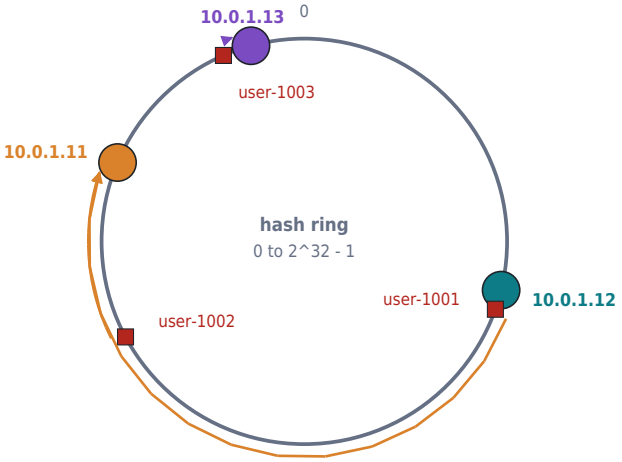
Output of LbSim.java TERMINAL

```
$ java LbSim.java
round robin
  server per request [0, 1, 2, 0, 1, 2, 0, 1, 2, 0, 1, 2]
  peak concurrent [1, 1, 3], mean queueing 3.00, slowest response 27
least connections
  server per request [0, 0, 0, 1, 1, 1, 2, 2, 2, 0, 1, 0]
  peak concurrent [2, 2, 1], mean queueing 0.58, slowest response 10
```

Round robin sends every heavy request (index 2, 5, 8, 11) to server 2, which accumulates a queue of three concurrent requests and an end-to-end time of 27 units for the slowest request. Least connections sees which server is busy and avoids it, so the slowest request needs 10 units. The model is deliberately simple, but it reproduces the real effect: round robin is optimal only for uniform requests.

Consistent hashing

The ring below shows the three servers at their real positions (the first 32 bits of the MD5 digest of the server address) with three keys. A key is served by the first server clockwise from its hash.



Lookup rule

hash(key); walk clockwise;
first server position wins
(wrap past 2^32 to the start)
user-1003 (4014382812) to .13

Remove 10.0.1.13

only its keys move: user-1003
now wraps around to .12
1,457 of 10,000 keys moved
(hash mod N: 6,705 moved)

100 positions per server

load 3895 / 3080 / 3025
instead of 5231 / 3312 / 1457
keys moved on removal: 3025
(exactly the removed server)

Figure 5.20. Each key walks clockwise to the next server position; removing 10.0.1.13 remaps only the keys that it owned, whereas hash mod N remaps most keys; 100 positions per server even out the load.

Ring.java JAVA

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Ring {
    static long hash(String s) throws Exception {
        byte[] d = MessageDigest.getInstance("MD5").digest(s.getBytes(StandardCharsets.UTF_8));
        long h = 0;
        for (int i = 0; i < 4; i++) h = (h << 8) | (d[i] & 0xFF);
        return h;                       // 32-bit position on the ring
    }

    static TreeMap<Long, String> build(List<String> nodes, int vnodes) throws Exception {
        TreeMap<Long, String> ring = new TreeMap<>();
        for (String n : nodes)
            for (int v = 0; v < vnodes; v++) ring.put(hash(n + "#" + v), n);
        return ring;
    }

    static String lookup(TreeMap<Long, String> ring, String key) throws Exception {
        Map.Entry<Long, String> e = ring.ceilingEntry(hash(key));
        return (e != null ? e : ring.firstEntry()).getValue();
    }

    public static void main(String[] args) throws Exception {
        List<String> nodes = List.of("10.0.1.11", "10.0.1.12", "10.0.1.13");
        TreeMap<Long, String> one = build(nodes, 1);
        System.out.println("Ring with 1 position per server:");
        one.forEach((k, v) -> System.out.printf("  %10d (%08x)  %s%n", k, k, v));
        for (String key : List.of("user-1001", "user-1002", "user-1003"))
            System.out.printf("  key %-10s hash %10d -> %s%n", key, hash(key), lookup(one, key));

        int keys = 10000;
        for (int vn : new int[] {1, 100}) {
            TreeMap<Long, String> before = build(nodes, vn);
            TreeMap<Long, String> after = build(List.of("10.0.1.11", "10.0.1.12"), vn);
            int moved = 0;
            int[] load = new int[3];
            for (int i = 0; i < keys; i++) {
                String k = "user-" + i;
                String a = lookup(before, k), b = lookup(after, k);
                load[nodes.indexOf(a)]++;
                if (!a.equals(b)) moved++;
            }
```

Ring.java (continued) JAVA

```
            System.out.printf("vnodes=%3d load %s, keys moved after removing .13: %d of %d%n",
                    vn, Arrays.toString(load), moved, keys);
        }
        int movedMod = 0;
        for (int i = 0; i < keys; i++) {
            long h = hash("user-" + i);
            if (h % 3 != h % 2) movedMod++;
        }
        System.out.println("hash mod N: keys moved when N changes 3 -> 2: "
                + movedMod + " of " + keys);
    }
}
```

Output of Ring.java TERMINAL

```
$ java Ring.java
Ring with 1 position per server:
  1239792742 (49e5bc66)  10.0.1.12
  3494228875 (d045b38b)  10.0.1.11
  4112989833 (f5273e89)  10.0.1.13
  key user-1001  hash 1307061317 -> 10.0.1.11
  key user-1002  hash 2885715928 -> 10.0.1.11
  key user-1003  hash 4014382812 -> 10.0.1.13
vnodes=  1 load [5231, 3312, 1457], keys moved after removing .13: 1457 of 10000
vnodes=100 load [3895, 3080, 3025], keys moved after removing .13: 3025 of 10000
hash mod N: keys moved when N changes 3 -> 2: 6705 of 10000
```

What to notice. With one position per server the ring is unbalanced (5,231 keys on the first server against 1,457 on the last). One hundred virtual positions per server bring the load close to an even split (about 3,900, 3,080 and 3,025 for 10,000 keys). Removing a server moves exactly the keys that it owned: 3,025 of 10,000 (the share of that server), whereas `hash mod N` moves 6,705 of 10,000 when N changes from 3 to 2, which would empty most of a cache.

Health checks

A balancer must know which instances can take traffic. **Active** checks send a probe at a fixed interval (TCP connect, or an HTTP request to a health path) and mark an instance down after a number of consecutive failures and up again after a number of successes. **Passive** checks observe real traffic and eject an instance after errors or timeouts. Spring Boot Actuator exposes `/actuator/health/liveness` (the process should be restarted if it fails) and `/actuator/health/readiness` (the instance should receive no traffic while it fails); balancers should probe the readiness endpoint, because an instance that is alive but still starting or overloaded must be taken out of rotation without being killed.

Detection is never instantaneous. Between a failure and the moment the check marks the instance down, a fraction of requests is still sent to it. The run in the worked trace below shows this window as `502` responses.

Sticky sessions

When session state lives in the memory of one instance, the balancer must send all requests of a user to that instance (**session affinity**, sticky sessions), through a balancer-issued cookie or an IP or header hash. The disadvantages are uneven load, loss of the session when the instance fails or is replaced during a deployment, and difficulty in scaling down. The preferred design is stateless instances with session data in a shared store (Redis, database) or in a signed token.

TLS termination, passthrough and re-encryption

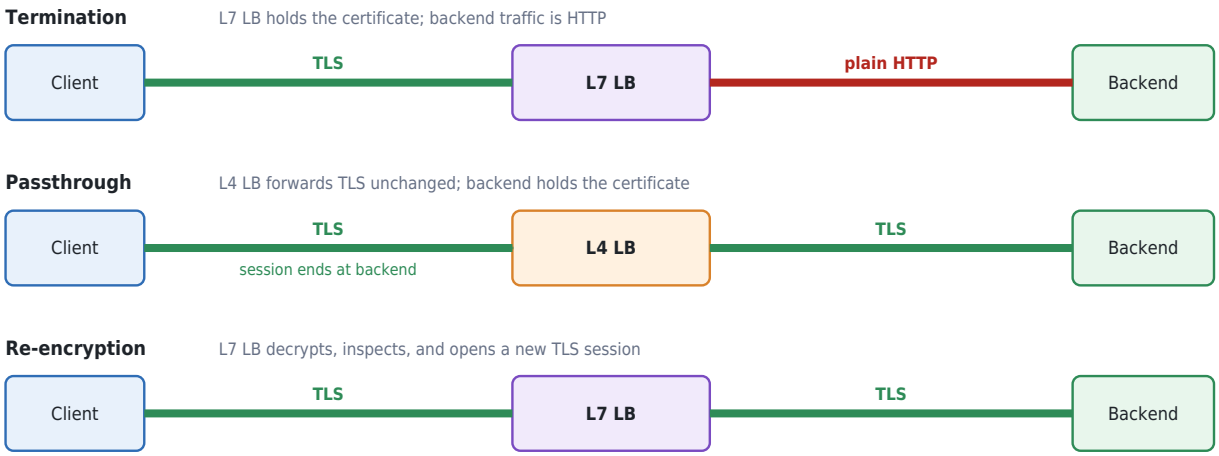


Figure 5.21. Termination removes TLS at the balancer, so the segment behind it is plain HTTP; passthrough keeps the session end to end but forces layer 4 balancing; re-encryption decrypts for inspection and opens a new TLS session to the back end.

- **Termination** centralises certificates and renewal and lets the balancer read HTTP for routing and caching. The internal hop is unprotected, which is acceptable only inside a trusted segment.
- **Passthrough** keeps the session end to end, so the balancer cannot inspect or route by HTTP (only by SNI), and every back end needs the certificate.
- **Re-encryption** decrypts at the balancer and connects to the back end over a new TLS session (often with mutual TLS), combining inspection and protection of the internal hop at the cost of two handshakes and double cryptographic work.

Gateways and service discovery

An **API gateway** is a layer 7 reverse proxy specialised for APIs. In addition to routing it performs authentication (validating JWT or API keys), rate limiting, request and response transformation, aggregation, retries, circuit breaking, logging and tracing, so that individual services need not implement these concerns.

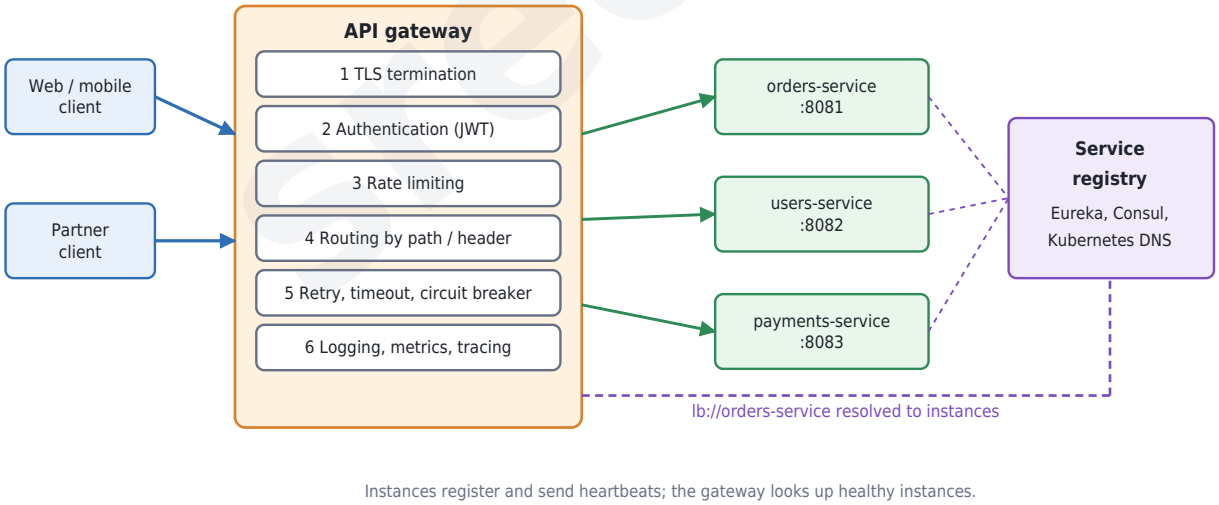


Figure 5.22. The gateway applies cross-cutting policies and routes each request to a service instance obtained from a registry that instances join on start-up and leave when they fail.

With dynamic instances (containers started and stopped by an orchestrator) addresses change constantly, so the balancer or gateway asks a **service registry** for the current instances. In **client-side discovery** (Spring Cloud LoadBalancer with Eureka or Consul) the caller fetches the list and balances itself. In **server-side discovery** a balancer or the platform does it: a Kubernetes `Service` gives a stable virtual IP and DNS name, and `kube-proxy` or the CNI distributes connections to ready pods.

5.5.2 Worked trace

The program `MiniLb.java` is a working layer 7 reverse proxy built with the JDK: a load balancer on port 9000 forwards requests round robin to three back ends on ports 9001 to 9003, adds `X-Forwarded-For`, and skips instances that a health check has marked down. The back end then stops, first without and then with the health check, to show the detection window.

MiniLb.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class MiniLb {
    static final int[] PORTS = {9001, 9002, 9003};
    static final boolean[] healthy = {true, true, true};
    static final AtomicInteger next = new AtomicInteger();
    static final HttpClient client =
            HttpClient.newBuilder().connectTimeout(Duration.ofMillis(300)).build();

    static HttpServer backend(int port) throws Exception {
        HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", port), 0);
        s.createContext("/", ex -> {
            String xff = ex.getRequestHeaders().getFirst("X-Forwarded-For");
            String host = ex.getRequestHeaders().getFirst("Host");
            byte[] b = ("backend:" + port + " saw X-Forwarded-For=" + xff + " Host=" + host)
                    .getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        s.start();
        return s;
    }

    static void healthCheck() {
        for (int i = 0; i < PORTS.length; i++) {
            try {
                HttpRequest probe = HttpRequest.newBuilder(
                        URI.create("http://127.0.0.1:" + PORTS[i] + "/health")).build();
                healthy[i] = client.send(probe, HttpResponse.BodyHandlers.discarding())
                        .statusCode() == 200;
            } catch (Exception e) {
                healthy[i] = false;
            }
        }
    }

    static String pick() {
        for (int tries = 0; tries < PORTS.length; tries++) {
            int i = Math.floorMod(next.getAndIncrement(), PORTS.length);
            if (healthy[i]) return "http://127.0.0.1:" + PORTS[i];
        }
        return null;
    }

    static void show(HttpClient user) throws Exception {
        for (int i = 0; i < 4; i++) {
            var r = user.send(HttpRequest.newBuilder(URI.create("http://127.0.0.1:9000/x")).build(),
                    HttpResponse.BodyHandlers.ofString());
            System.out.println(r.statusCode() + " " + r.body());
        }
    }

    public static void main(String[] args) throws Exception {
        List<HttpServer> backends = new ArrayList<>();
        for (int p : PORTS) backends.add(backend(p));
        HttpServer lb = HttpServer.create(new InetSocketAddress("127.0.0.1", 9000), 0);
```

MiniLb.java (continued) JAVA

```
        lb.createContext("/", ex -> {
            String target = pick();
            byte[] out;
            int code = 200;
            if (target == null) {
                out = "no healthy backend".getBytes(StandardCharsets.UTF_8);
                code = 503;
            } else {
                try {
                    HttpRequest rq = HttpRequest.newBuilder(URI.create(target + ex.getRequestURI()))
                            .header("X-Forwarded-For",
                                    ex.getRemoteAddress().getAddress().getHostAddress())
                            .build();
                    out = client.send(rq, HttpResponse.BodyHandlers.ofByteArray()).body();
                } catch (Exception e) {
                    out = "bad gateway".getBytes(StandardCharsets.UTF_8);
                    code = 502;
                }
            }
            ex.sendResponseHeaders(code, out.length);
            ex.getResponseBody().write(out);
            ex.close();
        });
        lb.start();

        HttpClient user = HttpClient.newHttpClient();
        System.out.println("-- round robin, all healthy");
        show(user);
        backends.get(1).stop(0);                      // backend 9002 fails
        System.out.println("-- 9002 stopped, before the health check runs");
        show(user);
        healthCheck();
        System.out.println("-- after the health check marks 9002 down");
        show(user);
        lb.stop(0);
        for (HttpServer s : backends) s.stop(0);
    }
}
```

Output of MiniLb.java TERMINAL

```
$ java MiniLb.java
-- round robin, all healthy
200 backend:9001 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9001
200 backend:9002 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9002
200 backend:9003 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9003
200 backend:9001 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9001
-- 9002 stopped, before the health check runs
502 bad gateway
200 backend:9003 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9003
200 backend:9001 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9001
502 bad gateway
-- after the health check marks 9002 down
200 backend:9003 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9003
200 backend:9001 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9001
200 backend:9003 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9003
200 backend:9001 saw X-Forwarded-For=127.0.0.1 Host=127.0.0.1:9001
```

What to notice, in order:

1. The requests rotate 9001, 9002, 9003 (round robin), and the balancer opens its own connection to each back end; the client never sees the back-end address.
2. `X-Forwarded-For=127.0.0.1` is the address of the original client, supplied by the proxy. Without this header the back end would see only the balancer address. The back end sees `Host=127.0.0.1:9001`, the back-end authority, because this proxy builds a new request; production proxies copy the original `Host` (nginx: `proxy_set_header Host $host;`) so that virtual hosting and redirect generation keep working.
3. After the 9002 instance stops and before any probe has run, every turn of the rotation that selects 9002 yields `502 bad gateway` (connection refused): one request in three fails, here the first and the fourth of the four requests. A balancer reports a back-end connection failure with 502, and 504 when the back end accepted the connection but did not answer within the timeout.

4. After the health check marks 9002 down, requests alternate between 9003 and 9001 and no error is returned. Clients see a failure only during the detection window, which is why balancers combine active checks with passive ejection and **retry** idempotent requests on another instance.

5.5.3 Hands-on

nginx as reverse proxy and load balancer

The configuration below (reference, not run here because nginx is not installed on the build machine) terminates TLS, balances two Spring Boot instances with least connections, passes the client address and original host, and sets timeouts and passive health checking.

nginx.conf (not run) CONF

```
upstream orders {
    least_conn;
    server 10.0.1.11:8080 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8080 max_fails=3 fail_timeout=10s;
    keepalive 32;                       # idle connections kept open to the back ends
}

server {
    listen 443 ssl;
    http2 on;
    server_name shop.example.com;
    ssl_certificate     /etc/ssl/shop.example.com.pem;
    ssl_certificate_key /etc/ssl/shop.example.com.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    location /api/ {
        proxy_pass http://orders;
        proxy_http_version 1.1;
        proxy_set_header Connection "";              # required for upstream keepalive
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;
        proxy_read_timeout    30s;
        proxy_next_upstream error timeout http_502 http_503;   # retry on another instance
    }
}
```

Directives that matter: `least_conn` selects the algorithm (`ip_hash` and `hash $request_uri consistent;` give affinity and consistent hashing); `max_fails` and `fail_timeout` implement passive health checks; `keepalive` reuses back-end connections and avoids a TCP handshake per request; `proxy_next_upstream` retries only on connection errors, timeouts and the listed status codes. `proxy_next_upstream` does not retry non-idempotent methods by default, which prevents duplicate `POST` requests.

Spring Boot behind a proxy

Because the application sees the proxy address, redirects and generated links would use `http://` and the internal host unless the forwarded headers are honoured. In Spring Boot the setting `server.forward-headers-strategy=framework` (or `native` for the container implementation) makes `request.getRemoteAddr()`, `isSecure()` and generated URLs reflect `X-Forwarded-For`, `X-Forwarded-Proto` and `X-Forwarded-Host`. The headers must be trusted only when they come from the proxy: a client can send its own `X-Forwarded-For`, so the proxy must overwrite or append and the application must not use the header for security decisions unless the immediate peer is a known proxy.

Spring Cloud Gateway

Spring Cloud Gateway is a reactive layer 7 gateway. A route consists of a predicate (which requests match) and filters (what happens to them). The YAML below sends `/api/orders/**` to a load-balanced service name resolved through discovery, limits the rate and retries idempotent requests. It is shown for reference and was not run here. The property namespace is `spring.cloud.gateway.server.webflux.*` in the 5.x line; earlier releases use `spring.cloud.gateway.routes`.

application.yml (not run) YAML

```yaml
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: orders
              uri: lb://orders-service
              predicates:
                - Path=/api/orders/**
              filters:
                - StripPrefix=1
                - name: Retry
                  args:
                    retries: 2
                    methods: GET
                    statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
                - name: RequestRateLimiter
                  args:
                    redis-rate-limiter.replenishRate: 10
                    redis-rate-limiter.burstCapacity: 20
```

The `RequestRateLimiter` filter is a token bucket stored in Redis with the parameters `replenishRate` (tokens per second) and `burstCapacity` (bucket size), the same algorithm as in the previous chapter; `lb://` asks the discovery client for instances of `orders-service`.

5.5.4 Failure modes

- **Detection window.** Between failure and the health check verdict, clients receive errors. Tune interval and threshold (for example probe every 5 s, down after 2 failures) and retry idempotent requests.
- **Health check that lies.** A liveness probe that only checks that the process answers keeps an instance whose database connection is broken in rotation. Readiness checks should include critical dependencies but not cascade (a database outage should not mark every instance of every service down).
- **Thundering herd after recovery.** A recovered instance with an empty cache and cold JIT receives a full share of traffic immediately. Use slow start (weight ramp-up) or gradual re-admission.
- **Idle timeout mismatch.** If the balancer idle timeout (for example 60 s) is shorter than the back-end keep-alive timeout, or the opposite, requests fail intermittently with 502 when a connection is closed at the moment it is reused. The back-end keep-alive timeout must be longer than the balancer idle timeout (Tomcat: `server.tomcat.keep-alive-timeout` and `server.tomcat.connection-timeout`).
- **Lost client address.** Logs and rate limits keyed on the balancer address penalise everyone at once. Use `X-Forwarded-For` (validated) or the PROXY protocol for layer 4.
- **Uneven distribution.** Long-lived connections (HTTP/2, gRPC, WebSocket) pin all requests of a client to one back end at layer 4, so new instances stay idle. Balance at layer 7 per request, or recycle connections periodically.
- **Single point of failure.** The balancer itself must be redundant: an active-passive pair with a virtual IP (VRRP), several instances behind anycast or DNS, or a managed cloud service.
- **Oversized headers and body limits.** Proxies enforce limits (nginx `client_max_body_size`, header buffer sizes) and answer `413` or `400` before the request reaches the application.

5.5.5 Questions to expect

1. What is the difference between a layer 4 and a layer 7 load balancer?

Layer 4 forwards connections on the basis of IP addresses and ports and never reads the payload; it is fast and protocol-agnostic. Layer 7 terminates the connection, reads the HTTP request and routes by host, path or header, with features such as TLS termination, retries, caching and rewriting, at higher CPU cost.

2. Compare round robin and least connections.

Round robin rotates through servers and is fair for uniform, short requests. Least connections routes to the server with the fewest active connections and copes with uneven request durations, because a server stuck on slow requests receives fewer new ones.

3. Why is consistent hashing used instead of hash mod N?

With `hash mod N` a change in N remaps most keys, which empties caches and breaks affinity. In a consistent hash ring only the keys adjacent to the added or removed server move (about 1/N of the keys), and virtual nodes make the distribution even.

4. How do health checks work, and what is the difference between liveness and readiness?

The balancer probes each instance at an interval and removes it after consecutive failures; passive checks eject instances based on live errors. Liveness tells the orchestrator that the process must be restarted; readiness tells the balancer that the instance must not receive traffic yet or any more.

5. What are sticky sessions and why are they avoided?

The balancer keeps a user on one instance, usually through a cookie. This is needed only when session state sits in instance memory. It leads to uneven load and lost sessions on failure or deployment, so shared session storage or stateless tokens are preferred.

6. Where should TLS terminate?

At the edge balancer in most designs, to centralise certificates and allow HTTP-level routing. Where internal traffic must be protected, the balancer re-encrypts to the back end, ideally with mutual TLS; passthrough keeps TLS end to end but limits the balancer to layer 4.

7. What does an API gateway do beyond load balancing?

It is the single entry point for APIs and applies authentication, rate limiting, routing, request transformation, response aggregation, circuit breaking, logging and tracing, so that services stay free of these concerns.

8. A user receives intermittent 502 errors after a deployment. What is checked?

Back-end instances that were stopped while still in rotation (health check interval and connection draining), a keep-alive timeout of the back end shorter than the idle timeout of the balancer, application crashes at start-up, and back-end listen backlog or thread pool exhaustion.

9. What is the difference between 502, 503 and 504?

502 means that the proxy received an invalid response or could not connect to the upstream; 503 means that the service or the proxy is unavailable or overloaded (no healthy back end); 504 means that the upstream did not answer within the proxy timeout.

KEY POINTS

- A reverse proxy represents servers; a load balancer distributes requests; an API gateway adds API policy. One product often does all three.
- Layer 4 balancing decides on addresses and ports and keeps TLS opaque; layer 7 terminates connections, reads HTTP and can route, retry and cache.
- Round robin suits uniform requests, least connections adapts to uneven ones, and consistent hashing moves only about 1/N of the keys when the server set changes.
- Health checks bound but do not remove the failure window; combine active and passive checks with retries of idempotent requests.
- TLS may terminate, pass through or be re-encrypted; always forward the client address and the original host and trust those headers only from the proxy.
- Keep-alive and idle timeouts must be consistent across the chain, or intermittent 502 responses result.

5.6 CDNs, Caching and High Availability

Two physical limits shape the delivery of web content: the propagation delay between a user and a distant server, and the capacity and reliability of a single data centre. A **content delivery network** (CDN) addresses the first by placing caching servers close to users. **High availability** (HA) addresses the second by removing single points of failure and by making clients and services tolerate the failures that remain. The two topics meet in one design principle: every component can fail, so every call needs a timeout, a bounded retry policy and a fallback.

5.6.1 Concept and mechanism

How a CDN serves a request

A CDN is a geographically distributed set of **points of presence** (PoP), each with caching reverse proxies (**edge servers**). The site owner points the public name at the CDN, usually with a `CNAME` record (`www.example.com` to a CDN host name) or by moving the DNS zone to the CDN. The user is directed to a nearby PoP by DNS-based steering (the answer depends on the resolver location) or by anycast. The edge server looks up the object in its cache using a **cache key** (normally scheme, host, path and selected query parameters and headers). On a hit it answers from memory or disk; on a miss it fetches the object from the **origin**, stores it according to the response headers, and returns it. Large CDNs insert an **origin shield**, a regional cache that merges the misses of many edges so that the origin sees one request per object per shield.

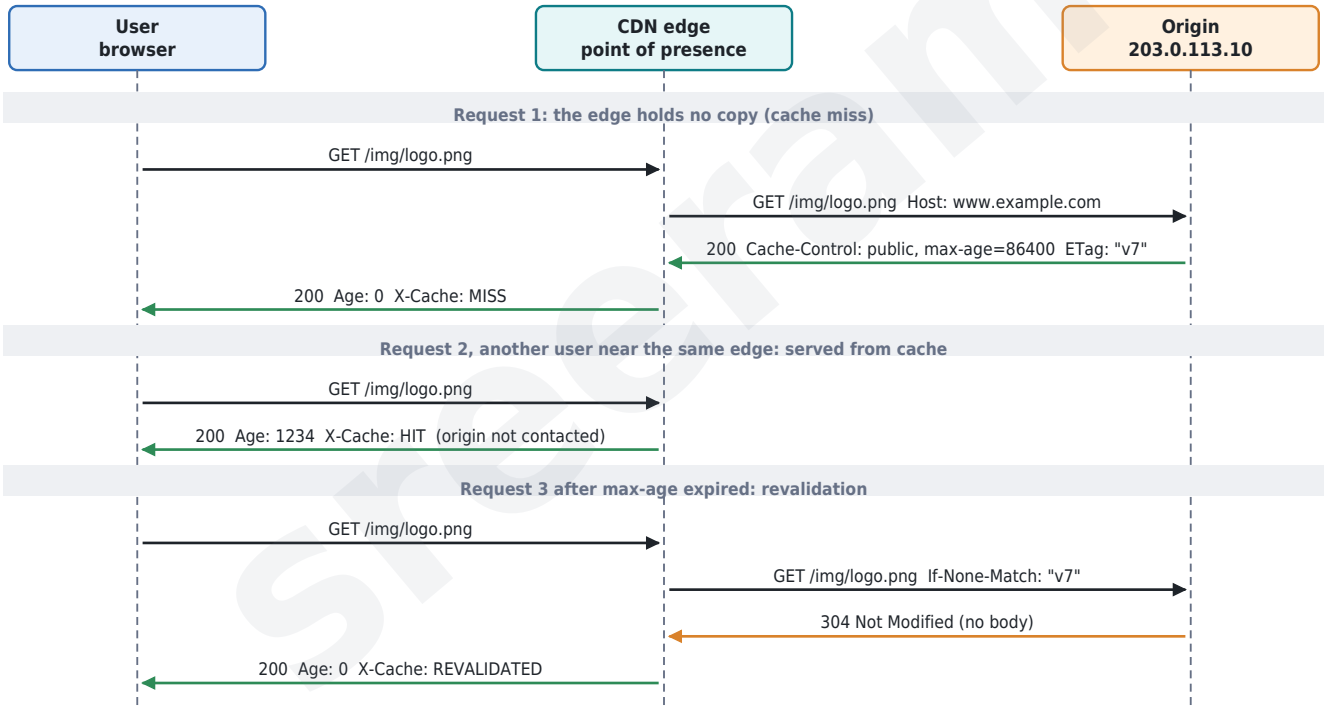


Figure 5.23. The first request to an edge is a cache miss that travels to the origin; later requests are served from the edge without contacting the origin; after expiry the edge revalidates with a conditional request and receives a bodiless 304.

A CDN reduces latency (fewer round trips over long distances, and TCP and TLS connections terminate near the user), reduces origin load and bandwidth, and absorbs traffic spikes and denial-of-service floods because the edge capacity exceeds that of a single origin. Most CDNs also terminate TLS, speak HTTP/2 and HTTP/3 to browsers, and offer a web application firewall.

Cache control headers

Caching behaviour is controlled by response headers (HTTP caching is defined in RFC 9111; the validators `ETag` and `Last-Modified` were covered in the earlier Part on HTTP caching).

Directive	Meaning
`max-age=N`	The response is fresh for N seconds in any cache, including the browser.
`s-maxage=N`	Freshness for shared caches (CDN, proxy) only; overrides `max-age` there. Allows a long CDN lifetime and a short browser lifetime.

Directive	Meaning
`public` / `private`	`private` forbids shared caches (per-user content); `public` permits them even for requests with an `Authorization` header.
`no-cache`	The cache may store the response but must revalidate before every reuse.
`no-store`	Do not store the response at all (sensitive data).
`stale-while-revalidate=N`	After expiry, serve the stale copy for up to N seconds while refreshing in the background (RFC 5861).
`stale-if-error=N`	Serve a stale copy for N seconds when the origin returns an error or is unreachable (RFC 5861).
`immutable`	The content will not change during its freshness lifetime; browsers skip revalidation on reload (RFC 8246).
`Vary: Accept-Encoding`	The cache key also includes the named request headers; `Vary: Cookie` makes the cache nearly useless.

Cache-Control directives that matter at the edge.

A frequent production pattern is versioned asset names (`app.4f3a9c.js`) served with `Cache-Control: public, max-age=31536000, immutable`, while the HTML document that references them is served with `no-cache` or a short `s-maxage`. A new deployment changes the file name, so no purge is needed. Objects that must change at a known time are removed with the purge (invalidation) API of the CDN.

Anycast

With **anycast** the same IP address is announced by BGP from many locations. Each router forwards packets towards the topologically nearest announcement, so users reach the closest PoP without any DNS logic, and a PoP that fails withdraws its route and traffic moves to the next nearest one. Anycast is the standard for public DNS resolvers and CDN edges, and it spreads the traffic of a DDoS attack over many sites. The limitation is that a routing change in the middle of a long TCP connection can move packets to a different PoP, which then has no state for that connection and answers with a reset; for this reason anycast suits short connections and QUIC, and CDNs add connection-aware forwarding between PoPs.

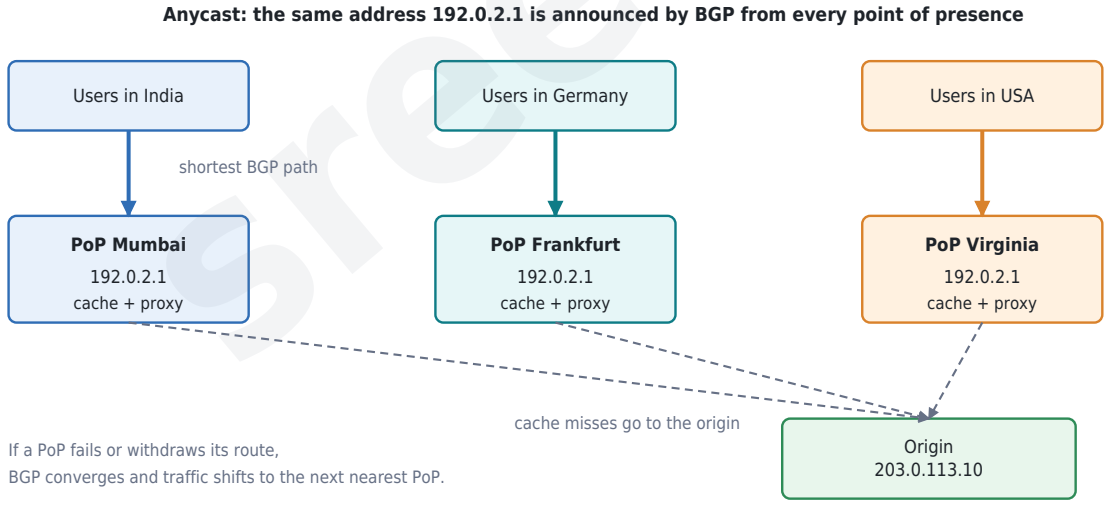


Figure 5.24. All points of presence announce the same address, and BGP delivers each user to the nearest one; cache misses are served from the origin, and a failed PoP is bypassed after routing converges.

Redundancy and failover

High availability is obtained by redundancy at every tier, with failure detection and a switch-over mechanism.

- **Active-active**: all nodes serve traffic (instances behind a balancer, multiple availability zones). Capacity must allow for the loss of a node (N+1 or N+2).
- **Active-passive**: a standby takes over on failure. A pair of balancers or routers shares a virtual IP address through VRRP or keepalived; when the master stops sending advertisements (default interval one second, three missed advertisements) the backup claims the address and announces it with gratuitous ARP.

- **Multi-zone and multi-region**: independent failure domains. Cross-region designs add data replication and the difficulty of consistency.
- **Split brain**: if the link between the two nodes fails but both stay alive, both may claim to be active. Quorum or a third arbiter prevents it.

Failover can happen at the DNS layer or at the balancer layer, and the two differ greatly in speed. DNS failover changes the answer of a name when a health check fails, but resolvers and clients cache the old answer until the **TTL** expires, and some clients and JVMs cache longer than the TTL (the Java property `networkaddress.cache.ttl` and the security setting of the same name). Balancer-level failover (virtual IP, health-checked pool) switches in seconds because the address does not change.

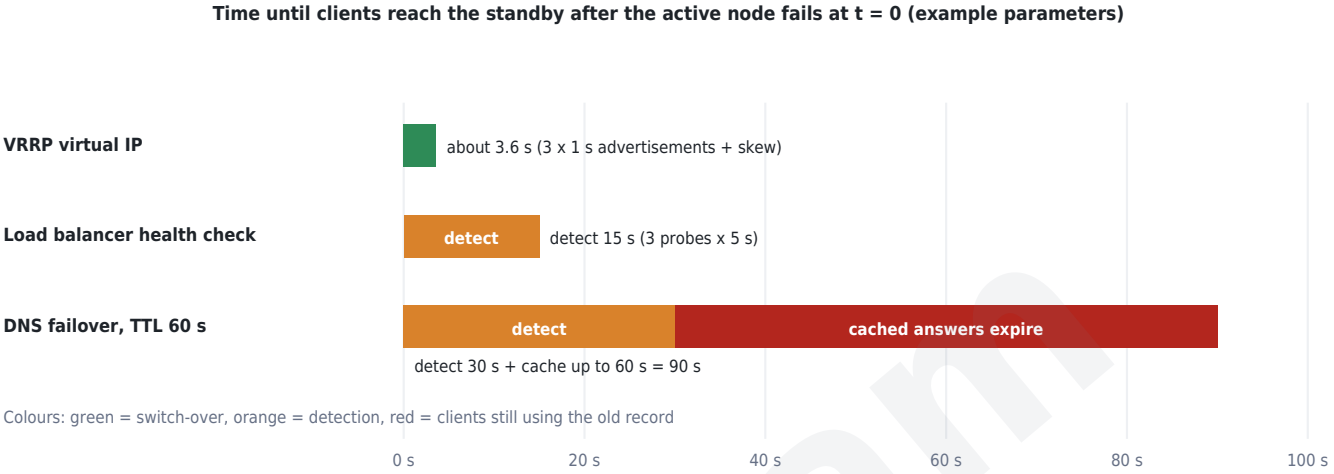


Figure 5.25. A virtual IP moves in a few seconds, a health-checked load balancer removes a failed instance after the probe interval, whereas DNS failover adds the record TTL on top of detection time (example parameters).

Timeouts, retries and backoff

Every network call needs a **timeout**, because a call that never completes holds a thread, a connection and memory. A **retry** repairs transient failures (packet loss, a restarting instance, a momentary overload), but retries multiply load, so they must be limited and spaced. The standard schedule is **exponential backoff** (delay doubles after each attempt up to a cap) with **jitter** (the delay is randomised, so that clients that failed together do not retry together). With full jitter the delay for attempt n is a uniform random value between 0 and min(cap, base x 2^n).

Retries are safe only for **idempotent** operations (see the REST chapter of the HTTP Part): `GET`, `PUT` and `DELETE` are idempotent by definition; `POST` is not, so retrying it can create duplicates. A client-generated **idempotency key** sent in a header (for example `Idempotency-Key: 7c9e6679`) allows the server to recognise a repeat and return the original result, which makes retrying a `POST` safe.

The retry storm

When a dependency slows down, every caller times out and retries. If three tiers each retry three times, one user request becomes 3 x 3 x 3 = 27 requests at the bottom tier, which is already overloaded. The extra load keeps it down after the original cause has gone: the failure is sustained by the retries (a **metastable failure**). The countermeasures are layered.

- Retry at one layer only (usually the one closest to the user), never at every tier.
- Cap retries with a **retry budget** (for example retries may not exceed 10 percent of the request rate).
- Use exponential backoff with jitter.
- Use a **circuit breaker** to stop calls to a failing dependency.
- Propagate a deadline, so that work for a caller that has already given up is dropped.
- Shed load at the server (return 503 quickly) instead of queueing unboundedly.

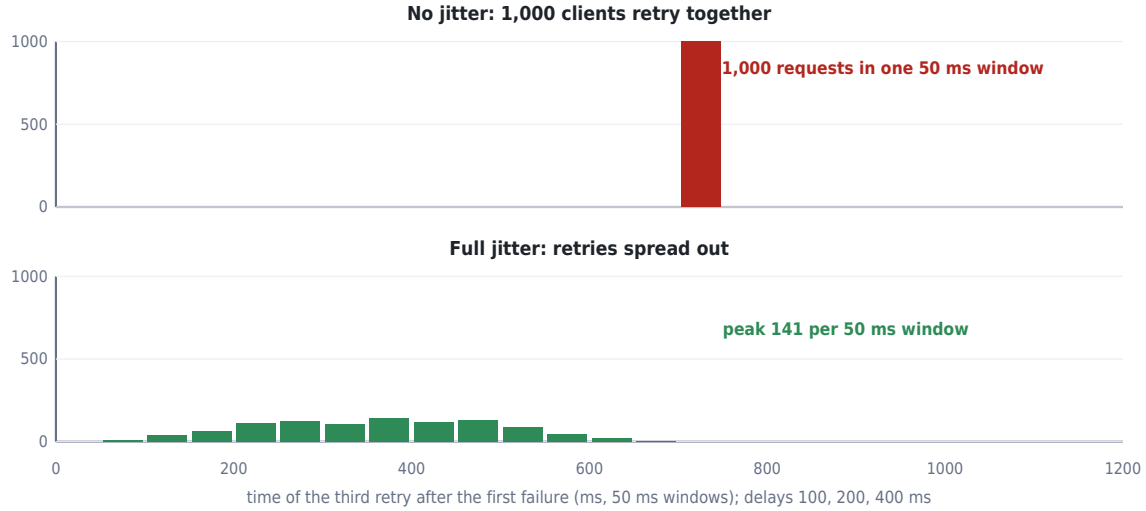


Figure 5.26. Without jitter all 1,000 clients that failed together retry in the same 50 ms window; full jitter spreads the same retries over about 650 ms with a peak of 141 per window.

Circuit breakers

A **circuit breaker** wraps calls to a dependency and tracks their outcomes. In the **closed** state calls pass through. When failures exceed a threshold the breaker **opens**: calls fail at once (or return a fallback) without touching the dependency, which relieves it and avoids holding threads for the full timeout. After a wait period the breaker becomes **half-open** and allows a few trial calls; if they succeed it closes, otherwise it opens again.

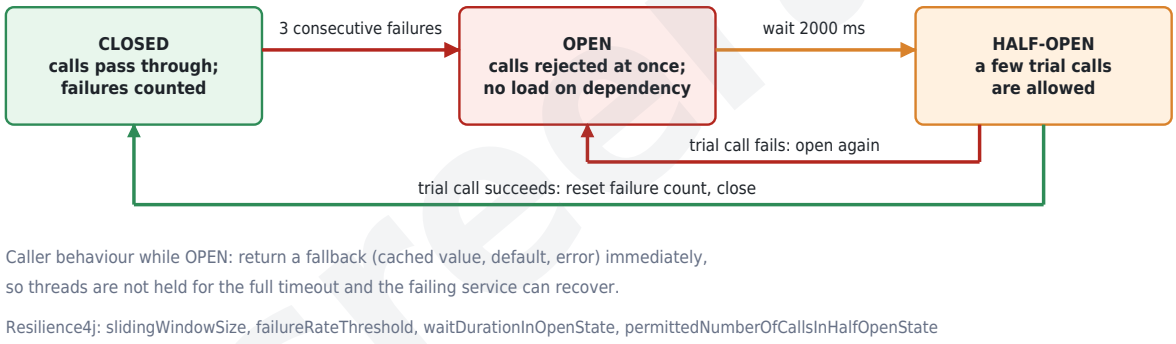


Figure 5.27. The breaker opens after three consecutive failures, rejects calls for 2000 ms, then allows a trial call whose result decides between closing and reopening.

5.6.2 Worked trace

An edge cache with real headers

The program implements a miniature CDN edge: an origin on port 9201 sends `Cache-Control: public, max-age=2` and a body that contains a version counter; an edge on port 9200 caches responses by path and honours `max-age`. A client requests the same path five times with pauses, and the edge adds `X-Cache` and `Age` headers.

EdgeCache.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;
```

EdgeCache.java (continued) JAVA

```java
public class EdgeCache {
    record Entry(byte[] body, long storedAtMs, long maxAgeSec) {}

    static final AtomicInteger originHits = new AtomicInteger();
    static final Map<String, Entry> cache = new ConcurrentHashMap<>();
    static final HttpClient http = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        HttpServer origin = HttpServer.create(new InetSocketAddress("127.0.0.1", 9201), 0);
        origin.createContext("/", ex -> {
            String text = "price list v" + originHits.incrementAndGet();
            byte[] b = text.getBytes(StandardCharsets.UTF_8);
            ex.getResponseHeaders().add("Cache-Control", "public, max-age=2");
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        origin.start();

        HttpServer edge = HttpServer.create(new InetSocketAddress("127.0.0.1", 9200), 0);
        edge.createContext("/", ex -> {
            String key = ex.getRequestURI().toString();
            long now = System.currentTimeMillis();
            Entry e = cache.get(key);
            String state;
            if (e != null && now - e.storedAtMs() < e.maxAgeSec() * 1000) {
                state = "HIT";
            } else {
                try {
                    var req = HttpRequest.newBuilder(
                            URI.create("http://127.0.0.1:9201" + key)).build();
                    var r = http.send(req, HttpResponse.BodyHandlers.ofByteArray());
                    e = new Entry(r.body(), now, 2);
                    cache.put(key, e);
                    state = "MISS";
                } catch (Exception ex2) {
                    ex.sendResponseHeaders(502, -1);
                    ex.close();
                    return;
                }
            }
            ex.getResponseHeaders().add("X-Cache", state);
            ex.getResponseHeaders().add("Age", String.valueOf((now - e.storedAtMs()) / 1000));
            ex.sendResponseHeaders(200, e.body().length);
            ex.getResponseBody().write(e.body());
            ex.close();
        });
        edge.start();

        HttpClient user = HttpClient.newHttpClient();
        long[] pauses = {0, 1100, 600, 1000, 100};
        for (int i = 0; i < pauses.length; i++) {
            Thread.sleep(pauses[i]);
            var req = HttpRequest.newBuilder(
                    URI.create("http://127.0.0.1:9200/prices.json")).build();
            var r = user.send(req, HttpResponse.BodyHandlers.ofString());
            System.out.printf("request %d: X-Cache=%s Age=%s body=\"%s\" origin hits so far=%d%n",
                    i + 1, r.headers().firstValue("x-cache").get(),
                    r.headers().firstValue("age").get(), r.body(), originHits.get());
        }
        edge.stop(0);
        origin.stop(0);
    }
}
```

Output of EdgeCache.java TERMINAL

```
$ java EdgeCache.java
request 1: X-Cache=MISS Age=0 body="price list v1" origin hits so far=1
request 2: X-Cache=HIT Age=1 body="price list v1" origin hits so far=1
request 3: X-Cache=HIT Age=1 body="price list v1" origin hits so far=1
request 4: X-Cache=MISS Age=0 body="price list v2" origin hits so far=2
request 5: X-Cache=HIT Age=0 body="price list v2" origin hits so far=2
```

Reading the result: request 1 finds an empty cache (`MISS`) and fills it from the origin. Requests 2 and 3, at 1.1 s and 1.7 s, are served from the edge (`HIT`) with a growing `Age`, and the origin hit counter does not change. Request 4 arrives at 2.7 s, after the two-second lifetime, so the edge fetches again (`MISS`, version 2). Request 5 is a hit once more. A real edge would also store `ETag`, send `If-None-Match` and accept a `304` to refresh without transferring the body, as in the sequence diagram.

Backoff and jitter

The program computes the delays of exponential backoff (base 100 ms, doubling, cap 5,000 ms) with and without full jitter, and compares the timing of the third retry of 1,000 clients that all failed at time 0.

Backoff.java JAVA

```java
import java.util.Random;

public class Backoff {
    static final long BASE_MS = 100, CAP_MS = 5000;

    static long delay(int attempt, boolean jitter, Random rnd) {
        long exp = Math.min(CAP_MS, BASE_MS << attempt);       // 100, 200, 400, 800, 1600 ...
        return jitter ? (long) (rnd.nextDouble() * exp) : exp; // full jitter: uniform in [0, exp)
    }

    public static void main(String[] args) {
        Random rnd = new Random(7);
        System.out.println("attempt  plain-delay-ms  full-jitter-sample-ms");
        for (int a = 0; a < 6; a++)
            System.out.printf("%7d  %14d  %21d%n",
                    a + 1, delay(a, false, rnd), delay(a, true, rnd));

        int clients = 1000;
        for (boolean jitter : new boolean[] {false, true}) {
            int[] perBucket = new int[100];          // 50 ms buckets, third retry only
            for (int c = 0; c < clients; c++) {
                long t = 0;
                for (int a = 0; a < 3; a++) t += delay(a, jitter, rnd);
                perBucket[(int) Math.min(99, t / 50)]++;
            }
            int peak = 0, used = 0;
            for (int v : perBucket) { peak = Math.max(peak, v); if (v > 0) used++; }
            System.out.printf("%s: third retry of %d clients lands in %d distinct 50 ms windows, "
                    + "peak %d%n", jitter ? "full jitter" : "no jitter  ", clients, used, peak);
        }
        System.out.println("Layered retries: 3 attempts at each of 3 tiers = "
                + (int) Math.pow(3, 3) + " calls reach the database per user request");
    }
}
```

Output of Backoff.java TERMINAL

```
$ java Backoff.java
attempt  plain-delay-ms  full-jitter-sample-ms
      1             100                     73
      2             200                    149
      3             400                    139
      4             800                    717
      5            1600                   1133
      6            3200                   1126
no jitter  : third retry of 1000 clients lands in 1 distinct 50 ms windows, peak 1000
full jitter: third retry of 1000 clients lands in 13 distinct 50 ms windows, peak 141
Layered retries: 3 attempts at each of 3 tiers = 27 calls reach the database per user request
```

Without jitter the whole population retries at the same instant (here 100 + 200 + 400 = 700 ms): the dependency sees a synchronised spike of 1,000 requests. With full jitter the same retries are spread over 13 windows with a maximum of 141. The jitter samples vary with the random seed; the figures above are for seed 7.

A circuit breaker under a dependency outage

The simulation drives a breaker (open after 3 consecutive failures, stay open for 2,000 ms) with one call every 500 ms. The dependency is down between 2,000 ms and 6,000 ms. The state column shows the state before each call.

Breaker.java JAVA

```java
public class Breaker {
    enum State { CLOSED, OPEN, HALF_OPEN }

    State state = State.CLOSED;
    int failures = 0;
    long openedAt = 0;
    final int threshold;
    final long openMs;

    Breaker(int threshold, long openMs) { this.threshold = threshold; this.openMs = openMs; }

    // returns what happened to the call at virtual time t
    String call(long t, boolean dependencyHealthy) {
        if (state == State.OPEN) {
            if (t - openedAt < openMs) return "rejected immediately (OPEN)";
            state = State.HALF_OPEN;                       // allow one trial call
        }
        boolean probe = state == State.HALF_OPEN;
        if (dependencyHealthy) {
            state = State.CLOSED;
            failures = 0;
            return probe ? "probe succeeded, circuit CLOSED" : "success";
        }
        failures++;
        if (probe || failures >= threshold) {
            state = State.OPEN;
            openedAt = t;
            return probe ? "probe failed, circuit OPEN again"
                         : "failure " + failures + ", circuit OPEN";
        }
        return "failure " + failures + " (timeout after 2000 ms)";
    }

    public static void main(String[] args) {
        Breaker b = new Breaker(3, 2000);
        for (long t = 0; t <= 9000; t += 500) {
            boolean healthy = t < 2000 || t >= 6000;       // dependency down from 2 s to 6 s
            System.out.printf("t=%4d ms  dependency %-4s  %-9s -> %s%n", t, healthy ? "up" : "DOWN",
                    b.state, b.call(t, healthy));
        }
    }
}
```

Output of Breaker.java TERMINAL

```
$ java Breaker.java
t=   0 ms  dependency up    CLOSED    -> success
t= 500 ms  dependency up    CLOSED    -> success
t=1000 ms  dependency up    CLOSED    -> success
t=1500 ms  dependency up    CLOSED    -> success
t=2000 ms  dependency DOWN  CLOSED    -> failure 1 (timeout after 2000 ms)
t=2500 ms  dependency DOWN  CLOSED    -> failure 2 (timeout after 2000 ms)
t=3000 ms  dependency DOWN  CLOSED    -> failure 3, circuit OPEN
t=3500 ms  dependency DOWN  OPEN      -> rejected immediately (OPEN)
t=4000 ms  dependency DOWN  OPEN      -> rejected immediately (OPEN)
t=4500 ms  dependency DOWN  OPEN      -> rejected immediately (OPEN)
t=5000 ms  dependency DOWN  OPEN      -> probe failed, circuit OPEN again
t=5500 ms  dependency DOWN  OPEN      -> rejected immediately (OPEN)
t=6000 ms  dependency up    OPEN      -> rejected immediately (OPEN)
t=6500 ms  dependency up    OPEN      -> rejected immediately (OPEN)
t=7000 ms  dependency up    OPEN      -> probe succeeded, circuit CLOSED
t=7500 ms  dependency up    CLOSED    -> success
t=8000 ms  dependency up    CLOSED    -> success
t=8500 ms  dependency up    CLOSED    -> success
t=9000 ms  dependency up    CLOSED    -> success
```

What to notice. The three failures between 2,000 and 3,000 ms each cost a full timeout. After the breaker opens, calls at 3,500, 4,000 and 4,500 ms are rejected immediately and cost nothing. At 5,000 ms the wait has elapsed and a trial call is sent, which fails because the dependency is still down, so the breaker reopens. At 7,000 ms the next trial call succeeds and the circuit closes. During the four seconds of outage the dependency receives four calls (three failures that opened the breaker and one trial call) instead of eight, and recovery is detected one wait period after the dependency returns.

5.6.3 Hands-on

Cache headers for a Spring Boot application

Spring MVC sets cache headers per response. The controller below marks a catalogue response as cacheable by browsers for 60 seconds and by CDNs for 10 minutes, with stale content served for a minute during revalidation (reference code, not run here).

CatalogController.java (not run) JAVA

```java
import java.time.Duration;
import java.util.List;
import org.springframework.http.CacheControl;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CatalogController {
    @GetMapping("/catalog")
    ResponseEntity<List<String>> catalog() {
        CacheControl cc = CacheControl.maxAge(Duration.ofSeconds(60))
                .sMaxAge(Duration.ofMinutes(10))
                .staleWhileRevalidate(Duration.ofSeconds(60))
                .cachePublic();
        return ResponseEntity.ok().cacheControl(cc).body(List.of("book", "pen"));
    }
}
```

Retries and circuit breakers with Resilience4j

In a Spring Boot service the retry and breaker policies are normally configured rather than hand-coded. The properties below (Resilience4j Spring Boot starter, reference sample, not run) define a breaker that opens when half of the last 20 calls fail, waits 10 seconds, and a retry policy of three attempts with exponential backoff and randomisation.

application.properties (not run) PROPERTIES

```properties
resilience4j.circuitbreaker.instances.inventory.sliding-window-size=20
resilience4j.circuitbreaker.instances.inventory.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.inventory.wait-duration-in-open-state=10s
resilience4j.circuitbreaker.instances.inventory.permitted-number-of-calls-in-half-open-state=3
resilience4j.retry.instances.inventory.max-attempts=3
resilience4j.retry.instances.inventory.wait-duration=200ms
resilience4j.retry.instances.inventory.enable-exponential-backoff=true
resilience4j.retry.instances.inventory.exponential-backoff-multiplier=2
resilience4j.retry.instances.inventory.enable-randomized-wait=true
```

Failover checks

To measure real failover behaviour, stop one instance and watch the client: the time until errors cease is the detection interval plus, for DNS, the TTL. The JVM caches successful lookups for 30 seconds by default when no security manager is installed (`networkaddress.cache.ttl` in `java.security`, default 30 in recent JDKs), and failed lookups for 10 seconds (`networkaddress.cache.negative.ttl`); services that depend on DNS failover must set these values explicitly, for example to 10 seconds, with the system property `-Dsun.net.inetaddr.ttl=10` or in the `java.security` file.

5.6.4 Failure modes

- **Cache poisoning of personalised content.** A response containing user data cached as `public` and served to other users. Symptom: users see each other pages. Prevention: `private` or `no-store` for personalised responses; `Vary` only on stable headers.
- **Cache fragmentation.** Tracking parameters, cookies or `Vary: Cookie` give every request a unique key and a zero hit ratio, so all traffic reaches the origin. Normalise the cache key.
- **Stale content after deployment.** Long lifetimes without versioned names or purge. Use content-hashed file names for assets and short or revalidated lifetimes for HTML.
- **Cache stampede.** When a popular object expires, many edges and requests hit the origin at once. Mitigation: request collapsing at the edge, origin shield, `stale-while-revalidate`, jittered lifetimes.
- **Failover slower than expected.** DNS TTLs and client-side caches delay switch-over far beyond the health-check interval. Lower the TTL before planned changes and verify resolver behaviour in the client runtime.

- **Retry storm.** Described above. Symptom: load stays high after the original fault has been fixed; remedy: remove retries at inner tiers, add budgets and jitter, shed load.
- **Timeouts longer than the caller deadline.** A service that waits 30 seconds for a dependency while its own caller gives up after 5 seconds keeps working for nobody. Inner timeouts must be shorter than outer ones.
- **Untested failover.** A standby that has never taken traffic often fails to do so (missing configuration, expired certificate, cold cache). Exercise failover regularly.
- **Breaker misconfiguration.** A threshold that is too sensitive opens on normal error noise; a wait period that is too long keeps a recovered dependency isolated.

5.6.5 Questions to expect

1. How does a CDN reduce latency?

It serves content from an edge server near the user, so the request travels a short distance and TCP and TLS handshakes complete over a short round-trip time; cache hits avoid the origin entirely, and the edge keeps warm connections to the origin for misses.

2. Explain `max-age`, `s-maxage` and `no-cache`.

`max-age` sets freshness in seconds for all caches; `s-maxage` does so for shared caches only and overrides `max-age` there; `no-cache` allows storage but requires revalidation (for example with `If-None-Match`) before each reuse, whereas `no-store` forbids storage.

3. What is anycast and what is its weakness?

One address is announced from many sites and routing delivers packets to the nearest. It gives low latency, automatic failover and distribution of attack traffic. A routing change during a long TCP connection can send later packets to a different site that has no state for the connection.

4. DNS failover versus load balancer failover?

DNS failover changes the answer after a failed health check but is delayed by the TTL and by client caches (minutes). A load balancer or virtual IP keeps the address and removes the failed instance in seconds. Both are often combined: DNS or anycast selects the site, a balancer selects the instance.

5. What is a retry storm and how is it prevented?

A slowdown causes timeouts, retries add load, and the extra load prolongs the slowdown. Layered retries multiply (3 x 3 x 3 = 27). Prevention: retry at one layer, use exponential backoff with jitter, limit retries with a budget, use circuit breakers and propagate deadlines.

6. Why add jitter to backoff?

Clients that fail together would otherwise retry together at 100, 300 and 700 ms and produce synchronised spikes. Random delays spread the retries over time, which lowers the peak load on the recovering service.

7. Which requests may be retried automatically?

Idempotent ones: `GET`, `HEAD`, `PUT`, `DELETE` and `OPTIONS`. A `POST` may be retried only if the server supports an idempotency key. A retry after a timeout is ambiguous, because the first attempt may have succeeded.

8. Describe the states of a circuit breaker.

Closed: calls pass and failures are counted. Open: calls fail fast without reaching the dependency. Half-open after a wait: a few trial calls decide whether to close the breaker (success) or reopen it (failure).

9. What is split brain and how is it avoided?

Two nodes of an active-passive pair both believe that they are active, usually after the link between them fails. Quorum (an odd number of voters), a witness node or fencing (power or network isolation of the other node) prevent it.

freeram

KEY POINTS

- A CDN serves cached content from nearby edges; behaviour is governed by `Cache-Control` (`max-age`, `s-maxage`, `stale-while-revalidate`) and `Vary`; versioned asset names avoid purges.
- Anycast routes users to the nearest PoP and removes failed sites by BGP withdrawal; DNS failover is bounded below by TTL and client caches.
- High availability needs redundancy, health checks and rehearsed failover; a virtual IP fails over in seconds, DNS in minutes.
- Every call needs a timeout; retries need backoff with jitter, budgets and idempotency; inner timeouts are shorter than outer ones.
- Layered retries multiply load (27 calls from 3 x 3 x 3) and sustain outages; circuit breakers and load shedding break the cycle.

5.7 Network Design for Microservices

In a monolith, a method call costs nanoseconds and fails only when the process fails. In a system of microservices the same call crosses a network: it costs milliseconds, can be slow, lost, duplicated or answered after the caller has given up, and it can fail while the caller and callee are both healthy. Network design for microservices consists of choosing how services find and call each other, bounding every wait, reusing connections economically, and making latency visible. The classic list of assumptions that do not hold (the network is reliable, latency is zero, bandwidth is infinite, the topology does not change) summarises the problem.

5.7.1 Concept and mechanism

North-south and east-west traffic

North-south traffic crosses the boundary of the cluster: a browser or mobile client talks to a gateway or ingress. **East-west** traffic stays inside: service to service, service to database, cache or message broker. A single external request usually fans out to several internal calls, so east-west traffic volume commonly exceeds north-south traffic by a large factor. The two have different controls: north-south traffic needs TLS from the public Internet, authentication, rate limiting and a WAF at the gateway; east-west traffic needs service identity (mutual TLS), authorisation between services, short timeouts and retries, and network policies that restrict which service may call which.

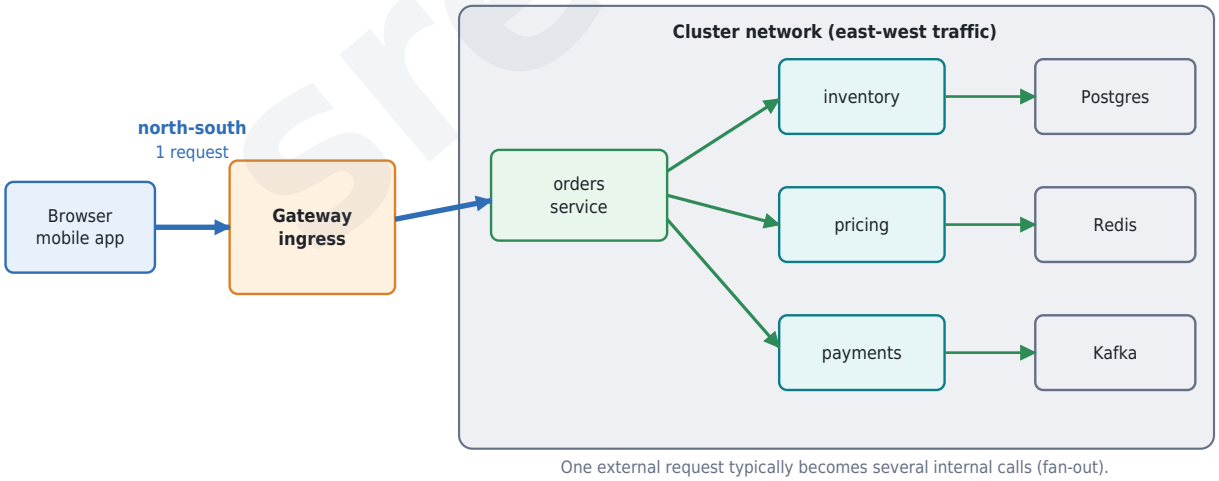


Figure 5.28. One external request (north-south) enters through the gateway and fans out into several internal calls (east-west) towards other services and data stores.

Service-to-service calls and discovery

Services call each other synchronously (HTTP/JSON, gRPC) or asynchronously through a broker (Kafka, RabbitMQ). Synchronous calls are simple but couple availability and latency: the caller is slow when the callee is slow. Asynchronous messaging decouples the two at the cost of eventual consistency and operational complexity.

In Kubernetes a `Service` gives a set of pods a stable name and virtual IP: `orders.shop.svc.cluster.local` resolves to a cluster IP, and `kube-proxy` (iptables or IPVS rules) or the CNI distributes each new **connection** to a ready pod. Two consequences matter for Java clients. First, distribution is per connection, not per request: an HTTP/1.1 client with a

keep-alive pool or a gRPC client with one long-lived HTTP/2 connection keeps talking to the same pod, so a scaled-up deployment may receive no traffic from existing clients. Second, a **headless service** (`clusterIP: None`) returns the pod addresses in DNS, which allows client-side balancing, subject to the JVM DNS cache (see the previous chapter). Alternatives are a layer 7 proxy or a service mesh that balances per request.

Fan-out and tail latency

If a request calls n services in parallel and waits for all of them, the request is slow whenever any one call is slow. When each call exceeds its 99th-percentile latency in 1 percent of cases, the probability that at least one of n calls is in the slow tail is 1 - 0.99^n.

Parallel calls (n)	Each call slow in 1 percent of cases	Each call slow in 0.1 percent of cases
1	1.0 percent	0.1 percent
5	4.9 percent	0.5 percent
20	18.2 percent	2.0 percent
50	39.5 percent	4.9 percent
100	63.4 percent	9.5 percent

Probability that a fan-out request meets at least one slow call.

The calculation (1 - 0.99^n, 1 - 0.999^n) assumes independent calls. It explains why p99 and p99.9 of individual services matter more than their averages once calls are chained or fanned out, and why timeouts and hedged requests are applied to the tail.

Timeouts and deadlines

A client call has several distinct waiting phases, each with its own timeout.

- **Connect timeout**: establishing the TCP connection (and often the TLS handshake). Fails when the peer or a firewall does not answer the SYN. A refused connection fails at once; a filtered one waits for the full timeout.
- **Pool acquisition timeout**: waiting for a free connection from the client pool.
- **Read (response) timeout**: waiting for response data after the request was sent. This detects a peer that accepted the connection and then stalled.
- **Total deadline**: the upper bound for the whole operation including retries.

Without explicit values many libraries wait indefinitely (the JDK `HttpClient` blocks forever when a request has no `timeout`, as stated in its documentation) or for very long periods (operating-system TCP defaults). Timeouts must **nest**: each inner call must be given less time than the caller will wait, otherwise the callee works for a caller that has already given up. Many frameworks therefore propagate a **deadline** (gRPC does it natively) so that each hop knows the remaining budget.

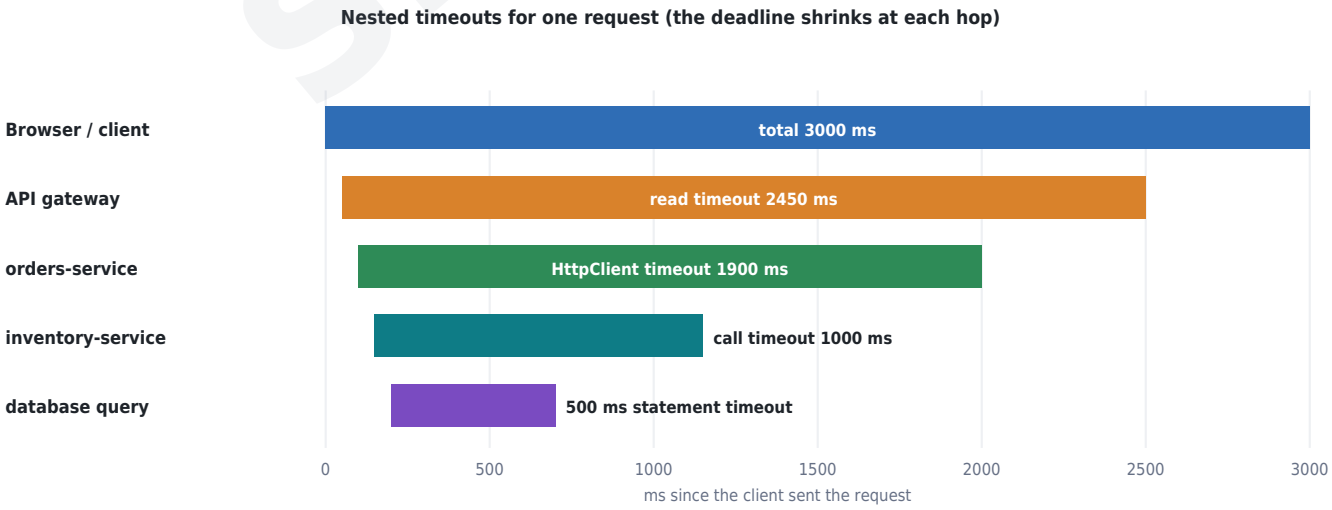


Figure 5.29. Each tier waits less than the tier above it, so the innermost call fails first and the error propagates upwards in time for the outer tiers to respond or retry within their own budgets.

Connection pools and keep-alive

Creating a connection costs a TCP handshake (one round trip), a TLS handshake (one more in TLS 1.3, two in TLS 1.2) and a slow-start phase. A **connection pool** keeps connections open and reuses them for later requests; HTTP/1.1 keep-alive and HTTP/2 multiplexing make this possible. A pool needs three decisions.

1. **Size.** By Little's law the number of requests in flight equals throughput times latency. A service that sends 500 requests per second to a dependency whose calls take 40 ms holds 500 x 0.040 = 20 connections busy on average. A pool of 20 is exactly at saturation, so the pool is sized with headroom for variation (for example 30 to 50), and the dependency must tolerate the sum of the pools of all callers.
2. **Idle eviction.** Connections idle for longer than the idle timeout of the server, load balancer or NAT device are closed by that device without notice. The client must close idle connections earlier than the shortest timeout on the path, or the next request fails with a connection reset.
3. **Maximum lifetime.** Recycling connections after some minutes lets a client discover new instances behind a Kubernetes Service or DNS name and rebalance load.

Service mesh and sidecars

A **service mesh** moves network concerns out of application code into a **sidecar proxy** (typically Envoy) that runs next to every service instance and intercepts its traffic. The application calls `localhost` or the service name; the sidecars establish mutual TLS between themselves with short-lived workload certificates issued by the mesh control plane, and implement retries, timeouts, circuit breaking, load balancing per request and uniform metrics and tracing. The benefit is uniform policy for services written in any language and encrypted, authenticated east-west traffic without changes to the code. The costs are extra proxy hops (typically a few milliseconds per call), memory and CPU per pod, and the operational complexity of the control plane; sidecarless designs and Spring Cloud / Resilience4j libraries are alternatives.

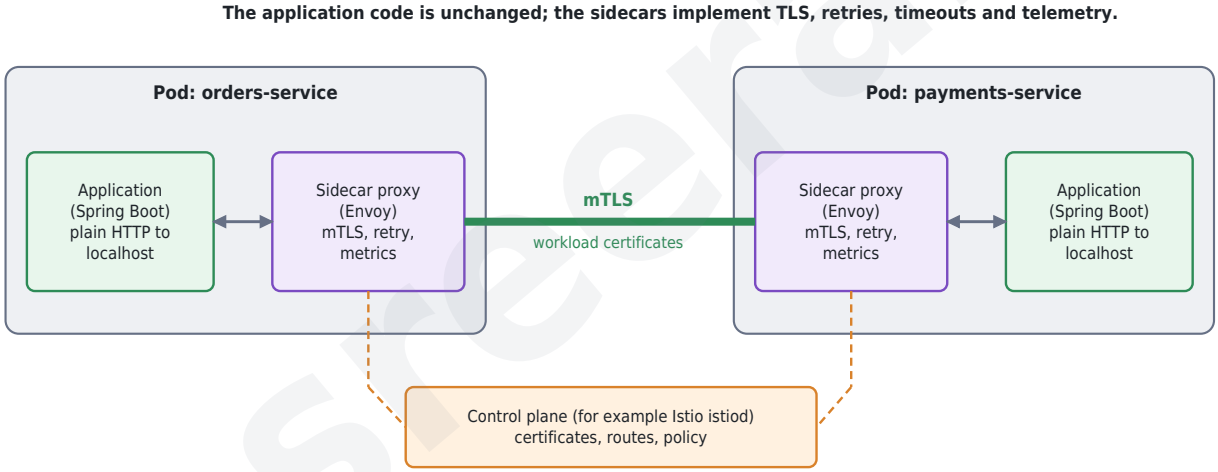


Figure 5.30. Each pod contains an application and a sidecar proxy; the sidecars carry mutual TLS between services using certificates distributed by the control plane, and apply retries, timeouts and telemetry without code changes.

Observing network latency

Latency problems are found with measurements, not with guesses. Three kinds of data are needed.

- **Metrics**: for each route and dependency the rate of requests, the error rate and the duration as a **histogram** (the RED method). Percentiles (p50, p95, p99) are derived from histograms; averages hide the tail. Spring Boot with Micrometer records `http.server.requests` and `http.client.requests` and, with the appropriate settings, connection pool statistics.
- **Distributed traces**: a trace identifier created at the gateway is propagated on every call, so one request can be followed across services and the time spent in each hop and in each network wait is visible. The W3C **Trace Context** standard defines the `traceparent` header, for example `traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01` (version, 16-byte trace id, 8-byte parent span id, flags).
- **Logs** that carry the trace id, so that log lines of one request can be joined.

At the network level, TCP statistics such as retransmissions and round-trip time per connection (`ss -ti` on Linux, not available on the build machine) show whether a slow call was caused by the network or by the application.

5.7.2 Worked trace

Why the average hides the problem

The program starts a local server whose handler answers in 5 ms in 97 percent of cases and in 120 ms in 3 percent of cases (a stand-in for garbage-collection pauses, cold caches or slow disks), then sends 400 sequential requests with `HttpClient` and reports the distribution.

Latency.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Arrays;
import java.util.Random;

public class Latency {
    public static void main(String[] args) throws Exception {
        Random rnd = new Random(42);
        HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", 8083), 0);
        s.createContext("/work", ex -> {
            try { Thread.sleep(rnd.nextInt(100) < 3 ? 120 : 5); } catch (InterruptedException e) { }
            ex.sendResponseHeaders(204, -1);
            ex.close();
        });
        s.start();
        HttpClient c = HttpClient.newHttpClient();
        HttpRequest rq = HttpRequest.newBuilder(URI.create("http://127.0.0.1:8083/work")).build();
        int n = 400;
        double[] ms = new double[n];
        for (int i = 0; i < n; i++) {
            long t0 = System.nanoTime();
            c.send(rq, HttpResponse.BodyHandlers.discarding());
            ms[i] = (System.nanoTime() - t0) / 1e6;
        }
        Arrays.sort(ms);
        double sum = 0;
        for (double v : ms) sum += v;
        System.out.printf("calls=%d mean=%.1f ms p50=%.1f ms p95=%.1f ms p99=%.1f ms max=%.1f ms%n",
                n, sum / n, ms[n / 2], ms[(int) (n * 0.95)], ms[(int) (n * 0.99)], ms[n - 1]);
        s.stop(0);
    }
}
```

Typical output of Latency.java (measured on the build machine; varies between runs) TERMINAL

```
$ java Latency.java
calls=400 mean=11.3 ms p50=6.9 ms p95=11.1 ms p99=126.5 ms max=203.4 ms
```

The mean is close to the typical latency and hides the problem: roughly one call in thirty takes about twenty times longer than the median. The 99th percentile exposes it. For a request that fans out to many such services the table of the previous section applies: with 3 percent slow calls per service, a request calling 20 of them in parallel meets at least one slow call with probability 1 - 0.97^20, about 46 percent.

Connection reuse and per-request timeouts

The next program shows two facts with real numbers. The server reports the client port of each request. A single `HttpClient` reuses one TCP connection for sequential requests (same client port). A new client for every request opens a new connection each time (a new port each time), which is what happens when a client object is created per call. The last step shows a per-request timeout: the handler sleeps 1,500 ms and the request allows 500 ms.

Pool.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.nio.charset.StandardCharsets;
```

Pool.java (continued) JAVA

```java
import java.time.Duration;

public class Pool {
    public static void main(String[] args) throws Exception {
        HttpServer s = HttpServer.create(new InetSocketAddress("127.0.0.1", 8081), 0);
        s.createContext("/port", ex -> {
            String port = String.valueOf(ex.getRemoteAddress().getPort());
            byte[] b = port.getBytes(StandardCharsets.UTF_8);
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        s.createContext("/slow", ex -> {
            try { Thread.sleep(1500); } catch (InterruptedException e) { }
            ex.sendResponseHeaders(204, -1);
            ex.close();
        });
        s.start();
        URI portUri = URI.create("http://127.0.0.1:8081/port");
        HttpClient hc = HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(2)).build();
        System.out.println("One HttpClient, sequential requests (keep-alive):");
        for (int i = 0; i < 3; i++) {
            var r = hc.send(HttpRequest.newBuilder(portUri).build(),
                    HttpResponse.BodyHandlers.ofString());
            System.out.println("  request " + (i + 1) + " -> client port " + r.body());
        }
        System.out.println("A new HttpClient (and so a new connection) per request:");
        for (int i = 0; i < 3; i++) {
            var fresh = HttpClient.newBuilder().version(HttpClient.Version.HTTP_1_1).build();
            var r = fresh.send(HttpRequest.newBuilder(portUri).build(),
                    HttpResponse.BodyHandlers.ofString());
            System.out.println("  request " + (i + 1) + " -> client port " + r.body());
        }
        long t0 = System.nanoTime();
        try {
            HttpRequest slow = HttpRequest.newBuilder(URI.create("http://127.0.0.1:8081/slow"))
                    .timeout(Duration.ofMillis(500)).build();
            hc.send(slow, HttpResponse.BodyHandlers.discarding());
        } catch (HttpTimeoutException e) {
            long ms = (System.nanoTime() - t0) / 1_000_000 / 100 * 100;
            System.out.println("per-request timeout after about " + ms + " ms: "
                    + e.getClass().getSimpleName() + ": " + e.getMessage());
        }
        s.stop(0);
    }
}
```

Output of Pool.java (client ports vary between runs) TERMINAL

```
$ java Pool.java
One HttpClient, sequential requests (keep-alive):
  request 1 -> client port 51230
  request 2 -> client port 51230
  request 3 -> client port 51230
A new HttpClient (and so a new connection) per request:
  request 1 -> client port 51238
  request 2 -> client port 51242
  request 3 -> client port 51248
per-request timeout after about 500 ms: HttpTimeoutException: request timed out
```

What to notice. In the first loop all three requests arrive from the same client port: one TCP connection, one handshake. In the second loop each request comes from a new ephemeral port, so each paid a handshake (and, on a real network, a TLS handshake), and each left a connection in `TIME_WAIT` on the side that closed it. At high request rates a client that creates connections per request exhausts its ephemeral port range (about 28,000 ports on Linux by default); the symptom is `Cannot assign requested address`. The `HttpClient` object is therefore created once and shared. The final line shows `HttpTimeoutException`, the failure that a service must be prepared to handle on every call.

Pool sizing by Little's law

A worked example. A service handles 500 requests per second at peak and each request calls a dependency that answers in 40 ms at the median and 200 ms at p99.

1. Average connections busy: 500 x 0.040 = 20.
2. During a slowdown to 200 ms the same rate needs 500 x 0.200 = 100 connections.
3. A pool limited to 30 connections then queues requests; the waiting time adds to every call until the pending-acquire timeout fires.
4. With 10 instances of the service the dependency must accept up to 10 x 100 = 1,000 connections in the slow case.

Consequences: the pool limit protects the dependency (a bulkhead) but turns a slowdown into queueing at the caller, so the acquisition timeout must be short and the failure handled; and the capacity of the dependency (its own connection limit, for example a database `max_connections`) is part of the same calculation.

5.7.3 Hands-on

Timeouts and pools in Spring Boot

Spring Boot 3.2 and later provide `RestClient`; the reference documentation for the current Spring Boot 4.x line configures timeouts globally with `spring.http.clients.connect-timeout` and `spring.http.clients.read-timeout` (earlier 3.x releases use a different prefix, so the reference of the Spring Boot version in use should be consulted). The sample below (not run here) sets them explicitly in code and shows a `WebClient` with a bounded Reactor Netty pool.

HttpClientConfig.java (not run) JAVA

```java
import io.netty.channel.ChannelOption;
import java.time.Duration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.reactive.ReactorClientHttpConnector;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.netty.http.client.HttpClient;
import reactor.netty.resources.ConnectionProvider;

@Configuration
public class HttpClientConfig {
    @Bean
    WebClient inventoryClient() {
        ConnectionProvider pool = ConnectionProvider.builder("inventory")
                .maxConnections(50)                          // Little's law plus headroom
                .pendingAcquireTimeout(Duration.ofSeconds(1)) // fail fast when exhausted
                .maxIdleTime(Duration.ofSeconds(20))         // below server/balancer idle time
                .maxLifeTime(Duration.ofMinutes(10))         // rebalance across new instances
                .evictInBackground(Duration.ofSeconds(30))
                .metrics(true)
                .build();
        HttpClient http = HttpClient.create(pool)
                .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 1000)
                .responseTimeout(Duration.ofSeconds(2));
        return WebClient.builder()
                .baseUrl("http://inventory.shop.svc.cluster.local:8080")
                .clientConnector(new ReactorClientHttpConnector(http))
                .build();
    }
}
```

The Reactor Netty reference states that the default pool size of a `ConnectionProvider` built without settings is 2 x the number of processors (minimum 16) and that the default pending acquire timeout is 45 seconds, which is far longer than any user request should wait; both defaults deserve explicit values.

Keep-alive tuning

Keep-alive timeouts must decrease along the path from the client to the server for idle connections to be closed by the client first.

Component	Setting	Rule
Caller HTTP client pool	`maxIdleTime` 20 s	Shorter than every device on the path
Load balancer or NAT idle timeout	60 s to 350 s depending on product	Longer than the caller idle time

Component	Setting	Rule
Server keep-alive (Tomcat)	`server.tomcat.keep-alive-timeout`	Longer than the balancer idle timeout if the balancer reuses connections to the server

Typical idle timeouts that must be ordered consistently (illustrative values).

If the order is reversed, the server or a middlebox silently closes a connection at the moment the client sends a request on it, and the request fails with a reset or `502`. The error is intermittent and appears at low traffic, when connections stay idle.

Network policies

East-west access can be restricted in Kubernetes with `NetworkPolicy` objects, which a supporting CNI enforces. The policy below (reference, not run) allows only pods labelled `app: orders` to reach the inventory pods on port 8080.

networkpolicy.yaml (not run) YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: inventory-allow-orders
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app: inventory
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: orders
      ports:
        - protocol: TCP
          port: 8080
```

5.7.4 Failure modes

- **No timeouts.** A single slow dependency holds threads or connections of every caller until the pool is exhausted, and the failure spreads upwards (cascading failure). Every outbound call needs connect, read and acquisition timeouts.
- **Pool exhaustion.** Symptoms: latency grows while CPU stays low, then `pending acquire` or `Timeout waiting for connection from pool` exceptions. Diagnosis: pool metrics (active, idle, pending). Fix: size from Little's law, shorten timeouts, add a circuit breaker.
- **Connection per request.** A client object or connection created for every call exhausts ephemeral ports and CPU (TLS handshakes) and leaves many `TIME_WAIT` sockets. Reuse one client.
- **Stale DNS or stale connections.** After a deployment or scale event, clients keep connecting to removed pod addresses through cached DNS answers or long-lived connections. Use a bounded `networkaddress.cache.ttl`, maximum connection lifetime and readiness-based endpoints.
- **Idle connection reset.** Intermittent resets on the first request after a quiet period; cause: idle timeout ordering (see the table above).
- **Unbalanced load with long-lived connections.** gRPC and HTTP/2 clients pin to one pod behind a layer 4 Service. Use a headless Service with client-side balancing, a layer 7 proxy or a mesh.
- **Chatty services.** A request that makes dozens of sequential internal calls (the N+1 pattern across services) multiplies latency. Batch calls, aggregate in a gateway or backend-for-frontend, cache, or change the service boundaries.
- **Retry amplification and thundering herd.** Covered in the previous chapter: limit retries, add jitter and budgets.
- **Large payloads and head-of-line blocking.** Big responses over a shared HTTP/1.1 connection delay small ones; large JSON bodies cost serialisation time. Use pagination, compression, HTTP/2 or binary protocols where appropriate.

sheeram

REAL WORLD
Teams often discover their timeout policy during an incident. A typical post-incident finding: the gateway timeout was 30 seconds, the service-to-service timeout 60 seconds and the database timeout unlimited, so slow queries kept thousands of threads busy for work whose callers had left long before. The remedy is a written timeout budget per tier, enforced by configuration review and by chaos tests that inject latency.

5.7.5 Questions to expect

1. What is the difference between north-south and east-west traffic?

North-south traffic enters or leaves the cluster (client to gateway). East-west traffic flows between services inside it. East-west traffic is usually larger because of fan-out, and is secured with mutual TLS, network policies and service-level authorisation rather than perimeter controls alone.

2. Which timeouts should every HTTP client configure?

A connect timeout, a read (response) timeout, and a pool acquisition timeout, plus an overall deadline that includes retries. Defaults are often infinite or very long. Inner timeouts must be shorter than the timeout of the caller.

3. Why do p99 latencies matter more in microservices?

A request that depends on many calls meets a slow tail call with probability 1 - (1 - q)^n. With a 1 percent tail and 20 parallel calls, that is about 18 percent of requests, so tail behaviour of dependencies dominates user-visible latency.

4. How is a connection pool sized?

From Little's law: connections in use = request rate x latency. Add headroom for latency variation, limit the total across all callers to what the dependency can accept, and set a short acquisition timeout so that exhaustion fails fast.

5. A Kubernetes Service has three pods but one receives all the traffic from a gRPC client. Why?

The Service balances per TCP connection. A gRPC client keeps one HTTP/2 connection open and multiplexes all calls over it, so all calls reach the pod that accepted that connection. Solutions: a headless Service with client-side balancing, a layer 7 proxy or mesh that balances per request, or periodic connection recycling.

6. What does a service mesh provide and what does it cost?

It provides mutual TLS, retries, timeouts, circuit breaking, per-request balancing and uniform telemetry through sidecar proxies, without code changes. The costs are added latency per hop, resource overhead per pod and the complexity of operating the control plane.

7. Why do intermittent connection resets appear on idle services?

An intermediary or the server closed an idle connection (idle timeout) and the client reused it. The client pool idle time must be shorter than the idle timeouts along the path, or connections must be validated before reuse.

8. How is network latency observed in a Spring Boot system?

Micrometer histograms for server and client requests (percentiles per route and dependency), distributed tracing with W3C `traceparent` propagation, logs correlated by trace id, and pool and TCP metrics (retransmissions, RTT). Averages alone are not used because they hide the tail.

9. Synchronous call or message broker?

Synchronous calls suit queries that need an immediate answer, at the price of coupled availability and latency. A broker decouples producer and consumer in time and absorbs bursts, suits notifications and workflows, and introduces eventual consistency, duplicates (consumers must be idempotent) and a new component to operate.

Preview

KEY POINTS

- Every microservice call is a network call: it can be slow, fail while both sides are healthy, or complete after the caller has given up; design for partial failure.
- Configure connect, read and acquisition timeouts on every client, and make timeouts nest so that inner calls fail before outer ones.
- Reuse connections through a shared client and size pools with Little's law; order idle timeouts so that the client closes idle connections first.
- Fan-out makes tail latency dominate: with 20 parallel calls and a 1 percent tail, 18 percent of requests are slow.
- Kubernetes Services balance per connection; long-lived HTTP/2 or gRPC connections need client-side or layer 7 balancing.
- Service meshes provide mTLS, retries and telemetry without code changes at the cost of extra hops and operational complexity.
- Measure latency with histograms, traces and correlated logs, and read percentiles rather than averages.

sreeram

PART 6

Practice, Troubleshooting and Interviews

Applies the preceding Parts to Java services, end-to-end traces, fault diagnosis, numerical problems and interview preparation.

In this part

6.1 Java Networking in Practice
6.2 What Happens When a URL Is Entered
6.3 Troubleshooting Playbook
6.4 Numerical Problems
6.5 Interview Question Bank
6.6 Cheat Sheets
6.7 Six-Week Study Plan

6.1 Java Networking in Practice

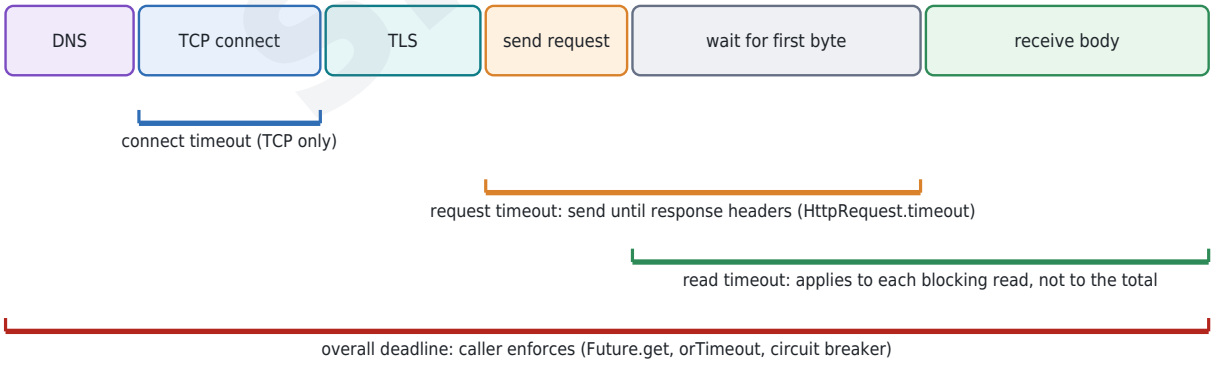
A Java service is a client and a server at the same time: it accepts connections from browsers or gateways and opens connections to databases and other services. Every call crosses the layers described in the preceding Parts, and every layer contributes a limit, a queue or a timeout. Most production incidents involving a Spring Boot application are caused by an unbounded wait, an exhausted pool, or two components with incompatible idle timeouts, not by a defect in the protocol itself. This chapter shows where those settings live and demonstrates their effect with experiments that run on the loopback interface.

6.1.1 Concept and mechanism

The three client timeouts

A blocking call over TCP has three distinct waiting phases, and each one needs its own bound. The **connect timeout** limits the time spent establishing the TCP connection (the three-way handshake). The **request timeout** limits the time from sending the request until the response headers arrive. The **read timeout** (the socket timeout, `SO_TIMEOUT`) limits the time of each individual blocking read. None of them limits the total duration of a call: a server that sends one byte every few seconds satisfies a read timeout indefinitely. An **overall deadline** must therefore be enforced by the caller.

Phases of one HTTPS call and the timeout that bounds each



A call without an overall deadline can last (read timeout x number of reads) even when every read is fast enough.

Figure 6.1. Each timeout covers a different span of one HTTPS call; only the overall deadline, which the caller must enforce, bounds the complete duration.

Java defaults are permissive. `java.net.http.HttpClient` has no connect timeout and `HttpRequest` has no request timeout unless configured, and `HttpURLConnection` waits indefinitely (timeout value 0) for connect and read. A default-configured client therefore hangs for as long as the operating system or a middlebox keeps the connection state alive.

Connection reuse

Opening a connection costs one round trip for TCP and one or two for TLS (see the latency budget in the end-to-end chapter of this Part). A client that keeps connections open and reuses them avoids both. HTTP/1.1 connections are persistent by default, and `HttpClient` keeps them in an internal pool keyed by host, port and protocol version. Reuse has two consequences. First, the number of TCP connections is bounded by concurrency, not by the number of requests. Second, a pooled connection can be closed by the peer or by a middlebox while it is idle, so the next request on it may fail; the client must either detect the closure or retry on a fresh connection.

Where a connection waits on the server

A Tomcat connector holds a connection in four places in sequence: the kernel SYN queue, the kernel accept queue, the set of connections tracked by the connector, and finally a worker thread while the request executes. Each stage has a Spring Boot property or a kernel parameter that bounds it.

Where a TCP connection waits inside a Tomcat server (Spring Boot property names where they apply)

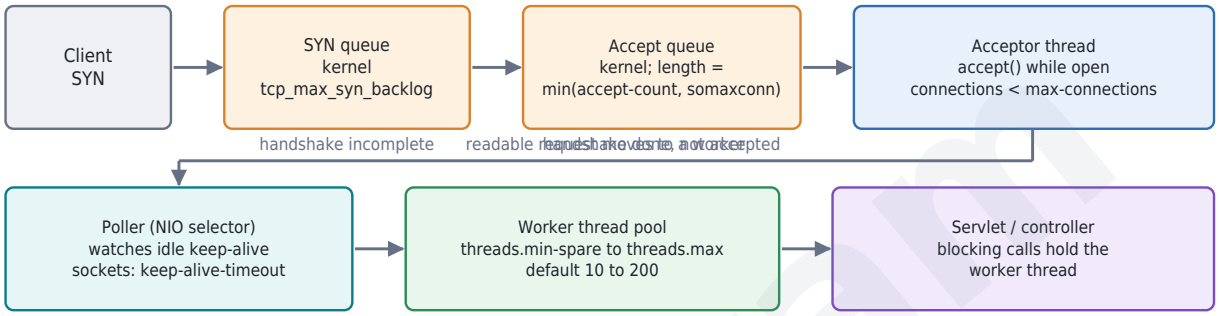


Figure 6.2. The accept queue and the worker pool are separate queues; a full accept queue produces connect timeouts at the client, whereas a saturated worker pool produces slow responses on established connections.

Property	Default	Effect
`server.tomcat.threads.max`	`200`	Upper bound on worker threads, therefore on requests executing at the same time.
`server.tomcat.threads.min-spare`	`10`	Threads kept alive when idle.
`server.tomcat.accept-count`	`100`	Length of the accept queue passed to `listen()`; the kernel caps it at `net.core.somaxconn`.
`server.tomcat.max-connections`	`8192`	Number of connections the connector accepts and tracks (NIO); further connections wait in the accept queue.
`server.tomcat.connection-timeout`	`Tomcat default (60 s)`	Time allowed for the request line to arrive after accept; also the idle limit when no keep-alive timeout is set.
`server.tomcat.keep-alive-timeout`	`connection timeout`	Idle time of a persistent connection before the server closes it (sends FIN).
`server.tomcat.max-keep-alive-requests`	`100`	Requests served on one connection before the server closes it.

Tomcat settings in Spring Boot and their effect at the TCP level.

The defaults in the table were checked against the Spring Boot reference for the current release; the connector defaults are Tomcat's own. With `spring.threads.virtual.enabled=true` (Spring Boot 3.2 and later) Tomcat runs requests on virtual threads, so `threads.max` no longer limits concurrency and downstream pools (database, HTTP client) become the effective bound.

File descriptors

Every open socket consumes one file descriptor, as does every open file. The per-process limit (`ulimit -n`, `LimitNOFILE` under systemd) is commonly 1024 in containers and shells unless raised. A service that holds thousands of connections, or leaks them, fails with `java.net.SocketException: Too many open files` for outgoing connects and `accept` failures on the server side. The limit must exceed `max-connections` plus the sizes of all client pools plus files in use.

6.1.2 Worked trace

The first experiment runs a local `HttpServer` and an `HttpClient` with explicit timeouts. Four behaviours are exercised: a normal call, a request timeout, a retry with exponential backoff and jitter, and a refused connection.

ClientDemo.java JAVA

```java
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.time.Duration;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.atomic.AtomicInteger;

public class ClientDemo {
    static final AtomicInteger flakyCalls = new AtomicInteger();

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 8081), 0);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/ok", ex -> reply(ex, 200, "ok"));
        server.createContext("/slow", ex -> {
            try { Thread.sleep(2000); } catch (InterruptedException ignored) { }
            reply(ex, 200, "late");
        });
        server.createContext("/flaky", ex -> {
            int n = flakyCalls.incrementAndGet();
            reply(ex, n <= 2 ? 503 : 200, n <= 2 ? "unavailable" : "recovered");
        });
        server.start();

        HttpClient client = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(1))
                .build();

        // 1. normal call
        HttpResponse<String> r = send(client, "/ok", Duration.ofSeconds(1));
        System.out.println("ok     -> " + r.statusCode() + " " + r.body()
                + " version=" + r.version());

        // 2. request timeout
        long t0 = System.nanoTime();
        try {
            send(client, "/slow", Duration.ofMillis(500));
        } catch (HttpTimeoutException e) {
            long ms = (System.nanoTime() - t0) / 1_000_000;
            System.out.println("slow   -> " + e.getClass().getSimpleName() + " after ~"
                    + (ms / 100) * 100 + " ms");
        }

        // 3. retry with exponential backoff and jitter (idempotent GET only)
        long cap = 100;
        for (int attempt = 1; attempt <= 4; attempt++) {
            r = send(client, "/flaky", Duration.ofSeconds(1));
            System.out.println("flaky  -> attempt " + attempt + " status " + r.statusCode());
            if (r.statusCode() < 500) {
                break;
            }
            long sleep = ThreadLocalRandom.current().nextLong(cap / 2, cap + 1);
```

ClientDemo.java (continued) JAVA

```
            System.out.println("          waiting " + sleep + " ms before the next attempt");
            Thread.sleep(sleep);
            cap *= 2;
        }

        // 4. connection refused (nothing listens on 8082)
        try {
            client.send(HttpRequest.newBuilder(URI.create("http://127.0.0.1:8082/")).build(),
                    HttpResponse.BodyHandlers.ofString());
        } catch (IOException e) {
            System.out.println("refused-> " + e.getClass().getName());
        }
        server.stop(0);
    }

    static HttpResponse<String> send(HttpClient c, String path, Duration timeout) throws Exception {
        HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:8081" + path))
                .timeout(timeout).GET().build();
        return c.send(req, HttpResponse.BodyHandlers.ofString());
    }

    static void reply(HttpExchange ex, int code, String body) throws IOException {
        byte[] b = body.getBytes();
        ex.sendResponseHeaders(code, b.length);
        ex.getResponseBody().write(b);
        ex.close();
    }
}
```

Run ClientDemo (waiting times vary between runs, typical output) TERMINAL

```
$ java ClientDemo.java
ok      -> 200 ok version=HTTP_1_1
slow    -> HttpTimeoutException after ~500 ms
flaky   -> attempt 1 status 503
           waiting 52 ms before the next attempt
flaky   -> attempt 2 status 503
           waiting 194 ms before the next attempt
flaky   -> attempt 3 status 200
refused-> java.net.ConnectException
```

Observations. The `/slow` handler needs two seconds, but the call ends after about 500 ms with `HttpTimeoutException`; the request timeout is the bound that applies. The refused call fails at once with `ConnectException` because the operating system answers the SYN with a reset (nothing listens on port 8082); this differs from a connect timeout, in which the SYN receives no answer at all. The retry loop waits a random time between half of the cap and the cap, and doubles the cap after each failure. The randomisation (jitter) prevents many clients from retrying at the same instant. A retry is safe here because `GET` is idempotent; retrying a `POST` without an idempotency key may execute the operation twice.

GOTCHA

The request timeout and the connect timeout are independent settings. `HttpClient.Builder.connectTimeout` applies to every request of that client. `HttpRequest.Builder.timeout` applies to one request. Setting only one of them leaves the other phase unbounded.

6.1.3 Hands-on

HTTP/2 and connection counts

`HttpClient` negotiates HTTP/2 by default. Over cleartext HTTP it sends an upgrade request and falls back to HTTP/1.1 when the server does not support it; over TLS it uses ALPN (application-layer protocol negotiation) to choose `h2`. A single HTTP/2 connection multiplexes concurrent requests as streams, so a pool of one connection per host is sufficient for many concurrent requests. The version actually used is available as `HttpResponse.version()`. Forcing `HttpClient.Version.HTTP_1_1` makes the client open one connection per concurrent request.

Spring Boot clients

Spring's `RestClient` delegates to a request factory, and the JDK client is one option. Timeouts belong in the factory or the underlying client, never left at defaults. The following fragments were not run (Spring is not installed on the build machine) and show the properties that matter.

ClientConfig.java JAVA

```
// RestClient on the JDK HttpClient: connect timeout on the client, read timeout on the factory
HttpClient jdk = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(2))
        .build();
JdkClientHttpRequestFactory factory = new JdkClientHttpRequestFactory(jdk);
factory.setReadTimeout(Duration.ofSeconds(5));
RestClient rest = RestClient.builder()
        .baseUrl("http://orders:8080")
        .requestFactory(factory)
        .build();

// WebClient on Reactor Netty: bounded pool, bounded wait for a connection, response timeout
ConnectionProvider pool = ConnectionProvider.builder("orders")
        .maxConnections(100)
        .pendingAcquireTimeout(Duration.ofSeconds(2))
        .maxIdleTime(Duration.ofSeconds(20))
        .maxLifeTime(Duration.ofMinutes(5))
        .build();
reactor.netty.http.client.HttpClient netty = reactor.netty.http.client.HttpClient.create(pool)
        .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 2000)
        .responseTimeout(Duration.ofSeconds(5));
WebClient web = WebClient.builder()
        .clientConnector(new ReactorClientHttpConnector(netty))
        .build();
```

The `maxIdleTime` of 20 seconds is chosen below the idle timeout of the typical peer (see the failure mode on stale connections in this chapter). `pendingAcquireTimeout` bounds the time a request waits for a free connection; without it a saturated pool turns into an unbounded queue.

Load test on the loopback interface

A load test needs a server with a known capacity so that the result can be compared with a prediction. The following program starts a server with 4 worker threads and 50 ms of work per request. The server has therefore a capacity of 4 / 0.05 s = 80 requests per second. The client runs 200 requests at increasing concurrency and records throughput, latency percentiles, and the number of distinct client ports seen by the server (one port per TCP connection).

LoadTest.java JAVA

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class LoadTest {
    public static void main(String[] args) throws Exception {
        Set<Integer> clientPorts = ConcurrentHashMap.newKeySet();
        HttpServer server = HttpServer.create(new InetSocketAddress("127.0.0.1", 8083), 128);
        server.setExecutor(Executors.newFixedThreadPool(4));   // 4 workers, like a small Tomcat
        server.createContext("/work", ex -> {
            clientPorts.add(ex.getRemoteAddress().getPort());
            try { Thread.sleep(50); } catch (InterruptedException ignored) { }
            byte[] b = "done".getBytes();
            ex.sendResponseHeaders(200, b.length);
            ex.getResponseBody().write(b);
            ex.close();
        });
        server.start();
```

LoadTest.java (continued) JAVA

```
        for (int concurrency : new int[] {1, 4, 16, 64}) {
            clientPorts.clear();
            run(concurrency, 200, clientPorts);
        }
        server.stop(0); System.exit(0);
    }

    static void run(int concurrency, int total, Set<Integer> ports) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest req = HttpRequest.newBuilder(URI.create("http://127.0.0.1:8083/work")).build();
        List<Long> lat = Collections.synchronizedList(new ArrayList<>());
        long start = System.nanoTime();
        try (ExecutorService pool = Executors.newFixedThreadPool(concurrency)) {
            List<Future<?>> fs = new ArrayList<>();
            for (int i = 0; i < total; i++) {
                fs.add(pool.submit(() -> {
                    long t = System.nanoTime();
                    client.send(req, HttpResponse.BodyHandlers.discarding());
                    lat.add((System.nanoTime() - t) / 1_000_000);
                    return null;
                }));
            }
            for (Future<?> f : fs) f.get();
        }
        double secs = (System.nanoTime() - start) / 1e9;
        List<Long> s = new ArrayList<>(lat);
        Collections.sort(s);
        long p50 = s.get(s.size() / 2);
        long p99 = s.get((int) (s.size() * 0.99) - 1);
        System.out.printf("concurrency=%2d  req/s=%6.1f  p50=%4d ms  p99=%4d ms  connections=%d%n",
                concurrency, total / secs, p50, p99, ports.size());
    }
}
```

Run LoadTest (measured on the build machine, typical) TERMINAL

```
$ java -Dsun.net.httpserver.nodelay=true LoadTest.java
concurrency= 1  req/s=  17.5  p50=  54 ms  p99=  75 ms  connections=1
concurrency= 4  req/s=  73.6  p50=  53 ms  p99=  60 ms  connections=4
concurrency=16  req/s=  77.4  p50= 204 ms  p99= 222 ms  connections=16
concurrency=64  req/s=  78.2  p50= 812 ms  p99= 818 ms  connections=64
```

Loopback load test: 4 worker threads, 50 ms of work per request (solid = measured, dashed = model)

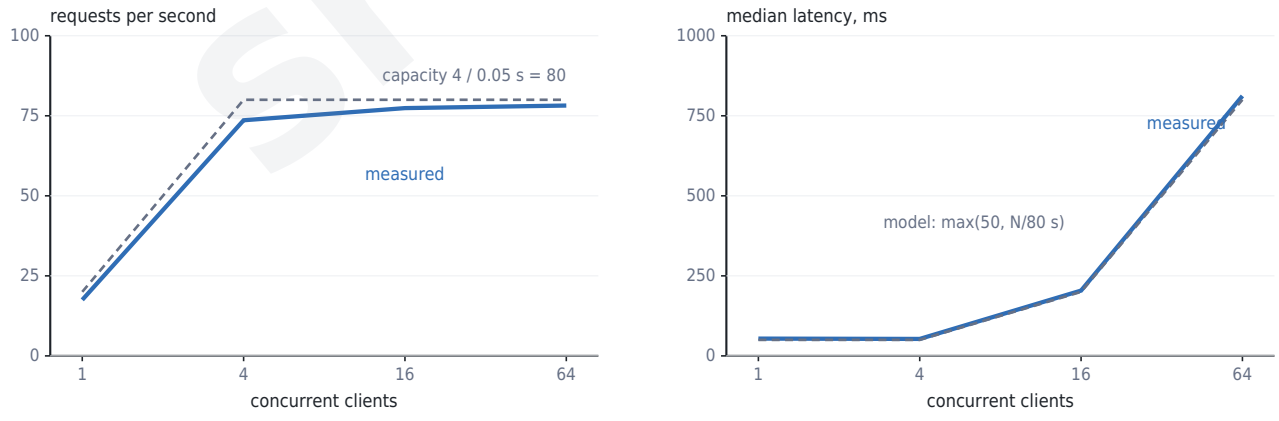


Figure 6.3. Throughput rises linearly until the 4 workers are busy and then stays at the capacity of about 80 requests per second, while latency grows in proportion to the number of waiting clients.

Reading the result. Up to 4 concurrent clients, every request finds an idle worker, so latency equals the service time (about 52 ms) and throughput grows with concurrency. Beyond 4 clients the throughput stays at 78 requests per second, the capacity of the worker pool, and the excess concurrency turns into queueing time. Little's law, $L = \lambda \times W$, predicts the latency: 64 clients at 78.2 requests per second give $W = 64 / 78.2 = 0.82$ s, matching the measured 812 ms. The connection count equals the concurrency in every row, which shows that `HttpClient` reuses each connection for successive requests (200 requests used at most 64 connections).

Nagle's algorithm and delayed acknowledgement measured

The command line above sets `sun.net.httpserver.nodelay=true`. Without it, the same program produced the following on the same machine.

Run LoadTest without the property (measured on the build machine, typical) TERMINAL

```
$ java LoadTest.java
concurrency= 1  req/s=  10.3  p50=  96 ms  p99= 117 ms  connections=1
concurrency= 4  req/s=  42.0  p50=  95 ms  p99= 101 ms  connections=4
concurrency=16  req/s=  76.7  p50= 203 ms  p99= 245 ms  connections=16
concurrency=64  req/s=  76.8  p50= 811 ms  p99= 855 ms  connections=64
```

At low concurrency the median latency is 96 ms instead of 52 ms: an extra delay of about 40 ms on every request. The JDK server writes the response headers and the body with separate small writes. Nagle's algorithm holds the second small segment until the first is acknowledged, and the receiving TCP delays that acknowledgement (about 40 ms on Linux) because it has no data to send back at that moment. A control run with 1 ms of work per request gave a median of 47 ms without the property and 4 ms with it, which isolates the effect. At high concurrency the acknowledgements are piggybacked on other traffic and the stall disappears. The mechanism is described in the flow-control chapter of the transport Part; the practical rule is to enable `TCP_NODELAY` on latency-sensitive request/response servers and to write each response with one `write` where possible.

File-descriptor exhaustion

The next program opens loopback connections without closing them under a deliberately low limit.

FdLeak.java JAVA

```java
import java.io.IOException;
import java.net.InetAddress;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class FdLeak {
    public static void main(String[] args) throws Exception {
        try (ServerSocket server = new ServerSocket(9100, 50, InetAddress.getLoopbackAddress())) {
            Thread acceptor = Thread.startVirtualThread(() -> {
                try { while (true) { server.accept(); } } catch (IOException ignored) { }
            });
            List<Socket> leaked = new ArrayList<>();
            try {
                while (true) {
                    leaked.add(new Socket(InetAddress.getLoopbackAddress(), 9100));
                }
            } catch (IOException e) {
                System.out.println("opened " + leaked.size() + " client sockets, then: " + e);
            }
        }
    }
}
```

Run FdLeak with a descriptor limit of 64 TERMINAL

```
$ (ulimit -n 64; java FdLeak.java)
opened 53 client sockets, then: java.net.SocketException: Too many open files
```

The failure is raised by `Socket` creation, not by the network. The remedy is to close sockets deterministically (try-with-resources), to bound pools, and to raise `ulimit -n` or `LimitNOFILE` to a value that matches the design (for example 65536 for a gateway). The count of open descriptors of a running process is visible with `ls /proc/<pid>/fd | wc -l`, or inside the JVM through `UnixOperatingSystemMXBean.getOpenFileDescriptorCount()`.

6.1.4 Failure modes

steram

Stale pooled connections

A connection idle in a client pool can be closed by the server (keep-alive timeout), by a load balancer (idle timeout), or by a NAT device that discards the translation entry. The client does not learn of the closure until it writes on the connection. The write succeeds, the peer answers with a reset, and the application sees `Connection reset`, `Broken pipe` or a premature end of stream, typically at a low and irregular rate. The rule is that the side that reuses connections must close idle connections earlier than every device that could close them for it: client pool idle time below the load balancer idle timeout, which is in turn below the server keep-alive timeout.

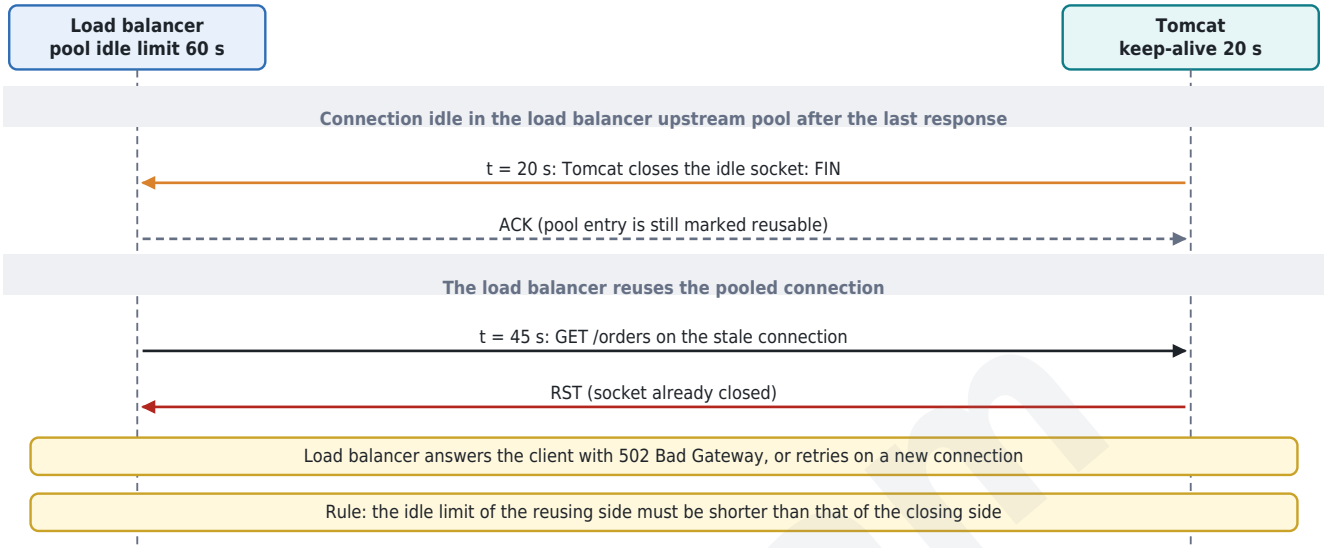


Figure 6.4. A connection that the server closed at 20 s is reused by the load balancer at 45 s; the first write on the stale connection is answered with a reset, which surfaces as a 502 response.

Unbounded waiting

A pool without a wait limit, a client without a connect timeout, or a server whose worker pool is saturated all converge on the same symptom: threads blocked in socket reads, rising latency, and eventually no free thread for health checks. Every outgoing call needs a connect timeout, a request or read timeout, a bounded pool, and a bounded queue for pool acquisition.

Retry amplification

If each of three layers retries a failing call three times, one failure produces $3 \times 3 \times 3 = 27$ attempts against the failing component. Retries must be limited to one layer, restricted to idempotent operations and transient errors (connect failures, 502, 503, 504), bounded by a total deadline, and spaced with jitter. A retry budget (for example at most 10 percent of the request volume) prevents a partial outage from becoming a complete one.

6.1.5 Questions to expect

Which timeouts must a Java HTTP client configure, and why?

Connect timeout, request or read timeout, and an overall deadline. Without the first, a dropped SYN blocks for the operating system limit (about two minutes on Linux with default retries). Without the second, a server that accepts but never answers blocks the thread indefinitely. The third bounds the sum, because the read timeout restarts after every received chunk.

What does `server.tomcat.accept-count` control?

The length of the accept queue: connections whose handshake completed but which Tomcat has not yet accepted. It is passed as the backlog of `listen()` and capped by `net.core.somaxconn`. When it is full, new SYNs or final ACKs are dropped and clients observe connect timeouts.

How does `threads.max` differ from `max-connections`?

`max-connections` bounds the number of open connections the connector tracks (idle keep-alive connections included). `threads.max` bounds the requests executing concurrently. Many more connections than threads is normal, because most connections are idle between requests.

Why is a retry on `POST` dangerous?

The first attempt may have been processed although the response was lost. A second attempt then repeats the effect (a duplicate payment). Safe retries require an idempotency key that the server stores and checks.

A service shows 40 ms of unexplained latency per request on loopback. What is a likely cause?

The interaction of Nagle's algorithm with delayed acknowledgement when a response is written in several small writes. Setting `TCP_NODELAY` or coalescing the writes removes it, as the experiment in this chapter shows.

How is the number of connections to a downstream service estimated?

By Little's law: connections in use equal the request rate multiplied by the mean latency. At 200 requests per second and 120 ms mean latency, about 24 connections are busy; a pool of 32 to 40 leaves headroom.

KEY POINTS

- Every phase of a call (connect, request or read, overall) needs an explicit bound; Java defaults are unbounded.
- Connection reuse removes handshake round trips, but pooled connections can be closed silently; idle limits must decrease from server to client.
- A Tomcat connection waits in the accept queue, then for a worker thread; `accept-count` and `threads.max` bound these two queues independently.
- Throughput saturates at pool size divided by service time; beyond that, latency grows linearly (Little's law).
- Descriptor limits, retries without budgets and unclosed responses are the usual causes of resource-exhaustion incidents.

6.2 What Happens When a URL Is Entered

The question "what happens when `https://www.example.com/index.html` is entered in a browser" is a standard interview question because a complete answer touches every layer: name resolution, link-layer addressing, routing, address translation, TCP, TLS, HTTP and rendering. This chapter traces one request from the keystroke to the first painted page, using one consistent set of addresses and header values. The trace is illustrative: the addresses belong to the documentation ranges (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and private ranges, and the MAC addresses to the documentation block 00:00:5E:00:53:00 to 00:00:5E:00:53:FF.

6.2.1 Concept and mechanism

The request passes through seven phases. Phases 1 to 3 obtain the information needed to send a packet (the server address and the next-hop link-layer address). Phases 4 and 5 build a reliable, encrypted channel. Phases 6 and 7 exchange the application data. Each phase starts only when the previous one has produced its result, which is why the first byte of a response needs several round trips.

Phase	Protocol	Result	Java counterpart
1	URL parsing, caches	Scheme https, host, port 443, path	`java.net.URI`
2	ARP	MAC address of the default gateway	handled by the operating system
3	DNS over UDP	203.0.113.10	`InetAddress.getByName`, `networkaddress.cache.ttl`
4	TCP handshake	Established connection, ISNs exchanged	`Socket.connect`, connect timeout
5	TLS 1.3	Authenticated, encrypted channel	`SSLContext`, truststore, SNI, ALPN
6	HTTP/2 request	HTML document	`HttpClient`, `HttpRequest`
7	Sub-resources, rendering	Painted page	not applicable to a Java client

Phases of the end-to-end trace with the protocol, the key result and the main Java counterpart.

6.2.2 Worked trace

Phases 1 to 3: local lookups, ARP and DNS

The browser parses the URL into scheme `https`, host `www.example.com`, port 443 (the default for the scheme) and path `/index.html`. It consults the HSTS list (which would force HTTPS), the HTTP cache and its own DNS cache, then asks the operating system to resolve the name; the system checks its resolver cache and the hosts file. All lookups miss in this trace.

The destination 203.0.113.10 is not in the client subnet (192.168.1.0/24), so a packet to it must be sent to the default gateway 192.168.1.1. The client has no entry for the gateway MAC address and broadcasts an ARP request. The gateway answers with a unicast ARP reply. The DNS query then follows as a UDP datagram to the configured resolver, which here is the home router acting as a forwarder.

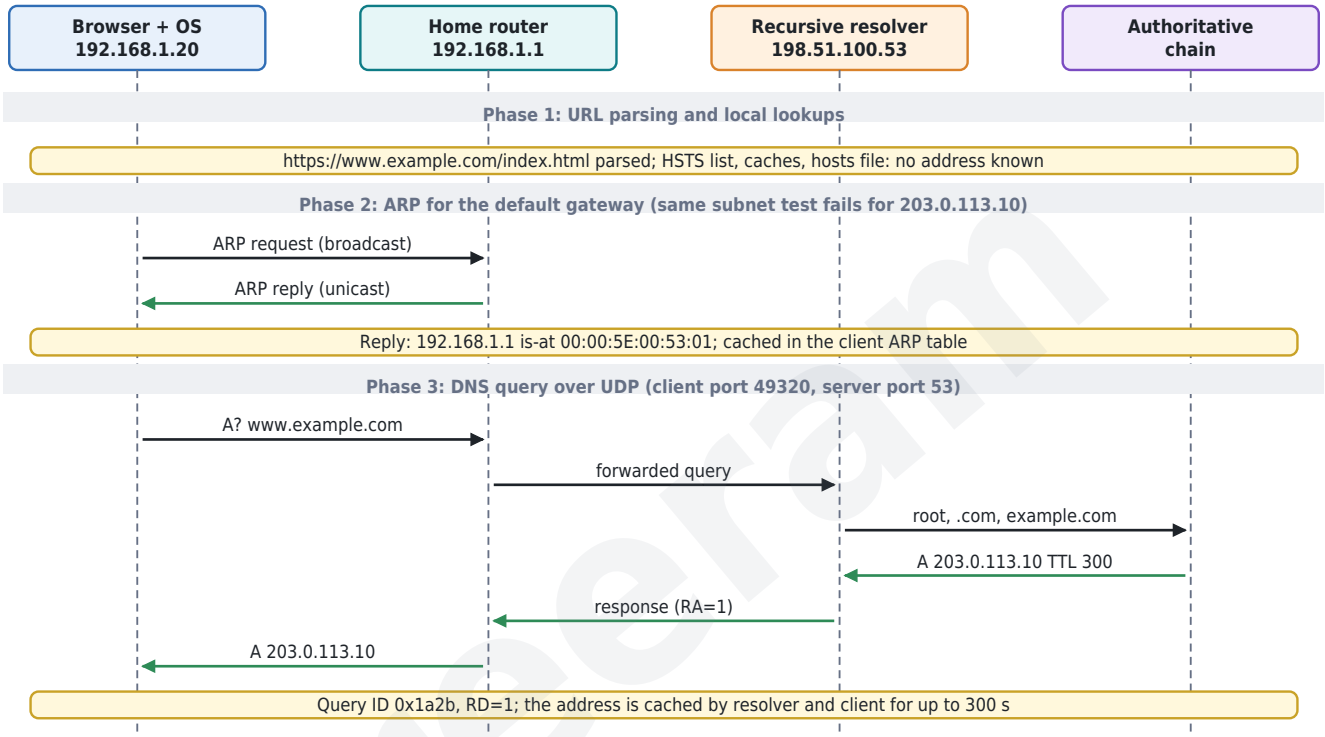


Figure 6.5. ARP resolves the gateway MAC address first; the DNS query then travels via the router to a recursive resolver, which contacts the authoritative chain and returns the A record with a TTL of 300 seconds.

Field	ARP request	DNS query (frame towards the gateway)
Ethernet destination	`ff:ff:ff:ff:ff:ff`	`00:00:5E:00:53:01`
Ethernet source	`00:00:5E:00:53:20`	`00:00:5E:00:53:20`
EtherType	`0x0806`	`0x0800`
Payload	`who-has 192.168.1.1, tell 192.168.1.20`	`IPv4 192.168.1.20 to 192.168.1.1, protocol 17`
Transport	`not applicable`	`UDP 49320 to 53, DNS ID 0x1a2b, flags 0x0100 (RD=1), QTYPE A, QCLASS IN`
Reply	`ARP reply: 192.168.1.1 is-at 00:00:5E:00:53:01`	`flags 0x8180 (QR=1, RD=1, RA=1), 1 answer: 203.0.113.10, TTL 300`

Header values of the first frames (ARP request and DNS query).

Note that the DNS query is addressed at the link layer to the gateway MAC address but at the network layer to the resolver (here also the gateway). If the configured resolver were 198.51.100.53 directly, the IP destination would be the resolver and the Ethernet destination would still be the gateway.

Phases 4 and 5: TCP and TLS

The client opens a TCP connection from an ephemeral port (49152) to port 443. The router performs source NAT (network address translation with port mapping) and records the translation so that return packets are mapped back. The server answers with SYN-ACK; each side picks an initial sequence number (ISN), and a SYN consumes one sequence number, so the acknowledgement values are the ISNs plus one.

TLS 1.3 then completes in one round trip: the ClientHello carries the server name (SNI), the offered application protocols (ALPN) and a key share; the server replies with its key share and, already encrypted, its certificate chain, a signature over the handshake (CertificateVerify) and Finished. The client validates the certificate chain, validity period and host name before sending its own Finished. It may send application data in the same flight.

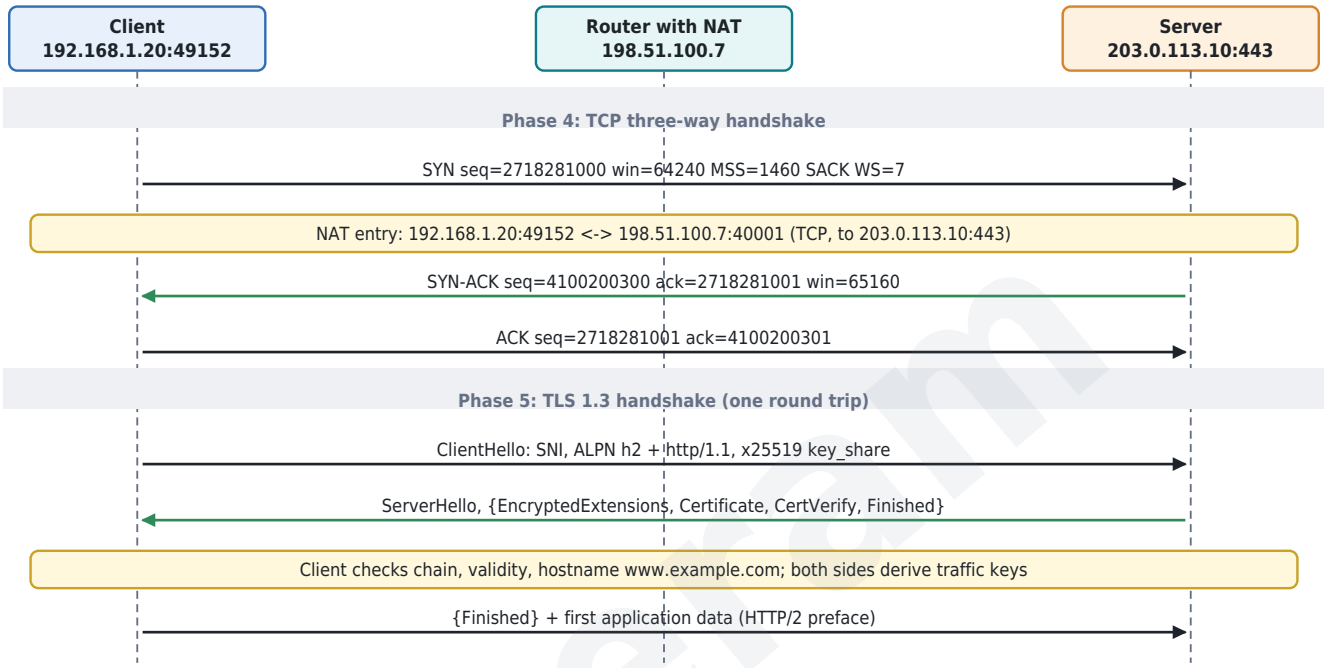


Figure 6.6. The TCP handshake consumes one round trip and the TLS 1.3 handshake one more; the router translates the source port 49152 to 40001 when the SYN passes through it.

Segment	Flags	Fields
`Client to server`	SYN	seq 2718281000, window 64240, MSS 1460, SACK permitted, window scale 7, 74-byte frame
`Server to client`	SYN, ACK	seq 4100200300, ack 2718281001, window 65160, MSS 1460
`Client to server`	ACK	seq 2718281001, ack 4100200301, no payload

TCP handshake values (illustrative ISNs; header options typical).

The 74-byte frame of the SYN is 14 bytes of Ethernet header, 20 bytes of IPv4 header and a 40-byte TCP header (20 bytes plus 20 bytes of options). Subsequent data segments usually carry a 32-byte header with the timestamp option.

Phase 6 and 7: HTTP and rendering

ALPN selected `h2`, so the client sends the HTTP/2 connection preface, a SETTINGS frame and a HEADERS frame on stream 1 with the pseudo-headers `:method GET`, `:scheme https`, `:authority www.example.com` and `:path /index.html`, compressed with HPACK. The server answers with HEADERS (`:status 200`, `content-type text/html`) and DATA frames. While the HTML is still arriving, the browser's parser discovers the references to `/app.css`, `/app.js` and `/logo.png` and requests them on streams 3, 5 and 7 of the same connection. Stream identifiers initiated by a client are odd.

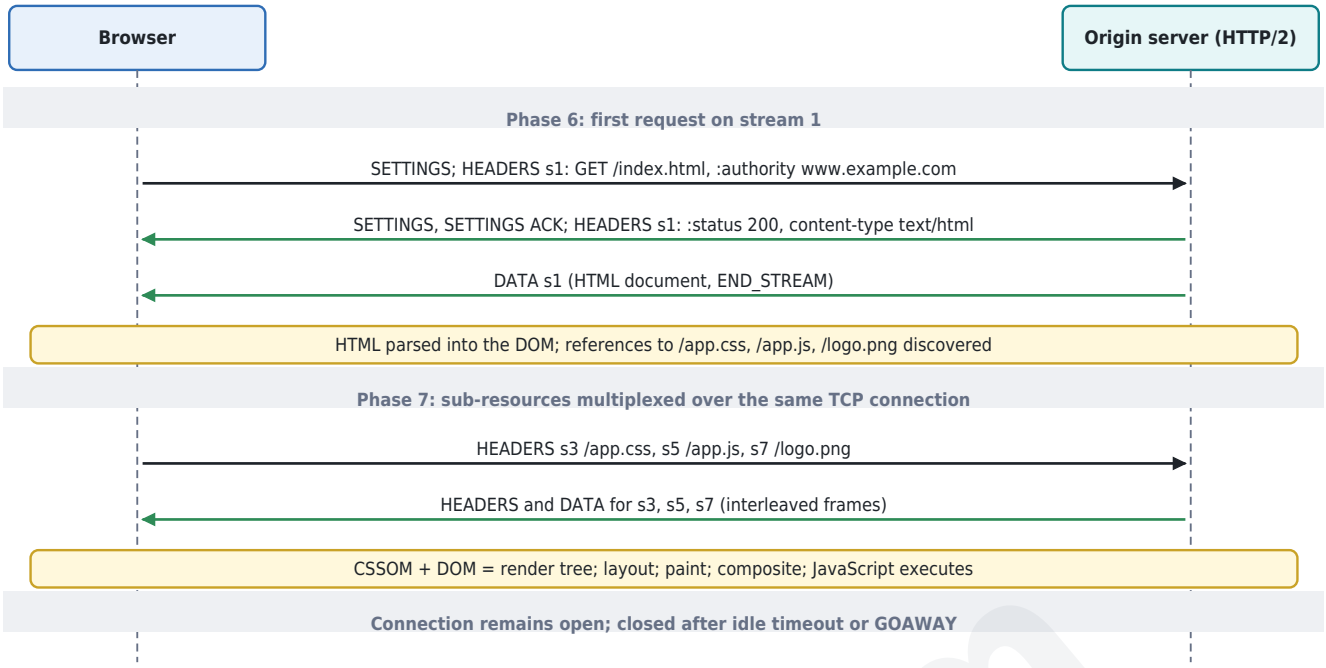


Figure 6.7. The first response and the three sub-resources share one TCP connection as separate HTTP/2 streams; after the last DATA frame the browser builds the render tree and paints.

Rendering is local work: the HTML is parsed into the DOM, the CSS into the CSSOM, both combine into the render tree, layout computes the geometry, the page is painted into layers and composited. Scripts without `async` or `defer` block parsing, and CSS blocks rendering, which is why both appear on the critical path.

What changes on the wire at each hop

The same TCP segment is carried in a new Ethernet frame at every router. The link-layer addresses change at each hop, the TTL decreases by one at each router, and NAT rewrites the source address and port once at the edge of the private network. The TCP sequence numbers and the destination address and port stay unchanged end to end.

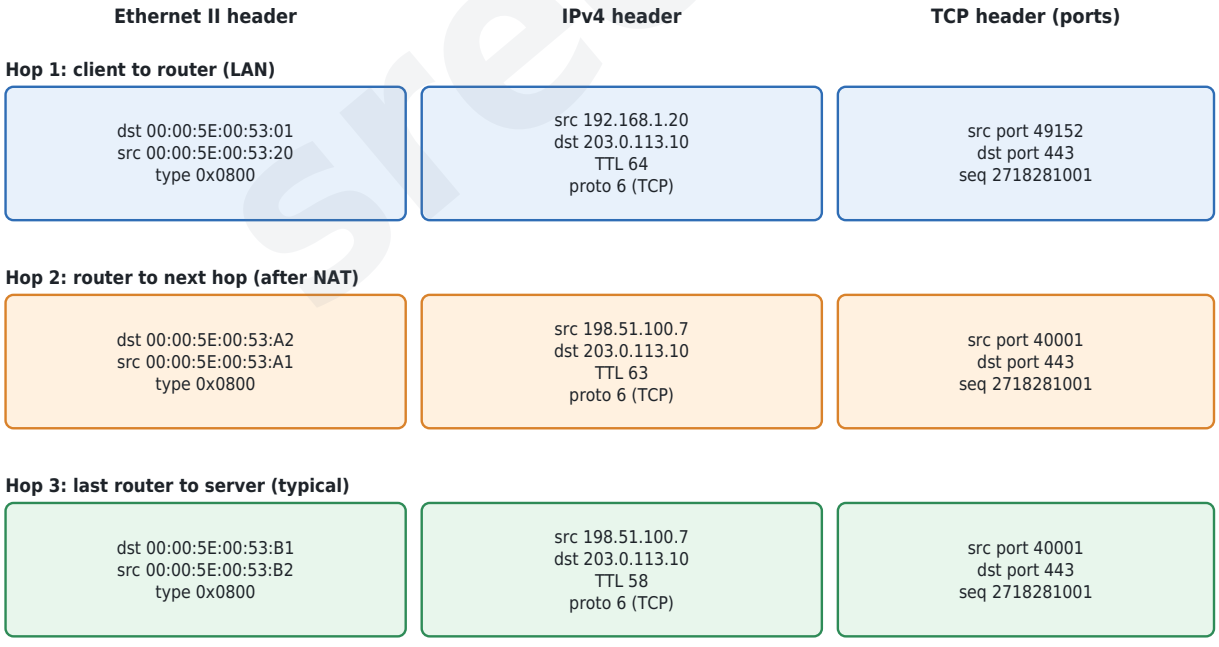


Figure 6.8. Across three hops the Ethernet addresses are replaced every time, the IPv4 TTL falls from 64 to 58, and only the first router rewrites the source address and port.

Where the time goes

Latency budget for the first byte of an HTTPS response

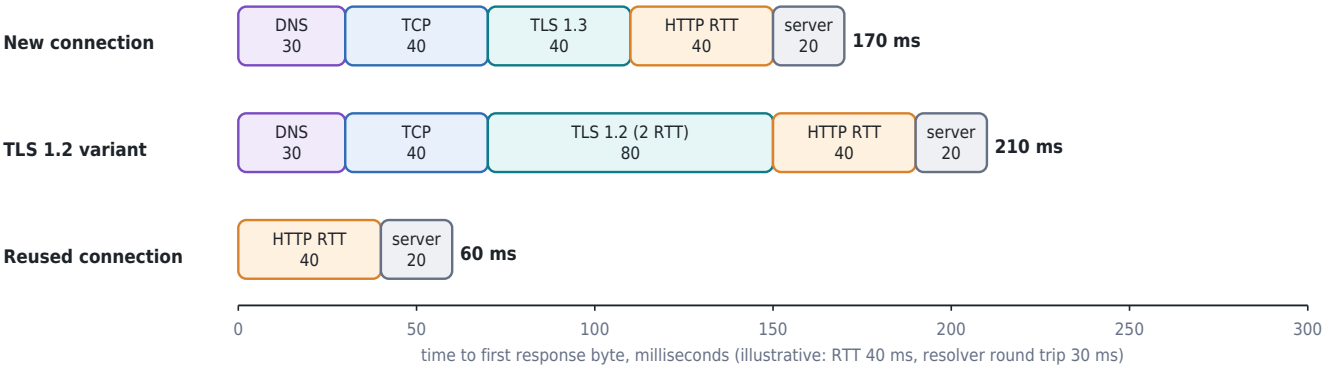


Figure 6.9. With a 40 ms round-trip time, a new HTTPS connection needs 170 ms before the first response byte, a TLS 1.2 handshake adds one more round trip, and a reused connection needs only 60 ms.

The budget explains three common optimisations. Connection reuse removes the TCP and TLS round trips. TLS 1.3 removes one of the two TLS round trips of TLS 1.2. A content delivery network reduces the round-trip time itself by terminating the connection close to the client. HTTP/3 over QUIC combines the transport and TLS handshakes into one round trip (see the QUIC chapter in the transport Part).

6.2.3 Hands-on

A local TLS server reproduces phases 4 to 6 on the loopback interface. The certificate is self-signed for `localhost` and the server is `openssl s_server`; `curl -v` prints the handshake steps in order.

Phases 4 to 6 with curl -v (trimmed; certificate dates vary) TERMINAL

```
$ openssl s_server -accept 9501 -cert c.pem -key k.pem -www -alpn http/1.1 &
$ curl -sS -v --cacert c.pem -o /dev/null https://localhost:9501/
* Host localhost:9501 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying 127.0.0.1:9501...
* Connected to localhost (127.0.0.1) port 9501
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519 / RSASSA-PSS
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: CN=localhost
*  subjectAltName: host "localhost" matched cert's "localhost"
*  SSL certificate verify ok.
> GET / HTTP/1.1
> Host: localhost:9501
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.0 200 ok
< Content-type: text/html
```

The order of the lines is the order of the phases: resolution (two addresses, IPv6 and IPv4), TCP connect, TLS messages (the same message names as in the diagram), certificate verification, then the HTTP request. The server in this experiment offers only HTTP/1.1, so ALPN selects it; a server that supports HTTP/2 would be selected as `h2`. The time of each phase is printed by the `-w` format of curl.

Time of each phase on loopback (seconds, typical) TERMINAL

```
$ curl -sS --cacert c.pem -o /dev/null -w 'namelookup %{time_namelookup}\n\
    connect %{time_connect}\nappconnect %{time_appconnect}\nstarttransfer %{time_starttransfer}\n\
    total %{time_total}\n' https://localhost:9501/
namelookup 0.000023
connect 0.000162
appconnect 0.004233
starttransfer 0.004609
total 0.004665
```

The values are cumulative from the start of the transfer. `connect` marks the end of the TCP handshake, `appconnect` the end of the TLS handshake, and `starttransfer` the arrival of the first response byte. On loopback the TLS handshake (about 4 ms, dominated by the asymmetric cryptography) is larger than the network time; on a real path each of the three differences grows by one round trip plus the cryptographic cost.

6.2.4 Failure modes

Phase	Failure	Symptom and first check
2	Gateway down, wrong subnet mask	Destination unreachable, incomplete ARP entry; `ip neigh` shows `INCOMPLETE` or `FAILED`
3	Resolver unreachable, NXDOMAIN, stale cache	`UnknownHostException`, curl exit 6; query the resolver directly, check the TTL of the cached record
4	Port closed, firewall drop, full accept queue	Immediate refusal (curl 7) or timeout (curl 28); `nc -vz host 443`
5	Expired or untrusted certificate, wrong host name, protocol mismatch	PKIX or handshake failure; `openssl s_client -connect host:443 -servername host`
6	Slow application, proxy timeout, wrong Host header	504, 502, 404 from a default virtual host; `curl -v` and the proxy log
7	Blocking script, large resources	High time to first paint with a healthy network; browser developer tools, waterfall view

What fails in each phase and how the failure appears.

6.2.5 Questions to expect

Summarise what happens when a URL is entered.

The browser parses the URL, checks caches and HSTS, resolves the name through the stub resolver and a recursive resolver, obtains the gateway MAC address with ARP, opens a TCP connection (three-way handshake, with NAT at the edge), performs a TLS handshake that authenticates the server and derives keys, sends an HTTP request, receives the response, and renders the page while fetching sub-resources over the same or additional connections.

Why is ARP needed when the destination is on another network?

ARP resolves the next hop, not the final destination. The frame must reach the default gateway, so the client resolves the gateway MAC address and addresses the frame to it, while the IP header carries the server address.

How many round trips precede the first response byte for a new HTTPS connection?

DNS (one, unless cached), TCP (one), TLS 1.3 (one) and the request itself (one): four. TLS 1.2 adds one. A reused connection needs one.

Which header fields change as a packet crosses routers, and which do not?

The Ethernet source and destination addresses change at every hop; the IPv4 TTL decreases and the header checksum is recomputed. NAT changes the source address and port (and the checksums) at the boundary. The destination address, the protocol, and the TCP sequence numbers do not change.

How does the client know that the server is genuine?

The server certificate chain must lead to a trusted root, be within its validity period, contain the requested host name in the subject alternative name, and the server must prove possession of the private key with the CertificateVerify signature over the handshake transcript.

Where does the DNS answer come from if the record is cached?

From the first cache that holds an unexpired record: the browser, the operating system, the router forwarder, or the recursive resolver. Each cache serves it until the TTL (300 s here) expires, which is why a changed record propagates gradually.

Why does HTTP/2 need only one connection per origin?

Streams multiplex requests and responses on one TCP connection with frame-level interleaving, so concurrent requests do not need separate handshakes. The trade-off is that one lost TCP segment stalls all streams, which HTTP/3 avoids.

What is the role of SNI and ALPN in the ClientHello?

SNI names the host so that a server with many certificates can select the right one; ALPN lists the application protocols the client supports (`h2`, `http/1.1`) so that the server can choose one during the handshake without an extra round trip.

KEY POINTS

- A new HTTPS request needs DNS, TCP, TLS and HTTP round trips in sequence; reuse and caching remove the first three.
- ARP resolves the next hop (the gateway), DNS resolves the server; the two addresses sit in different headers.
- Ethernet addresses change per hop, TTL per router, and NAT rewrites source address and port once; sequence numbers are end to end.
- TLS 1.3 completes in one round trip and may carry application data in the client's second flight.
- HTTP/2 multiplexes the page and its sub-resources as streams on a single connection.

6.3 Troubleshooting Playbook

Network faults are diagnosed by locating the layer that fails and then using the tool that observes that layer. The method is mechanical: state the symptom precisely (the exact error, the exact status code, the exact time), decide which layer can produce it, confirm with one measurement, and only then change a setting. A refused connection, a connection timeout and a connection reset are three different events with three different causes, and the first step in every investigation is to distinguish them.

6.3.1 Concept and mechanism

Method

Four rules keep an investigation short. First, reproduce the failure with the simplest client available (`curl`, `nc`) before blaming the application. Second, work along the path in order: name resolution, connection, handshake, request, response. Third, compare a working case with the failing case (another host, another port, another time) because the difference localises the fault. Fourth, change one variable at a time and record the result.

From symptom to layer to tool

Layer	Typical symptom	First tools
Application	5xx, wrong body, slow handler	curl -v, application logs, jstack, JFR
Security	certificate or handshake errors	openssl s_client, keytool -list, curl --cacert
Transport	refused, timeout, reset, retransmits	nc -vz, ss -tan, ss -ti, tcpdump 'tcp'
Network	unreachable, no route, MTU black hole	ping, traceroute, ip route, ping -M do -s
Link	no ARP reply, duplicate IP, flapping	ip neigh, arping, tcpdump -e arp
Physical	no carrier, errors, drops	ip -s link, ethtool

Figure 6.10. Each layer produces characteristic symptoms and is observed with specific tools, so the symptom selects the first tool to run.

Symptom	Layer	Tool	Next step
`UnknownHostException`, curl exit 6	DNS (application)	`getent hosts name`, `dig name`	Check resolver address, search domain, record existence, cached negative answer
Connection refused, curl exit 7	Transport	`nc -vz host port`, `ss -ltn` on the server	Confirm the listener, bind address (127.0.0.1 versus 0.0.0.0), port, container port mapping
Connect timeout, curl exit 28 before connect	Network or firewall	`traceroute`, security group and ACL review, `ss -ltn` (Recv-Q at the backlog limit)	Find where the SYN stops; check accept queue overflow counters
Connection reset, curl exit 56	Transport or application	`tcpdump 'tcp[tcpflags] & tcp-rst != 0'`	Identify which side sent the RST; compare idle timeouts
Read timeout, curl exit 28 after connect	Application	`jstack`, thread pool metrics, `ss -ti`	Look for blocked threads, slow dependency, retransmissions
HTTP 502	Gateway to upstream	gateway log, `curl` to the upstream directly	Upstream refused, reset or sent invalid data
HTTP 503	Gateway or application	health check state, pool metrics	No healthy upstream or load shedding
HTTP 504	Gateway to upstream	gateway timeout setting, upstream latency	Upstream slow; align timeouts
TLS error, curl exit 60, `PKIX path building failed`	Security	`openssl s_client -connect host:443 -servername host`	Chain, expiry, host name, truststore
Large requests hang, small ones work	Network (path MTU)	`ping -M do -s 1472 host`	Lower the MTU or clamp the MSS
Slow only at times, retransmissions	Network or host	`ss -ti`, `tcpdump`, interface counters	Locate loss; check congestion and queues

Symptom, probable layer, tool and next step.

The commands `dig`, `ss`, `ip`, `traceroute`, `tcpdump` and `ping` are not installed on the build machine of this book. The outputs in this chapter that come from tools on the build machine (curl, openssl, Java, `/proc`) are real; outputs marked typical are representative of those tools.

Connection failure tree

sreeram

Decision tree: a client cannot obtain a response from host:port

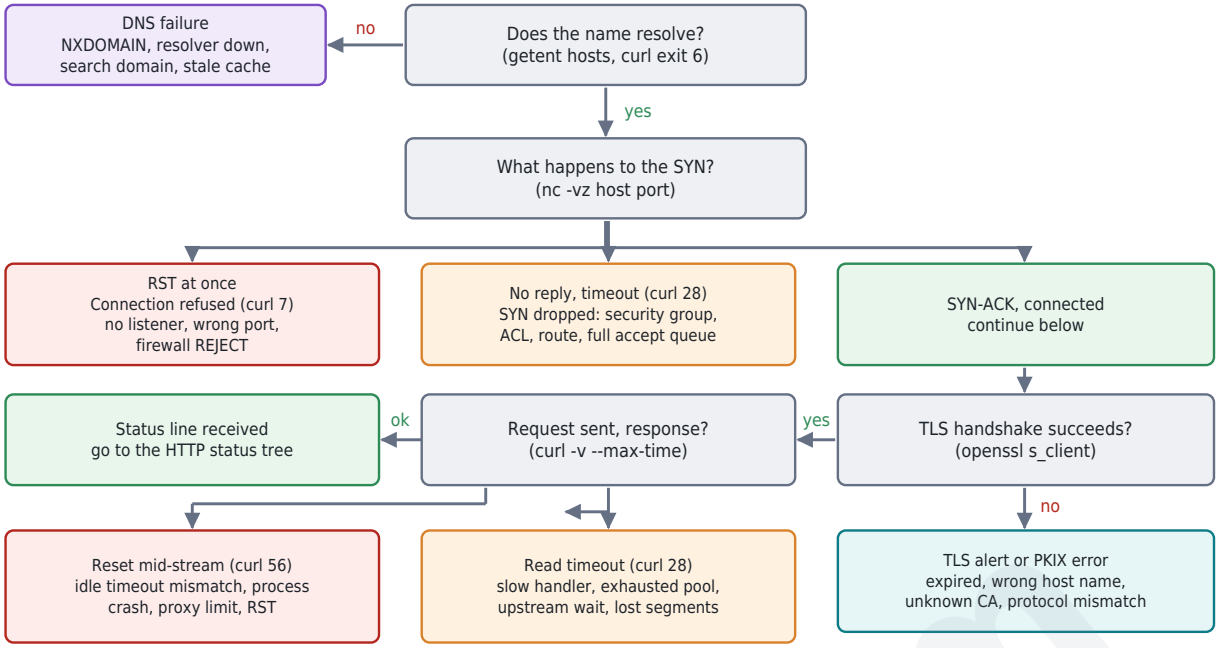


Figure 6.11. The tree separates failures by what the first packets do: a reset immediately after the SYN means refusal, silence means a drop, and a reset or silence after the connection means a problem above the handshake.

Reading the state of sockets without `ss`

On Linux, `ss` and `netstat` read the table `/proc/net/tcp`, in which the fourth column holds the TCP state as a hexadecimal code (01 ESTABLISHED, 06 TIME_WAIT, 08 CLOSE_WAIT, 0A LISTEN). The following script counts sockets per state and is used in the experiments below.

tcpstates.py PYTHON

```python
import collections
NAMES = {"01": "ESTABLISHED", "02": "SYN_SENT", "03": "SYN_RECV", "04": "FIN_WAIT1",
         "05": "FIN_WAIT2", "06": "TIME_WAIT", "07": "CLOSE", "08": "CLOSE_WAIT",
         "09": "LAST_ACK", "0A": "LISTEN", "0B": "CLOSING"}
count = collections.Counter()
for f in ("/proc/net/tcp", "/proc/net/tcp6"):
    try:
        for line in open(f).read().splitlines()[1:]:
            count[NAMES.get(line.split()[3], "?")] += 1
    except OSError:
        pass
for state, n in sorted(count.items()):
    print(f"{state:12s} {n}")
```

6.3.2 Worked trace

Distinguishing refused, timeout, reset and slow

Four loopback experiments produce the four failure classes with real curl output. A refusal needs no server: nothing listens on port 9.

Connection refused: RST in answer to the SYN TERMINAL

```
$ curl -sS -o /dev/null http://127.0.0.1:9
curl: (7) Failed to connect to 127.0.0.1 port 9 after 0 ms: Couldn't connect to server
```

A timeout is produced by a server that has called `listen(1)` but never calls `accept()`. After the accept queue is full, the kernel drops further SYN packets silently and the client retransmits the SYN until its own limit expires.

backlog.py PYTHON

```python
import socket, time, subprocess
s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("127.0.0.1", 9200))
s.listen(1)                                  # accept queue of length 1, accept() never called
fill = []
for i in range(4):                           # fill the accept queue and the SYN queue
    c = socket.socket()
    c.setblocking(False)
    try:
        c.connect(("127.0.0.1", 9200))
    except BlockingIOError:
        pass
    fill.append(c)
time.sleep(0.5)
r = subprocess.run(["curl", "-sS", "-o", "/dev/null", "--connect-timeout", "2",
                    "http://127.0.0.1:9200/"], capture_output=True, text=True)
print(r.stderr.strip(), "exit", r.returncode)
```

Connect timeout: the SYN is dropped, no answer arrives TERMINAL

```
$ python3 backlog.py
curl: (28) Failed to connect to 127.0.0.1 port 9200 after 2002 ms: Timeout was reached exit 28
```

The distinction matters in production: a server whose accept queue is full produces the same symptom as a firewall that drops packets. The counters `ListenOverflows` and `ListenDrops` in `nstat -az` (or `/proc/net/netstat`) increase in the first case and not in the second.

A reset after connection is produced by closing a socket with `SO_LINGER` set to zero, which sends RST instead of FIN. A read timeout is produced by a server that accepts and reads the request but never answers.

Reset by peer, and read timeout (curl exit codes 56 and 28) TERMINAL

```
$ python3 reset.py
curl: (56) Recv failure: Connection reset by peer exit 56
$ python3 slow.py
curl: (28) Operation timed out after 2002 milliseconds with 0 bytes received exit 28
```

Both curl messages carry exit code 28, but the first says `Failed to connect` (the handshake never completed) and the second `Operation timed out ... with 0 bytes received` (connected, no response). The wording identifies the phase.

Name resolution and TLS failures

DNS failure and TLS failures with real output TERMINAL

```
$ curl -sS -o /dev/null http://no-such-host.invalid/
curl: (6) Could not resolve host: no-such-host.invalid
$ curl -sS -o /dev/null https://localhost:9500/
curl: (60) SSL certificate problem: self-signed certificate
$ curl -sS -o /dev/null --cacert c.pem https://127.0.0.1:9500/
curl: (60) SSL: no alternative certificate subject name matches target host name '127.0.0.1'
$ curl -sS -o /dev/null http://localhost:9500/
curl: (52) Empty reply from server
```

The four lines show four distinct causes: an unresolvable name (the `.invalid` top-level domain is reserved and never resolves), an untrusted issuer, a certificate whose subject alternative names do not contain the address used, and plaintext HTTP sent to a TLS port, which the server closes without a reply. The Java equivalent of the second line is `SSLHandshakeException: PKIX path building failed: ... unable to find valid certification path to requested target`; the fix is to add the issuing certificate to the truststore of the JVM (or to a dedicated truststore passed with `-Djavax.net.ssl.trustStore`), never to disable verification.

502 versus 504 at a gateway

A gateway returns 502 when the upstream connection fails or returns garbage, and 504 when the upstream accepts but does not answer in time. The following gateway proxies to an upstream that is either absent (port 9399) or slow (three seconds); the proxy applies a connect timeout of 500 ms and a request timeout of one second.

Gateway.java JAVA

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.time.Duration;

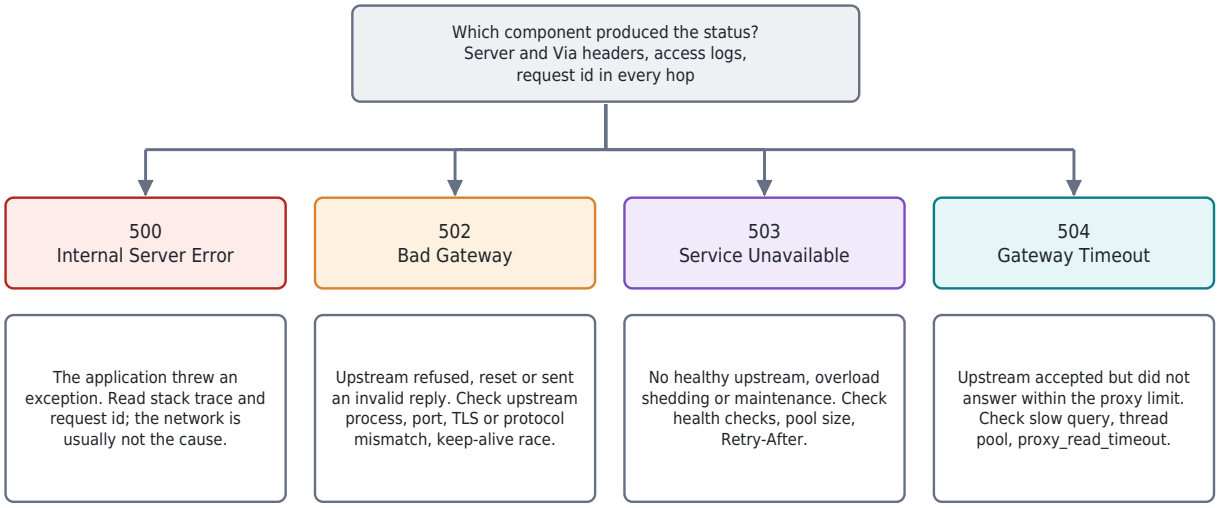
public class Gateway {
    public static void main(String[] args) throws Exception {
        HttpClient up = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(500)).build();
        HttpServer gw = HttpServer.create(new InetSocketAddress("127.0.0.1", 9300), 0);
        gw.createContext("/", ex -> {
            int port = ex.getRequestURI().getPath().equals("/down") ? 9399 : 9301;
            int code;
            try {
                HttpRequest r = HttpRequest.newBuilder(URI.create("http://127.0.0.1:" + port + "/"))
                        .timeout(Duration.ofSeconds(1)).build();
                code = up.send(r, HttpResponse.BodyHandlers.discarding()).statusCode();
            } catch (HttpTimeoutException e) {
                code = 504;                       // upstream accepted but did not answer in time
            } catch (IOException e) {
                code = 502;                       // upstream refused or reset the connection
            } catch (InterruptedException e) {
                code = 503;
            }
            ex.sendResponseHeaders(code, -1);
            ex.close();
        });
        gw.start();
        HttpServer slow = HttpServer.create(new InetSocketAddress("127.0.0.1", 9301), 0);
        slow.createContext("/", ex -> {
            try { Thread.sleep(3000); } catch (InterruptedException ignored) { }
            ex.sendResponseHeaders(200, -1);
            ex.close();
        });
        slow.start();
    }
}
```

Status codes produced by the gateway (measured, typical) TERMINAL

```
$ java -Dsun.net.httpserver.nodelay=true Gateway.java &
$ curl -s -o /dev/null -w "/down -> HTTP %{http_code} in %{time_total}s\n" \
    http://127.0.0.1:9300/down
/down -> HTTP 502 in 0.114705s
$ curl -s -o /dev/null -w "/slow -> HTTP %{http_code} in %{time_total}s\n" \
    http://127.0.0.1:9300/slow
/slow -> HTTP 504 in 1.009752s
```

The elapsed time is itself diagnostic: a 502 after a fraction of a second indicates a refused or reset connection; a 504 after exactly the configured limit (here 1.0 s) indicates a timeout. When the elapsed time of a 504 equals a round number such as 30 or 60 seconds, the proxy timeout setting is the first thing to inspect.

Decision tree: a 5xx status code is returned through a proxy or gateway



Compare timeouts along the chain: client > gateway > service > database avoids both orphaned work and duplicate retries.

A 502 or 504 seen by the client was generated by an intermediary, so its own log shows the upstream error.

Figure 6.12. The status code identifies the failing link: 500 points to the application, 502 and 504 to the connection between gateway and upstream, 503 to unavailability or shedding.

TIME_WAIT and CLOSE_WAIT

The state that a socket remains in after close identifies which side holds the problem. The side that closes first enters TIME_WAIT and keeps the connection tuple for twice the maximum segment lifetime (60 seconds on Linux), which prevents delayed segments of an old connection from being accepted by a new one. The side that receives the FIN enters CLOSE_WAIT and stays there until its application calls `close()`.

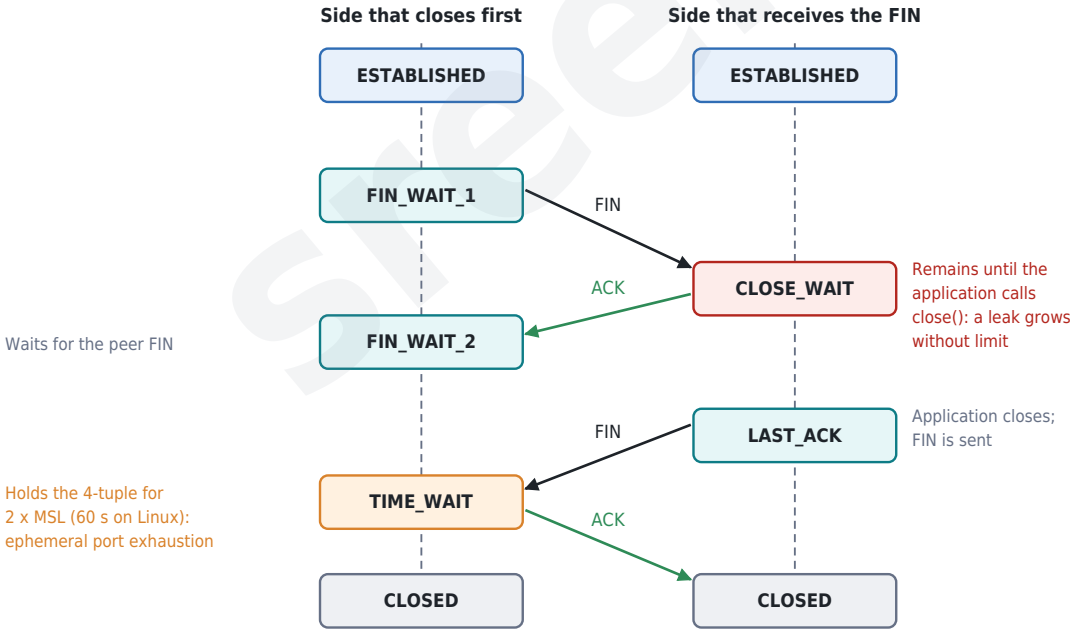


Figure 6.13. TIME_WAIT is a normal, bounded state of the side that closes first; CLOSE_WAIT is held by the passive side and grows without bound only when the application never closes the socket.

The next program makes 100 short connections. The server keeps the first 20 accepted sockets open (a deliberate leak) and closes the other 80 after reading the end of the stream, then the socket states of the machine are counted with the script above.

Churn.java JAVA

```java
import java.io.IOException;
import java.net.InetAddress;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class Churn {
    public static void main(String[] args) throws Exception {
        InetAddress lo = InetAddress.getLoopbackAddress();
        try (ServerSocket server = new ServerSocket(9400, 100, lo)) {
            List<Socket> leaked = new ArrayList<>();
            Thread srv = Thread.startVirtualThread(() -> {
                try {
                    while (true) {
                        Socket s = server.accept();
                        s.getInputStream().read();           // returns -1 when the client closes
                        if (leaked.size() < 20) leaked.add(s); // BUG: never closed -> CLOSE_WAIT
                        else s.close();
                    }
                } catch (IOException ignored) { }
            });
            for (int i = 0; i < 100; i++) {                   // 100 short-lived connections
                try (Socket c = new Socket(lo, 9400)) { c.getOutputStream().write(1); }
            }
            Thread.sleep(500);
            new ProcessBuilder("python3", "tcpstates.py").inheritIO().start().waitFor();
        }
    }
}
```

Socket states after 100 short connections (system-wide counts, measured) TERMINAL

```
$ java Churn.java
CLOSE_WAIT   20
ESTABLISHED  25
FIN_WAIT2    20
LISTEN       10
TIME_WAIT    91
```

Interpretation. The 20 leaked server sockets are in CLOSE_WAIT, and their 20 peers (the clients, which closed first) are in FIN_WAIT2, waiting for a FIN that the leaking side never sends. The 80 properly closed connections left TIME_WAIT entries on the client side; the counts are system-wide, so earlier experiments on the same machine add to the figure 91. A CLOSE_WAIT count that rises steadily is always an application defect (a response, stream or connection that is never closed). A large TIME_WAIT count is normal on a busy client or proxy and becomes a problem only when it approaches the number of available ephemeral ports.

Kernel parameters relevant to the states (build machine values) TERMINAL

```
$ cat /proc/sys/net/ipv4/ip_local_port_range /proc/sys/net/ipv4/tcp_fin_timeout \
    /proc/sys/net/core/somaxconn /proc/sys/net/ipv4/tcp_max_syn_backlog
32768   60999
60
4096
512
```

The range 32768 to 60999 contains 28232 ephemeral ports. All connections from one client address to one server address and port share that pool, because only the source port varies in the connection tuple. If every connection ends with the client closing first and stays in TIME_WAIT for 60 s, the sustainable rate is 28232 / 60 = about 470 new connections per second to that single destination; above it, `connect` fails with `Cannot assign requested address`. The remedy is connection reuse, not kernel tuning.

6.3.3 Hands-on

The remaining diagnostic commands are listed with their typical output. These tools are not installed on the build machine, so the outputs are representative and not captured here.

Typical output: socket summary and per-connection details (ss) TERMINAL

```
$ ss -tan state established '( dport = :8080 )'
Recv-Q Send-Q  Local Address:Port    Peer Address:Port
0      0       10.0.1.15:46012       10.0.2.20:8080
$ ss -ti dst 10.0.2.20
ESTAB 0 0 10.0.1.15:46012 10.0.2.20:8080
         cubic wscale:7,7 rto:204 rtt:1.2/0.6 mss:1448 cwnd:10 retrans:0/3 bytes_acked:5120
```

In the second command, `retrans:0/3` shows three retransmissions in the lifetime of the connection and `rto:204` the current retransmission timeout in milliseconds. For a listening socket, `ss -ltn` shows the current accept queue length in `Recv-Q` and its limit in `Send-Q`; `Recv-Q` equal to `Send-Q` means the accept queue is full.

Typical output: path MTU probe with the Don't Fragment bit (Linux ping) TERMINAL

```
$ ping -c 2 -M do -s 1472 192.0.2.50
PING 192.0.2.50 (192.0.2.50) 1472(1500) bytes of data.
1480 bytes from 192.0.2.50: icmp_seq=1 ttl=57 time=21.4 ms
$ ping -c 2 -M do -s 1472 198.51.100.80
PING 198.51.100.80 (198.51.100.80) 1472(1500) bytes of data.
From 10.0.0.1 icmp_seq=1 Frag needed and DF set (mtu = 1400)
```

An ICMP payload of 1472 bytes plus 28 bytes of headers equals the Ethernet MTU of 1500. The second target is reached through a path with MTU 1400 and the router reports it; when the ICMP message is filtered, the probe simply times out, which is the signature of a path MTU black hole.

6.3.4 Failure modes

The following incident descriptions use the structure symptom, evidence, cause, correction. The second and third relate to the experiments of this chapter and of the Java practice chapter; all five are described as typical cases, not as captured incidents.

Incident 1: intermittent 502 after quiet periods

Symptom: roughly one request in several thousand fails with 502, mostly on the first request after a quiet period, and never under constant load. Evidence: the gateway log shows `connection reset by peer` or `upstream prematurely closed connection` towards one upstream; the upstream access log contains no record of the failed requests. Cause: the application server closes idle connections after 20 seconds while the gateway reuses upstream connections for up to 60 seconds; the gateway occasionally writes on a connection that has just been closed (the race in the keep-alive diagram of the Java practice chapter). Correction: make the idle limit of the reusing side shorter than that of the closing side (gateway upstream keep-alive timeout below the server keep-alive timeout), and enable one retry for idempotent requests on a fresh connection.

Incident 2: CLOSE_WAIT growth and descriptor exhaustion

Symptom: after several days the service logs `Too many open files` and stops accepting connections; a restart repairs it. Evidence: `ls /proc/<pid>/fd | wc -l` grows linearly; the state count shows thousands of CLOSE_WAIT sockets towards one downstream service. Cause: responses are read partially or not closed on an error path (for example a body stream obtained with `BodyHandlers.ofInputStream()` that is not closed when a status check throws). Correction: use try-with-resources around every stream, prefer `ofString` or `ofByteArray` for small bodies, and alert on descriptor usage above 70 percent of the limit.

Incident 3: port exhaustion under load

Symptom: during a campaign the service fails with `java.net.BindException: Cannot assign requested address` when calling a payment provider; CPU is low. Evidence: tens of thousands of TIME_WAIT sockets to one address and port. Cause: a new client (and so a new connection) is created per request, which exhausts the 28232 ephemeral ports at about 470 connections per second to a single destination. Correction: one shared `HttpClient` per downstream, so that connections are pooled and reused.

Incident 4: large requests hang over a VPN

Symptom: small API calls succeed; uploads and large responses stall. Evidence: `ping -M do -s 1472` times out while `-s 1300` succeeds; capture shows full-size segments retransmitted without acknowledgement. Cause: a tunnel lowers the path MTU to about 1400 and the ICMP fragmentation-needed messages are filtered, so path MTU discovery fails. Correction: clamp the TCP MSS at the tunnel endpoint (for example to 1360), permit ICMP type 3 code 4, or set the interface MTU on the hosts.

Incident 5: all requests slow, CPU idle

Symptom: p99 latency rises from 80 ms to 12 s while CPU stays at 10 percent. Evidence: thread dump shows all worker threads blocked in `SocketInputStream.read` towards one dependency; the dependency latency graph shows 15 s responses. Cause: one slow dependency, no read timeout, and a worker pool equal in size to the concurrency, so all threads wait for it. Correction: read timeout and a bulkhead (a separate small pool per dependency) so that one slow dependency cannot occupy every worker.

6.3.5 Questions to expect

How are connection refused, connection timed out and connection reset distinguished?

Refused: the peer answered the SYN with RST, so the host is reachable but no process listens (or a firewall rejects). Timed out: no answer to the SYN, so a packet was dropped on the path, by a firewall, or by a full accept queue. Reset: the connection was established and then aborted by an RST from the peer or an intermediary.

What is the difference between 502, 503 and 504?

502: the gateway received an invalid response or no response because the upstream connection failed. 503: the service is unavailable (no healthy upstream, overload shedding, maintenance). 504: the gateway waited for the upstream and the time limit expired.

A service has thousands of sockets in TIME_WAIT. Is it a fault?

Not by itself. TIME_WAIT is the normal state of the side that closes first and lasts 60 s on Linux. It is a fault only if connection churn is excessive, which is corrected by reusing connections, or if ephemeral ports run out.

A service has thousands of sockets in CLOSE_WAIT. What does it indicate?

The peer closed its end and the local application did not close the socket: a resource leak in the application. Restarting hides it; the correction is closing responses and streams on every path.

All requests to one host time out but ping works. What is checked?

`ping` uses ICMP and does not prove that TCP port 443 is open. Check with `nc -vz host 443`, the security group and firewall rules for that port, the listening socket on the server, and the accept queue counters.

How is a path MTU problem recognised?

Small requests work and large ones hang after the handshake succeeds; a probe with the Don't Fragment bit fails above a certain size. The usual causes are tunnels and filtered ICMP; the fix is MSS clamping or a lower MTU.

How is a slow request attributed to DNS, connect, TLS or the server?

By the timing breakdown of the transfer: with `curl -w`, the differences between `time_namelookup`, `time_connect`, `time_appconnect` and `time_starttransfer` give the duration of each phase.

How does a Java `SSLHandshakeException: PKIX path building failed` differ from a certificate host name error?

PKIX failure means the chain does not lead to a certificate in the truststore (unknown issuer, missing intermediate); a host name error means the chain is valid but the subject alternative names do not match the name used. The fix for the first is the truststore, for the second the certificate or the URL.

sreferat

KEY POINTS

- The exact error text identifies the phase: refused (RST), timeout (silence), reset (abort after connect), read timeout (silence after request).
- 502 and 504 are generated by an intermediary; the time to failure shows whether the upstream refused or was slow.
- TIME_WAIT belongs to the side that closes first and is bounded; CLOSE_WAIT belongs to the passive side and signals an application leak.
- Ephemeral ports limit the rate of new connections to one destination (about 470 per second with 60 s TIME_WAIT); reuse connections.
- Change one variable at a time and compare a working case with the failing case.

6.4 Numerical Problems

Interviews and examinations on networking contain calculation problems with a short list of recurring types: delay and throughput, bandwidth-delay product, subnetting, retransmission timeout, congestion window evolution, checksums, aggregation, forwarding and switching. This chapter states twelve problems, solves each step by step, and verifies every answer with a program that was run for this book.

6.4.1 Concept and mechanism

Quantity	Formula
Transmission delay	`L / R (L in bits, R in bits per second)`
Propagation delay	`d / s (s about 2 × 10^8 m/s in copper or fibre)`
Store-and-forward delay over N links	`sum of (L / R_i + d_i / s) for each link, ignoring queueing`
Bandwidth-delay product	`R × RTT (bits in flight needed to fill the pipe)`
Window-limited throughput	`window / RTT`
Usable hosts in a subnet	`2^h − 2 (h = number of host bits)`
RTT estimator (RFC 6298)	`RTTVAR = 0.75 × RTTVAR + 0.25 × abs(SRTT − R); SRTT = 0.875 × SRTT + 0.125 × R; RTO = SRTT + 4 × RTTVAR`
Internet checksum	`one's complement of the one's complement sum of 16-bit words`
Little's law	`L = λ × W (items in system = arrival rate × time in system)`

Formulas used in this chapter.

6.4.2 Worked trace

Problem 1: store-and-forward delay

A 1500-byte packet crosses three links in series: 100 Mbit/s over 2 km, 10 Mbit/s over 1000 km, 100 Mbit/s over 2 km. Propagation speed is 2 × 10^8 m/s; processing and queueing are ignored. Find the end-to-end delay.

Solution. L = 1500 × 8 = 12000 bits. Transmission delays: 12000 / 100e6 = 0.12 ms, 12000 / 10e6 = 1.2 ms, 0.12 ms; sum 1.44 ms. Propagation delays: 2000 / 2e8 = 0.01 ms, 1e6 / 2e8 = 5 ms, 0.01 ms; sum 5.02 ms. Total delay = 1.44 + 5.02 = **6.46 ms**. The long link dominates propagation and the slow link dominates transmission.

Problem 2: pipelining of three packets

The sender transmits three such packets back to back. When does the third packet arrive?

Solution. The first packet arrives at 6.46 ms. The slowest link (10 Mbit/s, 1.2 ms per packet) spaces the packets by 1.2 ms from then on, because the router in front of it queues them. The third packet therefore arrives 2 × 1.2 ms later: 6.46 + 2.4 = **8.86 ms**. Sending three packets takes 8.86 ms, not 3 × 6.46 = 19.38 ms, because the links work in parallel on different packets. For a 4 MB file (32 Mbit) over a path whose slowest link is 5 Mbit/s the transfer time is dominated by the bottleneck: 32e6 / 5e6 = 6.4 s.

Problem 3: bandwidth-delay product and window

A path has 100 Mbit/s capacity and a round-trip time of 80 ms. (a) How many bytes must be in flight to keep the path full? (b) What throughput does a 65535-byte receive window allow? (c) Which window scale factor is needed?

Solution. (a) BDP = 100e6 × 0.080 = 8e6 bits = **1 000 000 bytes**. (b) Throughput = 65535 × 8 / 0.080 = 6.55 Mbit/s, which is **6.55 percent** of the capacity. (c) The scale shift s must satisfy 65535 × 2^s ≥ 1 000 000: 1 000 000 / 65535 = 15.26, so 2^s ≥ 15.26 and **s = 4** (a window of up to 1 048 560 bytes). The window scale option (RFC 7323) allows shifts up to 14.

Problem 4: equal-size subnets

Divide 192.168.10.0/24 into at least 6 subnets of equal size. Give mask, block size, hosts per subnet and the range of each subnet.

Solution. 2^3 = 8 ≥ 6, so 3 host bits are borrowed: prefix /24 + 3 = **/27**, mask 255.255.255.224. The block size is 256 − 224 = 32 addresses; hosts per subnet 2^5 − 2 = **30**. Subnet number i starts at 192.168.10.(32 × i).

i	Network	Usable hosts	Broadcast
0	`192.168.10.0/27`	`.1 to .30`	`.31`
1	`192.168.10.32/27`	`.33 to .62`	`.63`
2	`192.168.10.64/27`	`.65 to .94`	`.95`
3	`192.168.10.96/27`	`.97 to .126`	`.127`
4	`192.168.10.128/27`	`.129 to .158`	`.159`
5	`192.168.10.160/27`	`.161 to .190`	`.191`
6	`192.168.10.192/27`	`.193 to .222`	`.223`
7	`192.168.10.224/27`	`.225 to .254`	`.255`

The eight /27 subnets of 192.168.10.0/24 (the first six satisfy the requirement; two remain spare).

Problem 5: VLSM design

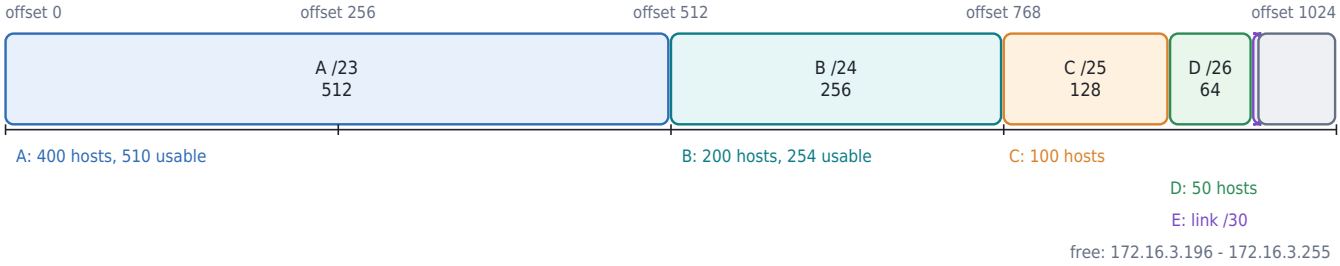
From 172.16.0.0/22 allocate subnets for departments of 400, 200, 100 and 50 hosts and one point-to-point link (2 hosts).

Solution. Sort by size, largest first, and give each the smallest block with at least hosts + 2 addresses: 400 + 2 = 402 needs 2^9 = 512 (/23); 202 needs 256 (/24); 102 needs 128 (/25); 52 needs 64 (/26); the link needs 4 (/30). Each block starts at the next multiple of its own size.

Name	Hosts	Block	Usable range	Broadcast
A	400	`172.16.0.0/23`	172.16.0.1 to 172.16.1.254	172.16.1.255
B	200	`172.16.2.0/24`	172.16.2.1 to 172.16.2.254	172.16.2.255
C	100	`172.16.3.0/25`	172.16.3.1 to 172.16.3.126	172.16.3.127
D	50	`172.16.3.128/26`	172.16.3.129 to 172.16.3.190	172.16.3.191
E	2	`172.16.3.192/30`	172.16.3.193 to 172.16.3.194	172.16.3.195

Allocation. All blocks lie inside 172.16.0.0/22 (1024 addresses); 964 are used.

VLSM allocation of 172.16.0.0/22 (1024 addresses, largest requirement first)



Each block starts on a multiple of its own size, so every block is a valid CIDR prefix.

Figure 6.14. The five blocks fill 964 of the 1024 addresses without overlap; allocating the largest block first keeps every block aligned to a multiple of its size.

Allocating in ascending order instead would leave gaps, because a /23 cannot start at the offset 4 or 8 following small blocks; alignment, not the sum of sizes, is the constraint.

Problem 6: retransmission timeout

RTT samples in order: 100, 120, 90 and 300 ms. Apply the RFC 6298 estimator and give SRTT, RTTVAR and RTO after each sample.

Solution. First sample R = 100: SRTT = 100, RTTVAR = 50, RTO = 100 + 4 × 50 = 300 ms. Second sample R = 120: RTTVAR = 0.75 × 50 + 0.25 × |100 − 120| = 42.5; SRTT = 0.875 × 100 + 0.125 × 120 = 102.5; RTO = 102.5 + 170 = 272.5. Third R = 90: RTTVAR = 0.75 × 42.5 + 0.25 × 12.5 = 35.0; SRTT = 0.875 × 102.5 + 0.125 × 90 = 100.94; RTO = 240.94. Fourth R = 300: RTTVAR = 0.75 × 35 + 0.25 × 199.06 = 76.02; SRTT = 0.875 × 100.94 + 37.5 = 125.82; RTO = **429.88 ms**.

Sample	R	SRTT	RTTVAR	RTO
1	100	100.00	50.00	300.00
2	120	102.50	42.50	272.50
3	90	100.94	35.00	240.94
4	300	125.82	76.02	429.88

After each sample (milliseconds).

The variance term makes a single outlier (300 ms) raise the RTO from 241 to 430 ms while the smoothed mean rises by only 25 ms. RFC 6298 also requires a minimum RTO of one second; the table shows the raw formula, and Linux applies a lower minimum of 200 ms. Samples from retransmitted segments are discarded (Karn's algorithm), and each timeout doubles the RTO.

Problem 7: congestion window evolution

A Reno sender starts with cwnd = 1 segment and ssthresh = 16 segments. Each round trip with no loss doubles cwnd in slow start (up to ssthresh) and adds one segment in congestion avoidance. Loss occurs in round 8, when cwnd = 20. Give cwnd per round if (a) the loss is detected by three duplicate acknowledgements and (b) by a timeout.

Solution. Rounds 0 to 8: 1, 2, 4, 8, 16 (slow start reaches ssthresh at round 4), then 17, 18, 19, 20 (congestion avoidance). (a) Fast retransmit and fast recovery: ssthresh = 20 / 2 = 10 and cwnd = 10, so rounds 9 onwards: 10, 11, 12, 13, 14, 15. (b) Timeout: ssthresh = 10 and cwnd = 1, so slow start repeats: 1, 2, 4, 8, then 10 (capped at ssthresh), 11.

sreeram

Reno congestion window for Problem 7 (initial window 1 segment; solid = fast retransmit, dashed = timeout)

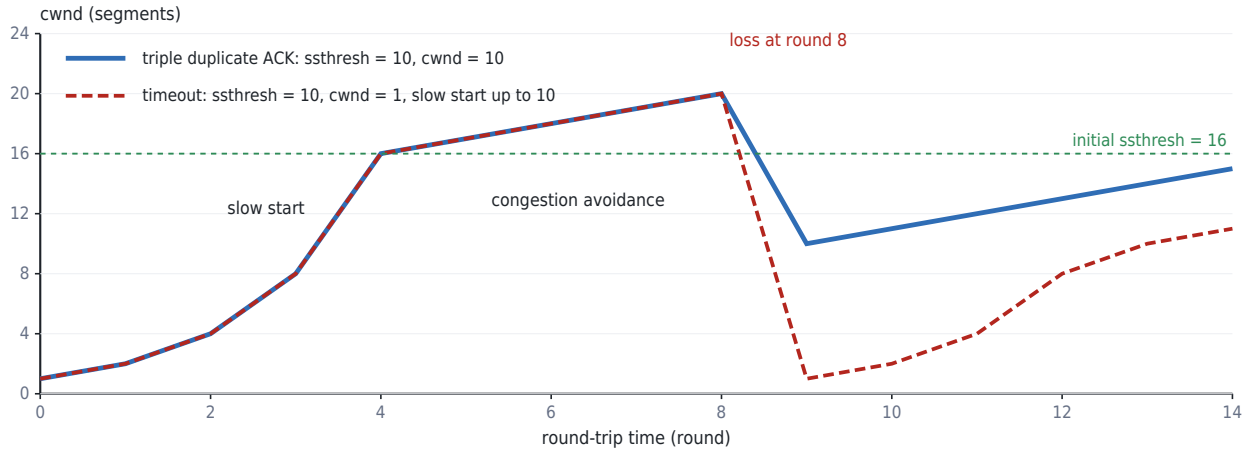


Figure 6.15. After a loss at cwnd = 20, triple duplicate ACKs halve the window to 10 and growth continues linearly, whereas a timeout resets the window to 1 and the sender needs five rounds to return to 10.

The comparison explains why duplicate acknowledgements are a milder signal than a timeout: the sender knows that later segments are still being delivered.

Problem 8: Internet checksum

Compute the header checksum of the IPv4 header with 16-bit words 4500 0073 0000 4000 4011 [checksum 0000] c0a8 0001 c0a8 00c7.

Solution. Add the words: 0x4500 + 0x0073 = 0x4573; + 0x4000 = 0x8573; + 0x4011 = 0xC584; + 0xC0A8 = 0x1862C; + 0x0001 = 0x1862D; + 0xC0A8 = 0x246D5; + 0x00C7 = 0x2479C. Fold the carry: 0x2 + 0x479C = 0x479E. Complement: 0xFFFF − 0x479E = **0xB861**. Verification: the sum of all words including the checksum is 0x2479C + 0xB861 = 0x2FFFD, folded 0xFFFD + 0x2 = 0xFFFF, which is the valid result.

Problem 9: route aggregation

Aggregate 10.1.0.0/24, 10.1.1.0/24, 10.1.2.0/24 and 10.1.3.0/24. Repeat without 10.1.2.0/24.

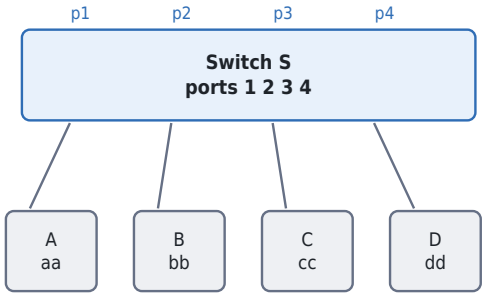
Solution. The third octets 0 to 3 are 000000xx in binary, so the common prefix is 24 − 2 = 22 bits: **10.1.0.0/22**. Without 10.1.2.0/24 the remaining three networks cannot form one prefix without also covering 10.1.2.0/24, which would attract traffic that does not belong to the aggregate. The best aggregation is **10.1.0.0/23** (covers .0 and .1) plus **10.1.3.0/24**.

Problem 10: learning switch

A switch with four ports and an empty table connects hosts A (port 1), B (port 2), C (port 3), D (port 4). Frames: A to B, B to A, C to A, A to D, D to broadcast. Give the ports on which each frame is sent and the table after each.

sreram

Problem 10: MAC learning on a 4-port switch (hosts A, B, C, D on ports 1 to 4)



Step	Frame	Action	Table after
1	A to B	B unknown: flood p2 p3 p4	aa=1
2	B to A	aa known: forward p1 only	aa=1 bb=2
3	C to A	forward p1 only	aa=1 bb=2 cc=3
4	A to D	dd unknown: flood p2 p3 p4	unchanged
5	D to ff:ff	broadcast: flood p1 p2 p3	adds dd=4

Unicast frames to a known address leave exactly one port; unknown unicast and broadcast leave every port except the ingress port.
The source address is learned from every frame; entries age out (300 s typical default).

Figure 6.16. Unknown unicast and broadcast frames are flooded to every port except the ingress port, whereas frames to learned addresses leave one port; every source address is learned on arrival.

Solution. (1) A to B: the table is empty, so the frame is flooded to ports 2, 3, 4; the table learns A on port 1. (2) B to A: A is known; the frame goes to port 1 only; B is learned on port 2. (3) C to A: port 1 only; C learned on port 3. (4) A to D: D is unknown; flood to 2, 3, 4; the table is unchanged. (5) D to broadcast: flooded to ports 1, 2, 3; D learned on port 4.

Problem 11: longest-prefix match

Routing table: 0.0.0.0/0 via gw; 10.0.0.0/8 via r1; 10.20.0.0/16 via r2; 10.20.30.0/24 via r3; 10.20.30.128/25 via r4. Find the next hop for 10.20.30.200, 10.20.30.5, 10.20.99.1, 10.9.9.9 and 8.8.4.4.

Solution. Collect every matching entry and select the one with the longest prefix.

Destination	Matches	Selected entry
10.20.30.200	5	10.20.30.128/25 via r4 (200 is in 128 to 255)
10.20.30.5	4	10.20.30.0/24 via r3 (5 is below 128)
10.20.99.1	3	10.20.0.0/16 via r2
10.9.9.9	2	10.0.0.0/8 via r1
8.8.4.4	1	0.0.0.0/0 via gw

Forwarding decisions.

Problem 12: pool and thread sizing with Little's law

A service receives 200 requests per second from a downstream client; the mean latency of the call is 120 ms. (a) How many connections are busy on average? (b) A Tomcat server with 200 threads handles requests of 0.5 s mean duration at 200 requests per second; is that sufficient? What happens when the duration rises to 5 s?

Solution. (a) $L = \lambda \times W = 200 \times 0.12 =$ **24** connections. A pool of 32 to 40 gives headroom for bursts. (b) Threads needed = 200 × 0.5 = 100, which is below 200 and sufficient. At 5 s the demand is 200 × 5 = 1000 threads, which exceeds 200: all workers block, the accept queue fills, and clients see timeouts. The load test of the Java practice chapter confirms the relation: 64 clients at 78.2 requests per second gave a mean latency of 64 / 78.2 = 0.82 s, and the measured median was 812 ms.

6.4.3 Hands-on

The solutions above were checked with two programs. The first (Java) covers Problems 1, 2, 3, 6, 7, 8 and 12.

Verify.java JAVA

```java
import java.util.ArrayList;
import java.util.List;

public class Verify {
    public static void main(String[] args) {
```

Verify.java (continued) JAVA

```
        // Problem 1 and 2: store-and-forward delay
        double bits = 1500 * 8;
        double[] rate = {100e6, 10e6, 100e6};
        double[] dist = {2_000, 1_000_000, 2_000};
        double tx = 0, prop = 0;
        for (int i = 0; i < 3; i++) {
            tx += bits / rate[i];
            prop += dist[i] / 2e8;
        }
        System.out.printf("P1 transmission %.2f ms, propagation %.2f ms, total %.2f ms%n",
                tx * 1e3, prop * 1e3, (tx + prop) * 1e3);
        double last = (tx + prop) + 2 * (bits / 10e6);
        System.out.printf("P2 third packet arrives at %.2f ms; 4 MB file: %.1f s%n",
                last * 1e3, 32e6 / 5e6);

        // Problem 3: bandwidth-delay product
        double bdp = 100e6 * 0.080 / 8;
        double cap = 65535 * 8 / 0.080;
        int shift = (int) Math.ceil(Math.log(bdp / 65535) / Math.log(2));
        System.out.printf("P3 BDP %.0f bytes, 64 KiB window limit %.2f Mbit/s, window scale %d%n",
                bdp, cap / 1e6, shift);

        // Problem 6: RFC 6298 estimator
        double[] r = {100, 120, 90, 300};
        double srtt = r[0], rttvar = r[0] / 2;
        System.out.printf("P6 R=%.0f SRTT=%.3f RTTVAR=%.3f RTO=%.3f%n",
                r[0], srtt, rttvar, srtt + 4 * rttvar);
        for (int i = 1; i < r.length; i++) {
            rttvar = 0.75 * rttvar + 0.25 * Math.abs(srtt - r[i]);
            srtt = 0.875 * srtt + 0.125 * r[i];
            System.out.printf("P6 R=%.0f SRTT=%.3f RTTVAR=%.3f RTO=%.3f%n",
                    r[i], srtt, rttvar, srtt + 4 * rttvar);
        }

        // Problem 7: Reno window per round
        List<Integer> w = new ArrayList<>();
        int cwnd = 1, ssthresh = 16;
        for (int round = 0; round <= 8; round++) {
            w.add(cwnd);
            cwnd = cwnd < ssthresh ? Math.min(cwnd * 2, ssthresh) : cwnd + 1;
        }
        System.out.println("P7 rounds 0 to 8: " + w);

        // Problem 8: Internet checksum of an IPv4 header
        int[] hdr = {0x4500, 0x0073, 0x0000, 0x4000, 0x4011,
                     0x0000, 0xc0a8, 0x0001, 0xc0a8, 0x00c7};
        long sum = 0;
        for (int x : hdr) {
            sum += x;
        }
        System.out.printf("P8 raw sum 0x%x, ", sum);
        while ((sum >> 16) != 0) {
            sum = (sum & 0xffff) + (sum >> 16);
        }
        System.out.printf("folded 0x%x, checksum 0x%04x%n", sum, (~sum) & 0xffff);

        // Problem 12: Little's law
        System.out.printf("P12 connections %.0f, measured L/lambda %.2f s%n",
                200 * 0.12, 64 / 78.2);
    }
}
```

sheram

Output of Verify.java TERMINAL

```
$ java Verify.java
P1 transmission 1.44 ms, propagation 5.02 ms, total 6.46 ms
P2 third packet arrives at 8.86 ms; 4 MB file: 6.4 s
P3 BDP 1000000 bytes, 64 KiB window limit 6.55 Mbit/s, window scale 4
P6 R=100 SRTT=100.000 RTTVAR=50.000 RTO=300.000
P6 R=120 SRTT=102.500 RTTVAR=42.500 RTO=272.500
P6 R=90 SRTT=100.938 RTTVAR=35.000 RTO=240.938
P6 R=300 SRTT=125.820 RTTVAR=76.016 RTO=429.883
P7 rounds 0 to 8: [1, 2, 4, 8, 16, 17, 18, 19, 20]
P8 raw sum 0x2479c, folded 0x479e, checksum 0xb861
P12 connections 24, measured L/lambda 0.82 s
```

The second program (Python, standard `ipaddress` module) covers the addressing problems 4, 5, 9, 10 and 11.

verify_addr.py PYTHON

```python
import ipaddress, math

# Problem 4: six subnets from 192.168.10.0/24 need 3 borrowed bits
subnets = list(ipaddress.ip_network("192.168.10.0/24").subnets(new_prefix=27))
for n in (subnets[0], subnets[1], subnets[7]):
    print("P4", n, "hosts", n.network_address + 1, "-", n.broadcast_address - 1,
          "broadcast", n.broadcast_address)

# Problem 5: VLSM, largest requirement first, from 172.16.0.0/22
base = int(ipaddress.ip_address("172.16.0.0"))
for name, hosts in (("A", 400), ("B", 200), ("C", 100), ("D", 50), ("E", 2)):
    bits = math.ceil(math.log2(hosts + 2))
    net = ipaddress.ip_network((base, 32 - bits))
    print("P5", name, hosts, "->", net, "usable", net.num_addresses - 2,
          "last", net.broadcast_address)
    base += net.num_addresses

# Problem 9: aggregation
four = [ipaddress.ip_network(f"10.1.{i}.0/24") for i in range(4)]
print("P9", list(ipaddress.collapse_addresses(four)))
gap = [ipaddress.ip_network(f"10.1.{i}.0/24") for i in (0, 1, 3)]
print("P9", list(ipaddress.collapse_addresses(gap)))

# Problem 10: learning switch
table = {}
frames = [("aa", "bb", 1), ("bb", "aa", 2), ("cc", "aa", 3), ("aa", "dd", 1), ("dd", "ff", 4)]
for src, dst, port in frames:
    table[src] = port
    out = [table[dst]] if dst in table else [p for p in (1, 2, 3, 4) if p != port]
    print("P10", f"{src}->{dst}", "out", out, "table", dict(table))

# Problem 11: longest-prefix match
routes = {"0.0.0.0/0": "gw", "10.0.0.0/8": "r1", "10.20.0.0/16": "r2",
          "10.20.30.0/24": "r3", "10.20.30.128/25": "r4"}
for dst in ("10.20.30.200", "10.20.30.5", "10.20.99.1", "10.9.9.9", "8.8.4.4"):
    hits = [p for p in routes if ipaddress.ip_address(dst) in ipaddress.ip_network(p)]
    best = max(hits, key=lambda p: ipaddress.ip_network(p).prefixlen)
    print("P11", dst, "matches", len(hits), "best", best, "->", routes[best])
```

Output of verify_addr.py TERMINAL

```
$ python3 verify_addr.py
P4 192.168.10.0/27 hosts 192.168.10.1 - 192.168.10.30 broadcast 192.168.10.31
P4 192.168.10.32/27 hosts 192.168.10.33 - 192.168.10.62 broadcast 192.168.10.63
P4 192.168.10.224/27 hosts 192.168.10.225 - 192.168.10.254 broadcast 192.168.10.255
P5 A 400 -> 172.16.0.0/23 usable 510 last 172.16.1.255
P5 B 200 -> 172.16.2.0/24 usable 254 last 172.16.2.255
P5 C 100 -> 172.16.3.0/25 usable 126 last 172.16.3.127
P5 D 50 -> 172.16.3.128/26 usable 62 last 172.16.3.191
P5 E 2 -> 172.16.3.192/30 usable 2 last 172.16.3.195
P9 [IPv4Network('10.1.0.0/22')]
P9 [IPv4Network('10.1.0.0/23'), IPv4Network('10.1.3.0/24')]
P10 aa->bb out [2, 3, 4] table {'aa': 1}
P10 bb->aa out [1] table {'aa': 1, 'bb': 2}
P10 cc->aa out [1] table {'aa': 1, 'bb': 2, 'cc': 3}
P10 aa->dd out [2, 3, 4] table {'aa': 1, 'bb': 2, 'cc': 3}
P10 dd->ff out [1, 2, 3] table {'aa': 1, 'bb': 2, 'cc': 3, 'dd': 4}
P11 10.20.30.200 matches 5 best 10.20.30.128/25 -> r4
```

Ireram

Output of verify_addr.py (continued) TERMINAL

```
P11 10.20.30.5 matches 4 best 10.20.30.0/24 -> r3
P11 10.20.99.1 matches 3 best 10.20.0.0/16 -> r2
P11 10.9.9.9 matches 2 best 10.0.0.0/8 -> r1
P11 8.8.4.4 matches 1 best 0.0.0.0/0 -> gw
```

6.4.4 Failure modes

Calculation errors in these problems are systematic. The most frequent are listed with the check that exposes each.

- Mixing bits and bytes. Link rates are in bits per second, sizes in bytes: multiply bytes by 8 before dividing by the rate. A result that is eight times too small or large indicates this error.
- Mixing 1000 and 1024. Rates use powers of ten (1 Mbit/s = 10^6 bit/s); memory sizes in operating systems often use powers of two. State the convention used.
- Forgetting the two reserved addresses of a subnet (network and broadcast) when sizing, or forgetting that a /31 point-to-point link (RFC 3021) and a /32 are special cases.
- Allocating VLSM blocks that are not aligned to a multiple of their own size; the resulting prefix is not a valid CIDR block.
- Applying the RTT estimator update in the wrong order: RTTVAR is updated first, using the old SRTT.
- Treating the congestion window as bytes in one place and segments in another; state the unit before computing.
- Taking the sum of per-link delays for a stream of packets, which ignores pipelining (Problem 2).

6.4.5 Questions to expect

How many usable hosts does a /26 contain, and what is its mask?

32 − 26 = 6 host bits: 2^6 − 2 = 62 usable hosts; mask 255.255.255.192.

Why must the window equal the bandwidth-delay product for full utilisation?

The sender may send one window before the first acknowledgement returns after one RTT. Utilisation is window / (R × RTT), so it reaches 100 percent only when the window equals R × RTT.

How is the next hop chosen when several routes match?

The longest matching prefix wins, regardless of the order of entries or metrics; metrics only select among routes with equal prefix length.

Why does a timeout reduce the congestion window more than three duplicate acknowledgements?

Duplicate acknowledgements show that segments after the lost one still arrive, so the network is congested only mildly; Reno halves the window. A timeout indicates that nothing is getting through, so the window falls to one segment and slow start restarts.

What does a receiver check with the Internet checksum?

It sums all 16-bit words of the header (including the checksum field) with end-around carry; the result must be 0xFFFF. It detects most single-bit and many multi-bit errors, but not reordered words.

How is the end-to-end delay of one packet over several links computed?

By adding, for every link, the transmission delay (L / R) and the propagation delay (distance / speed), plus processing and queueing delays when they are given.

KEY POINTS

- Convert units first (bytes to bits, kilometres to metres) and state the propagation speed assumption.
- The bandwidth-delay product is the window needed to fill a path; a 64 KiB window on 100 Mbit/s and 80 ms limits throughput to 6.55 Mbit/s.
- Subnet and VLSM problems reduce to powers of two and alignment; allocate the largest block first.
- RTO follows SRTT + 4 × RTTVAR; a timeout resets the window to one segment, three duplicate ACKs halve it.
- Verify every manual answer with a short program; all values in this chapter were verified.

sreeram

6.5 Interview Question Bank

The questions in this chapter are grouped by layer, from the link layer to Java practice, and each is followed by a model answer of two to five lines. The answers state the mechanism, not only the definition, because interviewers follow a definition with a request for the reason. Questions are numbered within each group. The recommended method is to read the question, answer aloud for thirty seconds, and then compare with the model answer.

6.5.1 Foundations and link layer

1. **Compare the OSI model and the TCP/IP model.** OSI has seven layers (physical, data link, network, transport, session, presentation, application); TCP/IP has four or five (link, internet, transport, application, with physical sometimes separated). The OSI session and presentation functions are part of the application layer in TCP/IP. TCP/IP is the model of the protocols that are deployed; OSI is a reference model.
2. **What is encapsulation?** Each layer adds its own header (and, at the link layer, a trailer) to the data from the layer above. An HTTP message becomes a TCP segment, then an IP packet, then an Ethernet frame. The receiver removes the headers in reverse order (decapsulation).
3. **Which protocol data unit belongs to which layer?** Application: message. Transport: segment (TCP) or datagram (UDP). Network: packet or datagram. Link: frame. Physical: bits.
4. **Distinguish a hub, a switch and a router.** A hub repeats every bit on all ports (one collision domain). A switch forwards frames by destination MAC address using a learned table (one collision domain per port). A router forwards packets between networks by destination IP address and separates broadcast domains.
5. **How does a switch learn?** For each arriving frame it records the source MAC address against the ingress port. A frame to a known destination leaves one port; unknown unicast and broadcast frames are flooded to all other ports. Entries age out after a timeout (300 seconds is a common default).
6. **What is the difference between a MAC address and an IP address?** A MAC address (48 bits) identifies an interface on one link and is used for delivery within a network; it changes at every hop. An IP address identifies a host's attachment to a network and is used for end-to-end routing; it normally stays the same across the path (apart from NAT).
7. **How does ARP work?** A host that must send to an IPv4 address on its subnet broadcasts "who has this address"; the owner replies with its MAC address and the sender caches the mapping. For a remote destination the host resolves the MAC address of the default gateway instead.
8. **What is a broadcast domain and what is a collision domain?** A broadcast domain is the set of devices that receive a link-layer broadcast; routers and VLANs bound it. A collision domain is the set of devices whose transmissions can collide on shared media; each switch port is its own collision domain, and full-duplex links have none.
9. **What does a VLAN do and how is it carried over a trunk?** A VLAN splits one physical switch into several logical broadcast domains. On a trunk link the frame carries an IEEE 802.1Q tag with a 12-bit VLAN identifier, which allows up to 4094 usable VLANs.
10. **What is the MTU?** The largest payload a link layer frame can carry; 1500 bytes for standard Ethernet. IP packets larger than the path MTU are fragmented (IPv4) or must be avoided by path MTU discovery.
11. **Name the four components of packet delay.** Processing delay, queueing delay, transmission delay (L / R) and propagation delay (distance / speed). Queueing is the only variable component and the cause of jitter.
12. **How do bandwidth, throughput and latency differ?** Bandwidth is the capacity of a link in bits per second. Throughput is the rate actually achieved, limited by the bottleneck link, protocol windows and loss. Latency is the time for a bit or packet to travel, independent of bandwidth.
13. **Packet switching versus circuit switching?** Circuit switching reserves capacity for the duration of a call (no queueing, wasted when idle). Packet switching shares links statistically, which gives better utilisation for bursty traffic at the cost of queueing and possible loss.
14. **Why does Wi-Fi use CSMA/CA instead of CSMA/CD?** A wireless station cannot listen while it transmits, and hidden terminals make collisions undetectable at the sender, so Wi-Fi avoids collisions (random backoff, acknowledgements, optional RTS/CTS) instead of detecting them.

INTERVIEW ANGLE
Follow-up chain: "A packet goes from a laptop to a server in another city. Which addresses change?" The MAC addresses change at every router hop; the IP addresses do not change, unless NAT rewrites the source address at the network edge; the TTL decreases at every router; port numbers do not change except for the NAT port mapping. This answer connects questions 4, 6 and 7 and is expected to be given without prompting.

6.5.2 The network layer

1. **List the main IPv4 header fields.** Version, header length, DSCP/ECN, total length, identification, flags and fragment offset, TTL, protocol, header checksum, source and destination addresses. The minimum header is 20 bytes.
2. **What are the TTL and the protocol fields for?** TTL is decremented by each router and the packet is discarded at zero, which stops routing loops (and makes traceroute possible). The protocol field names the payload type (1 ICMP, 6 TCP, 17 UDP).
3. **How does IPv4 fragmentation work?** A router splits a packet larger than the next link MTU into fragments sharing one identification value, each carrying a fragment offset (in 8-byte units) and the More Fragments flag. Only the destination reassembles. If the Don't Fragment bit is set, the router drops the packet and sends ICMP "fragmentation needed".
4. **What is CIDR and why was it introduced?** Classless inter-domain routing writes a network as address/prefix-length, so any prefix length can be used instead of fixed classes A, B and C. It reduces address waste and allows route aggregation.
5. **How many hosts are in a /27, and what is the mask?** Five host bits: 2^5 − 2 = 30 usable hosts; mask 255.255.255.224.
6. **Which address ranges are private?** 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16 (RFC 1918). 100.64.0.0/10 is shared address space for carrier-grade NAT, 169.254.0.0/16 is link-local, and 127.0.0.0/8 is loopback.
7. **How does NAT with port translation work?** The NAT device rewrites the source address and port of outgoing packets to its public address and a unique port, and records the mapping in a table; return packets are matched against the table and translated back. Many private hosts share one public address.
8. **What are the problems of NAT?** Hosts behind NAT cannot accept unsolicited inbound connections without port forwarding; protocols that carry addresses in the payload (FTP, SIP) need helpers; the table is per-flow state with timeouts, which breaks idle connections; end-to-end addressing is lost.
9. **Describe DHCP.** A client without an address broadcasts DISCOVER; servers answer with OFFER; the client sends REQUEST for one offer; the server confirms with ACK. The lease contains the address, mask, default gateway, DNS servers and lease time.
10. **How do ping and traceroute work?** Ping sends ICMP echo requests and measures the time to the echo replies. Traceroute sends packets with TTL 1, 2, 3 and so on; each router that discards a packet at TTL zero returns ICMP "time exceeded", revealing the hop.
11. **What is the difference between routing and forwarding?** Forwarding is the per-packet action of sending a packet to an outgoing interface using the forwarding table. Routing is the process (protocols and algorithms) that computes the table.
12. **How does a router choose among several matching routes?** The longest prefix wins. For equal prefix lengths the route with the lowest administrative distance and then the lowest metric is chosen.
13. **Distance vector versus link state.** Distance vector (RIP) routers exchange their distance tables with neighbours and apply Bellman-Ford; it converges slowly and can count to infinity. Link-state (OSPF) routers flood link information so that every router holds the full topology and runs Dijkstra's algorithm.
14. **What is BGP and what is an autonomous system?** An autonomous system is a network under one administrative policy with an AS number. BGP exchanges reachability between autonomous systems and selects paths by policy, not by shortest distance; it is the protocol that holds the Internet together.
15. **Why IPv6?** The 128-bit address space removes address scarcity and the need for NAT. IPv6 also has a simpler fixed header, no router fragmentation, and stateless address autoconfiguration (SLAAC). Transition uses dual stack and tunnelling.
16. **What is the default gateway?** The router on the local subnet to which a host sends every packet whose destination is outside the subnet.
17. **What is the loopback address and why is `localhost` not the same as the machine address?** 127.0.0.1 (and ::1) refers to the host itself and never leaves it. A server bound to 127.0.0.1 is unreachable from other machines, which explains a frequent deployment error.

6.5.3 The transport layer

1. **TCP versus UDP: when is each used?** TCP provides a reliable, ordered byte stream with flow and congestion control (HTTP, databases, SSH). UDP provides unreliable datagrams without connection setup (DNS, streaming, games, QUIC); reliability, if needed, is implemented above it.
2. **Describe the TCP three-way handshake.** Client sends SYN with its ISN; the server answers SYN-ACK with its own ISN and acknowledges the client ISN plus one; the client sends ACK acknowledging the server ISN plus one. The data phase then begins.
3. **Why three messages and not two?** Both directions must have their sequence numbers synchronised and acknowledged. With two messages the server could not know whether the client received its ISN, and delayed duplicate SYNs could create half-open connections.
4. **Why are initial sequence numbers random?** To avoid confusing segments of an old connection with a new one and to make blind injection by an off-path attacker harder (RFC 6528).
5. **How is a connection closed?** Each direction closes independently: FIN, ACK in one direction, then FIN, ACK in the other (four segments). A side that has sent FIN can still receive data (half-close). An RST aborts the connection immediately.
6. **What is TIME_WAIT and why does it exist?** The side that closes first waits 2 × MSL (60 s on Linux) after the final ACK. It ensures that the last ACK can be retransmitted if lost, and that delayed segments of the old connection expire before the same tuple is reused.
7. **What does a large number of CLOSE_WAIT sockets mean?** The peer closed and the local application never called `close()`: an application leak.
8. **What is a SYN flood and how is it mitigated?** An attacker sends many SYNs and never completes the handshake, filling the SYN queue. SYN cookies encode the state in the sequence number so that no queue entry is needed until the final ACK arrives; rate limiting and larger backlogs help.
9. **How does TCP achieve reliability?** Sequence numbers, cumulative acknowledgements, a retransmission timer (RTO from RTT estimates), fast retransmit after three duplicate ACKs, selective acknowledgements, and a checksum.
10. **How is the retransmission timeout computed?** RTO = SRTT + 4 × RTTVAR, where SRTT and RTTVAR are exponentially weighted averages of RTT samples (RFC 6298). The RTO doubles after each timeout, and samples of retransmitted segments are ignored.
11. **What is fast retransmit?** After three duplicate ACKs for the same sequence number the sender retransmits the missing segment without waiting for the timer, because the duplicates prove that later segments arrived.
12. **Flow control versus congestion control?** Flow control protects the receiver: the advertised receive window limits unacknowledged data. Congestion control protects the network: the sender maintains a congestion window and sends at most the minimum of the two windows.
13. **Explain slow start and congestion avoidance.** Slow start doubles cwnd every round trip from a small initial window until ssthresh or the first loss. Congestion avoidance then adds about one segment per round trip. On loss, ssthresh is set to half of the window; a timeout resets cwnd to one segment, fast retransmit halves it.
14. **What are Nagle's algorithm and delayed ACK, and how can they interact badly?** Nagle holds small segments while unacknowledged data is outstanding; delayed ACK holds an acknowledgement for up to about 40 ms. A request/response server that writes headers and body separately can stall for 40 ms on every response. `TCP_NODELAY` disables Nagle.
15. **What is a zero window?** The receiver advertises a window of zero because its buffer is full. The sender stops and probes periodically until a window update arrives.
16. **What identifies a TCP connection?** The four-tuple of source address, source port, destination address and destination port (the five-tuple adds the protocol). A server port can therefore serve many clients.
17. **What does `SO_REUSEADDR` do?** It allows a server to bind a port that has connections in TIME_WAIT, so that a restarted server does not fail with `Address already in use`.
18. **What is the listen backlog?** The length of the accept queue of completed connections waiting for `accept()`. When it is full, further connection attempts are dropped and clients time out.
19. **Why does TCP suffer from head-of-line blocking and how does QUIC avoid it?** TCP delivers a single ordered stream, so one lost segment delays all later data, even if it belongs to independent HTTP/2 streams. QUIC runs over UDP with independent streams, so a loss delays only its own stream.

20. **What is TCP keep-alive?** An option that sends probe segments on an idle connection (after 2 hours by default on Linux) to detect dead peers; application-level heartbeats are usually preferred because the default interval is long.

6.5.4 The application layer and the web

1. **Walk through DNS resolution.** The application asks the stub resolver, which asks a recursive resolver. On a cache miss the recursive resolver queries a root server (referral to the .com servers), a TLD server (referral to the authoritative servers of example.com) and the authoritative server, which returns the record. Every answer is cached for its TTL.
2. **Which DNS record types matter?** A (IPv4 address), AAAA (IPv6), CNAME (alias; cannot coexist with other records at the same name), MX (mail), NS (delegation), TXT (verification, SPF), SOA (zone parameters, negative-caching time), SRV (service host and port).
3. **How does the JVM cache DNS lookups?** `InetAddress` caches successful lookups for the value of the security property `networkaddress.cache.ttl` (30 seconds by default when no security manager is installed) and failures for `networkaddress.cache.negative.ttl` (10 seconds). Services behind a changing DNS name, such as a load balancer, need a finite TTL.
4. **Which HTTP methods are safe and which are idempotent?** Safe (no state change): GET, HEAD, OPTIONS. Idempotent (repeating has the same effect as once): GET, HEAD, OPTIONS, PUT, DELETE. POST and PATCH are not idempotent in general.
5. **Summarise the status code classes.** 1xx informational, 2xx success (200, 201, 204), 3xx redirection (301, 302, 304, 307, 308), 4xx client error (400, 401, 403, 404, 409, 429), 5xx server error (500, 502, 503, 504).
6. **What is the difference between 401 and 403?** 401 means the request lacks valid authentication; 403 means the identity is known and not permitted to perform the action.
7. **301, 302, 307 and 308: when is each used?** 301 and 308 are permanent redirects, 302 and 307 temporary. 307 and 308 require the client to repeat the same method and body; for 301 and 302 clients historically change POST to GET.
8. **HTTP/1.1 versus HTTP/2 versus HTTP/3.** HTTP/1.1: text, one request at a time per connection, so browsers open several connections. HTTP/2: binary frames, multiplexed streams on one TCP connection, HPACK header compression, but TCP head-of-line blocking. HTTP/3: HTTP semantics over QUIC (UDP), per-stream loss recovery, faster handshake, connection migration.
9. **What does HTTP keep-alive do?** It keeps the TCP connection open for further requests, which avoids repeated handshakes. In HTTP/1.1 it is the default; `Connection: close` disables it, and idle timeouts on both sides limit its lifetime.
10. **Why is the Host header required?** One IP address and port can serve many sites (virtual hosting). The Host header (or the TLS SNI) tells the server which site the request targets.
11. **How does HTTP caching work?** A response with `Cache-Control: max-age` is reused without contacting the server until it expires. After expiry the client revalidates with `If-None-Match` (ETag) or `If-Modified-Since`; an unchanged resource returns 304 with no body.
12. **How do cookies and sessions work?** The server sends `Set-Cookie` with a session identifier; the browser returns it in the `Cookie` header on matching requests. Attributes `Secure`, `HttpOnly` and `SameSite` limit transport, script access and cross-site sending. State is kept on the server.
13. **Session cookie versus JWT?** A session cookie is an opaque identifier resolved in a server-side store (easy to revoke, requires shared storage when scaled). A JWT is a signed token carrying claims, verified without a lookup (stateless, hard to revoke before expiry, and readable by the client because it is signed, not encrypted).
14. **What is CORS and who enforces it?** The browser's same-origin policy blocks scripts from reading responses from another origin (scheme, host, port). CORS lets the server opt in with `Access-Control-Allow-Origin` and related headers. The browser enforces it; `curl` and servers are not affected.
15. **What is a preflight request?** For requests that are not "simple" (methods other than GET, HEAD, POST, custom headers, JSON content type) the browser first sends an OPTIONS request with `Origin`, `Access-Control-Request-Method` and `Access-Control-Request-Headers`. The actual request is sent only if the response allows it.
16. **Why can `Access-Control-Allow-Origin: *` not be combined with credentials?** A wildcard would let any site send authenticated requests and read the response. With cookies the server must name the exact origin and send `Access-Control-Allow-Credentials: true`.

17. **WebSocket, SSE or long polling?** WebSocket: full-duplex messages over one connection after an HTTP upgrade (chat, games). SSE: one-way server-to-client event stream over plain HTTP with automatic reconnect (notifications). Long polling: a request held open until data exists; works everywhere, at the cost of repeated requests.
18. **How should a REST client retry safely?** Only idempotent operations, or operations protected by an idempotency key, only on transient errors (connect failure, 502, 503, 504, 429 with `Retry-After`), with exponential backoff, jitter and an overall deadline.
19. **What is the difference between 502, 503 and 504?** 502: invalid or no response from the upstream (connection refused or reset). 503: unavailable or overloaded. 504: the upstream did not answer within the time limit.
20. **What does gRPC use on the wire?** HTTP/2 with Protocol Buffers messages; it supports unary calls and client, server and bidirectional streaming.
21. **What is chunked transfer encoding?** A way to send a body of unknown length: the body is sent as chunks, each with a length prefix, ending with a zero-length chunk. HTTP/2 does not use it because frames carry lengths.

6.5.5 Security and infrastructure

1. **Describe the TLS 1.3 handshake.** ClientHello (versions, cipher suites, key share, SNI, ALPN); ServerHello with its key share; the remaining server messages are encrypted: certificate chain, CertificateVerify signature, Finished. The client verifies and answers with Finished. One round trip; keys come from an ephemeral Diffie-Hellman exchange.
2. **What is forward secrecy?** Session keys derive from ephemeral key exchanges that are discarded afterwards, so recorded traffic stays confidential even if the server's long-term private key is later compromised.
3. **How is a certificate validated?** The chain must lead to a trusted root; each signature must verify; every certificate must be within its validity period and not revoked; the host name must match a subject alternative name; and the server must prove possession of the private key.
4. **What is mutual TLS?** The server also requests and validates a client certificate, so both sides authenticate. It is used for service-to-service traffic and service meshes.
5. **Symmetric versus asymmetric encryption: why does TLS use both?** Asymmetric operations authenticate and establish a shared secret but are slow. Symmetric ciphers (AES-GCM, ChaCha20) encrypt bulk data quickly with that secret.
6. **What is the purpose of a MAC or HMAC?** It proves integrity and authenticity of a message with a shared key. TLS uses AEAD ciphers that combine encryption and authentication.
7. **What is the Java `PKIX path building failed` error?** The JVM could not build a chain from the server certificate to a trust anchor in its truststore, typically because of a private CA, a missing intermediate certificate or a proxy that re-signs traffic. The fix is to import the CA into the truststore.
8. **Stateless versus stateful firewall?** A stateless filter judges each packet by header fields. A stateful firewall tracks connections and admits return traffic of permitted flows automatically.
9. **What is a DMZ?** A separate network segment for public-facing services, isolated from the internal network, so that a compromised server cannot reach internal systems directly.
10. **Forward proxy versus reverse proxy?** A forward proxy acts for clients (filtering, caching, anonymity). A reverse proxy sits in front of servers and handles TLS termination, load balancing, caching and routing.
11. **Layer 4 versus layer 7 load balancing.** L4 balances connections using addresses and ports without parsing HTTP (fast, protocol-agnostic). L7 parses requests and can route by path, host or header, terminate TLS, and retry; it costs more CPU.
12. **Which load-balancing algorithms exist?** Round robin, weighted round robin, least connections, IP hash, and consistent hashing (keys map to a hash ring so that adding or removing a node moves only a fraction of the keys).
13. **What are sticky sessions and why are they discouraged?** The balancer sends a client to the same backend (by cookie or source address) to keep local session state. It skews load and loses sessions on failure; externalised session state is preferable.
14. **TLS termination versus passthrough?** Termination decrypts at the balancer (enables L7 features, centralises certificates; traffic behind it needs protection). Passthrough forwards encrypted bytes to the backend, which keeps end-to-end encryption but prevents L7 inspection.
15. **How does a CDN serve a request?** DNS or anycast directs the client to a nearby edge. A cache hit is answered at once; a miss goes to the origin, is stored according to `Cache-Control`, and is served to later clients. Latency falls and origin load shrinks.

16. **Explain CSRF, XSS and SSRF.** CSRF: a malicious site makes the victim's browser send an authenticated request; mitigated by `SameSite` cookies and tokens. XSS: attacker script runs in the victim's page; mitigated by output encoding and Content-Security-Policy. SSRF: the server is tricked into requesting an internal URL; mitigated by allow-lists and blocking internal addresses.
17. **What are ARP spoofing and DNS poisoning?** ARP spoofing forges ARP replies so that a victim sends traffic to the attacker; mitigated by dynamic ARP inspection and port security. DNS poisoning inserts a false record into a resolver cache; mitigated by DNSSEC, randomised ports and identifiers, and TLS.
18. **How do VPN types differ?** IPsec tunnels IP packets at the network layer (site to site); WireGuard is a small modern UDP-based tunnel; TLS-based VPNs tunnel through port 443 and cross restrictive firewalls.
19. **How does the OAuth 2.0 authorisation code flow work?** The client redirects the user to the authorisation server; after login and consent the server redirects back with a short-lived code; the client exchanges the code (with PKCE) at the token endpoint for an access token and calls the API with `Authorization: Bearer`.

6.5.6 Java and system design

1. **Which timeouts does a Java HTTP client need?** Connect timeout, request or read timeout, and an overall deadline. All are off by default in `HttpClient` and `HttpURLConnection`.
2. **What does `SO_TIMEOUT` control, and is it the same as the connect timeout?** `SO_TIMEOUT` bounds each blocking read on an established socket; the connect timeout bounds connection establishment. A slow server can keep a read timeout satisfied forever by sending data slowly.
3. **Blocking I/O, NIO and virtual threads: how do they differ?** Blocking I/O dedicates a platform thread to each connection. NIO with a selector multiplexes many connections on few threads but needs callback-style code. Virtual threads (final in Java 21) let blocking-style code scale, because a blocked virtual thread releases its carrier thread.
4. **How many threads and connections does Tomcat use?** A pool of up to `server.tomcat.threads.max` (200) worker threads handles requests; the NIO connector tracks up to `max-connections` (8192) connections; `accept-count` (100) is the queue behind them.
5. **How is a connection pool sized?** By Little's law: required connections = request rate × mean latency of the call, plus headroom. A pool that is too small queues requests; one that is too large can overload the downstream service.
6. **What causes `java.net.SocketException: Too many open files`?** The process reached its descriptor limit, through leaked sockets, large pools or a low `ulimit -n`. Close resources deterministically and raise the limit to match the design.
7. **How is a circuit breaker related to timeouts and retries?** The timeout bounds a single call, retries repeat transient failures, and the circuit breaker stops calls to a failing dependency after a failure rate threshold so that it can recover and the caller fails fast.
8. **How is CORS configured in Spring Boot?** With `@CrossOrigin` on a controller, or globally by registering `CorsRegistry` mappings in a `WebMvcConfigurer` (allowed origins, methods, headers, credentials), or through the Spring Security `cors()` configuration, which must run before the authentication filters so that preflight requests are answered.
9. **A request takes 3 s only on its first call after idle. List possible causes.** DNS cache expiry, a new TCP connection after the pooled one was closed, a TLS handshake without session resumption, a cold JIT or lazily initialised pool on the server, or a NAT or firewall entry that expired and forced a retransmission.

INTERVIEW ANGLE

Closing chain: "A Spring Boot service calls another service, and 1 request in 5000 fails with 502 at the gateway. How is it investigated?" Establish which hop produced the 502 (gateway log, `Server` and `Via` headers). Check whether failures follow idle periods, which indicates a stale connection: compare keep-alive and idle timeouts along gateway, load balancer and Tomcat. Confirm in the upstream access log that the failed requests never arrived. Correct the timeouts so that the reusing side closes first, and permit one retry for idempotent requests.

6.5.7 Questions to expect

The bank above is the question set; the following advice applies to delivering the answers. State the layer first, because it frames the answer. Give the mechanism before the name of the protocol feature. Quote one number where one exists (a header size, a timeout, a default port). Finish with the failure mode or trade-off. An answer that contains these four elements is normally sufficient, and the interviewer's follow-up then tests depth on one of them.

Opening question	Typical follow-ups
What happens when a URL is typed?	Which round trips occur? What is cached? Where does NAT act? What changes with HTTP/3?
TCP versus UDP	Why does DNS use UDP? When does DNS use TCP? How does QUIC add reliability?
What is TIME_WAIT?	Which side enters it? Why 60 seconds? What if ports run out? Why not disable it?
HTTP/1.1 versus HTTP/2	What is multiplexing? What remains blocked? Why does HTTP/3 exist?
What is CORS?	Who enforces it? What is a preflight? Why not use * with cookies?
502 versus 504	Who generates them? How is each reproduced? How are the timeouts ordered?
TLS handshake	What does the certificate prove? What is SNI? What is forward secrecy?
How is a connection pool sized?	What is Little's law? What if the downstream slows down? Which timeouts?

Frequently chained follow-up questions.

KEY POINTS

- Answer by layer: state the layer, the mechanism, one number, and the failure mode or trade-off.
- The classic questions (URL walk-through, TCP versus UDP, TIME_WAIT, HTTP versions, CORS, TLS handshake, NAT, 502 versus 504) must be answerable in under a minute each.
- Connect each theoretical answer to a Java setting: timeouts, pools, Tomcat properties, truststores, DNS cache TTL.
- Practise aloud; written recognition of an answer is weaker than producing it.

6.6 Cheat Sheets

This chapter collects reference tables for revision on the day before an interview. Each table is a summary; the mechanism behind every entry is explained in the chapter of the corresponding Part.

sreeram

6.6.1 Ports

Port	Protocol	Transport	Note
20, 21	FTP data, control	TCP	Plaintext; replaced by SFTP (over SSH, port 22)
22	SSH, SFTP	TCP	Encrypted remote shell and file transfer
25	SMTP	TCP	Server-to-server mail transfer
53	DNS	UDP and TCP	UDP for ordinary queries; TCP for large answers and zone transfers
67, 68	DHCP server, client	UDP	Broadcast-based address assignment
80	HTTP	TCP	Plaintext web
110, 995	POP3, POP3S	TCP	Mail retrieval
123	NTP	UDP	Time synchronisation
143, 993	IMAP, IMAPS	TCP	Mail access
161	SNMP	UDP	Network device management
389, 636	LDAP, LDAPS	TCP	Directory access
443	HTTPS	TCP (and UDP for HTTP/3)	TLS-protected web
465, 587	SMTPS, mail submission	TCP	Client mail submission
2181	ZooKeeper	TCP	Coordination service
3306	MySQL	TCP	Database
5432	PostgreSQL	TCP	Database
5672	AMQP (RabbitMQ)	TCP	Messaging
6379	Redis	TCP	In-memory store
8080	HTTP alternative	TCP	Common for application servers (Tomcat default)
9092	Kafka broker	TCP	Messaging
9200	Elasticsearch	TCP	Search
27017	MongoDB	TCP	Database

Well-known and common ports.

sreram

6.6.2 HTTP status codes

Code	Name	Meaning and typical cause
200	OK	Request succeeded
201	Created	Resource created; `Location` header names it
204	No Content	Success without body (DELETE, PUT)
301, 308	Moved Permanently, Permanent Redirect	Resource has a new URL; 308 preserves the method
302, 307	Found, Temporary Redirect	Temporary redirect; 307 preserves the method
304	Not Modified	Conditional request; cached copy is valid
400	Bad Request	Malformed syntax or validation failure
401	Unauthorized	Missing or invalid credentials
403	Forbidden	Authenticated but not permitted
404	Not Found	No such resource (or hidden)
405	Method Not Allowed	Method not supported by the resource
408	Request Timeout	Client too slow to send the request
409	Conflict	State conflict, for example a duplicate or version clash
413	Content Too Large	Body exceeds the server limit
415	Unsupported Media Type	Content type not accepted
429	Too Many Requests	Rate limit; see `Retry-After`
500	Internal Server Error	Unhandled exception in the application
502	Bad Gateway	Invalid or no response from upstream
503	Service Unavailable	Overload, no healthy upstream, maintenance
504	Gateway Timeout	Upstream did not answer in time

Status codes that appear most often.

6.6.3 Header cheat sheet

Header	Direction	Purpose
`Host`	Request	Target host (virtual hosting); mandatory in HTTP/1.1
`Authorization`	Request	Credentials, for example `Bearer <token>`
`Content-Type`, `Content-Length`	Both	Media type and size of the body
`Accept`, `Accept-Encoding`	Request	Acceptable media types and compressions
`Cookie`, `Set-Cookie`	Request, response	Session state; attributes `Secure`, `HttpOnly`, `SameSite`
`Cache-Control`	Both	`max-age`, `no-store`, `no-cache`, `private`, `public`
`ETag`, `If-None-Match`	Response, request	Validator and conditional request (304)
`Location`	Response	Redirect target or created resource
`Origin`	Request	Origin of a cross-site request (CORS)
`Access-Control-Allow-Origin`	Response	Origins allowed to read the response (CORS)
`Strict-Transport-Security`	Response	Force HTTPS for a period (HSTS)
`Retry-After`	Response	Seconds or date after which a retry is acceptable
`X-Forwarded-For`, `Forwarded`	Request	Original client address behind proxies
`Connection`, `Keep-Alive`	Both	Persistent connection control (HTTP/1.1)
`Upgrade`	Both	Protocol switch, for example `websocket`

Important HTTP headers.

6.6.4 TCP flags and header sizes

Flag	Meaning
SYN	Synchronise sequence numbers; opens a connection
ACK	The acknowledgement number is valid (set on almost every segment after the SYN)
FIN	Sender has finished sending; closes one direction
RST	Abort the connection immediately
PSH	Deliver buffered data to the application without delay
URG	Urgent pointer is valid (rarely used)
ECE, CWR	Explicit congestion notification signalling

TCP flags.

Header	Size	Note
Ethernet II	14 bytes (+4 FCS trailer)	802.1Q adds 4 bytes
IPv4	20 to 60 bytes	20 without options
IPv6	40 bytes	Fixed
TCP	20 to 60 bytes	32 with timestamps; 40 in the SYN with typical options
UDP	8 bytes	Source port, destination port, length, checksum
Standard Ethernet MTU	1500 bytes	Maximum TCP segment payload 1460 with plain headers

Typical header sizes.

6.6.5 Subnet quick table

Prefix	Mask	Addresses	Usable hosts
/8	255.0.0.0	16 777 216	16 777 214
/16	255.255.0.0	65 536	65 534
/20	255.255.240.0	4096	4094
/22	255.255.252.0	1024	1022
/23	255.255.254.0	512	510
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2
/31	255.255.255.254	2	2 (point-to-point, RFC 3021)
/32	255.255.255.255	1	single host

CIDR prefixes, masks and host counts.

Method: block size = 256 minus the interesting mask octet; the network address is the largest multiple of the block size not above the host value in that octet; the broadcast address is the next network minus one.

sreeram

6.6.6 Command cheat sheet

Command	Purpose
`curl -v URL`	Show DNS, connect, TLS and HTTP exchange
`curl -w '%{time_connect} %{time_starttransfer} ' -o /dev/null -s URL`	Phase timings
`nc -vz host port`	Test whether a TCP port accepts connections
`openssl s_client -connect host:443 -servername host`	TLS handshake, certificate chain, protocol and cipher
`dig +trace name`	Follow DNS delegation from the root
`ss -ltn` / `ss -tan` / `ss -ti`	Listening sockets with queues, all TCP sockets, TCP internals (RTT, retransmissions)
`ip addr`, `ip route`, `ip neigh`	Addresses, routing table, ARP cache
`ping -c 4 host`, `ping -M do -s 1472 host`	Reachability and path MTU
`traceroute host` / `mtr host`	Path and per-hop loss
`tcpdump -i eth0 -nn 'tcp port 443'`	Packet capture
`ls /proc/<pid>/fd`	One entry per open descriptor of a process (count the lines)
`keytool -list -keystore store.jks`	Contents of a Java keystore
`jstack <pid>`	Thread dump; find threads blocked in socket reads

Commands for diagnosis (Linux).

sreeram

6.6.7 Java network API cheat sheet

API	Use
`new ServerSocket(port, backlog)`	Listen; the backlog is the accept queue length hint
`serverSocket.accept()`	Blocks until a connection is complete; returns a `Socket`
`socket.connect(addr, timeoutMs)`	Connect with a connect timeout
`socket.setSoTimeout(ms)`	Read timeout (`SocketTimeoutException`)
`socket.setTcpNoDelay(true)`	Disable Nagle's algorithm
`socket.setKeepAlive(true)`	Enable TCP keep-alive probes
`serverSocket.setReuseAddress(true)`	`SO_REUSEADDR`; bind despite TIME_WAIT
`socket.shutdownOutput()`	Half-close: sends FIN, continues reading
`DatagramSocket`, `DatagramPacket`	UDP send and receive
`InetAddress.getByName(name)`	DNS lookup, cached by the JVM
`networkaddress.cache.ttl`	Security property for DNS cache lifetime (30 s default)
`HttpClient.newBuilder().connectTimeout(d)`	Client-wide connect timeout
`HttpRequest.newBuilder(uri).timeout(d)`	Per-request timeout
`HttpClient.Version.HTTP_2`	Preferred protocol version (negotiated)
`SocketChannel`, `Selector`	Non-blocking I/O with multiplexing
`SSLContext.getInstance("TLS")`	TLS configuration; truststore via `-Djavax.net.ssl.trustStore`
`-Djavax.net.debug=ssl:handshake`	Print TLS handshake details
`server.tomcat.threads.max`, `accept-count`, `max-connections`	Tomcat concurrency and queues
`server.tomcat.keep-alive-timeout`, `connection-timeout`	Tomcat idle and request-line timeouts
`spring.threads.virtual.enabled`	Run requests on virtual threads (Spring Boot 3.2 and later)

Java classes, methods and settings.

KEY POINTS

- Memorise the ports, status codes and the subnet table; derive everything else from the mechanisms.
- The timeouts and queue properties of the Java stack map to TCP events: connect timeout to SYN, accept-count to the accept queue, keep-alive timeout to FIN after idle.
- Use `curl -v`, `nc -vz` and `openssl s_client` first; they cover DNS, connect, TLS and HTTP.

6.7 Six-Week Study Plan

The plan assumes about ten hours per week and a reader who knows basic Java. Each week reads the chapters of one Part, runs the hands-on commands, solves the numerical problems of that layer, and ends with spoken answers to the matching questions of the interview bank. Revision uses the cheat sheets. The plan follows the order of the Parts because every layer relies on the one below it.

sreeram

Six-week study plan (about 10 hours per week), each week closes with a self-test

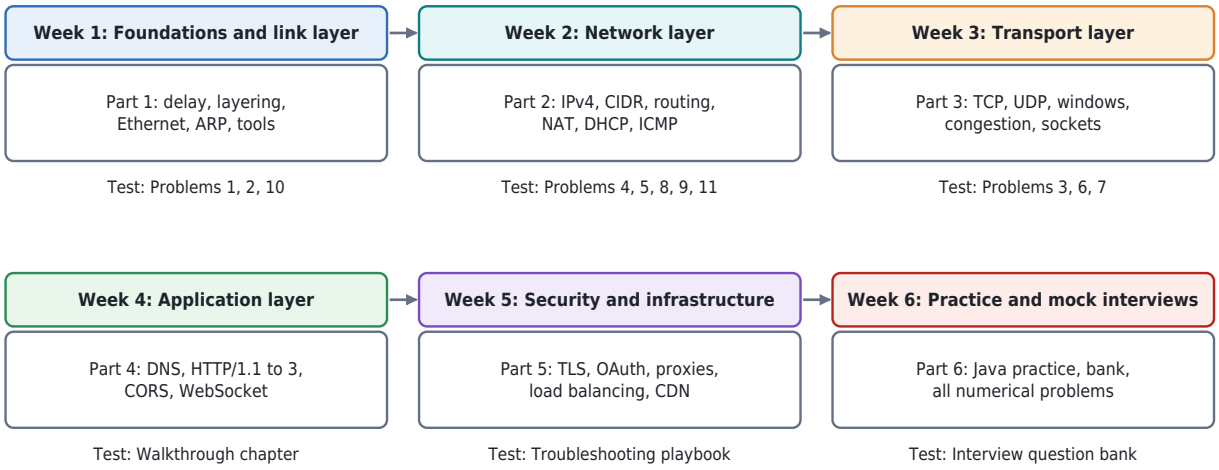


Figure 6.17. The six weeks follow the layers from the bottom up, with a numerical or practical test at the end of each week and the troubleshooting and interview material reserved for the final two weeks.

Week	Chapters to read	Hands-on and problems	Interview bank
1	What a network is; Delay, loss and throughput; Layered architecture; The link layer; Wireless basics; Tools	Run curl and nc against a local server; Problems 1, 2 and 10	Foundations and link layer
2	IPv4 addressing and the IPv4 header; Classless addressing and subnetting; IPv6; ICMP and diagnostics; Routing; NAT and DHCP; Cloud networking basics	Subnet calculator; Problems 4, 5, 8, 9 and 11	The network layer
3	Transport layer, ports and sockets; UDP; TCP segment and connection management; TCP reliability; Flow control; Congestion control; Sockets in Java; QUIC	Echo servers; Problems 3, 6 and 7	The transport layer
4	DNS; HTTP/1.1; HTTP caching; Cookies and sessions; HTTP/2 and HTTP/3; REST from the network; CORS; WebSocket, SSE and long polling; gRPC and other protocols	`curl -v` against a local `HttpServer`; the walk-through chapter	The application layer and the web
5	Cryptography for networking; TLS and HTTPS; Authentication in transit; Network security; Reverse proxies and load balancers; CDNs and high availability; Network design for microservices	`openssl s_server` and `s_client`; truststore exercises	Security and infrastructure
6	Java networking in practice; Troubleshooting playbook; What happens when a URL is entered; Numerical problems	Re-run the load test; reproduce the failure experiments; all 12 problems without notes	Whole bank, aloud, with a timer

Week-by-week plan with chapters named as in this book.

6.7.1 Daily rhythm

A session of ninety minutes has three parts. Forty minutes of reading with the diagrams redrawn by hand (the TCP state machine, the TLS handshake, the encapsulation of an HTTP request). Thirty minutes of hands-on work: run the commands and programs, change one parameter, and predict the effect before running. Twenty minutes of spoken answers to bank questions, recorded or given to a partner. Questions that cannot be answered in under a minute return to the review list of the next week.

6.7.2 Checkpoints

Week	The reader is able to
1	Explain encapsulation with header sizes; trace a MAC learning table; compute delay for a multi-link path
2	Subnet and aggregate by hand; apply longest-prefix match; explain NAT translation entries
3	Draw the TCP handshake with numbers; explain TIME_WAIT and CLOSE_WAIT; compute RTO and window evolution
4	Reproduce a preflight request; explain HTTP/2 multiplexing and its limit; use `curl -v` fluently
5	Draw the TLS 1.3 handshake; explain certificate validation; choose between L4 and L7 balancing
6	Diagnose a described incident from symptoms; answer the classic questions in under a minute each

Week-end checkpoints.

KEY POINTS

- Study bottom-up: each layer depends on the one below it, and so do the interview follow-ups.
- Pair every chapter with a hands-on command or program and with the matching bank questions.
- Redraw the main diagrams from memory and answer aloud; both reveal gaps that reading hides.
- Spend the final week on integration: the URL walk-through, incidents and the numerical problems.

sreeram